Smithian Fold Theory: A Zero-Axiom, Zero-Free-Parameter Unified Constants Object - Lean 4 Formal Verification

Smithian Fold Theory: the cumulative theory-of-everything manuscript
Maria Smith
Edition 0.2.1 - Unified Constants milestone | 1 November 2026
Previous published edition 0.2: DOI 10.5281/zenodo.23846751
Zenodo edition 0.2.1: DOI 10.5281/zenodo.23849998

Exact alpha. Seven sectors.
One rooted constants object.

308 / 2,781 completed V3 entries
17,849 audited theorems | Zero axiom dependencies

A cumulative mathematical manuscript. The fine-structure derivation opens the paper; the Unified Constants milestone joins the completed sector readings without changing the V3 model.

Contents

1. Opening result: the fine-structure constant 9
2. A cumulative paper, released at scientific milestones 10
3. Reading and reproducing this edition 11
4. Source and evidence links 11
5. Model and scope 12
6. Operational root 12
7. Structural One 14
8. Exact positive finite count 15
9. Exact positive part coordinate 16
10. Generated common-refinement pairing 17
11. Minimal Count extension in Fold 18
12. Complete finite Fold-word assembly and return 19
13. Formal representation and results 29
14. Correspondence and reproducibility 30
15. Extension of the formalisation 31
16. Source 31
17. Discrete collections and held selections 31
18. Finite collection induction 32
19. Complementary discrete relations 33
20. Composition of discrete maps 33
21. Primitive Fold maps and ordered composition 34
22. Least-size primitive generator 35
23. Ordered exact derivation traces 36
24. Finite combinatorial selections 37
25. Finite arrangements by held choice 38
26. Arrangement completeness and uniqueness 39
27. Disjoint class accounting 40
28. Combinatorial witnesses and complete grammar 41
29. Selection encoding and extension 41
30. Arrangement successor partition and provenance 42
31. Arrangement trace accounting and arithmetic linkage 43
32. Complete encoded selection output 43
33. Discrete collection integration 44
34. Ordered selection provenance 45
35. Raw choices as discrete held selections 45
36. Finite algebraic operation tables 46
37. Complete single-valued operation support 47
38. Exhaustive identity and association 47
39. Held return mates and commutation 48
40. Public return and commutation semantics 49
41. Operation-preserving maps 49
42. Total structure-map semantics 50
43. Original algebra witnesses and complete grammar 51
44. Algebra program correspondence 52
45. Algebraic induction base and carrier extension 52
46. Identity and association under carrier extension 53
47. Return mates under carrier extension 54
48. Operation maps under carrier extension 55
49. Concrete dictionary extension 55
50. Finite order and bound operations 56
51. Exact partial-order admission 57
52. Extremal witnesses and returned bounds 58
53. Conditional meet/join existence and monotonicity 58
54. Complete order grammar and program correspondence 59
55. Order induction base and comparison extension 60
56. Bound lists under carrier extension 61
57. Held-selection updates and extremal retention 62
58. Extremal selection of the added carrier member 63
59. Generated graphs, complete paths and cuts 63
60. Finite reachability search 64
61. Complete path witnesses for successful reachability 65
62. Reachability completeness and equivalence 66
63. Exact finite cycle traversal 67
64. Cycle detection and complete return paths 67
65. Graph witnesses and complete registered grammar 68
66. Graph induction and locality of path updates 69
67. Admitted graph extension, ordered pairs, cuts and balance 70
68. Held-partition updates and exact paired-balance conditions 70
69. Batch source-program correspondence 71
70. Complete Part Equivalence source correspondence 72
71. Finite Form source input correspondence 73
72. Completed mathematics source entries 74
73. Completion of the represented foundation and mathematics entries 75
74. Finite incidence and open-family derivations 76
75. Boundary-depth dimension 77
76. Ordered undirected path search 78
77. General shortest-path and reachability derivations 78
78. Completed finite geometry and topology entry 79
79. Exact events and common-refinement weights 80
80. Pair-cell factorization and exact modal summaries 81

81. Observation partitions and completed finite probability entry 82
82. Exact finite optimization and retained alternatives 83
83. Generated trajectories and predecessor records 84
84. First returns and complete registered perturbation stability 85
85. Finite logic and registered proof traces 87
86. Compositional paths and finite type constructors 88
87. Identity, transport and route comparison 89
88. Symbols and observation-relative distinction 90
89. Observation classes, complete distinction ledgers and extension 91
90. Encoding classes and exact inverse decoding 92
91. Closed encoding distinctions and exact composition 93
92. Encoding source extension and finite closure 94
93. Complete information support and position generation 95
94. Exact information profiles and independent position traces 96
95. Deterministic entropy partitions and observation refinement 98
96. Exact entropy parts and closed-pair monotonicity 99
97. Entropy successor closure and exact finite completion 100
98. Exact compression descriptions and reconstruction provenance 101
99. Prefix parsing and the complete-support shortening boundary 103
100. Finite channel admission and exact confusability 104
101. Complete finite capacity families and relation composition 105
102. Fresh-input closure of exact channel capacity 107
103. Registered action relations and exact correction 108
104. Complete position traces, detection and action succession 110
105. Complete coding support and held codebooks 111
106. Complete single-label error images and exact decoding 112
107. General coding action succession and retained source provenance 114
108. Exact observation incidence and conditional class restriction 115
109. Exact joint fractions and marginal-product comparison 116
110. Conditional fractions, resolution and retained dependence 117
111. Fresh-source joint and conditional succession 118
112. Complete mutual and conditional program correspondence 119
113. Exact conservation and loss of source distinctions 120
114. Composition and downstream retention 121
115. Exact predecessor records and recovery 122
116. Reversibility is equivalent to singleton fibres 123
117. Source succession and conservation of pair accounting 124
118. Complete conservation and transformation program correspondence 125
119. Classical state, observation classes and deterministic realization 126
120. Complete observation-class cardinality 126
121. Exact-whole closure and microstate succession 127
122. Classical and probabilistic operational witnesses 128
123. Complete classical and probabilistic correspondence 129
124. Quantum-information branch support and provenance 130
125. Basis records and joint support 130
126. Independently realized information-unit traces 131
127. Exact support observation and reconstruction 132
128. Canonical position succession 132
129. Quantum support program correspondence 133
130. Canonical Fold basis bijection 134
131. Exact generated transition systems 135
132. Retained execution composition and extension 135
133. Generated words and exact coordinate rewrite 136
134. Finite language derivation certificates 137
135. Finite recognition with retained accepting classes 138
136. Retained held-position rewrite traces 138
137. Registered rewrite boundaries and trace classes 139
138. Structural finite recursion with complete values 140
139. Fold binding and self-application calculus 141
140. Separate FORMX source components 142
141. Finite executable descriptions and halt traces 143
142. Circuit interfaces and complete evaluation traces 143
143. Operational process observations and failure traces 144
144. Process composition and exact decomposition 145
145. Registered step-to-trace simulation 145
146. Finite registered model translation and reconstruction 146
147. Interpretation of frozen generated descriptions 147
148. Recognition, decision and unresolved execution 148
149. Terminal witnesses and the held halting boundary 149
150. Effective finite enumeration and reconstruction 149
151. Explicit reductions and answer reconstruction 150
152. Finite relative computability classes 151
153. Universal-machine execution and recognition 152

154. Generated diagonal contradiction 153
155. Computational proof-system boundary 154
156. Declared relative answer interfaces 155
157. Hypercomputation admissibility boundary 155
158. Exact generated input extent 156
159. Time and retained-state resources 157
160. Circuit structural resources 158
161. Communication and query resources 159
162. Scheduled unresolved branch resources 159
163. Information retained for inverse execution 160
164. Independent and causally layered work 161
165. Finite resource comparison certificates 162
166. Resource-preserving reductions and completeness 162
167. Complete instance-family resource profiles 163
168. Exact approximation and retained objective distinction 164
169. Resource stratification by structural coordinates 165
170. Shortest exact descriptions at declared finite depth 165
171. Exact search on registered carriers 166
172. Exact ordering and complete recursion traces 167
173. Exact arithmetic execution certificates 167
174. Finite sequence matches and position reconstruction 168
175. Finite graph traversal certificates 169
176. Finite algebraic and geometric execution 169
177. Exact subproblem reuse and reconstruction 170
178. Complete optimization traversal 171
179. Scheduled algorithms and exact success support 171
180. Causally independent algorithm layers 172
181. Global replay of retained local process events 173
182. Exact distributed messages and causal reconstruction 174
183. Finite-prefix streaming and retained distinctions 175
184. Exact oriented numerical-error records 176
185. Complete numerical error-ledger reconstruction 177
186. Retained numerical update observations 178
187. Finite numerical boundary certificates 179
188. Exact symbolic transformations within the frozen grammar 180
189. Exact approximation-bound admission and source rows 181
190. Approximation with complete objective and execution ledgers 182
191. Generated program syntax and complete occurrence records 183
192. Complete name occurrences and registered scope 184
193. Binding-extension formation and exact substitution behavior 185
194. Capture preservation at the evaluator substitution boundary 186
195. Canonical replacement in the registered expression grammar 187
196. Complete source evaluation traces 188
197. Operational and compositional meaning correspondence 189
198. Positive semantic-value typing and preservation 190
199. Generated type support and exact interface derivations 191
200. Program equivalence on registered observations 192
201. Generated descent and terminal coverage 193
202. Correctness against the registered specification 194
203. Generated formal specification relations 195
204. Retained program transformation traces 196
205. Semantics-preserving compilation relations 197
206. Machine-checkable program evidence 197
207. Positive causal relation expansion 198
208. Native causal fixed-point execution 199
209. Exact causal operation boundary and output 200
210. Complete concurrency schedules and local observations 201
211. Computational causality and retained provenance 201
212. Structural generator three: exact recurrence derivation 202
213. Native first-return execution 203
214. Complete Structural Generator Three entry 204
215. Spatial Dimension Three 205
216. Boundary Rank Two 206
217. Inverse fine-structure cover construction 207
218. Terminal inverse fine-structure ratio 208
219. Charged-lepton invariant operations 209
220. Complete charged-lepton symmetric carrier 210
221. Terminal charged-lepton self-coupling 211
222. Terminal grammar and exact root brackets 212
223. Exact terminal mass-ratio refinements 213
224. Exact source rows and complete comparison record 214
225. Registered terminal prediction and empirical validation 215
226. Electroweak Fold-fibre mixing 216
227. Joint synchronization and retained readiness 217
228. Communication and exact record transfer 218
229. Distributed partial orders and retained event identities 219
230. Consensus under a declared communication and fault boundary 220

231. Agreement impossibility from indistinguishable predecessors 221
232. Replication,consistency observations and retained update order 222
233. Declared computational fault support and recovery 223
234. Distributed knowledge from local observations 224
235. Locality from registered communication neighborhoods 225
236. Computation over generated communication networks 226
237. Resource-relative one-wayness and exact security partitions 226
238. Observation-relative secrecy and disclosure 227
239. Integrity of protected records 228
240. Authentication relative to registered identity support 229
241. Hashing fibres, collisions and domain separation 230
242. Commitment binding, hiding and retained phases 231
243. Signature correctness and exact adversary support 232
244. Proof of knowledge and source-bound extraction 232
245. Zero knowledge, simulation and closed witness distinctions 233
246. Multiparty computation and explicit corruption boundaries 234
247. Adversarial computation and complete outcome ledgers 235
248. Observation-based and resource-bounded security 236
249. Classical security under an enlarged interface boundary 237
250. Inference from generated hypotheses and exact evidence 237
251. Sealed classification, prediction and target custody 238
252. Representation grammar and preserved distinctions 239
253. Generalization and exhaustive target-family certificates 240
254. Sample complexity and constructive teaching witnesses 241
255. Exact optimization over evidence-compatible hypotheses 241
256. Computational induction and exact successor tests 242
257. Search and planning through exact execution traces 243
258. Reinforcement computation through generated interaction traces 244
259. Multi-agent computation and returning joint traces 245
260. Trace-bound adaptation with preserved constitutional identity 245
261. Learning limits from indistinguishable evidence 246
262. Interpretability through exact verified decision certificates 247
263. Complete classical learning reports 248
264. Exact values for capability-closed prediction 248
265. Admission and exact generated operations 249
266. Complete finite prediction execution and trace accounting 250
267. Data-only prediction documents and the complete capability grammar 251
268. Exact reference forms and bounded approximations 252
269. Exact perturbations and registered stability 252
270. Local error transport and terminal provenance 253
271. Generated accumulated bounds for finite exact traces 254
272. Finite discretization and refinement provenance 255
273. Registered convergence traces and successor certificates 256
274. Typed scientific symbolic traces and normalization 257
275. Registered simulation and complete observation traces 258
276. Exact computational dynamics and orbit certificates 258
277. Inverse problems and retained predecessor fibres 259
278. Computational statistics from exact support traces 260
279. Complete coordinate products and high-dimensional computation 261
280. Many-body joint support and interaction records 262
281. Prediction seals and registered target access 263
282. Formal structures, observations and retained model comparisons 264
283. Reversible gates and exact phase-labelled support 265
284. Exact inverse phase and branch reconstruction 266
285. Admitted gate round trips 267
286. Complete joint-state composition 267
287. Complete observation and predecessor records 268
288. Complete repetition fault-mask enumeration 269
289. Arbitrary-depth repetition correction 270
290. Reversible-model grammar and operational controls 270
291. Complete finite reversible execution and interpretation 271
292. Fold quantum information unit 272
293. Joint-state provenance and marginal reconstruction 273
294. Complete generated support and phase retention 274

295. Exact phase period and predecessor interference 275
296. Exact component correlations and retained observation provenance 275
297. Exact measurement selection and complete reconstruction 276
298. Exact reversible gate branch correspondence 277
299. Exact circuit evaluation, inverse and observation ledger 277
300. Exact simulation of the frozen circuit-description grammar 278
301. Exact algorithm branch derivations and decoded outcomes 279
302. Exact live-support and measurement resource counts 280
303. Exact communication provenance and complete joint records 281
304. Exact coding reconstruction and disjoint error supports 281
305. Complete error census and successor recovery 282
306. Recovery at every contained circuit location 283
307. Exact simulator execution and observation correspondence 284
308. Complete quantum verification challenge schedules 284
309. Joint hypothesis-search provenance and retained alternatives 285
310. Bidirectional classical and quantum operational correspondence 286
311. Registered self-description and quantum computation limits 287
312. Complete target-package custody checks 288
313. Named-target admission and portable custody grammar 288
314. Data-only package admission and protected-tree retention 289
315. Physical observation, reference pairing and complete comparisons 290
316. Source-bound observation validation and prediction sealing 291
317. Half-One vacuum floor and complete finite oscillator spectrum 292
318. Exact cosmic-ratio continuations and outward source intervals 293
319. Dual quark cubics and first complete dyadic separation 294
320. Squared Planck/proton support and terminal transport 295
321. Local vacuum support and separately typed cosmic magnitude 296
322. Complete prime-sector construction 297
323. Exact Fold return and binary multiplicative order 298
324. Counted binary transient and complete orbit support 299
325. Complete cyclotomic counting and source-loop termination 300
326. Completed orbit, prime-sector and base hierarchy verification 301
327. Exact positive-ratio composition 301
328. Unique positive holding complement 302
329. Exact-relations operational witnesses and complete grammar 302
330. Positive local separation before casting 303
331. Least common return of every positive finite cycle family 303
332. Held orientation and exact shorter-part selection 304
333. Completed exact-relations verification 304
334. Positive finite refinement and exact counted return 304
335. Exact finite-difference identities 305
336. Complete finite dyadic closure 305
337. Exact rational execution of the finite-refinement witnesses 306
338. Complete finite traces and the typed object boundary 306
339. Completed finite-refinement and continuum-boundary verification 307
340. Terminal electroweak support from the promotion object 307
341. Exact terminal electroweak share and positive complement 308
342. Positive-take domain and arbitrary positive running levels 309
343. Complete labelled electroweak channel supports 309
344. General odd-denominator vacuum recurrence 310
345. Exact odd-denominator orbit interface and returned traces 310
346. Exact Fraction and held-rank recurrence correspondence 311
347. General success of the original odd-denominator first-return loop 312
348. Original vacuum recurrence witnesses 312
349. Completed odd-denominator vacuum recurrence 313
350. Complete reference-equivalence classes and retained records 313
351. Exact same-dimension reference transport 314
352. Finite canonical dimension composition 315
353. Complete finite decision of held dimension identity 316
354. Reference transport on equivalent generated descriptions 317
355. Completed measurement carrier, dimension and unit entries 317
356. Operational realization with a complete retained trace 318
357. Measured values as separate records 319
358. Complete retention of measurement uncertainty 319

359. Calibration with aligned comparison and uncertainty histories 320
360. Exact source-byte hashing equivalence 321
361. Dimensional consistency at every finite comparison 321
362. Completed measurement closure 322
363. Complete observed events and retained change 323
364. Duration from generated recurrence 324
365. Reference-relative location and held displacement orientation 325
366. Exact speed and oriented velocity 325
367. Completed initial mechanics chain 326
368. Canonical identity under the proved byte-hash equivalence 327
369. Completed dark-to-baryon fraction 327
370. Source-bound inertial continuation 328
371. Complete inertia–velocity pair cells 329
372. Exact oriented scalar changes 330
373. Acceleration from the complete velocity comparison 331
374. Force from source-bound momentum transfer 332
375. Mechanical work and retained energy transfer 332
376. Power as the rate of the retained energy transfer 334
377. Angular motion: exact phase in a retained closed recurrence 334
378. Angular momentum and source-bound torque 335
379. Complete constituent weighting and the composite centre 336
380. Closed mechanical transfer and exact carrier conservation 337
381. Reversible restoration, finite recurrence and exact period 337
382. Complete source-bound field support 338
383. Geometric dilution and the unique conserved share 338
384. Weak relational gravitation and complete inertial-source response 339
385. Conserved electric distinction and held orientation 340
386. Charge-normalized electric field and potential difference 340
387. Completed Hubble calibration from the generated partition and first return 341
388. Oriented electric transport and the magnetic response record 342
389. Linked magnetic change and complete cyclic electric work 343
390. Common electric-magnetic source and propagation closure 343
391. Gauge equivalence through complete observable records 344
392. Generated adjacency and completed causal accessibility 345
393. Terminal on-shell electroweak share 345
394. Detached radiative recurrence with its complete field carrier 347
395. Interaction classes from complete source, carrier and response traces 348
396. Exact interior change and boundary flow 348
397. Action and reaction as paired held transfer records 349
398. Period and frequency from complete recurrence counts 349
399. Adjacent propagation retaining source, frequency and phase 350
400. Exact wavelength, period and propagation speed 350
401. Complete labelled wave superposition 351
402. Common-cell phase-labelled predecessor merging 351
403. Aperture selection and complete reachable-support regeneration 352
404. Held transverse polarization 352
405. Recurrence-labelled support response and dispersion 353
406. Exact recurrence matching and resonance 353
407. Wave energy and held momentum along a causal path 354
408. Physical quantum support and preparation 354
409. Closed reversible physical evolution 355
410. Observables, selected fibres and retained records 356
411. Exact quantum branch-support weights 356
412. Record-preserving joint refinement and incompatibility 357
413. Physical spin as finite held cyclic action 357
414. Indistinguishability through complete exchange traces 358
415. Exclusion in the alternating exchange class 358
416. Tunnelling from complete generated support 359
417. Discrete spectra from exact boundary recurrence 360
418. Entanglement as nonfactorable complete joint support 360
419. Bell correlations and the omitted-dependence boundary 361
420. Contextuality and complete observation provenance 362
421. Decoherence through reversible environmental records 362
422. The environment-distinguished classical quotient 363
423. Complete remote fibres and no-signalling 364
424. Mass-energy from two propagation transfers 364

425. Particle-antiparticle reversal of held labels 365
426. Spin action and exchange-composition classes 366
427. Complete transitionwise conservation of physical labels 367
428. Minimal recurrent trace classes and the observed particle spectrum 367
429. Boundary-closed composite hadrons 368
430. Complete scattering channels and exact cross-section measure 369
431. Conserved particle decay transitions 369
432. State mixing as cyclic overlap of observation refinements 370
433. Nuclear binding and the positive released carrier 371
434. Finite nuclear recurrence classes and carrier transitions 371
435. Radioactivity and complete deterministic path weights 372
436. Complete nuclear reaction channels 373
437. Fission as conserved multiple-product decomposition 373
438. Fusion and the exact positive release 374
439. Exact colour coupling from complete Fold support 374
440. Closed charged-colour proton-to-Planck hierarchy 375
441. Exact charged and neutral source-count running 376
442. Exact vacuum screening and one-Fold exposure 377
443. Self-similar support and exact finite-tail convergence 377
444. Source-bound relational spacetime events 379
445. Canonical interval under reference relabeling 380
446. Causal accessibility on finite recurrence support 380
447. Complete prime-sector force and particle inventory 381
448. Exact source-boundary growth discrimination 382
449. Generated common support and finite convergence witnesses 383
450. Inertial-frame correspondence from complete invariant records 384
451. Post-seal running comparison and exact measurement intervals 385
452. Elementary causal support and the invariant limiting ratio 386
453. Relative clock accumulation on complete causal paths 387
454. Complete simultaneous support and relative spatial extent 387
455. Universal free response and local inertial–gravitational equivalence 388
456. Source-bound transport and relational curvature 389
457. Complete inertial recurrence along an unforced curved path 389
458. Realization and minimality of the complete geodesic trace 390
459. Complete inertial-energy source and gravitational boundary closure 391
460. Gravitational-wave propagation 392
461. Causal horizon boundary 392
462. Propagation redshift relation 393
463. Relational expansion observation 394
464. Background-radiation field relation 395
465. Gravitational lensing relation 396
466. Causal distance and observation-time relation 397
467. Finite structure growth and retained histories 398
468. Physics-to-astronomy/cosmology handoff 399
469. Exact partition closure and spatial flatness 399
470. Complete generated cosmic budget 401
471. Common finite scale axis and terminal electroweak transport 402
472. Terminal cosmic component transport and expansion 405
473. Unified Constants: the foundation-order object 409
474. Unified Constants: terminal continuations and cross-sector equality 411
475. Unified Constants: exact rooted incidence and complete certificate 413

Abstract

Smithian Fold Theory (SFT) develops a structural theory of everything with zero axioms and zero free scientific parameters. This cumulative preprint presents its one-to-one native Lean formalisation, preserving the definitions, dependency order, derivations, complete candidate grammars, controls and empirical boundaries of the existing V3 model. The opening result is the exact terminal inverse fine-structure ratio, $\alpha^{-1} = 503846395469/3676744786$, derived through generated binary covers and the complete finite promotion ladder before measurement. The Unified Constants milestone assembles a sixteen-field foundation-order vector and twenty-six terminal readings across seven sectors through one exact rooted incidence graph. Constructive proofs establish all seven sector paths, twelve generator-dependent changes, four held controls and fourteen common-order equalities; the complete 4,096-form grammar retains the original operational witnesses and controls. The manuscript develops the wider model from its operational root through mathematics, information, computation and the completed physics derivations, with mathematical expressions accompanying the exposition. This edition covers 308 completed entries of the 2,781-entry V3 census, 17,849 audited theorems and 38,713 audited definitions and theorems, all with zero axiom dependencies. The completed entries cover 142,354 original candidate forms. Source correspondence and reproducible verification evidence accompany the paper. Subsequent milestones will extend the same cumulative manuscript until the model's formalisation is complete.

1. Opening result: the fine-structure constant

The opening result connects the structural foundation of SFT to an exact dimensionless ratio. The complete derivation is carried into native Lean from the pinned V3 model. Each numerical block retains its generated structural role; the terminal order is fixed by exhaustion of the three-direction promotion object.

Let b = 2 be the positive predecessor of the independently derived generator c = 3. The least complete binary covers of the carriers c^c = 27 and c^(c + 1) = 81 have depths 5 and 7. The independent decompositions give 5 = 2 + 3 and 7 = 3 + 4. The depth inequalities are

$$2^4 = 16 < 27 \leq 32 = 2^5, \qquad 2^6 = 64 < 81 \leq 128 = 2^7.$$

Since successive positive binary powers increase, these inequalities exclude every earlier positive depth and establish least coverage. These generate the tower 2^7 = 128, boundary 3^2 = 9 and complete cover 2 x 5^3 = 250.

Promoting each of the three interchangeable directions once generates the complete finite ladder 125, 175, 245, 343. Each step replaces one down-depth factor by an up-depth factor. After all three directions have been promoted, no successor is admitted. Explicitly, the successive products are

$$5 \cdot 5 \cdot 5 = 125, \quad 7 \cdot 5 \cdot 5 = 175, \quad 7 \cdot 7 \cdot 5 = 245, \quad 7 \cdot 7 \cdot 7 = 343.$$

The remaining down-direction counts are respectively $3, 2, 1, 0$; a successor replaces one remaining down factor, so the fourth product exhausts the construction. The first rung, 125, supplies the base cover $250 =$

2×125 and hence the separate leading-order result; the terminal fourth-order assembly adds the reciprocal continuation built from the remaining rungs 175, 245 and 343. The terminal effective cover and inverse fine-structure ratio are:

$$D = 250 + \cfrac{1}{175 + \cfrac{1}{245 + \cfrac{1}{343}}},$$
$$\alpha^{-1} = 128 + 9\frac{D+1}{D} = \frac{503846395469}{3676744786},$$
$$\alpha = \frac{3676744786}{503846395469}.$$

The source evaluates the reciprocal continuation from the terminal rung backwards. Its intermediate rational values are

$$245 + \frac{1}{343} = \frac{84036}{343}, \qquad 175 + \frac{343}{84036} = \frac{14706643}{84036}, \qquad D = 250 + \frac{84036}{14706643} = \frac{3676744786}{14706643}.$$

All rung values are positive, so each reciprocal is defined. Substitution gives

$$128 + 9\frac{D+1}{D} = \frac{128(3676744786) + 9(3676744786 + 14706643)}{3676744786} = \frac{503846395469}{3676744786}.$$

The leading rung yields $\alpha^{-1} = 34259/250$. The terminal fourth order follows from the complete generated ladder. It is not selected by a numerical fit. The source construction reads no measured value of alpha. Its separately registered empirical comparison retains the original post-seal boundary.

Native proofs establish the least-cover property, complete promotion stages, decreasing remaining-direction count, terminal closure and exact rational identities. All 1,024 registered candidate forms are retained, together with the three operational witnesses and four controls. Complete source-program comparisons retain candidate order, decisions, reasons, output fields and hashes. The entry is SFT-PHYS-CONSTANT-INVERSE-FINE-STRUCTURE-001; its nine registered dependencies were completed before it.

In this edition, zero axioms means the audited Lean declarations report an empty axiom-dependency set. Zero free scientific parameters means that the scientific construction uses the generated V3 quantities and original constraints, without fitted coefficients or measurement-fed choices. Host indices and explicit inputs of general finite operations retain their source roles. The exact expression above and its proof constitute the opening result of the wider manuscript.

> Source, proof and provenance: V3 minimal_binary_cover, sft/physics/atomic_constants.py:52; V3 fine_structure_blocks, sft/physics/atomic_constants.py:65; V3 promotion_rungs, sft/physics/atomic_constants.py:87; V3 effective_cover, sft/physics/atomic_constants.py:102; V3 inverse_fine_structure, sft/physics/atomic_constants.py:117; V3 source derivation, claims/SFT-PHYS-CONSTANT-INVERSE-FINE-STRUCTURE-001/WHY_DERIVATION_CHECK.md:1. Native results: SFT.Physics.AlphaCovers.completeBinarySupport, AlphaCovers.lean:132; SFT.Physics.AlphaCovers.leastCompleteSupport, AlphaCovers.lean:136; SFT.Physics.AlphaBlocks.nativeBlockDerivation, AlphaBlocks.lean:51; SFT.Physics.AlphaPromotion.completeNativeLadder, AlphaPromotion.lean:100; SFT.Physics.AlphaRatio.terminalRatio, AlphaRatio.lean:127; SFT.Physics.AlphaRatio.automaticTerminal, AlphaRatio.lean:129; SFT.Physics.AlphaDomain.exactOrderDomain, AlphaDomain.lean:48. Timeline: Turn 270, line 2724; Timeline: Turn 270, line 2725; Timeline: Turn 271, line 2732; Timeline: Turn 271, line 2733. Evidence: Completion TURN_271_ALPHA_COMPLETION.

2. A cumulative paper, released at scientific milestones

The paper develops SFT as a unified theory and grows with the native verification of the full V3 model. Alpha introduced the first release. This edition adds the completed Unified Constants object and its seven-sector assembly. Future opening results will feature further major derivations when their source entries are

complete. Expanded editions are planned at 500 completed entries and every 250 completed entries thereafter, with a final edition at all 2,781 entries. Each edition incorporates all newly completed scientific material and identifies its corresponding source and verification snapshot.

The full scientific exposition follows in the same manuscript. The release includes the chronological timeline, coverage census, source correspondence, native Lean sources and retained verification evidence as separately accessible companion files. Development history belongs to the timeline; scientific definitions, derivations, methods and results belong to the paper.

Edition references: V3 source snapshot; completed-entry census; Unified Constants completion receipt; Timeline: Turn 515, line 5101. Representative native result: AlphaRatio.terminalRatio.

3. Reading and reproducing this edition

Start with the operational root in Section 6 and follow the registered dependencies through the structural and mathematical constructions. The physics sections include Structural Generator Three, Spatial Dimension Three, Boundary Rank Two, Inverse Fine Structure, Charged-Lepton Cubic, Charged-Lepton Terminal and Electroweak Fold Mixing. The complete alpha construction is developed under "Inverse fine-structure cover construction" and "Terminal inverse fine-structure ratio".

The immutable scientific code checkpoint for this Unified Constants review edition is 59172eee89576735796ec396f85e8527f0744130 in MettaMazza/SFT-v4-LEAN-VERIFIED. The V3 model is pinned to 76f1b34e9e9682989adcbb088797b4ae4ce4fcf1 in MettaMazza/ernos-labs-sft-platform. Lean is pinned to leanprover/lean4:v4.35.0. COVERAGE.json identifies completed source entries; reference/CORRESPONDENCE.json maps translated sources. The retained TURN_271_ALPHA_COMPLETION.json records the alpha derivation components, and TURN_279_PARITY_REPORT.json records the published-edition aggregate checks. TURN_515_UNIFIEDCONSTANTS_COMPLETION.json and the source-specific completion receipts identified in COVERAGE.json record the Unified Constants milestone. Entry completion and the count of individual theorems are reported separately.

To reproduce the repository checks, install the pinned Lean toolchain and run lake build, followed by python3 verification/check_current_parity.py from the repository root. The latter regenerates native output and performs the configured source-output and axiom checks. Source-specific completion reports retain the additional derivation and complete-program evidence.

Edition references: V3 source snapshot; completed-entry census; Unified Constants completion receipt; Timeline: Turn 515, line 5101. Representative native result: AlphaRatio.terminalRatio.

4. Source and evidence links

V3 model: https://github.com/MettaMazza/ernos-labs-sft-platform/tree/76f1b34e9e9682989adcbb088797b4ae4ce4fcf1
Verified Lean checkpoint: https://github.com/MettaMazza/SFT-v4-LEAN-VERIFIED/tree/59172eee89576735796ec396f85e8527f0744130
Alpha completion evidence: https://github.com/MettaMazza/SFT-v4-LEAN-VERIFIED/blob/59172eee89576735796ec396f85e8527f0744130/verification/TURN_271_ALPHA_COMPLETION.json

Unified Constants completion evidence: https://github.com/MettaMazza/SFT-v4-LEAN-VERIFIED/blob/59172eee89576735796ec396f85e8527f0744130/verification/TURN_515_UNIFIEDCONSTANTS_COMPLETION.json

Edition references: V3 source snapshot; completed-entry census; Unified Constants completion receipt; Timeline: Turn 515, line 5101. Representative native result: AlphaRatio.terminalRatio.

5. Model and scope

The source model is Smithian Fold Theory V3, repository MettaMazza/ernos-labs-sft-platform, commit 76f1b34e9e9682989adcbb088797b4ae4ce4fcf1. Its census registers 2,781 model-admitted claims. The complete formalisation follows all 2,781 entries in dependency order, preserving their original definitions, derivations, declared constraints, controls and empirical boundaries. Each translated component is bound to its V3 source, and full-model verification requires both native proofs of the registered formal statements and correspondence of the associated executable procedures. Empirical records retain their original evidential status. The results presented here cover 308 completed entries across the foundations, mathematics, information science, computation and physics. Their finite-generation derivations and exact source operations are presented below. A finite candidate census enumerates structural alternatives, while the operational theorems quantify over the inputs of the corresponding constructors. For example, Root classification covers every record with two arbitrary metadata strings and a Boolean presentation field; Count retention covers every generated finite trace and every proposed occurrence list; primitive minimality covers every finite word over the four normalized primitive maps. The size of a stored census therefore does not bound those operational quantifiers. Each source-entry completion binds its candidate proofs, operational derivations and executable correspondence; separately translated components retain their stated component scope. See recordClassificationExhaustive, RootDerivation.lean:41, proposalAcceptedIffExactRetention, CountRepresentation.lean:235 and leastGeneratorIff, PrimitiveMinimality.lean:84.

SFT distinguishes the structural One, generated positive finite counts and exact positive parts from host implementation indices. Its derivational constraints exclude added axioms, free or fitted scientific parameters, negative proof quantities, irrational or imaginary proof values, floating equality and completed infinity. Formalisation preserves the registered source boundary of each result.

Edition references: V3 source snapshot; completed-entry census; Unified Constants completion receipt; Timeline: Turn 515, line 5101. Representative native result: AlphaRatio.terminalRatio.

6. Operational root

Let $R = \mathrm{String} \times \mathrm{Bool} \times \mathrm{String}$ be the source record schema, retaining candidate identity, presentation flag and descriptive form. The two strings are arbitrary finite metadata; the Boolean records whether a challenge is presented at the operational proof boundary. Let P(r) be that presentation flag. Write U for unpresented-absence and O for presented-occurrence. Classification and acceptance are

$$\kappa(r) = U \text{ if } P(r) = \mathrm{false}; \kappa(r) = O \text{ if } P(r) = \mathrm{true}$$
$$A(r) = P(r)$$

The Boolean cases are exhaustive, so for every r∈R,

$$A(r) = \mathrm{true} \iff \kappa(r) = O$$
$$\kappa(r) = \kappa(s) \iff P(r) = P(s)$$

In the generated grammar G=[U,O], A(U)=false and A(O)=true. Filtering G therefore gives [O], proving both existence and uniqueness of the survivor. Deleting O gives filter_A([U])=[], while the modified acceptance A′(U)=A′(O)=true gives filter_A′(G)=[U,O]. Since U≠O, the modified survivor set is not unique. The typed boundary concerns these operational presentation records; neither identifier nor descriptive text enters P.

The registered operational challenge partition has exactly two classes: unpresented-absence and presented-occurrence. An unpresented challenge supplies no counterexample. A presented challenge supplies an occurrence and is therefore not nothing at the operational boundary.

The decision accepts precisely presented-occurrence. Exhaustive enumeration establishes that both registered classes occur in the candidate list. The unique-survivor theorem establishes that a candidate is accepted if and only if it is presented-occurrence. No premise about a particular presentation's internal content is introduced.

Every source-schema record belongs to one of the two presentation classes. Deleting the presented-occurrence class from the two-class census leaves no accepted candidate. The decision is definitionally the presentation predicate. Three additional native proofs establish the false-premise, tampered-artifact and fixed-boundary control outcomes. The source census, decisions, closure evidence and all four control records are emitted by native Lean definitions; every stored field and digest agrees with the pinned V3 artifacts.

The tampered-artifact premise is represented by an acceptance function that accepts both source classes. Its survivor set is exactly the original two-element candidate list. Constructive proofs establish that the two survivors are distinct and that no unique survivor exists. These statements formalise the source rejection condition while retaining the original control record for exact artifact correspondence.

The record-level formalisation retains exactly candidate identity, presentation flag and descriptive form from the source dictionary. Classification is derived from the presentation flag. For every record in this schema, its classified candidate has the same presentation status; classifying a generated representative returns that representative. Two records have the same class if and only if their presentation flags agree. Thus every such record maps into the registered exhaustive partition, independently of its identifier or descriptive text.

The source decision agrees with this classification for every record. Acceptance is equivalent to the presented-occurrence class and to the source operational presentation witness; an accepted record cannot be unpresented. Changing either metadata string changes neither acceptance nor its reason. Deleting presentation rejects the record. These statements formalise the source record decision and operational presentation relation, while preserving all three source fields and the generated evidence values.

The complete registered Root entry is formalised: its source definitions, operational derivation and closure statements, exhaustive candidate partition, four controls and program methods are represented. Fresh native execution reproduces the pinned census, decisions, closure, control records and their hashes; registration fields and refusal of unknown identifiers agree. The source program declares no prior dependencies, axioms or free scientific parameters. Completed source-entry totals and individual Lean theorem counts are reported separately in the current verification summary.

Source claim: SFT-ROOT-THERE-IS-NO-NOTHING.

The partition is exhaustive because Boolean elimination gives exactly the two cases of P(r). Applying the presentation projection to equal classes proves equal flags; conversely equal flags select the same classifier branch. Substituting this equality into the decision gives acceptance exactly at O for every record. Thus arbitrary changes in the metadata strings, including their length and notation, preserve both decision and reason. These steps are classificationExact, RootDerivation.lean:44, decisionAgreesWithClassification, RootDerivation.lean:54, everyAcceptedRecordHasSurvivorClass, RootDerivation.lean:58 and reasonIndependentOfMetadata, RootDerivation.lean:88. The source defines an operational challenge through presentation; the universal statement is over that explicitly constructed record schema.

> Source, proof and provenance: V3 decision_record, sft/foundation/root_theorem.py:84; V3 closure_record, sft/foundation/root_theorem.py:97. Native results: SFT.Foundation.Root.uniqueSurvivor, Root.lean:38; SFT.Foundation.Root.Derivation.classificationExact, RootDerivation.lean:44; SFT.Foundation.Root.Derivation.acceptanceIffWitnessedOccurrence, RootDerivation.lean:73; SFT.Foundation.Root.Gate.registrationWithAxiomRejected, RootGate.lean:75; SFT.Foundation.Root.Gate.registrationWithParameterRejected, RootGate.lean:82. Timeline, line 11; Timeline: Turn 011, line 169; Timeline: Turn 008, line 111. Evidence: Completion TURN_111_ROOT_COMPLETION.

7. Structural One

Let C={none, proper, complete} denote root coverage and E={no-extra, has-extra} denote added coverage. The source relation domain is C×E, with 3·2=6 classes. Its survival predicate is

$$S(c,e) \iff (c = \text{complete}) \wedge (e = \text{no-extra})$$

For c=none or c=proper the first conjunct fails; for e=has-extra the second fails. The pair (complete,no-extra) satisfies both, hence

$$\{(c,e) \in C \times E : S(c,e)\} = \{(\text{complete}, \text{no-extra})\}$$

This singleton is exact-self-whole. For every source record r, the classification map retains precisely its two relations, so S(r)⇔κ(r)=(complete,no-extra). Metadata changes preserve both relations and therefore preserve the decision. These relations express structural wholeness before any numerical magnitude is assigned.

A representation of the root occurrence is classified by its coverage—none, proper or complete—and by whether it adds unforced material. The product gives six classes: omitted-root, replacement-extra, fragmented-root, fragment-plus-extra, exact-self-whole and whole-plus-extra, in source enumeration order.

The representation survives precisely when its coverage is complete and it contains no extra material. Thus exact-self-whole is the unique surviving class. Omission, fragmentation and addition are rejected by the same ordered conditions as in V3. The One is structural self-wholeness, not a numerical magnitude introduced at this stage.

The record-level derivation uses the three source root-coverage relations and two source extra-coverage relations. Their generated product is proved identical, in order, to the six stored candidates. Classification retains both relations and is inverted by the generated representatives. The four source record fields, survival decision and ordered reason all agree with the generated records.

For every record in that relation domain, acceptance is equivalent to complete coverage together with no extra material, and hence to the exact-self-whole class. Replacing coverage by none or proper rejects the record; setting the extra relation to has-extra also rejects it. Acceptance and its reason are unchanged by arbitrary identifier or descriptive-text changes. These universal record statements formalise the source omission, fragmentation, addition and metadata-independence arguments without introducing a numerical magnitude.

The complete One source program preserves its registered dependency, all six ordered candidates, actual identifier-based decisions, closure and generality records, and four original controls. Native lookup correctness and the non-hash control outcomes are proved. Fresh execution matches every corresponding pinned field and digest, including refusal of an unknown identifier. Together with the general derivation above, this completes the registered entry.

Source claim: SFT-FOUNDATION-ONE-001. Registered dependency: the operational root.

> Source, proof and provenance: V3 survives, sft/foundation/one.py:69; V3 closure_record, sft/foundation/one.py:101. Native results: SFT.Foundation.One.uniqueSurvivor, One.lean:61; SFT.Foundation.One.survivorAccepted, One.lean:64; SFT.Foundation.One.Derivation.acceptedIffCompleteWithoutAddition, OneDerivation.lean:87; SFT.Foundation.One.Derivation.acceptedIffExactSelfWhole, OneDerivation.lean:98. Timeline, line 9; Timeline: Turn 013, line 184. Evidence: Completion TURN_112_ONE_COMPLETION.

8. Exact positive finite count

Let B be the base generation trace with its sole occurrence position o. For an extension of trace t, write lift for the embedding of prior positions and ν for the fresh terminal, with ν outside the image of lift. Define complete retention recursively by

$$R(B) = [o]$$
$$R(\mathrm{extend}(t)) = \mathrm{map}(\mathrm{lift}, R(t)) +\!\!+ [\nu]$$

The base is nonempty and contains its generated position exactly once. If R(t) contains every generated position exactly once, injectivity of lift preserves those occurrences and their uniqueness; freshness of ν prevents a repetition when it is appended. Thus induction establishes nonemptiness, complete coverage and exact-once retention at every finite extension.

For a proposed finite retention L, let m_L(x) be the number of occurrences of generated position x, and Extra(L) indicate an outside-trace occurrence. The admitted condition is

$$(\forall x \mathrm{generated} byt, m_L(x) = 1) \wedge \neg \mathrm{Extra}(L)$$

Deleting a generated x makes m_L(x)=0; duplicating it makes m_L(x)>1; an outside addition makes Extra(L) true. Each modification therefore fails one explicit conjunct. Retaining only map(lift,R(t)) at an extension fails at ν. The grammar count is 2+2·2·2=10: two extra-material cases with no coverage, and two coverage cases (proper or complete), two multiplicities and two extra-material cases. Its unique admitted class is complete, once, no-extra.

V3 defines positive finite count through the complete generation trace of a nonempty finite succession of structural-One occurrences. Every generated occurrence is retained once and no ungenerated occurrence is added.

Its representation grammar has ten classes. With no coverage, multiplicity is not applicable and the two extra-material classes remain. With proper or complete coverage, multiplicity is once or duplicated, independently paired with absent or present extra material. Only complete-coverage__once__no-extra survives. The formal classification retains this exact domain, including the absence of a multiplicity choice in the no-coverage case.

The ten classes record coverage, multiplicity and outside addition for a proposed occurrence list. For every generated finite trace t and every such proposal L, the classifier computes these relations and selects one of the ten classes. Its semantic equivalence is complete retention of every generated position, no repeated retained position and no ungenerated occurrence. The base/successor construction below supplies an accepted proposal at every finite depth; the ten-class census records the possible relation outcomes at each depth. classificationExhaustive, CountRepresentation.lean:137 and proposalAcceptedIffExactRetention, CountRepresentation.lean:235 establish these two links.

The base/successor argument is encoded as finite generation syntax: a structural-One base followed by extensions, each retaining prior occurrence positions and introducing a fresh terminal. Positions denote generated occurrence identities within the trace. They are not interpreted as a numerical magnitude. The complete representation is constructed by retaining the base and appending each newly generated terminal once.

Structural induction proves that every generated position occurs in the complete trace, that no position is repeated, and that the trace is nonempty. The previous complete trace is proper coverage of an extension: all its positions remain, but the fresh terminal is absent. Appending that terminal restores completeness. Repeating any generated position violates exact multiplicity; removing a position loses complete coverage. The two source complete-generation clauses determine a unique trace identity at every finite

extension. Finite proposed representations are then classified by their retained generated positions and presence of ungenerated material. The latter is represented by the source outside-trace relation; extra-object content is not a criterion of the source classifier.

Constructive position equality and membership establish the semantic connection to the ten-class grammar. Complete coverage is equivalent to retaining every generated position. The multiplicity predicate is equivalent to absence of repeated retained positions. The extra-material predicate is equivalent to the presence of an ungenerated occurrence. For every finite proposal, survival is therefore equivalent to complete exact-once retention with no ungenerated material. Canonical complete proposals survive; any missing position, repeated retained position or outside addition is rejected. The previous complete prefix of an extension is rejected because it omits the new terminal.

Execution comparisons cover 493 proposals over one-, two- and three-occurrence traces, reaching all ten source classes and matching their decisions and reasons. These bounded comparisons supplement the universal Lean equivalence; they do not replace it.

The complete Count source program preserves its registered dependency, all ten ordered candidates, actual identifier-based decisions, closure and generality records, and four original controls. Native lookup correctness and the non-hash control outcomes are proved. Fresh execution matches every corresponding pinned field and digest, including refusal of an unknown identifier. Together with the general derivation above, this completes the registered entry.

Source claim: SFT-FOUNDATION-COUNT-001. Registered dependency: the structural One.

> Source, proof and provenance: V3 closure_record, sft/foundation/count.py:111. Native results: SFT.Foundation.Count.TraceDerivation.finiteGenerationPreservesExactRetention, CountTrace.lean:176; SFT.Foundation.Count.Representation.proposalAcceptedIffExactRetention, CountRepresentation.lean:235; SFT.Foundation.Count.uniqueSurvivor, Count.lean:90; SFT.Foundation.Count.survivorAccepted, Count.lean:93. Timeline: Turn 014, line 199; Timeline: Turn 015, line 214; Timeline, line 13. Evidence: Completion TURN_112_COUNT_COMPLETION.

9. Exact positive part coordinate

Let W be the nonempty generated fibre trace of the registered partition and H(x) the held-selection predicate. The selected trace is S=filter_H(W). For every fibre x,

$$x \in S \iff x \in W \wedge H(x)$$

Consequently S⊆W, and filtering cannot duplicate a fibre absent repetition in W. A witness x∈W with H(x) makes S nonempty. At the host-record level this gives 0<length(S)≤length(W). The coordinate retains the generation records themselves:

$$\mathrm{coord}(S,W) = (S,W), \pi_1(\mathrm{coord}(S,W)) = S, \pi_2(\mathrm{coord}(S,W)) = W$$

If a coordinate has these same two projections, ordered-pair equality identifies it with (S,W). The registered admission condition is the conjunction of complete partition coverage, disjointness, equal fibres, an internal held selection, presence of the held record, presence of the whole record and absence of an extra coordinate. Failure of any one conjunct rejects the form. No cross-partition equivalence follows at this stage; that relation is derived in the subsequent common-refinement construction.

The Part derivation registers a finite partition of the structural One and a held selection. Its seven relation domains classify partition coverage, overlap, fibre equality, selection relation, held-count presence, whole-count presence and unforced-extra-coordinate presence. Their source-ordered product contains 192 forms. A native theorem establishes that acceptance is equivalent to all seven requirements: complete, disjoint,

equal-fibred, within-partition, both counts present and no extra coordinate. Exactly that form survives. Removing either count, omitting whole coverage, allowing overlap or unequal fibres, using an empty or outside selection, or adding an unforced coordinate is proved to fail.

For a registered whole's finite generated fibre trace, a held predicate constructs the selected trace. Constructive proofs establish containment and absence of repeated fibres. A witnessed held fibre makes the selection nonempty. Its host-record length cannot exceed the whole trace length; this expresses the finite containment bound without installing a new magnitude in the model. The exact coordinate is the pair of held and whole generation records. Its projections retain those two identities, and the pair is unique from them. Both records are nonempty for a witnessed selection.

The complete Part source program retains the original One and Count dependencies, all 192 candidates, every decision payload, closure and generality certificate, and four controls. Fresh actual identifier-based execution reproduces every pinned field and digest and refuses unknown identifiers. The original boundary control retains the source statement that cross-partition equivalence belongs to a later derivation. Together with the general coordinate proofs, these results complete the registered Part entry.

Source claim: SFT-FOUNDATION-PART-001. Registered dependencies: structural One and Count. Equivalence of coordinates from different partitions is not assumed here; it is the subject of the separately registered common-refinement derivation.

> Source, proof and provenance: V3 closure_record, sft/foundation/part.py:140. Native results: SFT.Foundation.Part.SelectionDerivation.witnessedCoordinatePositiveAndContained, PartSelection.lean:107; SFT.Foundation.Part.acceptanceIffExactCriteria, Part.lean:129. Timeline: Turn 016, line 230; Timeline, line 27. Evidence: Completion TURN_113_PART_COMPLETION.

10. Generated common-refinement pairing

For generated fibre traces A and B, write A⊠B for the ordered list [(a,b): a∈A, b∈B], with A traversed first and B second. Held selections H_A⊆A and H_B⊆B lift to

$$L_A = H_A \boxtimes B, L_B = A \boxtimes H_B$$

Every lifted cell belongs to A⊠B, since both its origins belong to the corresponding whole traces. If these two lifted lists have equal length n, the generated index pairing sends the i-th cell of L_A to the i-th cell of L_B for each 0≤i<n. Its inverse sends the i-th cell back. Thus g(f(x))=x and f(g(y))=y; injectivity follows by applying g to f(x)=f(x′), and onto coverage follows from y=f(g(y)). Unequal lengths cannot supply this complete index pairing.

Identity uses f(x)=x, reversal exchanges f and g, and chaining uses $f_2 \circ f_1$ with inverse $g_1 \circ g_2$. Under a shared descendant trace D, the refinement map is (x,d)↦(f(x),d), with inverse (y,d)↦(g(y),d). Both inverse identities follow by substitution. In the one-of-two/two-of-four example, the selected lifts have lengths 1·4=2·2=4; the order pairing therefore covers both lifts completely.

The common-refinement construction generates cells by traversing each source A label and then each source B label. The selected A lift retains all B descendants of each held A label; the selected B lift retains all A origins of each held B label. Every generated cell has source origins, and each pair of source origins occurs in the construction. The executable label formatting and order are preserved exactly.

The registered generated-bijection procedure pairs finite host indices in order and checks equal trace lengths and bounds. Native proofs establish that the bounds always hold, that acceptance is equivalent to equal finite trace lengths, and that every index on both accepted sides occurs in the generated pairing. A constructive pairing supplies forward and inverse maps with both inverse identities; injectivity and onto coverage follow. Identity, reversal and chaining establish the registered pairing laws. Uniform refinement retains the shared descendant identity and preserves both inverse identities. For the concrete pair-cell

procedure, equal selected-trace lengths remain equal after appending the same descendant trace by nested generation. This is proved structurally without assuming a multiplication law. The source one-of-two/two-of-four example accepts; the unequal source example rejects.

Source claim: SFT-FOUNDATION-PART-EQUIVALENCE-001. Dependencies: One, Count and Part. The exact nine-domain product has 1,024 forms. A constructive product-membership proof establishes exhaustive enumeration. The source identifier-equality decision is proved equivalent to all nine witness requirements; exactly the registered witness survives. Eleven adverse relation cases are proved rejected. Complete selected-origin lifting is proved contained in the common refinement, and the accepted generated pairing is both one-to-one and onto on the lifted trace positions. The complete original program and its evidence records match the pinned V3 source outputs.

> Source, proof and provenance: V3 pair_cells, sft/foundation/part_equivalence.py:86; V3 has_generated_bijection, sft/foundation/part_equivalence.py:98. Native results: SFT.Foundation.CommonRefinement.generatedBijectionIffEqualLength, CommonRefinement.lean:70; SFT.Foundation.CommonRefinement.uniformRefinementPreservesGeneratedPairing, CommonRefinement.lean:232; SFT.Foundation.RefinementSelection.liftedPairingOnto, RefinementSelection.lean:55; SFT.Foundation.PartEquivalence.pairingOntoOnlyRejected, PartEquivalence.lean:289; SFT.Foundation.PartEquivalence.extraHasExtraRejected, PartEquivalence.lean:298. Timeline: Turn 017, line 245; Timeline: Turn 018, line 262; Timeline: Turn 018, line 260. Evidence: Completion TURN_116_PART_EQUIVALENCE_COMPLETION.

11. Minimal Count extension in Fold

Write B for the base Count stage and S(t) for its extension. Define u⩽t when u is a generated prefix of t. A minimal nonidentity stage t satisfies

$$t \neq B \text{ and } \forall u \preceq t, (u \neq B \Longrightarrow u = t)$$

Every t≠B is an extension and contains S(B) as a prefix. Substitution u=S(B) into minimality forces t=S(B). Conversely the only prefixes of S(B) are B and S(B), so S(B) satisfies the condition. Therefore

$$\text{minimal_nonidentity}(t) \iff t = S(B)$$

Its generated positions are precisely lift(o) and ν, with lift(o)≠ν by freshness. The complete retained trace is [lift(o),ν]. A later stage S(S(t)) has a proper nonidentity prefix S(B), so cannot be minimal; B fails nonidentity itself.

Fold classifies a generated Count stage as identity, first extension or later extension. Structural proofs identify those classes exactly with the base, its immediate successor and a successor of a successor. The generated-stage prefix relation retains each earlier stage under further extension. Every nonidentity finite trace contains the first extension. The first extension is nonidentity and every nonidentity prefix of it is itself; it is therefore minimal. Conversely, any minimal nonidentity stage must equal that first extension. Identity and every later finite extension are consequently excluded as minimal Fold stages.

The first extension retains the base occurrence and the appended occurrence. Its generated-position domain is exhausted by these two distinct positions, and its complete record retains each once. This formalises the source first-extension count argument without introducing signed numerical labels, a fitted quantity or a maximum finite depth. The source equality, disjointness, whole-preservation, distinct-label and return relations are retained in the complete grammar and characterized by the stage acceptance theorem below.

The seven source relation domains generate 288 forms in the original order. The native decision retains the exact source conjunction and ordered explanations. Exhaustive membership, unique survival and acceptance iff all seven requirements are established. Nine adverse criterion values are rejected. A source-record adapter derives the extension field from the actual Count stage and preserves the other six fields.

Its acceptance is equivalent to minimal nonidentity of that stage together with complete whole coverage, disjoint and equal fibres, distinct-held labels, present return and absence of extra data. Identity and every later stage are rejected for every choice of the remaining source fields.

The complete Fold program preserves all 288 original candidates and the One, Count and Part dependencies. Fresh native execution matches every census field, actual identifier-based decision, trace/proof digest, closure/generality certificate and original control result. Unknown identifiers are refused. These results accompany the general minimal-stage derivation and source-relation acceptance equivalence, completing the registered Fold entry.

Source claim: SFT-FOUNDATION-FOLD-001. Dependencies: One, Count and Part.

> Source, proof and provenance: V3 closure_record, sft/foundation/fold.py:131. Native results: SFT.Foundation.FoldDerivation.minimalNonidentityUnique, FoldDerivation.lean:88; SFT.Foundation.FoldStage.stageAcceptanceIff, FoldStage.lean:59; SFT.Foundation.Fold.returnRelationAbsentRejected, Fold.lean:224; SFT.Foundation.Fold.extraHasExtraRejected, Fold.lean:233. Timeline: Turn 018, line 263; Timeline: Turn 018, line 273; Timeline, line 9. Evidence: Completion TURN_114_FOLD_COMPLETION.

12. Complete finite Fold-word assembly and return

Fold-word assembly and complete records.

> Source: V3 sft/foundation/fold_assembly.py:23. Proof: SFT.Foundation.FoldAssembly.returnPredecessorUnique; SFT.Foundation.FoldAssembly.allFiniteSupportsUnique; SFT.Foundation.FoldAssembly.supportIffLengthAndHeldLabels; SFT.Foundation.FoldAssemblyTrace.canonicalAssemblyFacts; SFT.Foundation.AssemblySupport.supportAcceptanceIff; SFT.Foundation.AssemblySets.sourceUniquenessIff; SFT.Foundation.AssemblyRecords.recordAcceptanceIff. Provenance: Timeline, line 287; Timeline, line 303; Timeline, line 319; Timeline, line 320; Timeline, line 333.

Let a and b denote the two distinct held labels and ε the empty structural word. Define the ordered supports by

$$W_0 = [\varepsilon]$$
$$W_{n+1} = \mathrm{concat}_{w \in W_n}[w \cdot a, w \cdot b]$$

A successor w·x retains its prefix w and final label x; removing that final label gives ρ(w·x)=w. If w·x=v·y then terminal-label equality gives x=y and prefix equality gives w=v. Together with a≠b this proves the two branches are disjoint and preserves uniqueness inductively.

Induction on n proves

$$w \in W_n \iff |w| = n \ \wedge \ (\forall x \in w,\ x \in \{a, b\})$$

At n=0 the only such word is ε. At n+1, separating the final label yields a length-n prefix satisfying the induction hypothesis and one of the two admitted labels, hence exactly one generated branch. Conversely each generated branch has the asserted length and label membership. The host-record cardinalities obey $|W_0|=1$ and $|W_{(n+1)}|=2|W_n|$, hence $|W_n|=2^n$. This count follows from complete recursive generation, not a premise replacing it.

The assembly construction begins with the structural empty One word. Each successor appends held-a and held-b, in that order, to every preceding word. The nonnegative extension index is the source host recursion input; the empty word is a structural form. Negative host extension inputs are rejected.

Appending a held label retains both the predecessor word and the last label. The return relation recovers the predecessor by removing precisely the appended terminal label. Both distinct branches return to the same predecessor, and a successor has only one such predecessor. These properties hold at every finite generated depth. Appending different labels or extending different predecessor words produces distinct successor words; induction establishes uniqueness of every complete support.

A word belongs to the support after n extensions if and only if it has host-record length n and every label belongs to the source pair. This universal equivalence establishes completeness and absence of additional words. Every retained successor has a return to the preceding support. An outside label or incorrect length excludes a proposed word; repeating a generated word violates uniqueness, and omitting one violates complete coverage. These statements are established by constructive finite-list and generation induction without an imported exponentiation law or a maximum depth.

Source claim: SFT-FOUNDATION-FOLD-ASSEMBLY-001. Dependencies: Count and Fold. The source keeps this recursive assembly claim separate from minimal Fold. Its seven relation domains produce 192 source-ordered forms. Exhaustive enumeration, equivalence of the original identifier decision with the seven criteria, unique survival and rejection of eight adverse values are established. The complete original program and its evidence records match the pinned V3 source outputs.

The canonical assembly retains every stage from the empty source word through the final finite depth. Each transition explicitly retains predecessor, appended label and successor. Its successor uniquely determines that transition. Every generated child at every earlier level has both its transition and return recorded in the full trace. Conversely, each stored stage, transition and return arises from the source construction; each return recovers its recorded predecessor.

A native proof certificate establishes the seven accepted relation requirements on this canonical construction: complete stage coverage, consistent lengths, complete support, word uniqueness, transition retention, return retention and absence of added records. The accepted source witness is realised at every positive finite depth. This result concerns the canonical construction. Proposed word supports additionally admit the executable semantic classification below; source coverage, transition and return relations remain independent inputs.

For an arbitrary finite proposed word support, executable membership and universal finite-list tests determine the source length, support-completeness, uniqueness and extra-word relations. A native theorem establishes acceptance exactly when the coverage relation is complete, every proposed word has the required length, every generated word is retained, no word is repeated, transitions and returns are present, every proposed word is generated and no extra rule is supplied. The coverage, transition and return fields are preserved as independent source relations; their truth is not inferred from a word list alone.

The source finite-set operations are represented constructively. Duplicate removal preserves precisely the original members and produces a duplicate-free list. Equality of the original length and this finite-set cardinality is equivalent to uniqueness. Two word sets compare equal exactly when their membership predicates agree. The original four assembly controls retain their predicates and serialized result hashes; the depth-one-through-five uniqueness/length check follows from the native all-finite theorem. The source literal boundary text check remains distinct from semantic exclusion results.

The finite assembly record representation includes predecessor–label–successor transitions and child–parent returns. Acceptance is equivalent to retention of every required transition and return, exclusion of ungenerated records, complete source coverage, correct word lengths, complete word support, word uniqueness and absence of extra words or rules. Canonical records satisfy these conditions. Missing required transitions, missing required returns and incorrect returns are rejected. Coverage remains the explicit source relation. Registration, census, decisions and closure are represented with the original source fields and ordering; candidate lookup uses the source identifier.

Positive casting and exact part admission.

> Source: V3 sft/foundation/exact_operations.py:47; V3 sft/engine/exact.py:17. Proof: SFT.Foundation.ExactCast.castHasRemovalTrace; SFT.Foundation.ExactCast.removalUnique; SFT.Foundation.ExactCast.castAddingOne; SFT.Foundation.ExactCast.completeTurnsRetainOne; SFT.Foundation.ExactPartCarrier.admissionIff; SFT.Foundation.ExactPartCarrier.guardedTakeAccepted; SFT.Foundation.ExactPartCarrier.takeStrictlyBelowOne. Provenance: Timeline, line 346; Timeline, line 361.

For positive integers n,d, define the retained numerator recursively by c_d(n)=n when n≤d, and c_d(n)=c_d(n−d) when n>d. In the latter branch 0<n−d<n, so the recursion terminates. Induction along the removals establishes 0<c_d(n)≤d and n=c_d(n)+kd for a nonnegative host removal count k. If r,r′∈{1,...,d} represent the same residue modulo d, then their difference has magnitude below d and is divisible by d, hence r=r′. This proves uniqueness of the retained representative. It follows that c_d(n+d)=c_d(n) and c_d(kd)=d for every positive whole-turn count k. The terminal representative of a complete turn is d/d, the One.

The exact casting operation acts on a positive count ratio n/d by removing one complete denominator while the numerator exceeds it. A structural recursion bound derived from n suffices for every positive d. A native removal-trace theorem proves termination and a unique retained numerator r with 0 < r ≤ d. Admitted numerators remain fixed; adding a complete denominator to a positive numerator does not change its retained value. Every positive whole number of complete turns retains d, representing the One. These results apply to all finite positive count pairs. The numerator-level result supplies the casting stage of the normalized primitive operations below.

The Fraction admission map preserves the reduced numerator and denominator exactly. An admitted part maps back to the same Fraction, and admission yields that part if and only if the original Fraction is its reduced-pair representation. Zero, signed values and values above the One are rejected; every reduced positive pair through the One is admitted. For admitted a and b, the strict Take guard is the cross-product inequality between their count pairs. Its difference numerator is positive and strictly smaller than the product denominator; equal and reversed Take inputs are rejected. Self-junction followed by cast has a positive numerator no larger than its denominator, and a complete One is Fold-fixed. The normalized operations reduce their output count pairs to the canonical Fraction representation.

Fraction normalization and primitive operations.

> Source: V3 sft/foundation/exact_operations.py:47; V3 sft/engine/exact.py:17. Proof: SFT.Foundation.FractionReduction.gcdDivides; SFT.Foundation.FractionReduction.commonDivisorDividesGcd; SFT.Foundation.FractionReduction.reducedCoprime; SFT.Foundation.ExactOperations.normalizedValuePreserved; SFT.Foundation.ExactOperations.normalizationStable; SFT.Foundation.ExactOperations.takeAdmitted; SFT.Foundation.ExactOperations.unisonIff. Provenance: Timeline, line 1636; Timeline, line 379.

For a positive pair (n,d), let g=gcd(n,d) and N(n,d)=(n/g,d/g). Divisibility by g gives positive integral quotients, and any common divisor of both quotients would enlarge the removed common factor; therefore gcd(n/g,d/g)=1. The identity (n/g)d=n(d/g) preserves the ratio exactly. For admitted parts a=n/d and b=m/e, the strict Take guard a>b is ne>md, and the retained difference is

$$\mathrm{Take}(a,b) = N(ne - md, de)$$

Because 0<ne−md<de for admitted inputs satisfying the guard, the result is a positive part below the One. Equality ne=md and reversal ne<md fail the guard. Self-junction and Fold retain their source order:

$$\mathrm{Junction}(a,a) = N(2n,d)$$
$$\mathrm{Fold}(a) = \mathrm{normalize}(\mathrm{cast}(\mathrm{Junction}(a,a)))$$

Unison is equivalent to ne=md, the exact input cross-product equality.

Canonical Fraction construction divides the numerator and denominator by their greatest common divisor. Constructive proofs identify the native GCD computation, establish its divisibility and greatest-common-divisor properties, and show that the reduced quotient pair is coprime. Both quotients are positive, reduction preserves exact cross-product value, and an admitted input remains through the One. Normalization is stable on an already reduced part. The integer-pair constructor retains the source rejection messages for signed, zero and beyond-One inputs.

Fold first reduces the self-junction Fraction, then casts complete Ones and reduces the retained part. Guarded Take constructs and reduces its exact positive difference. Both operations return reduced admitted parts. Unison normalizes the quotient and compares it to the One; a native theorem makes this equivalent to equality of the input cross-products. Every part is in unison with itself, and the complete One is Fold-fixed. The proof arguments apply to all finite positive count pairs and use no denominator bound.

The Exact Operations grammar contains the complete 128-form product of domain, full-turn representation, Fold composition, Take guard, unison identity, trace custody and added-data classes. Enumeration is exhaustive; the source identifier decision is equivalent to all seven required criteria, and exactly the registered survivor satisfies them. Each adverse class is rejected. The trace-custody relation remains an explicit source criterion.

The positive Fraction cast adapter rejects zero and signed inputs and returns the normalized cast result for every positive reduced input. The four original controls retain their predicates: reversed Take rejection, changed-source identity, casting two complete Ones to the One, and the four specified hostile carrier constructions. Their full output fields and hashes agree with the pinned source. The eight literal closure fields are preserved.

Half-One selection and self-junction.

> Source: V3 sft/foundation/half_one.py:31. Proof: SFT.Foundation.HalfOneDerivation.selfJunctionOneIff; SFT.Foundation.HalfOneDerivation.properSelectionIsHalf; SFT.Foundation.HalfOneDerivation.exchangePreservesCoordinate. Provenance: Timeline, line 409.

The first Fold has the two positions already established by minimal Count extension. Every held selection is empty, one of the two singletons, or the whole pair. Its nonempty coordinate classes are precisely one-of-two and One. Every proper nonempty selection therefore normalizes to half-One. Exchanging the two positions is involutive and preserves the selection cardinality and exact coordinate.

Let x=n/d be reduced and positive. Junction(x,x)=One implies 2n=d. Then gcd(n,d)=gcd(n,2n)=n, so reducedness forces n=1 and consequently d=2. Conversely x=1/2 has self-junction 1. Thus

$$\mathrm{Junction}(x,x) = \mathrm{One} \iff x = 1/2$$

Complement gives 1−1/2=1/2, while Fold(1/2)=cast(1)=1. The complete One is Fold-fixed because Fold(1)=cast(2)=1.

For every admitted reduced positive part, normalized self-junction is the One if and only if the part is half-One. The proof uses the full positive numerator/denominator domain: self-junction forces the denominator to be twice the numerator, while reducedness forces the numerator to be one. Half-One complements to itself and Folds to the One; the One is Fold-fixed. At half-One, complement and half-One phase translation have different values. These results formalize the registered complete-partition argument without a sampled rational search.

Source claim: SFT-FOUNDATION-HALF-ONE-001. Dependencies: minimal Fold, Exact Operations and Part Equivalence.

The Half-One candidate grammar is the complete 128-form product of selection class, self-junction, complement, Fold image, One fixedness, antipode terminology and added-data class. Native proofs establish exhaustive membership, equivalence of the source decision to its seven criteria, unique survival and rejection of every adverse class. The source whole-selection control witness is retained in the generated census. The executable self-junction-to-One predicate holds exactly at half-One by the general reduced-part uniqueness theorem.

All four Half-One controls retain the source predicates and serialized fields. The altered-coordinate control compares half-One and one-of-three; the boundary evaluates the actual complement and Fold operations. Current Lean output agrees with the four complete source control records, including hashes, and all nine literal closure fields.

Exact Fold fibres, phase and uniform divisions.

> Source: V3 sft/foundation/fold_dynamics.py:31. Proof: SFT.Foundation.ExactValueIdentity.reducedValueIff; SFT.Foundation.FoldDynamics.lowerFoldsToImage; SFT.Foundation.FoldDynamics.firstTwoCycle; SFT.Foundation.FoldCastLaws.foldCountFormula; SFT.Foundation.FoldPhaseLaws.generatedPreimagesDistinct; SFT.Foundation.FoldPhaseFull.fullPhaseCollision; SFT.Foundation.FoldPhaseFull.fullPhaseInvolution; SFT.Foundation.FoldFibres.exactTwoPreimages; SFT.Foundation.FoldUniform.exactEvenImageSet; SFT.Foundation.FoldUniformOrder.adjacentGap; SFT.Foundation.FoldSorted.foldedUniformDivision; SFT.Foundation.FoldUniformCheck.everyEvenDivisionRemainsUniform. Provenance: Timeline, line 440; Timeline, line 441; Timeline, line 455; Timeline, line 456; Timeline, line 470; Timeline, line 485; Timeline, line 500; Timeline, line 515; Timeline, line 530; Timeline, line 545.

Reduced positive Fractions are equal if and only if their cross-products are equal. Constructive remainder scaling and GCD scaling establish denominator identity, followed by numerator identity. This result connects exact count-pair equations to equality of the native normalized parts. Normalization represents any equal admitted ratio, and casting an admitted reduced part preserves it.

For an arbitrary admitted image y, the lower generated Fold preimage is the normalized ratio y/2. Its doubled numerator does not exceed its denominator, and native Fold of this preimage is exactly y. Phase translation is implemented as normalized addition of half-One followed by cast; the generated preimage pair retains the source lower and translated-upper definitions. The executable uniformity predicate is connected to the arbitrary-count division proofs below.

The exact orbit one-of-three to two-of-three and back to one-of-three is proved using the native Fold operation. Its two values are distinct. Two steps return to the source, while every positive step count below two fails to return. Source claim: SFT-FOUNDATION-FOLD-DYNAMICS-001; dependencies: Exact Operations and Half-One.

Positive scaling of a count-pair representation scales its complete-One removal trace and retained numerator. Normalization commutes with casting, yielding an exact raw count-pair formula for Fold of every normalized part. These identities preserve the original operation while allowing its arbitrary-rational branch equations to be proved.

Write C for casting to the positive rational circle (0,1] and F(x)=C(2x). Define phase T(x)=C(x+1/2). For $0<x\le 1/2$, T(x)=x+1/2; for $1/2<x\le 1$, T(x)=x−1/2. In both branches a second application returns x, so T(T(x))=x. Doubling either branch changes 2x by exactly one complete turn, hence F(T(x))=F(x).
For any $0<y\le 1$, the two generated inverse images are

$$L(y) = y/2, U(y) = (y+1)/2$$

They satisfy $0<L(y)\le 1/2<U(y)\le 1$ and

$$F(L(y)) = C(y) = y, F(U(y)) = C(y+1) = y$$

Conversely if F(x)=y, the first-half branch gives 2x=y and x=L(y); the second-half branch gives 2x−1=y and x=U(y). Therefore

$$F(x) = y \iff x = L(y) \text{ or } x = U(y)$$

with $L(y)\neq U(y)$. At y=1 the pair is (1/2,1), retaining the complete-turn representative at the endpoint.

For a part in the first half-circle, half-One phase translation has the exact normalized coordinate (twice the numerator plus the denominator)/(twice the denominator). Doubling this translated coordinate adds one complete One before cast, so it shares the original Fold image. Applying this result to the generated lower preimage proves that the generated upper preimage also Folds to every admitted image. The upper lies strictly beyond the half-circle boundary, while the lower lies on or below it; hence the two generated preimages are distinct. This establishes validity and distinctness of the generated pair. Every preimage is one of these two forms: a first-half part is recovered as the lower preimage of its Fold image, while a second-half part translates to the first half and is recovered using phase involution. Thus, for every admitted p and y, Fold(p) = y if and only if p is y/2 or cast(y/2 plus half-One). Distinctness and this exhaustive equivalence establish the exact two-preimage law over the entire positive rational circle.

The phase laws hold throughout the positive rational circle, including both half-circle branches and One. Complete-One removal is preserved under the affine half-One shift and under doubling. Consequently Fold(phase(x)) = Fold(x) and phase(phase(x)) = x for every admitted part x. The proofs retain the source normalization and cast order and remove the complete One introduced by two successive half-One translations.

For positive n define D_n=[1/n,2/n,...,n/n], with each coordinate normalized. For 1≤i≤n,

$$F(i/(2n)) = i/n$$
$$F((i+n)/(2n)) = C(i/n+1) = i/n$$

Every index 1≤j≤2n lies either at j=i or at j=i+n for exactly one such i, so the complete image has precisely the members of D_n. Cross multiplication gives i/n<j/n exactly when i<j. Thus D_n is strictly ordered, has n distinct entries and ends at n/n=One. Adjacent differences are

$$\frac{i+1}{n} - \frac{i}{n} = \frac{1}{n}$$

Sorting and removing duplicate image values therefore yields exactly D_n; at n=1 this is [One].

For every positive n, the source uniform division is represented by the ordered normalized parts i/n with indices from one through n. The native enumeration retains every generated index. For every i from one through n, Fold maps both i/(2n) and (i+n)/(2n) to i/n. Conversely, every index in the even division belongs to one of these branches. Therefore a part occurs among the Fold images of the complete 2n-division if and only if it occurs in the n-division. Distinct indices give distinct normalized coordinates, and the terminal n/n coordinate is One. These arbitrary-count results establish the exact image membership before sorting and gap evaluation.

Exact normalized rational ordering agrees with the original cross-product comparison. Consequently the complete n-division is strictly increasing, has no repeated parts and contains exactly n entries. For each adjacent generated index i,i+1, native Take of i/n from (i+1)/n is exactly the normalized part 1/n. This is established by factoring the reduced numerators and denominators through their GCDs and preserving the exact rational difference. At n=1 the generated list is exactly [One]. Together with terminal One and the even-division image correspondence, these results establish the order, cardinality and gap properties of the uniform image domain.

The sorted distinct Fold image is implemented over the exact reduced-rational carrier. Cross-product identity removes repeated values, and cross-product order determines the ascending output. Native proofs establish input membership preservation and strict output ordering for every finite input list. Two strictly ordered rational lists with the same members are equal. Applying this uniqueness result to the complete even division proves the executable equality folded_distinct(uniform_division(2n)) = uniform_division(n) for every positive n. The resulting list has exactly n entries; no generated input index is omitted before Fold and duplicate-image identification.

The native uniformity predicate preserves signed adjacent differences: positive and negative differences carry their exact normalized magnitudes, and equal neighboring parts give zero. Equality of these gap values is decidable and is proved equivalent to equality of their representations. The singleton-gap-set check holds precisely when a gap occurs and every gap in the list equals it. Empty lists fail; singleton lists pass exactly at One. Longer lists require one distinct signed gap and terminal One, matching the source predicate.

Complete generated-prefix witnesses establish equal positive gaps and the final coordinate for every prefix. Consequently the executable predicate accepts uniform_division(n) for every positive n and accepts folded_distinct(uniform_division(2n)) for every positive n. Separate fresh execution comparisons agree with exact Fraction evaluation on 156 lists covering lengths zero through three and positive, negative, zero and mixed gaps.

The Fold Dynamics candidate grammar retains the complete 128-form product of domain closure, fibre cardinality, phase collision, phase involution, uniform-division preservation, first cycle and extra-data class. The source identifier decision is equivalent to all seven stated criteria, and exactly the registered survivor satisfies them. Native proofs establish complete enumeration and rejection of every adverse axis value.

The Fold Dynamics controls retain the original 35 rational images generated for denominators two through eight and all six even divisions two through twelve. The two-preimage and phase control predicates hold for every admitted image by the arbitrary-domain laws; their finite executions preserve the source enumeration. The uniformity and first-cycle boundary check follows from the arbitrary even-division theorem and exact one-third/two-thirds return. All four serialized control records, including receipt hashes, and all eight closure fields agree with the pinned V3 artifacts and source literals.

The Fold Dynamics program adapter preserves the original registration fields, complete census, candidate lookup, decision proof hashes and depth-independent closure evidence. Fresh execution agrees with the pinned artifacts for all 128 lookup decisions and every census, closure and control field and hash. Candidate lookup depends only on the original identifier, and an unregistered identifier is rejected.

Finite Form productions and raw validation.

> Source: V3 sft/foundation/form_grammar.py:32. Proof: SFT.Foundation.FormConstruction.leafCountIsFoldCountPlusOne; SFT.Foundation.FormConstruction.encodingInjective; SFT.Foundation.FormValidation.generatedFormsValid; SFT.Foundation.FormValidation.edgeTailIgnored; SFT.Foundation.FormAcceptance.validityIffFiniteProduction. Provenance: Timeline, line 577; Timeline, line 590; Timeline, line 605.

The next foundational construction is the finite Form grammar. Its two productions are the structural One leaf and a Fold node with arbitrary recursively generated left and right subforms, distinct held-a/held-b edges and the explicit return marker. The native tuple encoding preserves these source constructors and is injective. Structural induction applies to every generated finite form; each form has a positive number of One leaves, with leaf count equal to Fold-node count plus one. The five original examples include distinct left-deep and right-deep nonuniform forms. Write $L(t)$ and $F(t)$ for the leaf and Fold-node counts. The productions give

$$L(\mathrm{One}) = 1, \quad F(\mathrm{One}) = 0, \qquad L(\mathrm{Fold}(u,v)) = L(u) + L(v), \quad F(\mathrm{Fold}(u,v)) = F(u) + F(v) + 1.$$

The base therefore satisfies $L = F + 1$. Substituting the two induction hypotheses at a Fold node gives

$$L(u) + L(v) = (F(u) + 1) + (F(v) + 1) = F(\mathrm{Fold}(u,v)) + 1.$$

For encoding injectivity, unequal root tags cannot have equal tuple encodings. Equal Fold encodings have equal first and second child encodings separately; applying the induction hypothesis to each child

recovers the original ordered pair of subforms. The raw source validator is translated on the finite tuple/string representation described below.

On finite nested tuple/list/string values, native form validation preserves the source predicate and its evaluation order. The One tuple is accepted; a Fold tuple requires the original tag, labels and return marker before recursive child validation. Every encoded generated Form is accepted by structural induction, including the original nonuniform examples. One-child and same-label nodes are rejected. The source accesses only the first two items of each edge tuple, so trailing edge items remain ignored. Missing indexed items retain their IndexError outcomes, while earlier false conditions suppress later accesses. Fresh execution agrees with the explicit source predicate on 235 tuple/string cases. For every represented finite tuple/list/string value, successful validation is equivalent to a finite production trace. The forward direction follows structural recursion on the value; the reverse direction follows induction on the trace. Each production retains the original Boolean tag, edge-label and return tests, including arbitrary ignored trailing edge items. Thus this characterization preserves the source acceptance predicate rather than imposing canonical two-item edges. The accepted root is a tuple; each held edge independently uses a tuple or list, with arbitrary ignored trailing edge values. The One production and these four Fold productions exhaust successful validation, as expanded in Section 71.

The registered Form Grammar classifies the complete product of two base choices, three child arities, three edge-label relations, two child-provenance choices, two return choices, two termination choices and two extra-constructor choices. All 288 proposals are enumerated in source order. Native proofs establish exhaustive membership and exactly one accepted proposal: the One base with two distinctly held-labelled generated children, return, finite-leaf termination and no additional constructor. Each of the nine adverse field values is rejected. Fresh execution preserves every original candidate identifier, exact form, acceptance and ordered reason. All four original control records and their receipt hashes agree with the pinned artifacts; the seven closure fields agree with the source. The original boundary control retains its literal text predicate; finite production-trace results are separately established by the structural proofs above.

Canonical form identity and raw equivalence.

> Source: V3 sft/foundation/form_enforcement.py:29. Proof: SFT.Foundation.FormIdentity.traceInjective; SFT.Foundation.FormIdentity.equivalenceIffIdentity; SFT.Foundation.FormCanonicalRaw.acceptedHasCanonicalForm; SFT.Foundation.FormEquivalenceRaw.canonicalOutputsDetermineEquivalence. Provenance: Timeline, line 637; Timeline, line 638; Timeline, line 652.

Canonical form enforcement assigns the One leaf its terminal One tuple and each Fold node the ordered tuple Fold, held-a, left canonical trace, held-b, right canonical trace, return. Structural induction proves this map injective on all generated finite Forms. Canonical-trace equivalence is therefore precisely form identity and is reflexive, symmetric and transitive. At the One constructor, equality is immediate. At a Fold constructor, equality of the ordered traces identifies the left trace with the left trace and the right with the right. The two induction hypotheses recover the corresponding children; the distinct One/Fold tags exclude mixed constructors. Equality itself then supplies reflexivity, symmetry and transitivity. Equality of Fold traces preserves the two corresponding child traces separately. The source partial trace and asymmetric child swap are rejected. An executable comparison on generated Forms is proved equivalent to canonical-trace equality.

On the finite tuple/list/string domain, the raw canonicalizer validates before constructing the trace, propagates validation errors and preserves ignored edge tails. Every generated input produces its stated canonical trace. More generally, every accepted raw value produces the canonical trace of a generated Form. Fresh execution matches 235 explicit source-predicate trace and error cases. The complete Form Enforcement product contains 192 identity proposals; exhaustive membership, unique acceptance and each adverse-condition rejection are established. All candidate fields and decisions and the four original

controls with their receipt hashes match the pinned artifacts. The accepted-production induction covers arbitrary finite child depth and all four edge-container combinations; the complete program correspondence is given below.

Raw form equivalence canonicalizes the left input before the right and propagates the first encountered error. A constructive recursive equality decision on finite tuple/string values is proved equivalent to their exact equality. Consequently, two generated inputs compare successfully as equivalent exactly when the generated Forms are identical. For arbitrary represented inputs with successful canonicalization, equivalence is precisely equality of the resulting traces. Fresh execution agrees with the source formulas on 2,352 raw input pairs, including malformed forms, ignored trailing edge items and error precedence.

Measured-value boundary.

> Source: V3 sft/foundation/measurement_boundary.py:30. Proof: SFT.Foundation.MeasurementProtocol.acceptanceIff; SFT.Foundation.MeasurementProtocol.sourceExpression; SFT.Foundation.MeasurementProtocol.booleanSpecialization. Provenance: Timeline, line 654.

The measured-value boundary preserves the source Boolean protocol: acceptance is equivalent to opening the target after sealing, leaving the law unchanged, retaining all rows and identifying the source. Each violated condition forces rejection regardless of the other flags. With Boolean flags s, m, r, i for post-seal opening, law mutation, complete row retention and source identification, the exact predicate is

$$A(s, m, r, i) = s \wedge \neg m \wedge r \wedge i.$$

Expanding the conjunction shows that acceptance forces each of the four requirements; supplying all four reduces every conjunct to true. The source's non-Boolean operation follows ordered short-circuit evaluation, returning the selected operand; its Boolean specialization is precisely this predicate. All 16 Boolean inputs agree with the source conjunction. The complete seven-axis interface product has 256 proposals, with one surviving outward interface and nine independently rejected adverse field values. Every proposal's order, identifier, exact form, decision and reason agrees with the pinned records. All four original control records and their hashes and all eight closure fields also agree. These results establish the Boolean protocol and finite interface classification. The source operand-preserving protocol, including its non-Boolean return values, is retained by the native protocol implementation.

Exact trace arithmetic.

> Source: V3 sft/mathematics/exact_arithmetic/law.py:13. Proof: SFT.Mathematics.ExactTraceArithmetic.pairCellsLeftDistribute; SFT.Mathematics.ExactTraceArithmetic.pairCellsAssociative; SFT.Mathematics.ExactTraceArithmetic.pairCellCardinality; SFT.Mathematics.ExactTracePairing.exactPairingComplete; SFT.Mathematics.ExactTracePairing.pairingUnique; SFT.Mathematics.ExactTraceDistribution.pairCellsRightDistribute; SFT.Mathematics.ExactTraceRemainder.sameFormIffNoUnmatched. Provenance: Timeline, line 669; Timeline, line 684; Timeline, line 699; Timeline, line 717.

Exact Arithmetic represents an atom as a finite string-label path and a trace as a finite sequence of atoms. The trace constructor rejects empty or repeated label inputs. Disjoint junction concatenates nonempty traces only when they have no shared atom. Pair-cell product retains every left/right pair in the source order and flattens the two atom paths by concatenation. Native induction establishes associativity of this flattening at arbitrary finite trace lengths and distribution over concatenation in the left input. Its complete output cardinality is the product of the two input lengths, and positive input traces produce a nonempty result.

Exact pairing returns the source zip precisely for equal input lengths, including its specified empty-input result. Oriented remainder retains unmatched atoms in source order and reports incomparable-held, left-held, right-held, or same-form with the original empty-One representation. No additional distinctness condition is imposed on raw operation inputs. Fresh execution agrees with all five source operations on 81

pairs covering empty, disjoint, overlapping, repeated and nested-path inputs, together with eight trace-constructor cases. The registered arithmetic laws and complete generated-law evidence are supplied by the pairing, distribution, remainder and program results below.

Exact pairing preserves both complete input sequences: projecting every pair onto its first coordinate recovers the left trace in order, and projecting onto its second recovers the right trace in order. Both output cardinalities equal the input lengths. A returned pairing exists exactly when those lengths agree, and the ordered pairing is unique. These results apply to arbitrary finite source traces, including duplicate atoms and the specified empty pairing, without replacing occurrences by sets.

The Exact Arithmetic LawSpec contributes eight binary representation dimensions. Their complete 256-candidate product is enumerated in source order. Native proofs establish membership for every candidate, equivalence between identifier acceptance and all eight admitted choices, unique acceptance and rejection of each adverse coordinate. Fresh execution matches all candidate identifiers, coordinate-form strings, decisions and ordered rejection reasons from the pinned generated-law artifacts. This finite classification accompanies the operation proofs and the complete registered-law evidence described below.

Right-distribution of pair-cell refinement preserves the complete generated output as a permutation. The proof constructs finite adjacent swaps and block interchanges, retaining every occurrence and its multiplicity even when flattened atom paths coincide. This relation accounts for the source row-major traversal order. Together with ordered left-distribution and flattened associativity, it establishes both distributive directions by cell provenance. The five original arithmetic witnesses—junction associativity, six-cell cardinality, flattened product associativity, exact pairing and held remainder—are evaluated by native operations and proved passing, with their exact source names, statements and results retained.

The complete native Form Grammar program output agrees with the pinned artifacts: registration fields, all 288 actual identifier lookups and their decision proof hashes, every census trace hash, closure, controls and unknown-identifier rejection. The program correspondence accompanies the complete finite-production equivalence and canonical-form construction. These proofs and the retained dependency registration complete the Form Grammar entry.

The oriented-remainder output is nonempty for every represented finite trace pair. Its same-form tag holds precisely when both source unmatched filters are empty; the corresponding output is the original empty-One trace. This characterization retains the source predicate on raw inputs, including duplicates, without replacing it by ordered trace equality.

The arithmetic closure retains the registered exact result, laws, operational witnesses, minimality, named uniqueness, induction base and step, and exclusions. The original tampered-artifact control performs both complete enumerations: survivor filtering and distinct identifier counting. Fresh execution yields one survivor and 256 distinct identifiers; all four control records and receipt hashes and all eight closure fields match the pinned source artifacts.

The native Form Enforcement, Measurement Boundary and Exact Arithmetic programs preserve the registered program-method outputs in full. Fresh execution agrees with all 704 candidate lookups across these three programs, every census trace and decision proof hash, all registration fields and provenance classes, closure and generality evidence, and control records. Unknown identifiers retain the source failure: KeyError for the two foundation programs and StopIteration for the generated mathematics program. Arithmetic generality serialization retains the original witness dataclass fields. These complete program correspondences accompany the general Form, protocol and exact-trace operation proofs. Their source-bound completion records retain the dependencies and applicable verification evidence for each entry.

13. Formal representation and results

Each Lean candidate constructor denotes exactly one registered V3 candidate record. These constructors are encodings of the existing finite grammar. Field projections retain source text and field values. Decision functions implement the original branch predicates and rejection order.

For each translated grammar, the formalisation proves:

(a) Every candidate belongs to the enumerated domain.
(b) Acceptance is equivalent to equality with the source survivor.
(c) The source survivor is accepted.

The candidate proofs use constructor elimination, reflexivity and explicit list membership to exhaust the source coordinate products. Operational coverage is established from the constructors themselves: Count uses base/successor induction on generation traces; Fold-word assembly proves membership exactly by length and held-label membership; the primitive theorem compares an arbitrary nonempty word with the one-letter Fold witness. Consequently the corresponding completeness statements range over every finite trace, support depth or primitive word permitted by those definitions. proposalAcceptedIffExactRetention, CountRepresentation.lean:235 and leastGeneratorIff, PrimitiveMinimality.lean:84 give explicit examples of these universal statements. At the current scientific checkpoint, the audit covers 17,849 theorems and 38,713 explicit definitions and theorems, all with empty axiom-dependency sets. Finite classification, source closure, control records and admission procedures have separate proof and correspondence obligations; the completion receipts bind their applicable results to each completed entry.

On the annotated registration domain, passing also entails empty axiom and free-parameter lists and no missing dependencies under the source authority lookup. The typed Root registration serializes identically to its source record. Registration diagnostic order is checked on 272 cases.

The admission engine’s forcing gate preserves four ordered requirements: decision identifiers match the generated census in order; exactly one decision survives; every decision provides an explicit reason; every proof identity has canonical SHA-256 syntax. On the source-typed Candidate and CandidateDecision domains, a constructive theorem establishes that an empty violation list implies a survivor list of length one. The original Root decisions have one survivor, while marking both decisions as surviving produces two and is proved to be rejected by this gate. This connects the source tampering premise to the engine’s actual rejection condition.

Closure evidence retains its source scope, exact boundary, minimality and uniqueness flags, proof identity and optional generality identity. Within the source-typed field domains, passing the closure gate requires a nonblank boundary and successful minimality and uniqueness flags; depth-independent closure also requires a canonical SHA-256 generality identity. The control gate retains the required-kind, duplicate-kind, outcome, behavior-text and receipt-identity checks in source order. Passing implies that every supplied control has passed. Root closure with minimality removed or with its generality identity absent, and Root controls with failed status flags, are constructively proved to be rejected. The engine-gate theorems prove rejection from the failed fields themselves. The operational Root theorem is supplied by the separate record derivation: classification preserves presentation, exhausts the two presentation cases for every source-schema record, and equates acceptance with witnessed occurrence. recordClassificationExhaustive, RootDerivation.lean:41 and acceptanceIffWitnessedOccurrence, RootDerivation.lean:73 state that derivation directly.

The candidate census retains the source generation rule, grammar boundary, signed expected cardinality, completeness identity and ordered candidate records. The census gate preserves the eight source checks and their diagnostic order. Passing requires nonblank generation and boundary text, a positive expected cardinality, matching candidate count and no duplicated identifiers. A constructive witness expresses each passing expected cardinality as a natural successor. Root censuses with an incorrect registered count or

duplicated candidate are proved rejected. Executable correspondence covers 1,440 census variants, including negative and zero counts, Unicode identifiers, duplicated identifiers, blank forms and malformed identities, together with three direct Root cases.

> Source, proof and provenance: V3 candidate_records, sft/foundation/root_theorem.py:50; V3 receipt_dict, sft/engine/engine.py:497; V3 canonical_value, sft/engine/canonical.py:13; V3 registration, sft/engine/model.py:158. Native results: SFT.Foundation.Root.Gate.registrationWithAxiomRejected, RootGate.lean:75; SFT.Foundation.Root.Gate.registrationWithParameterRejected, RootGate.lean:82; SFT.Engine.passingRegistrationHasNoFreeParameters, RegistrationGate.lean:108; SFT.Engine.passingRegistrationNoMissingDependencies, RegistrationGate.lean:113; SFT.Engine.positiveCardinalityHasSuccessor, CensusGate.lean:63; SFT.Engine.passingCensusPositiveWitness, CensusGate.lean:72; SFT.Engine.emptyControlDetailsAllPassed, EvidenceGates.lean:131; SFT.Engine.passingControlsAllPassed, EvidenceGates.lean:142; SFT.Engine.eqOneOfBeq, DecisionGate.lean:78; SFT.Engine.passingDecisionGateHasOneSurvivor, DecisionGate.lean:86. Timeline: Turn 008, line 111; Timeline: Turn 338, line 3318; Timeline: Turn 010, line 148; Timeline: Turn 009, line 132; Timeline: Turn 008, line 110. Evidence: Completion TURN_111_ROOT_COMPLETION.

Gate-specific source and conclusions: V3 _registration_violations; V3 _census_violations; V3 _decision_violations; V3 _closure_violations; V3 _control_violations; SFT.Engine.passingRegistrationHasNoAxioms; SFT.Engine.passingCensusMatchesCount; SFT.Engine.passingCensusNoDuplicateIdentifiers; SFT.Engine.passingClosureHasBoundary; SFT.Engine.passingClosureHasMinimality; SFT.Engine.passingClosureHasUniqueness; SFT.Engine.passingDepthIndependentHasCertificate.

14. Correspondence and reproducibility

The Lean implementation uses leanprover/lean4:v4.35.0. The build command is lake build. Audit.lean reports the dependencies of each theorem. Parity.lean emits the candidate records and their computed decisions. Each verification invocation rebuilds the current Lean source, generates a declaration audit and executes the current parity program. The retained aggregate artifact comparison checks 47,206 ordered rows against the pinned V3 census and elimination records; all 47,206 agree on candidate identifier, exact form, decision and reason.

The public formalisation repository is https://github.com/MettaMazza/SFT-v4-LEAN-VERIFIED.

Canonical JSON uses the source ASCII-escaping, compact-separator and sorted-key rules on the Root evidence domain. SHA-256 is implemented constructively with its fixed algorithm constants. All 38,713 audited explicit definitions and theorems have no axiom dependencies. Sixteen byte-message checks and ten JSON checks agree with Python standard-library results; these execution checks are separate from a general proof of algorithm correspondence, which remains outstanding. The fixed Root boundary is ASCII, so its lowercase and substring operations preserve the source predicate exactly. These host operations introduce no free scientific parameters.

The typed forcing-gate implementation is additionally checked on 384 combinations of identifier order and cardinality, survivor flags, reason content and hash syntax. The Unicode whitespace predicate agrees with the host language over all 1,114,112 code points.

Closure/control correspondence is checked on 432 closure variants, 576 control variants and five direct Root cases. The typed control adapter has a native equality proof preserving the serialized source control fields.

The four Fold control outputs, including receipt hashes, and the seven literal closure fields agree with the pinned source. Assembly execution checks cover depths zero through six, four arbitrary prior-word inputs and six signed host indices; the native completeness, uniqueness and return proofs apply to every finite depth. The source literal boundary text predicate is preserved separately from the native exclusion of words containing labels outside the held pair.

Fresh execution comparisons cover 9,240 proposed-support/source-relation combinations and 385 finite-set inputs. The former reach 144 grammar classes; the complete 192-form assembly grammar is verified separately. All four assembly control records, receipt hashes and seven closure fields agree with the pinned V3 artifacts.

Complete Assembly program outputs agree with the pinned source registration, census, all 192 identifier lookup decisions, closure and four controls, including their hashes. An unregistered identifier is rejected.

The machine-readable correspondence lists the V3 source function and source hash for every scientific declaration. The comparison script does not execute or import the V3 scientific modules and is not part of the Lean model.

> Source, proof and provenance: V3 extend, sft/foundation/fold_assembly.py:23; V3 candidate_records, sft/foundation/root_theorem.py:50; V3 receipt_dict, sft/engine/engine.py:497; V3 canonical_value, sft/engine/canonical.py:13; V3 registration, sft/engine/model.py:158. Native results: SFT.Foundation.Root.Evidence.registrationAdapterPreservesRecord, RootEvidence.lean:60; SFT.Foundation.Root.Evidence.typedControlResultMatchesRecord, RootEvidence.lean:139; SFT.Foundation.AssemblyRecords.missingReturnRejected, AssemblyRecords.lean:199; SFT.Foundation.AssemblyRecords.incorrectReturnRejected, AssemblyRecords.lean:209. Timeline: Turn 111, line 1519; Timeline: Turn 023, line 333. Evidence: Completion TURN_111_ROOT_COMPLETION; Completion TURN_115_ASSEMBLY_COMPLETION.

15. Extension of the formalisation

The completed foundation entries include the source closure arguments, controls and program correspondence developed here. Their registered dependencies lead through common-refinement equivalence, Fold and the downstream model. This cumulative edition presents 308 completed source entries; subsequent extensions follow the remaining V3 census and its dependency order. The scientific content remains that of the pinned V3 source.

Edition references: V3 source snapshot; completed-entry census; Unified Constants completion receipt; Timeline: Turn 515, line 5101. Representative native result: AlphaRatio.terminalRatio.

16. Source

Maria Smith. Smithian Fold Theory V3. https://github.com/MettaMazza/ernos-labs-sft-platform/tree/76f1b34e9e9682989adcbb088797b4ae4ce4fcf1

Edition references: V3 source snapshot; completed-entry census; Unified Constants completion receipt; Timeline: Turn 515, line 5101. Representative native result: AlphaRatio.terminalRatio.

17. Discrete collections and held selections

Write C=[c₁,…,c_n] for the ordered source collection. Admission requires n>0 and c_i≠c_j whenever i≠j. For a held list H, successful selection requires $\mathrm{Distinct}(H) \wedge \forall h \in H,\ h \in C$. The source rejects either an outside member or a repeated member: negating both rejection tests gives this conjunction. When both tests are false, its empty/nonempty branch supplies precisely the following two outputs. If H=[] the result is the structural empty-One form; otherwise the result retains H itself. The complete ordered pair support is

$$P(A,B) = \mathrm{concat}_{a \in A}[(a,b) : b \in B]$$

List induction gives (a,b)∈P(A,B) ⇔ a∈A ∧ b∈B. Thus every admitted held relation R⊆P(A,B) retains both origins of each pair. A fresh successor is C′=C++[x] with x∉C; pairwise distinctness of C plus freshness proves distinctness of C′. For a source a, define Out_R(a) as the retained list of pairs whose first endpoint is

a. The source one-image condition is ∀a∈A, length(Out_R(a))=1. This counts pair occurrences, including any repetition, rather than replacing the list by a set.

For finite tuple-of-string forms, generated collections retain their exact input order and reject an empty collection or repeated forms. A successful held selection retains only members of its source collection; the native membership procedure is equivalent to list membership. An empty held selection returns the structural empty-One form, whereas a nonempty selection returns the tuple of held forms. Generated relations retain their accepted pair sequence within the complete Cartesian pair support. Fresh successor construction appends the new form and rejects an already retained member. The total-single-valued predicate counts exactly one outgoing pair for each source member and is true on an empty source. The implementation preserves these source guards, including the distinction between an empty collection and a collection containing an empty form. These operation results combine with the registered candidate grammar and the general discrete laws developed in Sections 18–20.

For arbitrary finite source collections, membership of an ordered pair in complete pair support is equivalent to membership of its two endpoints in their respective collections. Every pair in a successful generated relation therefore has an origin in the left collection and a target in the right. The total-single-valued predicate is equivalent to an exact singleton condition on the retained outgoing pair list for every source member; repeated pairs remain counted as repeated occurrences. Fresh succession preserves collection uniqueness, and appending a fresh form to an accepted generated collection yields another accepted generated collection. These proofs apply at arbitrary finite size by structural induction, without a depth cutoff.

The discrete candidate grammar enumerates the complete product of eight binary questions: collection identity, canonical identity, membership, pair-support relations, map law, generation order, induction closure and additional constructors. All 256 proposals are represented. Exhaustive membership and unique-survivor proofs establish exactly the source all-admitted coordinate, with each rejected coordinate incompatible with acceptance. Every source identifier, exact form, decision and ordered rejection reason agrees under fresh execution. The six original operational witnesses retain their source constructions and predicates. The closure record and all four original control records agree with pinned V3, including one actual enumerated survivor and 256 distinct candidate identifiers. These results preserve the registered finite grammar and supplement the general collection, relation and succession proofs.

> Source, proof and provenance: V3 generated_collection, sft/mathematics/discrete_mathematics/law.py:14; V3 held_selection, sft/mathematics/discrete_mathematics/law.py:20; V3 is_total_single_valued, sft/mathematics/discrete_mathematics/law.py:33. Native results: SFT.Mathematics.DiscreteObjects.anyFalseAtMember, DiscreteObjects.lean:97; SFT.Mathematics.DiscreteObjects.heldSelectionContained, DiscreteObjects.lean:114; SFT.Mathematics.DiscreteObjects.completePairIff, DiscreteRelations.lean:5; SFT.Mathematics.DiscreteObjects.totalMapSingleRetainedPair, DiscreteRelations.lean:80; SFT.Mathematics.DiscreteObjects.freshSuccessorCollection, DiscreteSuccession.lean:68. Timeline: Turn 050, line 747; Timeline: Turn 051, line 762; Timeline: Turn 051, line 763. Evidence: Completion TURN_118_SFT-MATH-DISCRETE-001_COMPLETION.

Exact operation and proof anchors: heldSelection, DiscreteObjects.lean:L26.

18. Finite collection induction

Let Generated(C) mean that C is obtained by a singleton base [x] and steps C↦C++[y] with y∉C. The forward induction establishes

$$\mathrm{Generated}(C) \Longrightarrow C \neq [] \text{ and } C \text{ has no repeated members}$$

Conversely, split any admitted nonempty list into its prefix and final member. A singleton supplies the base. For a longer list, the prefix is nonempty and distinct, and the final member is absent from it; induction constructs the prefix and then the fresh append. Hence Generated(C) is equivalent to the source admission condition. For any property Q, the two premises

$$\forall x, Q([x])$$
$$\forall C, y, \text{Generated}(C) \wedge Q(C) \wedge y \notin C \Longrightarrow Q(C +\!\!+ [y])$$

therefore imply Q(C) for every admitted C. No bound on the finite list length enters this argument.

The accepted domain of the discrete collection operation is exactly characterized by finite construction traces with a singleton base and freshness-preserving append steps. Every such construction yields a nonempty distinct collection accepted by the source operation, and every accepted collection has such a construction. Consequently, a property established for every singleton and preserved by each fresh append holds for every accepted collection, at arbitrary finite length. The converse decomposition recovers both uniqueness of the preceding collection and freshness of the last member. These results concern the exact tuple-of-string operational domain and introduce no additional scientific constructors or free parameters.

> Source, proof and provenance: V3 successor_trace, sft/mathematics/discrete_mathematics/law.py:37. Native results: SFT.Mathematics.DiscreteObjects.acceptedCollectionTrace, DiscreteInduction.lean:100; SFT.Mathematics.DiscreteObjects.acceptedIffConstructionTrace, DiscreteInduction.lean:105. Timeline: Turn 053, line 791. Evidence: Completion TURN_118_SFT-MATH-DISCRETE-001_COMPLETION.

19. Complementary discrete relations

For supported R, define R^c=filter_{p∉R}(P(A,B)), retaining the canonical support order. Filter membership gives

$$p \in R^c \iff p \in P(A,B) \wedge p \notin R$$

Consequently R and R^c have no common member. Every p∈P(A,B) either belongs to R or fails its decidable finite membership test and belongs to R^c. They therefore cover P(A,B). Applying the membership equation twice, with R⊆P(A,B), gives $p \in (R^c)^c \Leftrightarrow p \in R$. This establishes the involution on membership; it does not replace arbitrary held-list order by a claim of list equality.

For accepted generated source collections, the complete Cartesian pair support contains each pair once. Holding precisely the support pairs absent from a given relation produces its complement within that same support, preserving support order. The complement satisfies the original generated-relation acceptance guard. Its membership is equivalent to source-endpoint membership together with absence from the held relation; it is disjoint from the held relation, and the two cover the complete support. For an accepted relation, taking the complement twice restores its membership. This is a membership result: the source permits arbitrary held-pair order, which need not equal the canonical support order.

> Source, proof and provenance: V3 generated_relation, sft/mathematics/discrete_mathematics/law.py:26. Native results: SFT.Mathematics.DiscreteObjects.complementOfGeneratedCollectionsAccepted, DiscreteComplement.lean:151; SFT.Mathematics.DiscreteObjects.doubleComplementMembership, DiscreteComplement.lean:157. Timeline: Turn 054, line 804. Evidence: Completion TURN_118_SFT-MATH-DISCRETE-001_COMPLETION.

20. Composition of discrete maps

For R on A×B and S on B×C, the composition enumerates (a,c) for every retained occurrence (a,b)∈R linked to (b,c)∈S. Therefore

$$(a,c) \in S \circ R \iff \text{there exists } b \text{ with } (a,b) \in R \text{ and } (b,c) \in S$$

If Out_R(a)=[(a,b_a)] and Out_S(b_a)=[(b_a,c_a)], the nested enumeration for a contains precisely [(a,c_a)]. Thus length(Out_(S∘R)(a))=1. Endpoint containment follows from a∈A and c_a∈C. With unique input pairs, two identical output occurrences would require the same unique first link and same unique second link, so no duplicate output is introduced. This derives preservation of the actual singleton-list map condition.

Relation composition retains each pair of linked occurrences through their equal intermediate endpoint. Every composed pair has witnesses in both input relations, and every linked pair is retained. For a total single-valued first relation whose targets belong to the second relation's source, and a total single-valued second relation, the outgoing composed pair list for every original source member is a singleton. Thus composition preserves the original exact one-image predicate. When both input relations satisfy their generated-relation guards on the shared interface, the composed relation also satisfies that guard on the first source and final target: its endpoints are supported and its pairs are unique. These results preserve finite occurrence counts and require no fitted values or additional scientific parameters.

The native discrete program adapter preserves the registered program interface and its complete outputs. Fresh execution agrees with all 256 actual candidate lookups, every candidate trace and decision proof hash, the census completeness certificate, registration fields, closure and generality hashes, and all four control records. Lookup is determined by the candidate identifier, and an unknown identifier retains the source StopIteration failure. Generality serialization retains the original witness fields. These program-method correspondences accompany the general collection induction and relation-composition theorems. Their quantifiers range over arbitrary finite source collections satisfying the original guards; the complete 256-form census records the structural alternatives. Together they supply the completed Discrete entry and its source-bound dependency registration.

> Source, proof and provenance: V3 is_total_single_valued, sft/mathematics/discrete_mathematics/law.py:33. Native results: SFT.Mathematics.DiscreteObjects.totalSupportedRelationDistinct, DiscreteComposition.lean:138; SFT.Mathematics.DiscreteObjects.generatedMapCompositionAccepted, DiscreteComposition.lean:143. Timeline: Turn 055, line 818. Evidence: Completion TURN_118_SFT-MATH-DISCRETE-001_COMPLETION.

21. Primitive Fold maps and ordered composition

On admitted reduced parts $0<x\leq 1$, write $I(x)=x$, $S(x)=x^2$, $K(x)=1$ and $F(x)=\text{cast}(2x)$. Finite word evaluation satisfies $E([],x)=x$ and $E(f::w,x)=E(w,f(x))$; induction gives $E(u++v,x)=E(v,E(u,x))$. The identities $I(x)=I(y)\Rightarrow x=y$ and $K(x)=K(y)=1$ identify the identity and constant behaviors. If $S(x)=S(y)$, positive cross-values have equal squares. If either cross-value were strictly smaller, monotonicity of positive multiplication would make its square strictly smaller too. Both strict alternatives are impossible, so the cross-values agree and reduced-ratio identity forces $x=y$. For $0<x<1$, multiplication by positive x preserves $x<1$, giving $x^2<x$. Repeated application yields $S^n(x)<x$ for every positive n, excluding a return; $x=1$ is fixed. Finally $F(1/2)=F(1)=1$ with $1/2\neq 1$, while $F(1/3)=2/3$ and $F(2/3)=1/3$. Fold is therefore noninjective and has a nonstatic recurrent pair.

The declared primitive self-maps are identity, square, constant One and Fold on reduced positive exact parts through One. Native evaluation retains each original operation, including exact rational normalization for square, and executes arbitrary finite words from left to right. Concatenating two words is equivalent to executing the first followed by the second. Typed primitive words and their original string names have identical execution; unknown names retain the source ValueError. Identity is injective and fixes every input; constant One collapses every input to the fixed One. Squaring preserves its exact rational value and the admitted part domain. Fold is noninjective because half-One and One share an image, and has a nonstatic recurrent orbit through one-third and two-thirds. These primitive behavior results supply the size-one cases of the all-finite-word least-generator theorem in Section 22. Its proof uses the Fold witness to bound any least generating word by length one, then eliminates the other three primitive cases.

The square primitive is injective on the entire admitted exact-part domain. Equality of normalized squared parts implies equality of the original squared cross-products, hence equality of their positive cross-values and of the reduced parts. For every proper part, its square is strictly smaller in exact cross-product order and remains proper. Structural induction then establishes strict contraction at every positive finite iteration count. A positive-time return under square occurs if and only if the input is the fixed One, excluding nonstatic recurrent square orbits throughout the declared domain.

> Source, proof and provenance: V3 primitive, sft/foundation/primitive_map_uniqueness.py:23; V3 apply_word, sft/foundation/primitive_map_uniqueness.py:31. Native results: SFT.Foundation.PrimitiveMaps.wordAppend, PrimitiveMaps.lean:53; SFT.Foundation.PrimitiveMaps.foldNonstaticReturn, PrimitiveMaps.lean:83; SFT.Foundation.PrimitiveMaps.foldNoninjective, PrimitiveMaps.lean:91; SFT.Foundation.PrimitiveMaps.squareIterationsFixOne, PrimitiveSquare.lean:139; SFT.Foundation.PrimitiveMaps.squarePositiveReturnIffOne, PrimitiveSquare.lean:148; SFT.Foundation.PrimitiveMaps.takeSelfRejected, PrimitiveReductions.lean:42; SFT.Foundation.PrimitiveMaps.guardedRemovalNotTotal, PrimitiveReductions.lean:46. Timeline: Turn 057, line 846; Timeline: Turn 058, line 859; Timeline: Turn 061, line 899. Evidence: Completion TURN_119_SFT-FOUNDATION-PRIMITIVE-MAP-UNIQUENESS-001_COMPLETION.

22. Least-size primitive generator

The declared generating predicate is noninjectivity together with nonstatic finite recurrence. At word size one, I fails noninjectivity; S fails noninjectivity and has no nonstatic return; K has only the fixed image One; F satisfies both conditions by the preceding exact identities. Hence F is the unique admitted size-one primitive. A finite word of later size has size>1, so cannot be a least-size generator when F already has size one. Appending one primitive increases size by one and preserves this exclusion. The executed prefix count is

$$4^1 + 4^2 + 4^3 = 4 + 16 + 64 = 84$$

with every ordered word retained. The domain is every finite ordered word over identity, square, constant One and Fold, with generation defined by the two operational properties above. The 84 executed words are its complete size-one-through-three prefix.

For the ordered words just constructed, generation is represented by noninjectivity together with nonstatic finite recurrence. The native behavior proofs exclude identity, square and constant One and establish that Fold generates. Fold is therefore the unique least-size generating word. This result quantifies over every finite word, beyond the executed size-three prefix. Appending a primitive increases construction size by one and preserves the nonminimality of every later word. The complete executed prefix contains all 84 words of sizes one, two and three; each source acceptance decision is equivalent to the derived least-generator predicate. Fresh execution preserves every word, one-based rank, identifier, exact form, decision and reason. The normalization stage uses the source operations on reduced positive parts: multiplication by One retains its operand, self-product gives square, self-junction followed by complete-One cast gives Fold, and a positive whole number of complete turns casts to One. Raw self-junction fails the self-map condition at One, while guarded removal fails it at equal operands. The source uses these reductions at normalized primitive cost through two source occurrences to obtain the four primitive operational forms; ordered composition then generates words of arbitrary finite length. The least-size theorem quantifies that resulting word grammar with the stated noninjectivity-and-recurrence predicate.

The native primitive-map program preserves all 84 actual candidate lookups, complete census and trace hashes, decision proof hashes, registration fields, closure and positive-word-size generality certificate, and original control records. The artifact control checks the full record count, actual survivor count and fourth-record identifier, and its passing result has a zero-axiom proof. Exact preimage and recurrent-orbit controls retain their original computations. Unknown candidate identifiers retain the source KeyError failure. Fresh complete program execution matches the pinned V3 artifacts.

The stated primitive normalization reductions hold for the native exact operations. Multiplication by One on either side preserves every admitted part; self-product is square and casting self-junction is Fold. Any positive Fraction whose numerator is a positive integral multiple of its denominator casts to the One. Raw self-junction is not a total self-map of admitted parts, since its value at One exceeds One. Guarded removal rejects equal operands for every admitted part and therefore is not total. These results retain the source’s declared normalized grammar and operation guards.

More explicitly, let $G(w)$ mean that the evaluated word is noninjective and has a positive-time return at an input it does not fix. Write $M(w)$ for $G(w)$, nonemptiness of w, and least length among all nonempty words satisfying G. Since $G([F])$, minimality gives $|w| \leq 1$; nonemptiness gives $1 \leq |w|$. Hence $|w| = 1$, and the four singleton behavior cases force $w = [F]$. Conversely [F] generates and every nonempty competitor has length at least one. Therefore

$$M(w) \iff w = [F] \qquad (w \in \{I, S, K, F\}^*).$$

For any word with $|w| > 1$, comparison with [F] already contradicts minimality. Appending a primitive p gives $|w +\!\!+ [p]| = |w| + 1 > 1$, preserving this exclusion at every further finite extension. This is the proof of leastGeneratorIff, PrimitiveMinimality.lean:84, everyLargerWordNonminimal, PrimitiveMinimality.lean:89 and successorPreservesNonminimality, PrimitiveMinimality.lean:100. The census-to-operation bridge sourceAcceptanceIffLeastGenerator, PrimitiveMapGrammar.lean:106 equates each executed candidate decision with this same least-generator predicate.

> Source, proof and provenance: V3 closure_record, sft/foundation/primitive_map_uniqueness.py:70. Native results: SFT.Foundation.PrimitiveMaps.appendPrimitiveIncreasesSize, PrimitiveMinimality.lean:94; SFT.Foundation.PrimitiveMaps.successorPreservesNonminimality, PrimitiveMinimality.lean:100; SFT.Foundation.PrimitiveMapGrammar.wordInjective, PrimitiveMapGrammar.lean:103; SFT.Foundation.PrimitiveMapGrammar.sourceAcceptanceIffLeastGenerator, PrimitiveMapGrammar.lean:106. Timeline: Turn 059, line 872; Timeline: Turn 059, line 871. Evidence: Completion TURN_119_SFT-FOUNDATION-PRIMITIVE-MAP-UNIQUENESS-001_COMPLETION.

23. Ordered exact derivation traces

For an accepted operation o and current value x, let T_o(x) be its exact result. A constructed row records o, the ordered inputs, y=T_o(x), and the canonical identity H(y). Replay accepts the row only when its recomputed y and H(y) both equal the record. With R denoting replay, the base and successor equations are

$$R([], x) = x$$
$$R(\mathrm{row}(o, x) :: t, x) = R(t, T_o(x))$$

when the recorded fields match; otherwise replay rejects. Induction proves replay of every constructed finite trace, and splitting at a prefix gives R(u++v,x)=R(v,R(u,x)) whenever the prefix succeeds. In the concrete three-move trace,

$$1/3 \to F2/3 \to \mathrm{Take}(1, \cdot)1/3 \to F2/3$$

The terminal equals the direct first Fold result, while an unregistered operation or a recorded output unequal to the recomputed value fails the corresponding replay condition.

A derivation row retains its operation name, ordered input identities, exact output and output identity. The native replay function starts at one-of-three and executes the registered Fold or guarded Take operation in source order. Each result must match both the recorded exact output and its canonical SHA256 identity. Unknown operations and mismatched outputs retain the original rejection behavior.

Constructive induction establishes that every successfully constructed finite operation sequence replays to its computed terminal value. If a recorded prefix replays to a given exact part, appending a row constructed from the next accepted operation extends replay to that operation’s exact result. Concatenation preserves replay through the intermediate terminal. These theorems have no finite-length cutoff and add no scientific parameters or axioms. They formalise the base/successor replay component of SFT-FOUNDATION-DERIVATION-TRACE-001. The registered source dependencies and complete executable evidence are verified alongside these general replay results.

The concrete three-move derivation computes Fold(one-of-three), Take(One, two-of-three), and Fold(one-of-three), retaining the exact outputs two-of-three, one-of-three, and two-of-three. Its terminal agrees with the independent direct Fold route. All ordered input identities and exact output identities are retained. Replacing the final operation with Select is rejected as unregistered. Changing the final exact output to one-of-three while preserving its other fields is rejected by replay. Native proofs establish these outcomes and the corresponding boundary control. Fresh native execution agrees with the source artifacts for all four original control records and their receipt hashes, and with independent exact arithmetic and hashing for every row and mutation.

The derivation-record grammar is the complete product of eight source axes: source custody, dependency coverage, step order, intermediate exactness, operation registration, replay equality, terminal identity and added-data status. Its 256 records retain source enumeration order, including no-extra before has-extra on the final axis. Exhaustive membership and cardinality are proved, and acceptance is equivalent to the conjunction of the eight admitted criteria. The unique survivor is the complete source-bound, dependency-complete, ordered, exact, registered, identically replayed, terminal-bound record without extra data. Each rejected axis excludes acceptance.

Fresh native program execution reproduces every candidate, every actual identifier lookup and the complete census, trace, decision, closure and generality digests. Registration, original control receipts and unknown-identifier behavior also agree with pinned V3. The classifier and program-correspondence results accompany the operational theorem for arbitrary finite operation lists: the empty list replays its initial value, and each successful registered step reduces replay of the constructed head-and-tail record to replay of its tail. Concatenation retains the intermediate exact part. The complete entry combines this induction with all 256 structural candidates and the source-bound dependency registration.

> Source, proof and provenance: V3 replay, sft/foundation/derivation_trace.py:50; V3 closure_record, sft/foundation/derivation_trace.py:80. Native results: SFT.Foundation.DerivationTrace.generatedTraceReplays, DerivationTrace.lean:84; SFT.Foundation.DerivationTrace.replayAppend, DerivationTrace.lean:107; SFT.Foundation.DerivationTraceGrammar.terminal_rejected_rejected, DerivationTraceGrammar.lean:237; SFT.Foundation.DerivationTraceGrammar.addition_rejected_rejected, DerivationTraceGrammar.lean:246. Timeline: Turn 062, line 912; Timeline: Turn 064, line 939. Evidence: Completion TURN_119_SFT-FOUNDATION-DERIVATION-TRACE-001_COMPLETION.

24. Finite combinatorial selections

Selections are generated by retaining the previous ordered family and appending an ordered copy with the next carrier label held at the end of each trace. A native finite construction relation records the absent and held choices at each successor. Structural induction proves that this relation describes exactly the generated raw traces, that every retained label belongs to the carrier, and that fresh-member absent and held branches are disjoint. For a unique carrier, raw selection traces are unique at every finite length. The output-trace count at a successor is the sum of the two generated branch lengths; no closed-form counting formula is introduced.

The executable output conversion replaces the empty trace with the source singleton empty-One label and preserves trace count. Selection input guards and fresh-extension guards retain their exact domains and errors. The original three-label example and its fresh-extension construction are proved in source order. Fresh Lean comparisons cover 341 selection inputs and 32 recurrence inputs, including the literal empty-One encoding. The uniqueness theorem concerns pre-conversion choice traces. Sections 29 and the subsequent carrier constructions state the precise encoding and discrete-admission results.

Write list concatenation as ++, the empty trace as [], and the ordered raw family on a carrier C as S(C). The construction is

$$S([]) = [[]]$$
$$S(C \mathbin{+\!\!+} [x]) = S(C) \mathbin{+\!\!+} [u \mathbin{+\!\!+} [x] : u \in S(C)]$$

where the second list retains the order of S(C). Define H(C,u) by the base H([],[]) and the two successor rules H(C,u)⇒H(C++[x],u) and H(C,u)⇒H(C++[x],u++[x]). At the base, membership in [[]] is exactly u=[]. At a successor, membership in a concatenation is membership in either branch; membership in its mapped branch means u=v++[x] for some v∈S(C). Substituting the induction hypothesis in these two cases gives

$$u \in S(C) \iff H(C,u)$$

Induction on H also gives y∈u⇒y∈C. If x∉C, no old trace u can equal v++[x], since the right side contains x and containment would force x∈C. Appending [x] is injective: u++[x]=v++[x]⇒u=v. Therefore distinctness of the old family implies distinctness of each branch and disjointness between them, proving distinctness at the successor. Length of concatenation and preservation of length by mapping give

$$|S(C \mathbin{+\!\!+} [x])| = |S(C)| + |S(C)|$$

For C=[a,b,c], the ordered raw result is [[],[a],[b],[a,b],[c],[a,c],[b,c],[a,b,c]]. The output map e sends [] to the source empty-One trace and leaves every nonempty trace unchanged. Thus |map(e,S(C))|=|S(C)|; this count identity does not by itself assert injectivity of the encoding. The public generator admits C precisely when C≠[] and its labels are distinct.

> Source, proof and provenance: V3 selections, sft/mathematics/combinatorics/law.py:24; V3 recurrence_extend, sft/mathematics/combinatorics/law.py:45. Native results: SFT.Mathematics.CombinatorialSelections.recurrenceExistingRejected, CombinatorialSelections.lean:182; SFT.Mathematics.CombinatorialSelections.exactFreshExtension, CombinatorialSelections.lean:189. Timeline: Turn 065, line 953. Evidence: Completion TURN_118_SFT-MATH-COMBINATORICS-001_COMPLETION.

25. Finite arrangements by held choice

The native arrangement generator retains the source carrier order: each available label is held in turn, all occurrences of that held label are removed from the recursive carrier, and the held label is prepended to every resulting suffix. Nonempty unique carriers retain the source admission guard. On this domain, each remainder removes exactly one label, remains unique, and is nonempty whenever the parent has at least two members. Recursion is justified by strictly decreasing carrier length, without a depth cutoff or an imported counting formula.

Well-founded induction proves that every generated arrangement is a permutation of the unique source carrier. Thus every output retains all original labels with their multiplicities and introduces none. Fresh Lean execution matches independently generated permutations in exact order for 64 admitted carriers, comprising 748 arrangement rows, within 341 input cases including rejected empty and repeated-label carriers. The unbounded output-permutation theorem is distinct from these finite execution checks. Converse enumeration completeness and uniqueness of the complete output family are established for every nonempty unique finite carrier.

For a held label x define R(C,x) by deleting all occurrences of x while preserving the order of all other labels:

$$R([], x) = []$$
$$R(y :: C, x) = R(C, x) \text{ if } y = x, \text{ and } y :: R(C, x) \text{ otherwise}$$

Its membership law is y∈R(C,x) ⇔ (y∈C and y≠x). For distinct C and x∈C, induction on C yields

$$|R(C,x)| + 1 = |C|, \mathrm{distinct}(R(C,x))$$

Indeed, at the occurrence of x its removal contributes exactly one, and distinctness excludes further occurrences; before it, every retained head adds one to both lengths. If |C|≥2, the remainder is nonempty, and its smaller length justifies the recursive call.

Let A(C) be the raw arrangement family. Its source equations are

$$A([]) = [], A([x]) = [[x]]$$
$$A(C) = \mathrm{concat}_{x \in C}[x :: u : u \in A(R(C,x))] \qquad (|C| \geq 2)$$

The empty input is rejected by the public guard. Write u~C for list permutation, retaining multiplicities. Removing the held occurrence gives C~x::R(C,x). By the induction hypothesis u∈A(R(C,x)) implies u~R(C,x), hence

$$x :: u \sim x :: R(C,x) \sim C$$

The singleton base is immediate. Well-founded induction on |C| therefore proves that every generated arrangement permutes its carrier.

> Source, proof and provenance: V3 arrangements, sft/mathematics/combinatorics/law.py:12. Native results: SFT.Mathematics.CombinatorialArrangements.arrangementsEmpty, CombinatorialArrangements.lean:182; SFT.Mathematics.CombinatorialArrangements.everyArrangementPermutesCarrier, CombinatorialArrangements.lean:187. Timeline: Turn 066, line 967. Evidence: Completion TURN_118_SFT-MATH-COMBINATORICS-001_COMPLETION.

26. Arrangement completeness and uniqueness

Every permutation of a nonempty unique carrier is generated by the original arrangement function. The proof takes the first label of a proposed permutation, establishes its occurrence in the carrier, and identifies the suffix as a permutation of the exact held-label remainder. Well-founded induction then supplies the recursive suffix, which the existing held-choice branch prepends in source order. Together with output preservation, this proves membership in the generated family if and only if a form permutes the carrier.

Each held-label branch is duplicate-free by recursive uniqueness and injectivity of prepending a fixed label. Different branches have distinct first labels and are disjoint. Their source-ordered concatenation is therefore duplicate-free. The combined theorem applies directly to every successful call of the guarded arrangements API: its output contains every carrier permutation exactly once. The result is uniform in finite carrier size and uses no counting formula or depth bound.

For the converse, take v~C with C nonempty and distinct. Permutations preserve length, so v is nonempty; write v=x::u. Preservation of membership gives x∈C. Applying removal of x to the permutation gives

$$R(x :: u, x) \sim R(C,x)$$

Distinctness is preserved by permutation, so x∉u and R(x::u,x)=u. Thus u~R(C,x). In the nonsingleton case the remainder is admitted and shorter, and induction gives u∈A(R(C,x)); the x branch then supplies x::u∈A(C). In the singleton case equal length and membership force the sole output. Combining both directions yields

$$v \in A(C) \iff v \sim C$$

For uniqueness, x::u=x::v implies u=v, so prepending preserves the recursive family's distinctness. An intersection of branches held at x and y would give x::u=y::v and hence x=y; distinct carrier labels exclude this between different branches. Consequently the ordered concatenation has no duplicate traces. For every successful guarded call arrangements(C)=ok(L),

$$\mathrm{distinct}(L) \text{ and } (v \in L \iff v \sim C)$$

This establishes exact enumeration and uniqueness together, for every admitted finite carrier.

> Source, proof and provenance: V3 arrangements, sft/mathematics/combinatorics/law.py:12. Native results: SFT.Mathematics.CombinatorialArrangements.arrangementFamilyDistinct, ArrangementCompleteness.lean:182; SFT.Mathematics.CombinatorialArrangements.acceptedArrangementsCompleteUnique, ArrangementCompleteness.lean:198. Timeline: Turn 067, line 981. Evidence: Completion TURN_118_SFT-MATH-COMBINATORICS-001_COMPLETION.

27. Disjoint class accounting

For the classifier's resulting text labels, the class ledger retains keys in first-occurrence order and appends each input occurrence to its matching class. Empty input and repeated items retain their original behavior. Native proofs establish that class insertion and complete ledger construction preserve the input family as a permutation, hence preserve every occurrence and the total count. Class keys are unique.

Lookup of any class equals the stable filter of the input by that class label. This equality preserves both multiplicity and within-class order, and implies that different labels have disjoint classes. These results hold for arbitrary finite inputs and classification functions at the source text-label boundary. Fresh Lean execution matches independent first-occurrence ordering and per-class filtering in 1,026 cases, including identity, constant and the original a/other classification witness.

Let I(L,t,x) insert the occurrence x into the ledger L under the text label t. Its equations are

$$I([],t,x) = [(t,[x])]$$
$$I((k,G)::L,t,x) = (k, G +\!\!+ [x])::L \text{ if } k = t$$
$$I((k,G)::L,t,x) = (k,G)::I(L,t,x) \text{ if } k \neq t$$

Starting with L=[], process input occurrences in their original order using t=f(x). An existing key remains in its position; a new key reaches the empty suffix and is appended. This proves first-occurrence key order and uniqueness of keys by induction.

Write F(L) for concatenation of the groups. In either insertion branch,

$$F(I(L,t,x)) \sim F(L) +\!\!+ [x]$$

where ~ denotes permutation with multiplicity. For a matching group, moving x past the subsequent groups proves this relation; for a nonmatching group, retain that head group and apply induction to the suffix. Iterating gives F(ledger(C,f))~C and therefore |F(ledger(C,f))|=|C|, including repeated occurrences.

For lookup Q(L,u), insertion satisfies

$$Q(I(L,t,x),u) = Q(L,u) +\!\!+ [x] \text{ if } t = u$$
$$Q(I(L,t,x),u) = Q(L,u) \text{ otherwise}$$

Induction on the input then yields Q(ledger(C,f),u)=filter(x↦f(x)=u,C). This stable filter retains precisely the original occurrences of class u in their original order. In particular x∈Q(ledger(C,f),u) iff x∈C and f(x)=u. If u≠v and an item belonged to both groups, these two equalities would give u=f(x)=v, a contradiction. This proves disjointness directly, without signed inclusion–exclusion.

> Source, proof and provenance: V3 disjoint_class_ledger, sft/mathematics/combinatorics/law.py:33. Native results: SFT.Mathematics.CombinatorialLedger.differentClassesDisjoint, CombinatorialLedger.lean:180; SFT.Mathematics.CombinatorialLedger.emptyLedger, CombinatorialLedger.lean:189. Timeline: Turn 068, line 995. Evidence: Completion TURN_118_SFT-MATH-COMBINATORICS-001_COMPLETION.

28. Combinatorial witnesses and complete grammar

The five registered operational witnesses are retained: six unique ordered arrangements of three labels; exact carrier coverage of each arrangement; eight distinct selection forms including the empty One; the original a/other class ledger; and the fresh-member extension count. Each witness is evaluated from the translated operations and has a native proof. The six arrangement rows are derived through the original recursive equations in exact source order.

The combinatorial candidate grammar contains all 256 products of carrier coverage, choice rule, provenance, duplication policy, symmetry identity, class accounting, recurrence closure and extra-formula status. Native proofs establish exhaustive membership, cardinality, acceptance exactly at the all-admitted coordinate, and rejection of each inadmissible coordinate. Fresh native program execution reproduces all candidate lookups, source records, census and trace identities, decision proofs, closure and generality certificates, registration fields and four original control receipts. The general operational derivations accompany these finite grammar and program-correspondence results. The combinatorics entry is complete against its registered V3 source; the encoding results retain their precise domains as stated below.

The held-choice recursion on [a,b,c] produces, in order,

$$[a,b,c],[a,c,b],[b,a,c],[b,c,a],[c,a,b],[c,b,a]$$

There are six distinct traces, and direct inspection of each trace gives length three and the exact label set {a,b,c}. The selection recurrence of Section 24 produces its eight listed traces, with [] encoded as the empty One. For the original classifier f(a)=a-class and f(b)=f(c)=other-class, the ledger is [(a-class,[a]), (other-class,[b,c])], whose flattening is [a,b,c]. Extending the four encoded selections on [a,b] by c retains those four and appends [c],[a,c],[b,c],[a,b,c], giving eight traces. Thus all five witness predicates follow from the same executed constructors.

For the grammar, write its eight binary coordinate domains as $D_1,\ldots,D_8$ in the source order listed above. The complete ordered Cartesian product is $D_1\times\cdots\times D_8$; successive two-way branching gives $2^8=256$ candidates. Its acceptance predicate is the conjunction of the eight coordinate predicates. The all-admitted tuple satisfies each conjunct. Conversely, acceptance forces each coordinate to its admitted value, so coordinate equality identifies that tuple uniquely. A rejected value in any coordinate falsifies its own conjunct. The 256 coordinate tuples classify structural conditions. The operational domain is generated independently by the absent/held selection recursion and the recursive choice of every remaining arrangement label. Their membership and uniqueness inductions cover arbitrary finite admitted carriers; the coordinate proof above exhausts the corresponding structural alternatives. Sections 29–35 then retain the empty-One encoding, source order and fresh-member conditions of those constructions.

> Source, proof and provenance: V3 arrangements, sft/mathematics/combinatorics/law.py:12; V3 selections, sft/mathematics/combinatorics/law.py:24. Native results: SFT.Mathematics.CombinatoricsWitnesses.freshExtensionPassed, CombinatoricsWitnesses.lean:76; SFT.Mathematics.CombinatoricsWitnesses.allWitnessesPass, CombinatoricsWitnesses.lean:80. Timeline: Turn 069, line 1009. Evidence: Completion TURN_118_SFT-MATH-COMBINATORICS-001_COMPLETION.

29. Selection encoding and extension

For a raw selection form, source extension commutes with empty-form encoding exactly when that raw form differs from the singleton literal empty-One marker. A native equivalence proves both directions. For distinct admitted carriers without that literal marker, encoding preserves distinct selection forms. With a fresh label different from the marker, the original recurrence operation on encoded selections equals the original selections operation on the carrier extended by that label. These conditions are explicit theorem premises; the executable input guards remain those of v3. The source accepts the literal marker as a

label, and its singleton output is proved to contain two identical encoded forms. The full-domain result uses raw ordered-choice provenance and exact encoded membership. Marker-separation remains a premise of the encoded injectivity theorem, rather than an additional executable input guard.

Let m=["empty-One"] be the literal marker trace, e([])=m and e(x::u)=x::u. Define extension X(v,z)=[z] if v=m, and v++[z] otherwise. If u=[], then X(e(u),z)=[z]=e(u++[z]). If u is nonempty and u≠m, encoding leaves u unchanged and extension appends z, giving the same equality. At u=m, however, the left side is [z] and the right side is m++[z], with lengths one and two. Consequently

$$X(e(u), z) = e(u +\!\!+ [z]) \iff u \neq m$$

On raw forms different from m, encoding is injective. Two nonempty forms are unchanged; if one is empty, equality of the encoded outputs forces the other to be m, excluded by the premise. If the carrier contains no literal marker label, containment of raw selections excludes m from every raw form. Encoding therefore preserves distinctness on that domain.

Mapping e over the raw successor equation and applying the commutation identity yields

$$\mathrm{map}(e, S(C +\!\!+ [z])) = \mathrm{map}(e, S(C)) +\!\!+ \mathrm{map}(v \mapsto X(v, z), \mathrm{map}(e, S(C)))$$

When z∉C and z≠"empty-One", it occurs in neither an ordinary encoded selection nor the encoded empty trace; hence the recurrence's original freshness guard passes. With the original nonempty/distinct carrier guard, this establishes the exact successful API recurrence. The executable guard itself does not exclude the marker: on C=["empty-One"], S(C)=[[],m] and map(e,S(C))=[m,m], exactly as in V3. The unrestricted correspondence is instead

$$v \in \mathrm{map}(e, S(C)) \iff \exists u, H(C, u) \text{ and } e(u) = v$$

It retains both raw derivations even when their displayed encodings coincide. No marker restriction is added to the source operation.

> Source, proof and provenance: V3 recurrence_extend, sft/mathematics/combinatorics/law.py:45. Native results: SFT.Mathematics.CombinatorialSelections.selectionRecurrenceExact, SelectionEncoding.lean:129; SFT.Mathematics.CombinatorialSelections.encodedLiteralBehavior, SelectionEncoding.lean:144. Timeline: Turn 070, line 1023. Evidence: Completion TURN_118_SFT-MATH-COMBINATORICS-001_COMPLETION.

30. Arrangement successor partition and provenance

For every nonempty distinct finite carrier and every fresh label, a form belongs to the original successor arrangement family if and only if it is obtained by inserting the fresh label between a prefix and suffix of a prior generated arrangement. The proof removes the fresh label from the successor carrier, identifies the remaining form through permutation completeness, and reconstructs every inserted form through the same original generator. The prefix and suffix are uniquely determined: freshness prevents the inserted label from being mistaken for any prior occurrence. Thus the successor family is exhausted by disjoint insertion-position classes with retained prior construction provenance. This result is uniform over finite carrier size and leaves the source-ordered enumeration unchanged.

Let C be nonempty and distinct, and z∉C. Removal distributes over concatenation, R(U++V,z)=R(U,z)++R(V,z); therefore R(C++[z],z)=C. If v∈A(C++[z]), permutation preservation places z in v. Splitting at that occurrence gives v=B++z::D. Moving z past B gives B++z::D~z::(B++D). Removal of z from the successor permutation now yields B++D~C, so completeness supplies B++D∈A(C).

Conversely, if B++D∈A(C), then B++D~C. Prepending z and moving it to the indicated position gives B++z::D~C++[z]. Successor completeness therefore generates that inserted form. Hence

$$v \in A(C +\!\!+ [z]) \iff \exists B, D, B +\!\!+ D \in A(C) \text{ and } v = B +\!\!+ z :: D$$

For uniqueness, suppose B++z::D=B′++z::D′. Neither prefix contains z, because each belongs to a prior arrangement of C. If one prefix is empty and the other is not, equality of the first labels forces z into the nonempty prefix, a contradiction. Otherwise equality of first labels reduces to the same problem on the tails. Induction gives B=B′ and D=D′, retaining the unique prior arrangement and insertion position.

> Source, proof and provenance: V3 arrangements, sft/mathematics/combinatorics/law.py:12. Native results: SFT.Mathematics.CombinatorialArrangements.insertionPositionUnique, ArrangementSuccessor.lean:79; SFT.Mathematics.CombinatorialArrangements.freshArrangementUniqueProvenance, ArrangementSuccessor.lean:104. Timeline: Turn 071, line 1036. Evidence: Completion TURN_118_SFT-MATH-COMBINATORICS-001_COMPLETION.

31. Arrangement trace accounting and arithmetic linkage

The count of the arrangement family is obtained directly from its ordered held-choice branches. A singleton carrier generates one arrangement. At each larger carrier, the length of the complete generated trace equals the ordered sum of the generated remainder-family lengths; prepending the held label preserves each branch length. This equation is derived from the enumerated trace and does not replace its generation with a closed-form answer.

For every admitted carrier of at least two labels, the first held-choice branch and the concatenation of the remaining branches are both nonempty and disjoint. Applying the existing exact-arithmetic disjoint junction to these actual traces succeeds under its original guards and returns the original complete arrangement family. This connects combinatorial branch accounting directly to the translated arithmetic operation while preserving source order and every generated form.

For a supplied carrier C of at least two labels, define its held branch B_x=map(u↦x::u,A(R(C,x))). The original recursion states A(C)=concat_(x∈C) B_x in carrier order. Mapping preserves length and concatenation adds lengths, so

$$|A(C)| = \sum_{x \in C} |B_x| = \sum_{x \in C} |A(R(C,x))|$$
$$|A([x])| = 1$$

The sum is the finite ordered fold of actual branch lengths, with host zero only as the empty accumulator; the arrangement family is still generated in full.

For C=first::second::rest with distinct labels, each relevant remainder is a nonempty distinct carrier. Its identity permutation is generated by completeness, proving A(R(C,x)) nonempty. Thus B_first and the concatenation of the remaining branches are nonempty. An intersection would equate first::u with x::v for some x in second::rest, forcing first=x and contradicting distinctness. The arithmetic overlap test consequently returns false, and both nonempty guards pass. Its exact operation therefore gives

$$\mathrm{disjointJunction}(B_{\mathrm{first}}, concat_{x \in \mathrm{second::rest}} B_x) = ok(A(C))$$

This equality retains every source-ordered branch and connects the count to the actual arithmetic junction.

> Source, proof and provenance: V3 arrangements, sft/mathematics/combinatorics/law.py:12. Native results: SFT.Mathematics.CombinatorialArrangements.arrangementCountRecurrence, ArrangementCounting.lean:80; SFT.Mathematics.CombinatorialArrangements.arrangementCountBase, ArrangementCounting.lean:87. Timeline: Turn 072, line 1049. Evidence: Completion TURN_118_SFT-MATH-COMBINATORICS-001_COMPLETION.

32. Complete encoded selection output

For every successful call to the original selections operation, output membership is equivalent to the existence of a raw absent/held construction whose empty-form encoding equals the returned form. This equivalence covers the full string-label input domain of the source API. Every such output is a positive

trace and retains the structural empty One, derived by choosing the absent branch at each carrier extension.

If the carrier omits the literal empty-One label, each encoded result has a unique raw-form preimage and the output is duplicate-free. These uniqueness conclusions retain their explicit premise; the full-domain membership and completeness theorem requires no such exclusion. The preimage result identifies raw forms, without asserting equality of proof terms or erasing the source construction order.

Let E(C)=map(e,S(C)), using the raw held-choice family and encoding already defined. The list-map membership rule gives

$$v \in E(C) \iff \exists u, u \in S(C) \text{ and } e(u) = v$$

Substituting u∈S(C)⇔H(C,u) proves the stated full-domain correspondence. The base H([],[]) extends to H(C,[]) by taking the absent constructor at every successor. Since e([])=m, the empty-One trace m belongs to E(C); therefore E(C) is nonempty. Each encoded form is itself nonempty: the empty raw form becomes the singleton marker, and every other raw form is retained.

If the carrier contains no marker label, suppose H(C,u), H(C,w) and e(u)=v=e(w). Both raw forms avoid m by containment, and encoding injectivity gives w=u. This establishes a unique raw-form preimage, while distinctness of the raw family yields distinctness of E(C). For an accepted call selections(C)=ok(L), the original guard supplies nonempty/distinct C and the implementation supplies L=E(C). Substitution yields the membership and positivity statements for the actual returned family, with encoded uniqueness requiring only its separately stated marker-absence premise.

> Source, proof and provenance: V3 selections, sft/mathematics/combinatorics/law.py:24. Native results: SFT.Mathematics.CombinatorialSelections.acceptedSelectionsComplete, SelectionCompleteness.lean:47; SFT.Mathematics.CombinatorialSelections.acceptedSelectionsCompleteUnique, SelectionCompleteness.lean:62. Timeline: Turn 073, line 1062. Evidence: Completion TURN_118_SFT-MATH-COMBINATORICS-001_COMPLETION.

33. Discrete collection integration

Every arrangement family over a nonempty distinct carrier satisfies the original discrete generated-collection operation and has its finite singleton/successor construction trace. Each held-choice branch is a nonempty, duplicate-free subset of the complete family, so the original held-selection operation accepts it and retains its exact tuple representation.

The encoded selection family likewise has a generated-collection construction trace when the carrier is distinct and omits the literal empty-One label. The concrete composition on the singleton literal-marker carrier is also established: selections returns two equal encoded forms, and generated_collection rejects that result through its existing duplicate guard. The translated operations preserve this source behavior. No unconditional collection-admission conclusion is inferred for all encoded selections.

Write G(L) for the generated-collection operation. Its source guard requires L nonempty and its member forms distinct. For an admitted arrangement carrier C, the identity permutation C belongs to A(C), so A(C) is nonempty; Section 26 supplies its distinctness. Consequently

$$G(A(C)) = ok(A(C))$$

The existing collection construction theorem then supplies CollectionTrace(A(C)): a nonempty distinct family is built from its first singleton by appending each remaining fresh form, preserving source order.

For the held branch B_x, membership in the branch supplies membership in A(C) through the exact concatenation equation. Its recursive suffix family is nonempty and distinct, and prepending x is injective, so B_x has both properties. Thus every held-selection guard passes and

$$\mathrm{heldSelection}(A(C), B_x) = ok(\mathrm{collectionValue}(B_x))$$

For E(C), Section 32 supplies nonemptiness and, when the literal marker is absent, distinctness. Hence G(E(C))=ok(E(C)) and its CollectionTrace follows in the same way. At the literal-marker singleton, however, E(["empty-One"])=[m,m]; the duplicate guard rejects this exact family. This is the original composition of the two operations, without a new restriction on the selections input domain.

> Source, proof and provenance: V3 arrangements, sft/mathematics/combinatorics/law.py:12; V3 selections, sft/mathematics/combinatorics/law.py:24. Native results: SFT.Mathematics.CombinatorialCollections.arrangementHeldBranchAdmitted, CombinatorialCollections.lean:43; SFT.Mathematics.CombinatorialCollections.encodedLiteralCollectionRejected, CombinatorialCollections.lean:59. Timeline: Turn 074, line 1073. Evidence: Completion TURN_118_SFT-MATH-COMBINATORICS-001_COMPLETION.

34. Ordered selection provenance

A raw form is generated by the absent/held recurrence if and only if it is an order-preserving subsequence of the carrier. Both directions are established constructively: every recurrence step preserves source order, while every ordered choice can be built by retaining or skipping carrier positions. Encoding this relation gives the exact membership characterization of the returned selection family, including the empty-One conversion.

Every held successor is the existing Fold word-append operation on the prior raw form. Its return operation recovers that exact prior form, and its last-label operation recovers the appended label. This retains the local construction provenance of every held step. The word-operation identity does not extend the fixed binary Fold support to arbitrary string labels.

Write u⊑C for an order-preserving sublist. The base []⊑[] matches the base held-choice constructor. If u⊑C, skipping a new final label gives u⊑C++[x], while retaining it gives u++[x]⊑C++[x]. Induction on H(C,u) therefore proves H(C,u)⇒u⊑C. For the converse, sublist construction either skips a leading carrier label or retains that label at the front of both lists. Prefixing an absent or held choice is itself derived by induction on the existing held-choice construction. These two prefix constructions translate the sublist induction into H(C,u). Thus

$$u \in S(C) \iff H(C,u) \iff u \sqsubseteq C$$
$$v \in E(C) \iff \exists u, u \sqsubseteq C \text{ and } e(u) = v$$

For each held successor, appendLabel(u,x)=u++[x]. The established word identities give

$$\mathrm{returnWord}(\mathrm{appendLabel}(u,x)) = \mathrm{some}(u)$$
$$\mathrm{lastLabel}(\mathrm{appendLabel}(u,x)) = \mathrm{some}(x)$$

The held-choice successor rule simultaneously places this appended word in S(C++[x]). These equalities recover the exact local predecessor and label without changing the support of the binary Fold grammar.

> Source, proof and provenance: V3 selections, sft/mathematics/combinatorics/law.py:24. Native results: SFT.Mathematics.CombinatorialSelections.encodedMembershipIffOrderedChoice, SelectionProvenance.lean:52; SFT.Mathematics.CombinatorialSelections.heldExtensionRetainsFoldProvenance, SelectionProvenance.lean:64. Timeline: Turn 075, line 1084. Evidence: Completion TURN_118_SFT-MATH-COMBINATORICS-001_COMPLETION.

35. Raw choices as discrete held selections

Each raw selection of a distinct carrier retains distinct labels in carrier order. Representing each label by its singleton form gives a generated discrete carrier collection. Under the same representation, every raw selection is accepted by the original discrete held-selection operation: a nonempty choice returns its exact collection tuple, while an empty choice returns the original structural empty-One form. The proof uses

retained membership and distinctness and requires no exclusion of the literal empty-One label. This relation concerns raw choices and explicitly represented member forms; it does not identify the differently nested encoded outputs of the two source functions.

For any sublist u⊑C, membership is retained: x∈u⇒x∈C. Induction on the sublist constructors also preserves distinctness. A skipped carrier head leaves the induction result unchanged; a retained head was absent from the carrier tail and hence from its sublist, so it remains fresh in the selected trace. Thus distinct(C) and u∈S(C) imply distinct(u).

Define the singleton representation J(C)=map(x↦[x],C). It is injective on labels because [x]=[y] implies x=y. Therefore nonempty distinct C yields nonempty distinct J(C), and generatedCollection(J(C))=ok(J(C)). For u∈S(C), retained membership gives J(u)⊆J(C), and injectivity gives distinct(J(u)). Substitution in the original held-selection guards yields

$$\mathrm{heldSelection}(J(C), J(u)) = \begin{cases} \mathrm{ok}(\mathrm{formValue}(m)), & u = [], \\ \mathrm{ok}(\mathrm{collectionValue}(J(u))), & u \neq []. \end{cases}$$

The empty branch is the original empty-One case; the nonempty branch retains the nested singleton forms. All of these statements permit the literal marker as a carrier label because the map x↦[x] remains injective there.

> Source, proof and provenance: V3 selections, sft/mathematics/combinatorics/law.py:24. Native results: SFT.Mathematics.CombinatorialSelections.carrierMemberCollection, SelectionDiscreteLink.lean:35; SFT.Mathematics.CombinatorialSelections.rawSelectionDiscreteAdmission, SelectionDiscreteLink.lean:43. Timeline: Turn 076, line 1096. Evidence: Completion TURN_118_SFT-MATH-COMBINATORICS-001_COMPLETION.

36. Finite algebraic operation tables

The operation-table constructor retains the registered carrier and ordered input/output rows. It first requires a nonempty distinct carrier, then checks that row count equals the complete ordered pair support and that the input sets coincide, and finally checks every result belongs to the carrier. These are the original source checks in their original order. Accepted tables preserve their data, cover exactly the carrier pair support, and have carrier-closed results.

Application filters the retained rows by the requested ordered input pair and returns a result only when exactly one row matches. Empty or multiple matches retain the source error. Fresh execution matches 526 constructor cases and 2630 lookup cases, including complete two-member operations in every row order and malformed inputs. General single-valuedness and total lookup follow from the constructor conditions as established below; the remaining algebraic property derivations are tracked separately.

For the retained carrier C, define the ordered support P(C)=concat_(a∈C)[(a,b):b∈C]. For the row list R, let K(R) project each row (a,b,c) to its input pair (a,b). The constructor evaluates, in order,

$$C \neq [] \text{ and } \mathrm{distinct}(C)$$
$$|R| = |P(C)| \text{ and } (\forall p, p \in K(R) \iff p \in P(C))$$
$$\forall (a,b,c) \in R, c \in C$$

The second line expresses the actual two-direction membership test together with the length test; it does not assume row-input uniqueness. Passing these checks returns exactly the retained pair of lists (C,R), without sorting or replacing the supplied rows.

For a requested pair (a,b), define M_R(a,b)=map((x,y,z)↦z,filter((x,y,z)↦x=a and y=b,R)). Application returns ok(c) when M_R(a,b)=[c], and the original error when the list is empty or has at least two entries. Constructor closure supplies carrier membership for every output of a matching retained row. The next section derives singleton matching from the constructor's actual coverage and length checks.

> Source, proof and provenance: V3 __post_init__, sft/mathematics/algebraic_structures/law.py:18; V3 apply, sft/mathematics/algebraic_structures/law.py:28. Native results: SFT.Mathematics.AlgebraOperations.singleMatchApplied, AlgebraOperations.lean:96; SFT.Mathematics.AlgebraOperations.noMatchRejected, AlgebraOperations.lean:101. Timeline: Turn 077, line 1108. Evidence: Completion TURN_118_SFT-MATH-ALGEBRA-001_COMPLETION.

37. Complete single-valued operation support

The constructor's input coverage and row cardinality jointly force the projected input rows to be a permutation of the complete ordered pair support. This follows constructively for arbitrary finite size: locate the first support element in the proposed rows, remove that occurrence, and retain coverage and equal length for the remaining support. Since the carrier is distinct, the complete pair support is distinct, hence so is the projected row-input list.

Consequently, for every pair of carrier members the actual matching-row filter contains exactly one row. The translated apply operation returns its result, and the constructor's closure check proves that result remains in the registered carrier. No uniqueness assumption is added to the source guards; it is derived from their original coverage and cardinality conditions.

The finite coverage lemma is: if a list P has distinct members, every member of P occurs in Q, and |P|=|Q|, then P∼Q. For P=[], length equality forces Q=[]. For P=x::P′, coverage splits Q=B++x::D. Distinctness gives x∉P′. Each y∈P′ still occurs in B++D: its supplied occurrence in Q cannot be the removed x. Cancelling the one removed position from the length equality gives |P′|=|B++D|. Induction supplies P′∼B++D, and restoring x and moving it across B gives x::P′∼Q.

For distinct C, P(C) is distinct: within each fixed first coordinate, (a,b)=(a,b′) implies b=b′; between distinct first coordinates, pair equality is impossible. The constructor's coverage and count conditions therefore instantiate the lemma to give

$$P(C) \sim K(R), \text{ hence } \operatorname{distinct}(K(R))$$

For a,b∈C, the pair (a,b) belongs to P(C) and thus K(R). Filter R by this pair. Existence guarantees a matching row. Projected-input distinctness excludes every second matching row, so the actual filter is [(a,b,c)] for one retained row. Mapping to results gives M_R(a,b)=[c]. Application consequently returns ok(c), and the original closure check gives c∈C. Totality and single-valuedness follow from the original guards for arbitrary admitted finite carrier size.

> Source, proof and provenance: V3 __post_init__, sft/mathematics/algebraic_structures/law.py:18; V3 apply, sft/mathematics/algebraic_structures/law.py:28. Native results: SFT.Mathematics.AlgebraOperations.inputTestIff, AlgebraTableCoverage.lean:159; SFT.Mathematics.AlgebraOperations.acceptedLookupTotal, AlgebraTableCoverage.lean:178. Timeline: Turn 078, line 1122. Evidence: Completion TURN_118_SFT-MATH-ALGEBRA-001_COMPLETION.

38. Exhaustive identity and association

The identity test first checks that the proposed identity belongs to the carrier, then evaluates left and right preservation for each carrier member in source order. Its successful result is equivalent to both applications returning that member for every carrier member. The conjunction preserves source short-circuiting and lookup exceptions.

The associativity test traverses every ordered carrier triple. For each triple it evaluates the left composite before the right composite in the original order. On accepted operation tables, successful exhaustive checking is equivalent to equality of the two actual composed results for every triple. Totality and closure of the constructor supply the intermediate operation results, so no associativity law is assumed. Fresh execution agrees on 156 property calls over 39 table representations, including every two-member binary operation in two orders and malformed diagnostic representations.

Let μ(a,b) denote the successful result of applyOperation on an accepted table with carrier C. Section 37 establishes μ(a,b)∈C for a,b∈C. The checked traversal has the equations allChecked(p,[])=ok(true); on x::X it evaluates p(x), continues on X only after ok(true), returns ok(false) after false, and propagates an error. Induction on X proves

$$\mathrm{allChecked}(p, X) = ok(\mathrm{true}) \iff \forall x \in X, p(x) = ok(\mathrm{true})$$

For an identity cell, success of the left comparison forces μ(e,a)=a; only then is the right comparison evaluated, and success forces μ(a,e)=a. Combining this equivalence with the initial membership guard and traversal gives

$$\mathrm{hasIdentity}(e) = ok(\mathrm{true}) \iff e \in C \text{ and } \forall a \in C, \mu(e,a) = a = \mu(a,e)$$

For association, the evaluated sequence is u=μ(a,b), l=μ(u,c), v=μ(b,c), r=μ(a,v). Closure supplies u,v∈C, so all four lookups exist. The cell is true exactly when l=r. Applying the traversal equivalence at each of the three nested carrier lists yields

$$\mathrm{isAssociative} = ok(\mathrm{true}) \iff \forall a, b, c \in C, \mu(\mu(a,b),c) = \mu(a,\mu(b,c))$$

This derives the meaning of exhaustive checking; it does not assert that every accepted table is associative.

> Source, proof and provenance: V3 has_identity, sft/mathematics/algebraic_structures/law.py:35; V3 is_associative, sft/mathematics/algebraic_structures/law.py:42. Native results: SFT.Mathematics.AlgebraOperations.hasIdentityTrueIff, AlgebraProperties.lean:88; SFT.Mathematics.AlgebraOperations.acceptedAssociativityMeaning, AlgebraProperties.lean:135. Timeline: Turn 079, line 1134. Evidence: Completion TURN_118_SFT-MATH-ALGEBRA-001_COMPLETION.

39. Held return mates and commutation

The return-mate operation retains the source identity precheck. For each carrier member it exhausts candidate mates in carrier order and tests return to identity on both sides. Exactly one matching candidate is required; zero or multiple matches return None, while lookup exceptions propagate. Successful candidate filtering retains precisely those carrier members satisfying both return equations. Successful mate-row construction preserves the carrier order and provides a singleton candidate search for each returned row.

Commutation is evaluated on every ordered carrier pair, comparing both operation results with the original left-before-right evaluation. Its successful result is equivalent to all pair comparisons passing, and each comparison is equivalent to equality of the actual results. Fresh execution matches 156 return-mate and commutation calls across 39 table representations, covering all two-member operations in two orders and malformed diagnostics.

For a proposed identity e and an item a, define the successful candidate list K_e(a) by checked filtering of C. A candidate b is retained precisely when both ordered operations return e. Induction on the filter gives

$$b \in K_e(a) \iff b \in C \text{ and } \mathrm{applyOperation}(a,b) = ok(e) \text{ and } \mathrm{applyOperation}(b,a) = ok(e)$$

The recursion retains carrier order. Its empty base returns []; each successful head test either prepends that head or omits it, and any evaluated error propagates. Return-row construction accepts the item only if K_e(a)=[b]. With successful later rows R it prepends (a,b), so projection onto first coordinates reproduces the original item list by induction. An empty or multi-element candidate list returns None at that step.

For commutation, evaluate l=μ(a,b) and then r=μ(b,a). The cell returns true exactly when l=r. Applying the checked-traversal equivalence twice gives exhaustive agreement of all ordered pair cells. The candidate and commutation calculations use the retained table operation; neither replaces it with a named algebraic

law.

> Source, proof and provenance: V3 return_mates, sft/mathematics/algebraic_structures/law.py:49; V3 is_commutative, sft/mathematics/algebraic_structures/law.py:64. Native results: SFT.Mathematics.AlgebraOperations.commutationExhaustiveIff, AlgebraReturns.lean:182; SFT.Mathematics.AlgebraOperations.commutationCellExact, AlgebraReturns.lean:191. Timeline: Turn 080, line 1147. Evidence: Completion TURN_118_SFT-MATH-ALGEBRA-001_COMPLETION.

40. Public return and commutation semantics

The return-mate API produces a successful row list if and only if its identity test succeeds and every carrier member has an exhaustive candidate search containing exactly one mate. Successful output has the same carrier projection in the same order. Every returned mate belongs to the carrier, returns to identity on both sides, and is the only carrier member satisfying both equations for that item.

For accepted operation tables, the commutation test succeeds if and only if the actual operation outputs agree for every ordered pair and its reversal. The constructor's totality supplies both outputs in the converse direction, so the equivalence does not rely on vacuous comparisons of missing operations.

Let Success(e) mean that returnMates produces ok(Some(R)) for some row list R. Its initial branch requires hasIdentity(e)=ok(true). Row recursion then gives

$$\mathrm{Success}(e) \iff \mathrm{hasIdentity}(e) = ok(\mathrm{true}) \text{ and } \forall a \in C, \exists b, \mathrm{returnCandidates}(e, a) = ok([b])$$

For the forward implication, the successful row projection contains every original a, and its corresponding row supplies the singleton search. For the reverse implication, take the mate supplied for the head, construct later rows by induction on the tail, and prepend the resulting pair. The empty tail closes with Some([]).

For a returned (a,b), singleton membership and the candidate-filter equivalence give b∈C and μ(a,b)=e=μ(b,a). Any other c∈C satisfying both equations belongs to the same singleton [b], hence c=b. Thus uniqueness follows from exhaustive singleton filtering. Similarly, on an accepted table, total lookup supplies μ(a,b) and μ(b,a) for every pair. Substituting these values into the cell equivalence proves

$$\mathrm{isCommutative} = ok(\mathrm{true}) \iff \forall a, b \in C, \mu(a, b) = \mu(b, a)$$

Totality is used in the reverse implication to construct the successful comparisons, not merely to rule out counterexamples.

> Source, proof and provenance: V3 return_mates, sft/mathematics/algebraic_structures/law.py:49; V3 is_commutative, sft/mathematics/algebraic_structures/law.py:64. Native results: SFT.Mathematics.AlgebraOperations.returnedMateUniqueMeaning, AlgebraReturnMeaning.lean:57; SFT.Mathematics.AlgebraOperations.acceptedCommutationMeaning, AlgebraReturnMeaning.lean:77. Timeline: Turn 081, line 1160. Evidence: Completion TURN_118_SFT-MATH-ALGEBRA-001_COMPLETION.

41. Operation-preserving maps

The mapping relation is converted to the source dictionary semantics: the first occurrence fixes a key's position, and subsequent occurrences replace its value. Native proofs establish replacement lookup, preservation of other lookups and the last occurrence's value. The preservation operation retains the source sequence of checks: dictionary cardinality, presence of every source origin with a target-carrier image, then the operation equation for every ordered source pair.

Each equation compares the image of the source operation result with the target operation applied to the two input images, in the source evaluation order. Successful preservation is equivalent to all original checks passing; each successful cell comparison is equivalent to equality of its actual results. Fresh

execution agrees on 1555 complete dictionary comparisons and 6220 preservation calls across repeated-key, extra-key and external-image mappings. Dictionary invariants and accepted-table totality are established below.

Write put(D,a,v) for the source dictionary update and lookup(D,a) for its optional lookup. Structural induction on D gives

$$\mathrm{lookup}(\mathrm{put}(D,a,v),a) = \mathrm{Some}(v)$$
$$\mathrm{lookup}(\mathrm{put}(D,a,v),b) = \mathrm{lookup}(D,b) \text{ for } b \neq a$$

At a matching key, replacement leaves its position unchanged; if no key matches, recursion appends the new pair. Processing concatenated mapping lists satisfies run(D,U++V)=run(run(D,U),V). Therefore a final occurrence (a,v) fixes lookup(build(U++[(a,v)]),a)=Some(v), preserving last-value semantics rather than rejecting repeated keys.

Let D be the built dictionary and f its lookup value where defined. The first two preservation guards require |D|=|C_source| and, for every a∈C_source, a defined f(a)∈C_target. For each ordered source pair, the cell evaluates in order

$$u = \mu_{\mathrm{source}}(a,b), f(u), f(a), f(b), v = \mu_{\mathrm{target}}(f(a),f(b))$$

and then compares f(u) with v. Given these successful lookups, the cell is true exactly when

$$f(\mu_{\mathrm{source}}(a,b)) = \mu_{\mathrm{target}}(f(a),f(b))$$

The two nested checked traversals turn successful preservation into the conjunction of the original cardinality guard, image-membership guard and this equality at every source pair. Missing dictionary entries and operation failures retain their source error path and evaluation order.

> Source, proof and provenance: V3 preserves_operation, sft/mathematics/algebraic_structures/law.py:68. Native results: SFT.Mathematics.AlgebraOperations.preservationCellExact, AlgebraMaps.lean:90; SFT.Mathematics.AlgebraOperations.preservesOperationTrueIff, AlgebraMaps.lean:109. Timeline: Turn 082, line 1172. Evidence: Completion TURN_118_SFT-MATH-ALGEBRA-001_COMPLETION.

42. Total structure-map semantics

Dictionary updates preserve distinct keys, and the complete dictionary construction therefore has one entry per retained origin. Lookup succeeds exactly at a stored key. The source image-carrier check is equivalent to successful lookup of an image in the target carrier.

For accepted source and target operation tables, those image checks supply the mapped inputs and the image of every source operation result; constructor closure supplies membership of the intermediate source result. Each preservation comparison consequently terminates with a Boolean result rather than a missing-key or operation error. The public preservation operation succeeds exactly when its original size and image checks hold and the image of every source operation result equals the corresponding target operation result.

Let keys(D) project dictionary origins. If its keys are distinct, put(D,a,v) either replaces an existing value without changing keys or appends a previously absent key. Induction over the input relation therefore gives distinct(keys(build(mapping))). Lookup induction gives

$$(\exists v, \mathrm{lookup}(D,a) = \mathrm{Some}(v)) \iff a \in \mathrm{keys}(D)$$

Unfolding the image guard gives

$$\mathrm{imageInCarrier}(D,T,a) = \mathrm{true} \iff \exists v \in T, \mathrm{imageAt}(D,a) = ok(v)$$

For a,b∈C_source, accepted-table totality produces u=μ_source(a,b)∈C_source. The image guard supplies f(u),f(a),f(b), with f(a),f(b)∈C_target. Target totality then produces μ_target(f(a),f(b)). Consequently every checked cell terminates with the Boolean comparison f(u)=μ_target(f(a),f(b)). Substitution in the exhaustive pair traversal yields the exact public equivalence

$$\begin{gathered}\mathrm{preservesOperation} = ok(\mathrm{true}) \iff \\ |D| = |C_{\mathrm{source}}| \text{ and } (\forall a \in C_{\mathrm{source}}, f(a) \text{ exists in } C_{\mathrm{target}}) \\ \text{and } (\forall a, b \in C_{\mathrm{source}}, f(\mu_{\mathrm{source}}(a,b)) = \mu_{\mathrm{target}}(f(a), f(b)))\end{gathered}$$

The dictionary and table facts supply each value used in this equality.

> Source, proof and provenance: V3 preserves_operation, sft/mathematics/algebraic_structures/law.py:68. Native results: SFT.Mathematics.AlgebraOperations.checkedPreservationCellTotal, AlgebraMapMeaning.lean:84; SFT.Mathematics.AlgebraOperations.acceptedPreservationMeaning, AlgebraMapMeaning.lean:111. Timeline: Turn 083, line 1185. Evidence: Completion TURN_118_SFT-MATH-ALGEBRA-001_COMPLETION.

43. Original algebra witnesses and complete grammar

Both original operation tables are retained and accepted by the translated constructor. The six registered witnesses are proved from their actual computations: complete four-row coverage, structural One identity, exhaustive association, exact two-sided return mates, commutation of the symmetric table together with failure of commutation for the lawful left-projection table, and preservation by the canonical self-map. All original names and statements are retained.

The algebra grammar contains all 512 products of its nine source dimensions. Native proofs establish exhaustive membership, exact cardinality, the unique all-preserving survivor, equivalence of acceptance to every original criterion, and rejection of each inadmissible coordinate. Fresh execution matches every candidate record’s order, identifier, survivor decision, exact form and reason against the pinned source artifacts. These finite grammar proofs accompany the general operation derivations and complete-program correspondence. The algebra entry is recorded complete against its registered V3 source.

Abbreviate the witness labels One and held by O and H. Its four ordered cells are

$$O \cdot O = O, O \cdot H = H, H \cdot O = H, H \cdot H = O$$

These cover all pairs and stay in {O,H}; O preserves both labels. In the ordered triple sequence OOO, OOH, OHO, OHH, HOO, HOH, HHO, HHH, both parenthesizations evaluate respectively to

$$O, H, H, O, H, O, O, H$$

Thus association is checked on all eight triples. The mate searches give K_O(O)=[O] and K_O(H)=[H], producing [(O,O),(H,H)]. The table is symmetric. By contrast, the second lawful table has a·a=a, a·b=a, b·a=b, b·b=b; for its distinct labels a·b=a≠b=b·a, so commutation fails. The self-map f(O)=O,f(H)=H satisfies f(x·y)=f(x)·f(y) on every cell by direct substitution. These computations establish the six original witness predicates.

The nine binary grammar axes generate 2^9=512 ordered tuples. Acceptance is equivalent to the conjunction of the nine admitted-coordinate conditions. The all-preserving tuple satisfies that conjunction; any accepted tuple equals it coordinate by coordinate. A rejected value falsifies its coordinate condition, establishing the exclusion proofs while retaining the full product enumeration.

> Source, proof and provenance: V3 has_identity, sft/mathematics/algebraic_structures/law.py:35; V3 return_mates, sft/mathematics/algebraic_structures/law.py:49. Native results: SFT.Mathematics.AlgebraWitnesses.structureMapPassed, AlgebraWitnesses.lean:65; SFT.Mathematics.AlgebraWitnesses.allWitnessesPass, AlgebraWitnesses.lean:69. Timeline: Turn 084, line 1197. Evidence: Completion TURN_118_SFT-MATH-ALGEBRA-001_COMPLETION.

44. Algebra program correspondence

The translated algebra program retains its original registration and dependencies, all 512 candidate records and lookup decisions, completeness certificate, closure and generality payloads, and four original controls. Candidate lookup depends only on the registered identifier, and an unknown identifier retains the source StopIteration outcome. Fresh native execution reproduces the pinned source artifacts, including every census trace identity, decision proof identity, closure and generality certificate, control receipt and registration field. These correspondence results accompany the general operation, return and map derivations above. The registered algebra entry and its dependencies are complete in the coverage ledger.

The original program constructs each candidate from the ordered Cartesian product of its declared choice lists, joins its coordinate names to form the identifier, and retains the coordinate/form record. Decision lookup first finds that exact identifier in the complete product, then compares it with the joined all-admitted identifier. A missing identifier therefore fails lookup. The reason scan stops at the first rejected coordinate in declared order. Census hashes retain the complete candidate record; closure hashes retain the closure record and ordered decisions; each control retains its own observed result and receipt hash. Registration uses the same specification and source identity. These are the particular program operations compared with their native records.

> Source, proof and provenance: V3 has_identity, sft/mathematics/algebraic_structures/law.py:35. Native results: SFT.Mathematics.AlgebraEvidence.registrationHasNoFreeParameters, AlgebraEvidence.lean:83; SFT.Mathematics.AlgebraEvidence.lookupDependsOnlyOnIdentifier, AlgebraEvidence.lean:86. Timeline: Turn 085, line 1210. Evidence: Completion TURN_118_SFT-MATH-ALGEBRA-001_COMPLETION.

Program correspondence sources and proofs: V3 candidate_records, mathematics/generated_law.py:L117; V3 registration, mathematics/generated_law.py:L230; V3 generate_candidates, mathematics/generated_law.py:L245; V3 decide_candidate, mathematics/generated_law.py:L264; V3 closure_evidence, mathematics/generated_law.py:L285; V3 control_records, mathematics/generated_law.py:L187; AlgebraGrammar.generatedCardinality, L127; AlgebraGrammar.candidatesComplete, L129; AlgebraGrammar.acceptanceIffExactCriteria, L184; AlgebraEvidence.registrationData, L17; AlgebraEvidence.censusData, L37; AlgebraEvidence.decisionData, L46; AlgebraEvidence.closureData, L61.

45. Algebraic induction base and carrier extension

The One carrier has exactly its self pair. Its closed operation returns One, and both identity and association hold. More generally, closure and complete pair coverage force the sole result of any accepted singleton carrier to be its member; identity and association follow for every such accepted table.

Appending a fresh member retains the original nested-loop order of pair generation. Each existing left-member row gains its cell with the fresh right member, followed by the fresh left-member row and the fresh self cell. The enlarged support partitions into old pairs, old-to-fresh pairs, fresh-to-old pairs and the self pair. Exactly the last three classes are new, and a distinct original carrier with a fresh added member generates a distinct complete pair support. These results establish the base and pair-generation component of the original algebra induction; the property checks on an extended operation remain explicit obligations.

For a singleton carrier [o], pair support is [(o,o)]. Accepted-table totality and closure give $\mu(o,o)\in[o]$, forcing $\mu(o,o)=o$. Hence $o\cdot o=o$ establishes both identity equations, and both association branches reduce to $\mu(\mu(o,o),o)=\mu(o,\mu(o,o))=o$.

Let $z\notin C$. Distributing the ordered pair generator over $C++[z]$ gives the exact list equation

$$P(C +\!\!+ [z]) =$$
$$concat_{a \in C}([(a,b) : b \in C] +\!\!+ [(a,z)])$$
$$+\!\!+ [(z,b) : b \in C] +\!\!+ [(z,z)]$$

This preserves the interleaving of each old row with its new final cell. Pair membership consequently separates into four cases:

$$(a \in C \text{ and } b \in C), (a \in C \text{ and } b = z)$$
$$(a = z \text{ and } b \in C), \text{ or } (a = z \text{ and } b = z)$$

Freshness makes these cases disjoint. Excluding membership in P(C) leaves exactly the final three cases, so no new pair is omitted and no old pair is reclassified as new. Distinctness of C and freshness of z give distinct(C++[z]); applying pair-product distinctness proves the enlarged support has no duplicate pairs. This constructs the complete extension support; the chosen outputs and their property checks are handled by the subsequent extension results.

> Source, proof and provenance: V3 __post_init__, sft/mathematics/algebraic_structures/law.py:18. Native results: SFT.Mathematics.AlgebraOperations.acceptedSingletonAssociation, AlgebraInduction.lean:139; SFT.Mathematics.AlgebraOperations.newPairSupportExactly, AlgebraInduction.lean:155. Timeline: Turn 086, line 1222. Evidence: Completion TURN_118_SFT-MATH-ALGEBRA-001_COMPLETION.

46. Identity and association under carrier extension

The exhaustive check on an appended domain evaluates the original prefix first. A false result or exception stops that evaluation; only a successful prefix proceeds to the added domain. Identity on an enlarged carrier requires preservation of each old member and of the added member on both sides. When the old identity holds and all old operation cells are preserved, the enlarged identity holds exactly when its two equations on the added member hold.

Every enlarged-carrier triple either consists entirely of old members or contains the added member in at least one position. For an accepted old table, closure keeps both intermediate old pair results in the old carrier. Preserving old operation cells therefore preserves the full composed old triple calculations. If the old table is associative, the enlarged table is associative exactly when every triple containing the added member passes its original association check. All seven new-coordinate combinations are included; no enumeration or scientific guard is removed.

Write the old carrier list as C, the appended label as z, and C' = C ++ [z]. Write μ and μ' for the old and extended operations, with μ'(a,b) = μ(a,b) for a,b ∈ C. Each equality here denotes successful lookup as well as equality of the resulting label. The exhaustive Boolean check A satisfies

$$A(P, C +\!\!+ [z]) = \begin{cases} \text{error}(e), & A(P,C) = \text{error}(e), \\ \text{ok}(\text{false}), & A(P,C) = \text{ok}(\text{false}), \\ P(z), & A(P,C) = \text{ok}(\text{true}). \end{cases}$$

If e is an old two-sided identity, preservation of old cells gives μ'(e,a) = a = μ'(a,e) for every a ∈ C. Thus

$$e \text{ is an identity on } C' \iff \mu'(e,z) = z \ \wedge \ \mu'(z,e) = z.$$

For old a,b,c, closure gives u = μ(a,b) ∈ C and v = μ(b,c) ∈ C. Consequently the intermediate and outer lookups are all preserved:

$$\mu'(\mu'(a,b),c) = \mu'(u,c) = \mu(u,c), \qquad \mu'(a,\mu'(b,c)) = \mu'(a,v) = \mu(a,v).$$

The old associativity equality therefore transfers. Expanding each coordinate of C' gives the old block C×C×C and seven remaining blocks:

$$(z,C,C),\ (C,z,C),\ (C,C,z),\ (z,z,C),\ (z,C,z),\ (C,z,z),\ (z,z,z).$$

Here each displayed triple denotes the corresponding Cartesian block of labels, retaining list enumeration. Under old associativity and closure, associativity on C' is therefore equivalent to

$$\forall a,b,c \in C', \quad (a = z \vee b = z \vee c = z) \Longrightarrow \mu'(\mu'(a,b),c) = \mu'(a,\mu'(b,c)).$$

Every required lookup must succeed; an error is not an equality witness.

> Source, proof and provenance: V3 has_identity, sft/mathematics/algebraic_structures/law.py:35; V3 is_associative, sft/mathematics/algebraic_structures/law.py:42. Native results: SFT.Mathematics.AlgebraOperations.oldAssociationCellPreserved, AlgebraExtension.lean:121; SFT.Mathematics.AlgebraOperations.associationPreservedByExtension, AlgebraExtension.lean:141. Timeline: Turn 087, line 1234. Evidence: Completion TURN_118_SFT-MATH-ALGEBRA-001_COMPLETION.

47. Return mates under carrier extension

The exhaustive mate filter preserves carrier order and propagates lookup errors. With old operation cells retained, each old member’s enlarged search consists of its original candidate search followed by the added candidate’s two-sided return test. An originally unique mate remains the only mate precisely when that last test returns false. A passing added candidate creates a second match, and an error propagates.

Given prior successful return mates and preservation of old cells, successful return mates on the enlarged carrier are equivalent to three obligations: both identity equations hold for the added member; its return test fails for every old member; and its own exhaustive search over the enlarged carrier has exactly one mate. The original identity precheck and singleton requirement remain intact.

Define R_e(a,b) as the checked predicate μ(a,b) = e and μ(b,a) = e, and let F_C(a) be the ordered list obtained by filtering C with this predicate. Primes denote the extended operation and carrier. For a ∈ C, preservation of old cells gives R'_e(a,b) = R_e(a,b) for every old b. Hence, after a successful old search,

$$F_{C'}(a) = \begin{cases} F_C(a) \mathbin{+\!\!+} [z], & R'_e(a,z) = \mathrm{ok}(\mathrm{true}), \\ F_C(a), & R'_e(a,z) = \mathrm{ok}(\mathrm{false}), \\ \mathrm{error}(u), & R'_e(a,z) = \mathrm{error}(u). \end{cases}$$

An old search error propagates before the appended test is reached. If F_C(a) = [m_a], the successful alternatives have lengths 2 and 1 respectively. Therefore

$$\exists m\ F_{C'}(a) = [m] \iff R'_e(a,z) = \mathrm{ok}(\mathrm{false}),$$

and in the successful case the retained mate is exactly m_a. This is a singleton-list condition, preserving the original enumeration and multiplicities. Combining this equation for every old a with the identity result of section 46 yields

$$\begin{aligned} \mathrm{ReturnSuccess}(C',e) \iff\ & [\mu'(e,z) = z \wedge \mu'(z,e) = z] \\ & \wedge [\forall a \in C,\ R'_e(a,z) = \mathrm{ok}(\mathrm{false})] \\ & \wedge [\exists m,\ F_{C'}(z) = [m]]. \end{aligned}$$

This equivalence assumes prior successful return mates and preserved old cells. The last search still examines all of C', including the appended candidate.

> Source, proof and provenance: V3 return_mates, sft/mathematics/algebraic_structures/law.py:49. Native results: SFT.Mathematics.AlgebraOperations.oldSearchUniqueIffFreshFails, AlgebraReturnExtension.lean:113; SFT.Mathematics.AlgebraOperations.returnMatesPreservedByExtension, AlgebraReturnExtension.lean:134. Timeline:

> Turn 088, line 1246. Evidence: Completion TURN_118_SFT-MATH-ALGEBRA-001_COMPLETION.

48. Operation maps under carrier extension

The original map-admission checks split into dictionary cardinality, images for old members and the added member, and operation preservation for old and newly formed source pairs. All left, right and self pairs involving the added member remain required.

When old source and target operation cells and old dictionary images are retained, source closure ensures that the old operation result also retains its image. Consequently the complete old preservation-cell calculation is unchanged. Old image coverage transfers when the enlarged target retains its old carrier. Given prior map success and these preservation conditions, enlarged map success is equivalent to the original dictionary-size guard, target membership of the added member's image, and every new source-pair preservation check.

Let C,D be the old source and target carriers, μ,ν their operations, and f the dictionary produced by the original mapping constructor. The extended objects are C' = C ++ [z], D', μ',ν',f'. The admission condition is

$$|f'| = |C'|, \qquad \forall a \in C',\ f'(a) \in D', \qquad \forall a, b \in C',\ f'(\mu'(a,b)) = \nu'(f'(a), f'(b)).$$

The size is the number of entries in the constructed dictionary, not the length of an unprocessed mapping input. All images and operation lookups in the displayed equations must exist. Suppose the old table was accepted, the old map passed, D ⊆ D', and the extended operations and images agree on their old domains. For a,b ∈ C, closure places r = μ(a,b) in C. Thus

$$\begin{aligned} f'(\mu'(a,b)) &= f'(r) = f(r) = f(\mu(a,b)), \\ \nu'(f'(a), f'(b)) &= \nu'(f(a), f(b)) = \nu(f(a), f(b)). \end{aligned}$$

The old map equality identifies the two final expressions, so all old pair checks transfer. Likewise f'(a) = f(a) ∈ D ⊆ D' transfers old image coverage. Splitting C'×C' leaves the old block and (C,z), (z,C), (z,z). Consequently the remaining necessary and sufficient checks are

$$\begin{aligned} &|f'| = |C'|, \qquad f'(z) \in D', \\ &\forall a, b \in C',\ (a = z \vee b = z) \Longrightarrow f'(\mu'(a,b)) = \nu'(f'(a), f'(b)). \end{aligned}$$

This reduction preserves the original size guard, image coverage and every newly formed pair; it follows by transferring the old checks, not by dropping them.

> Source, proof and provenance: V3 preserves_operation, sft/mathematics/algebraic_structures/law.py:68. Native results: SFT.Mathematics.AlgebraOperations.oldPreservationCellPreserved, AlgebraMapExtension.lean:71; SFT.Mathematics.AlgebraOperations.operationMapPreservedByExtension, AlgebraMapExtension.lean:100. Timeline: Turn 089, line 1258. Evidence: Completion TURN_118_SFT-MATH-ALGEBRA-001_COMPLETION.

49. Concrete dictionary extension

For a successful operation map from an accepted source table, the generated dictionary keys are a permutation of the source carrier. A fresh source member therefore has no existing dictionary entry. The original update operation appends its key and image at the end, increasing cardinality by one and preserving every old lookup. The fresh lookup returns its assigned image.

These facts discharge the dictionary-size and old-image preservation conditions for the concrete appended mapping. With the old source and target operations retained and the old target carrier included, that mapping preserves the enlarged operation exactly when the fresh image belongs to the enlarged target and every new source pair passes the operation equation. The dictionary's original first-key order and last-value replacement behavior are retained.

Let I = buildImages(M) for the original mapping list M. Successful map admission gives |I| = |C| and a defined image for every a ∈ C. Since the accepted carrier is distinct, coverage and equal length imply

$$\mathrm{keys}(I) \sim C,$$

where ~ denotes list permutation. For z ∉ C, membership preservation under permutation gives z ∉ keys(I), hence lookup(I,z) = none. The original update therefore takes its absent-key branch:

$$I' = \mathrm{buildImages}(M +\!\!+ [(z,v)]) = I +\!\!+ [(z,v)], \qquad |I'| = |I| + 1 = |C| + 1 = |C +\!\!+ [z]|.$$

For every a ∈ C, freshness implies a ≠ z, so lookup(I',a) = lookup(I,a); meanwhile lookup(I',z) = some(v). Substitution into section 48 discharges the size and old-image obligations and gives

$$\mathrm{MapSuccess}(M +\!\!+ [(z,v)]) \iff v \in D' \ \wedge\ \forall a, b \in C',\ (a = z \vee b = z) \Longrightarrow f'(\mu'(a,b)) = \nu'(f'(a), f'(b)).$$

This uses the accepted old source, successful old map, preserved old source and target cells, and inclusion D ⊆ D'. Repeated keys elsewhere retain the original replacement rule; freshness is used only for this appended key.

> Source, proof and provenance: V3 preserves_operation, sft/mathematics/algebraic_structures/law.py:68. Native results: SFT.Mathematics.AlgebraOperations.appendedFreshImageValidIff, AlgebraDictionaryExtension.lean:82; SFT.Mathematics.AlgebraOperations.freshDictionaryMapExtensionIff, AlgebraDictionaryExtension.lean:94. Timeline: Turn 090, line 1269. Evidence: Completion TURN_118_SFT-MATH-ALGEBRA-001_COMPLETION.

50. Finite order and bound operations

The order kernel retains the original nonempty distinct carrier and distinct relation checks, rejects external endpoints, and exhaustively evaluates reflexivity, antisymmetry and transitivity. Totality requires both partial-order success and comparability of every generated carrier pair. The latter equivalence is proved natively.

Lower and upper bounds retain the nonempty held-selection guard and reject external held members. Successful results contain exactly the carrier members satisfying every held comparison, in original carrier order. Greatest and least witnesses are selected only when the complete matching list has exactly one entry. Meet and join retain the bound-selection errors and conditional extremal result. Monotonicity retains dictionary replacement semantics and checks both endpoint images and every source comparison.

Fresh execution matches 1404 order, bound and extremal records and 13932 monotonicity comparisons, covering all two-member relations together with malformed carriers/relations, repeated and outside selections, and repeated-key mappings. These execution results accompany the general partial-order and extremal proofs. The complete 512-candidate program comparison also matches the pinned V3 registration, census, decisions, closure, controls and unknown-identifier behavior.

Write a ≤_R b for (a,b) ∈ R. For a held selection H with H ≠ [] and every member in C, the actual bound lists are the ordered filters

$$L_R(H) = [x \in C \mid \forall h \in H,\ x \le_R h], \qquad U_R(H) = [x \in C \mid \forall h \in H,\ h \le_R x].$$

The bracket notation denotes list filtering, retaining carrier order and multiplicity. Their membership equivalences follow directly from filter membership and the finite all-test. For a bound list B, form the complete extremal lists

$$G_R(B) = [g \in B \mid \forall b \in B,\ b \le_R g], \qquad E_R(B) = [e \in B \mid \forall b \in B,\ e \le_R b].$$

The greatest selector returns some(g) exactly when G_R(B) = [g]; the least selector similarly requires E_R(B) = [e]. Thus, after successful bound selection,

$$\mathrm{meet}(C,R,H) = \mathrm{ok}(\mathrm{some}(m)) \iff G_R(L_R(H)) = [m], \qquad \mathrm{join}(C,R,H) = \mathrm{ok}(\mathrm{some}(j)) \iff E_R(U_R(H)) = [j].$$

An empty or outside held selection returns the original error before extremal selection. Total order is also conditional:

$$\mathrm{Total}(C,R) \iff \mathrm{Partial}(C,R) \wedge \forall a,b \in C,\ (a \le_R b \vee b \le_R a).$$

For a monotone map f from source relation R to target relation S, the comparison condition is (a,b) ∈ R ⇒ (f(a),f(b)) ∈ S, with the original dictionary and endpoint-image admission checks retained. No comparison absent from R is supplied by the map check.

> Source, proof and provenance: V3 lower_bounds, sft/mathematics/order_lattice/law.py:33; V3 upper_bounds, sft/mathematics/order_lattice/law.py:39. Native results: SFT.Mathematics.OrderOperations.emptyLowerSelectionRejected, OrderOperations.lean:137; SFT.Mathematics.OrderOperations.emptyUpperSelectionRejected, OrderOperations.lean:140. Timeline: Turn 091, line 1281. Evidence: Completion TURN_118_SFT-MATH-ORDER-LATTICE-001_COMPLETION.

Exact operation and proof anchors: meet, OrderOperations.lean:L45; join, OrderOperations.lean:L49; lowerBoundsMembership, OrderOperations.lean:L113; upperBoundsMembership, OrderOperations.lean:L125.

51. Exact partial-order admission

For arbitrary represented finite carrier and relation lists, the partial-order check succeeds if and only if the carrier is nonempty and distinct, the relation is distinct, every relation endpoint belongs to the carrier, every carrier member has its reflexive cell, every pair of reversed relation cells has equal endpoints, and every pair of composable relation cells has its composite in the relation.

The native equivalence follows each original guard and exhaustive traversal. The transitivity condition covers all pairs of relation cells with matching middle endpoints; nonmatching cells impose no composite requirement. The reverse implication establishes success from precisely these same conditions.

With Distinct denoting absence of repeated list entries, the complete admission equivalence is

$$\begin{aligned}\mathrm{Partial}(C,R) \iff\ & C \neq [] \wedge \mathrm{Distinct}(C) \wedge \mathrm{Distinct}(R) \\ & \wedge [\forall (a,b) \in R,\ a \in C \wedge b \in C] \\ & \wedge [\forall a \in C,\ (a,a) \in R] \\ & \wedge [\forall a,b,\ (a,b) \in R \wedge (b,a) \in R \Rightarrow a = b] \\ & \wedge [\forall a,b,c,\ (a,b) \in R \wedge (b,c) \in R \Rightarrow (a,c) \in R].\end{aligned}$$

For each finite traversal the elementary equivalence is

$$\mathrm{all}(P,[x_1,\ldots,x_n]) = \mathrm{true} \iff \bigwedge_{i=1}^{n} P(x_i) = \mathrm{true}.$$

Applying it to carrier cells proves the reflexive clause. Applying it twice to relation cells proves antisymmetry and transitivity. In the latter traversal, a pair of cells (a,b),(u,c) passes automatically when b ≠ u; when b = u, its check is exactly (a,c) ∈ R. Therefore every composable pair is tested, and the displayed transitive clause supplies precisely the checks needed for the reverse implication. Combining these traversal equivalences with the three initial guards and endpoint check proves both directions of admission.

> Source, proof and provenance: V3 is_partial_order, sft/mathematics/order_lattice/law.py:12. Native results: SFT.Mathematics.OrderOperations.transitiveCheckIff, OrderMeaning.lean:93; SFT.Mathematics.OrderOperations.partialOrderMeaning, OrderMeaning.lean:115. Timeline: Turn 092, line 1293. Evidence: Completion TURN_118_SFT-MATH-ORDER-LATTICE-001_COMPLETION.

52. Extremal witnesses and returned bounds

Greatest and least operations return a value exactly when the complete filtered list of matching candidates is the singleton containing that value. This characterization retains occurrence multiplicity on unrestricted input lists. Every returned value belongs to the supplied bounds, dominates them in the appropriate direction, and is the only bound with that property. For distinct bounds, those semantic conditions also imply successful return.

Meet succeeds exactly when lower-bound generation succeeds and its complete result has the returned unique greatest witness; join has the corresponding upper-bound and unique least criterion. Every returned meet belongs to the carrier, lies below all held members, and is above every carrier lower bound. Every returned join belongs to the carrier, lies above all held members, and is below every carrier upper bound. Bound-generation errors and absence or multiplicity of matching extrema retain the original outcomes.

Using the ordered extremal filters G_R and E_R of section 50, successful greatest selection gives G_R(B) = [g]. Filter membership then yields

$$g \in B, \qquad \forall b \in B,\ b \leq_R g, \qquad \forall r \in B,\ (\forall b \in B,\ b \leq_R r) \Rightarrow r = g.$$

Indeed, any such rival r belongs to G_R(B), and membership of [g] forces r = g. The least case reverses both comparisons. Conversely, if B is distinct, filtering preserves distinctness; membership of g together with exclusion of every different rival forces the entire matching list to be [g]. Distinctness matters here: two occurrences of the same matching label produce a two-entry list and the original selector returns none.

For successful lower-bound generation B = L_R(H), substitute its membership equivalence into the greatest result:

$$m \in C, \quad \forall h \in H,\ m \leq_R h, \quad \forall x \in C,\ (\forall h \in H,\ x \leq_R h) \Rightarrow x \leq_R m.$$

The dual substitution for B = U_R(H) gives

$$j \in C, \quad \forall h \in H,\ h \leq_R j, \quad \forall x \in C,\ (\forall h \in H,\ h \leq_R x) \Rightarrow j \leq_R x.$$

These conclusions follow from actual returned witnesses. They do not assume that an extremum exists for every held selection.

> Source, proof and provenance: V3 unique_greatest, sft/mathematics/order_lattice/law.py:45; V3 unique_least, sft/mathematics/order_lattice/law.py:50. Native results: SFT.Mathematics.OrderOperations.meetGreatestLowerBound, OrderExtrema.lean:149; SFT.Mathematics.OrderOperations.joinLeastUpperBound, OrderExtrema.lean:160. Timeline: Turn 093, line 1305. Evidence: Completion TURN_118_SFT-MATH-ORDER-LATTICE-001_COMPLETION.

53. Conditional meet/join existence and monotonicity

The original bound guard succeeds exactly for a nonempty held selection whose members belong to the carrier. Bound generation then returns its full ordered filter. On a distinct carrier, both bound lists are distinct. For an accepted partial order and a valid held selection, meet returns a given value if and only if it is a carrier lower bound above every other carrier lower bound. Join has the dual equivalence. Antisymmetry supplies extremal uniqueness, so the converses imply actual successful evaluation.

Monotonicity succeeds exactly when both endpoint images exist for every source comparison and their ordered image pair belongs to the target relation. Dictionary replacement semantics are retained; an empty source relation passes its exhaustive check.

The six original witnesses are proved using their actual operations and original names/statements: the diamond is a partial order, its arms remain incomparable, the chain is total while the diamond is not, the arms meet at One and join at whole, and the canonical identity mapping preserves every diamond comparison. Fresh execution reproduces all six witness rows.

Assume Partial(C,R), H ≠ [], and H ⊆ C. The bound guard then returns its ordered filter, and distinctness of C implies distinctness of that filter. If m is a greatest lower bound and r is another greatest member of L_R(H), then

$$r \le_R m \quad \text{and} \quad m \le_R r \quad \Longrightarrow \quad r = m$$

by antisymmetry. Thus the greatest-filter list is exactly [m] and evaluation returns ok(some(m)). The least-upper-bound proof is dual. Together with section 52 this establishes

$$\mathrm{meet}(C, R, H) = \mathrm{ok}(\mathrm{some}(m)) \iff m \text{ satisfies the displayed greatest-lower-bound conditions},$$

and the analogous join equivalence. No existence is asserted when those conditions have no witness.

For the actual mapping dictionary I, monotonicity has the exact finite interpretation

$$\forall (a,b) \in R,\ \exists u, v,\ \mathrm{lookup}(I,a) = \mathrm{some}(u) \wedge \mathrm{lookup}(I,b) = \mathrm{some}(v) \wedge (u,v) \in S.$$

A missing endpoint image makes its cell false; an empty R has no cells to fail. The original diamond uses C = [One,left,right,whole] and

$$R = \{(x,x) : x \in C\} \cup \{(One,left),(One,right),(One,whole),(left,whole),(right,whole)\}.$$

Neither (left,right) nor (right,left) occurs. Direct complete bound selection gives

$$L_R([left,right]) = [One], \qquad U_R([left,right]) = [whole],$$

so the corresponding extremal filters return meet One and join whole. The three-member chain has the reflexive cells and (a,b),(b,c),(a,c), giving comparability of every pair. Finally the identity dictionary satisfies (f(a),f(b)) = (a,b) ∈ R for each relation cell, proving the sixth witness directly.

> Source, proof and provenance: V3 meet, sft/mathematics/order_lattice/law.py:55; V3 join, sft/mathematics/order_lattice/law.py:59; V3 is_monotone, sft/mathematics/order_lattice/law.py:63. Native results: SFT.Mathematics.OrderOperations.acceptedMeetIff, OrderBoundExistence.lean:58; SFT.Mathematics.OrderOperations.acceptedJoinIff, OrderBoundExistence.lean:81; SFT.Mathematics.OrderOperations.monotoneMeaning, OrderMaps.lean:5. Timeline: Turn 094, line 1317; Timeline: Turn 094, line 1318. Evidence: Completion TURN_118_SFT-MATH-ORDER-LATTICE-001_COMPLETION.

54. Complete order grammar and program correspondence

The order/lattice grammar retains all 512 combinations of its nine original dimensions: carrier identity, relation provenance, partial-order witnesses, comparability, bound generation, meet uniqueness, join uniqueness, monotonicity and extra-order status. Native proofs establish complete membership, cardinality, exact acceptance criteria, the sole all-admitted survivor and rejection of each inadmissible coordinate. Every combination remains in the source order.

The native program retains its original registration and dependencies, complete candidate records, identifier lookup behavior, completeness and trace identities, decision proof identities, closure and generality payloads, and four original controls. Fresh execution matches all 512 candidate records and complete program fields against the pinned source artifacts. Unknown identifiers retain StopIteration.

These results accompany the general order, bound, monotonicity and carrier-extension derivations for the completed order/lattice entry. Completion of this entry is distinct from completion of the entire SFT formalisation.

Let A_i be the ordered two-choice list for source dimension i, with rejected choice followed by admitted choice. The complete candidate list is the ordered Cartesian product

$$\Gamma = A_1 \times \cdots \times A_9, \qquad |\Gamma| = \prod_{i=1}^{9} |A_i| = 2^9 = 512.$$

Every typed candidate has one member of each A_i and therefore belongs to Γ. For a candidate c, let p_i(c) assert that its ith coordinate is the original admitted choice. Exhaustive evaluation establishes

$$\mathrm{survives}(c) = \mathrm{criteriaDecision}(c), \qquad \mathrm{criteriaDecision}(c) = \mathrm{true} \iff \bigwedge_{i=1}^{9} p_i(c) \iff c = c_*.$$

Here c_* is the tuple of all nine admitted choices. A rejected coordinate contradicts the corresponding conjunct, so no candidate with any rejected coordinate survives. These equations describe the complete registered grammar; the order-operation derivations in sections 50–53 establish the mathematical content tested by its admitted choices. The program comparison checks all 512 source-ordered records and their accompanying fields, rather than sampling only c_*.

The original program constructs each candidate from the ordered Cartesian product of its declared choice lists, joins its coordinate names to form the identifier, and retains the coordinate/form record. Decision lookup first finds that exact identifier in the complete product, then compares it with the joined all-admitted identifier. A missing identifier therefore fails lookup. The reason scan stops at the first rejected coordinate in declared order. Census hashes retain the complete candidate record; closure hashes retain the closure record and ordered decisions; each control retains its own observed result and receipt hash. Registration uses the same specification and source identity. These are the particular program operations compared with their native records.

> Source, proof and provenance: V3 is_partial_order, sft/mathematics/order_lattice/law.py:12. Native results: SFT.Mathematics.OrderEvidence.registrationHasNoFreeParameters, OrderEvidence.lean:83; SFT.Mathematics.OrderEvidence.lookupDependsOnlyOnIdentifier, OrderEvidence.lean:86. Timeline: Turn 095, line 1331. Evidence: Completion TURN_118_SFT-MATH-ORDER-LATTICE-001_COMPLETION.

Program correspondence sources and proofs: V3 candidate_records, mathematics/generated_law.py:L117; V3 registration, mathematics/generated_law.py:L230; V3 generate_candidates, mathematics/generated_law.py:L245; V3 decide_candidate, mathematics/generated_law.py:L264; V3 closure_evidence, mathematics/generated_law.py:L285; V3 control_records, mathematics/generated_law.py:L187; OrderGrammar.generatedCardinality, L127; OrderGrammar.candidatesComplete, L129; OrderGrammar.acceptanceIffExactCriteria, L184; OrderEvidence.registrationData, L17; OrderEvidence.censusData, L37; OrderEvidence.decisionData, L46; OrderEvidence.closureData, L61.

55. Order induction base and comparison extension

The One carrier with its identity relation is both a partial and total order. Its complete lower and upper bound lists for the held One selection are the singleton One, and both meet and join return One.

For a fresh carrier addition preserving every old comparison cell, the original endpoint and distinct-relation guards are retained. Partial-order success on the enlarged carrier is equivalent to the new reflexive cell, antisymmetry for every reversed pair touching the fresh member, and closure for every composable triple

touching it. Old-only obligations follow from the prior order and retained comparisons. Given prior totality and an admitted enlarged partial order, totality of the enlarged order is equivalent to comparison between the fresh member and each old member.

The base carrier and relation are C₁ = [One] and R₁ = [(One,One)]. Their single relation cell simultaneously supplies reflexivity, antisymmetry, transitivity and comparability. Direct filtering gives

$$L_{R_1}([One]) = U_{R_1}([One]) = [One], \qquad \text{meet} = \text{join} = \text{ok}(\text{some}(One)).$$

For C' = C ++ [z], assume Partial(C,R), z ∉ C, distinct R', all endpoints of R' in C', and exact retention (a,b) ∈ R' ⇔ (a,b) ∈ R for old a,b. Then

$$\begin{aligned}\text{Partial}(C', R') \iff\ & (z,z) \in R' \\ & \wedge [\forall a, b,\ (a,b),(b,a) \in R' \wedge (a = z \vee b = z) \Rightarrow a = b] \\ & \wedge [\forall a, b, c,\ (a,b),(b,c) \in R' \wedge (a = z \vee b = z \vee c = z) \Rightarrow (a,c) \in R'].\end{aligned}$$

To prove the reverse direction, classify each endpoint as old or equal to z. All-old cells reduce through retention to the old laws; every remaining case is one of the displayed new checks. Freshness preserves carrier distinctness, while relation distinctness and endpoint admission remain explicit hypotheses. If the old order is total and the extended relation is already a partial order, the same split gives

$$\text{Total}(C', R') \iff \forall a \in C,\ (z,a) \in R' \vee (a,z) \in R'.$$

The old-old comparisons transfer, and (z,z) is supplied by extended reflexivity.

> Source, proof and provenance: V3 is_partial_order, sft/mathematics/order_lattice/law.py:12. Native results: SFT.Mathematics.OrderOperations.partialOrderExtensionIff, OrderInduction.lean:12; SFT.Mathematics.OrderOperations.totalOrderExtensionIff, OrderInduction.lean:62. Timeline: Turn 096, line 1343. Evidence: Completion TURN_118_SFT-MATH-ORDER-LATTICE-001_COMPLETION.

56. Bound lists under carrier extension

Successful bound generation establishes that the held selection is nonempty and contained in the original carrier. When every old comparison cell is retained and that held selection remains fixed, every old candidate has the same bound-check result in the enlarged relation.

The enlarged lower-bound list is exactly the original ordered list followed by the added member when it lies below every held member; otherwise the list is unchanged. The upper-bound list has the dual formula. These are equalities of the actual generated lists, preserving order and occurrence multiplicity, and require no additional distinctness assumption.

Let the held selection H be valid on C, and let R' retain all old comparisons. For every x ∈ C and h ∈ H, retention identifies both lower and upper comparison tests. Hence the complete old-candidate filters are unchanged. Using the list identity

$$\text{filter}(P, C \mathbin{+\!\!+} [z]) = \text{filter}(P, C) \mathbin{+\!\!+} \text{filter}(P, [z]),$$

the actual extended bound lists are

$$L_{C',R'}(H) = L_{C,R}(H) \mathbin{+\!\!+} \begin{cases} [z], & \forall h \in H,\ z \le_{R'} h, \\ \text{empty list}, & \text{otherwise}, \end{cases}$$

$$U_{C',R'}(H) = U_{C,R}(H) \mathbin{+\!\!+} \begin{cases} [z], & \forall h \in H,\ h \le_{R'} z, \\ \text{empty list}, & \text{otherwise}. \end{cases}$$

The original successful bound result proves H ≠ [] and H ⊆ C ⊆ C', so the extended guard also succeeds. These formulas do not require freshness of z or distinctness of C: an appended repeated label remains an appended occurrence, exactly as in the source lists.

> Source, proof and provenance: V3 lower_bounds, sft/mathematics/order_lattice/law.py:33; V3 upper_bounds, sft/mathematics/order_lattice/law.py:39. Native results: SFT.Mathematics.OrderOperations.lowerBoundsCarrierExtension, OrderBoundExtension.lean:78; SFT.Mathematics.OrderOperations.upperBoundsCarrierExtension, OrderBoundExtension.lean:96. Timeline: Turn 097, line 1354. Evidence: Completion TURN_118_SFT-MATH-ORDER-LATTICE-001_COMPLETION.

57. Held-selection updates and extremal retention

Adding a carrier member to an already valid held selection filters the existing lower-bound list by comparison below that member, and filters the upper-bound list by comparison above it. These are equalities of the generated lists, retaining their order and multiplicity. On an accepted partial order, the old meet remains the meet exactly when it lies below the added held member; the old join remains exactly when it lies above it.

For an admitted carrier extension that retains all old comparisons, an existing meet remains exactly when the added carrier member, if it is a lower bound, lies below that meet. The join condition is dual: an added upper bound must lie above the old join. The conditions preserve the original conditional existence of extrema and include all old carrier candidates through the complete prior bound properties.

For an added held member y ∈ C, the finite all-test splits as

$$[\forall h \in H +\!\!+ [y],\ x \le_R h] \iff [\forall h \in H,\ x \le_R h] \wedge x \le_R y.$$

Consequently, for previously valid H,

$$L_R(H +\!\!+ [y]) = [x \in L_R(H) \mid x \le_R y], \qquad U_R(H +\!\!+ [y]) = [x \in U_R(H) \mid y \le_R x].$$

Assume an accepted partial order and prior returned meet m and join j. If m ≤_R y, then m remains a lower bound of the enlarged held selection; every new lower bound was an old lower bound and therefore lies below m. Conversely any returned meet m must lie below y. The dual argument proves

$$\mathrm{meet}(H +\!\!+ [y]) = \mathrm{ok}(\mathrm{some}(m)) \iff m \le_R y,$$

$$\mathrm{join}(H +\!\!+ [y]) = \mathrm{ok}(\mathrm{some}(j)) \iff y \le_R j.$$

For carrier extension C' = C ++ [z], instead keep H fixed, retain all old comparisons, and assume Partial(C',R'). Every old lower-bound candidate is still bounded by m; only the appended candidate can add an obligation. Thus

$$\mathrm{meet}(C', R', H) = \mathrm{ok}(\mathrm{some}(m)) \iff [(\forall h \in H,\ z \le_{R'} h) \Rightarrow z \le_{R'} m].$$

Similarly,

$$\mathrm{join}(C', R', H) = \mathrm{ok}(\mathrm{some}(j)) \iff [(\forall h \in H,\ h \le_{R'} z) \Rightarrow j \le_{R'} z].$$

If z is not a bound, the corresponding implication is vacuous; if it is a bound, its comparison with the prior extremum is required. The accepted-order extremum equivalences of section 53 turn these complete bound conditions into actual successful returns.

> Source, proof and provenance: V3 lower_bounds, sft/mathematics/order_lattice/law.py:33; V3 meet, sft/mathematics/order_lattice/law.py:55. Native results: SFT.Mathematics.OrderOperations.meetRetainedOnCarrierExtension, OrderHeldExtension.lean:129; SFT.Mathematics.OrderOperations.joinRetainedOnCarrierExtension, OrderHeldExtension.lean:156. Timeline: Turn 098, line 1366. Evidence: Completion TURN_118_SFT-MATH-ORDER-LATTICE-001_COMPLETION.

58. Extremal selection of the added carrier member

For an accepted carrier extension retaining all original comparisons, the added member is selected as meet exactly when it lies below every held member and above every member of the original lower-bound list. Dually, it is selected as join exactly when it lies above every held member and below every original upper bound. The original bound lists arise from successful nonempty held selections. Every original bound participates in these conditions; reflexivity supplies the added member's self-comparison, and accepted partial-order antisymmetry supplies extremal uniqueness. No existence of an old meet or join is required.

Let B = L_(C,R)(H) be the successfully generated old lower-bound list, and let C' = C ++ [z] carry an accepted relation R' retaining all old comparisons. Then

$$\mathrm{meet}(C', R', H) = \mathrm{ok}(\mathrm{some}(z)) \iff [\forall h \in H,\ z \leq_{R'} h] \wedge [\forall b \in B,\ b \leq_{R'} z].$$

Necessity follows by applying the returned-meet conditions to every old lower bound. For sufficiency, any lower-bound candidate in C' is either old, hence belongs to B by retained comparisons, or is z itself. The displayed condition handles the former; reflexivity handles the latter. The accepted-order equivalence then yields the actual singleton extremum. Dually, for B = U_(C,R)(H),

$$\mathrm{join}(C', R', H) = \mathrm{ok}(\mathrm{some}(z)) \iff [\forall h \in H,\ h \leq_{R'} z] \wedge [\forall b \in B,\ z \leq_{R'} b].$$

Only successful old bound generation is assumed: B need not have had an extremum before the extension.

> Source, proof and provenance: V3 meet, sft/mathematics/order_lattice/law.py:55; V3 join, sft/mathematics/order_lattice/law.py:59. Native results: SFT.Mathematics.OrderOperations.freshMeetIff, OrderFreshExtrema.lean:5; SFT.Mathematics.OrderOperations.freshJoinIff, OrderFreshExtrema.lean:39. Timeline: Turn 099, line 1377. Evidence: Completion TURN_118_SFT-MATH-ORDER-LATTICE-001_COMPLETION.

59. Generated graphs, complete paths and cuts

Graph construction checks a nonempty canonical node collection, duplicate-free edge relations and membership of every edge endpoint in the carrier, in that order. Path validation retains the nonempty path, every node membership and every adjacent ordered edge. The recursively generated adjacency list is proved equal to the original path zipped with its tail. The identity path is admitted exactly for a node in the carrier.

At a shared interface node, the adjacency list of the composed path equals the complete first adjacency list followed by the complete second adjacency list. Consequently the composed path is valid if and only if both component paths are valid. This is an equivalence for arbitrary finite prefixes and suffixes, without erasing any transition.

A successful cut returns exactly the original edge list filtered by unequal held-endpoint membership, preserving edge order and multiplicity. Its membership criterion is proved directly. The internal balance operation is equivalent to nonempty ingress, equal ingress and egress trace counts, and separate distinctness of both trace collections. The executable criterion is retained exactly.

Fresh native execution matches 4597 independently reconstructed records for graph construction, paths, adjacency, cuts and balance, including malformed and repeated inputs. The complete native GraphProgram also matches the pinned V3 registration, all 512 candidate decisions, census and decision hashes, closure, controls and unknown-identifier behavior.

For node list V and edge list E, successful construction requires

$$V \neq [], \quad \mathrm{Distinct}(V), \quad \mathrm{Distinct}(E), \quad \forall (u, v) \in E,\ u, v \in V.$$

For a nonempty path P = [v₀,…,vₙ], the complete adjacent list is Adj(P) = [(v₀,v₁),…,(vₙ₋₁,vₙ)], equivalently zip(P,tail(P)). Path admission is exactly

$$P \neq [] \wedge (\forall v \in P,\ v \in V) \wedge (\forall e \in \mathrm{Adj}(P),\ e \in E).$$

At interface z, take P = A ++ [z] and Q = z :: B. The concatenation retains z once, and the exact adjacency identity is

$$\mathrm{Adj}(A +\!\!+ (z :: B)) = \mathrm{Adj}(A +\!\!+ [z]) +\!\!+ \mathrm{Adj}(z :: B).$$

Splitting node membership and the exhaustive edge check across this concatenation proves validity of the composite if and only if both components are valid. For a nonempty held selection H ⊆ V, the cut is the ordered filter

$$\mathrm{Cut}_H(E) = [(u,v) \in E \mid (u \in H) \operatorname{xor} (v \in H)].$$

Thus an edge occurs in the result exactly when it occurs in E and exactly one endpoint is held. Finally, for incoming and outgoing trace lists I,O, the original balance predicate is

$$I \neq [] \wedge |I| = |O| \wedge \mathrm{Distinct}(I) \wedge \mathrm{Distinct}(O).$$

This is the complete executable predicate; no additional conservation condition is inserted.

> Source, proof and provenance: V3 path_is_valid, sft/mathematics/graph_network/law.py:28; V3 crossing_cut, sft/mathematics/graph_network/law.py:65; V3 internal_balance, sft/mathematics/graph_network/law.py:73. Native results: SFT.Mathematics.GraphOperations.adjacentPairsComposition, GraphOperations.lean:130; SFT.Mathematics.GraphOperations.pathCompositionIff, GraphOperations.lean:140. Timeline: Turn 100, line 1389. Evidence: Completion TURN_118_SFT-MATH-GRAPH-NETWORK-001_COMPLETION.

60. Finite reachability search

Reachability retains the original identity case: equal origin and target return exactly origin membership in the node carrier. Otherwise the queue starts with the origin and the visited trace is empty. Each iteration removes the queue head, checks target identity before visited membership, skips a previously visited head, or appends a new head to visited and appends every matching unvisited edge target to the queue in original edge order.

Termination follows from finite support and queue length. Initially the support consists of the origin and every edge target. Every queued node and edge target lies in that support or in visited. A new visit removes the held node from the auxiliary unseen support, strictly decreasing its length. A visited skip leaves support unchanged and strictly shortens the queue. Their lexicographic order is well founded. The auxiliary support does not decide search success, change queue order, omit duplicate enqueues or impose a depth bound.

The native search satisfies the exact fixed-point equation and the original empty-queue, target-hit, visited-skip and fresh-expansion equations. Fresh execution matches 8,573 reachability decisions and 10,280 ordered outgoing lists across every directed relation on three nodes, with additional duplicate-edge and external-endpoint stress inputs. Cycle search and its full return-path correspondence are formalised below.

Let Q be the queue, S the visited list and t the target. Define Out_E(u,S) to be the ordered list of edge targets v for edges (u,v) ∈ E with v ∉ S. The exact recursive cases are

$$\begin{aligned}
\mathrm{Search}([], S) &= \mathrm{false}, \\
\mathrm{Search}(u :: Q, S) &= \mathrm{true} && \text{if } u = t, \\
&= \mathrm{Search}(Q, S) && \text{if } u \neq t \text{ and } u \in S, \\
&= \mathrm{Search}(Q +\!\!+ \mathrm{Out}_E(u, S +\!\!+ [u]), S +\!\!+ [u]) && \text{otherwise.}
\end{aligned}$$

The target test precedes the visited test. Initially Q = [origin] and S = []; equal origin and target are handled separately by origin ∈ V. Let U be the auxiliary unseen list, initially origin :: map(target,E). Coverage states that every queued node and every edge target lies in U or S. A fresh head u therefore belongs to U. Removing all its occurrences strictly decreases |U|; a visited skip leaves U unchanged and decreases |Q|. Each recursive call consequently decreases the lexicographic measure

$$(|U|, |Q|).$$

Finite-list length makes this order well founded. Expansion still appends every source-ordered unvisited target, including duplicate enqueues; the auxiliary measure proves termination without changing the displayed search equations.

> Source, proof and provenance: V3 reachable, sft/mathematics/graph_network/law.py:34. Native results: SFT.Mathematics.GraphOperations.searchSkipsVisited, GraphReachability.lean:134; SFT.Mathematics.GraphOperations.searchExpandsFresh, GraphReachability.lean:147. Timeline: Turn 101, line 1402. Evidence: Completion TURN_118_SFT-MATH-GRAPH-NETWORK-001_COMPLETION.

61. Complete path witnesses for successful reachability

For every graph admitted by the original constructor and every origin and target, a true reachability result entails a complete path witness. The witness is a finite node list whose first node is the origin, whose final node is the target, and which satisfies the original path validator. Thus it retains every node membership and every ordered adjacent edge. The theorem requires no separate assumption that the queried origin belongs to the carrier.

Outgoing-list membership is equivalent to a retained edge from the held node to the queried node together with that node's absence from visited. Every property of queued nodes that is preserved by each retained edge is preserved through the exact queue loop. Applying this invariant to complete path witnesses establishes search soundness. The initial origin is retained as an alternative until its first edge supplies carrier membership; the equal-origin/target case obtains membership directly from the original identity check. Successful construction supplies both endpoint memberships for every retained edge.

Path extension appends the new target to the complete prior path and uses the shared-interface composition theorem, preserving its origin and every earlier transition. The resulting proof covers arbitrary finite admitted graphs, rather than only the enumerated execution cases. Search completeness is established by the closed-set argument below.

For a graph G, define the complete witness predicate

$$\mathrm{PathWitness}_G(o,t) \iff \exists A,\ \mathrm{Valid}_G(A +\!\!+ [t]) \wedge \mathrm{head}(A +\!\!+ [t]) = o.$$

The queue proof uses a property P closed under retained edges:

$$(u,v) \in E \wedge P(u) \Rightarrow P(v).$$

If P holds for every queued node, a target hit establishes P(t). A visited skip preserves P on the shorter queue. A fresh expansion preserves it on the old remainder and every appended target by edge closure. Induction along the decreasing search measure therefore proves search success implies P(t).

Choose P(v) to mean v = o or PathWitness_G(o,v). This holds initially even if o has not separately been admitted as a node. On the first traversed edge (o,v), accepted graph endpoints supply o ∈ V and v ∈ V; the identity path [o] therefore extends to [o,v]. Every later edge extends the complete prior witness. If o ≠ t, the alternative t = o is impossible, leaving a path witness. If o = t, the original identity check itself supplies o ∈ V and the singleton witness. Thus

$$\mathrm{Reachable}_G(o,t) = \mathrm{true} \Longrightarrow \mathrm{PathWitness}_G(o,t)$$

for every admitted graph and arbitrary queried endpoints.

> Source, proof and provenance: V3 reachable, sft/mathematics/graph_network/law.py:34. Native results: SFT.Mathematics.GraphOperations.reachableHasPath, GraphPathSoundness.lean:125; SFT.Mathematics.GraphOperations.admittedReachabilityHasPath, GraphPathSoundness.lean:153. Timeline: Turn 102, line 1417. Evidence: Completion TURN_118_SFT-MATH-GRAPH-NETWORK-001_COMPLETION.

62. Reachability completeness and equivalence

Every complete valid origin-to-target path is found by the original finite queue search. The proof preserves the invariant that every edge leaving a visited node ends either in visited or in the frontier. Removing an already visited queue head preserves that invariant. Expanding a new head preserves it because every unvisited outgoing target is appended, in the original edge order.

If search terminates unsuccessfully, the resulting exhausted finite node collection contains the initial frontier and visited nodes, excludes the target, and is closed under every retained edge. A complete adjacent path beginning inside such a collection remains inside it at every transition. Consequently a valid path to the target contradicts unsuccessful search. This establishes completeness without bounding path length or discarding repeated queue entries.

Combining completeness with soundness gives the exact equivalence, for every admitted finite graph: reachability returns true if and only if a complete valid path starts at the queried origin and ends at the queried target. Identity reachability holds exactly for carrier nodes. Complete path witnesses compose at their shared interface, yielding the original transitivity law for reachability.

The visited/frontier invariant is

$$u \in S \wedge (u, v) \in E \Longrightarrow v \in S \vee v \in Q.$$

If a queue head is skipped, it already belongs to S, so removing that occurrence from Q preserves coverage. If a fresh head is expanded, it is appended to S and every outgoing target not in the new S is appended to Q. The displayed invariant is therefore preserved in both recursive cases.

Assuming the target is absent from S, unsuccessful search supplies a finite exhausted list X with

$$S \subseteq X, \qquad Q \subseteq X, \qquad t \notin X, \qquad u \in X \wedge (u, v) \in E \Rightarrow v \in X.$$

At the initial state, o ∈ Q ⊆ X. Induction over adjacent cells of any complete valid o-to-t path propagates membership in X from its first node to its last, forcing t ∈ X, a contradiction. Hence every path witness is found. Combining both directions gives

$$\mathrm{Reachable}_G(o, t) = \mathrm{true} \iff \mathrm{PathWitness}_G(o, t).$$

For admitted G, shared-interface path composition then proves

$$\mathrm{Reachable}_G(o, m) \wedge \mathrm{Reachable}_G(m, t) \Rightarrow \mathrm{Reachable}_G(o, t),$$

while Reachable_G(v,v) is equivalent to v ∈ V. No bound on path length enters either implication.

> Source, proof and provenance: V3 reachable, sft/mathematics/graph_network/law.py:34. Native results: SFT.Mathematics.GraphOperations.admittedReachabilityIffPath, GraphPathCompleteness.lean:160; SFT.Mathematics.GraphOperations.admittedReachabilityTransitive, GraphPathCompleteness.lean:190. Timeline: Turn 103, line 1430. Evidence: Completion TURN_118_SFT-MATH-GRAPH-NETWORK-001_COMPLETION.

63. Exact finite cycle traversal

The cycle procedure retains the original outer node order and recursively scans every edge in its original order. Edges whose origin differs from the current node are skipped. An edge to a node already in the held path returns true immediately. Otherwise its target is appended to the held path and visited recursively; a false result resumes scanning the remaining edges. No branch or edge is removed.

Finite termination follows from an auxiliary list containing every edge target. Every target remains either in that unseen support or in the held path. A recursive call is made only for a target absent from the held path; that target therefore occurs in unseen support, and removing it strictly decreases the support length. Each local edge scan is structurally recursive on the remaining edge list. Neither argument introduces a depth cutoff.

The native procedure satisfies its exact recursive equation. Empty edge sets and nodes without outgoing edges cannot yield a cycle, and a retained outgoing edge to a held-path member forces a true result. Fresh execution matches 3,084 graph-cycle decisions and 14,392 recursive visits across all directed relations on three nodes and additional malformed/repeated input stresses. General equivalence with complete nontrivial return paths is established below.

Let Visit(u,H) denote recursive cycle exploration from node u with ordered held path H, and Scan(u,H,L) its scan of the remaining ordered edge list L. With e = (a,b), the exact source cases are

$$\begin{aligned} \mathrm{Scan}(u, H, []) &= \mathrm{false}, \\ \mathrm{Scan}(u, H, e :: L) &= \mathrm{Scan}(u, H, L) && \text{if } a \neq u, \\ &= \mathrm{true} && \text{if } a = u \text{ and } b \in H, \\ &= \mathrm{Visit}(b, H +\!\!+ [b]) \vee \mathrm{Scan}(u, H, L) && \text{otherwise.} \end{aligned}$$

The final disjunction short-circuits in the displayed order: the scan resumes only when the recursive visit returns false. Each Visit scans the entire original E, and the outer result is

$$\mathrm{CycleExists}(G) = \mathrm{any}_{u \in V}\, \mathrm{Visit}(u, [u]),$$

in original node order. For recursive calls, auxiliary coverage places every edge target in unseen U or H. Since a recursive target b is absent from H, it occurs in U; removing its occurrences gives |U'| < |U|. Local scan recursion instead shortens L. These two finite decreases establish termination while preserving every branch and the ordered held path. An empty E makes every scan false; a matching edge to H makes its scan true immediately.

> Source, proof and provenance: V3 directed_cycle_exists, sft/mathematics/graph_network/law.py:52. Native results: SFT.Mathematics.GraphOperations.cycleVisitImmediateReturn, GraphCycles.lean:112; SFT.Mathematics.GraphOperations.edgelessGraphHasNoCycle, GraphCycles.lean:118. Timeline: Turn 104, line 1443. Evidence: Completion TURN_118_SFT-MATH-GRAPH-NETWORK-001_COMPLETION.

64. Cycle detection and complete return paths

For every admitted finite graph, the original cycle procedure returns true if and only if there is a complete valid path with at least one edge whose final node occurs earlier in the path. The starting node belongs to the graph carrier; every node and adjacency is checked by the original path validator. Self-loops satisfy the one-edge case. Longer witnesses may contain repeated nodes, with no extra simplicity restriction.

The proof first establishes exact correspondence between the recursive traversal and a finite return trace recording every successive edge target. A trace either closes along a retained edge to an already held node, or advances along a retained edge to a new node and continues with that node appended to the held path. Every such trace is nonempty, contains only retained adjacent edges, and yields a valid node path on an admitted graph. Its terminal target belongs to the held path together with the preceding trace prefix.

Conversely, a complete path ending at a prior node produces a return trace. Traversal follows its retained edges until the first held-node return; if a repeated node occurs before the supplied terminal return, that earlier return supplies the witness. Thus neither the equivalence nor the executable traversal requires paths to be simple. The proof applies to arbitrary finite admitted graphs and does not depend on the finite execution comparisons.

The retained ReturnTrace relation has exactly two constructors. A closing edge (u,v) ∈ E with v ∈ H yields trace [v]. An advancing edge (u,v) ∈ E with v ∉ H followed by a return trace T from (v,H ++ [v]) yields v :: T. Induction over these constructors proves nonemptiness, every required adjacent edge, and a closing decomposition

$$T = A +\!\!+ [t], \qquad t \in H +\!\!+ A.$$

Induction over the exact recursive scan proves it returns true exactly when such a trace exists. With H = [u], graph admission supplies all endpoint memberships and converts u :: T to a complete valid path. Thus the exact graph-level equivalence is

$$\mathrm{CycleExists}(G) = \mathrm{true} \iff \exists u, A, t,\ u \in V \wedge \mathrm{Valid}_G(u :: (A +\!\!+ [t])) \wedge t \in u :: A.$$

The displayed path has at least one edge, including when A is empty. Conversely, follow its adjacent edges until the first target already in the held prefix. That edge supplies the closing constructor; all earlier edges supply advancing constructors. Any earlier repetition closes sooner and is still detected, so simplicity is neither assumed nor imposed.

> Source, proof and provenance: V3 directed_cycle_exists, sft/mathematics/graph_network/law.py:52. Native results: SFT.Mathematics.GraphOperations.admittedCycleIffCompleteReturnPath, GraphCycleTrace.lean:236. Timeline: Turn 105, line 1456. Evidence: Completion TURN_118_SFT-MATH-GRAPH-NETWORK-001_COMPLETION.

65. Graph witnesses and complete registered grammar

The six original graph witnesses are retained with their exact names, statements and executable predicates: valid chain path, missing-edge rejection, directed reachability, explicit cycle return, exact crossing cut and paired internal balance. The chain and cycle examples satisfy the original graph constructor. Positive reachability and return results follow from the general path theorems; the negative chain cases follow from the exact traversal equations.

The registered graph grammar exhausts all 512 combinations of its nine binary structural coordinates: nodes, edges, orientation, path, connectivity, cycle, cut, balance and additional rules. Native proofs establish complete enumeration, cardinality, equality of source classification with the preservation criteria, the unique all-preserving survivor, and rejection when any coordinate is replaced by its source alternative. No candidate is omitted.

Fresh native execution of the complete graph program matches the pinned registration, census, all 512 actual candidate lookups and decisions, trace and proof hashes, closure and generality records, four original controls and unknown-identifier behavior. These results accompany the general path/search and cycle/return equivalences. The source graph induction results appear below; the graph entry and its complete native program correspondence are recorded as complete.

The chain witness uses V = [a,b,c], E = [(a,b),(b,c)]. Therefore Adj([a,b,c]) = E, whereas Adj([a,c]) = [(a,c)] is absent. Its concrete results are

$$\mathrm{Valid}([a,b,c]) = \mathrm{true}, \quad \mathrm{Valid}([a,c]) = \mathrm{false}, \quad \mathrm{Reachable}(a,c) = \mathrm{true}, \quad \mathrm{Reachable}(c,a) = \mathrm{false}.$$

The reverse search exhausts immediately because c has no outgoing edge. Appending (c,a) supplies the return path [a,b,c,a], so cycle detection is true on the augmented graph and false on the chain. Holding H = [a] gives Cut_H(E) = [(a,b)]. The balance witness uses I = [[i1],[i2]] and O = [[o1],[o2]]: both are

distinct, I is nonempty and |I| = |O| = 2.

For the original nine binary grammar axes, Γ = A₁×⋯×A₉ has

$$|\Gamma| = 2^9 = 512, \qquad \mathrm{survives}(c) = \mathrm{true} \iff \bigwedge_{i=1}^{9} [c_i = \mathrm{admitted}_i] \iff c = c_*.$$

Complete product membership covers every typed tuple. Equality with the original source decision is checked over that full product, and a rejected coordinate contradicts the corresponding conjunct. These grammar results accompany the six operational witnesses and general graph proofs.

> Source, proof and provenance: V3 directed_cycle_exists, sft/mathematics/graph_network/law.py:52. Native results: SFT.Mathematics.GraphWitnesses.pairedBalancePassed, GraphWitnesses.lean:107; SFT.Mathematics.GraphWitnesses.allWitnessesPass, GraphWitnesses.lean:111. Timeline: Turn 106, line 1469. Evidence: Completion TURN_118_SFT-MATH-GRAPH-NETWORK-001_COMPLETION.

66. Graph induction and locality of path updates

The One graph with no edges is admitted by the original constructor, supplies its identity path and identity reachability, and has no directed cycle. Under extensions retaining prior nodes and edges, every existing complete path and return trace remains valid. Consequently prior reachability on an admitted graph and prior detected cycles are retained.

For a carrier extended by a fresh node and an appended edge selection, any newly valid path whose nodes all belong to the old carrier must contain an added edge absent from the old relation. When each added edge has the fresh node as an endpoint and that node is absent from the old carrier, such a path cannot use an added edge. Hence the original and extended path-validation results are exactly equal on every sequence of old nodes, including rejected sequences. This establishes the locality of path updates in the source fresh-node induction step.

These results retain the explicit conditions of the extension. They do not impose new constructor guards or restrict the original graph operations. Ordered pair-support generation, cut and paired-balance extension are established below; held-selection characterization is also established below. These are results of the completed graph entry.

The base graph is G₁ = ([One],[]), with Valid([One]) = true, Reachable(One,One) = true and CycleExists(G₁) = false. If V ⊆ V' and E ⊆ E', every valid old path retains all node and edge checks. Every old return trace likewise retains each edge and held-prefix comparison. The path and cycle equivalences therefore transfer positive reachability and cycle results.

For V' = V ++ [z] and E' = E ++ A, suppose a sequence P contains only old nodes, is rejected by G and accepted by G'. The accepted path is nonempty and satisfies all node checks already in V. Hence some adjacency must account for the change:

$$\exists e \in \mathrm{Adj}(P), \quad e \in A \wedge e \notin E.$$

Now assume z ∉ V and every edge of A touches z. Both endpoints of every adjacency in P belong to P and hence to V. No such adjacency can touch z, contradicting the displayed witness. Thus

$$\forall P \text{ with nodes in } V, \quad \mathrm{Valid}_{(V{+}{+}[z],E{+}{+}A)}(P) = \mathrm{Valid}_{(V,E)}(P).$$

The equality includes rejected and empty sequences. It follows from the explicit fresh-endpoint condition, without adding a restriction to the original path validator.

> Source, proof and provenance: V3 path_is_valid, sft/mathematics/graph_network/law.py:28; V3 reachable, sft/mathematics/graph_network/law.py:34. Native results: SFT.Mathematics.GraphOperations.adjacentEndpointsInPath, GraphInduction.lean:94; SFT.Mathematics.GraphOperations.freshOnlyEdgesPreserveOldPathDecision,

> GraphInduction.lean:108. Timeline: Turn 107, line 1482. Evidence: Completion TURN_118_SFT-MATH-GRAPH-NETWORK-001_COMPLETION.

67. Admitted graph extension, ordered pairs, cuts and balance

The graph constructor succeeds exactly when its node list is nonempty and distinct, its edge list is distinct, and every endpoint belongs to the node list. If a genuinely fresh node is appended, a distinct added-edge selection whose endpoints lie in the enlarged carrier and whose edges touch that fresh node is disjoint from the old relation. The combined graph therefore satisfies precisely the original constructor checks.

The complete ordered node-pair product extends row by row: each old left-node row retains all old right-node cells followed by its fresh-right cell; the final fresh-left row contains every old right node followed by the fresh self-pair. The equality retains the full generated order and does not assert that every generated pair is selected as an edge.

For an already successful held cut selection, extending the node carrier and appending edges returns exactly the old cut list followed by the added edges whose endpoint held-membership differs. Selection validity follows from the original nonempty and carrier-membership guard; no distinctness requirement is added to the held list. For an internally balanced pair of ingress/egress trace lists, appending one previously absent trace to each side preserves equal counts and separate distinctness, hence preserves the original balance predicate.

Let G = (V,E) be admitted, z ∉ V, and A a distinct added-edge list whose endpoints lie in V' = V ++ [z] and whose every edge touches z. If e belonged to both E and A, an old endpoint would equal z, contradicting z ∉ V. Thus E and A are disjoint, their concatenation is distinct, and all constructor guards hold for G' = (V',E ++ A).

Writing row(u,V) = [(u,v):v ∈ V] as an ordered list, the full pair-support identity is

$$\mathrm{Pairs}(V') = \mathrm{concat}_{u \in V}(\mathrm{row}(u, V) \mathbin{+\!\!+} [(u, z)]) \mathbin{+\!\!+} (\mathrm{row}(z, V) \mathbin{+\!\!+} [(z, z)]).$$

This distributes map and flatMap over append and retains each old row's order. For a previously successful cut with fixed held H, filter distribution gives

$$\mathrm{Cut}_H(E \mathbin{+\!\!+} A) = \mathrm{Cut}_H(E) \mathbin{+\!\!+} [e \in A \mid (e_1 \in H) \operatorname{xor} (e_2 \in H)].$$

For balanced ingress and egress lists I,O and fresh respective traces i,o,

$$|I \mathbin{+\!\!+} [i]| = |I| + 1 = |O| + 1 = |O \mathbin{+\!\!+} [o]|.$$

Fresh append preserves each list's distinctness, and the incoming list remains nonempty, proving the original balance predicate after extension.

> Source, proof and provenance: V3 crossing_cut, sft/mathematics/graph_network/law.py:65; V3 internal_balance, sft/mathematics/graph_network/law.py:73. Native results: SFT.Mathematics.GraphOperations.balancePairedExtension, GraphExtension.lean:112; SFT.Mathematics.GraphOperations.freshGraphPairSupportOrder, GraphExtension.lean:124. Timeline: Turn 108, line 1494. Evidence: Completion TURN_118_SFT-MATH-GRAPH-NETWORK-001_COMPLETION.

68. Held-partition updates and exact paired-balance conditions

Appending a carrier node to a valid held selection replaces each endpoint-membership test by membership in the old selection or in that appended singleton. Filtering the complete original edge list by the resulting unequal-membership test gives exactly the updated cut, including the original allowance for repeated held nodes. An edge whose endpoints differ from the appended node has an unchanged cut decision.

For an admitted old graph, a genuinely fresh node belongs to neither endpoint of any old edge. If that node is appended to both carrier and held selection while new edges are appended, the resulting cut is exactly the prior cut followed by the added edges crossing the enlarged held partition. The equation preserves original list order.

Appending one ingress trace and one egress trace yields internal balance if and only if the original trace counts agree, each original trace list is distinct, and neither appended trace already occurs on its respective side. This characterization includes empty original lists: their first trace pair is balanced. For an already balanced pair, the exact remaining requirements are freshness on both sides. No restriction on trace contents is added to the original operation.

Define the endpoint indicator χ_H(x) by membership in the held list. Appending y gives χ_(H++[y])(x) = χ_H(x) ∨ (x=y). Therefore the updated cut is exactly

$$[e \in E \mid (\chi_H(e_1) \vee e_1 = y) \operatorname{xor} (\chi_H(e_2) \vee e_2 = y)].$$

If neither endpoint is y, both added equalities are false and the old cut decision is unchanged. When y = z is fresh to an admitted old graph, this applies to every old edge. For H' = H ++ [z], the combined carrier/edge extension consequently satisfies

$$\mathrm{Cut}_{H'}(E +\!\!+ A) = \mathrm{Cut}_H(E) +\!\!+ \mathrm{Cut}_{H'}(A),$$

where the rightmost term denotes the same ordered crossing-edge filter. Repeated members in H affect neither the indicator nor the source's valid-selection guard.

The exact paired-balance equivalence is

$$\mathrm{Balance}(I +\!\!+ [i], O +\!\!+ [o]) \iff |I| = |O| \wedge \mathrm{Distinct}(I) \wedge \mathrm{Distinct}(O) \wedge i \notin I \wedge o \notin O.$$

The appended incoming list is automatically nonempty; equal successor lengths cancel to |I| = |O|, and distinctness of each append is exactly prior distinctness plus absence of the new trace. In particular Balance([i],[o]) = true for arbitrary traces i,o, including empty traces, because trace contents are unrestricted by this operation.

> Source, proof and provenance: V3 crossing_cut, sft/mathematics/graph_network/law.py:65; V3 internal_balance, sft/mathematics/graph_network/law.py:73. Native results: SFT.Mathematics.GraphOperations.balancedPairRequiresFreshTokens, GraphPartitionBalance.lean:108; SFT.Mathematics.GraphOperations.firstTokenPairBalanced, GraphPartitionBalance.lean:118. Timeline: Turn 109, line 1506. Evidence: Completion TURN_118_SFT-MATH-GRAPH-NETWORK-001_COMPLETION.

69. Batch source-program correspondence

In the recorded forty-four-program verification batch, native execution reproduces the pinned V3 registration fields, complete candidate censuses, all 23,910 actual identifier-based decisions, closure and generality records, original control results and every embedded digest. The complete classification comparison across all 44 represented entries covers 23,910 candidates; Exact Operations and Half-One include their complete source program adapters. The batch contains 3,278 theorem proofs and 6,394 audited definitions/theorems with zero axiom dependencies. These are the recorded batch totals, not the cumulative totals of the growing formalisation.

Fold Assembly combines the general finite-support, uniqueness, transition, return and proposal-classification proofs with exact execution of its original 192-candidate program. This completes the registered assembly entry after its Count and Fold dependencies. Completion is recorded separately from output parity alone. The Form adapters preserve tuple and list edge inputs; the protocol adapter preserves the original operand-returning behavior for literal host records and proves its Boolean specialization.

> Source, proof and provenance: V3 has_identity, sft/mathematics/algebraic_structures/law.py:35; V3 arrangements, sft/mathematics/combinatorics/law.py:12; V3 generated_collection, sft/mathematics/discrete_mathematics/law.py:14; V3 path_is_valid, sft/mathematics/graph_network/law.py:28; V3 is_partial_order, sft/mathematics/order_lattice/law.py:12. Native results: SFT.Mathematics.DiscreteEvidence.registrationHasNoFreeParameters, DiscreteEvidence.lean:83; SFT.Mathematics.DiscreteEvidence.lookupDependsOnlyOnIdentifier, DiscreteEvidence.lean:86; SFT.Mathematics.CombinatoricsEvidence.registrationHasNoFreeParameters, CombinatoricsEvidence.lean:83; SFT.Mathematics.CombinatoricsEvidence.lookupDependsOnlyOnIdentifier, CombinatoricsEvidence.lean:86; SFT.Mathematics.AlgebraEvidence.registrationHasNoFreeParameters, AlgebraEvidence.lean:83; SFT.Mathematics.AlgebraEvidence.lookupDependsOnlyOnIdentifier, AlgebraEvidence.lean:86; SFT.Mathematics.OrderEvidence.registrationHasNoFreeParameters, OrderEvidence.lean:83; SFT.Mathematics.OrderEvidence.lookupDependsOnlyOnIdentifier, OrderEvidence.lean:86; SFT.Mathematics.GraphEvidence.registrationHasNoFreeParameters, GraphEvidence.lean:83; SFT.Mathematics.GraphEvidence.lookupDependsOnlyOnIdentifier, GraphEvidence.lean:86. Timeline: Turn 056, line 832; Timeline: Turn 069, line 1011; Timeline: Turn 085, line 1210; Timeline: Turn 095, line 1331; Timeline: Turn 106, line 1471. Evidence: Completion TURN_118_SFT-MATH-DISCRETE-001_COMPLETION; Completion TURN_118_SFT-MATH-COMBINATORICS-001_COMPLETION; Completion TURN_118_SFT-MATH-ALGEBRA-001_COMPLETION; Completion TURN_118_SFT-MATH-ORDER-LATTICE-001_COMPLETION; Completion TURN_118_SFT-MATH-GRAPH-NETWORK-001_COMPLETION.

Program correspondence sources and proofs: V3 candidate_records, mathematics/generated_law.py:L117; V3 registration, mathematics/generated_law.py:L230; V3 generate_candidates, mathematics/generated_law.py:L245; V3 decide_candidate, mathematics/generated_law.py:L264; V3 closure_evidence, mathematics/generated_law.py:L285; V3 control_records, mathematics/generated_law.py:L187; AlgebraGrammar.generatedCardinality, L127; AlgebraGrammar.candidatesComplete, L129; AlgebraGrammar.acceptanceIffExactCriteria, L184; AlgebraEvidence.registrationData, L17; AlgebraEvidence.censusData, L37; AlgebraEvidence.decisionData, L46; AlgebraEvidence.closureData, L61; CombinatoricsGrammar.generatedCardinality, L116; CombinatoricsGrammar.candidatesComplete, L118; CombinatoricsGrammar.acceptanceIffExactCriteria, L170; CombinatoricsEvidence.registrationData, L17; CombinatoricsEvidence.censusData, L37; CombinatoricsEvidence.decisionData, L46; CombinatoricsEvidence.closureData, L61; DiscreteGrammar.generatedCardinality, L116; DiscreteGrammar.candidatesComplete, L118; DiscreteGrammar.acceptanceIffExactCriteria, L170; DiscreteEvidence.registrationData, L17; DiscreteEvidence.censusData, L37; DiscreteEvidence.decisionData, L46; DiscreteEvidence.closureData, L61; GraphGrammar.generatedCardinality, L127; GraphGrammar.candidatesComplete, L129; GraphGrammar.acceptanceIffExactCriteria, L184; GraphEvidence.registrationData, L17; GraphEvidence.censusData, L37; GraphEvidence.decisionData, L46; GraphEvidence.closureData, L61; OrderGrammar.generatedCardinality, L127; OrderGrammar.candidatesComplete, L129; OrderGrammar.acceptanceIffExactCriteria, L184; OrderEvidence.registrationData, L17; OrderEvidence.censusData, L37; OrderEvidence.decisionData, L46; OrderEvidence.closureData, L61.

70. Complete Part Equivalence source correspondence

The registered common-refinement entry combines the general nested pair-cell, complete lifting, constructive bijection and uniform-refinement proofs with all 1,024 original witness forms. Its complete native program reproduces every registered candidate and actual identifier-based decision, closure/generality certificate, control record and digest. The original tampered-artifact predicate counts accepted candidates in the complete enumeration and accepts the specified equal lifted traces; the unequal example is rejected. The three non-hash control outcomes have native proofs. Registration retains only the original One, Count and Part dependencies and introduces no axioms or free scientific parameters. Together these results complete the registered Part Equivalence entry.

For ordered source lists A,B, write P(A,B) = concat_(a in A)[label(a,b): b in B]. The lifted selections are L_A = P(A_held,B_all) and L_B = P(A_all,B_held). The source generates index pairs by zipping [0,...,|L_A|-1] with [0,...,|L_B|-1]. Every generated index is bounded, so its acceptance condition reduces exactly to

$$\mathrm{hasGeneratedBijection}(L_A, L_B) = \mathrm{true} \iff |L_A| = |L_B|.$$

With equal lengths n, the generated pairs are [(i,i): 0 ≤ i < n]; every left and right index occurs. The forward and backward maps satisfy g(f(i)) = i and f(g(j)) = j. For a uniform descendant type C, the refined maps (a,c) ↦ (f(a),c) and (b,c) ↦ (g(b),c) retain both inverse identities. On the ordered source lists, induction over A and distribution of length over append prove that equal source lengths give equal lengths of P(A,C), hence preserve the generated pairing. Origin cells satisfy (a,b) ∈ originCells(A,B) iff a ∈ A and b ∈ B; labels retain this origin witness without requiring label injectivity.

The original equal-selection control has lifted lengths 1·4 = 2·2 = 4. The unequal control has lengths 1·3 = 3 and 2·1 = 2, so the same predicate rejects it. The complete grammar retains its eight binary choices and one four-way pairing choice, giving 2^8·4 = 1,024 forms.

> Source, proof and provenance: V3 has_generated_bijection, sft/foundation/part_equivalence.py:98. Native results: SFT.Foundation.CommonRefinement.equalSourceControl, CommonRefinement.lean:238; SFT.Foundation.CommonRefinement.unequalSourceControl, CommonRefinement.lean:242; SFT.Foundation.RefinementSelection.liftedPairingOneToOne, RefinementSelection.lean:48; SFT.Foundation.RefinementSelection.liftedPairingOnto, RefinementSelection.lean:55; SFT.Foundation.PartEquivalence.pairingOntoOnlyRejected, PartEquivalence.lean:289; SFT.Foundation.PartEquivalence.extraHasExtraRejected, PartEquivalence.lean:298. Timeline: Turn 017, line 245; Timeline: Turn 018, line 262; Timeline: Turn 018, line 260. Evidence: Completion TURN_116_PART_EQUIVALENCE_COMPLETION.

71. Finite Form source input correspondence

The source validator requires a tuple at a Form root and accesses either tuple or list containers at the held edges. The formalisation retains both containers and their mixed combinations, arbitrary finite child forms and ignored trailing edge items. Its acceptance equivalence characterizes precisely the finite One/Fold productions over this representation. Original short-circuit order preserves false returns and index errors, including distinct tuple and list error messages. Canonicalization validates before recursively emitting the ordered trace, and every accepted represented input has a generated canonical Form. Canonical equality retains the established identity and equivalence laws. Fresh native execution reproduces all 288 Grammar and 192 Enforcement candidate decisions, complete evidence values and hashes. These results complete both registered entries in dependency order.

Let T denote a tuple constructor and E range independently over tuple or list constructors at each edge. The finite acceptance productions are

$$\frac{}{\mathrm{Accepted}(T[\mathrm{One}])}, \qquad \frac{\mathrm{Accepted}(l) \quad \mathrm{Accepted}(r)}{\mathrm{Accepted}(T[\mathrm{Fold}, E_L[\text{held-a}, l, \ldots], E_R[\text{held-b}, r, \ldots], \mathrm{return}])}.$$

The trailing items at each edge are arbitrary and ignored, while the outer tuple has exactly the displayed length. Case analysis on the validator's ordered tests extracts one of these productions from a true result. Induction on a production proves the converse, retaining all four tuple/list edge combinations:

$$\mathrm{valid}(v) = \mathrm{ok}(\mathrm{true}) \iff \mathrm{Accepted}(v).$$

Validation errors propagate unchanged; a false validation returns the source's outside-grammar error before canonical recursion. For accepted inputs the canonical trace C obeys

$$C(T[\mathrm{One}]) = T[\mathrm{One}], \qquad C(\mathrm{FoldRaw}(l, r)) = T[\mathrm{Fold}, \text{held-a}, C(l), \text{held-b}, C(r), \mathrm{return}].$$

Here FoldRaw denotes any of the four admitted edge-container combinations above. Induction on the acceptance production constructs f = One in the base case and f = Fold(f_l,f_r) in the recursive case. Thus valid(v) = ok(true) implies the existence of a finite Form f with canonical(v) = ok(canonicalTrace(f)). This derivation preserves both child order and error short-circuiting.

> Source, proof and provenance: V3 valid_form, sft/foundation/form_grammar.py:36. Native results: SFT.Foundation.FormConstruction.noExtraProduction, FormConstruction.lean:75; SFT.Foundation.FormConstruction.encodingInjective, FormConstruction.lean:80; SFT.Foundation.FormValidation.missingListLabelErrors, FormValidation.lean:176; SFT.Foundation.FormValidation.missingListChildErrors, FormValidation.lean:179; SFT.Foundation.FormAcceptance.validOfAccepted, FormAcceptance.lean:168; SFT.Foundation.FormAcceptance.validityIffFiniteProduction, FormAcceptance.lean:192. Timeline: Turn 039, line 577; Timeline: Turn 040, line 590; Timeline: Turn 041, line 605. Evidence: Completion TURN_117_SFT-FOUNDATION-FORM-GRAMMAR-001_COMPLETION.

72. Completed mathematics source entries

Exact Arithmetic, Discrete Objects, Combinatorics, Algebra, Order and Lattice, Graph and Network, Geometry and Topology, Probability and Statistics, Optimization, Dynamical Systems, Logic and Proof, and Category/Type/Composition combine their retained source-mapped derivations, complete candidate grammars, original operational witnesses and controls with passing full-program comparisons. Their registered dependencies are complete. These twelve mathematics entries form part of the completed census reported in the model and scope section; their native proofs have zero axiom dependencies. The source-component and executable evidence records distinguish these completed entries from the remaining translated portions.

> Source, proof and provenance: V3 has_identity, sft/mathematics/algebraic_structures/law.py:35; V3 generated_collection, sft/mathematics/discrete_mathematics/law.py:14; V3 trace, sft/mathematics/exact_arithmetic/law.py:13; V3 path_is_valid, sft/mathematics/graph_network/law.py:28; V3 is_partial_order, sft/mathematics/order_lattice/law.py:12. Native results: SFT.Mathematics.ExactTraceArithmetic.positivePairCells, ExactTraceArithmetic.lean:130; SFT.Mathematics.ExactTraceArithmetic.remainderTags, ExactTraceArithmetic.lean:132; SFT.Mathematics.DiscreteEvidence.registrationHasNoFreeParameters, DiscreteEvidence.lean:83; SFT.Mathematics.DiscreteEvidence.lookupDependsOnlyOnIdentifier, DiscreteEvidence.lean:86; SFT.Mathematics.AlgebraEvidence.registrationHasNoFreeParameters, AlgebraEvidence.lean:83; SFT.Mathematics.AlgebraEvidence.lookupDependsOnlyOnIdentifier, AlgebraEvidence.lean:86; SFT.Mathematics.OrderEvidence.registrationHasNoFreeParameters, OrderEvidence.lean:83; SFT.Mathematics.OrderEvidence.lookupDependsOnlyOnIdentifier, OrderEvidence.lean:86; SFT.Mathematics.GraphEvidence.registrationHasNoFreeParameters, GraphEvidence.lean:83; SFT.Mathematics.GraphEvidence.lookupDependsOnlyOnIdentifier, GraphEvidence.lean:86. Timeline: Turn 045, line 669; Timeline: Turn 056, line 832; Timeline: Turn 085, line 1210; Timeline: Turn 095, line 1331; Timeline: Turn 106, line 1471. Evidence: Completion TURN_118_SFT-MATH-EXACT-ARITHMETIC-001_COMPLETION; Completion TURN_118_SFT-MATH-DISCRETE-001_COMPLETION; Completion TURN_118_SFT-MATH-ALGEBRA-001_COMPLETION; Completion TURN_118_SFT-MATH-ORDER-LATTICE-001_COMPLETION; Completion TURN_118_SFT-MATH-GRAPH-NETWORK-001_COMPLETION.

Program correspondence sources and proofs: V3 candidate_records, mathematics/generated_law.py:L117; V3 registration, mathematics/generated_law.py:L230; V3 generate_candidates, mathematics/generated_law.py:L245; V3 decide_candidate, mathematics/generated_law.py:L264; V3 closure_evidence, mathematics/generated_law.py:L285; V3 control_records, mathematics/generated_law.py:L187; AlgebraGrammar.generatedCardinality, L127; AlgebraGrammar.candidatesComplete, L129; AlgebraGrammar.acceptanceIffExactCriteria, L184; AlgebraEvidence.registrationData, L17; AlgebraEvidence.censusData, L37; AlgebraEvidence.decisionData, L46; AlgebraEvidence.closureData, L61; DiscreteGrammar.generatedCardinality, L116; DiscreteGrammar.candidatesComplete, L118; DiscreteGrammar.acceptanceIffExactCriteria, L170; DiscreteEvidence.registrationData, L17; DiscreteEvidence.censusData, L37; DiscreteEvidence.decisionData, L46; DiscreteEvidence.closureData, L61; GraphGrammar.generatedCardinality, L127; GraphGrammar.candidatesComplete, L129; GraphGrammar.acceptanceIffExactCriteria, L184; GraphEvidence.registrationData, L17; GraphEvidence.censusData, L37; GraphEvidence.decisionData, L46; GraphEvidence.closureData, L61;

OrderGrammar.generatedCardinality, L127; OrderGrammar.candidatesComplete, L129; OrderGrammar.acceptanceIffExactCriteria, L184; OrderEvidence.registrationData, L17; OrderEvidence.censusData, L37; OrderEvidence.decisionData, L46; OrderEvidence.closureData, L61.

73. Completion of the represented foundation and mathematics entries

The initial 21 represented entries and Geometry/Topology are complete against their registered V3 source content. The six-entry verification batch covers Measurement Boundary, Exact Operations, Half-One, Fold Dynamics, Primitive Map Uniqueness and Derivation Trace, reproducing all 980 identifier-based candidate decisions and every original registration, census, closure, control and digest field. The complete operational and half-One programs retain the original 128-form enumerations and original dependency lists. Their program correspondence accompanies the previously established general positive-domain and first-Fold derivations.

The measurement protocol preserves Python conjunction semantics on its literal host records: truth testing controls evaluation, a false all-rows operand is returned unchanged, and an admitted source operand is returned unchanged. A nonempty source identifier therefore remains that identifier. Boolean inputs agree with the original Boolean protocol theorem. Constructive integer cases distinguish zero from positive and negative host integers without axioms. These host values are external protocol data, not added scientific parameters.

These foundation and mathematics batches contribute to the cumulative completed census; their batch sizes are not the total coverage of the growing formalisation.

Write t(x) for host truth testing. For Boolean protocol fields p (target before seal), m (measurement changes law), r (all rows) and s (identified source), the exact acceptance theorem is

$$A(p,m,r,s) = \neg p \wedge \neg m \wedge r \wedge s, \qquad A(p,m,r,s) = \text{true} \iff p = \text{false} \wedge m = \text{false} \wedge r = \text{true} \wedge s = \text{true}.$$

For general literal host values, define x & y = y when t(x) is true and x otherwise. The source expression is ((¬t(p) & ¬t(m)) & r) & s, where negated truth values are Boolean host values. Splitting on the first two truth tests gives the exact operand-returning equation

$$A_{\text{host}}(p,m,r,s) = \begin{cases} \text{false}, & t(p) \vee t(m), \\ r, & \neg t(p) \wedge \neg t(m) \wedge \neg t(r), \\ s, & \neg t(p) \wedge \neg t(m) \wedge t(r). \end{cases}$$

Truth testing is false for null, zero, empty text and empty arrays/objects; it is true for nonzero integers and nonempty containers/text, and agrees with a Boolean's value. Consequently A_host(false,false,true,"source-id") = "source-id" and A_host(false,false,[],"source-id") = []. Restricting all four operands to Booleans recovers A by exhaustive Boolean cases, without coercing arbitrary source identifiers to Boolean outputs.

> Source, proof and provenance: V3 form_name, sft/foundation/count.py:48; V3 candidate_records, sft/foundation/fold.py:48; V3 form_name, sft/foundation/one.py:42; V3 candidate_id, sft/foundation/part.py:53; V3 candidate_records, sft/foundation/root_theorem.py:50. Native results: SFT.Foundation.Root.Evidence.registrationAdapterPreservesRecord, RootEvidence.lean:60; SFT.Foundation.Root.Evidence.typedControlResultMatchesRecord, RootEvidence.lean:139; SFT.Foundation.One.Evidence.tamperedArtifactPassed, OneEvidence.lean:121; SFT.Foundation.One.Evidence.boundaryPassed, OneEvidence.lean:123; SFT.Foundation.Count.Evidence.tamperedArtifactPassed, CountEvidence.lean:123; SFT.Foundation.Count.Evidence.boundaryPassed, CountEvidence.lean:125; SFT.Foundation.Part.Evidence.tamperedArtifactPassed, PartEvidence.lean:128; SFT.Foundation.Part.Evidence.boundaryPassed, PartEvidence.lean:130; SFT.Foundation.Fold.Evidence.runControls, FoldEvidence.lean:87. Timeline: Turn 111, line 1519; Timeline: Turn 112, line 1525; Timeline: Turn 112, line 1528;

> Timeline: Turn 113, line 1535; Timeline: Turn 114, line 1542. Evidence: Completion TURN_111_ROOT_COMPLETION; Completion TURN_112_ONE_COMPLETION; Completion TURN_112_COUNT_COMPLETION; Completion TURN_113_PART_COMPLETION; Completion TURN_114_FOLD_COMPLETION.

74. Finite incidence and open-family derivations

The incidence predicate accepts exactly a nonempty distinct cell carrier and a distinct boundary relation whose endpoints belong to that carrier and whose pairs are not self-incidences. The adjacency predicate, which does not itself impose the incidence admission guard, accepts exactly two different cells with a shared retained face. General native proofs establish this equivalence, symmetry and irreflexivity.

Each proposed open is projected onto carrier order and an empty projection is represented by the original empty-One literal. The finite topology check retains the original canonical-set cardinality comparison, presence of empty One and the whole carrier, and exhaustive pairwise union/intersection closure. Its acceptance theorem states precisely these conditions. Structural induction then establishes membership of every finite generated join and intersection of admitted canonical opens. Off-carrier labels and collisions with the literal empty-One marker retain their original executable behavior.

Continuity constructs its image map with the last supplied row for each key, requires exactly the source point keys, and tests each target open's source-ordered inverse image against the supplied source open family. No extra topology validation or normalization of the supplied source family is inserted. Native proofs establish the exact key-support and inverse-image conditions and last-row precedence. The original incidence, adjacency, finite-topology and continuity witnesses hold. The path-search operation, returned-path soundness, general minimality and unreachable-case equivalence are established below.

For a carrier list C and boundary list B, the admission and adjacency equivalences are

$$\begin{aligned}\mathrm{Inc}(C,B) &\iff C \neq [] \wedge \mathrm{Distinct}(C) \wedge \mathrm{Distinct}(B)\\ &\quad \wedge \forall (a,b) \in B,\ a \in C \wedge b \in C \wedge a \neq b,\\ \mathrm{Adj}_B(x,y) &\iff x \neq y \wedge \exists f,\ (f,x) \in B \wedge (f,y) \in B.\end{aligned}$$

The ordered face list is F_B(c) = [a : (a,c) occurs in B]. Swapping the two face-membership witnesses proves adjacency symmetry; x ≠ x excludes self-adjacency. Neither implication inserts an incidence guard into Adj.

Let e denote the original empty-One literal, and write E(L) = e if L = [], otherwise L. For a point list P, define

$$\begin{aligned}N_P(U) &= E([x \in P \mid x \in U]),\\ U \vee_P V &= E([x \in P \mid x \in U \vee x \in V]),\\ U \wedge_P V &= E([x \in P \mid x \in U \wedge x \in V]),\\ K &= \mathrm{distinctWords}([N_P(U) : U \in \mathcal{O}]).\end{aligned}$$

These are ordered-list operations with the source literal e, including its original collision behavior. The topology predicate is exactly

$$P \neq [] \wedge \mathrm{Distinct}(P) \wedge |K| = |\mathcal{O}| \wedge e \in K \wedge P \in K \wedge \forall U,V \in K,\ U \vee_P V \in K \wedge U \wedge_P V \in K.$$

The generated join has J([])=e and J(U::R)=U∨_P J(R); the generated intersection has I([])=P and I(U::R)=U∧_P I(R). For a list R of members of K, induction begins with e∈K or P∈K and applies the displayed binary closure at each cons, proving J(R),I(R)∈K.

For a supplied mapping list M, f_M(x) is the last value whose key is x, if present. Hence f_(M++[(x,y)])(x)=y. Define the inverse-open list by V_M(U)=E([x∈P : f_M(x) is present and belongs to U]). Then the source continuity predicate is precisely

$$\mathrm{keys}(M) \subseteq P \ \wedge \ P \subseteq \mathrm{keys}(M) \ \wedge \ \forall U \in \mathcal{O}_{\mathrm{target}},\ V_M(U) \in \mathcal{O}_{\mathrm{source}}.$$

The inclusions express list membership, so duplicate mapping keys remain admitted and use last-row precedence. The supplied source family is tested directly.

> Source, proof and provenance: V3 incidence_is_well_formed, sft/mathematics/geometry_topology/law.py:14; V3 is_finite_topology, sft/mathematics/geometry_topology/law.py:68; V3 is_continuous, sft/mathematics/geometry_topology/law.py:86. Native results: SFT.Mathematics.GeometryTopology.sourceTopologyWitness, GeometryTopology.lean:300; SFT.Mathematics.GeometryTopology.sourceContinuityWitness, GeometryTopology.lean:302. Timeline: Turn 120, line 1583. Evidence: Completion TURN_124_GEOMETRY_COMPLETION.

75. Boundary-depth dimension

For a registered finite incidence complex, the dimension function returns one for a cell with no faces and one plus the maximum of the recursively derived face dimensions otherwise. The active descent stack rejects a return to a cell already on that stack. Independent branches share the parent stack, preserving the original push/pop behavior; an unrelated cycle outside the queried descent does not cause rejection. Termination follows constructively by removing each newly visited label from the finite support. This support introduces no depth bound or scientific parameter.

A native inductive derivation records every face in source order, its own complete derivation and the maximum rule at the parent. The successful computation is equivalent in both directions to this derivation for every admitted input and result. All successful dimensions are positive; the original carrier-admission and return-cycle error conditions are retained. Execution reproduces the original triangle dimension of three and edge dimension of two, together with leaf, invalid-carrier, reachable-cycle, unrelated-cycle and shared-descendant cases.

With visiting stack S and ordered faces F_B(c)=[f_1,...,f_k], successful depth evaluation follows

$$d_S(c) = \begin{cases} \mathrm{error}(\text{return cycle}), & c \in S, \\ 1 + \max(0, d_{S{+}{+}[c]}(f_1), \ldots, d_{S{+}{+}[c]}(f_k)), & c \notin S, \end{cases}$$

where any child error is propagated in source order before a numerical maximum is returned. The maximum of an empty child list is zero, giving a leaf dimension of one. Every child receives S++[c], rather than the final stack from a sibling branch.

For successful children with ordered heights h_1,...,h_k, the native derivation rule is

$$\frac{c \notin S \qquad D_B(S{+}{+}[c], f_1, h_1) \quad \cdots \quad D_B(S{+}{+}[c], f_k, h_k)}{D_B(S, c, 1 + \max(0, h_1, \ldots, h_k))}.$$

The auxiliary support initially contains c followed by all first endpoints of B. At a fresh node, removing every occurrence of c strictly decreases its length; each child remains covered by unseen support or the visiting stack. Well-founded induction therefore yields both directions

$$\mathrm{boundaryDepth}(B, S, c) = \mathrm{ok}(h) \iff D_B(S, c, h).$$

Here boundaryDepth denotes a state carrying the stated support-coverage invariant. Under Inc(C,B) and c∈C, the public dimension call starts with S=[] and has the same equivalence. Every successful h is a successor and thus h>0. In the original witness, p,q,r have no faces and dimension 1; pq and qr have dimension 1+max(1,1)=2; triangle has dimension 1+max(2,2)=3.

> Source, proof and provenance: V3 cell_dimension, sft/mathematics/geometry_topology/law.py:20. Native results: SFT.Mathematics.GeometryTopology.boundaryDepthIffDerivation, GeometryDimension.lean:224; SFT.Mathematics.GeometryTopology.dimensionIffDerivation, GeometryDimension.lean:228. Timeline: Turn 121, line 1593. Evidence: Completion TURN_124_GEOMETRY_COMPLETION.

76. Ordered undirected path search

The path-distance procedure checks only membership of its two endpoints in the supplied carrier. Equal endpoints return the original empty-One value. For different endpoints, the queue stores both each held node and its full path. A newly visited node scans the original edge list in order, extending only to unvisited neighbors; the first target extension is returned. Pending extensions retain duplicate occurrences and source order. Edge labels outside the supplied carrier remain traversable, as in the source procedure.

The formal implementation has a constructive termination proof over unseen edge-endpoint support and queue length. Its extension decision is proved equivalent to the literal early-return edge scan. Every returned nonidentity path has a native inductive trace beginning at the origin and extended by original undirected incidences to the target. Endpoint rejection, empty-One identity, exhausted queue, visited-node skipping and fresh expansion are explicit native laws. Source-case execution preserves reversed-edge traversal, off-carrier intermediate labels, self-edges, duplicate queue entries and edge-order tie selection. Exhaustive checks on all 64 directed edge subsets over three labels and all 576 endpoint pairs agree with independent enumeration of simple paths for reachability and minimal length. The finite checks accompany the general shortest-path and unreachable-case theorems below.

For each source edge (a,b), the neighbor of u is b when a=u, otherwise a when b=u, and absent otherwise. Let N_E(u,V) be the resulting ordered neighbor list after removing members of V; duplicate occurrences remain. For queue head (u,p), tail Q and visited list V, put V'=V++[u]. The loop equations are

$$\begin{aligned} L(V,[]) &= \mathrm{None}, \\ L(V,(u,p)::Q) &= L(V,Q) && (u \in V), \\ L(V,(u,p)::Q) &= \mathrm{Some}(p++[t]) && (u \notin V,\ t \in N_E(u,V')), \\ L(V,(u,p)::Q) &= L\big(V', Q++[(v,p++[v]) : v \in N_E(u,V')]\big) && (u \notin V,\ t \notin N_E(u,V')). \end{aligned}$$

The initial queue is [(o,[o])]. The unseen support initially contains o and both endpoint lists of E. Skipping a visited head reduces queue length; fresh expansion removes its node from unseen support. Thus (|unseen|,|queue|) strictly decreases lexicographically on every recursive call.

Returned-path soundness uses the constructors

$$\frac{}{T_E(o,o,[o])}, \qquad \frac{T_E(o,u,p) \quad ((u,v) \in E \vee (v,u) \in E)}{T_E(o,v,p++[v])}.$$

Every queued extension has this trace, so a target extension does too. The public endpoint guard is o,t∈C; it returns an error otherwise, returns Some(e) for o=t, and invokes L for distinct endpoints. This guard imposes no carrier-membership test on intermediate edge endpoints.

> Source, proof and provenance: V3 path_distance, sft/mathematics/geometry_topology/law.py:45. Native results: SFT.Mathematics.GeometryTopology.scanNeighborsDecision, GeometryPaths.lean:369; SFT.Mathematics.GeometryTopology.sourceScanDecision, GeometryPaths.lean:383. Timeline: Turn 122, line 1601. Evidence: Completion TURN_124_GEOMETRY_COMPLETION.

77. General shortest-path and reachability derivations

The search queue preserves nondecreasing path lengths and a width of at most one edge. When a node is visited, every adjacent node that remains unvisited has a queued route bounded by any path to that visited node followed by the edge. This invariant, together with the retained origin route, supplies a frontier route no longer than any proposed trace to an unvisited endpoint. Since the target is returned upon discovery and is never queued, the final extension is no longer than any alternative trace to that target.

For every finite source edge list and distinct admitted endpoints, every undirected trace therefore yields a successful returned path of no greater length. Combined with returned-path soundness, this proves minimality of the actual result against all finite undirected alternatives. The procedure returns None if and only if no such trace exists. The proof retains the original carrier guard and does not require unique edges, globally admitted edge endpoints, or exclusion of loops. Equal endpoints retain the separate source identity result, empty One. These statements are general native proofs, with zero axiom dependencies; they do not depend on a finite regression bound.

For a queue Q, define B_Q(v,n) to mean that some (v,p)∈Q has |p|≤n. The preserved frontier conditions are

$$o \in V \vee B_Q(o,1), \qquad \forall u \in V,\ T_E(o,u,p) \wedge (u \sim_E v) \Rightarrow v \in V \vee B_Q(v,|p|+1),$$

where u~_E v means either orientation occurs in E. Queue order requires each head path to be no longer than every later path; queue width requires |p_2|≤|p_1|+1 for every two queued paths. Skipping a visited head preserves frontier coverage. Fresh expansion supplies each still-unvisited neighbor with the extended head path and retains the old queue tail; the width bound ensures the appended extensions preserve order.

Induction on T_E(o,v,p) gives a queued path of length at most |p| whenever v∉V. If v is also absent from the queue, the stronger bound is |q|+1≤|p|: the last trace edge either has a visited predecessor, which would force v to be queued, or has an unvisited predecessor to which the induction bound applies. The target is never queued because it is returned immediately upon discovery. At that return, queue order gives

$$|p_{\text{head}} + +[t]| = |p_{\text{head}}| + 1 \le |q| + 1 \le |p_{\text{alternative}}|.$$

Consequently, for distinct admitted endpoints, every alternative trace yields a successful result r with |r| ≤|p_alternative|. Determinism identifies r with any already returned result, proving minimality. Finally,

$$\text{pathDistance}(C,E,o,t) = \text{ok}(\text{None}) \iff \neg\exists p,\ T_E(o,t,p) \qquad (o,t \in C,\ o \neq t).$$

A trace contradicts None by completeness; a successful result contradicts absence of traces by soundness. These implications quantify over arbitrary finite edge lists and finite alternative traces.

> Source, proof and provenance: V3 path_distance, sft/mathematics/geometry_topology/law.py:45. Native results: SFT.Mathematics.GeometryTopology.pathDistanceComplete, GeometryShortest.lean:291; SFT.Mathematics.GeometryTopology.pathDistanceShortest, GeometryShortest.lean:302; SFT.Mathematics.GeometryTopology.pathDistanceNoneIff, GeometryShortest.lean:312. Timeline: Turn 123, line 1609. Evidence: Completion TURN_124_GEOMETRY_COMPLETION.

78. Completed finite geometry and topology entry

The ten registered binary dimensions generate all 1,024 kernels in the original order. Native proofs establish product membership, exact cardinality, agreement of identifier acceptance with the original structural criteria, the unique admitted kernel, and rejection upon replacement of each required coordinate. All six source operational witnesses are evaluated by the translated operations; the original depth and path witnesses are additionally obtained from the general native derivations and source queue equations. The closure evidence retains the original five laws, induction base and successor, exclusions and control records. The connectedness coordinate requires that no two nonempty disjoint admitted opens jointly exhaust the carrier. The deformation coordinate requires a retained lawful reverse for each elementary change and preservation of the registered incidence invariants. In the source product these are the admitted alternatives named no-disjoint-open-separation and reversible-incidence-trace. The native grammar retains these exact choices, and its acceptance theorem requires both together with the other eight coordinates. The separate operational proofs for this classification are the incidence, depth, path, open-family and inverse-image computations described in Sections 74–77.

Fresh native execution reproduces every original registration, census, decision, closure, control and digest field, including all 1,024 identifier-based decisions and refusal of an unknown identifier. Together with the general incidence, dimension, adjacency, shortest-path, topology and continuity derivations, these results complete the registered Geometry/Topology entry. This entry is included in the cumulative completed census reported in the model and scope section.

Writing each registered coordinate as admitted or rejected, the full geometry grammar is the ordered product of the ten two-element coordinate lists. Thus

$$|\mathcal{G}| = \prod_{i=1}^{10} 2 = 1,024, \qquad \mathrm{accept}(c) = \mathrm{true} \iff \bigwedge_{i=1}^{10} (c_i = \mathrm{admitted}) \iff c = c_*.$$

The coordinates are cells, incidence, dimension, adjacency, distance, opens, connectedness, continuity, deformation and addition. Product membership is proved by selecting each coordinate in its original two-element list. Conjunction elimination proves rejection if any coordinate is replaced by its rejected alternative; conjunction introduction proves acceptance of c_*. These enumeration results accompany the operational derivations in the preceding sections.

The original program constructs each candidate from the ordered Cartesian product of its declared choice lists, joins its coordinate names to form the identifier, and retains the coordinate/form record. Decision lookup first finds that exact identifier in the complete product, then compares it with the joined all-admitted identifier. A missing identifier therefore fails lookup. The reason scan stops at the first rejected coordinate in declared order. Census hashes retain the complete candidate record; closure hashes retain the closure record and ordered decisions; each control retains its own observed result and receipt hash. Registration uses the same specification and source identity. These are the particular program operations compared with their native records.

> Source, proof and provenance: V3 incidence_is_well_formed, sft/mathematics/geometry_topology/law.py:14. Native results: SFT.Mathematics.GeometryEvidence.registrationHasNoFreeParameters, GeometryEvidence.lean:83; SFT.Mathematics.GeometryEvidence.lookupDependsOnlyOnIdentifier, GeometryEvidence.lean:86. Timeline: Turn 124, line 1616. Evidence: Completion TURN_124_GEOMETRY_COMPLETION.

Program correspondence sources and proofs: V3 candidate_records, mathematics/generated_law.py:L117; V3 registration, mathematics/generated_law.py:L230; V3 generate_candidates, mathematics/generated_law.py:L245; V3 decide_candidate, mathematics/generated_law.py:L264; V3 closure_evidence, mathematics/generated_law.py:L285; V3 control_records, mathematics/generated_law.py:L187; GeometryGrammar.generatedCardinality, L138; GeometryGrammar.candidatesComplete, L140; GeometryGrammar.acceptanceIffExactCriteria, L198; GeometryEvidence.registrationData, L17; GeometryEvidence.censusData, L37; GeometryEvidence.decisionData, L46; GeometryEvidence.closureData, L61.

79. Exact events and common-refinement weights

An admitted event preserves its supplied order and is a duplicate-free held selection of a complete nonempty support. The native admission results establish support positivity and uniqueness, event uniqueness and inclusion, and the bound of held cardinality by support cardinality. Empty events retain the source empty-One marker. The original literal-marker comparison is preserved when a registered state itself has the label empty-One.

A nonmarker event weight is the reduced positive held-count/whole-count fraction. Native proofs establish exact cross-multiplication equality with the original counts, positive numerator, coprime numerator and denominator, and the through-One bound. Conditioning first validates the event and then the condition in the original order. Its common refinement selects exactly the support states belonging to both held sets; an empty refinement or marker condition returns empty One. Every fractional result is the exact

refinement-count/condition-count ratio and is bounded by the whole. No fitted quantity, stochastic premise or additional scientific parameter enters these operations. The observation-class construction and complete probability program are included below.

For support S and held list H, admission is exactly the conjunction S≠[], Distinct(S), Distinct(H), and H⊆S, with support checked before H. Its event result is e when H=[] and H otherwise, where e is the source empty-One literal. For nonmarker results, let a=|H|, b=|S|, g=gcd(a,b). Reduction gives

$$n = a/g, \qquad d = b/g, \qquad ad = nb, \qquad n > 0, \quad d > 0, \quad \gcd(n,d) = 1, \quad n \le d.$$

The cardinality bound a≤b is constructive: for H=x::R, remove x from S; distinctness ensures R is still contained in that remainder, induction bounds |R|, and removal strictly decreases |S|. Thus |H|≤|S|, and positive-fraction normalization preserves the through-One bound. The marker branch is tested on the returned event value itself, so a nonempty held list equal to e retains the original marker behavior.

For admitted event H and condition C, the common-refinement list is

$$R = [x \in S \mid x \in H \wedge x \in C], \qquad x \in R \iff x \in S \wedge x \in H \wedge x \in C.$$

After validating H and then C, the procedure returns e if the condition result equals e or R=[]. Otherwise it reduces |R|/|C| to n/d and proves

$$|R|d = n|C|, \qquad 0 < n \le d, \qquad \gcd(n,d) = 1.$$

Filtering preserves distinctness of S and gives R⊆C, so the same cardinality argument yields |R|≤|C|. The original first error is propagated before any later condition check.

> Source, proof and provenance: V3 exact_event, sft/mathematics/probability_statistics/law.py:14; V3 exact_weight, sft/mathematics/probability_statistics/law.py:22; V3 conditional_weight, sft/mathematics/probability_statistics/law.py:29. Native results: SFT.Mathematics.ProbabilityEvents.commonEventBound, ProbabilityEvents.lean:248; SFT.Mathematics.ProbabilityEvents.conditionalThroughOne, ProbabilityEvents.lean:255. Timeline: Turn 125, line 1625. Evidence: Completion TURN_127_PROBABILITY_COMPLETION.

80. Pair-cell factorization and exact modal summaries

The independence operation extracts marginal labels in their first-occurrence order and compares the supplied joint support with their complete ordered Cartesian product. The native product-filter identity establishes that retaining pair cells by both held-event predicates yields the Cartesian product of the retained marginal cells. Its cardinality is the product of the two retained marginal cardinalities. For duplicate-free held selections contained in their marginal supports, these cardinalities equal the supplied held counts; cross-multiplication therefore proves exact joint-weight factorization. The operational procedure retains the source rejection of incomplete or differently ordered products, its nonempty-event guard, raw event-list lengths and empty-joint division error.

The modal summary counts every occurrence in the complete trace, forms the greatest count over first-occurrence labels and retains every label attaining it in that order. Native proofs characterize membership exactly as trace membership with frequency at least that of every other trace label. Every nonempty trace has a modal label, the output contains no duplicate labels, and every tied alternative is retained. The empty trace retains the original rejection. These operations introduce no stochastic parameter.

Let A and B be the first-occurrence marginal lists extracted from joint J, and let P(A,B) list every (a,b) in outer-A, inner-B order. For held lists H,K, filter distribution gives the exact ordered identity

$$[(a,b) \in P(A,B) \mid a \in H \wedge b \in K] = P([a \in A \mid a \in H], [b \in B \mid b \in K]).$$

Induction over A proves the identity row by row; distributing length over append gives |P(A,B)|=|B|·|A|. When H,K are distinct and contained in their respective distinct supports, each filtered marginal has the held cardinality. Hence, for the complete ordered product J=P(A,B), with J_HK the retained joint list,

$$|J_{HK}| = |K|\,|H|, \qquad |J| = |B|\,|A|, \qquad |J_{HK}|\,|A|\,|B| = |H|\,|K|\,|J|.$$

For nonempty admitted events and joint support, this is the exact source cross-multiplication test for joint-weight factorization. The executable procedure still uses raw |H| and |K| outside these theorem hypotheses: it does not silently deduplicate them. Its preliminary ordered-product equality test also retains the original rejection of reordered joint lists.

For a trace X, define c_X(a)=|[x∈X:x=a]| and let L be its first-occurrence labels. The maximum recurrence and mode list are

$$m_X([]) = 0, \qquad m_X(a :: R) = \max(c_X(a), m_X(R)), \qquad M(X) = [a \in L \mid c_X(a) = m_X(L)].$$

Induction establishes that m_X(L) bounds every label count and is at most any common upper bound. Therefore

$$a \in M(X) \iff a \in X \wedge \forall b \in X,\ c_X(b) \le c_X(a).$$

For nonempty L, the recurrence attains its maximum either at its head or at the inductively selected tail label, proving existence of a mode. Filtering a distinct first-occurrence list preserves distinctness and order. If c_X(b)=c_X(a) for a retained a and b∈X, the same maximality inequality proves b is retained, including every tie.

> Source, proof and provenance: V3 independent, sft/mathematics/probability_statistics/law.py:40; V3 exact_mode, sft/mathematics/probability_statistics/law.py:56. Native results: SFT.Mathematics.ProbabilityIndependence.independentEmptyJoint, ProbabilityIndependence.lean:266; SFT.Mathematics.ProbabilityIndependence.independentCompleteDecision, ProbabilityIndependence.lean:273; SFT.Mathematics.ProbabilityModes.exactModeEmpty, ProbabilityModes.lean:127; SFT.Mathematics.ProbabilityModes.exactModeNonempty, ProbabilityModes.lean:129. Timeline: Turn 126, line 1635. Evidence: Completion TURN_127_PROBABILITY_COMPLETION.

81. Observation partitions and completed finite probability entry

An observation assigns each admitted support state its final dictionary value. Repeated mapping keys retain their last value, and admission requires equality of mapping-key and support sets. Native proofs establish that every support state has an image and that the image trace retains the support length. Distinct class labels are taken in first-occurrence order. Each class consists exactly of the support rows mapped to its label, retaining their original order and multiplicity. Every support state is covered, each emitted class is nonempty, and a state cannot belong to classes with different labels. Observation therefore closes distinctions between deterministic support states without adding a stochastic evolution law.

The complete nine-dimensional source product contains 512 candidate kernels. Native enumeration and criteria proofs establish the unique all-admitted kernel and rejection of each substituted coordinate. All six original operational witnesses evaluate successfully; their joint passing result is kernel checked. Native execution matches every source registration, census, decision, closure, control and digest field for all 512 identifier lookups, and preserves unknown-identifier refusal. The original induction, derivation exposition, correspondence terminology and empirical exclusions are retained. These results complete the V3 finite probability and descriptive-statistics entry. Inferential claims about natural measurements retain V3's requirement for preregistered blind validation through the empirical engine.

Write f_M(x) for the final mapping value at key x. Admission is the pair of membership inclusions keys(M)⊆S and S⊆keys(M). Induction on the mapping list proves

$$(\exists y,\ f_M(x) = \mathrm{Some}(y)) \iff x \in \mathrm{keys}(M).$$

Thus every admitted support row contributes exactly one value to the image trace Y, and induction on S gives |Y|=|S|. Define L=firstLabels(Y) and F_y=[x∈S:f_M(x)=Some(y)]. The emitted rows are [(y,F_y):y∈L], with exact membership law

$$(y, T) \in \mathrm{classRows}(S, M) \iff (\exists x \in S,\ f_M(x) = \mathrm{Some}(y)) \wedge T = F_y.$$

Every emitted row therefore has a witness x and is nonempty. For each x∈S, the admitted mapping supplies y and hence its emitted class F_y containing x. If x∈F_y and x∈F_z, then Some(y)=f_M(x)=Some(z), so y=z. This proves coverage and separation of differently labelled classes while retaining support order and any repeated support occurrences. The observation operation's key-set guard does not add a distinctness requirement to its supplied support list.

The nine binary probability coordinates give 2^9=512 kernels. The conjunction of their original admission criteria holds exactly at the all-admitted tuple; each rejected coordinate falsifies its corresponding conjunct. The finite classification and operational witness results accompany the general event, factorization, mode and partition derivations above.

> Source, proof and provenance: V3 observation_classes, sft/mathematics/probability_statistics/law.py:65. Native results: SFT.Mathematics.ProbabilityObservations.observationRejected, ProbabilityObservations.lean:179; SFT.Mathematics.ProbabilityObservations.emptyObservation, ProbabilityObservations.lean:186. Timeline: Turn 127, line 1642. Evidence: Completion TURN_127_PROBABILITY_COMPLETION.

82. Exact finite optimization and retained alternatives

The optimization carrier is nonempty and canonical. Its feasibility relation retains the final verdict for every candidate key and must cover exactly the carrier's keys. The feasible selection preserves candidate order and contains precisely the candidates with true verdicts. A preference cell must join two distinct feasible forms. The undominated selection retains every feasible occurrence with no incoming preference cell; a native theorem supplies a distinct feasible dominator for every excluded feasible form. No acyclicity or extra ordering assumption is imposed by the source procedure.

A unique optimum is returned exactly when the complete held list is a singleton. Every other result retains the original equivalence-class tag and full list, including ties, incomparable alternatives and an empty result from a preference cycle. Constraints require membership in every supplied allowed set. Pareto dominance requires every criterion to admit the directed comparison and at least one to witness strict preference; self-comparisons cannot eliminate candidates. Native frontier membership and elimination-witness results establish complete selection and retention of incomparable forms, including the original empty-criteria behavior.

The approximation certificate accepts the original signed-integer domain and rejects a nonpositive optimum or candidate count below that optimum. Every accepted result is the reduced positive optimum-count/candidate-count part. Native ratio and through-One results establish exactness without rounding, negative penalties or a tuning parameter. Shared-interface decomposition retains the source's complete-factorization condition; no additional decomposition algorithm is introduced.

All 512 kernels in the original nine-dimensional grammar are retained, with native enumeration, criteria, uniqueness and coordinate-rejection proofs. All six original operational witness computations pass. Full native program execution matches every pinned registration, candidate, decision, closure, control and digest field and the unknown-identifier error. These results complete the finite optimization entry; complexity and particular objective-family laws retain their original placement in the algorithms and complexity branches.

For canonical nonempty carrier C and verdict rows R, the key guard is keys(R)⊆C and C⊆keys(R). With v_R(x) the last verdict at x, feasibility is the ordered filter

$$F = [x \in C \mid v_R(x) = \mathrm{Some}(\mathrm{true})], \qquad v_{R{+}{+}[(x,b)]}(x) = \mathrm{Some}(b).$$

A preference list P is admitted exactly when every (a,b)∈P has a,b∈F and a≠b. For nonempty F, its retained list U obeys

$$x \in U \iff x \in F \wedge \forall y,\ (y,x) \notin P.$$

If x∈F but x∉U, the failed filter supplies a pair (y,x)∈P; preference admission then gives y∈F and y≠x. This witness need not itself be undominated: cyclic preferences retain their original possibility of U=[]. Classification returns unique(x) iff U=[x]; otherwise it returns the source retained-equivalence-class tag with U unchanged. Constraints [(name_i,A_i)] accept x iff x∈A_i for every supplied row.

For criteria Q=[(name_i,P_i)], the source dominance predicate is

$$D_Q(a,b) \iff [\forall i,\ (a,b) \in P_i \vee a = b] \wedge [\exists i,\ (a,b) \in P_i].$$

The Pareto filter separately excludes self-elimination and has the exact law

$$x \in \mathrm{Frontier}(F,Q) \iff x \in F \wedge \forall y \in F,\ y \neq x \Rightarrow \neg D_Q(y,x).$$

An excluded feasible x has a distinct feasible y satisfying D_Q(y,x), obtained from the source existential test. If each distinct competitor lacks the directed pair in at least one criterion, its universal dominance clause fails and x remains. For Q=[], the existential clause is false, so Frontier(F,[])=F.

The signed-integer approximation call accepts exactly 0<o≤c. For accepted counts, put g=gcd(o,c), n=o/g and d=c/g. The normalized part satisfies

$$od = nc, \qquad 0 < n \leq d, \qquad \gcd(n,d) = 1.$$

Negative inputs, o=0 and o>c take the original error branch; no rounding or selected numerical tolerance enters the ratio. The nine binary grammar coordinates enumerate 2^9=512 candidates; their criteria conjunction accepts only the all-admitted tuple, with each substituted rejected coordinate falsifying its own conjunct.

> Source, proof and provenance: V3 undominated, sft/mathematics/optimization/law.py:22; V3 exact_approximation, sft/mathematics/optimization/law.py:55. Native results: SFT.Mathematics.OptimizationSelection.exactOptimumResult, OptimizationSelection.lean:197; SFT.Mathematics.OptimizationSelection.constraintsMeaning, OptimizationSelection.lean:202; SFT.Mathematics.OptimizationBounds.approximationRejectsZero, OptimizationBounds.lean:143; SFT.Mathematics.OptimizationBounds.approximationRejectsInvalidBound, OptimizationBounds.lean:147. Timeline: Turn 128, line 1651. Evidence: Completion TURN_128_OPTIMIZATION_COMPLETION.

83. Generated trajectories and predecessor records

Finite dynamics admission requires a nonempty canonical state list, distinct retained transition cells and both endpoints of every cell inside the state carrier. Trajectory validation has the separate original domain: it requires a nonempty trace and each adjacent ordered pair in the supplied transition relation. It introduces no additional carrier-membership guard. Native membership results establish these exact conditions, and composition at a shared canonical interface is equivalent to validity of both component trajectories.

Elapsed time retains the complete ordered adjacent-transition trace. Its length is one fewer than the number of retained states; every trace with at least two states has positive transition count. A singleton trajectory has empty-One duration and an empty trajectory is rejected. Fixed forms retain precisely the source rows with identical source and target, while predecessor records retain every source row mapped to the requested target, preserving order and multiplicity.

Record-based reversibility requires inclusion of every original transition in a duplicate-free retained ledger. Native results show that each source predecessor remains recoverable from that ledger and that omission of any original transition fails the check. Extra retained rows remain governed by the original inclusion test; no stronger equality condition is imposed. First-return selection, registered-perturbation reachability and the complete dynamical program are included below.

Let S be the state list, E the ordered transition list, and A(p) the ordered adjacent pairs of trajectory p. The admission predicates are

$$\begin{aligned} \mathrm{Dynamics}(S,E) &\iff S \neq [] \wedge \mathrm{Distinct}(S) \wedge \mathrm{Distinct}(E) \wedge \forall (a,b) \in E,\ a,b \in S, \\ \mathrm{Trajectory}_E(p) &\iff p \neq [] \wedge \forall e \in A(p),\ e \in E. \end{aligned}$$

Thus [x] is a valid trajectory for any x, since its adjacent-pair list is empty. The empty trajectory is rejected. For lists L,R and common interface x,

$$A(L ++ [x] ++ R) = A(L ++ [x]) ++ A(x :: R).$$

Distributing universal membership over this append proves trajectory validity of the joined list iff both component trajectories are valid. This establishes composition without adding a state-carrier guard.

The time operation retains the actual adjacent transition list:

$$\mathrm{Time}([x]) = e, \qquad \mathrm{Time}(x :: y :: R) = (x,y) :: A(y :: R), \qquad |A(x :: R)| = |R|.$$

The final equation follows by induction on R: the empty tail gives zero and each cons contributes one adjacent pair. Hence a trajectory with at least two states has positive transition count. The source distinguishes the singleton marker e=["empty-One"] from a transition list; Time([]) returns its original error.

Filtering E before projecting its first component gives

$$\mathrm{Fixed}(E) = [a : (a,b) \in E,\ a = b], \qquad \mathrm{Pred}_E(t) = [a : (a,t) \text{ occurs in } E].$$

Therefore x∈Fixed(E) iff (x,x)∈E, and x∈Pred_E(t) iff (x,t)∈E, while the lists retain source order and repeated occurrences. For a retained ledger L, the exact reversibility test is

$$\mathrm{Reversible}(E,L) \iff E \subseteq L \wedge \mathrm{Distinct}(L).$$

If (x,t)∈E and the test passes, inclusion gives (x,t)∈L and hence x∈Pred_L(t). If an original edge is missing from L, inclusion is false and the test fails. Extra ledger rows are not excluded by this predicate.

> Source, proof and provenance: V3 trajectory_valid, sft/mathematics/dynamical_systems/law.py:18; V3 predecessor_records, sft/mathematics/dynamical_systems/law.py:41. Native results: SFT.Mathematics.DynamicsTraces.retainedPredecessor, DynamicsTraces.lean:144; SFT.Mathematics.DynamicsTraces.missingRecordRejected, DynamicsTraces.lean:149. Timeline: Turn 129, line 1660. Evidence: Completion TURN_130_DYNAMICS_COMPLETION.

84. First returns and complete registered perturbation stability

The first-return operation scans terminal positions in source order and selects the earliest equal prior state. The returned trace is the exact inclusive slice between those positions. Native proofs establish that the prefix before the repeated terminal contains no duplicate states and that no matching state precedes the selected starting occurrence. No return is produced exactly when the entire trajectory is duplicate-free. Every returned slice has at least two state occurrences with equal endpoints; when the original trajectory is valid, the slice retains valid transitions throughout.

Registered perturbation stability uses the original directed breadth-first queue. The target is checked before the visited test; visiting appends the current state, and successors are appended in transition-row order with the source duplicate behavior retained. A constructive finite-support measure proves termination without an arbitrary search bound. The native equivalence proves successful search exactly when a finite retained transition derivation reaches the terminal. Stability holds precisely when this is true for every registered perturbation. Empty perturbation lists retain the original vacuous result, and identity reachability requires no additional state-carrier admission.

All 1,024 kernels in the ten-dimensional source grammar are retained and verified. All seven original operational witnesses pass, with stability obtained from explicit transition derivations. Full native execution matches every original registration, candidate decision, closure, control and digest field, including unknown-identifier refusal. Together with the trajectory and predecessor results, these complete the exact finite Dynamical Systems entry. Continuum differential descriptions retain their original status as possible measured or finite-approximation correspondences, not premises of the entry.

Let suffix(x,S) return the suffix beginning at the first occurrence of x in S, or None if x is absent. The return scan obeys

$$R(S,[]) = \mathrm{None}, \qquad R(S, x :: T) = \begin{cases} \mathrm{Some}(H ++ [x]), & \mathrm{suffix}(x,S) = \mathrm{Some}(H), \\ R(S ++ [x], T), & x \notin S. \end{cases}$$

Starting with S=[], every recursive step appends a fresh state, so S remains distinct until the first successful suffix lookup. Induction on the unscanned tail gives

$$\mathrm{firstReturn}(p) = \mathrm{None} \iff \mathrm{Distinct}(p).$$

If the result is Some(h), the same induction and the first-suffix theorem construct lists B,M,A and terminal x such that

$$p = (B ++ x :: M) ++ x :: A, \qquad h = (x :: M) ++ [x], \qquad \mathrm{Distinct}(B ++ x :: M), \qquad x \notin B.$$

Thus no terminal repetition occurs before the displayed second x, and the selected starting x has no earlier matching occurrence. The returned length is |M|+2. For a valid trajectory p, applying the trajectory-composition equivalence first at the repeated terminal and then at the selected start proves validity of h.

For directed transitions E, write O_E(u,V)=[b:(u,b) occurs in E and b∉V], retaining row order and duplicates. The stability search with target t has equations

$$\begin{aligned} B_t(V,[]) &= \mathrm{false}, \\ B_t(V, u :: Q) &= \mathrm{true} && (u = t), \\ B_t(V, u :: Q) &= B_t(V, Q) && (u \neq t,\ u \in V), \\ B_t(V, u :: Q) &= B_t(V ++ [u], Q ++ O_E(u, V ++ [u])) && (u \neq t,\ u \notin V). \end{aligned}$$

The initial frontier is [s]. The finite support is s followed by all transition targets; fresh expansion removes u from unseen support, while visited-node skipping shortens the frontier. The lexicographic pair (|unseen|,|frontier|) therefore decreases at each recursive call.

The transition derivation is generated by s↝_E s and the rule s↝_E u, (u,v)∈E ⇒ s↝_E v. Every queued node has such a derivation, proving successful search sound. Conversely, failed search supplies an exhausted visited set containing s, closed under E and excluding t. Induction on a derivation s↝_E t puts t in that set, a contradiction. Hence

$$B_t([],[s]) = \mathrm{true} \iff s \rightsquigarrow_E t, \qquad \mathrm{Stable}(E,t,P) = \mathrm{true} \iff \forall s \in P,\ s \rightsquigarrow_E t.$$

The universal condition is true for P=[], while the identity constructor proves t↝_E t without a carrier guard. Any registered s lacking a derivation falsifies stability. The ten binary source coordinates enumerate 2^10=1,024 kernels, and their admission conjunction selects exactly the all-admitted tuple.

> Source, proof and provenance: V3 first_return, sft/mathematics/dynamical_systems/law.py:32; V3 stable_under_registered_perturbations, sft/mathematics/dynamical_systems/law.py:52. Native results: SFT.Mathematics.DynamicsReturns.returnedTraceLength, DynamicsReturns.lean:166; SFT.Mathematics.DynamicsReturns.returnedTrajectoryValid, DynamicsReturns.lean:173; SFT.Mathematics.DynamicsStability.terminalIdentity, DynamicsStability.lean:62; SFT.Mathematics.DynamicsStability.failedPerturbationRejects, DynamicsStability.lean:66. Timeline: Turn 130, line 1666; Timeline: Turn 130, line 1667. Evidence: Completion TURN_130_DYNAMICS_COMPLETION.

85. Finite logic and registered proof traces

A proposition retains a nonempty distinguished form and one of the two original held orientations. A proof step retains nonempty ordered premises, its conclusion and a nonempty rule name. The native constructors preserve the exact source guards and error cases; round-trip results establish that their represented values retain every original field. Denial switches orientation without changing the form, is involutive and produces a distinct oriented proposition. Conjunction retains both constituents with the original joint tag. Disjunction retains a supplied generated branch and both ordered alternatives, rejecting a branch outside that pair.

The proof checker walks the supplied steps in order. Every premise must already be available, and the complete rule-name/premise-tuple/conclusion record must occur in the registry before the conclusion is appended. Native equivalence establishes that acceptance is exactly an inductive trace satisfying those conditions at every step. Relative soundness follows by induction: any property held by the initial premises and preserved by every applicable registered rule holds for all resulting available propositions. This is relative registered-rule preservation, with no additional logical axiom or authority-based admission.

Consistency checks that no admitted proposition has its held denial also admitted. Registered-grammar completeness separately requires distinct generated propositions and a held decision for each proposition or its denial. These checks retain their original empty and duplicate-list behavior and make no global completeness claim beyond the supplied grammar.

All 1,024 source kernels and all seven operational witnesses are retained and verified. Native program execution matches every pinned registration, census, decision, closure, control and digest field, including unknown-identifier refusal. These results complete the finite generated Logic and Proof entry. Computability, self-reference, undecidability and incompleteness retain their original placement in the later formal-computation branch.

Represent a proposition as p=(f,o), with nonempty form f and orientation o∈{held,complementary-held}. Denial is d(f,o)=(f,flip(o)), so the two orientation cases establish

$$d(d(p)) = p, \qquad \mathrm{form}(d(p)) = \mathrm{form}(p), \qquad d(p) \neq p.$$

Conjunction returns ("joint-complete",p,q). Disjunction with supplied witness h returns ("held-alternative",h,(p,q)) exactly when h=p or h=q; otherwise it returns the source error. A proof step e=(P,c,r) requires P≠[] and a nonempty rule name r. Reconstructing a proposition or step from these same fields returns the original represented value.

Let A be available propositions, R the registry of complete triples (rule name, ordered premises, conclusion), and E the ordered steps. The checker equation is

$$\mathrm{Check}(A, [], R) = \mathrm{true}, \qquad \mathrm{Check}(A, e :: E, R) = \begin{cases} \mathrm{false}, & P_e \not\subseteq A, \\ \mathrm{false}, & P_e \subseteq A \ \wedge \ (r_e, P_e, c_e) \notin R, \\ \mathrm{Check}(A + +[c_e], E, R), & P_e \subseteq A \ \wedge \ (r_e, P_e, c_e) \in R. \end{cases}$$

The premise condition is membership of each listed premise; the registry check retains the entire ordered premise tuple. The inductive accepted trace has a done constructor for [] and a step constructor with precisely the two displayed admission premises and an accepted tail at A++[c_e]. Induction on E proves Check(A,E,R)=true iff this accepted trace exists.

For any property Φ of propositions, assume every member of A satisfies Φ and each registered triple preserves Φ from all of its premises to its conclusion. At an accepted step, premise inclusion transfers Φ from A to P_e; the registered preservation hypothesis gives Φ(c_e). Every member of A++[c_e] therefore satisfies Φ. Induction through the accepted tail yields

$$\forall q \in \mathrm{afterSteps}(A, E),\ \Phi(q).$$

This is the stated relative preservation theorem with its hypotheses retained explicitly. The two separate checks have exact meanings

$$\begin{aligned} \mathrm{Consistent}(A) &\iff \forall p \in A,\ d(p) \notin A, \\ \mathrm{GrammarComplete}(G, D) &\iff \mathrm{Distinct}(G) \wedge \forall p \in G,\ p \in D \vee d(p) \in D. \end{aligned}$$

An opposing pair directly falsifies consistency. Empty proof traces are accepted and an empty generated grammar satisfies its completeness check. The latter check does not add consistency of D as another condition. The ten binary source dimensions give 2^10=1,024 candidate kernels with the original unique all-admitted tuple.

> Source, proof and provenance: V3 proof_is_valid, sft/mathematics/logic_proof/law.py:49; V3 consistent, sft/mathematics/logic_proof/law.py:64. Native results: SFT.Mathematics.LogicForms.stepRoundTrip, LogicForms.lean:106; SFT.Mathematics.LogicForms.stepRejectsEmptyPremises, LogicForms.lean:117; SFT.Mathematics.LogicInference.emptyProofValid, LogicInference.lean:128; SFT.Mathematics.LogicInference.emptyGrammarComplete, LogicInference.lean:130. Timeline: Turn 131, line 1679; Timeline: Turn 131, line 1680. Evidence: Completion TURN_131_LOGIC_COMPLETION.

86. Compositional paths and finite type constructors

The native arrow constructor retains name, source, target and the complete nonempty trace, with precisely the original nonempty-field admission rule. Identity constructs the original empty-One return record. Composition rejects unequal interfaces; every successful composite retains the first source, second target and both traces in their original order. Canonical paths remove exactly those items whose text begins with the registered empty-One-return prefix. Canonicalization distributes over trace concatenation. Consequently both successful groupings of three arrows have the same interfaces and canonical transition path, by constructive list association.

Type membership uses the complete source key-set comparison and the last registered verdict for each key. It preserves carrier order and repeated occurrences. The product is the full ordered pair support, with membership equivalent to membership in both factors. The sum retains the original left-held and right-held tags; its membership theorem identifies exactly the corresponding source alternative. Empty and repeated carriers retain the original source behavior. These results formalise the path-association and finite type-constructor components of the registered compositional entry.

Write an arrow as a=(n_a,s_a,t_a,T_a). Its constructor admits it exactly when the name, source, target and trace are nonempty. Composition in the order a then b first requires t_a=s_b and then applies this same constructor to

$$J(a, b) = (n_b + \circ + n_a,\ s_a,\ t_b,\ T_a + + T_b).$$

For a trace T, let K(T) filter out every item with the source byte prefix "empty-One-return". This includes every matching prefix, rather than only the exact identity record. Filter distribution over append proves

$$K(T_a + +T_b) = K(T_a) + +K(T_b).$$

For successful three-arrow compositions, both groupings have source s_a and target t_c. Their canonical paths agree because

$$(K(T_a) + +K(T_b)) + +K(T_c) = K(T_a) + +(K(T_b) + +K(T_c)),$$

by induction establishing list-append associativity. This is the source's interface and canonical-path equality.

For carrier C and predicate rows R, admission requires keys(R)⊆C and C⊆keys(R), with membership rather than multiplicity comparison. The type list is [x∈C:v_R(x)=Some(true)], where v_R uses the last verdict at x. Thus empty C and R return [], while C=[a,a], R=[(a,false),(a,true)] return [a,a]. For ordered factor lists A,B,

$$\begin{aligned} A \times B &= \mathrm{concat}_{a \in A}[(a,b) : b \in B], \\ (a,b) \in A \times B &\iff a \in A \wedge b \in B, \\ A \sqcup B &= [(\text{left-held}, a) : a \in A] + +[(\text{right-held}, b) : b \in B], \\ (t,x) \in A \sqcup B &\iff (t = \text{left-held} \wedge x \in A) \vee (t = \text{right-held} \wedge x \in B). \end{aligned}$$

The product equivalence follows by extracting an outer-list member and then an inner mapped member; its converse constructs those same occurrences. Sum membership splits the append and recovers the exact tag from its mapped pair. Both constructions preserve duplicates and order.

> Source, proof and provenance: V3 compose, sft/mathematics/category_type_composition/law.py:29; V3 product_forms, sft/mathematics/category_type_composition/law.py:51; V3 sum_forms, sft/mathematics/category_type_composition/law.py:55. Native results: SFT.Mathematics.CategoryPaths.association, CategoryPaths.lean:87; SFT.Mathematics.CategoryPaths.identityFields, CategoryPaths.lean:102; SFT.Mathematics.CategoryTypes.emptyType, CategoryTypes.lean:77; SFT.Mathematics.CategoryTypes.duplicateType, CategoryTypes.lean:79. Timeline: Turn 132, line 1688; Timeline: Turn 132, line 1689. Evidence: Completion TURN_133_CATEGORY_COMPLETION.

87. Identity, transport and route comparison

For every nonempty source object, the exact original identity constructor succeeds. Its empty-One return contributes no canonical transition. Successful left and right identity compositions retain the other arrow's source, target and canonical path. Composition of source-admitted arrows exists precisely when the first target equals the second source; original field admission and complete trace retention are preserved. These results derive the registered identity and interface conditions constructively.

The native transport predicate retains the exact source procedure: the last object and arrow mapping for each key is used, every supplied arrow must have its name and endpoints mapped, and its image endpoints must equal the mapped endpoints. Native equivalence gives these conditions for every supplied arrow. The predicate retains the source's empty-input behavior and does not introduce additional executable checks.

Naturality replays each supplied nonempty route in order, propagating the original first composition failure. The resulting trace is exactly the concatenation of the input arrow traces. Route comparison succeeds precisely when source, target and the complete canonical traces agree. Empty input retains the source indexing error. All 1,024 original kernels, seven operational witnesses and complete generated-program outputs are verified against the pinned V3 records, completing the registered finite compositional entry.

For nonempty object x, the identity record is

$$I_x = (\mathrm{id}{:}x,\ x,\ x,\ [\text{empty-One-return}{:}x]), \qquad K(T_{I_x}) = [].$$

The prefix calculation removes its sole trace item. Consequently successful left and right identity compositions preserve the other arrow's interfaces, with canonical paths []++K(T_a)=K(T_a) and K(T_a)++[]=K(T_a). For admitted a,b, their concatenated trace is nonempty, the joined name contains the composition symbol, and the retained source/target are nonempty. Thus J(a,b) is admitted, proving

$$(\exists c,\ \mathrm{compose}(a,b) = \mathrm{ok}(c)) \iff t_a = s_b.$$

Let O and M be the last-value object and arrow lookup maps. The transport predicate is exactly

$$\forall a \in A,\ \exists b, s, t,\ M(n_a) = \mathrm{Some}(b) \wedge O(s_a) = \mathrm{Some}(s) \wedge O(t_a) = \mathrm{Some}(t) \wedge s_b = s \wedge t_b = t.$$

Lookup satisfies lookup(R++[(k,v)],k)=Some(v), by induction on R. Empty A makes the universal predicate true. No additional path-composition or identity test is inserted into this source predicate.

Route replay begins at its first arrow and repeatedly composes with the next. Formally FoldRest(a,[])=ok(a); FoldRest(a,b::R) propagates a composition error immediately, or recurses on its successful result and R. The empty route gives the original tuple-index error. On every successful nonempty route [a_1,...,a_n], induction using trace retention gives

$$s_{\mathrm{result}} = s_{a_1}, \qquad T_{\mathrm{result}} = T_{a_1} + + \cdots + + T_{a_n}.$$

The left route is replayed before the right. Once both return l,r, the comparison succeeds exactly when

$$s_l = s_r \wedge t_l = t_r \wedge K(T_l) = K(T_r).$$

The first left failure is propagated without replaying the right; a right failure is propagated after successful left replay. These are the complete source route-comparison conditions. The ten binary grammar coordinates enumerate 2^10=1,024 candidates, with acceptance equivalent to their original all-admitted conjunction.

> Source, proof and provenance: V3 identity, sft/mathematics/category_type_composition/law.py:25; V3 functor_preserves, sft/mathematics/category_type_composition/law.py:59; V3 naturality_commutes, sft/mathematics/category_type_composition/law.py:75. Native results: SFT.Mathematics.CategoryIdentity.identityNameNonempty, CategoryIdentity.lean:115; SFT.Mathematics.CategoryIdentity.identityExists, CategoryIdentity.lean:120; SFT.Mathematics.CategoryTransport.leftPathError, CategoryTransport.lean:139; SFT.Mathematics.CategoryTransport.rightPathError, CategoryTransport.lean:143. Timeline: Turn 133, line 1696; Timeline: Turn 133, line 1697. Evidence: Completion TURN_133_CATEGORY_COMPLETION.

88. Symbols and observation-relative distinction

A source symbol retains its canonical form and held label as an ordered pair. Alphabet admission requires a nonempty distinct list of these pairs and nonempty fields, with the original guard order and without changing source labels. Observation validation requires a nonempty alphabet, equal row and alphabet counts, distinct observation source pairs, exact source-set coverage and nonempty observation labels.

The native lookup selects the label only when exactly one source row matches the full symbol pair. Constructive results establish that every member of a validated alphabet has exactly one nonempty observed label and that each returned label retains its original observation row. Two source-admitted labels produce a retained distinction exactly when those labels differ. The operation is irreflexive, including its original early return before observation lookup, and is symmetric on a validated alphabet. These results formalise the alphabet, observation-admission and pairwise-distinction components of the registered symbol entry.

For symbol s=(form,label), let Held(s) mean both fields are nonempty. The alphabet constructor checks A≠[] and Distinct(A), then checks Held(s) for every s∈A and returns A unchanged. For observation rows O=[(s_i,l_i)], write K=map(first,O). The separate observation predicate is exactly

$$A \neq [] \wedge |O| = |A| \wedge \mathrm{Distinct}(K) \wedge K \subseteq A \wedge A \subseteq K \wedge \forall (s,l) \in O,\ l \neq \text{empty string}.$$

Define L_O(s)=[l:(s,l) occurs in O], retaining row order and matching the entire symbol pair. Lookup has the exact success equivalence

$$\mathrm{observedLabel}(O,s) = \mathrm{ok}(l) \iff L_O(s) = [l].$$

Coverage supplies a row for s∈A, and distinct source keys exclude a second matching row. Thus validation implies a unique lookup result l; its original row (s,l)∈O and the final validation clause imply that l is nonempty. Zero or multiple matching rows retain the original lookup error.

Distinction first returns ok(false) when s=t, even if lookup would fail. Otherwise it looks up s and then t, propagating the first error. For successful labels l,r, its result is

$$\mathrm{distinguishable}(O,s,t) = \mathrm{ok}(\mathrm{decide}(l \neq r)).$$

If s=t, lookup determinism forces l=r, so this equation also agrees with the early-return branch. Inequality symmetry proves symmetry whenever both lookups succeed, and therefore for members of a validated alphabet. Irreflexivity follows directly from the early branch without validation.

> Source, proof and provenance: V3 validate_observation, sft/information_science/symbols_distinguishability/law.py:21; V3 distinguishable, sft/information_science/symbols_distinguishability/law.py:39. Native results: SFT.InformationScience.SymbolObservation.validatedLookup, SymbolObservation.lean:210; SFT.InformationScience.SymbolObservation.validatedSymmetry, SymbolObservation.lean:221. Timeline: Turn 134, line 1708. Evidence: Completion TURN_135_SYMBOL_COMPLETION.

89. Observation classes, complete distinction ledgers and extension

Observation classes retain the first occurrence order of the observed labels and the original alphabet order of every source fibre. Constructive results establish that their labels are distinct, their fibres are nonempty, every source symbol occurs in its corresponding fibre, and a symbol cannot belong to two fibres with unequal labels. This partitions the complete admitted alphabet without replacing or erasing microform identity.

The pair ledger generates every original positional pair with the earlier symbol first. Its separate retained and closed passes preserve the source order and original lookup-error behavior. On validated observations both passes succeed, their outputs are disjoint, and their union exhausts the generated pair list. A singleton alphabet has no pair distinction.

Appending a fresh symbol generates one new pair with each prior symbol in the original nested-loop order, while preserving prior pairs. Appending its one observation row supplies its unique nonempty label and preserves every old lookup and distinction; every member of the extended alphabet still has a nonempty observed label. Under coarsening, a previously closed distinction remains closed, and a previously retained distinction closes exactly when its image labels are merged. These results establish the source successor and coarsening clauses.

All 512 original symbol kernels and all six operational witnesses are retained and verified. Fresh native execution matches the pinned registration, census, decisions, closure, controls and digest fields, including unknown-identifier refusal. This completes the registered finite Symbol Distinction entry, preserving its original exclusion of linguistic meaning, occurrence frequencies and downstream information quantities.

Collecting the unique labels in alphabet order produces rows R with map(first,R)=A. Their membership equivalence is

$$(s,l) \in R \iff s \in A \wedge \mathrm{observedLabel}(O,s) = \mathrm{ok}(l).$$

Let F_l=[s:(s,l) occurs in R] and L=firstLabels(map(second,R)). The class list [(l,F_l):l∈L] has distinct labels. Every l∈L comes from a row, proving F_l is nonempty; every collected row places its symbol in the corresponding fibre. If s∈F_l and s∈F_r, the two successful lookups are equal and hence l=r. Collection and filtering preserve alphabet order rather than replacing symbols with their labels.

The positional-pair generator has the exact recurrence

$$P([]) = [], \qquad P(a :: R) = [(a, b) : b \in R] + +P(R).$$

On validated observations, let D(p) be the defined Boolean distinction for each pair. The retained and closed passes return, in generated order,

$$U = [p \in P(A) : D(p) = \text{true}], \qquad C = [p \in P(A) : D(p) = \text{false}].$$

Their membership laws imply U∩C is empty: a shared pair would have both Boolean results. Every generated pair has one of those results, proving exhaustiveness as membership in U or C. This does not assert that concatenating the two passes reconstructs the original interleaved order. A singleton alphabet generates P([a])=[].

For a fresh symbol z, define E_z([])=[] and

$$E_z(a :: R) = ([(a, b) : b \in R] + +[(a, z)]) + +E_z(R).$$

Distributing map over append proves P(A++[z])=E_z(A). Thus every prior pair remains and every old a contributes (a,z), in its original nested-loop position. Appending observation row (z,l) gives

$$L_{O++[(z,l)]}(s) = L_O(s) + + \begin{cases} [l], & z = s, \\ [], & z \neq s. \end{cases}$$

For z∉A and validated O, key coverage gives z∉keys(O), so its new lookup is uniquely l. Old lookups are unchanged because z differs from every old symbol. When l is nonempty, every member of A++[z] consequently has a nonempty label, and each old distinction result is unchanged.

For a coarsening map m and fine/coarse lookups related by l↦m(l), the labelled-distinction equation gives

$$D_{\text{fine}}(s, t) = \text{true} \wedge D_{\text{coarse}}(s, t) = \text{false} \iff l \neq r \wedge m(l) = m(r).$$

If l=r, their images are equal and both distinctions are closed. These implications retain the stipulated fine and coarse lookup correspondences. The nine binary symbol coordinates enumerate 2^9=512 kernels, with the original conjunction selecting the all-admitted tuple.

> Source, proof and provenance: V3 observation_classes, sft/information_science/symbols_distinguishability/law.py:45; V3 distinction_ledger, sft/information_science/symbols_distinguishability/law.py:57. Native results: SFT.InformationScience.SymbolClasses.classLabels, SymbolClasses.lean:163; SFT.InformationScience.SymbolClasses.classLabelsDistinct, SymbolClasses.lean:166; SFT.InformationScience.SymbolLedger.coarseningClosed, SymbolLedger.lean:187; SFT.InformationScience.SymbolLedger.coarseningClosesExactly, SymbolLedger.lean:197; SFT.InformationScience.SymbolExtension.successorLookup, SymbolExtension.lean:40; SFT.InformationScience.SymbolExtension.priorDistinctionPreserved, SymbolExtension.lean:59. Timeline: Turn 135, line 1715; Timeline: Turn 135, line 1716; Timeline: Turn 135, line 1717. Evidence: Completion TURN_135_SYMBOL_COMPLETION.

90. Encoding classes and exact inverse decoding

Encoding admission retains the original nonempty, distinct source and code-support lists, complete equal-length source relation, distinct origin keys and code-support membership. Codewords are exact held-label lists; their original empty-word behavior is preserved. Source classes occur in first-use encoding order and

retain source order and multiplicity. Native results characterize membership, distinct class keys, source provenance and nonempty used classes on admitted inputs.

The exact decoder exists if and only if every used codeword class contains a single source occurrence. Its ordered inverse relation preserves those codeword keys. Decoder lookup requires exactly one matching image and retains the original error on zero or multiple matches. Constructive proofs establish that applying the resulting decoder to every row of an admitted lossless encoding returns the original canonical source. The raw operations retain their original empty and malformed-input behavior independently of encoding admission.

For source list S, code-support list W and encoding rows E, the exact admission condition is

$$\begin{aligned} &S \neq [] \wedge W \neq [] \wedge \mathrm{Distinct}(S) \wedge \mathrm{Distinct}(W) \\ &\wedge |E| = |S| \wedge \mathrm{keys}(E) \subseteq S \wedge S \subseteq \mathrm{keys}(E) \\ &\wedge \mathrm{Distinct}(\mathrm{keys}(E)) \wedge \forall (s,w) \in E,\ w \in W. \end{aligned}$$

Each word w is a list of held labels; this guard does not require w itself to be nonempty. Put U=firstCodewords(map(second,E)) and F_w=[s∈S:(s,w)∈E]. Then

$$s \in F_w \iff s \in S \wedge (s,w) \in E, \qquad (w,T) \in \mathrm{sourceClasses}(S,E) \iff w \in \mathrm{map}(\mathrm{second},E) \wedge T = F_w.$$

First-occurrence filtering gives distinct U in encoding order, while each F_w preserves source order. On admitted inputs, every used w has an encoding row whose source lies in S, so every emitted class is nonempty.

The inverse operation traverses these classes in order. It maps (w,[s]) to (w,s), rejects a class of zero or multiple occurrences, and returns the empty decoder on an empty class list. Induction over the class list proves

$$(\exists D,\ \mathrm{exactDecoder}(S,E) = \mathrm{Some}(D)) \iff \forall (w,F) \in \mathrm{sourceClasses}(S,E),\ \exists s,\ F = [s].$$

On success, the decoder keys equal the class keys in order, and (w,s)∈D iff (w,[s]) is a source class. Decoder lookup succeeds exactly when [s':(w,s') occurs in D]=[s]. For an admitted row (s,w)∈E, source provenance gives s∈F_w; the singleton-class condition forces F_w=[s]. Hence (w,s)∈D. Distinct class keys give exactly one decoder match, establishing

$$\mathrm{decode}(w,D) = \mathrm{ok}(s) \quad \text{for every } (s,w) \in E.$$

This is the exact left-inverse result, retaining the admitted-encoding and successful-decoder hypotheses. The raw class/inverse operations retain their separate empty-input behavior.

> Source, proof and provenance: V3 source_classes, sft/information_science/encoding_decoding/law.py:29; V3 exact_decoder, sft/information_science/encoding_decoding/law.py:37; V3 decode, sft/information_science/encoding_decoding/law.py:44. Native results: SFT.InformationScience.EncodingClasses.sourceProvenance, EncodingClasses.lean:157; SFT.InformationScience.EncodingClasses.admittedClassNonempty, EncodingClasses.lean:165; SFT.InformationScience.EncodingDecoder.decodeExactlyOnce, EncodingDecoder.lean:154; SFT.InformationScience.EncodingDecoder.decoderLeftInverse, EncodingDecoder.lean:164. Timeline: Turn 136, line 1725; Timeline: Turn 136, line 1726. Evidence: Completion TURN_138_ENCODING_COMPLETION.

91. Closed encoding distinctions and exact composition

The closed-distinction ledger enumerates source pairs in their original positional order and compares each source's first encoding image. On admitted encodings the computation is total. A pair occurs in the closed ledger precisely when it is one of the generated source pairs and both members belong to the same retained codeword class. The empty and singleton source cases retain empty ledgers; missing first images retain the original error behavior on unvalidated inputs.

Encoding composition requires distinct second-stage interface keys and an image for every first-stage codeword. Native existence is equivalent to these exact conditions. Every source occurrence remains in its original order, and an output relation holds precisely when a first-stage source-to-codeword relation joins a second-stage codeword-to-output relation. No source alphabet or injectivity condition beyond the original composition guard is added.

Let f_E(s) be the first encoding image for s, when present. The source pair list obeys P([])=[] and P(a::R)=[(a,b):b∈R]++P(R). The closed-pair operation looks up a before b, returns its original StopIteration-derived error if either image is missing, and otherwise tests equality of the two images. On admitted inputs its ordered ledger C therefore satisfies

$$(a,b) \in C \iff (a,b) \in P(S) \wedge \exists w,\ a \in F_w \wedge b \in F_w.$$

In one direction, a shared class supplies both encoding rows; distinct admitted source keys identify their first images as w. In the other, equal first images supply the two original rows and, with a,b∈S, give membership in F_w. The pair generator returns [] on empty or singleton sources, and the selection retains its source order.

For first-stage E and second-stage rows H, composition admission is exactly

$$\mathrm{Distinct}(\mathrm{keys}(H)) \wedge \forall (s,w) \in E,\ w \in \mathrm{keys}(H).$$

Coverage supplies a second-stage image for each row; induction on E constructs the complete output. Conversely, a successful public composition must have passed this guard. The recursive application maps (s,w)::R to (s,f_H(w)) followed by the recursively mapped tail, propagating any missing-image error. Thus a successful output J obeys

$$\mathrm{keys}(J) = \mathrm{keys}(E), \qquad (s,v) \in J \iff \exists w,\ (s,w) \in E \wedge (w,v) \in H.$$

The first equality is an ordered-list identity, preserving repeated source occurrences. For the second, induction supplies the intermediate lookup; distinct second-stage keys make lookup equivalent to the recorded row (w,v). No separate source-alphabet validation or injectivity premise is added to this composition condition.

> Source, proof and provenance: V3 closed_distinctions, sft/information_science/encoding_decoding/law.py:51; V3 compose_encodings, sft/information_science/encoding_decoding/law.py:61. Native results: SFT.InformationScience.EncodingMaps.interfaceMembership, EncodingMaps.lean:150; SFT.InformationScience.EncodingMaps.composedRow, EncodingMaps.lean:185; SFT.InformationScience.EncodingLoss.emptyLoss, EncodingLoss.lean:141; SFT.InformationScience.EncodingLoss.singletonLoss, EncodingLoss.lean:143. Timeline: Turn 137, line 1733; Timeline: Turn 137, line 1734. Evidence: Completion TURN_138_ENCODING_COMPLETION.

92. Encoding source extension and finite closure

For a fresh source, appending its one encoding pair leaves every previous source-to-code membership unchanged. Every codeword class retains its original source order and gains the fresh source at the end exactly when it is the newly assigned codeword. The used-codeword set gains precisely that word. A single source and one codeword have the singleton inverse. If an encoding has an exact decoder and the appended word is unused, all old singleton classes remain singleton and the new class is the singleton fresh source; therefore the extended exact decoder exists. If the appended word is already used, its nonempty class gains the fresh source and is no longer singleton; the exact decoder is absent and that merged class remains explicitly recorded.

These source-extension statements hold for arbitrary finite source lists satisfying the original admission and freshness conditions. They do not rely on the witness source size. All 1,024 original encoding kernels and all five operational witnesses are retained. Fresh native execution agrees with every pinned

registration, census, decision, closure, control and digest field, including unknown-identifier refusal. The Encoding and Decoding entry is complete within its original finite representation boundary; code-length value, compression, channel reliability and correction remain subsequent source subjects.

Let S'=S++[z] and E'=E++[(z,w)], where z∉S and E is admitted on S. Key coverage implies z∉keys(E). For every old s∈S and word v, splitting membership in the appended relation gives

$$(s,v) \in E' \iff (s,v) \in E, \qquad (z,v) \in E' \iff v = w.$$

The new pair cannot match an old source because s≠z. Conversely, no old pair has source z, so its only new image is w. Induction through the ordered source filter therefore establishes

$$F'_v = F_v +\!+ \begin{cases} [z], & v = w, \\ [], & v \neq w, \end{cases} \qquad v \in \mathrm{images}(E') \iff v \in \mathrm{images}(E) \vee v = w.$$

These are respectively an ordered-list identity and a membership equivalence; the old source order is retained in every class. The singleton base is computed directly:

$$\mathrm{exactDecoder}([z],[(z,w)]) = \mathrm{Some}([(w,z)]).$$

Suppose the old exact decoder exists and w is unused. For any old used v, v≠w, so F'_v=F_v remains singleton by the old decoder theorem. Since w is unused, F_w=[] and hence F'_w=[z]. The displayed image-membership equivalence exhausts the new classes: each is either an old singleton or this new singleton. Applying the singleton-class criterion constructs an extended exact decoder.

If w is already used, admission supplies an old member of F_w. Then F'_w=F_w++[z] cannot be singleton: writing F_w=x::R, equality with [y] would force R++[z]=[], contradicted by z's membership in that append. The new encoding still emits the class for w, so a successful inverse would require this nonsingleton list to be singleton. Therefore

$$w \in \mathrm{images}(E) \implies \mathrm{exactDecoder}(S', E') = \mathrm{None}.$$

The successor proof quantifies over the original finite lists and every fresh source; it does not assume a particular codeword length, nor replace a merged class with an arbitrary representative. The ten binary source coordinates enumerate 2^10=1,024 kernels, with their original admission conjunction selecting exactly the all-admitted tuple.

> Source, proof and provenance: V3 validate_encoding, sft/information_science/encoding_decoding/law.py:13; V3 exact_decoder, sft/information_science/encoding_decoding/law.py:37. Native results: SFT.InformationScience.EncodingExtension.reusedClosesDecoder, EncodingExtension.lean:130; SFT.InformationScience.EncodingExtension.singletonBase, EncodingExtension.lean:146. Timeline: Turn 138, line 1744. Evidence: Completion TURN_138_ENCODING_COMPLETION.

93. Complete information support and position generation

The information-quantity support operation requires at least two distinct held labels and a nonempty canonical position trace. Its complete ordered Cartesian support contains exactly the words whose length equals the position-trace length and whose labels belong to the held alphabet. Native proofs establish completeness and duplicate freedom for every finite admitted input. Appending one position extends every prior word once with every held label in the original product order.

For every generated prefix and suffix, inserting any held label produces a member of the corresponding complete support. Inserting two distinct held labels with the same prefix and suffix produces distinct words. These statements establish the arbitrary finite independent-position support relation. Support composition retains the original left-outer/right-inner pair order and concatenates the two source words. Each output occurs precisely through such a source pair; raw duplicate or variable-length input supports retain their original behavior. No logarithmic value or additional information scale is introduced.

For an ordered held alphabet L, the source word generator is

$$W_L(0) = [[]], \qquad W_L(n+1) = \mathrm{concat}_{a \in L}[a :: w : w \in W_L(n)].$$

The public support operation first requires |L|≥2 and Distinct(L), then requires a nonempty distinct position list P. It returns W_L(|P|). Induction on n proves the complete membership equivalence

$$w \in W_L(n) \iff |w| = n \wedge \forall a \in w,\ a \in L.$$

At n=0 the sole generated word is the empty list. At a successor, extracting a flatMap row gives a∈L and a generated tail of length n. Conversely a word of length n+1 has a head a∈L and a tail satisfying the induction hypothesis, reconstructing that same generated row.

Duplicate freedom follows by induction too. Within a row, a::u=a::v forces u=v, preserving distinct tails. Across different head rows, a::u=b::v forces a=b, contradicting distinct head labels. Appending one position has the ordered-list identity

$$W_L(n+1) = \mathrm{concat}_{w \in W_L(n)}[w + +[a] : a \in L].$$

To prove it, append-extension distributes over concatenated rows and commutes with a fixed head prefix. The base n=0 gives the ordered singleton words; the successor follows by distributing over the original outer label rows. Thus the identity preserves the exact generated order, not only membership.

For u∈W_L(n), v∈W_L(m) and a∈L, the membership characterization and append-length equation yield

$$u + +v \in W_L(n+m), \qquad u + +a :: v \in W_L(n + (m+1)).$$

If a≠b, equality u++a::v=u++b::v would cancel the common prefix by induction and then force a=b by head equality. Hence the two held alternatives are distinct at every finite position with the same surrounding context.

For arbitrary nonempty support lists A,B, composition returns the original nested concatenation

$$C(A,B) = \mathrm{concat}_{u \in A}[u + +v : v \in B], \qquad w \in C(A,B) \iff \exists u \in A, \exists v \in B,\ w = u + +v.$$

The equivalence follows by extracting or constructing the outer and inner mapped occurrences. Empty factors take the original error branch. On raw supports no distinctness or equal-length guard is added, so repeated concatenations and variable-length words retain their source behavior.

> Source, proof and provenance: V3 complete_word_support, sft/information_science/information_quantity/law.py:15; V3 distinction_pairs, sft/information_science/information_quantity/law.py:23. Native results: SFT.InformationScience.QuantitySupport.firstPositionWords, QuantitySupport.lean:216; SFT.InformationScience.QuantitySupport.positionSuccessorOrder, QuantitySupport.lean:221; SFT.InformationScience.QuantityPairs.newPairIncluded, QuantityPairs.lean:154; SFT.InformationScience.QuantityPairs.priorPairsRetained, QuantityPairs.lean:163. Timeline: Turn 139, line 1754; Timeline: Turn 140, line 1761. Evidence: Completion TURN_140_QUANTITY_COMPLETION.

94. Exact information profiles and independent position traces

For every admitted support, the positional pair construction contains each unordered nonidentity pair once, with its earlier support member first. The native derivation establishes endpoint provenance, absence of self-pairs and reversed duplicates, complete pair coverage and unique records. Observation admission requires unique image keys and exact support-key coverage before the support-pair validation. Empty observation labels remain permitted. The profile retains the support trace, first-occurrence observation-class order, retained distinctions with unequal images, and closed distinctions with equal images. These ledgers are disjoint and exhaust the complete support pairs. Fine observation retains every pair and closes none; coarsening cannot reopen a closed distinction and can only remove retained distinctions.

The position-trace computation explicitly tests both first held labels for every generated prefix and suffix at each position. Constructive proofs show that the complete support contains both alternatives at every finite depth and that unequal held labels give distinct words. The admitted computation returns exactly one one-Fold-distinction record for every original position, in order. Appending a position preserves the preceding unit trace and appends its unit; composition concatenates the position traces. Together with complete word generation and pair coverage, these results establish the original finite successor clause without a depth table or logarithmic proof quantity.

All 512 quantity kernels and all five operational witnesses are retained. Fresh native execution matches the pinned registration, census, decisions, closure, controls and all digest fields, including unknown-identifier refusal. Information Quantity is complete within the original exact finite boundary; conventional logarithmic units remain post-seal correspondence summaries rather than SFT proof values.

For distinct nonempty support S, generate P([])=[] and P(a::R)=[(a,b):b∈R]++P(R). Induction proves that every pair has endpoints in S and unequal endpoints. Each head-row pair is unique, all tail pairs are unique by induction, and a head-row pair cannot equal a tail pair because its first endpoint a does not occur in R. For any distinct a,b∈S, either one is the current head, producing the corresponding ordered pair, or both lie in the tail and induction applies. Thus

$$a, b \in S \wedge a \neq b \implies (a, b) \in P(S) \vee (b, a) \in P(S), \qquad (a, b) \in P(S) \implies (b, a) \notin P(S).$$

The original nonidentity filter consequently leaves P(S) unchanged. Together with duplicate freedom, these facts establish exactly one record for each unordered distinct pair.

For observation rows O, admission requires Distinct(keys(O)), keys(O)⊆S and S⊆keys(O). It precedes the nonempty/distinct support check. The image f_O(w) is its first lookup value, with the raw missing-lookup fallback retained; under admission every support word has a matching row and the fallback is not used. Empty label strings remain valid. The profile is

$$\big(S,\ \mathrm{firstLabels}([f_O(w) : w \in S]),\ U_O,\ C_O\big),$$

where U_O=[(a,b)∈P(S):f_O(a)≠f_O(b)] and C_O=[(a,b)∈P(S):f_O(a)=f_O(b)]. Equality case analysis proves each generated pair belongs to one pass; equality and inequality cannot both hold, proving disjointness. A class label l occurs iff some w∈S satisfies f_O(w)=l, and first-occurrence filtering makes those labels distinct while preserving their order.

For a fine observation, f_O(a)=f_O(b) implies a=b on S. Since every generated pair has unequal endpoints, every pair is retained and none is closed. For compatible coarse images f_C=m∘f_O on the pair endpoints, equality is preserved by m, giving

$$C_O \subseteq C_C, \qquad U_C \subseteq U_O.$$

The second inclusion follows by contradiction: equal fine images would force equal coarse images, contradicting coarse retention.

Let L begin with its first two distinct held labels a,b, and let n=|P|. At position i, with j positions following it, the actual source check is

$$\forall u \in W_L(i),\ \forall v \in W_L(j),\ u{+}{+}a :: v \in W_L(i + (j + 1)) \wedge u{+}{+}b :: v \in W_L(i + (j + 1)).$$

The independent-position theorem proves both membership clauses for every prefix and suffix; prefix cancellation proves the two words differ. Induction over P increments i and decreases the remaining tail, retaining i+|remaining|=n. Hence every check succeeds on the admitted complete support and the returned trace is exactly

$$T(P) = [(p, \text{one-Fold-distinction}) : p \in P], \qquad |T(P)| = |P|.$$

Map distribution over append yields T(P++Q)=T(P)++T(Q), and in particular T(P++[z])=T(P)++[(z,one-Fold-distinction)]. Invalid support inputs propagate their original error before these position checks. The nine binary source coordinates enumerate 2^9=512 kernels with the original all-admitted conjunction.

> Source, proof and provenance: V3 information_profile, sft/information_science/information_quantity/law.py:34; V3 distinction_position_trace, sft/information_science/information_quantity/law.py:52. Native results: SFT.InformationScience.QuantityProfile.coarseningPreservesClosed, QuantityProfile.lean:154; SFT.InformationScience.QuantityProfile.coarseningOnlyLoses, QuantityProfile.lean:164; SFT.InformationScience.QuantityPositions.traceComposition, QuantityPositions.lean:94; SFT.InformationScience.QuantityPositions.traceSuccessor, QuantityPositions.lean:97. Timeline: Turn 140, line 1762. Evidence: Completion TURN_140_QUANTITY_COMPLETION.

95. Deterministic entropy partitions and observation refinement

The exact entropy partition requires nonempty duplicate-free microstate support, unique observation keys and exact support-key coverage. Class labels follow their first observed occurrence in the original support order; each fibre retains its microstates in that order. Native proofs establish complete coverage, disjoint classes, nonempty fibres and distinct members. Unresolved distinctions are the complete positional pair list within each class. Empty and singleton lists return structural empty One; the general empty-One result is equivalent to at most one input member. The complete closed-pair output flattens these class ledgers in class order.

The original refinement operation has a broader raw domain than partition admission: it tests only the two observation key sets against support and retains dictionary last-row behavior for repeated keys. Its native characterization states exactly that equality under the fine observation implies equality under the coarse observation for every support pair. The relation is reflexive on matching-key observations and transitive. For admitted exact partitions, refinement holds if and only if every fine class is contained in a coarse class. These results retain deterministic observation provenance without introducing a stochastic source or scalar entropy value.

Let S be the microstate support and O the observation rows. Partition admission is

$$S \neq [] \wedge \mathrm{Distinct}(S) \wedge \mathrm{Distinct}(\mathrm{keys}(O)) \wedge \mathrm{keys}(O) \subseteq S \wedge S \subseteq \mathrm{keys}(O).$$

Writing f_O(s) for the source's last-row lookup, define F_l=[s∈S:f_O(s)=Some(l)]. The emitted partition is [(l,F_l):l∈firstLabels(observedValues(O,S))]. An emitted row has a witnessing source state, so its fibre is nonempty. Key coverage supplies each state's image, hence its fibre; equality of the deterministic lookup proves that intersecting fibres have equal labels. Filtering the distinct support preserves distinctness within every fibre and its original order.

For any finite member list M, let P(M) be its positional pair list. The structural uncertainty result is

$$U(M) = \begin{cases} \mathrm{emptyOne}, & P(M) = [], \\ \mathrm{pairs}(P(M)), & P(M) \neq []. \end{cases}$$

The pair recurrence gives P([])=P([x])=[]; for x::y::R its first row contains (x,y), so it is nonempty. Therefore U(M)=emptyOne iff M has at most one occurrence, even on the raw list domain. This structural result is separate from any ordinary microstate string named "empty-One". For an admitted partition [(l_i,F_i)], the full closed ledger is concat_i P(F_i), retaining class order and each within-class pair order.

The raw refinement predicate requires key-set coverage for both observations but does not add their partition-admission guards. Its exact condition is

$$O_f \preceq_S O_c \iff \mathrm{keysMatch}(S, O_f) \wedge \mathrm{keysMatch}(S, O_c) \wedge \forall a, b \in S,\ f_{O_f}(a) = f_{O_f}(b) \Rightarrow f_{O_c}(a) = f_{O_c}(b).$$

Reflexivity follows from identity of the equality implication. For transitivity, apply the fine-to-middle implication and then the middle-to-coarse implication to the same pair. Repeated raw keys retain last-row lookup, and the empty support with empty observations satisfies the predicate.

For two admitted partitions, select a base member x of any fine fibre F. Every y∈F has the same fine image as x; refinement forces its coarse image to equal x's, placing all of F in the coarse fibre containing x. Conversely, if every fine fibre lies in some coarse fibre, equal fine images put a,b into the same fine fibre and hence into one coarse fibre, proving equal coarse images. Thus

$$O_f \preceq_S O_c \iff \forall F \in \mathcal{P}_f,\ \exists C \in \mathcal{P}_c,\ F \subseteq C,$$

where the quantifiers range over the member lists of the admitted labelled partition rows. This proves the source refinement relation from finite observation records without assigning a scalar entropy value.

> Source, proof and provenance: V3 exact_partition, sft/information_science/entropy_uncertainty/law.py:14; V3 refines, sft/information_science/entropy_uncertainty/law.py:52. Native results: SFT.InformationScience.EntropyPartition.closedResult, EntropyPartition.lean:124; SFT.InformationScience.EntropyPartition.closedMembership, EntropyPartition.lean:129; SFT.InformationScience.EntropyRefinement.containmentImpliesRefinement, EntropyRefinement.lean:71; SFT.InformationScience.EntropyRefinement.refinementIffContainment, EntropyRefinement.lean:95. Timeline: Turn 141, line 1772. Evidence: Completion TURN_143_ENTROPY_COMPLETION.

96. Exact entropy parts and closed-pair monotonicity

Each entropy record retains its observation label, exact positive support part, complete microstate trace and structural unresolved-distinction value. The native construction preserves the partition's record order and uses the exact reduced fraction of the class cardinality over the complete support cardinality. A cross-product identity proves preservation of that rational value, and the part is positive and at most the whole. Literal empty-One microstate labels remain ordinary retained microstates; structural empty-One uncertainty is the separate unresolved-distinction result.

Refinement implies inclusion of each fine class's positional pairs in its containing coarse class. The complete closed-pair ledger has no repeated pair records, both within a class and across disjoint classes. Native finite-list counting therefore proves that a finer admitted observation has no more closed pairs than an admitted coarser observation it refines. The result applies to arbitrary finite admitted support and retains every pair's source coordinates.

For an admitted support S and partition [(l_i,F_i)], put N=|S| and m_i=|F_i|. Admission and nonempty fibres give N>0 and m_i>0. The exact record constructor is

$$R_i = (l_i, \mathrm{normalizePositive}(m_i, N), F_i, U(F_i)).$$

Writing the normalized positive fraction as (a_i,b_i), normalization proves

$$m_i b_i = a_i N, \qquad a_i > 0, \quad b_i > 0.$$

Since F_i is an ordered filter of S, induction on S gives m_i≤N: retaining a head increments both lengths, while discarding it increments only the support length. Normalization therefore yields a_i≤b_i and hence 0<a_i/b_i≤1. The ledger recursion emits R_i followed by the recursively constructed tail. Induction on the partition proves that projecting each record to (label,microstates) returns the original partition exactly, while each unresolved-distinction field remains U(F_i). A partition error is returned unchanged; no record is manufactured for an inadmissible input.

Let P(X) denote the positional pairs of a list. Its ordered recurrence is

$$P([]) = [], \qquad P(x :: X) = [(x, y) : y \in X] \mathbin{++} P(X).$$

For predicates p,q with p(x)⇒q(x) on S, induction through this recurrence proves

$$P(\mathrm{filter}_p\, S) \subseteq P(\mathrm{filter}_q\, S).$$

If p rejects the head, every pair comes from the tail and survives the induction. If p accepts it, q accepts it too; a head pair survives because its second coordinate also satisfies the implication, and a tail pair survives by induction. Thus the proof retains the source orientation of pairs, rather than replacing positional pairs by arbitrary ordered pairs.

Write C(O)=concat_i P(F_i) for the complete closed ledger. Under admitted fine and coarse observations, refinement supplies a containing coarse fibre for every fine fibre. Applying filtered-pair containment inside each such fibre gives

$$O_f \preceq_S O_c \implies C(O_f) \subseteq C(O_c).$$

For a distinct support, P(S) has no duplicates: head pairs have different second coordinates, the tail is duplicate-free by induction, and a head pair cannot occur in the tail because its first coordinate is absent there. Each fibre inherits distinctness. Across different fibres, a repeated pair would place its first coordinate in both fibres, forcing their labels equal, contrary to distinct emitted labels. Consequently C(O) is duplicate-free for every admitted partition.

Finally, for a duplicate-free list A contained in B, remove every occurrence of the head a of A from B. Membership of a makes the filtered list strictly shorter; distinctness of A ensures its tail is still contained in that filtered list. Induction gives

$$|\,\mathrm{tail}(A)| \le |B \setminus \{a\}| < |B|, \qquad |A| \le |B|.$$

Applying this finite-list argument to the closed ledgers establishes

$$O_f \preceq_S O_c \implies |C(O_f)| \le |C(O_c)|.$$

This is the declared finite closed-distinction monotonicity, with exact rational support parts retained separately in the records.

> Source, proof and provenance: V3 entropy_ledger, sft/information_science/entropy_uncertainty/law.py:31; V3 closed_pairs, sft/information_science/entropy_uncertainty/law.py:43. Native results: SFT.InformationScience.EntropyLedger.rowPartThroughOne, EntropyLedger.lean:139; SFT.InformationScience.EntropyLedger.recordClass, EntropyLedger.lean:151; SFT.InformationScience.EntropyPairs.pairSubsetLength, EntropyPairs.lean:141; SFT.InformationScience.EntropyPairs.refinementPairCount, EntropyPairs.lean:156. Timeline: Turn 142, line 1779; Timeline: Turn 142, line 1780. Evidence: Completion TURN_143_ENTROPY_COMPLETION.

97. Entropy successor closure and exact finite completion

For one deterministic microstate, the exact partition is one class whose native normalized support part is One and whose unresolved ledger is structural empty One. Adding a fresh state preserves every prior image and appends the new observation in source order. Its assigned class appends precisely that state; every other class retains its member trace. The pair successor preserves the original nested order and contains exactly the old class pairs together with one pair from each prior same-class member to the fresh state. Each returned class part satisfies the exact member-count/new-support-count ratio. The changed total is retained in the weights of all classes.

The general successor derivation, exact weighted class ledger and refinement containment/count results complete the original finite entropy statement. All 512 candidate kernels and all five operational witnesses are retained; fresh native program output matches the pinned census, actual lookup decisions, registration, closure, controls and digest fields. The retained 9805 independent input comparisons cover partition/closed pairs, raw unresolved pairs, refinement and normalized Fraction records, including the

original invalid and duplicate input domains. Entropy and Uncertainty is complete at V3's declared finite boundary. Conventional scalar Shannon summaries remain subject to the separately declared source correspondence map.

For a fresh state x∉S, let S'=S++[x] and O'=O++[(x,v)]. Last-row lookup gives f_{O'}(x)=v. Induction over the observation rows, using x≠s for every old state s, gives f_{O'}(s)=f_O(s). Consequently the ordered observation and fibre equations are

$$\mathrm{observedValues}(O', S') = \mathrm{observedValues}(O, S) \mathbin{+\!\!+} [v], \qquad F'_l = F_l \mathbin{+\!\!+} \begin{cases} [x], & v = l, \\ [], & v \neq l. \end{cases}$$

Freshness preserves distinct support and distinct observation keys. The old key coverage extends in both directions by the same appended key x, so partition admission is preserved. The assigned fibre appends x; the others retain their complete ordered traces, including when v is a newly occurring label.

The ordered successor pairs are described by E_x([])=[] and

$$E_x(y :: R) = \big([(y, z) : z \in R] \mathbin{+\!\!+} [(y, x)]\big) \mathbin{+\!\!+} E_x(R).$$

Expanding the positional-pair recurrence and distributing the head map over R++[x] proves P(M++[x])=E_x(M) by induction. Splitting each concatenation in that induction also proves

$$p \in E_x(M) \iff p \in P(M) \ \vee \ \exists y \in M,\ p = (y, x).$$

Thus each new pair appears at its original nested-loop position, alongside the preserved prior pairs. This formula applies directly to the changed fibre's unresolved-pair contents.

For any returned successor record of label l with normalized part (a'_l,b'_l), the exact-part identity and the fibre equation give

$$\big(|F_l| + \mathbf{1}_{v=l}\big) b'_l = a'_l (|S| + 1).$$

All class weights therefore use the new total, including unchanged fibres. At the base S=[x], O=[(x,v)], key coverage and distinctness hold directly, and the partition is [(v,[x])]. Normalization of (1,1) gives numerator and denominator both 1, while P([x])=[] gives structural empty-One uncertainty. These base and successor equations retain the exact finite construction.

> Source, proof and provenance: V3 entropy_ledger, sft/information_science/entropy_uncertainty/law.py:31. Native results: SFT.InformationScience.EntropyExtension.singletonBase, EntropyExtension.lean:194; SFT.InformationScience.EntropyExtension.singletonWholePart, EntropyExtension.lean:201. Timeline: Turn 142, line 1787. Evidence: Completion TURN_143_ENTROPY_COMPLETION.

98. Exact compression descriptions and reconstruction provenance

A dictionary description is admitted precisely when its dictionary and token trace are nonempty, dictionary keys are unique and every retained token has a dictionary entry. The executable source allows empty dictionary blocks and empty string labels; the translation retains these domains. Reconstruction concatenates each referenced block in token order. The native lookup theorem connects each selected block to its unique retained dictionary row, and the provenance theorem identifies the supplying token and block for every reconstructed cell.

Losslessness is equivalent to a nonempty source message and exact equality with the reconstructed message. An empty source message returns false without attempting reconstruction, preserving the source short circuit. Malformed descriptions retain the original reconstruction error, while a valid unequal reconstruction gives the original redundancy error. The redundancy trace retains a token/cell pair for each produced occurrence; projecting its cells recovers the exact source message, and its length equals the source length.

The resource record retains the dictionary rows in order, every token cell in order, the reconstructed trace and their complete resource concatenation. Native count identities retain every dictionary row and token occurrence. One retained token reconstructs exactly its registered block; appending any valid token appends precisely its block to the message and one cell to the resource trace. These results establish the source token-successor clause. Independent comparisons cover 1720 reconstruction/resource cases and 6880 lossless/redundancy cases, including duplicate keys, missing tokens, empty blocks, repeated tokens and empty-message behavior. Prefix parsing and the complete fixed/short support boundary remain distinct components of the Compression and Redundancy entry.

Write D for the ordered dictionary, T for the token list and B_D(t) for first-key lookup's block. Description admission is exactly

$$V(D,T) \iff D \neq [] \wedge \mathrm{Distinct}(\mathrm{keys}(D)) \wedge T \neq [] \wedge \forall t \in T,\ t \in \mathrm{keys}(D).$$

Neither nonempty blocks nor nonempty string labels are added to this predicate. With C_D(T)=concat_{t∈T}B_D(t), reconstruction obeys

$$\mathrm{reconstruct}(D,T) = \mathrm{ok}(W) \iff V(D,T) \wedge W = C_D(T).$$

Admission supplies a dictionary row for each token; distinct keys make its first lookup equal that row's block. Membership in concatenation then proves both directions of the provenance statement

$$c \in W \iff \exists t, U,\ t \in T \wedge (t,U) \in D \wedge c \in U.$$

The forward direction selects a supplying token block; the reverse direction uses unique-key lookup to insert that block's cell into the concatenation. Repeated tokens retain repeated output occurrences.

The lossless result is true precisely when M≠[] and reconstruct(D,T)=ok(M). For M=[], the result is false immediately, including for an invalid description. For a nonempty message, reconstruction errors propagate; successful unequal reconstruction yields false. Redundancy returns the original exact-reconstruction error for a false result and propagates reconstruction errors otherwise.

Define the ordered occurrence ledger

$$Q_D(T) = \mathrm{concat}_{t \in T}[(t,c) : c \in B_D(t)].$$

Projection commutes with concatenation, and projecting each block's pairs returns that block. Induction over T therefore proves map(second,Q_D(T))=C_D(T). For a lossless description this is M, and length preservation under mapping yields |Q_D(T)|=|M|. The ledger retains occurrences rather than merely the set of supplying tokens.

The resource traces are

$$A(D) = [\,[\mathrm{dictionary}, t] +\!+ U : (t,U) \in D\,], \qquad H(T) = [\,[\mathrm{token}, t] : t \in T\,], \qquad R(D,T) = A(D) +\!+ H(T).$$

Their outer list lengths satisfy |A(D)|=|D|, |H(T)|=|T| and |R(D,T)|=|D|+|T|. These count retained resource rows, not character lengths. For a valid description and a dictionary row (t,U), appending t preserves admission and gives

$$C_D(T +\!+ [t]) = C_D(T) +\!+ U, \qquad R(D, T +\!+ [t]) = R(D,T) +\!+ [[\mathrm{token}, t]].$$

The decoding equality follows by distributing concatenation over appended token lists and using unique-key lookup. For the singleton token list [t], the same argument gives reconstruct(D,[t])=ok(U), including U=[]. This establishes the base and token-successor clauses without changing the executable domain.

> Source, proof and provenance: V3 reconstruct, sft/information_science/compression_redundancy/law.py:24; V3 description_resource_trace, sft/information_science/compression_redundancy/law.py:31. Native results: SFT.InformationScience.CompressionDescription.redundancyExact, CompressionDescription.lean:219;

> SFT.InformationScience.CompressionDescription.resourceSuccessor, CompressionDescription.lean:232; SFT.InformationScience.CompressionCanonical.inheritedFixedSupport, CompressionCanonical.lean:5; SFT.InformationScience.CompressionCanonical.inheritedNoInjectiveShortCode, CompressionCanonical.lean:10. Timeline: Turn 144, line 1797; Timeline: Turn 145, line 1807. Evidence: Completion TURN_145_COMPRESSION_COMPLETION.

99. Prefix parsing and the complete-support shortening boundary

The prefix-family predicate rejects empty families, empty words, repeated words and any distinct pair for which the shorter word is the initial segment of the longer. A native common-initial-segment argument shows that two parses of the same stream must begin with the same admitted word. Cancelling that word and repeating over the finite traces proves unique parsing for arbitrary admitted concatenations. Explicit dictionary token boundaries are retained by the reconstruction representation.

Fixed support retains the complete ordered Cartesian product at the specified width. The shorter support concatenates all products at positive widths below that width, in increasing width order. Native membership and count proofs establish the complete original enumeration and its strict multiplicity-count inequality whenever the raw helper accepts. The pigeonhole argument rules out a pairwise-distinct shorter assignment to every generated occurrence. For set-injective coding of distinct messages, the distinct-label requirement is inherited from the registered Information Quantity complete_word_support domain, rather than added to Compression's executable guard. A native bridge proves that this canonical support is exactly the Compression product and establishes the no-injective-short-code theorem on that domain. The broader raw helper accepts repeated labels; its list-count result is preserved, without treating repeated equal words as distinct messages. The native raw helper therefore preserves repeated alphabet labels and their output multiplicities without an additional distinctness guard.

Compression and Redundancy retains all 1,024 kernels and all five operational witnesses. Fresh native execution matches every original program field, including actual candidate lookups, controls and digests. The cumulative 11,626 source-expression comparisons cover exact descriptions, errors, prefix families and complete fixed/short supports. The result applies at every admitted finite width: the recursively generated full support is larger than the complete shorter support, and injectivity on the canonical distinct-label support would contradict that strict count inequality. Prefix parsing separately covers every finite concatenation over an admitted prefix-free family.

For a family W of words, the source tests proper prefixes by

$$\mathrm{properPrefix}(u,v) \iff |u| < |v| \wedge \mathrm{take}_{|u|}(v) = u.$$

Prefix admission requires W≠[], every word nonempty, distinct words, and no proper-prefix relation between distinct members. Suppose two admitted parses begin with u and v and have equal concatenations, u++A=v++B. Cancelling equal leading cells recursively proves that either v=u++r or u=v++r. If r is nonempty, the shorter word is a proper prefix of the longer, contradicting admission; if r is empty, u=v. Cancel this common first word and apply induction to the remaining parses. A nonempty parse cannot equal an empty parse because its first word is nonempty. Thus for any two finite parses L,R over W,

$$\mathrm{concat}(L) = \mathrm{concat}(R) \implies L = R.$$

For an ordered label list A, define the complete word list recursively by

$$W_0 = [\,[\,]\,], \qquad W_{n+1} = \mathrm{concat}_{a \in A}[a :: w : w \in W_n].$$

The raw fixed-support helper admits exactly |A|≥2 and a nonempty position list P, and returns $W_{|P|}$. It does not test distinctness of A. Every list occurrence is retained. With q=|A|, distributing list length over the outer concatenation gives $|W_{n+1}|$=q$|W_n|$ and hence $|W_n|$=q^n. Membership is exactly word length n with every cell drawn from A.

Define T_0=[] and T_{n+1}=T_n++W_{n+1}. Then

$$w \in T_n \iff 0 < |w| \le n \wedge \forall c \in w,\ c \in A, \qquad |T_n| = \sum_{j=1}^{n} q^j.$$

The shorter nonempty support for a width-N source is T_{N-1}, preserving increasing width order. For q≥2, its strict count bound follows without omitting any generated word. At n=0, $|T_0|$=0<q=$|W_1|$. If $|T_n|$<$|W_{n+1}|$, then

$$|T_{n+1}| = |T_n| + |W_{n+1}| < 2|W_{n+1}| \le q|W_{n+1}| = |W_{n+2}|.$$

Therefore every admitted raw fixed support satisfies $|T_{N-1}|$<$|W_N|$, including when repeated labels produce repeated equal words.

For any duplicate-free list C contained in a finite list T, the removal induction gives |C|≤|T|: remove the head of C from T, use the head's membership to make T strictly shorter, and use C's distinctness to keep its tail contained in the remainder. If a proposed code f maps every generated full-support word into T_{N-1} and the full output list map(f,W_N) is duplicate-free, this yields

$$|W_N| = |\operatorname{map}(f, W_N)| \le |T_{N-1}| < |W_N|,$$

a contradiction. This establishes the occurrence-list result on the raw helper's exact domain.

For the registered complete_word_support domain, admission already supplies distinct alphabet labels. Induction through the Cartesian-product construction makes W_N duplicate-free: different leading labels separate blocks, and the tail induction separates words within each block. Injectivity of f on W_N then makes map(f,W_N) duplicate-free. The canonical bridge returns the identical W_N from the Compression helper, so the preceding contradiction proves the no-injective-short-code result on that inherited domain. The bridge adds no condition to the raw executable helper and preserves both source domains.

> Source, proof and provenance: V3 prefix_free, sft/information_science/compression_redundancy/law.py:15; V3 universal_shortening_impossible, sft/information_science/compression_redundancy/law.py:71. Native results: SFT.InformationScience.CompressionPrefix.uniqueParse, CompressionPrefix.lean:103; SFT.InformationScience.CompressionSupport.noInjectiveShortCode, CompressionSupport.lean:183. Timeline: Turn 442, line 3944; Timeline: Turn 182, line 2066. Evidence: Completion TURN_145_COMPRESSION_COMPLETION.

100. Finite channel admission and exact confusability

An admitted finite channel retains nonempty duplicate-free input and output carriers and a duplicate-free relation within those carriers. Every input has at least one image; unused output-carrier entries remain permitted. The native constructor preserves the original order of carrier, repeated-cell, endpoint and totality checks, including their distinct errors. Empty string labels remain allowed.

Output support filters the retained relation by the queried source and preserves every corresponding target in relation order. Native theorems establish exact cell membership, membership within the output carrier, nonemptiness for admitted inputs and absence of repeated targets. A query outside the input carrier retains the original error. Confusability is equivalent to the existence of a retained target shared by both inputs. The overlap is symmetric, and every admitted input is self-confusable because its support is nonempty. No probability or error threshold enters these predicates.

The distinguishable-code predicate retains its original nonempty, unique-input and input-coverage guards and tests every positional input pair. Its native characterization establishes that admitted codes have pairwise disjoint output supports. Every admitted single input forms a distinguishable singleton code. Independent source-expression comparisons pass 3,913 constructor cases, 600 output-support cases,

2,400 confusability cases and 6,000 code cases, covering malformed and empty carriers, duplicate cells, missing transport, empty labels, repeated held inputs and invalid queries. Complete capacity selection, composition and the fresh-carrier successor are established below.

Write a channel as (I,O,R), where I and O are ordered carriers and R is the ordered list of transport cells. Constructor success is equivalent to

$$I \neq [] \wedge O \neq [] \wedge \mathrm{Distinct}(I) \wedge \mathrm{Distinct}(O) \wedge \mathrm{Distinct}(R) \wedge R \subseteq I \times O \wedge \forall x \in I,\ \exists y,\ (x,y) \in R.$$

The constructor checks carrier validity first, then repeated cells, then endpoint containment, then totality; the first failed condition returns its corresponding source error. Success retains I,O,R exactly. No output-surjectivity requirement is imposed.

For an input x, define the ordered support

$$S_R(x) = \mathrm{map}(\mathrm{second}, \mathrm{filter}_{\mathrm{first}=x}\ R).$$

Unpacking filter and map membership proves y∈S_R(x) iff (x,y)∈R. Totality therefore supplies a support member for every admitted x, while endpoint containment places every support member in O. Distinctness follows because two filtered cells have the same first coordinate x, so equality of their second coordinates implies equality of the cells. A query succeeds exactly for x∈I and returns this ordered support; other queries return the outside-input error.

The overlap predicate has the exact characterization

$$\mathrm{overlap}_R(x,z) \iff \exists y,\ (x,y) \in R \wedge (z,y) \in R.$$

Swapping the two conjuncts proves symmetry. Totality supplies the same witness twice to prove self-confusability for admitted inputs. The public confusability operation checks the left input's support, then the right input's support, preserving that error order, and returns overlap when both queries succeed.

Let P(H) be the positional-pair list of a held input list H. Its code predicate is

$$\mathrm{Code}(H) \iff H \neq [] \wedge \mathrm{Distinct}(H) \wedge H \subseteq I \wedge \forall (x,z) \in P(H),\ \neg\, \mathrm{overlap}_R(x,z).$$

A shared support target would witness overlap and contradict the final clause. Conversely, absence of any shared target makes that Boolean overlap false for every tested pair. For H=[x] with x∈I, the pair list is empty, so all code conditions hold directly. These are exact finite relations and guards, without probability weights.

> Source, proof and provenance: V3 confusable, sft/information_science/channels_capacity/law.py:37; V3 __post_init__, sft/information_science/channels_capacity/law.py:20. Native results: SFT.InformationScience.ChannelRelation.rawSupportDistinct, ChannelRelation.lean:207; SFT.InformationScience.ChannelRelation.admittedSelfConfusable, ChannelRelation.lean:218. Timeline: Turn 146, line 1816. Evidence: Completion TURN_148_CHANNEL_COMPLETION.

101. Complete finite capacity families and relation composition

Channel candidate generation enumerates each positive selection size in increasing order and, within that size, every input combination in source-index order. A native selection relation characterizes precisely these retained subsequences. The enumeration theorem establishes exact length, carrier membership and complete generation; distinct admitted inputs yield duplicate-free selections and candidates. Filtering retains exactly the original distinguishable codes. Every admitted input supplies a singleton candidate, so the admitted candidate family is nonempty.

The capacity computation takes the greatest candidate length and retains every candidate attaining it, in the original enumeration order. Native proofs establish the maximum bound, attainment and equivalence between returned membership and being a distinguishable canonical selection at least as large as every other such selection. No tie rule removes an alternative. Admitted capacity is nonempty and contains no repeated records. A one-input channel returns exactly its singleton code family.

Composition requires equality of the ordered interface carriers. It traverses the first relation and then each matching second relation row, keeping the first occurrence of each resulting endpoint cell. Native proofs establish that composite membership is equivalent to a retained two-edge path through the interface, and that admitted channels compose to a total admitted channel. If two inputs share an intermediate output, totality of the second channel supplies a common final output; composition therefore cannot restore that merged distinction.

Independent source-expression comparisons pass 186 combination enumerations, 399 complete candidate/capacity families and 6,272 compositions, covering all retained ties, repeated raw combination labels, relation-row permutations, first-occurrence duplicates and ordered-interface mismatch errors. The fresh-input successor and complete candidate program complete the Channel Capacity entry as established below.

A canonical selection is an order-preserving subsequence of the carrier, generated by the empty, skip and take constructors. The complete size-k enumeration obeys

$$B_0(I) = [\,[]\,], \qquad B_{k+1}([]) = [], \qquad B_{k+1}(x :: I) = \mathrm{map}(x :: \cdot, B_k(I)) +\!+ B_{k+1}(I).$$

Induction on the carrier matches the first branch to take and the second to skip, proving H∈B_k(I) iff |H|=k and H is a canonical selection of I. With T_0(I)=[] and T_{k+1}(I)=T_k(I)++B_{k+1}(I), this gives

$$H \in T_n(I) \iff 0 < |H| \le n \wedge \mathrm{Chosen}(H, I).$$

Every chosen selection has length at most |I|. Filtering T_{|I|}(I) by Code therefore gives exactly all canonical distinguishable codes. For distinct I, take and skip branches cannot share a selection: a take selection contains the head x, whereas x is absent from the remaining carrier. Induction proves distinctness within each size; unequal lengths separate different sizes. Filtering preserves that distinctness. Any admitted input supplies a singleton candidate, establishing nonemptiness.

For the candidate list C, the maximum recurrence is

$$M([]) = 0, \qquad M(H :: C) = \begin{cases} M(C), & |H| < M(C), \\ |H|, & \mathrm{otherwise}. \end{cases}$$

Induction proves both |H|≤M(C) for every H∈C and M(C)≤b whenever every candidate has length at most b. On a nonempty list, either the head attains the recurrence value or the tail induction supplies a maximizer. Consequently the retained family K=[H∈C:|H|=M(C)] satisfies

$$H \in K \iff H \in C \wedge \forall G \in C,\ |G| \le |H|.$$

Every tie is retained in the original enumeration order. Nonempty C gives nonempty K; distinct C gives distinct K. The raw empty-candidate case retains the source max-empty-sequence error. For a one-input carrier [x], the candidate list and capacity family are both [[x]].

For composition, the ordered output carrier of the first channel must equal the ordered input carrier of the second. Let

$$P(R_1, R_2) = \mathrm{concat}_{(x,y)\in R_1}[(x, z) : (y, z) \in R_2], \qquad R_{21} = \mathrm{firstCells}(P(R_1, R_2)).$$

The nested enumeration retains relation order; firstCells skips cells already seen and otherwise emits the cell before continuing with it added to the seen list. Induction proves that a cell survives this scan iff it occurs in the input and was absent from the initial seen list, and that the output is duplicate-free. Starting with no seen cells preserves membership, so

$$(x,z) \in R_{21} \iff \exists y,\ (x,y) \in R_1 \wedge (y,z) \in R_2.$$

Endpoint containment follows from the two component relations. For each first-channel input x, its totality supplies y; interface equality makes y an admitted second-channel input, whose totality supplies z. This proves composite totality and hence admission with the retained endpoint carriers.

Finally, if x and x' share y in R_1, second-channel totality supplies one z with (y,z)∈R_2. The path characterization gives both (x,z) and (x',z) in R_{21}. Therefore

$$\mathrm{overlap}_{R_1}(x,x') \implies \mathrm{overlap}_{R_{21}}(x,x').$$

Thus the no-restored-distinction statement follows from the actual finite relation composition and its totality, with no replacement transport rule.

> Source, proof and provenance: V3 capacity_family, sft/information_science/channels_capacity/law.py:51; V3 compose_channels, sft/information_science/channels_capacity/law.py:62. Native results: SFT.InformationScience.ChannelSelections.throughDistinct, ChannelSelections.lean:166; SFT.InformationScience.ChannelSelections.candidatesDistinct, ChannelSelections.lean:178; SFT.InformationScience.ChannelCapacity.singletonCapacity, ChannelCapacity.lean:120; SFT.InformationScience.ChannelCapacity.familyDistinct, ChannelCapacity.lean:126; SFT.InformationScience.ChannelComposition.compositionPaths, ChannelComposition.lean:150; SFT.InformationScience.ChannelComposition.noRestoredDistinction, ChannelComposition.lean:156. Timeline: Turn 147, line 1823; Timeline: Turn 147, line 1824. Evidence: Completion TURN_148_CHANNEL_COMPLETION.

102. Fresh-input closure of exact channel capacity

Appending a fresh input and its complete nonempty, duplicate-free support within the held output carrier preserves channel admission. Output support for every prior input is unchanged, while the new input has exactly the appended support in its original order. Every prior confusability decision remains unchanged. The new decision with each prior input is precisely the overlap of its retained support with the new support.

Every prior distinguishable code remains valid. Its extension by the fresh input is distinguishable exactly when each prior member has disjoint support from the fresh input. The native exhaustive successor theorem establishes that every generated code is an old code, the fresh singleton, or such a permitted extension of an old code, and conversely every one of those alternatives is generated. This closes arbitrary finite carrier growth while retaining the full candidate family used by the greatest-cardinality selection.

All 1,024 channel kernels and all five original operational witnesses are retained. Fresh native execution matches every original registration, census, actual candidate lookup, closure, control and digest field. The cumulative 20,505 source-expression comparisons include 735 fresh-input support/code checks alongside constructor, support, confusability, capacity and composition cases. Channel Capacity is complete at the source finite zero-error boundary; probabilistic error rates and asymptotic coding claims remain outside that original boundary.

Let C=(I,O,R) be admitted, let x∉I be fresh, and let T be a nonempty distinct list contained in O. The source extension is

$$I' = I +\!\!+ [x], \qquad O' = O, \qquad R' = R +\!\!+ [(x,t) : t \in T].$$

Distinctness of I' follows from freshness. The new cells are distinct because their second coordinates are distinct; no new cell can coincide with an old cell, since endpoint containment would put x in I. Old endpoints remain inside the extended carriers, and new endpoints lie there by T⊆O. Old inputs retain a transport witness, while nonempty T supplies one for x. These four facts establish the extended constructor's admission conditions.

Filtering distributes over relation concatenation. For the fresh rows alone,

$$S_{[(x,t):t\in T]}(s) = \begin{cases} T, & s = x, \\ [], & s \neq x. \end{cases}$$

Hence S_{R'}(s)=S_R(s) for every old s. No old cell begins at x, so S_R(x)=[] and S_{R'}(x)=T. The preserved support equations imply

$$\mathrm{overlap}_{R'}(s,u) = \mathrm{overlap}_{R}(s,u) \quad (s,u \in I), \qquad \mathrm{overlap}_{R'}(s,x) \iff \exists t \in T,\ (s,t) \in R.$$

For any held list H contained in I, its nonemptiness, distinctness and old pair decisions are unchanged, proving Code_{C'}(H) iff Code_C(H). If H is already a code, freshness preserves distinctness of H++[x]. Its positional pairs consist exactly of old pairs and pairs (s,x) for s∈H, in the established interleaved source order. Therefore

$$\mathrm{Code}_{C'}(H +\!\!+ [x]) \iff \forall s \in H,\ \neg\, \mathrm{overlap}_{R'}(s,x).$$

For the converse restriction used below, a nonempty H contained in I with Code_{C'}(H++[x]) inherits distinctness and all old pair tests from that longer code. The old overlap identity then proves Code_C(H).

Write K(C) for the complete candidate list, before selecting maximum cardinality. Induction through the take/skip constructors shows that any canonical selection of I++[x] either omits the final x and is chosen from I, or equals H++[x] for H chosen from I. In the latter case H=[] yields the fresh singleton; otherwise the restriction argument supplies an old code and the extension test supplies separation. This proves the exhaustive membership equivalence

$$L \in K(C') \iff L \in K(C) \ \vee\ L = [x] \ \vee \exists H \in K(C),\ L = H +\!\!+ [x] \wedge \forall s \in H,\ \neg\, \mathrm{overlap}_{R'}(s,x).$$

Conversely, an old selection remains chosen in the appended carrier; [x] is a singleton code; and appending x to a chosen old code remains a canonical selection and passes exactly the stated separation tests. Thus every listed alternative is generated, and none is missing. This is a membership characterization of the complete existing enumeration, not a replacement ordering for that enumeration. The maximum-family construction continues to act on the full ordered candidate list.

> Source, proof and provenance: V3 capacity_family, sft/information_science/channels_capacity/law.py:51. Native results: SFT.InformationScience.ChannelExtension.chosenAppendNew, ChannelExtension.lean:232; SFT.InformationScience.ChannelExtension.candidateAlternatives, ChannelExtension.lean:239. Timeline: Turn 148, line 1831. Evidence: Completion TURN_148_CHANNEL_COMPLETION.

103. Registered action relations and exact correction

An error relation retains nonempty duplicate-free source and received word carriers, duplicate-free source/image/action triples, and at least one image for every source. Every triple lies within the held carriers and has a nonempty action label. Empty words and unused received entries remain permitted. Distinct actions may connect the same source and image. The native constructor retains the original guard order and errors.

Image and predecessor extraction filters the original action relation, then removes repeated words in first-occurrence order. Native membership equivalences establish that a word is retained exactly when a corresponding action cell exists. Images of an admitted source are nonempty and lie in the received

carrier; every retained predecessor lies in the source carrier. Both lists are duplicate-free. An unused received entry has an empty predecessor class, preserving the original domain.

Correction is unique exactly when the complete registered predecessor list is a singleton. The unique record retains that source; every other list, including the empty list, is returned intact as a closed source class. Native proofs establish that a uniquely corrected source has a registered action cell and that any registered predecessor of that received word equals the corrected source. No likelihood, distance rule or inferred predecessor enters the construction. Independent source-expression comparisons pass 7,693 constructor cases, 738 image extractions, 738 predecessor extractions and 1,476 correction decisions/records.

Let S and V be source and received word lists, and R the ordered list of triples (s,v,a). Admission requires nonempty distinct S,V, distinct R, and

$$\forall (s,v,a) \in R,\ s \in S \wedge v \in V \wedge a \neq \text{empty string}, \qquad \forall s \in S,\ \exists v, a,\ (s,v,a) \in R.$$

The checks occur in this order: carriers, repeated triples, endpoint/action conditions, totality. Success returns the original lists. Empty words remain allowed; different actions on the same source/image pair remain distinct triples.

Let F be the first-occurrence word filter. Its scan skips a word already in the seen list, otherwise emits it and adds it to that list. Induction proves membership in the scan output iff membership in its input and absence from its initial seen list. Emitted words cannot recur because each is thereafter seen; hence F preserves membership and produces a distinct ordered list. Define

$$I_R(s) = F([v : (s,v,a) \in R]), \qquad P_R(v) = F([s : (s,v,a) \in R]).$$

Filter and projection membership, followed by membership preservation of F, give

$$v \in I_R(s) \iff \exists a,\ (s,v,a) \in R \iff s \in P_R(v).$$

Thus totality makes each admitted source's image list nonempty, and endpoint containment places images in V and predecessors in S. Querying an image list requires s∈S; querying predecessors requires v∈V. An unused admitted v has P_R(v)=[], whereas a query outside V retains its error.

The correction record is defined by the complete predecessor list:

$$\text{record}(P) = \begin{cases} \text{uniqueSource}(s), & P = [s], \\ \text{closedSourceClass}(P), & |P| \neq 1. \end{cases}$$

Splitting a list into empty, singleton, and at-least-two cases proves |P|=1 iff P=[s] for some s, and record(P)=uniqueSource(s) iff P=[s]. Therefore

$$\text{correction}(v) = \text{ok}(\text{uniqueSource}(s)) \iff v \in V \wedge P_R(v) = [s].$$

Membership of s in that singleton supplies a registered action (s,v,a). Any other registered predecessor t belongs to P_R(v) by the same membership equivalence and must equal s. If |P_R(v)|≠1, the record returns the entire P_R(v), including the empty list, with no selected replacement predecessor.

> Source, proof and provenance: V3 uniquely_correctable, sft/information_science/noise_error/law.py:61; V3 correction_record, sft/information_science/noise_error/law.py:65. Native results: SFT.InformationScience.NoiseRelation.admittedImagesInside, NoiseRelation.lean:242; SFT.InformationScience.NoiseRelation.admittedPredecessorsInside, NoiseRelation.lean:250; SFT.InformationScience.NoiseCorrection.correctionRegistered, NoiseCorrection.lean:105; SFT.InformationScience.NoiseCorrection.noUnrecordedCorrection, NoiseCorrection.lean:112. Timeline: Turn 149, line 1838; Timeline: Turn 149, line 1839. Evidence: Completion TURN_150_NOISE_COMPLETION.

104. Complete position traces, detection and action succession

Word comparison requires equal generated position carriers and retains the original error for unequal widths. Complete ordered position pairs are filtered by label inequality, retaining each changed position and both its source and received labels. Positions are rendered with the original one-based position labels. Native indexed membership proves exact correspondence at every position of arbitrary finite words. The resulting trace is structural empty One exactly when the words are identical; otherwise it is the complete ordered changed-position trace.

The detection function retains identity as true. For nonidentical equal-width forms, detection is exactly received-word nonmembership in the valid codebook. No source-membership guard, probability threshold or distance measure is added. Unequal-width errors propagate unchanged.

The identity-action base admits a single source and yields its singleton predecessor class, unique correction and empty-One trace. At each finite successor, appended registered action cells extend the received support by their exact images. Native admission preservation retains the original carrier and cell conditions. Complete image and predecessor membership after extension is equivalent to prior membership or a new matching action cell. First-occurrence order equations retain the original traversal and deduplication behavior. These results cover arbitrary finite action additions, including shared images, without selecting among ambiguous predecessors.

Noise and Error retains all 1,024 kernels, all five operational witnesses and both admitted witness relations. Fresh native execution matches every original registration, census, actual candidate lookup, closure, control, hash and unknown-identifier result. The cumulative 18,867 source-expression comparisons include 1,600 trace, 6,400 detection and 222 successor comparisons. The entry is complete at its finite structural boundary; stochastic frequencies, physical noise mechanisms and asymptotic reliability remain outside that original claim.

At an initial index k, the position-pair recursion emits (k,s_0,v_0) and continues at k+1 on the two tails; it stops when either word ends. Its indexed membership theorem is

$$(i,a,b) \in Q_k(s,v) \iff \exists j,\ i = k + j \wedge s[j] = \mathrm{Some}(a) \wedge v[j] = \mathrm{Some}(b).$$

The head case has j=0; the tail case increments j and uses k+1+j=k+(j+1). For equal-width words, induction also gives |Q_k(s,v)|=|s|. Filtering Q_k by a≠b gives exactly the changed positions, retaining order and both labels. Presentation maps (i,a,b) to ("position-" followed by the decimal representation of i+1,a,b).

For equal widths, the changed list is empty iff s=v: identical heads are removed and induction applies to the tails; unequal heads would be retained and contradict emptiness. Conversely, identical words have no unequal head at any step. Consequently

$$\mathrm{errorTrace}(s,v) = \mathrm{ok}(\mathrm{emptyOne}) \iff s = v.$$

For equal-width unequal words, the trace is the complete mapped changed list. Unequal widths return the common-position-carrier error before presentation. For any supplied valid-word list B, detection therefore satisfies

$$\mathrm{detectable}(B,s,v) = \mathrm{ok}\left(\begin{cases} \mathrm{true}, & s = v, \\ v \notin B, & s \neq v \end{cases}\right) \quad \text{when } |s| = |v|.$$

The unequal-width error propagates unchanged; no membership guard on s is introduced.

For appended action cells A, the exact extension is

$$S' = S, \qquad V' = F(V +\!\!+ [v : (s,v,a) \in A]), \qquad R' = R +\!\!+ A.$$

Assume the original relation is admitted, A is distinct and disjoint from R, and every added cell has its source in S and a nonempty action label. Concatenation preserves distinct triples; first-occurrence filtering preserves a nonempty distinct received carrier. Old endpoints remain admitted, new images enter V' by construction, and every old totality witness remains in R'. These facts prove extension admission.

Distributing filter and map over concatenation gives the exact order equations

$$P_{R'}(v) = F([s : (s, v, a) \in R] +\!+ [s : (s, v, a) \in A]),$$

$$I_{R'}(s) = F([v : (s, v, a) \in R] +\!+ [v : (s, v, a) \in A]).$$

Membership is consequently old membership or existence of a matching added action cell, in both directions. Shared images retain every predecessor through this same construction. At the identity base, S=V=[s] and R=[(s,s,"identity")]. The guards hold directly, P_R(s)=[s], correction returns uniqueSource(s), and the unchanged word has empty-One trace. This proves the base and finite action-successor clauses on the registered domain.

> Source, proof and provenance: V3 error_trace, sft/information_science/noise_error/law.py:43; V3 detectable, sft/information_science/noise_error/law.py:54. Native results: SFT.InformationScience.NoiseTrace.detectionMeaning, NoiseTrace.lean:179; SFT.InformationScience.NoiseTrace.unequalWidthError, NoiseTrace.lean:193; SFT.InformationScience.NoiseExtension.identityCorrection, NoiseExtension.lean:144; SFT.InformationScience.NoiseExtension.identityBase, NoiseExtension.lean:147. Timeline: Turn 150, line 1845. Evidence: Completion TURN_150_NOISE_COMPLETION.

105. Complete coding support and held codebooks

Coding support enumerates the full ordered Cartesian product of at least two distinct labels over a nonempty position list. Its native membership characterization retains exactly every word of the held width whose labels belong to the alphabet. The cardinality is the alphabet size raised to that width, and distinct alphabet labels give duplicate-free support. The original position-list domain is preserved, including repeated position names.

A codebook returns the original held selection, in order, when it is nonempty, duplicate-free and contained in the supplied support. Native admission and membership proofs show that selecting from complete coding support gives every retained codeword the same position width and allowed labels. No additional guard is imposed on the supplied support itself.

Codeword difference retains the complete indexed changed-position trace and both labels at each change. Equality gives empty One; unequal widths retain Coding's original common-carrier error. The general position correspondence follows the established exact Noise and Error derivation.

The repetition constructor maps each supplied label to its constant word over the full position list. Its original helper accepts repeated labels, preserving output multiplicity and order. Under complete support's distinct-label condition, nonempty width makes the repetition map injective and its words distinct; each word belongs to the generated support. Independent source-expression comparisons pass 600 complete-support cases, 600 repetition cases, 1,785 held-codebook cases and 1,600 difference cases.

For an alphabet list A and position list P, coding support succeeds precisely when

$$|A| \geq 2 \wedge \mathrm{Distinct}(A) \wedge P \neq [].$$

Put n=|P|. The output is the complete ordered word list W_n(A), with W_0(A)=[[]] and W_{n+1}(A)=concat_{a∈A}[a::w:w∈W_n(A)]. The head/tail decomposition proves by induction

$$w \in W_n(A) \iff |w| = n \wedge \forall a \in w,\ a \in A, \qquad |W_n(A)| = |A|^n.$$

For distinct A, different leading labels separate the generated blocks and the tail induction excludes repeats within each block. Thus the complete support is distinct. Position names enter this constructor through their list length; no distinct-position-name condition is added.

For an arbitrary supplied support U and held list H, the codebook result has the exact characterization

$$\mathrm{codebook}(U,H)=\mathrm{ok}(K)\iff H\neq[]\wedge\mathrm{Distinct}(H)\wedge H\subseteq U\wedge K=H.$$

The operation therefore retains the exact order and contents of H. If U=W_n(A) is admitted complete support, every w∈K belongs to U, so the preceding membership theorem gives |w|=n and every cell in A. For two retained words u,v, their two width equalities imply |u|=|v|. No guard on U beyond held-list membership is introduced by codebook itself.

For equal-width u,v, differenceTrace uses exactly the Noise and Error changed-position construction. Consequently

$$\mathrm{differenceTrace}(u,v)=\mathrm{ok}(\mathrm{emptyOne})\iff u=v,$$

and its changes result consists exactly of the one-based printed positions where u and v differ, with both labels retained. On unequal widths it returns Coding's own common-position-carrier error, rather than substituting Noise and Error's error text.

Define the constant word r_P(a)=map(λp.a,P). Repetition admission requires only |A|≥2 and P≠[], and returns [r_P(a):a∈A] in alphabet-list order. Induction on P gives

$$|r_P(a)|=|P|,\qquad b\in r_P(a)\Longrightarrow b=a.$$

Thus any emitted word from a label a∈A belongs to admitted W_n(A). Since P is nonempty, equality r_P(a)=r_P(b) forces equality of their first cells, a=b. The map is therefore injective, and distinct A yields a distinct repetition list. Repeated labels in the broader repetition helper remain repeated output words; this proof uses the distinctness already supplied by complete-support admission and does not change that helper's guard.

> Source, proof and provenance: V3 complete_support, sft/information_science/coding_theory/law.py:15; V3 codebook, sft/information_science/coding_theory/law.py:21. Native results: SFT.InformationScience.CodingSupport.repeatedInjective, CodingSupport.lean:166; SFT.InformationScience.CodingSupport.repetitionDistinct, CodingSupport.lean:172. Timeline: Turn 151, line 1854. Evidence: Completion TURN_153_CODING_COMPLETION.

106. Complete single-label error images and exact decoding

Single-label image generation begins with the unchanged source word, then visits every retained position and every supplied label in order, skipping only labels equal to the held label at that position. Native membership is equivalent to identity or to a decomposition into an unchanged prefix, one different supplied replacement label and an unchanged suffix. This equivalence includes both completeness and exclusion of unregistered replacements. Every image has the source width, and generated changes are nonidentity. A source in complete coding support has all its images in that support. The raw helper retains repeated supplied labels and their repeated outputs.

The complete error family retains each source occurrence with its full image list. Detection holds exactly when each image is either its source or outside the codebook. Correction holds exactly when every positional pair of source image families has no common received word. Native equivalences establish these conditions over the full generated lists, including raw duplicate source occurrences.

Decoding filters source occurrences in their original order by exact received-image membership. A singleton gives the original unique-source record; any other length gives the entire closed source class. The native correction theorem proves that disjoint source families yield an empty or singleton filtered list.

Consequently, every registered received image of a correctable code decodes to its source. Conversely, every unique decoded source is a retained codeword with that received image in its registered family. No distance or prior is used. Independent source-expression comparisons pass 1,600 image lists, 465 complete families, 465 detection decisions, 465 correction decisions and 2,790 decoding records.

For the supplied label list A, the complete changed-image recursion is

$$D_A([]) = [], \qquad D_A(h :: t) = [a :: t : a \in A,\ a \neq h] +\!\!+ [h :: v : v \in D_A(t)].$$

The image list is E_A(w)=[w]++D_A(w). The first branch changes the first position; the second retains it and recursively changes a later position. Induction, splitting on whether the unchanged prefix is empty, proves the exact equivalence

$$v \in D_A(w) \iff \exists p, h, t, a,\ w = p +\!\!+ (h :: t) \wedge a \in A \wedge a \neq h \wedge v = p +\!\!+ (a :: t).$$

The forward proof extracts the replacement from the head filter or prepends the unchanged head to the tail induction's prefix. Conversely, an empty prefix selects the head branch; a nonempty prefix selects the recursive branch. Thus E_A(w) contains precisely identity and every registered single-label replacement. Mapping and filtering retain the original label multiplicities and visit order.

Each branch preserves length, giving |v|=|w| for every v∈E_A(w). A changed image cannot equal w: cancelling the unchanged prefix would give a=h, contradicting the replacement guard. If all labels of w lie in A, each changed image also uses only A, since its unchanged cells retain that property and its replacement was selected from A. Together with preserved width, complete-support membership proves closure of all images inside the admitted support.

For a code list C, the family [(w,E_A(w)):w∈C] has exactly |C| rows, one per source occurrence. Expanding the Boolean all/any tests yields

$$\mathrm{Detect}(C, A) \iff \forall w \in C, \forall v \in E_A(w),\ v = w \vee v \notin C,$$

$$\mathrm{Correct}(C, A) \iff \forall (u, w) \in P(C),\ \neg\exists v,\ v \in E_A(u) \wedge v \in E_A(w),$$

where P(C) is the full positional pair list. The pair condition retains raw source occurrences rather than silently deduplicating C.

The received source class is the ordered filter

$$H_{C,A}(v) = [w \in C : v \in E_A(w)], \qquad w \in H_{C,A}(v) \iff w \in C \wedge v \in E_A(w).$$

Decoding applies the singleton/closed-class record constructor to this exact list. Hence decode(v)=uniqueSource(w) iff H_{C,A}(v)=[w]; every nonsingleton result retains H_{C,A}(v) intact.

To prove correct decoding, induct on C. If its head has no image v, the filtered list equals the tail's class and correction restricts to the tail's positional pairs. If the head does have image v, every tail occurrence must reject v: accepting it would give a shared image for that head–tail positional pair, contradicting Correct. The filtered tail is therefore empty and the whole class is the head singleton. Thus

$$\mathrm{Correct}(C, A) \Longrightarrow H_{C,A}(v) = [] \ \vee\ \exists w,\ H_{C,A}(v) = [w].$$

When w∈C and v∈E_A(w), class membership excludes the empty case and identifies the singleton as w, proving decode(v)=uniqueSource(w). Conversely, a unique-source record gives that singleton class; its member must be a retained source with v in its generated family. This proves both recovery and provenance using the full registered image enumeration.

> Source, proof and provenance: V3 single_label_images, sft/information_science/coding_theory/law.py:38; V3 decode_received, sft/information_science/coding_theory/law.py:68. Native results: SFT.InformationScience.CodingImages.changedLabels, CodingImages.lean:115; SFT.InformationScience.CodingImages.imageInCompleteSupport, CodingImages.lean:139; SFT.InformationScience.CodingDecoding.correctableDecode, CodingDecoding.lean:146; SFT.InformationScience.CodingDecoding.decodeRegistered, CodingDecoding.lean:159. Timeline: Turn 152, line 1860; Timeline: Turn 152, line 1861. Evidence: Completion TURN_153_CODING_COMPLETION.

107. General coding action succession and retained source provenance

A registered error family assigns its complete finite image list to each retained codeword. The general native detection, correction and decoding loops are definitionally the original single-label loops when instantiated with those original images. The singleton-action base supplies one image per codeword; correction then holds exactly when every positional pair has different action images.

Appending an action's registered images preserves the old image list and its order. The detection successor theorem is an equivalence: extended detection holds exactly when both prior and added images satisfy the original codebook-membership rule. The correction successor theorem checks every old–old, old–new, new–old and new–new pair of image supports. Their joint disjointness is equivalent to correction of the extended family. The received source class extends by exactly the sources whose new image lists contain that form. These proofs apply to arbitrary finite registered actions and preserve raw list multiplicity.

The stated total source/codeword relation retains one codeword for each registered source. Native recovery proves that every received image of that codeword in a correctable family decodes to the held codeword while preserving the source row. The same recovery result follows after an action addition precisely when the complete successor disjointness conditions hold. No injective message assignment beyond the stated source-map constraint is imposed.

The original width-three repetition example is a held codebook and its complete single-label image families are disjoint. The one-position uncoded control fails correction; a width-two repetition family also fails, giving the exact finite separation witness for the retained width. Coding includes all 2,048 original kernels and all six original operational witnesses. The cumulative 19,745 source-expression comparisons include 9,375 action-successor detection, correction and decoding outputs. Coding remains within the original finite classical boundary, without a stochastic error model, metric threshold, imported asymptotic rate or quantum-code claim.

Fresh native execution of the complete Coding program matches all 2,048 original candidate lookups together with every registration, census, closure, control and digest field and the unknown-identifier rejection. These results accompany the native source derivations and complete the Coding entry on its original finite classical domain.

Let F(w) be the ordered image list registered for codeword w. For action cells R, its registered-image constructor is [v:(w,v,a)∈R], retaining all matching occurrences. Membership is equivalent to existence of a matching action a. The general detection, correction and source-class definitions are exactly those of section 106 with E_A(w) replaced by F(w); substituting E_A recovers the original single-label operations by definition.

For a singleton action f, put F_f(w)=[f(w)]. Two such lists overlap iff their sole values agree, so

$$\mathrm{Correct}(C, F_f) \iff \forall (u,w) \in P(C),\ f(u) \neq f(w).$$

For old and added families F,G, define (F⊕G)(w)=F(w)++G(w). Membership splits into the old or added list. Applying that split to the universal detection condition proves both directions of

$$\mathrm{Detect}(C, F \oplus G) \iff \mathrm{Detect}(C, F) \wedge \mathrm{Detect}(C, G).$$

Define X_C(F,G) to mean that, for every positional pair (u,w)∈P(C), no received word belongs to both F(u) and G(w). Expanding the two append memberships yields precisely four cases:

$$\mathrm{Correct}(C, F \oplus G) \iff \mathrm{Correct}(C, F) \wedge X_C(F, G) \wedge X_C(G, F) \wedge \mathrm{Correct}(C, G).$$

For the forward direction, each alleged shared image in one case embeds into the two extended lists and contradicts their disjointness. Conversely, an overlap in the extended lists belongs to one of old–old, old–new, new–old or new–new, and the corresponding conjunct excludes it. Both cross orientations are retained because P(C) uses source order.

The extended source class remains the single ordered filter

$$H_{C,F\oplus G}(v) = [w \in C : v \in F(w) \vee v \in G(w)].$$

Each source occurrence is tested once against the disjunction; the two separate filtered lists are not concatenated. This preserves the actual source-list multiplicity even when both families contain v for the same source.

For registered source labels S, code list C and row list E, the source-map condition is

$$\forall s \in S,\ \exists c \in C,\ (s, c) \in E \wedge \forall d,\ (s, d) \in E \Rightarrow d = c.$$

Select that retained c for a source s. Under Correct(C,F), the general source-class induction proves that every v∈F(c) decodes to uniqueSource(c). The row (s,c) is retained alongside this recovery statement. Different source labels are not required by this condition to have different codewords. Under the four successor conditions, apply the same result to F⊕G; this establishes recovery after the action addition without strengthening the source-map domain.

For the concrete alphabet [a,b] and width three, the complete lists are

$$E(aaa) = [aaa, baa, aba, aab], \qquad E(bbb) = [bbb, abb, bab, bba].$$

They are disjoint; every changed word lies outside [aaa,bbb], and baa occurs only in the aaa family. The complete support has 2^3=8 distinct words, while aaa and bbb differ at all three positions. These computations give the retained support, difference, detection, correction and decoding witnesses. For width two,

$$E(aa) = [aa, ba, ab], \qquad E(bb) = [bb, ab, ba],$$

so ab and ba witness failure of correction. At width one, E(a)=[a,b] and E(b)=[b,a] overlap as well. These full finite lists establish the protected and unprotected witness results directly.

> Source, proof and provenance: V3 detectable_code, sft/information_science/coding_theory/law.py:52; V3 correctable_code, sft/information_science/coding_theory/law.py:59. Native results: SFT.InformationScience.CodingActions.singletonOverlap, CodingActions.lean:253; SFT.InformationScience.CodingActions.singletonCorrection, CodingActions.lean:264; SFT.InformationScience.CodingProvenance.encodedImageRecovered, CodingProvenance.lean:16; SFT.InformationScience.CodingProvenance.addedActionsRecoverEncoded, CodingProvenance.lean:24. Timeline: Turn 153, line 1867; Timeline: Turn 153, line 1868. Evidence: Completion TURN_153_CODING_COMPLETION.

108. Exact observation incidence and conditional class restriction

Mutual and conditional information begins with nonempty duplicate-free support and an observation assigning exactly one label to every support form. The native validator retains the original support-first error order, observation-key uniqueness and exact support coverage. Admitted dictionary rows preserve insertion order, while class generation visits support order and retains each observed label on its first occurrence. Empty labels and literal empty-One source names remain in the original raw domain.

Native validation equivalence connects this admission to the existing exact observation partition. Each class contains exactly the support forms mapped to its label; classes are nonempty, cover the support and are disjoint for different labels. The complete marginal-label product retains every pair in left-major/right-minor order. Each joint member belongs precisely to the held support and both corresponding observation classes.

Conditional membership first restricts to a given class, then filters by the target observation. Native coverage and disjointness prove that these target restrictions partition every given class, retaining source order and exact provenance. Independent source-expression comparisons pass 637 validations, 637 class outputs, 5,832 joint-member filters, 5,832 conditional-member filters and 648 complete ordered label products.

Let S be the ordered support and O the observation rows. Validation succeeds exactly when

$$S \neq [] \wedge \mathrm{Distinct}(S) \wedge \mathrm{Distinct}(\mathrm{keys}(O)) \wedge \mathrm{keys}(O) \subseteq S \wedge S \subseteq \mathrm{keys}(O).$$

The support conditions are tested before the map conditions, and success returns O unchanged. These are precisely the exact-partition admission conditions: splitting or recombining the conjunction proves the bridge in both directions. Write f_O(s) for lookup and F_O(a)=[s∈S:f_O(s)=Some(a)]. Observed labels L_O are retained on first occurrence while scanning S. Then a∈L_O iff some s∈S has image a, and the emitted classes are [(a,F_O(a)):a∈L_O]. The witnessing s makes each emitted fibre nonempty; total lookup supplies a fibre for every s; deterministic lookup makes two fibres sharing a member have equal labels.

For observations L,R, the full pair enumeration is concat_{a∈L_L}[(a,b):b∈L_R]. Unpacking map and concatenation membership proves that (a,b) occurs iff both marginal labels occur. Define

$$J_{ab} = [s \in S : f_L(s) = \mathrm{Some}(a) \wedge f_R(s) = \mathrm{Some}(b)].$$

Filter membership proves exactly these three conditions: source membership and the two image equalities. Conditional restriction, with given observation G and target T, is

$$C_{ab} = \mathrm{filter}_{f_T = \mathrm{Some}(b)}\, F_G(a), \qquad s \in C_{ab} \iff s \in S \wedge f_G(s) = \mathrm{Some}(a) \wedge f_T(s) = \mathrm{Some}(b).$$

For any s∈F_G(a), target-map coverage supplies b and places s in C_{ab}; this b occurs in L_T. If s also belongs to C_{ac}, equality of its target lookup forces b=c. Thus the restrictions cover the given fibre and different target labels are disjoint, with all filters preserving source order.

> Source, proof and provenance: V3 observation_classes, sft/information_science/mutual_conditional_information/law.py:25. Native results: SFT.InformationScience.MutualObservation.conditionalCovers, MutualObservation.lean:154; SFT.InformationScience.MutualObservation.conditionalDisjoint, MutualObservation.lean:163. Timeline: Turn 155, line 1882. Evidence: Completion TURN_158_MUTUAL_COMPLETION.

109. Exact joint fractions and marginal-product comparison

The joint ledger visits every pair of observed labels in left-major/right-minor order. Each row retains the exact support-ordered joint member list, its normalized part of the whole support, both normalized marginal parts, their normalized product and the original comparison label. An empty joint cell retains the original empty-One presentation and empty-One-below-product relation. Literal empty-One source labels remain admitted.

Native proofs establish positive marginal counts, exact count-ratio values of every present joint cell and both marginals, the product fraction value, and the four comparison cases through exact integer cross-products. Full label projection proves that every marginal-label pair is retained in source order; admitted ledger evaluation preserves left-then-right validation. Independent comparisons pass 1,360 complete ledgers, 144 guard-order cases and 252 rational comparison cases, bringing the observation and joint-ledger comparison total to 15,342.

Put N=|S|, l_a=|F_L(a)|, r_b=|F_R(b)| and j_{ab}=|J_{ab}|. Validation gives N>0; each emitted marginal label has a witnessing state, giving l_a,r_b>0. Normalize the marginal ratios to positive reduced pairs (u_a,v_a) and (x_b,y_b). Their exact value equations are

$$l_a v_a = u_a N, \qquad r_b y_b = x_b N.$$

The normalized product (p,q) satisfies (u_a x_b)q=p(v_a y_b), with p,q positive. If J_{ab} is empty, its count part is structural empty One and its displayed member list is ["empty-One"]. Otherwise its part (h,k) is the normalized positive count ratio, with

$$j_{ab} k = hN, \qquad h > 0, \quad k > 0, \quad \gcd(h,k) = 1.$$

Nonempty member lists are displayed unchanged, so a literal source name "empty-One" remains present. Comparison is defined by the complete four-way case split

$$\text{relation}(J_{ab}, p/q) = \begin{cases} \text{empty-One-below-product}, & J_{ab} = [], \\ \text{equal}, & J_{ab} \neq [] \wedge hq = pk, \\ \text{joint-below-product}, & J_{ab} \neq [] \wedge hq < pk, \\ \text{joint-above-product}, & J_{ab} \neq [] \wedge pk < hq. \end{cases}$$

The empty constructor is distinct from every positive fraction. For a fraction, equality is tested first, then less-than; when both fail, natural-number order gives the reverse strict inequality. These cases establish the comparison labels without rounding.

Each row is constructed from its two labels, presented members, joint part, both marginal parts, their product and this comparison. Induction over the complete label-pair list proves that projecting the row labels returns the input pair list exactly. Hence no absent joint cell or marginal-label combination is dropped. The ledger validates the left observation first and the right observation second, retaining the first error encountered; successful evaluation returns those complete rows.

> Source, proof and provenance: V3 joint_ledger, sft/information_science/mutual_conditional_information/law.py:33. Native results: SFT.InformationScience.MutualParts.compareBelow, MutualParts.lean:112; SFT.InformationScience.MutualParts.compareAbove, MutualParts.lean:142; SFT.InformationScience.MutualJoint.completePair, MutualJoint.lean:115; SFT.InformationScience.MutualJoint.ledgerAdmitted, MutualJoint.lean:123. Timeline: Turn 156, line 1888. Evidence: Completion TURN_158_MUTUAL_COMPLETION.

110. Conditional fractions, resolution and retained dependence

The conditional ledger enumerates every target label inside every given label in the original given-major order. Each row retains the full intersection and its exact normalized ratio to the given class. Native proofs establish the empty-cell criterion, positive intersection count and fraction value, complete ordered label-pair coverage and admitted evaluation. These rows use the previously established conditional class partition.

The executable resolution rule counts, within each given class, precisely the presented member lists unequal to the source empty-One sentinel and requires one such row together with a nonempty target dictionary. This exact raw comparison is retained even for a source form literally named empty-One. Native equivalences characterize that count rule without adding a domain restriction.

Factorization is exactly the conjunction of joint-cell presence and equality with the marginal product across every ledger row. Dependence retains, in original order, all and only the rows whose comparison is unequal, returning the source sentinel when none remain. Native all-cell and retained-cell equivalences accompany 1,504 independent comparisons for each of the conditional ledger, resolution, factorization and complete dependence ledger. The cumulative observation and ledger comparison total is 21,358.

For given label a and target label b, write g_a=|F_G(a)| and c_{ab}=|C_{ab}|. Every emitted given label has g_a>0. An empty C_{ab} receives structural empty One; a nonempty cell has a normalized positive part (u,v) satisfying

$$c_{ab}v = ug_a, \qquad c_{ab} > 0.$$

Conditional rows enumerate L_G×L_T in given-major order, retaining the presented intersection and this part. Induction on the row constructor proves that the label projection is exactly that full product. Validation nevertheless follows the executable argument order: target first, then given. Both this error order and the different row-enumeration order are preserved.

Let e=["empty-One"] be the source's literal list sentinel, and let R be the presented conditional rows. The exact counted-row predicate and resolution rule are

$$B_a(r) \iff r.\mathrm{givenLabel} = a \wedge r.\mathrm{members} \neq e,$$

$$\mathrm{Resolved} \iff (\forall (a, F_G(a)) \text{ in the given classes}, \ |[r \in R : B_a(r)]| = 1) \wedge \text{target dictionary} \neq [].$$

Expanding the Boolean conjunction, filter and all predicates proves this characterization. It deliberately compares the presented member list: a real singleton source list ["empty-One"] is excluded by the same equality test, even if its exact fraction is positive. No extra source-name guard changes that behavior.

For joint rows Q, the factorization predicate is

$$\mathrm{Factorizes} \iff \forall r \in Q, (\exists p, \ r.\mathrm{jointPart} = \mathrm{part}(p)) \wedge r.\mathrm{relation} = \mathrm{equal}.$$

An empty joint constructor returns false; a positive constructor tests comparison equality. Induction over Q establishes the all-cell equivalence. The responsible list is D=[r∈Q:r.relation≠equal], so filter membership proves that it contains all and only unequal rows in their existing order. Dependence is structural empty One exactly when D=[], and otherwise retains D in full. Errors from joint-ledger construction propagate through both decisions.

> Source, proof and provenance: V3 conditional_ledger, sft/information_science/mutual_conditional_information/law.py:78; V3 conditionally_resolved, sft/information_science/mutual_conditional_information/law.py:103; V3 factorizes, sft/information_science/mutual_conditional_information/law.py:115. Native results: SFT.InformationScience.MutualConditional.completePair, MutualConditional.lean:63; SFT.InformationScience.MutualConditional.ledgerAdmitted, MutualConditional.lean:71; SFT.InformationScience.MutualDecisions.dependenceEmpty, MutualDecisions.lean:112; SFT.InformationScience.MutualDecisions.rawSentinelExcluded, MutualDecisions.lean:118. Timeline: Turn 157, line 1895. Evidence: Completion TURN_158_MUTUAL_COMPLETION.

111. Fresh-source joint and conditional succession

For a fresh source form, each observation extends by its one assigned label. Native admission preservation keeps total observation coverage and duplicate-free support. The joint member list after extension is exactly the old list followed by the fresh source when both assigned labels match the cell, and exactly the old list otherwise. Thus precisely the designated joint cell receives the new source. Conditional intersections satisfy the same ordered successor equation with given and target labels in their original roles.

The exact joint fraction is recomputed against the extended support count. The conditional fraction is recomputed against the extended given-class count, using the retained intersection plus its possible new member. Native ratio equations establish these updates for arbitrary finite support. The singleton base has

one joint member and one conditional member with normalized whole part. Marginal class and label succession use the established observation-extension derivation.

All five original operational witnesses pass native proof: half-parts conditioned on the first coordinate, exact factorization of the complete grid, empty grid dependence, resolution by the copied observation, and nonfactorization with retained dependence cells for the nontrivial copy. Independent fresh-source comparisons pass 1,344 each of conditional ledgers, resolution, factorization and full dependence ledgers, bringing the cumulative source-expression total to 26,734.

Append a fresh x∉S with assigned left and right labels l,r. Freshness preserves support/key distinctness, and appending the same source key to each observation preserves total coverage. Old lookups remain unchanged; lookup of x gives its appended label. Induction through the joint filter therefore gives

$$J'_{ab} = J_{ab} +\!+ \begin{cases} [x], & l = a \wedge r = b, \\ [], & \text{otherwise.} \end{cases}$$

The base case evaluates the two label equalities at x. In the induction step, old lookup equality preserves the head's keep/drop decision and the tail induction supplies the appended term. Thus only the designated cell gains x. Conditional restriction equals the joint filter with given and target in that order; substituting their assigned labels g,t yields

$$C'_{ab} = C_{ab} +\!+ \begin{cases} [x], & g = a \wedge t = b, \\ [], & \text{otherwise.} \end{cases}$$

Write δ_{ab}=1 when both labels match and 0 otherwise. For any positive successor joint part (u',v'), the prior joint-value theorem and the exact list equation give

$$(|J_{ab}| + \delta_{ab})v' = u'(|S| + 1).$$

For a positive successor conditional part (h',k'), given-fibre succession supplies the changed denominator as well:

$$(|C_{ab}| + \mathbf{1}_{g=a\wedge t=b})k' = h'(|F_G(a)| + \mathbf{1}_{g=a}).$$

Empty cells retain the empty constructor until they gain a member. In the singleton base, both joint and conditional lists are [x]; normalization of (1,1) yields numerator and denominator 1. These equations cover the complete finite update, including newly occurring labels.

For the witness support [aa,ab,ba,bb], the first- and second-coordinate marginal classes each have size 2, and every coordinate joint cell has size 1. Thus each joint part is 1/4=(1/2)(1/2), every conditional part is 1/2, factorization holds and the dependence list is empty. Comparing the first coordinate with its copy gives diagonal counts 2 and off-diagonal counts 0. The diagonal parts 1/2 exceed the product 1/4; off-diagonal cells are empty-below-product. Hence factorization fails and all four unequal rows remain. Each given copy-class has exactly one nonsentinel presented target cell, establishing the resolution witness.

> Source, proof and provenance: V3 joint_ledger, sft/information_science/mutual_conditional_information/law.py:33; V3 conditional_ledger, sft/information_science/mutual_conditional_information/law.py:78. Native results: SFT.InformationScience.MutualExtension.jointPartSuccessor, MutualExtension.lean:93; SFT.InformationScience.MutualExtension.conditionalPartSuccessor, MutualExtension.lean:106. Timeline: Turn 158, line 1902. Evidence: Completion TURN_158_MUTUAL_COMPLETION.

112. Complete mutual and conditional program correspondence

Fresh native execution of all 1,024 original mutual and conditional information candidate lookups matches the pinned census, registration, closure, controls and every retained digest, including rejection of an unknown identifier. Together with the exact ledger, decision and arbitrary finite source-successor proofs,

this completes the entry on its original deterministic finite domain. No logarithm, prior, stochastic cause or added scalar measure is introduced.

The candidate grammar is the full ordered product of ten binary axes: support, observations, marginals, joint incidence, conditional restriction, mutual comparison, factorization, deterministic status, generality and added measure. Each axis enumerates its rejected value followed by its admitted value. Consequently

$$|\mathcal{G}| = \prod_{i=1}^{10} 2 = 2^{10} = 1{,}024.$$

For any candidate, its chosen constructor belongs to its corresponding two-element list. Repeated concatenation membership, followed by the final map membership, places that candidate in the generated list. The source identifier decision agrees on every constructor combination with the conjunction of all ten admitted-field tests. Therefore

$$\mathrm{accept}(c) \iff \bigwedge_{i=1}^{10} (c_i = \mathrm{admitted}) \iff c = (\mathrm{admitted}, \ldots, \mathrm{admitted}).$$

Any rejected field contradicts the corresponding projection of that equality and therefore rejects the candidate. The complete grammar classification accompanies the observation, fraction, decision and arbitrary finite successor derivations above; those derivations supply the mathematical content of the retained kernel.

The original program constructs each candidate from the ordered Cartesian product of its declared choice lists, joins its coordinate names to form the identifier, and retains the coordinate/form record. Decision lookup first finds that exact identifier in the complete product, then compares it with the joined all-admitted identifier. A missing identifier therefore fails lookup. The reason scan stops at the first rejected coordinate in declared order. Census hashes retain the complete candidate record; closure hashes retain the closure record and ordered decisions; each control retains its own observed result and receipt hash. Registration uses the same specification and source identity. These are the particular program operations compared with their native records.

> Source, proof and provenance: V3 validate_observation, sft/information_science/mutual_conditional_information/law.py:14. Native results: SFT.InformationScience.MutualEvidence.registrationHasNoFreeParameters, MutualEvidence.lean:83; SFT.InformationScience.MutualEvidence.lookupDependsOnlyOnIdentifier, MutualEvidence.lean:86. Timeline: Turn 158, line 1901. Evidence: Completion TURN_158_MUTUAL_COMPLETION.

Program correspondence sources and proofs: V3 candidate_records, information_science/generated_law.py:L117; V3 registration, information_science/generated_law.py:L230; V3 generate_candidates, information_science/generated_law.py:L245; V3 decide_candidate, information_science/generated_law.py:L264; V3 closure_evidence, information_science/generated_law.py:L285; V3 control_records, information_science/generated_law.py:L187; MutualGrammar.generatedCardinality, L138; MutualGrammar.candidatesComplete, L140; MutualGrammar.acceptanceIffExactCriteria, L198; MutualEvidence.registrationData, L17; MutualEvidence.censusData, L37; MutualEvidence.decisionData, L46; MutualEvidence.closureData, L61.

113. Exact conservation and loss of source distinctions

A transformation assigns exactly one image to each member of complete duplicate-free source support. The native validator preserves the original source-first and map-second guard order and errors. Image names remain unrestricted strings. Complete source pairs visit each source form followed by every later source form in the original order.

The transformation ledger classifies each pair by exact image equality. Different images retain the distinction; equal images close it. Native membership equivalences prove that these two ledgers disjointly exhaust the complete source-pair list. Empty ledgers have the original empty-One presentation. The full record retains source support, first-occurrence image support, both pair ledgers and exact predecessor fibres.

For an admitted transformation, a closed pair lies in a common image fibre. Conversely, every source pair whose two members lie in a common image fibre is closed. These statements retain the exact source provenance of loss. Independent comparisons pass 813 validations, 121 complete ordered pair outputs and 813 full transformation records, including empty-string and literal empty-One labels, reversed mappings and invalid source/map cases.

For source list S and transformation rows T, admission requires S≠[], distinct S, distinct keys(T), and key-set equality with S. Support is checked before the map, and success retains T unchanged. Let f_T(s) denote its admitted image. The complete pair recursion is

$$P([]) = [], \qquad P(x :: R) = [(x, y) : y \in R] +\!+ P(R).$$

The two ordered subledgers are

$$R_T = [(x, y) \in P(S) : f_T(x) \neq f_T(y)], \qquad C_T = [(x, y) \in P(S) : f_T(x) = f_T(y)].$$

Filter membership proves each defining equivalence. Decidable equality places every pair in one of the two lists; simultaneous membership would assert both equality and its negation. Thus these ledgers exhaust the original pairs without overlap, retaining their source traversal order. Empty lists are presented as structural empty One, and nonempty lists are retained intact.

The full record is (S,L_T,present(R_T),present(C_T),[(a,F_T(a)):a∈L_T]), where L_T is the first-occurrence image-label list and F_T(a)=[s∈S:f_T(s)=a]. For a closed pair, total lookup supplies its first endpoint's label a; equality gives that same label for the second endpoint, placing both in F_T(a). Conversely, two endpoints of a source pair in F_T(a) have equal images and the pair is closed. This proves the correspondence between closed distinctions and their exact predecessor fibres.

> Source, proof and provenance: V3 transformation_ledger, sft/information_science/conservation_loss_transformation/law.py:33. Native results: SFT.InformationScience.ConservationLedger.closedFibre, ConservationLedger.lean:119; SFT.InformationScience.ConservationLedger.fibreCloses, ConservationLedger.lean:131. Timeline: Turn 160, line 1916. Evidence: Completion TURN_164_CONSERVATION_COMPLETION.

114. Composition and downstream retention

Transformation composition first validates the source map, derives its middle support in source-order first occurrence, and validates the second map on that complete middle support. Its output retains one row per original source form in the original order. Native proofs establish exact successive lookup and preservation of transformation admission.

When two source forms have equal first images, their composed images are equal. Every closed upstream pair therefore remains closed. Since the retained and closed ledgers are disjoint and exhaustive, each retained composite pair must have been retained upstream. The native downstream-retention theorem proves this subset relation for arbitrary finite admitted compositions, and establishes that the original executable law returns true. No external record or newly created distinction is introduced. Independent comparisons pass 648 complete compositions and 648 downstream results, including the original validation errors and reordered maps; the cumulative comparison total is 3,043.

Composition validates T on S, forms its complete first-occurrence image support L_T, and validates U on L_T. The emitted rows are

$$V = [(s, f_U(f_T(s))) : s \in S].$$

The implementation's lookup default is unreachable on these admitted queries: totality supplies f_T(s), its occurrence puts it in L_T, and U is total on L_T. Induction on the row construction gives keys(V)=S and image_V(s)=Some(f_U(f_T(s))) for s∈S. Therefore V preserves distinct keys, coverage and source order and is an admitted transformation.

If (x,y)∈C_T, then f_T(x)=f_T(y). Applying f_U yields f_U(f_T(x))=f_U(f_T(y)), so

$$C_T \subseteq C_V.$$

Suppose a retained composite pair were not retained upstream. Its source-pair membership and exhaustive equality split would place it in C_T, hence C_V, contradicting disjointness of R_V and C_V. Consequently

$$R_V \subseteq R_T.$$

The executable downstream law removes only the presentation constructor and tests membership in R_T for every pair of R_V. The subset result supplies every test, so the all predicate returns true for admitted compositions. Validation errors retain their original order through the ledger, composition and final-ledger calls.

> Source, proof and provenance: V3 compose, sft/information_science/conservation_loss_transformation/law.py:53; V3 downstream_retention_law, sft/information_science/conservation_loss_transformation/law.py:80. Native results: SFT.InformationScience.ConservationComposition.contentsPresent, ConservationComposition.lean:149; SFT.InformationScience.ConservationComposition.downstreamTrue, ConservationComposition.lean:152. Timeline: Turn 161, line 1922. Evidence: Completion TURN_164_CONSERVATION_COMPLETION.

115. Exact predecessor records and recovery

The reverse record visits the predecessor fibres in their original order. A singleton fibre returns its source label directly; every other fibre returns the original held-predecessor-labels record with its complete ordered member list. Native equivalences establish precisely when each form is produced. Every source with the recorded image equals the unique predecessor in a singleton record. For a held record, membership is exactly source membership together with the recorded image. Two distinct retained predecessors exclude a unique record.

The executable reversibility decision retains V3's comparison of the number of distinct admitted dictionary images with the number of source forms. Independent comparisons pass 813 such decisions and 813 complete reverse records, including original errors and unrestricted string labels. The cumulative transformation comparison total is 4,669. The native count-decision equivalence with singleton fibres is established below.

For a predecessor member list M, the exact record cases are

$$B(M) = \begin{cases} \mathrm{unique}(s), & M = [s], \\ \mathrm{heldLabels}(M), & |M| \neq 1. \end{cases}$$

Splitting M into empty, singleton and longer lists proves B(M)=unique(s) iff M=[s], and B(M)=heldLabels(H) iff H=M and |M|≠1. Reverse rows map (a,F_T(a)) to (a,B(F_T(a))) without reordering fibres. Thus a unique record for a supplies F_T(a)=[s]. Any source t with image a belongs to that fibre and hence equals s. A held record satisfies

$$t \in H \iff t \in S \wedge f_T(t) = a.$$

If two different predecessors lie in M, uniqueness would force both to equal its alleged singleton member and contradict their difference. No member is selected from a merged fibre.

Let D=map(second,T) be the dictionary-value list, and let L(D) retain its first occurrences. After validation, the executable reversibility result is exactly

$$\mathrm{isReversible}(S,T) = \mathrm{ok}(|L(D)| = |S|).$$

This is the original count expression; the next derivation relates it to the fibre-based recovery records.

> Source, proof and provenance: V3 reverse_record, sft/information_science/conservation_loss_transformation/law.py:69. Native results: SFT.InformationScience.ConservationReverse.distinctPredecessorsNotUnique, ConservationReverse.lean:116; SFT.InformationScience.ConservationReverse.reversibleAdmitted, ConservationReverse.lean:126. Timeline: Turn 162, line 1929. Evidence: Completion TURN_164_CONSERVATION_COMPLETION.

116. Reversibility is equivalent to singleton fibres

For every transformation admitted by the original validator, the executable distinct-image count test returns true exactly when every observed predecessor fibre contains one source form. The native proof retains the original dictionary-value count. First-occurrence deduplication preserves list length exactly when the value list has no repetitions. Unique source keys establish exact correspondence between dictionary values and the images obtained by traversing the admitted source support. Their distinct-image counts are equal, regardless of dictionary row order.

Distinct mapped source values are equivalent to injectivity on the held source support. Injectivity makes each nonempty image fibre a singleton; conversely, singleton fibres identify any two sources with the same image. These implications prove both directions of V3's reversibility statement on arbitrary finite admitted supports, without a new guard or an assumed inverse. The established reverse-record recovery theorems then give direct recovery for singleton fibres and complete retained predecessor alternatives for merged fibres.

A first-occurrence scan can never lengthen a list. If it skips a head already seen, its tail output has length at most the tail length and therefore strictly less than the original list length. Hence equality of input and output lengths forces every head to be retained and, recursively, absent from the later list. Conversely, a distinct list with no initially seen member is retained unchanged. Starting with an empty seen list gives

$$|L(X)| = |X| \iff \mathrm{Distinct}(X).$$

Let D=map(second,T) and V=[f_T(s):s∈S]. A value in D has a dictionary row; unique keys make lookup return that row's value, and coverage puts its key in S, so the value occurs in V. Conversely, lookup of any admitted s yields a retained dictionary row and thus a value of D. Therefore D and V have identical value membership. Their first-occurrence lists are distinct and mutually contained; finite distinct-list length bounds in both directions give |L(D)|=|L(V)|. Since |V|=|S|,

$$|L(D)| = |S| \iff |L(V)| = |V| \iff \mathrm{Distinct}(V).$$

On distinct S, injectivity of f_T makes V distinct: a repeated head value would identify its source with a later source, contradicting distinctness, and the tail follows by induction. Conversely, if V is distinct, equal mapped values cannot occur at two different source positions; splitting source membership into head and tail proves injectivity. Thus the original count test is equivalent to injectivity on S.

Each observed fibre is nonempty and distinct. Under injectivity, choose one of its members s; every other member has the same image and must equal s, so the fibre is [s]. Conversely, two sources with equal images lie in the same observed fibre, and singleton fibres make those sources equal. Combining the equivalences proves

$$\mathrm{isReversible}(S,T) = \mathrm{ok}(\mathrm{true}) \iff f_T \text{ is injective on } S \iff \forall a \in L_T,\ \exists s,\ F_T(a) = [s].$$

The argument uses membership and distinct counts, so arbitrary admitted dictionary-row order is preserved. Recovery follows from the existing reverse records, without postulating an inverse function.

> Source, proof and provenance: V3 is_reversible, sft/information_science/conservation_loss_transformation/law.py:64. Native results: SFT.InformationScience.ConservationCardinality.reversibleIffSingletonFibres, ConservationCardinality.lean:243. Timeline: Turn 163, line 1936. Evidence: Completion TURN_164_CONSERVATION_COMPLETION.

117. Source succession and conservation of pair accounting

The singleton transformation has one source, one held image, one singleton predecessor fibre and empty retained and closed pair ledgers. Native evaluation proves the complete original record. Adding a fresh source and one image row preserves the admitted transformation conditions. Every image fibre retains its old members and appends the new source exactly when its label matches the added image.

The complete pair list after extension retains the original source traversal order, adding one pair between each prior source and the fresh source. A retained successor pair is exactly a previously retained pair or a new old/fresh pair with unequal images. A closed successor pair is exactly a previously closed pair or a new old/fresh pair with equal images. These native equivalences establish exhaustive successor accounting for arbitrary finite support.

All five original operational witnesses pass native proof: identity conservation, exact prefix loss, the identity/prefix reversibility boundary, complete held predecessor records and downstream erasure. Fresh-source comparisons pass 1,395 validations, 15 complete ordered pair lists and 1,395 full ledgers. The cumulative source-expression comparison total is 7,474.

For fresh x∉S and appended row (x,v), put S'=S++[x] and T'=T++[(x,v)]. The established observation successor preserves admission, retains all old images, and gives image_{T'}(x)=Some(v). Its exact fibre update is

$$F_{T'}(a) = F_T(a) \mathbin{++} \begin{cases} [x], & v = a, \\ [], & v \neq a. \end{cases}$$

The pair list retains its nested source order. Explicitly, E_x([])=[] and E_x(y::R)=([(y,z):z∈R]++[(y,x)])++E_x(R), and P(S++[x])=E_x(S). Membership is exactly old pair membership or a new pair (s,x) with s∈S. Substituting the old and fresh image equations into the two filters yields

$$p \in R_{T'} \iff p \in R_T \vee \exists s \in S,\ p = (s, x) \wedge f_T(s) \neq v,$$

$$p \in C_{T'} \iff p \in C_T \vee \exists s \in S,\ p = (s, x) \wedge f_T(s) = v.$$

Both directions follow from the pair split: old endpoints keep their images; a new pair compares exactly its old image with v. The formulas classify the complete ordered successor list, without replacing it by a differently ordered concatenation of pair classes.

At the singleton base, S=[s], T=[(s,a)], the ledger is ([s],[a],emptyOne,emptyOne,[(a,[s])]). For the witness S=[aa,ab,ba,bb], P(S) has six pairs. Identity retains all six and closes none. Prefix projection closes exactly (aa,ab) and (ba,bb), retaining the other four; its fibres are [aa,ab] and [ba,bb]. Identity has four distinct images and is reversible, whereas prefix projection has two and is not. Both prefix reverse rows retain their two predecessor labels. Composing prefix projection with a↦one,b↦one closes all six pairs and leaves no retained pair, establishing the downstream-erasure witness.

> Source, proof and provenance: V3 transformation_ledger, sft/information_science/conservation_loss_transformation/law.py:33. Native results: SFT.InformationScience.ConservationExtension.singletonValid, ConservationExtension.lean:110;

> SFT.InformationScience.ConservationExtension.singletonBase, ConservationExtension.lean:114. Timeline: Turn 164, line 1943. Evidence: Completion TURN_164_CONSERVATION_COMPLETION.

118. Complete conservation and transformation program correspondence

Fresh native execution matches all 1,024 original candidate lookups, registration, census, closure, controls, hashes and unknown-identifier rejection. Together with exhaustive pair accounting, exact composition, downstream retention, reversibility equivalence, predecessor recovery and arbitrary finite source succession, this completes the entry on its original deterministic finite domain. No scalar conservation measure, assumed inverse or unrecorded source distinction is introduced.

The complete registered grammar has ten binary axes: support, map, provenance, pair accounting, loss, reversal, composition, conservation, generality and addition. Each axis generates its rejected constructor followed by its admitted constructor, giving

$$|\mathcal{G}| = 2^{10} = 1{,}024.$$

Choosing the appropriate member of each axis list and applying nested product membership proves that every typed candidate is enumerated. Exhausting the ten constructor choices establishes equality between the source identifier decision and the conjunction of the ten admitted-field tests. Hence

$$\mathrm{accept}(c) \iff \bigwedge_{i=1}^{10} c_i = \mathrm{admitted} \iff c = (\mathrm{admitted}, \ldots, \mathrm{admitted}).$$

Projecting this equality onto any rejected field gives a contradiction, proving rejection for each failed criterion. This complete finite classification accompanies the source-pair, composition, fibre, recovery and successor derivations above.

The original program constructs each candidate from the ordered Cartesian product of its declared choice lists, joins its coordinate names to form the identifier, and retains the coordinate/form record. Decision lookup first finds that exact identifier in the complete product, then compares it with the joined all-admitted identifier. A missing identifier therefore fails lookup. The reason scan stops at the first rejected coordinate in declared order. Census hashes retain the complete candidate record; closure hashes retain the closure record and ordered decisions; each control retains its own observed result and receipt hash. Registration uses the same specification and source identity. These are the particular program operations compared with their native records.

> Source, proof and provenance: V3 validate_transformation, sft/information_science/conservation_loss_transformation/law.py:12. Native results: SFT.InformationScience.ConservationEvidence.registrationHasNoFreeParameters, ConservationEvidence.lean:83; SFT.InformationScience.ConservationEvidence.lookupDependsOnlyOnIdentifier, ConservationEvidence.lean:86. Timeline: Turn 164, line 1942. Evidence: Completion TURN_164_CONSERVATION_COMPLETION.

Program correspondence sources and proofs: V3 candidate_records, information_science/generated_law.py:L117; V3 registration, information_science/generated_law.py:L230; V3 generate_candidates, information_science/generated_law.py:L245; V3 decide_candidate, information_science/generated_law.py:L264; V3 closure_evidence, information_science/generated_law.py:L285; V3 control_records, information_science/generated_law.py:L187; ConservationGrammar.generatedCardinality, L138; ConservationGrammar.candidatesComplete, L140; ConservationGrammar.acceptanceIffExactCriteria, L198; ConservationEvidence.registrationData, L17; ConservationEvidence.censusData, L37; ConservationEvidence.decisionData, L46; ConservationEvidence.closureData, L61.

119. Classical state, observation classes and deterministic realization

A classical information state retains one member of the complete nonempty duplicate-free support together with that support as provenance. A total observation partitions the same support by its labels in first-occurrence order. Each probability row retains its entire ordered microstate class and the reduced positive fraction of complete support occupied by that class. Native proofs identify the exact class membership, retained order and fraction count equation. Deterministic pushforward constructs corresponding image rows with complete predecessor classes and exact parts.

Observation first requires exactly one row for its observed label. A singleton member class returns its held classical state; every other retained class returns all its members. The native singleton equivalence and unresolved-member identity preserve this boundary. Deterministic realization visits the probability rows in order, pairing every original microstate with the deterministic-member tag while retaining the class label and its exact support part. Source-expression comparisons cover 6,296 support, ledger, resolution and realization outputs and error cases, including empty labels and original validation order. Exact-whole closure and the source microstate-successor construction are established below.

Canonical support admission is S≠[] and Distinct(S). A classical-state request succeeds exactly when this holds and h∈S, returning (heldState=h,supportProvenance=S). For observation O, probability validation checks the support first, then distinct keys and complete key coverage. Pushforward validation has the same successful domain but checks duplicate map keys before support validation, retaining the original short-circuit errors.

For each first-occurrence label a, let F_a=[s∈S:f_O(s)=a], m_a=|F_a| and N=|S|. The probability row is (a,F_a,(u_a,v_a)), with the positive reduced part satisfying

$$m_a v_a = u_a N, \qquad u_a > 0, \quad v_a > 0, \quad \gcd(u_a, v_a) = 1.$$

The row maps from the entropy ledger preserve its label, members and part exactly. Induction on those maps proves that projecting label/member pairs returns the original ordered partition. Pushforward uses the same record with image/predecessor field names, so its source provenance and fraction equation are identical.

Resolution first forms H=[r in rows:r.label=a]. It returns the outside-observation error unless H=[r] is exactly a singleton. For that sole row,

$$\mathrm{resolveMembers}(M) = \mathrm{classicalState}(s) \iff M = [s].$$

The empty and longer-list cases return unresolvedClass(M) with M unchanged. Deterministic realization maps each row to (a,[(s,"deterministic-member"):s∈F_a],(u_a,v_a)). Projecting first coordinates recovers F_a by list induction, while the label and fraction are unchanged by construction. The realization therefore exposes every original member in order.

> Source, proof and provenance: V3 classical_state, sft/information_science/classical_probabilistic_information/law.py:20; V3 deterministic_realization, sft/information_science/classical_probabilistic_information/law.py:82. Native results: SFT.InformationScience.ClassicalSupport.probabilityPartition, ClassicalSupport.lean:88; SFT.InformationScience.ClassicalSupport.pushforwardProbability, ClassicalSupport.lean:96; SFT.InformationScience.ClassicalResolution.realizationOutput, ClassicalResolution.lean:74; SFT.InformationScience.ClassicalResolution.realizationProvenance, ClassicalResolution.lean:80. Timeline: Turn 166, line 1958; Timeline: Turn 167, line 1962. Evidence: Completion TURN_170_CLASSICAL_COMPLETION.

120. Complete observation-class cardinality

For every total observation of a finite support, the sum of the full fibre cardinalities over the first-occurrence observed labels equals the support cardinality. The proof counts each source occurrence against distinct observed labels: exactly its own image contributes once. Induction over the support then

reconstructs the complete count. No equal-class-size assumption is used.

The raw exact-whole helper retains its Fraction input domain, including signed and zero fractions. Integer cross-multiplication accumulates the exact numerator and denominator; the nonempty ledger is whole precisely when those values are equal. Generated probability and pushforward rows supply their original positive reduced fractions to this calculation. 3,614 independent source-expression comparisons pass, including 2,801 raw fraction lists and 813 generated-ledger or original-error cases. The universal reduced-fraction bridge and exact-whole result are established below.

Let L be the distinct list of observed labels, and define

$$H_L(a) = \sum_{b \in L} \mathbf{1}_{b=a}, \qquad K(S, O, L) = \sum_{a \in L} |F_O(a)|.$$

If a∉L, every summand in H_L(a) is zero. If a∈L and L is distinct, its unique matching position contributes one and every other position contributes zero, so H_L(a)=1. For f_O(s)=a, expanding each fibre filter at the head gives

$$K(s :: S, O, L) = H_L(a) + K(S, O, L).$$

Total observation coverage supplies such an a∈L for each source. Induction over S, starting from K([],O,L)=0, therefore yields K(S,O,L)=|S|. This counts every source occurrence exactly once without assuming equal fibre sizes.

The raw exact-sum helper begins with (n,d)=(0,1) and, for each rational p/q, updates

$$(n, d) \longmapsto (nq + pd, dq).$$

Here n,p are integers and d,q are natural denominators; zero and negative input fractions remain admitted. Cross-multiplication gives (nq+pd)/(dq)=n/d+p/q, so induction represents the exact rational sum. The executable whole test is false on an empty input list; on a nonempty list it tests n=d after all updates. Generated positive reduced parts enter this same helper through their exact numerator and denominator, without changing its broader raw domain.

> Source, proof and provenance: V3 probability_ledger, sft/information_science/classical_probabilistic_information/law.py:27; V3 pushforward_ledger, sft/information_science/classical_probabilistic_information/law.py:58. Native results: SFT.InformationScience.ClassicalLedger.probabilityProjection, ClassicalLedger.lean:77; SFT.InformationScience.ClassicalLedger.pushforwardProjection, ClassicalLedger.lean:84. Timeline: Turn 166, line 1958. Evidence: Completion TURN_170_CLASSICAL_COMPLETION.

121. Exact-whole closure and microstate succession

Every generated probability ledger and deterministic pushforward ledger reconstructs the exact whole. The proof tracks the numerator and denominator of the original exact sum through each retained class fraction. Its invariant relates the accumulated numerator to the count of represented microstates over the fixed support count. Reduced class fractions satisfy this relation by their native count equations. Complete class coverage gives the full support count, and cancellation of that positive count establishes equality of the accumulated numerator and denominator. Thus the executable exact-whole test returns true for every admitted ledger.

The singleton base retains its sole classical state with complete provenance, one observation class and one exact whole part. Extending admitted support by one fresh microstate preserves admission. Its assigned class gains exactly that state in source order, every other class retains its members, and each reduced fraction satisfies the updated membership-to-support ratio. The successor ledger again

reconstructs the exact whole. These statements apply at arbitrary finite depth and preserve deterministic membership throughout. Full candidate enumeration and complete program correspondence are verified below.

For generated rows, let N>0 be the support size, m_i each row's microstate count and a_i/b_i its reduced part. The native row equation is a_iN=m_i b_i. At an intermediate sum with numerator n, denominator d and represented count u, use the invariant nN=ud. Adding one row gives

$$(nb_i + a_i d)N = (nN)b_i + (a_i N)d = udb_i + m_i b_i d = (u + m_i)(db_i).$$

Thus the same invariant holds after each exact update. Starting from (0,1,0), induction over all rows gives n_final N=(Σ_i m_i)d_final. The class-count theorem supplies Σ_i m_i=N; cancellation of positive N yields n_final=d_final. The rows cannot be empty because their member count is N>0. Hence the executable exact-whole predicate returns true.

The natural-number accumulator for these positive rows agrees with the signed raw helper after embedding its numerator into the integers: their recurrences have identical multiplication and addition updates. Probability and pushforward row maps preserve the part list exactly. Consequently the preceding argument proves exact-whole closure for both generated ledger types, not only for unreduced count fractions.

For fresh x∉S with assigned label v, validation remains admitted after S'=S++[x], O'=O++[(x,v)]. Old lookup values persist, and the assigned fibre gains x:

$$F'_a = F_a +\!\!+ \begin{cases} [x], & a = v, \\ [], & a \neq v. \end{cases}$$

Every successor row with part a'_i/b'_i therefore satisfies

$$(|F_i| + \mathbf{1}_{i=v})b'_i = a'_i(N+1).$$

Applying the universal closure result to the newly admitted ledger proves that it again reconstructs the whole. At the singleton base S=[s], O=[(s,a)], the state is (s,[s]), the projected ledger is [(a,[s])], and its part is 1/1. The same base and successor construction preserves deterministic-member provenance at every finite stage.

> Source, proof and provenance: V3 ledger_is_exact_whole, sft/information_science/classical_probabilistic_information/law.py:77. Native results: SFT.InformationScience.ClassicalWhole.completeClassCount, ClassicalWhole.lean:112; SFT.InformationScience.ClassicalWhole.originalClassCount, ClassicalWhole.lean:124; SFT.InformationScience.ClassicalClosure.probabilityExactWhole, ClassicalClosure.lean:116; SFT.InformationScience.ClassicalClosure.pushforwardExactWhole, ClassicalClosure.lean:126; SFT.InformationScience.ClassicalExtension.successorLedgerWhole, ClassicalExtension.lean:41; SFT.InformationScience.ClassicalExtension.successorPartValue, ClassicalExtension.lean:49. Timeline: Turn 168, line 1968; Timeline: Turn 169, line 1975; Timeline: Turn 169, line 1976. Evidence: Completion TURN_170_CLASSICAL_COMPLETION.

122. Classical and probabilistic operational witnesses

All six original operational witnesses pass native proof: held-state provenance, two equal half classes, unequal three-quarter and one-quarter parts, singleton versus unresolved-class resolution, deterministic prefix pushforward reconstructing the whole, and exposed deterministic-member traces. Complete 1,024-candidate program correspondence is verified below.

For the original witness support S=[aa,ab,ba,bb], requesting held state aa returns (aa,S). Prefix observation gives the ordered classes F_a=[aa,ab] and F_b=[ba,bb], so both reduced fractions are 2/4=1/2. The unequal observation instead gives [aa,ab,ba] and [bb], with parts 3/4 and 1/4. These

compute the held-state, half-class and unequal-class witnesses directly.

Fine observation labels each microstate by itself, so the aa row has members [aa] and resolves to classicalState(aa). Under prefix observation, label a has members [aa,ab] and returns that exact unresolved class. Prefix pushforward retains the same two predecessor lists and its exact sum is

$$(0,1) \xrightarrow{1/2} (1,2) \xrightarrow{1/2} (4,4),$$

so the whole test succeeds. The unequal example similarly gives (0,1)→(3,4)→(16,16). Finally, realization returns [(aa,deterministic-member),(ab,deterministic-member)] in the first prefix row and the corresponding ba,bb pairs in the second. Every emitted member has the required tag while its source label, row label and fraction are preserved. These are the six complete operational witness evaluations.

> Source, proof and provenance: V3 resolve_observation, sft/information_science/classical_probabilistic_information/law.py:47. Native results: SFT.InformationScience.ClassicalWitnesses.allWitnessesPass, ClassicalWitnesses.lean:35. Timeline: Turn 170, line 1982. Evidence: Completion TURN_170_CLASSICAL_COMPLETION.

123. Complete classical and probabilistic correspondence

All 1,024 original candidates and six operational witnesses are retained. Fresh native execution matches original registration, complete census, actual identifier-based decisions, closure, controls, hashes and unknown-identifier rejection. Together with the exact class ledger, singleton-only resolution, deterministic realization, universal whole closure and arbitrary microstate succession, this completes the registered finite correspondence. Empirical frequencies retain their separately registered blind-data boundary.

The grammar enumerates ten binary choices: support, classical state, observation, parts, meaning, resolution, transformation, normalization, generality and addition. Each axis retains its rejected and admitted alternatives in source order, so

$$|\mathcal{G}| = 2^{10} = 1{,}024.$$

Membership in each two-element axis list constructs membership in the complete nested product. Constructor case analysis proves that the source identifier decision equals the conjunction of all ten admitted-field decisions; their conjunction identifies exactly the all-admitted candidate:

$$\mathrm{accept}(c) \iff \bigwedge_{i=1}^{10} c_i = \mathrm{admitted} \iff c = (\mathrm{admitted}, \ldots, \mathrm{admitted}).$$

Each rejected field contradicts the corresponding projection of this equality. The full candidate classification accompanies the ledger, resolution, realization, exact-whole and arbitrary finite successor derivations in this block.

The original program constructs each candidate from the ordered Cartesian product of its declared choice lists, joins its coordinate names to form the identifier, and retains the coordinate/form record. Decision lookup first finds that exact identifier in the complete product, then compares it with the joined all-admitted identifier. A missing identifier therefore fails lookup. The reason scan stops at the first rejected coordinate in declared order. Census hashes retain the complete candidate record; closure hashes retain the closure record and ordered decisions; each control retains its own observed result and receipt hash. Registration uses the same specification and source identity. These are the particular program operations compared with their native records.

> Source, proof and provenance: V3 canonical_support, sft/information_science/classical_probabilistic_information/law.py:14. Native results: SFT.InformationScience.ClassicalEvidence.registrationHasNoFreeParameters, ClassicalEvidence.lean:83; SFT.InformationScience.ClassicalEvidence.lookupDependsOnlyOnIdentifier, ClassicalEvidence.lean:86. Timeline: Turn 170, line 1981. Evidence: Completion TURN_170_CLASSICAL_COMPLETION.

Program correspondence sources and proofs: V3 candidate_records, information_science/generated_law.py:L117; V3 registration, information_science/generated_law.py:L230; V3 generate_candidates, information_science/generated_law.py:L245; V3 decide_candidate, information_science/generated_law.py:L264; V3 closure_evidence, information_science/generated_law.py:L285; V3 control_records, information_science/generated_law.py:L187; ClassicalGrammar.generatedCardinality, L138; ClassicalGrammar.candidatesComplete, L140; ClassicalGrammar.acceptanceIffExactCriteria, L198; ClassicalEvidence.registrationData, L17; ClassicalEvidence.censusData, L37; ClassicalEvidence.decisionData, L46; ClassicalEvidence.closureData, L61.

124. Quantum-information branch support and provenance

The support correspondence generates every held-label word across a nonempty canonical position trace. Its original validator requires at least two distinct labels and distinct positions. Native correspondence with the information-quantity construction preserves the full Cartesian order and proves exhaustive word membership and duplicate freedom. Empty string labels remain allowed.

The superposition-equivalent record retains the complete branch family and pairs each word with its full position-label trace. Projecting these records recovers the original branch family in order; projecting each trace recovers both its original word and every canonical position. The amplitude field retains the original not-admitted-at-support-boundary marker. 3,200 independent comparisons cover full support/provenance outputs and original errors. Information-unit traces, basis records, joint support and observation/reconstruction are established below; the canonical basis-name correspondence is established below.

Write L for the ordered label list and P for the ordered position list, with n=|P|. Admission is precisely |L|≥2, Distinct(L), n>0 and Distinct(P). The recursive complete support is

$$W_0(L) = [[]], \qquad W_{n+1}(L) = \mathrm{concat}_{\ell \in L}[\ell :: w : w \in W_n(L)].$$

Induction on n proves w∈W_n(L) if and only if |w|=n and every label of w belongs to L: the successor case removes the first label and applies the induction hypothesis to the tail. Distinct labels and distinct tails make each block duplicate-free, and different first labels separate different blocks. This establishes duplicate freedom of the complete support.

For each admitted w, define T_P(w)=zip(P,w). The returned branch record is (W_n,[(w,T_P(w)):w∈W_n],not-admitted-at-support-boundary). Equal lengths permit simultaneous induction on P and w, giving

$$\pi_1(T_P(w)) = P, \qquad \pi_2(T_P(w)) = w.$$

Projecting the word field of the outer record likewise returns W_n in its original order. Thus the complete alternative family and each branch's full origin are retained by the construction itself.

> Source, proof and provenance: V3 complete_fold_support, sft/information_science/quantum_information_correspondence/law.py:15; V3 superposition_equivalent_support, sft/information_science/quantum_information_correspondence/law.py:41. Native results: SFT.InformationScience.QuantumSupport.zipRetainsPositions, QuantumSupport.lean:79; SFT.InformationScience.QuantumSupport.admittedProvenance, QuantumSupport.lean:89. Timeline: Turn 172, line 1995. Evidence: Completion TURN_179_QUANTUM_COMPLETION.

125. Basis records and joint support

The basis correspondence retains each original word together with its source colon-delimited basis name. Projecting the retained words reconstructs complete input support exactly, and distinct words yield distinct word/name records. The name field alone is not injective over the original unrestricted string-label domain: labels a and a:a at two canonical positions generate the distinct words (a,a:a) and (a:a,a), both

named basis:a:a:a. The accepted support, distinct words and identical names are established by native proofs. Consequently, retained-record correspondence and name-only uniqueness are separate propositions; the latter does not hold over the complete executable domain.

Joint support requires two nonempty duplicate-free word carriers and enumerates every left/right pair in source order, retaining both coordinates and their concatenation. Native membership equivalence establishes complete pair coverage; coordinate recovery preserves both source words even when concatenated values coincide. 993 independent output comparisons cover exact basis and joint records, errors, empty words and delimiter-containing labels.

For a word w=[w_1,...,w_n], the executable name is B(w)="basis:"++intercalate(":",w), and the basis record list is [(w,B(w)):w∈S]. Projection onto its first component gives S. Equality of two records immediately gives equality of their words, establishing injectivity of the retained-record map without any condition on delimiters inside labels. The raw example satisfies

$$B([\mathrm{a},\mathrm{a{:}a}]) = \mathrm{basis{:}a{:}a{:}a} = B([\mathrm{a{:}a},\mathrm{a}]),$$

while the two first coordinates differ. The fixed inherited Fold labels used in the registered derivation have the separate name-injectivity proof below.

For admitted left and right carriers A and C, the joint construction is

$$J(A,C) = \mathrm{concat}_{a\in A}[(a, c, a \mathbin{+\!+} c) : c \in C].$$

Expanding the nested list maps proves (a,c,z)∈J(A,C) exactly when a∈A, c∈C and z=a++c. Equality of two emitted triples forces equality of both first and second coordinates by projection. Hence the joint correspondence retains the original pair even if concatenation alone would erase its split.

> Source, proof and provenance: V3 basis_correspondence, sft/information_science/quantum_information_correspondence/law.py:35; V3 joint_support, sft/information_science/quantum_information_correspondence/law.py:50. Native results: SFT.InformationScience.QuantumBasis.delimiterWordsDifferent, QuantumBasis.lean:57; SFT.InformationScience.QuantumBasis.delimiterBasisAccepted, QuantumBasis.lean:59; SFT.InformationScience.QuantumJoint.cellCoordinates, QuantumJoint.lean:43; SFT.InformationScience.QuantumJoint.jointLeftSuccessor, QuantumJoint.lean:46. Timeline: Turn 173, line 2001; Timeline: Turn 173, line 2002. Evidence: Completion TURN_179_QUANTUM_COMPLETION.

126. Independently realized information-unit traces

Each canonical position retains a four-field record: its position name, the one-Fold-information-distinction tag, the all-first-label baseline word and the word differing only at that position by the second held label. Native proofs place both words in the full generated support, identify the baseline with repeated first labels and establish distinctness of the alternatives for distinct labels. The admitted trace has exactly one row per position and preserves the entire position order at arbitrary finite depth. 1,600 independent complete-record and original-error comparisons pass, including empty strings and delimiter-containing labels.

Let a and b be the first two distinct admitted labels. For zero-based position i<n, the baseline and its independently changed alternative are

$$u_i = a^i \mathbin{+\!+} [a] \mathbin{+\!+} a^{n-i-1} = a^n, \qquad v_i = a^i \mathbin{+\!+} [b] \mathbin{+\!+} a^{n-i-1}.$$

Here a^k means the list of k copies of a. Both words have length n and every component belongs to L, so complete-support membership puts both in W_n(L). If u_i=v_i, cancelling the common prefix of i copies gives equal heads a=b, contradicting distinctness. The four-field row is (P_i,one-Fold-information-distinction,u_i,v_i).

The recursive implementation emits the head-position row and increments i before processing the remaining positions. Its invariant is i+|remaining positions|=n. Induction preserves this invariant, proves membership of both words in every row, and gives exactly n rows whose position projection is P. Thus the construction establishes an independent distinction at every position for arbitrary finite depth.

> Source, proof and provenance: V3 information_unit_trace, sft/information_science/quantum_information_correspondence/law.py:23. Native results: SFT.InformationScience.QuantumUnits.traceLength, QuantumUnits.lean:101; SFT.InformationScience.QuantumUnits.tracePositions, QuantumUnits.lean:107. Timeline: Turn 174, line 2007. Evidence: Completion TURN_179_QUANTUM_COMPLETION.

127. Exact support observation and reconstruction

Observation retains every branch with the selected image in source order, together with its complete complement and a record of every original branch and image. An empty complement retains the original empty-One marker. Native membership proofs establish disjoint and exhaustive accounting of retained and closed alternatives. Projecting the complete reconstruction record recovers the original support in order. On canonical nonempty support, reconstruction accepts this record and returns exactly the pre-observation support.

The raw observation helper preserves its original domain, including repeated support words when the observation keys cover that support set exactly. Reconstruction separately rejects repeated branch keys, as in the source. 4,660 independent comparisons cover full observation records, reconstruction outputs and original errors, including empty words, empty labels, repeated support, reversed mapping order and literal empty-One branch labels.

Let O have distinct word keys whose set equals the support set, and write f_O(w) for lookup of the image of w. For selected image y the emitted lists are

$$R_y = [w \in S : f_O(w) = y], \qquad C_y = [w \in S : w \notin R_y], \qquad E = [(w, f_O(w)) : w \in S].$$

Observation rejects an empty R_y. Otherwise it returns y, R_y, the presented C_y (using the empty-One marker when empty), and E. Filter membership gives w∈R_y iff w∈S and f_O(w)=y; the complementary filter gives w∈C_y iff w∈S and w∉R_y. These equivalences prove disjointness and exhaustive accounting of S while preserving each filtered order.

List induction gives $\pi_1(E)=S$. Reconstruction first requires E nonempty and Distinct($\pi_1(E)$), then returns $\pi_1(E)$. Consequently canonical nonempty duplicate-free S is recovered exactly. The raw observation guard checks distinct observation keys and coverage in both directions; it does not require distinct S. Repeated support occurrences therefore remain present in E, and the separate reconstruction guard rejects them exactly as specified by the source.

> Source, proof and provenance: V3 observe_support, sft/information_science/quantum_information_correspondence/law.py:56; V3 reconstruct_support, sft/information_science/quantum_information_correspondence/law.py:73. Native results: SFT.InformationScience.QuantumObservation.reconstruction, QuantumObservation.lean:122; SFT.InformationScience.QuantumObservation.observedReconstruction, QuantumObservation.lean:134. Timeline: Turn 175, line 2013. Evidence: Completion TURN_179_QUANTUM_COMPLETION.

128. Canonical position succession

Adding one fresh canonical position preserves support admission and extends every prior branch by every held label in the original Cartesian order. The extended support remains duplicate-free. Each branch provenance is exactly its previous position-label trace followed by the fresh position and chosen label; a complete information-unit trace exists for the extended support. The first-position construction retains all

held labels, with the original two-label instance producing its two branches. All six original operational witnesses pass native proof on their fixed source inputs. Name-only bijection holds throughout the registered fixed-label Fold support, as established below.

For fresh p∉P, admission persists because P++[p] is nonempty and duplicate-free. Define the ordered extension E_L(S)=concat_{w∈S}[w++[ℓ]:ℓ∈L]. Distribution over concatenated blocks, followed by induction on depth, proves the ordered identity

$$W_{n+1}(L) = E_L(W_n(L)).$$

The base is W_1(L)=[[ℓ]:ℓ∈L]. In the successor proof, each existing leading-label block is extended internally, preserving both its leading label and the source's order. For |w|=|P|, simultaneous list induction also gives

$$\mathrm{zip}(P \mathbin{+\!\!+} [p], w \mathbin{+\!\!+} [\ell]) = \mathrm{zip}(P, w) \mathbin{+\!\!+} [(p, \ell)].$$

The support-admission theorem then supplies duplicate freedom of the extended support, and the information-unit construction supplies its full position trace. At the original two-label base, P=[p1] produces [[held-a],[held-b]]. These base and successor results establish the construction at each finite canonical depth.

> Source, proof and provenance: V3 complete_fold_support, sft/information_science/quantum_information_correspondence/law.py:15. Native results: SFT.InformationScience.QuantumExtension.firstPositionSupport, QuantumExtension.lean:49; SFT.InformationScience.QuantumExtension.twoLabelBase, QuantumExtension.lean:53. Timeline: Turn 176, line 2019. Evidence: Completion TURN_179_QUANTUM_COMPLETION.

129. Quantum support program correspondence

Fresh native execution reproduces all 1,024 original candidate lookups and all registration, census, closure, control, hash and unknown-identifier outputs. All six operational witnesses hold for the original fixed held-a/held-b support. These program results retain the source declarations verbatim; they do not establish name-only injectivity for arbitrary accepted string labels. The delimiter collision refutes name-only injectivity over arbitrary accepted string labels. The registered Fold Assembly generator fixes its labels to held-a and held-b, and the quantum grammar refers to the two held Fold labels. Native proofs establish name-only injectivity and the unique inverse on this generated domain at every finite depth; the arbitrary-label collision is outside that registered grammar and remains faithfully represented in the raw helper behavior.

The complete grammar has ten binary axes: unit, support, basis, provenance, superposition, composition, observation, record, generality and addition. Each is enumerated in rejected/admitted order. The nested product therefore contains 2^{10}=1,024 candidates, and membership of a constructor in each binary axis proves membership of every candidate in this product. Exhaustive constructor analysis establishes

$$\mathrm{survives}(c) = \mathrm{criteriaDecision}(c), \qquad \mathrm{survives}(c) = \mathrm{true} \iff \bigwedge_{i=1}^{10} (c_i = \mathrm{admitted}).$$

The conjunction fixes the unique all-admitted survivor. Its support, basis and provenance fields refer to the complete inherited Fold construction proved in the surrounding sections.

The six original witnesses use L=[held-a,held-b] and three positions. Complete support contains eight distinct words; the unit trace has three rows; the basis has eight records with eight distinct names; and the superposition record retains the exact amplitude-boundary marker. The joint witness combines two singleton-word carriers on each side, emitting four pair cells. Finally, observation by the first held label retains four of the eight branches, while the complete image record reconstructs all eight. These are direct evaluations of the same general constructions, alongside the exhaustive candidate output comparison.

The original program constructs each candidate from the ordered Cartesian product of its declared choice lists, joins its coordinate names to form the identifier, and retains the coordinate/form record. Decision lookup first finds that exact identifier in the complete product, then compares it with the joined all-admitted identifier. A missing identifier therefore fails lookup. The reason scan stops at the first rejected coordinate in declared order. Census hashes retain the complete candidate record; closure hashes retain the closure record and ordered decisions; each control retains its own observed result and receipt hash. Registration uses the same specification and source identity. These are the particular program operations compared with their native records.

> Source, proof and provenance: V3 complete_fold_support, sft/information_science/quantum_information_correspondence/law.py:15. Native results: SFT.InformationScience.QuantumEvidence.registrationHasNoFreeParameters, QuantumEvidence.lean:83; SFT.InformationScience.QuantumEvidence.lookupDependsOnlyOnIdentifier, QuantumEvidence.lean:86. Timeline: Turn 179, line 2041. Evidence: Completion TURN_179_QUANTUM_COMPLETION.

Program correspondence sources and proofs: V3 candidate_records, information_science/generated_law.py:L117; V3 registration, information_science/generated_law.py:L230; V3 generate_candidates, information_science/generated_law.py:L245; V3 decide_candidate, information_science/generated_law.py:L264; V3 closure_evidence, information_science/generated_law.py:L285; V3 control_records, information_science/generated_law.py:L187; QuantumGrammar.generatedCardinality, L138; QuantumGrammar.candidatesComplete, L140; QuantumGrammar.acceptanceIffExactCriteria, L198; QuantumEvidence.registrationData, L17; QuantumEvidence.censusData, L37; QuantumEvidence.decisionData, L46; QuantumEvidence.closureData, L61.

130. Canonical Fold basis bijection

The registered Fold Assembly generator supplies precisely held-a and held-b. Quantum support with these inherited labels contains exactly the words in Fold Assembly at the same finite depth. The original colon-joined basis names uniquely identify these words: the native proof establishes exact character decoding, cancels the basis prefix and recovers the held-label sequence. Hence all emitted names are distinct and every emitted name has one and only one source word. This result holds at arbitrary finite depth and preserves the original executable functions on their broader raw input domains. Together with the unit, provenance, pair-support, observation, reconstruction and succession proofs and the complete 1,024-candidate program correspondence, it completes the registered quantum-information support entry.

Put H=[held-a,held-b]. Complete-support membership and the inherited Fold generator give the same condition,

$$w \in W_n(H) \iff |w| = n \ \wedge \ (\forall x \in w,\ x = \text{held-a} \ \vee \ x = \text{held-b}).$$

To prove name injectivity, decode the name's bytes into characters. Each character of the fixed prefix, delimiter and held labels belongs to the explicit finite alphabet {h,e,l,d,-,a,b,:,s,i}; character-list induction proves exact decoding on that alphabet. Equality B(u)=B(v) therefore yields equality of their character lists. Cancelling the fixed “basis:” prefix leaves equality of the colon-joined label sequences.

For nonempty sequences, prefixing a colon produces blocks “:held-a” or “:held-b”, each of length seven. Their seventh characters distinguish the alternatives. Equality forces the same first label; cancelling the common seven-character block reduces to equality of the remaining blocks. Induction recovers the entire label sequence. The empty case is separated by the presence or absence of a first block. Hence

$$u, v \in W_n(H),\ B(u) = B(v) \implies u = v.$$

For any emitted name z, membership in the mapped name list supplies a word w with B(w)=z. The preceding implication proves that any other such source word equals w. Thus every emitted name has exactly one preimage, and the complete basis-name list is duplicate-free at every admitted finite depth.

> Source, proof and provenance: V3 complete_fold_support, sft/information_science/quantum_information_correspondence/law.py:15; V3 basis_correspondence, sft/information_science/quantum_information_correspondence/law.py:35. Native results: SFT.InformationScience.QuantumCanonical.admittedNamesInjective, QuantumCanonical.lean:224; SFT.InformationScience.QuantumCanonical.basisInverse, QuantumCanonical.lean:238. Timeline: Turn 179, line 2042. Evidence: Completion TURN_179_QUANTUM_COMPLETION.

131. Exact generated transition systems

A transition system retains the ordered generated state carrier, held action labels and source/action/target rows. Construction first checks state uniqueness, then action uniqueness and nonempty carriers; it then checks each row in order for carrier membership and repeated source/action coordinates. The relation may be partial. A requested step succeeds exactly when its matching target list is a singleton, preserving the original missing-transition boundary.

Execution first checks that the initial state belongs to the carrier, then retains that initial state and every successive generated target, propagating the first failed step. Native proofs identify successful execution exactly with an inductive chain of the original step results. Every admitted trace begins at its registered initial state, has one position for every action plus the initial position, and remains entirely within the state carrier. Constructor admission is equivalent to the exact original carrier and ordered-row conditions. 3,397 independent comparisons cover complete outputs and original errors, including duplicate states/actions, empty strings, missing transitions, repeated coordinates, escaped rows and failure order.

Write a system as (S,A,R), where each retained row is (s,a,t). Row admission requires s,t∈S and a∈A. Scanning the relation maintains the list K of previously seen (source,action) coordinates: an empty remainder succeeds; a head row succeeds only if it is carrier-bound and (s,a)∉K, after which validation continues with (s,a)::K. Together with distinct, nonempty S and A, this is exactly constructor admission, including its ordered failure behavior.

Define T_R(s,a)=[t:(s,a,t) occurs in R]. The executable step satisfies

$$\mathrm{step}(s,a) = \mathrm{ok}(t) \iff T_R(s,a) = [t].$$

The retained execution relation has the two clauses

$$E(s,[],[s]), \qquad \frac{\mathrm{step}(s,a) = \mathrm{ok}(t) \quad E(t,u,\tau)}{E(s,a::u,s::\tau)}.$$

Induction on the requested action list proves runFrom(s,u)=ok(τ) iff E(s,u,τ), by matching each successful step and recursive result; either error branch propagates unchanged. The outer run additionally requires s∈S. Induction on E gives head(τ)=s and |τ|=|u|+1. Each successful step selects a retained row, whose validation gives its target in S. Starting with s∈S, the same induction therefore puts every trace state in S.

> Source, proof and provenance: V3 __post_init__, sft/computation/operations.py:31; V3 step, sft/computation/operations.py:45; V3 run, sft/computation/operations.py:51. Native results: SFT.Computation.StateTransition.runFromMeaning, StateTransition.lean:94; SFT.Computation.StateTransition.createMeaning, StateTransition.lean:163; SFT.Computation.StateTransition.executionStates, StateTransition.lean:230. Timeline: Turn 180, line 2050. Evidence: Completion TURN_182_STATE_COMPLETION.

132. Retained execution composition and extension

Every successful execution admits an exact decomposition into its retained predecessor positions and terminal state. Two such paths with a common terminal/initial interface compose by concatenating their action traces and predecessor records, retaining the shared state exactly once. An empty action trace is the identity path; appending one generated action retains the complete previous trace and appends its generated target. These statements hold for arbitrary finite executions.

For every admitted system, a requested transition succeeds if and only if its exact source/action/target row belongs to the retained relation. Original constructor validation excludes any second target row at the same coordinate. Consequently, an admitted extension that retains the earlier state carrier and transition rows preserves every earlier successful execution with its exact trace. This applies to appended carriers and rows and to both components embedded in a common admitted assembly. Compatibility is decided by the original constructor; no additional transition or scheduling rule is imposed.

Make the terminal interface explicit by writing P(s,u,b,t) for a retained path with actions u, predecessor list b and terminal t. Its identity clause is P(s,[],[],s); its successor clause prepends the current state and action after an exact successful step. Induction in both directions establishes

$$E(s,u,\tau) \iff \exists b,t,\ \tau = b +\!+ [t] \ \wedge\ P(s,u,b,t).$$

If P(s,u,b,m) and P(m,v,c,t), induction on the first path gives

$$P(s,u +\!+ v, b +\!+ c, t), \qquad \mathrm{runFrom}(s,u +\!+ v) = \mathrm{ok}((b +\!+ c) +\!+ [t]).$$

The identity case returns the second path. The successor case retains the first edge and applies the induction hypothesis to its remainder. Taking the second path to be the single edge m→t yields the old complete trace b++[m] followed by [t], with no repeated interface inserted.

For an admitted relation, the seen-coordinate invariant excludes any later row at a previously encountered coordinate. Induction through validation therefore gives T_R(s,a)=[t] whenever (s,a,t)∈R. Together with step membership, this proves step(s,a)=ok(t) iff (s,a,t)∈R. If an admitted extension retains S and every row of R, each old step still succeeds with the same target. Induction on the original execution preserves its exact trace, and retained initial-state membership preserves the outer run. Appended carriers and common admitted assemblies are direct instances of these two inclusions.

> Source, proof and provenance: V3 run, sft/computation/operations.py:51. Native results: SFT.Computation.StateComposition.nativeComposition, StateComposition.lean:54; SFT.Computation.StateExtension.extendRun, StateExtension.lean:87. Timeline: Turn 181, line 2056. Evidence: Completion TURN_182_STATE_COMPLETION.

133. Generated words and exact coordinate rewrite

The generated-word operation retains the original empty-One marker and then every complete word layer up to the given finite position depth, preserving both layer order and Cartesian order. Native induction identifies the exact frontier at every depth and proves that one new position appends the entire next layer. The original unique-alphabet, unique-position and nonempty-alphabet conditions remain unchanged; the literal empty-One symbol remains allowed.

Exact rewriting validates the unique position trace, equal word/position lengths and selected held position. The output retains the full position trace, changes the selected coordinate to its replacement and preserves every other coordinate and total length. 2,952 independent full-output/error comparisons and all three original formal-computation witnesses pass. Fresh native execution matches every original 256-candidate lookup, registration, census, closure, control and digest field. These results complete Exact State and Transition Law on its registered generated-finite boundary; no language, machine model or resource bound is introduced.

Let F_n be the current word frontier and G_n the retained cumulative output. The source recurrence is

$$F_0 = [[]], \quad G_0 = [[\text{empty-One}]], \qquad F_{n+1} = \mathrm{concat}_{w \in F_n}[w +\!+ [a] : a \in A], \quad G_{n+1} = G_n +\!+ F_{n+1}.$$

The ordered word-extension identity proves F_n=$W_n(A)$. Substitution into the second recurrence gives G_n= [[empty-One]]++$W_1(A)$++⋯++$W_n(A)$, retaining every layer and its order. Admission requires distinct A and P and nonempty A; the depth n=|P| may be zero. Membership is exactly the marker word or a word of

positive length at most n whose symbols all belong to A. The host empty frontier [] and the one-symbol marker word [empty-One] remain different objects.

For an admitted rewrite, P is distinct, |w|=|P| and the selected position h lies in P. The output is the coordinatewise map

$$R_{h,r}(w)_i = \begin{cases} r, & P_i = h, \\ w_i, & P_i \neq h. \end{cases}$$

Simultaneous induction over P and w proves

$$\mathrm{zip}(P, R_{h,r}(w)) = [(p, \text{if } p = h \text{ then } r \text{ else } x) : (p, x) \in \mathrm{zip}(P, w)].$$

The empty case is immediate; each successor retains its position and emits exactly the selected replacement or original symbol before the inductive tail. The same induction gives $|R_{h,r}(w)|$=|w|. Thus the displayed trace proves the selected-coordinate change and every unchanged coordinate together.

> Source, proof and provenance: V3 generated_words, sft/computation/operations.py:62; V3 exact_rewrite, sft/computation/operations.py:78. Native results: SFT.Computation.StateWords.rewriteLength, StateWords.lean:107; SFT.Computation.StateWords.rewriteCoordinate, StateWords.lean:118. Timeline: Turn 182, line 2066. Evidence: Completion TURN_182_STATE_COMPLETION.

134. Finite language derivation certificates

The language formation rules retain the empty construction frontier and append a held alphabet symbol at each finite successor. Native derivation trees are sound and complete for the exact generated word layer: a word belongs to that layer if and only if it has the corresponding finite derivation certificate. The certificate fixes its depth, admits only symbols in the generated alphabet and retains a recoverable predecessor at each successor. Concatenating two derivations produces the concatenated word at the summed construction depth.

For the complete cumulative language output, membership is exactly the original empty-One marker or a positive-depth derivation bounded by the registered position trace. The marker and the empty host frontier remain distinct, and literal marker-valued alphabet symbols retain their original behavior. The shared native operation proofs and all three claim-specific witnesses combine with full 256-candidate program/output/hash agreement to complete the registered Fold Formal-Language and Grammar Law. The seed-and-append equivalence holds for every finite depth n; a call with position depth N returns precisely the marker together with all positive-depth derivations through N.

The native finite certificate $D_A(n, w)$ has precisely the source's seed and append clauses:

$$D_A(0, []), \qquad \frac{D_A(n, w) \quad a \in A}{D_A(n+1, w +\!\!+ [a])}.$$

Soundness follows by induction on the certificate: the seed belongs to $W_0(A)$, and ordered extension inserts each appended held symbol into $W_{n+1}(A)$. Conversely, membership in $W_{n+1}(A)$ decomposes into a predecessor w∈$W_n(A)$ and a∈A with result w++[a]. Induction on n constructs the corresponding append certificate. Hence

$$w \in W_n(A) \iff D_A(n, w), \qquad D_A(n, w) \Longrightarrow |w| = n \ \wedge \ \forall a \in w,\ a \in A.$$

Every successor certificate therefore has a recoverable predecessor and final held symbol. Substituting this equivalence into cumulative-output membership gives w∈G_N iff w=[empty-One] or there exists 0<n≤N with $D_A(n, w)$. This disjunction preserves literal marker-valued symbols in ordinary generated words as well as the distinguished initial marker.

For $D_A(m,u)$ and $D_A(n,v)$, induction on the second certificate proves D_A(m+n,u++v). The seed uses u++[]=u. Its append case applies one append to the induction hypothesis and reassociates (u++v)++[a]=u++(v++[a]). Thus concatenation retains the complete finite construction certificate at the summed depth.

> Source, proof and provenance: V3 generated_words, sft/computation/operations.py:62. Native results: SFT.Computation.LanguageDerivation.productionPredecessor, LanguageDerivation.lean:68; SFT.Computation.LanguageDerivation.composeDerivations, LanguageDerivation.lean:79. Timeline: Turn 183, line 2076. Evidence: Completion TURN_183_LANGUAGE_COMPLETION.

135. Finite recognition with retained accepting classes

The finite-recognition kernel retains its generated states, input alphabet, original transition relation, initial state, accepting class and complete run. Its accepting-class statement is represented by a native predicate requiring the original successful execution and a terminal state held in that class. This predicate is proved equivalent to the retained-path representation; it does not introduce a new executable source method.

Accepted traces preserve the initial state, full trace length and admitted input alphabet. The identity trace accepts exactly when its state belongs to the accepting class; an empty class admits no accepted trace. Matching paths compose with every trace position retained, and an admitted carrier/transition extension preserves earlier accepted traces. A one-symbol successor retains its prior trace and uses the original generated target. All results hold at arbitrary finite depth. The 256-candidate program, witness, control and hash correspondence completes the Fold Finite Automaton entry. The acceptance equivalence quantifies over arbitrary admitted finite transition systems, requested finite input words and held accepting classes; each accepted trace is reconstructed from the original transition rows.

For accepting class F, the retained acceptance statement is

$$\mathrm{Accepted}(s,u,F,\tau) \iff s \in S \ \wedge\ \exists b,t,\ \tau = b +\!\!+ [t] \ \wedge\ P(s,u,b,t) \ \wedge\ t \in F.$$

The terminal-state recursion returns t on b++[t], by induction on b. Applying the execution/path correspondence in each direction therefore proves

$$\mathrm{Accepted}(s,u,F,\tau) \iff \mathrm{run}(s,u) = \mathrm{ok}(\tau) \ \wedge\ \exists t,\ \mathrm{terminalState}(\tau) = \mathrm{some}(t) \ \wedge\ t \in F.$$

This immediately gives rejection for F=[] and, for s∈S, identity acceptance iff s∈F. The execution theorem gives head(τ)=s and |τ|=|u|+1. In an admitted system every edge has its action in A; induction over the path places every requested symbol in A. If F is held in S, the retained accepting terminal also lies in S.

Composition uses the path equation of Section 132 and membership of its final t in F. For a one-symbol extension, a successful step from the previous terminal m to t∈F gives the accepted trace (b++[m])++[t] on input u++[a]. Finally, an admitted carrier/row extension preserves the exact run and leaves its terminal and accepting membership unchanged; the displayed equivalence transfers acceptance directly. These deductions retain all intermediate states at arbitrary finite depth.

> Source, proof and provenance: V3 run, sft/computation/operations.py:51. Native results: SFT.Computation.AutomatonRecognition.acceptedInputAlphabet, AutomatonRecognition.lean:104; SFT.Computation.AutomatonRecognition.acceptedSuccessor, AutomatonRecognition.lean:111. Timeline: Turn 184, line 2086. Evidence: Completion TURN_184_AUTOMATON_COMPLETION.

136. Retained held-position rewrite traces

The exact coordinate rewrite retains its original position validation and output behavior. Replacing the same held position twice has precisely the effect of the second replacement; repeating an identical replacement is idempotent. Replacements at distinct held positions commute. Native proofs establish

these equations for arbitrary finite words with the original matching position trace. An admitted rewrite permits another replacement at the same position without changing its admission conditions.

Retained step records associate rule identity, source word, held replacement position, replacement symbol and successor word. Replaying a record uses the unchanged exact rewrite operation. Each valid record preserves word length and carries the complete selected/unchanged coordinate correspondence. Finite chains compose at their exact successor/source interface, preserve length and retain a replay proof for every recorded step. These typed records express the declared provenance fields; they do not add an executable operation to V3.

For a fixed matching position trace P, write R_{h,a}(w) for the coordinate rewrite. At each coordinate p the overwrite equation reduces to

$$\text{if } p = h \text{ then } b \text{ else } (\text{if } p = h \text{ then } a \text{ else } x) = \text{if } p = h \text{ then } b \text{ else } x.$$

Induction on the paired position/word lists therefore gives R_{h,b}(R_{h,a}(w))=R_{h,b}(w). Setting b=a proves idempotence. For h≠k, the three cases p=h, p=k and neither give the same value in either order, proving

$$R_{k,b}(R_{h,a}(w)) = R_{h,a}(R_{k,b}(w)).$$

Length preservation leaves the rewrite admission conditions unchanged after the first replacement: P remains distinct, the word still has length |P|, and h is still held in P. Thus overwrite also holds for the admitted executable results.

A record r=(ruleIdentity,h,a,w,v) replays precisely when exactRewrite(w,h,P,a)=ok(v). Its coordinate theorem gives the entire retained map from (p,x) to (p,if p=h then a else x), and |v|=|w|. Define T(w,[],w) as the identity trace; prepend r to T(v,rs,t) only when r replays from w to v. Induction over this trace gives |t|=|w| and replay validity for every member of the record list. Induction over a first trace ending at m, with a second beginning at m, proves T(w,rs++ss,t). Every recorded rewrite and its rule identity remain in that concatenation.

> Source, proof and provenance: V3 exact_rewrite, sft/computation/operations.py:78. Native results: SFT.Computation.RewritingDerivation.traceComposition, RewritingDerivation.lean:101; SFT.Computation.RewritingDerivation.traceRecordsReplay, RewritingDerivation.lean:108. Timeline: Turn 185, line 2094. Evidence: Completion TURN_186_REWRITING_COMPLETION.

Exact operation and proof anchors: overwriteRows, RewritingDerivation.lean:L8; commuteRows, RewritingDerivation.lean:L26; rewriteSuccessorAdmitted, RewritingDerivation.lean:L49; rewriteIdempotent, RewritingDerivation.lean:L58.

137. Registered rewrite boundaries and trace classes

The finite relation of registered rewrite applications retains complete records, including rule identity, source word, selected position, replacement and successor. Admitted applications belong to this retained relation, have both endpoints in the generated carrier and replay the original exact rewrite operation. Every admitted application has an exact source-symbol occurrence at its held position, a source-word decomposition around that occurrence and the replacement at that same coordinate. Missing registrations and endpoints outside the generated carrier are excluded by the declared boundary; the raw executable helper keeps its original guards.

A terminal word has no admitted outgoing application. Consequently its retained traces are empty and it cannot begin a cyclic or branching trace. A cyclic trace retains a nonempty return path, which can be repeated by exact composition. A branching trace retains two admitted applications from the same word with distinct successors; each yields its own complete one-step trace. Identity, serial composition and one-record succession preserve the retained relation. Extending the registered application relation and

generated carrier preserves all previous traces. These predicates formalise the declared finite trace classes; no claim of universal termination or confluence is introduced. Full 256-candidate native program agreement and the original three operational witnesses complete the registered Rewriting entry.

For registered records Q and carrier C, define A(r) by r∈Q, source(r)∈C, successor(r)∈C and exact replay. A retained trace additionally requires its initial word in C and A(r) for each record. For an admitted record, equal word/position lengths and membership of its held position h imply an occurrence (h,x)∈zip(P,w). Induction through that zipped list locates the occurrence and produces a decomposition

$$w = u +\!\!+ (x :: v), \qquad (h, a) \in \mathrm{zip}(P, \mathrm{successor}(r)).$$

Here a is the record's replacement. The coordinate theorem supplies the second statement at the same held position. Failure of record registration or either carrier membership directly contradicts the corresponding conjunct of A(r).

Terminal(w) means w∈C and no admitted record has source w. Any nonempty retained trace has a head record whose source is w, contradicting this condition; hence every retained trace starting there has an empty record list. This excludes both a nonempty return cycle and the two outgoing records required for branching.

For a retained return trace T(w,rs,w), serial composition gives T(w,rs++rs,w). For two admitted records r and s from w with different successors, the replay constructor followed by identity gives T(w, [r],successor(r)) and T(w,[s],successor(s)), retaining the distinct endpoints. Appending an admitted record to a trace ending at its source is the same serial construction. If Q⊆Q′ and C⊆C′, each record keeps its replay proof and both endpoint memberships transfer by inclusion; induction is unnecessary for the unchanged trace itself. These inclusions establish preservation of every previous retained trace under the registered extension.

> Source, proof and provenance: V3 exact_rewrite, sft/computation/operations.py:78. Native results: SFT.Computation.RewritingClasses.retainedSuccessor, RewritingClasses.lean:153; SFT.Computation.RewritingClasses.retainedExtension, RewritingClasses.lean:160. Timeline: Turn 186, line 2102. Evidence: Completion TURN_186_REWRITING_COMPLETION.

138. Structural finite recursion with complete values

The recursion operation validates unique successor-position identities, then retains the supplied base and every value produced by its exact step relation. Empty bases, empty successor traces and empty string labels retain their original behavior. Native derivation chains agree exactly with the executable result and uniquely determine all intermediate values. The full trace has one value per successor plus the base.

At every finite depth, appending a generated successor appends exactly its step result and preserves the complete earlier trace. The terminal value supplies the base interface for subsequent recursion. Composition concatenates predecessor records with the second complete trace, retaining the shared interface once; its admission uses the original distinct-position check on the combined trace. The base and successor equations establish exact intermediate-value provenance and terminal composition for every finite distinct-position list, with no fixed maximum length. 800 independent output/error comparisons and all 256 original native program comparisons pass, completing the registered Fold Recursion Law.

Let f(v,p) be the supplied exact step on list-valued v, and let V(b,P) retain all values. Its defining equations are

$$V(b, []) = [b], \qquad V(b, p :: P) = b :: V(f(b, p), P).$$

Structural recursion returns ok(V(b,P)) exactly when P has distinct position identities. The derivation relation has the matching base clause D(b,[],[b]) and successor clause taking D(f(b,p),P,τ) to D(b,p::P,b::τ). Induction on P constructs D(b,P,V(b,P)); induction on any certificate D(b,P,τ) proves

τ=V(b,P). Thus the executable trace and the full derivation chain agree uniquely. The same recursion gives head(V)=b and |V(b,P)|=|P|+1.

Let t(b,P) be the recursively computed terminal and B(b,P) the predecessor list, omitting that terminal. Induction gives

$$V(b,P) = B(b,P) \mathbin{+\!\!+} [t(b,P)],$$

$$t(b, L \mathbin{+\!\!+} R) = t(t(b,L),R), \qquad V(b, L \mathbin{+\!\!+} R) = B(b,L) \mathbin{+\!\!+} V(t(b,L),R).$$

The empty-left case is immediate. In the successor case, retain b, replace the recursive base by f(b,p), and apply the induction hypothesis to the remaining left positions. Taking R=[p] yields

$$V(b, P \mathbin{+\!\!+} [p]) = V(b,P) \mathbin{+\!\!+} [f(t(b,P),p)].$$

The executable composition and successor equations require precisely the original distinctness check on the combined position trace. For every pair (p,v) in zip(P,B(b,P)), induction also places f(v,p) in V(b,P): the head pair yields the next base, and each tail pair uses the recursive proof. This retains the exact step result for every successor, including empty list-valued bases.

> Source, proof and provenance: V3 structural_recursion, sft/computation/operations.py:85. Native results: SFT.Computation.RecursionDerivation.recursionComposition, RecursionDerivation.lean:108; SFT.Computation.RecursionDerivation.stepRelation, RecursionDerivation.lean:114. Timeline: Turn 454, line 3992. Evidence: Completion TURN_187_RECURSIVE-FUNCTION_COMPLETION.

139. Fold binding and self-application calculus

The registered FORM-LAMBDA-CALCULUS entry derives the generated Fold kernel from admitted finite forms, exact relations, distinction ledgers and composition. Its complete product contains 256 combinations of eight structural requirements. Native exhaustive proofs establish that precisely the complete carrier, canonical held identity, source-bound exact relation, complete trace, explicit boundary, interface-preserving composition, structural successor and no-extra-premise combination survives. Generated term grammar, held binder identity, free and bound ledgers, capture-safe substitution, exact reduction and normal or cyclic trace retain their registered structural meaning.

The entry's source-bound operational kernel is the exact transition system, generated words and held-position rewrite. Successful execution reconstructs every result from its initial form and actual retained relation rows. The structural identity trace has no predecessor. Appending a permitted step preserves the entire earlier trace and adds its exact successor. Matching interfaces compose without discarding intermediate records, and admitted extension retains previous runs.

Normal traces have no successful outgoing registered step; their identity execution is retained and they cannot form a nonempty cycle. Cyclic traces retain a nonempty return path, replay through the exact source execution, and compose with themselves while retaining both copies of the trace. These native results quantify over arbitrary finite paths of the source transition kernel. A normal state permits only its identity path; a cyclic state carries an explicit nonempty return path whose repetitions are supplied by serial composition.

All 256 original program decisions, three operational witnesses, controls, registration, closure and hashes agree with the pinned entry. The registered Fold Binding and Self-Application Calculus Law is complete within its original boundary. Source: claims/SFT-COMP-FORM-LAMBDA-CALCULUS-001/execution.py and its six-file source manifest; current computation publication v1.5.1 section 14.

Using the retained path P(s,u,b,t), induction on its constructors gives

$$(x,a) \in \mathrm{zip}(b,u) \Longrightarrow \exists y,\ (x,a,y) \in R.$$

The head case uses the actual row selected by the successful step; the tail case applies the induction hypothesis. For s∈S, replay is run(s,u)=ok(b++[t]). The identity path has u=b=[]; appending a successful edge t→v produces run(s,u++[a])=ok((b++[t])++[v]). The serial and admitted-extension equations of Section 132 preserve these exact records.

The normal and cyclic trace classes are

$$N(s) \iff s \in S \wedge \forall a, t,\ \mathrm{step}(s,a) \neq \mathrm{ok}(t),$$

$$C(s) \iff s \in S \wedge \exists u, b,\ u \neq [] \wedge P(s,u,b,s).$$

A path from a normal state is either identity or has a first successful edge. The latter contradicts N(s), so its action list must be empty, and N(s) excludes C(s). Identity still executes to [s]. A cyclic witness replays to b++[s], and composing it with itself gives

$$\mathrm{run}(s, u \mathbin{+\!\!+} u) = \mathrm{ok}((b \mathbin{+\!\!+} b) \mathbin{+\!\!+} [s]).$$

This retains both complete traversals of the cycle within the registered transition kernel.

> Source, proof and provenance: V3 run, sft/computation/operations.py:51. Native results: SFT.Computation.BindingTrace.traceReplay, BindingTrace.lean:57; SFT.Computation.BindingTrace.successorRetainsTrace, BindingTrace.lean:64. Timeline: Turn 193, line 2153. Evidence: Completion TURN_193_BINDING_COMPLETION.

140. Separate FORMX source components

The later FORMX source defines variable, application and abstraction tuples, free-name membership, whole-tree renaming and substitution. Their native translation preserves the exact finite fresh-name search and exhaustion behavior. Structural induction establishes the application and abstraction execution equations at arbitrary term depth, including mapping composition and error propagation. 2,052 source-expression comparisons supplement these equations. These translated helper components belong to the separately registered FORMX entry; its completion is not claimed here.

Free-name membership follows FV(var(x))={x}, FV(app(u,v))=FV(u)∪FV(v), and FV(abs(b,t))=FV(t)\{b}. Structural induction identifies these occurrence clauses with the executable Boolean. Whole-tree renaming changes every matching variable and binder label. Induction preserves term size and proves identity renaming. For whole-tree mapping M_f, induction also gives

$$M_f(M_g(t)) = M_{f \circ g}(t), \qquad \mathrm{substituteWith}(M_g(t), f, x, r) = \mathrm{substituteWith}(t, f \circ g, x, r).$$

Application substitution evaluates the left child first, propagates its error if present, then evaluates the right child and constructs the application. Substitution into abs(b,t) returns it unchanged when b=x. Otherwise, if b∉FV(r), it substitutes in the body and retains b. If b∈FV(r), it selects the first c in [u,v,w,z] with c∉FV(t), c∉FV(r) and c≠x, renames the whole body from b to c, substitutes into that renamed body and constructs abs(c,result). Exhaustion returns StopIteration. Induction on the search proves the selected-name conditions. Induction on the term proves that every propagated substitution error is that same exhaustion error. Mapping composition establishes the exact abstraction execution equation at arbitrary depth.

> Source, proof and provenance: V3 free, formx_001_022_laws_v1.py:L125; V3 rename, formx_001_022_laws_v1.py:L134; V3 substitute, formx_001_022_laws_v1.py:L143; freeMeaning, BindingTerms.lean:L35; renameSize, BindingTerms.lean:L68; renameIdentity, BindingTerms.lean:L74; freshBoundary, BindingTerms.lean:L138; mapCompose, BindingCorrespondence.lean:L5; mappedExecution, BindingCorrespondence.lean:L12; applicationEquation, BindingCorrespondence.lean:L51; abstractionEquation, BindingCorrespondence.lean:L59; substitutionError, BindingCorrespondence.lean:L131; Timeline, line 2117; Timeline, line 2123.

141. Finite executable descriptions and halt traces

The Abstract-Machine entry retains generated description and store words, its configuration carrier, exact transition rows, initial configuration and complete execution trace. Its word boundary is precisely the original generated-word support, including the empty-One marker and all finite admitted words through the declared positions. Held-position replacement preserves the exact generated store coordinates. For every admitted alphabet and position list, description membership is exactly the empty-One marker or a positive-length held-symbol word whose length is at most that list's length. This characterizes the complete description support generated by each call.

Native induction and execution correspondence establish uniqueness of the complete trace and membership of every configuration in the admitted carrier. A retained halt trace records its terminal no-outgoing-step boundary separately from exhaustion of requested actions. It replays through the original source run, contains one predecessor configuration per action, admits no nonempty continuation, and composes with a preceding path at the held interface. The identity halt is its base. Existing exact successor and extension results preserve every prior trace record.

The complete 256-candidate structural derivation, original three witnesses, four controls and every native program field and digest agree with the pinned entry. These results complete the registered Fold Abstract-Machine Law within its original description boundary.

A halt certificate is H(s,u,b,t) := s∈S ∧ P(s,u,b,t) ∧ N(t), where N(t) is the no-successful-outgoing-step predicate. Path reconstruction gives

$$H(s,u,b,t) \Longrightarrow \mathrm{run}(s,u) = \mathrm{ok}(b +\!\!+ [t]), \qquad |b| = |u|.$$

Two execution certificates with the same initial state and actions both equal the result of runFrom; equality of their successful results gives equality of the full traces. Admission and row membership put every configuration in S. Any nonidentity continuation from t begins with a successful edge, contradicting N(t); hence every continuation has an empty action list. For N(s), H(s,[],[],s) is the identity halt. Composing P(s,u,b,m) with H(m,v,c,t) yields H(s,u++v,b++c,t), retaining the terminal boundary. Description admission is exactly w=[empty-One] or 0<|w|≤|positions| with all symbols held in the alphabet, as proved by generated-word membership.

> Source, proof and provenance: V3 run, sft/computation/operations.py:51; V3 source derivation, claims/SFT-COMP-FORM-ABSTRACT-MACHINE-001/WHY_DERIVATION_CHECK.md:1. Native results: SFT.Computation.MachineExecution.noUnregisteredConfiguration, MachineExecution.lean:47; SFT.Computation.MachineExecution.descriptionBoundary, MachineExecution.lean:56. Timeline: Turn 194, line 2159. Evidence: Completion TURN_194_ABSTRACT_MACHINE_COMPLETION.

142. Circuit interfaces and complete evaluation traces

Finite connected local transformations retain the generated node/wire carrier and typed interfaces supplied by the admitted Graph and Category dependencies. The native evaluation relation records successive successful original Category compositions. Induction proves complete ordered concatenation of all local trace rows, preservation of the initial interface, rejection of interface mismatches, serial composition and arbitrary finite successor extension.

When component interfaces are registered graph nodes and their local transformations are registered wires, evaluation supplies an original complete graph path and its terminal output remains in the node carrier. The original cycle detector is equivalent to a complete retained return path. Its false result excludes every such return; declared cycles retain their full path evidence. No external gate model is imported.

The full 256-candidate structural derivation, original operational witnesses, controls and complete native program agree with the pinned entry. This completes Fold Circuit Syntax and Evaluation within its generated-finite boundary. Resource size and depth comparisons remain with the source Complexity entries.

A successful Category composition a;b requires target(a)=source(b) and returns source(a), target(b), and trace(a)++trace(b). Evaluation begins with identity and repeatedly applies this exact composition. For locals L=[c_1,...,c_n] evaluated from a to r, induction gives

$$\mathrm{source}(r) = \mathrm{source}(a), \qquad \mathrm{trace}(r) = \mathrm{trace}(a) \mathbin{+\!\!+} \mathrm{trace}(c_1) \mathbin{+\!\!+} \cdots \mathbin{+\!\!+} \mathrm{trace}(c_n).$$

The successor case substitutes the composition equation and reassociates concatenation. A mismatched next interface contradicts its required successful edge. Induction over a first evaluation composes it with a second; taking the second to contain one component proves successor extension.

For registered graph components, each local endpoint is a held node and their pair is a held edge. Interface equality joins this edge to the current output. Induction concatenates these paths and places the final output in the node carrier. The source cycle predicate is true exactly when a held node n starts a valid path n::(b++[t]) with t∈n::b. If the predicate is false, any such return would imply true by the return-path theorem. This proves the stated feed-forward exclusion and retains complete path evidence for a declared cycle.

> Source, proof and provenance: V3 run, sft/computation/operations.py:51; V3 source derivation, claims/SFT-COMP-FORM-CIRCUIT-001/WHY_DERIVATION_CHECK.md:1. Native results: SFT.Computation.CircuitComposition.successor, CircuitComposition.lean:67; SFT.Computation.CircuitComposition.earlierRowsRetained, CircuitComposition.lean:73; SFT.Computation.CircuitGraph.declaredCycleTrace, CircuitGraph.lean:40; SFT.Computation.CircuitGraph.feedForwardExcludesReturn, CircuitGraph.lean:47. Timeline: Turn 195, line 2167. Evidence: Completion TURN_195_CIRCUIT_COMPLETION.

143. Operational process observations and failure traces

An admitted operational process retains its initial generated configuration, exact enabled transition rows, requested choices and full configuration trace. Successful and failed observations remain the original source outputs. Native failure records are equivalent to the original execution error branches at arbitrary finite depth. Each record decomposes the requested choices into a successfully executed prefix, the failing action and its unexecuted suffix. The complete successful prefix replays exactly up to the failure configuration.

A recorded failure excludes a successful result on the same execution. Prepending a successful path preserves every prior choice/configuration and propagates the same original error. Terminal processes retain their explicit no-outgoing-step boundary; cyclic processes retain a nonempty return path that replays and composes without losing records. These results use the unchanged source transition system and its admitted finite trace operations. No process algebra, distributed timing or stochastic selection is introduced.

All 256 original grammar members and native lookup decisions, 3 operational witnesses, 4 controls, registration/closure records and hashes agree with pinned V3. These results complete the registered Fold Operational-Process Law within its original generated-finite boundary.

A failure certificate F(s,u,b,q,e) stops when step(s,a)=error(e), with u=a::rest, b=[] and q=s. Its continued clause prepends a successful edge s→t to a failure from t, retaining s in b. Induction follows the executable propagation of errors. Conversely, induction on u splits its first step into error or success and reconstructs the corresponding certificate. Thus

$$\mathrm{runFrom}(s,u) = \mathrm{error}(e) \iff \exists b, q,\ F(s,u,b,q,e).$$

Induction on the certificate extracts u=v++a::z, P(s,v,b,q) and step(q,a)=error(e). The prefix replays as runFrom(s,v)=ok(b++[q]); z is the unexecuted suffix. Distinct error/success constructors exclude a successful result for the same input. Prepending P(s,v,b,m) to F(m,u,c,q,e) gives F(s,v++u,b++c,q,e), by induction on the prefix. For s∈S, the outer run passes its initial-state check and emits the same error.

> Source, proof and provenance: V3 run, sft/computation/operations.py:51; V3 source derivation, claims/SFT-COMP-FORM-OPERATIONAL-PROCESS-001/WHY_DERIVATION_CHECK.md:1. Native results: SFT.Computation.ProcessFailure.serialFailure, ProcessFailure.lean:79; SFT.Computation.ProcessFailure.admittedFailureObserved, ProcessFailure.lean:88. Timeline: Turn 196, line 2173. Evidence: Completion TURN_196_OPERATIONAL_PROCESS_COMPLETION.

144. Process composition and exact decomposition

Every retained serial execution decomposes at any declared cut in its action trace. Native induction reconstructs the intermediate configuration and both complete component paths; their predecessor records concatenate to the original trace. Conversely, exact interface composition reconstructs that original path. Each component replays through the unchanged source execution kernel.

Independent parallel records retain both systems, their initial configurations, action choices and original observations separately. Component execution reconstructs the observed pair, and unchanged component records determine unchanged observations. The exact source Category interface rule retains both local traces and rejects mismatch. These records add no shared state, timing, communication or scheduler. Concurrency involving communication or shared causality remains downstream as in V3.

The original 256-candidate grammar, witnesses, controls and complete native program agree with the pinned source. These results complete the registered Fold Process Composition and Decomposition Law within its original boundary.

For a declared cut u++v, induction on u establishes

$$P(s, u +\!+ v, b, t) \iff \exists m, c, d,\ b = c +\!+ d \ \wedge P(s, u, c, m) \ \wedge P(m, v, d, t).$$

At the empty cut choose m=s and c=[]; the second path is the original path. For a successor cut, remove the first edge, apply the induction hypothesis to the remainder, and prepend the edge and source state to the first component. The reverse direction is serial composition. Native replay gives component outputs c++[m] and d++[t], while the original output is (c++d)++[t].

For independent systems L,R, initials l,r and actions u,v, reconstruction consists of

$$O_L = \mathrm{run}_L(l, u), \qquad O_R = \mathrm{run}_R(r, v).$$

Pairing these equalities reconstructs (O_L,O_R), including either component's original error result. Equal systems, initials and action lists force equal observations by substitution. This product retains two independent original executions. Category composition instead uses the exact interface and trace equation of Section 142.

> Source, proof and provenance: V3 run, sft/computation/operations.py:51; V3 source derivation, claims/SFT-COMP-FORM-COMPOSITION-001/WHY_DERIVATION_CHECK.md:1. Native results: SFT.Computation.AssemblyDecomposition.componentObservationUnique, AssemblyDecomposition.lean:62; SFT.Computation.AssemblyDecomposition.parallelComponents, AssemblyDecomposition.lean:72. Timeline: Turn 197, line 2179. Evidence: Completion TURN_197_COMPOSITION_COMPLETION.

145. Registered step-to-trace simulation

The model-equivalence construction retains a translation between the two registered generated grammars. For a translation satisfying the source step-or-trace simulation condition, native induction constructs a complete target path for every finite source path. The translated terminal interface is

preserved, and the original target execution replays the complete generated path when the translated initial configuration belongs to its carrier.

The overhead ledger retains one complete target-action segment for each source step. Flattening concatenates all generated target actions in order; appending a source step appends exactly its target segment. No constant overhead or one-target-step restriction is introduced. Identity simulation and composition of successive simulations preserve the exact path relation. Registered finite encoding and exact reverse decoding connect these traces to the held model descriptions.

The supplied translation e satisfies StepSimulation exactly when every successful source step s→t has a target path P_T(e(s),a,b,e(t)) for some finite target actions a and predecessors b. Induction over a source path constructs a ledger L with one target-action segment per source step:

$$P_S(s,u,b,t) \Longrightarrow \exists L, c, \quad |L| = |u| \ \wedge \ P_T(e(s), \mathrm{flatten}(L), c, e(t)).$$

The identity uses empty ledger and identity target path. At a successor, apply the supplied step simulation to the first edge, prepend its segment to the inductively obtained ledger, and compose the two target paths. When e(s) belongs to the target carrier, replay returns c++[e(t)]. The ledger satisfies flatten(L++K)=flatten(L)++flatten(K), by induction on L; in particular appending one segment appends exactly that segment to the target actions. Identity simulation uses the original one-step path. Composing two simulations applies the second path-simulation theorem to the first simulation's finite segment, preserving the twice-translated endpoints.

> Source, proof and provenance: V3 run, sft/computation/operations.py:51; V3 source derivation, claims/SFT-COMP-FORM-MODEL-EQUIVALENCE-001/WHY_DERIVATION_CHECK.md:1. Native results: SFT.Computation.ModelSimulation.flattenedAppend, ModelSimulation.lean:60; SFT.Computation.ModelSimulation.ledgerSuccessor, ModelSimulation.lean:69. Timeline: Turn 198, line 2186. Evidence: Completion TURN_199_MODEL_EQUIVALENCE_COMPLETION.

146. Finite registered model translation and reconstruction

The description translation composes the original finite encoding lookup with the target description decoder. Validated tables and their exact decoders provide total lookup on each registered carrier. Shared used-code coverage supplies a target description for every source description and derives its exact reverse reconstruction. Uniqueness follows from the unchanged lookup and decoder operations.

For an admitted original system, every successful native execution reconstructs its complete path and terminal carrier membership. The registered step-or-trace simulation constructs the corresponding target execution and its complete action-segment ledger. The finite-table reconstruction then decodes the translated terminal back to the original terminal. Applying this construction in each direction establishes the registered bidirectional execution correspondence under the two supplied simulation conditions. Equivalence retains V3’s boundary of the two registered generated model grammars.

All 256 original candidates, native lookup decisions, 3 witnesses, 4 controls and complete program fields and hashes agree with pinned V3. Together with the native encoding, trace and reconstruction results, this completes the registered Fold Computational-Model Equivalence Law.

Write E_1,E_2 for the validated finite encoding tables and D_1,D_2 for their exact decoders. Translation is the original relational composition

$$T_{E,D}(x,y) \iff \exists w, \ \mathrm{lookup}(E,x) = \mathrm{some}(w) \ \wedge \ \mathrm{decode}(w,D) = \mathrm{ok}(y).$$

For held x, validation supplies its code w and the exact decoder gives decode(w,D_1)=ok(x). Used-code coverage places w in E_2, yielding a row (y,w) with y in the target carrier. Its decoder gives decode(w,D_2)=ok(y), while validated lookup at y returns w. Thus T_{E_1,D_2}(x,y) and T_{E_2,D_1}(y,x). Two

translations from the same x have equal lookup codes and then equal decoder results, proving uniqueness.

A successful source run decomposes into b++[t], with t in the admitted source carrier. Registered reconstruction puts e(s) in the target carrier and supplies the reverse translation from e(t) to t. Section 145 then constructs the target run and ledger:

$$|L| = |u|, \qquad \mathrm{run}_T(e(s), \mathrm{flatten}(L)) = \mathrm{ok}(c +\!\!+ [e(t)]), \qquad T_{E_2,D_1}(e(t), t).$$

The same construction in the other direction uses its separately supplied simulation and coverage conditions. The reverse terminal correspondence follows from the validated tables themselves.

> Source, proof and provenance: V3 run, sft/computation/operations.py:51; V3 source derivation, claims/SFT-COMP-FORM-MODEL-EQUIVALENCE-001/WHY_DERIVATION_CHECK.md:1. Native results: SFT.Computation.ModelEncoding.registeredReconstruction, ModelEncoding.lean:55; SFT.Computation.ModelEncoding.registeredExecution, ModelEncoding.lean:75. Timeline: Turn 199, line 2191. Evidence: Completion TURN_199_MODEL_EQUIVALENCE_COMPLETION.

147. Interpretation of frozen generated descriptions

A frozen description retains its complete generated transition system, initial configuration and action input. The original finite encoding and exact decoder recover that held request. A uniform interpretation adapter then applies the unchanged source execution kernel. For every admitted description, native proofs establish exact agreement with its machine execution, recover the complete path and terminal from every successful observation, and preserve the original errors. Failed decoding or absence from the frozen registry supplies no interpretation. For each admitted finite description registry with its exact decoder, the same interpreter handles every held request: it decodes the name, selects the retained machine/input/action triple and returns that triple’s original run. The grammar depth belongs to the supplied description construction; the interpretation theorem imposes no separate fixed depth cutoff.

Resource records retain the original numbered observation trace and occupied support. The initial observation is counted: a successful execution records one more observation than requested actions. Native proofs reconstruct every recorded observation and occupied label. The same original base, successor and composition operations preserve complete traces and terminal results under interpretation; no external machine model or fixed resource overhead is imposed.

All 256 original grammar members and full program outputs match pinned V3. 36 additional fresh native resource comparisons cover guard precedence, repeated observations, Unicode and position labels. These results complete the registered Fold Universal-Computation Law.

A frozen request q stores (machine,initial,actions). Interpretation first decodes code w to name n, then looks up q in the registry. Its successful selection equation is

$$\mathrm{decode}(w, D) = \mathrm{ok}(n), \quad \mathrm{lookup}(G, n) = \mathrm{some}(q) \Longrightarrow I_G(w) = \mathrm{some}(\mathrm{run}_{q.machine}(q.initial, q.actions)).$$

A decoder error or missing registry lookup yields no interpretation. Conversely, case analysis of an observed interpretation recovers the exact decoded name, registry row and unchanged run producing it. For distinct registry names and validated encoding with exact decoder, finite round-trip reconstruction supplies such a code for every held request.

If the observed run succeeds with τ, path reconstruction gives τ=b++[t], P(q.initial,q.actions,b,t), and |b|=|q.actions|. Thus |τ|=|q.actions|+1, including the initial observation. Equality of the deterministic interpretation result gives uniqueness of its observation, including original errors. An empty action request with held initial state returns that state's singleton trace. Successor and serial path equations apply to the selected unchanged run.

For trace τ and separately supplied occupied list O, resource admission requires Distinct(O), τ≠[] and O≠[]. Its exact ledger is ([(transition-i,τᵢ)] starting at i=1, O, |τ|, |O|). Induction on τ proves that projecting the numbered rows recovers τ and that their count is |τ|. The other three fields retain O, |τ| and |O| directly. Substituting the successful-execution length equation gives timeCount=|actions|+1; no equality between occupied support and trace is required.

> Source, proof and provenance: V3 run, sft/computation/operations.py:51; V3 source derivation, claims/SFT-COMP-FORM-UNIVERSALITY-001/WHY_DERIVATION_CHECK.md:1. Native results: SFT.Computation.UniversalInterpretation.observationUnique, UniversalInterpretation.lean:90; SFT.Computation.UniversalInterpretation.identityTrace, UniversalInterpretation.lean:97. Timeline: Turn 200, line 2197. Evidence: Completion TURN_200_UNIVERSALITY_COMPLETION.

148. Recognition, decision and unresolved execution

The generated description ledger retains every admitted description in order under its original numbered position. Its original duplicate and nonempty guards remain exact. Held acceptance and rejection labels complement each other; other labels are excluded by the original complement operation.

A decision retains a complete original execution path terminating at an accepting or rejecting label. An unresolved record retains a finite path whose current endpoint carries neither answer. Both replay through the unchanged execution kernel. An unresolved prefix supplies no answer and does not establish whether a later prefix will terminate. A generated one-state loop admits arbitrary finite unresolved prefixes without a terminating decision.

Total decision applies to every registered input, retaining its initial configuration and exact action input. Native proofs reconstruct its complete terminating observation and establish the successor decomposition of the domain obligation. Terminating decisions exclude continuation at their terminal boundary. These results preserve the source distinction between partial recognition and total decision without importing a conventional machine model or completed infinite trace.

The full 256 original computability grammar and complete native program agree with pinned V3. 13 additional fresh native comparisons cover exact enumeration, guard order, held complements and excluded labels. These results complete the registered Fold Recognition and Decision Law.

Define a decision by a halt certificate together with terminal∈{accept,reject}. An unresolved prefix instead retains P(s,u,b,c), initial membership, c≠accept and c≠reject. Both replay to b++[terminal/current], but either unresolved inequality contradicts the corresponding decision label. The explicit halt boundary excludes a further successful step from a decision.

For a finite input list Q, total decision is the quantified statement that each held (initial,actions) has such a decision certificate. List membership yields the exact successor equation

$$\mathrm{Decides}(q :: Q) \iff (\exists b, t, \mathrm{Decision}(q.initial, q.actions, b, t)) \wedge \mathrm{Decides}(Q).$$

The forward direction selects head and tail memberships; the reverse direction splits membership into those cases. Each certificate reconstructs the original terminating observation.

For the admitted one-state system with edge held --fold→ held, induction on n constructs a path with n fold actions and n held predecessors, ending at held. Since held is neither answer label, every such prefix is unresolved. Any supposed decision ends in the sole carrier state held; its existing fold edge contradicts the halt boundary. Hence these arbitrary finite prefixes do not supply a terminating decision.

> Source, proof and provenance: V3 complete_enumeration, sft/computation/operations.py:104; V3 run, sft/computation/operations.py:51; V3 source derivation, claims/SFT-COMP-CBL-RECOGNITION-DECISION-001/WHY_DERIVATION_CHECK.md:1. Native results: SFT.Computation.RecognitionDomain.complementAccept, RecognitionDomain.lean:48; SFT.Computation.RecognitionDomain.complementBoundary, RecognitionDomain.lean:51;

> SFT.Computation.RecognitionTrace.loopNotTotal, RecognitionTrace.lean:73; SFT.Computation.RecognitionTrace.loopPrefix, RecognitionTrace.lean:78. Timeline: Turn 201, line 2203. Evidence: Completion TURN_201_RECOGNITION_DECISION_COMPLETION.

149. Terminal witnesses and the held halting boundary

A halting witness retains the complete finite execution path and an explicit terminal boundary. Native reconstruction yields the unchanged source observation trace, including its terminal state. The original held complement operation exchanges acceptance and rejection and has no fixed point on those labels. An admitted generated stop process halts on its identity trace; an admitted generated self-loop has no terminating finite path. These processes realize the two complementary terminal behaviors within the original transition kernel.

The contradiction certificate retains the exact decoded self-description, selected registered process and prediction at that same code. A correct prediction and a process realizing its held complement cannot both hold. When total prediction includes a supported registered self-description and its complementary execution, instantiation at that description produces the contradiction. The quantified contradiction requires the displayed decoded self-description and its supported complementary execution. Totality supplies an answer at that same code, and correctness then contradicts the complementary process; these are the exact hypotheses consumed by the native theorem.

The complete 256-candidate grammar and every native program output field and hash agree with pinned V3. These results complete the registered Fold Halting Boundary Law.

Halts(M,s) means that some finite actions, predecessor list and terminal form a halt certificate. Define Correct(accept,M,s) by its halting implication and Correct(reject,M,s) by its nonhalting implication, retaining both clauses in the source predicate. The held complement satisfies

$$c(\mathrm{accept}) = \mathrm{reject}, \qquad c(\mathrm{reject}) = \mathrm{accept}, \qquad c(v) \neq v.$$

If both a prediction v and its complement c(v) are correct for the same process, case analysis on v supplies Halts(M,s) and its negation. The generated stop system has its identity halt because it has no edges. The generated one-state loop cannot halt: carrier membership forces the terminal to held, where the fold self-edge contradicts the required no-outgoing-step condition. They realize the complement of reject and accept respectively.

For the registered contradiction, decode one retained code w to n, look up its selected process M, and obtain predictor(w)=v. Correctness at that same code gives Correct(v,M,held); its supported complement record gives Correct(c(v),M,held). The preceding two-case argument yields contradiction. Total prediction is used only to obtain v at the retained self-description; its explicit support supplies the selected complementary process. The derivation keeps those exact registration and self-support premises.

> Source, proof and provenance: V3 self_negating_boundary, sft/computation/operations.py:95; V3 source derivation, claims/SFT-COMP-CBL-HALTING-001/WHY_DERIVATION_CHECK.md:1. Native results: SFT.Computation.HaltingBoundary.registeredPredictionContradiction, HaltingBoundary.lean:95; SFT.Computation.HaltingBoundary.totalInternalPredictionExcluded, HaltingBoundary.lean:110. Timeline: Turn 202, line 2209. Evidence: Completion TURN_202_HALTING_COMPLETION.

150. Effective finite enumeration and reconstruction

The original enumeration retains every admitted description in order under its canonical numbered position. Native positional reconstruction recovers that exact label and description at every position in the finite support; positions beyond the support supply no row. The original distinct-description guard excludes duplicates, and projection recovers the entire admitted description sequence.

The one-description base has its original first position. Arbitrary finite block extension retains every prior row and offsets the new block by exactly the earlier support length. An admitted single-description successor appends its next canonical row without changing any predecessor. The block-extension induction applies to every finite description list and every finite appended block; the row offset is exactly the retained prefix length.

Each retained description links through the original finite encoding and exact decoder to its complete registered machine request. Interpretation reproduces the original execution with the same initial configuration and action input. Enumeration itself supplies no additional termination or answer claim.

All 256 original candidates and every full native program field and hash agree with pinned V3. These results complete the registered Fold Effective-Enumeration Law.

Let E_k(D) enumerate descriptions starting at integer k. Its clauses are E_k([])=[] and E_k(d::D)=(position-k,d)::E_{k+1}(D). Induction gives the second-coordinate projection D and length |D|. For zero-based index i with D[i]=d, induction on D and i proves

$$E_k(D)[i] = (\text{position-}(k+i), d).$$

The successor case uses (k+1)+i=k+(i+1). If i≥|D|, recursive positional lookup returns none. Validation requires distinct, nonempty D, so the projected admitted descriptions remain duplicate-free.

Induction on a first block L establishes the exact offset equation

$$E_k(L \mathbin{+\!\!+} R) = E_k(L) \mathbin{+\!\!+} E_{k+|L|}(R).$$

The one-description base is [(position-1,d)]. Taking R=[fresh] gives the admitted successor row (position-(1+|L|),fresh), with all prior rows unchanged. For a held indexed description, finite encoding round-trip supplies its supported code; registry lookup and interpretation then return the unchanged machine request's observation. This links each reconstructed position to execution without adding a termination conclusion.

> Source, proof and provenance: V3 complete_enumeration, sft/computation/operations.py:104; V3 source derivation, claims/SFT-COMP-CBL-ENUMERATION-001/WHY_DERIVATION_CHECK.md:1. Native results: SFT.Computation.EnumerationTrace.oneDescription, EnumerationTrace.lean:74; SFT.Computation.EnumerationTrace.recognizerLink, EnumerationTrace.lean:80. Timeline: Turn 203, line 2215. Evidence: Completion TURN_203_ENUMERATION_COMPLETION.

151. Explicit reductions and answer reconstruction

A registered reduction retains its source and target domains, finite input encoder, target procedure and answer decoder. Original validated encoding tables and exact decoders establish total translation on the held source carrier. The target procedure retains the complete original request, execution trace and terminal boundary; decoding its answer uses the original finite lookup and decoder operations.

The registered bidirectional correctness trace links source answers to decoded target answers. Native forward reconstruction yields the full target run and source answer decoding, while reverse reconstruction recovers the source answer from that target result. Encoding alone does not introduce an answer-preservation premise.

Composition retains the intermediate input and both input translations. The final answer decodes through the second reduction and then the first, retaining both intermediate and source answers. Native proofs establish both directions of this correspondence and preserve both generated-domain witnesses. The identity law preserves both the exact input and the decoded answer; finite successor laws retain the original table-validation boundary.

All 256 original candidates and all complete native program fields and hashes agree with pinned V3. These results complete the registered Fold Computability-Reduction Law.

Let I(x,y) and D(b,a) be the supplied input and answer translations, each defined by finite encoding lookup followed by exact decoding. Totality on source S and target T means ∀x∈S, ∃y∈T, I(x,y). The registered answer correspondence is the explicit equivalence

$$A_S(x,a) \iff \exists b,\ A_T(y,b) \wedge D(b,a) \qquad (x \in S,\ I(x,y)).$$

Given A_S(x,a), totality supplies y and the forward implication supplies b. A target procedure answer retains a selected request and halt certificate, so replay gives its full trace before++[b] and normal terminal b. Expanding D(b,a) retains the actual code word, encoding lookup and answer decoder result. Conversely, a target answer and its decoding yield A_S(x,a) by the reverse implication.

For successive reductions with intermediate answer a_1 and final answer a_2, substitute the second correspondence into the first:

$$A_S(x,a) \iff \exists a_2, a_1,\ A_T(z,a_2) \wedge D_2(a_2,a_1) \wedge D_1(a_1,a).$$

Totality supplies and retains both intermediate input y and final input z in their respective domains. The two reverse implications prove the reverse direction in the opposite order. With the same validated table and its exact inverse, translation uniqueness forces output=input; applying this to both input and held answer proves identity preservation. The domain successor law separates the new head input's translated witness from totality on the old tail, by the two cases of list membership.

> Source, proof and provenance: V3 run, sft/computation/operations.py:51; V3 source derivation, claims/SFT-COMP-CBL-REDUCTION-001/WHY_DERIVATION_CHECK.md:1. Native results: SFT.Computation.ReductionTrace.identityMappingExact, ReductionTrace.lean:123; SFT.Computation.ReductionTrace.identityPreserves, ReductionTrace.lean:130. Timeline: Turn 204, line 2221. Evidence: Completion TURN_204_REDUCTION_COMPLETION.

152. Finite relative computability classes

Registered problems and their certified reductions form an original generated finite graph. Its exact reachability operation supplies complete retained paths. Every reduction step on such a path retains its finite encoders, decoders and bidirectional correctness certificate. Native path composition supplies transitive closure without replacing the intermediate interfaces.

Mutual reachability is reflexive on registered problems, symmetric and transitive. A degree class retains exactly the mutually reachable members of the original ordered problem carrier. Each representative belongs to its class; mutually reducible representatives give the same members, and overlapping classes imply mutual reducibility. These statements require no quotient axiom.

Strict one-way reachability is transitive and excludes mutual equivalence. A finite incomparability certificate excludes complete paths in both directions. Strict order and incomparability are independent of the chosen equivalent representatives. The singleton One supplies the base class, and retained nodes and edges preserve every prior mutual reachability witness. Degrees remain finite generated relation classes, with no completed transfinite hierarchy.

All 256 original candidates and every complete native program field and hash agree with pinned V3. These results complete the registered Fold Degrees-of-Computability Law.

Write x↝y for true reachability in the admitted finite problem graph. Define x~y iff x↝y and y↝x, and retain the ordered class [y∈nodes:x~y]. Filter membership proves that this list contains exactly the held mutually reachable nodes. Identity paths give x~x, swapping the two paths gives symmetry, and path composition in both directions gives transitivity. Hence x~x′ transports class membership in both directions. If two classes share z, composing x↝z↝y and y↝z↝x proves x~y.

Strict reachability is x≺y iff both nodes are held, x⇝y and not y⇝x. If x≺y and y≺z, composition gives x⇝z. A reverse z⇝x would compose with y⇝z to contradict not y⇝x. Thus strict reachability is transitive and excludes mutuality. Incomparability requires both reachability tests false; any complete path in either direction would force its test true.

For equivalent replacements x∼x′ and y∼y′, transport is the path composition

$$x' \rightsquigarrow x \rightsquigarrow y \rightsquigarrow y'.$$

Applying the same construction in reverse proves transport of absence by contradiction. Therefore strictness and incomparability preserve equivalent representatives. Retaining graph nodes and edges retains each original path and thus every mutual witness; the singleton graph [One] with no edges has class [One]. Finally, complete reachability reconstruction gives an ordered path whose every adjacent pair is a registered edge. The supplied edge certificate then recovers each reduction's total input map and bidirectional answer map along that path.

> Source, proof and provenance: V3 run, sft/computation/operations.py:51; V3 source derivation, claims/SFT-COMP-CBL-DEGREES-001/WHY_DERIVATION_CHECK.md:1. Native results: SFT.Computation.DegreeClasses.incomparableRepresentative, DegreeClasses.lean:149; SFT.Computation.DegreeClasses.oneDegree, DegreeClasses.lean:158. Timeline: Turn 205, line 2227. Evidence: Completion TURN_205_DEGREES_COMPLETION.

153. Universal-machine execution and recognition

The frozen description grammar retains exact machine requests: the original generated machine, its initial input and every requested action. Native finite encoding and decoding reconstruct each admitted request. Prefix-free description streams have a unique complete parse. Interpretation returns precisely the original machine execution, preserving its complete trace and original errors.

A terminal execution retains its output and excludes further transitions. Conversely, a successful interpreted trace with its explicit terminal boundary reconstructs the original halt witness. Accepted and rejected answers retain their exact labels. A continuing finite trace remains distinct from a terminal decision. The original generated loop admits arbitrary finite continuing prefixes while its registered input is not a total decision domain. The interpretation equation preserves each terminal witness and each continuing prefix separately. In particular, for every finite n the loop has an unresolved n-step prefix, while its retained self-edge excludes a terminal decision.

The identity trace supplies the base. Serial execution retains the intermediate state and concatenated action and trace rows. Retained decoding and registry selections preserve all prior interpreted observations; every newly admitted encoded request is covered by the same finite grammar theorem. All 256 original candidates and complete program fields and hashes agree with pinned V3. These results complete the registered Fold Universal-Machine Execution Law.

For a decoded code and selected request q, exact observation is

$$I(w) = \mathrm{some}(o) \iff \mathrm{run}_{q.machine}(q.initial, q.actions) = o.$$

Both directions follow by expanding interpretation and using the successful decoder and registry lookup. A halt certificate therefore yields I(w)=some(ok(b++[t])), |b|=|actions|, and no successful outgoing step at t. Conversely, a successful interpreted trace reconstructs its path; supplying the stated normal-terminal boundary completes the halt certificate. A decision retains t=accept or t=reject. An unresolved prefix replays through the same equation while its endpoint inequalities exclude a decision. Original run errors likewise remain the identical interpreted errors.

For prefix-free code support, two held codeword lists L,R with concatenate(L)=concatenate(R) satisfy L=R by the complete unique-parse theorem. If an extension preserves decoding at w and every previous successful registry selection, interpretation-origin reconstruction supplies the same request and run; substituting those retained selections preserves the exact observation. The original loop supplies an unresolved prefix at every finite depth and fails total decision on its registered input, as proved in Section 148.

> Source, proof and provenance: V3 run, sft/computation/operations.py:51; V3 source derivation, claims/SFT-COMP-CBL-UNIVERSAL-MACHINE-001/WHY_DERIVATION_CHECK.md:1. Native results: SFT.Computation.UniversalMachineTrace.retainedInterpretation, UniversalMachineTrace.lean:85; SFT.Computation.UniversalMachineTrace.noOracleFromReplay, UniversalMachineTrace.lean:98. Timeline: Turn 206, line 2233. Evidence: Completion TURN_206_UNIVERSAL_MACHINE_COMPLETION.

154. Generated diagonal contradiction

The registered undecidability construction applies on languages supporting exact self-encoding and complementation. The native diagonal constructor applies the original complementary-verdict operation and realizes its result in a generated machine with its complete identity-halt trace. Both held verdicts are supported, and the original failure boundary is retained.

Self-application retains the same generated code, decoded description and selected request used to query the proposed decider. Totality supplies a held answer on that description. The exact diagonal constructor supplies its complement and the corresponding native interpreted execution. Correctness would equate the proposed answer with that observed complementary answer, contradicting the proved absence of a complement fixed point. Thus a total correct internal decider is excluded on the explicitly registered self-encoding/complement domain.

The contradiction retains each decoding, selection, construction and execution witness. Exact retained decoding and registry selections preserve self-support under extension. The identity trace supplies the base; original path composition preserves intermediate interfaces and complete action/trace records. All 256 original candidates and complete native program fields and hashes agree with pinned V3, completing the Fold Undecidable-Problem Law.

Let c(v) be the original complement of a held verdict. The diagonal constructor returns the request

$$q_v = (M_v, c(v), []), \qquad M_v = (\,[c(v)],\,[\mathrm{fold}],\,[]\,).$$

The machine is admitted: its singleton carriers are distinct and nonempty, and its empty transition relation passes validation. Its identity path halts at c(v), since no outgoing row exists, and run(q_v)=ok([c(v)]). Both held verdicts produce this request; an invalid verdict propagates the original complement error.

Self-support retains a single code w decoded to n such that any predictor answer v there selects the corresponding constructed q_v at n. Totality supplies a held v with predictor(w)=ok(v). The native construction and selected registry row give

$$I(w) = \mathrm{some}(\mathrm{ok}([c(v)])).$$

Correctness for this same code requires v=c(v), contradicting the complement's no-fixed-point theorem. This instantiation proves exclusion of a total correct internal decider on the stated self-supported domain. If an extension preserves decoding and the selected request, the same w,n and constructor witness continue to establish self-support; the contradiction retains its exact source interface.

> Source, proof and provenance: V3 self_negating_boundary, sft/computation/operations.py:95; V3 source derivation, claims/SFT-COMP-CBL-UNDECIDABILITY-001/WHY_DERIVATION_CHECK.md:1. Native results: SFT.Computation.DiagonalTrace.nativeDiagonal, DiagonalTrace.lean:26; SFT.Computation.DiagonalTrace.noTotalInternalDecider, DiagonalTrace.lean:96. Timeline: Turn 207, line 2239. Evidence: Completion TURN_207_UNDECIDABILITY_COMPLETION.

155. Computational proof-system boundary

A checked proof record retains the original inference checker, each registered premise and rule, the complete accumulated proposition state, and its original consistency test. Native reconstruction yields every step of the accepted trace. A record containing both a proposition and its denied form fails the consistency condition. Original finite enumeration retains each positional proof name and the selected complete proof-step list.

For the registered self-encoding and complement domain, an answer-correct internal verifier has a specific missing decision: its decoded self-description cannot receive a held answer without reconstructing the native diagonal contradiction. Hence complete internal decision is excluded. The result also applies to answers backed by checked proof records when those records preserve the corresponding execution answers. The conclusion concerns the exact decoded self-code supplied by SelfSupport and the answer-preservation condition on checked records. Instantiating those premises at that code yields the complementary-answer contradiction.

The identity proof preserves a consistent initial state. Complete traces compose at their actual accumulated proposition interface, with exact final-state reconstruction and the retained consistency condition. Original enumeration and self-description extension preserve prior records. All 256 original candidates and full native program fields and hashes agree with pinned V3, completing the Fold Incompleteness-Boundary Law.

For initial proposition list A and proof steps L, the checker accepts the empty trace. At a successor step e it requires every premise in A and the exact tuple (rule identity,premises,conclusion) in the registered rules, then continues from A++[conclusion(e)]. Induction equates these executable checks with the retained AcceptedTrace constructors. A checked record additionally requires consistency of the final accumulated list F(A,L), namely

$$\forall p \in F(A,L), \quad \mathrm{denied}(p) \notin F(A,L).$$

Thus a retained proposition and its denied form directly contradict a checked record. Identity requires precisely consistency of A. Induction on a first accepted trace proves serial acceptance at the actual intermediate list, while induction on its step list gives

$$F(A, L \mathbin{+\!\!+} R) = F(F(A,L), R).$$

The second checked record's consistency therefore proves consistency of the composite final state. Finite positional reconstruction supplies each proof's exact numbered name; successful registry lookup supplies its original step list.

For an answer-correct predictor on the retained self-supported domain, choose the specific decoded self-code from SelfSupport. Any held answer there constructs and selects its complementary request, contradicting correctness by Section 154. Hence that code admits no held answer, and totality would contradict this explicit missing decision. If every returned answer is backed by a checked record and such records preserve the corresponding execution answer, these two premises establish the same correctness predicate; the identical missing-code argument excludes proof-backed totality.

> Source, proof and provenance: V3 self_negating_boundary, sft/computation/operations.py:95; V3 source derivation, claims/SFT-COMP-CBL-INCOMPLETENESS-001/WHY_DERIVATION_CHECK.md:1. Native results: SFT.Computation.ProofBoundary.checkedComposition, ProofBoundary.lean:64; SFT.Computation.ProofBoundary.noProofBackedTotality, ProofBoundary.lean:100. Timeline: Turn 208, line 2245. Evidence: Completion TURN_208_INCOMPLETENESS_COMPLETION.

156. Declared relative answer interfaces

An oracle interface retains its named query alphabet and declared query/answer relation. Each answer record retains the selected source identity and exact query and answer. Native provenance reconstruction proves that every record belongs to the selected declared relation; undeclared source identities and answer pairs are excluded. The relation is retained without imposing a new single-valuedness condition.

Finite ordered ledgers retain nested queries and subsequent records. Validity at a nested call is equivalent to validity of its named outer record, nested ledger and continuation. The base computation retains its original native execution path and held initial state alongside the complete external ledger. Replay reproduces that exact base execution and every declared external source; terminal recognition preserves the original accept/reject classification.

The identity path and empty external ledger supply the base. Serial composition preserves the intermediate state, concatenated actions and complete ordered answer record. Registry extensions retaining earlier interface selections preserve all prior nested certificates. These results retain the registered oracle boundary without unexplained computational power. All 256 original candidates and complete native program fields and hashes agree with pinned V3, completing the Fold Relative and Oracle-Computation Law.

A record r is registered exactly when lookup of its source identity selects interface I, its query lies in I.queries, and (query,answer) lies in I.relation. The ordered ledger clauses and flattened records are

$$\mathrm{records}(\mathrm{done}) = [], \qquad \mathrm{records}(\mathrm{call}(r, N, R)) = r :: (\mathrm{records}(N) +\!+ \mathrm{records}(R)).$$

Validity of a call is equivalent to registration of r and validity of both N and R. Induction splits record membership into head, nested ledger or continuation, proving registration of every emitted record. An absent identity contradicts its selected lookup; an absent answer pair contradicts membership in that uniquely selected interface.

Ledger append preserves the first record and its nesting while appending through its continuation. Induction gives records(append(L,K))=records(L)++records(K) and preserves validity. A relative trace is Valid(L) ∧ s∈S ∧ P(s,u,b,t). Native replay returns b++[t], and the membership induction supplies every external record's declared source. The identity uses the empty ledger and identity path. Serial composition combines ledger append with path composition, retaining actions u++v and predecessors b++c. Registry extensions that preserve successful interface selections preserve validity by induction at every nested call. Terminal decision labels transfer unchanged from the original decision certificate.

> Source, proof and provenance: V3 run, sft/computation/operations.py:51; V3 source derivation, claims/SFT-COMP-CBL-RELATIVE-ORACLE-001/WHY_DERIVATION_CHECK.md:1. Native results: SFT.Computation.OracleTrace.replay, OracleTrace.lean:118; SFT.Computation.OracleTrace.relativeComposition, OracleTrace.lean:138. Timeline: Turn 209, line 2251. Evidence: Completion TURN_209_RELATIVE_ORACLE_COMPLETION.

157. Hypercomputation admissibility boundary

A claimed computation retains its generated operational transition system, complete native execution, literal resource record and declared external-answer ledger. Native reconstruction recovers the entire transition path, occupied support, observation count and provenance of every external answer. A differing observed output, failed execution or undeclared oracle source excludes the submitted record.

The claimed input domain is generated at arbitrary finite depth. An explicit certificate retains the base and successor obligations; native induction establishes verification across that domain. A refuted generated record excludes such a certificate. A verified sample establishes V(r) for each of its members. The depth certificate establishes V(r) for every member of every generated D_n by induction. If a refuted record

belongs to some D_n, instantiating either universal claim shows that it is absent from the verified sample and rules out complete-domain verification. The original self-encoding/complement boundary remains applicable to internal total prediction.

These are admissibility conditions, preserving the source requirement for an executable law and depth-independent certificate. They introduce no unexplained computational power. Retained domain extensions preserve prior records. All 256 original candidates and complete native program fields and hashes agree with pinned V3, completing the Fold Hypercomputation Admissibility-Limit Law.

For a disclosed record r, verification is the conjunction of admitted machine construction, run(request)=ok(observed), successful resourceLedger(observed,occupied), and validity of its declared external ledger. Expanding these four conjuncts reconstructs

$$observed = b +\!\!+ [t], \quad P(initial, actions, b, t), \quad \pi_2(timeTrace) = observed, \quad spaceSupport = occupied, \quad timeCount = |actions| + 1.$$

Every external answer is registered by the ledger induction. A different successful actual output contradicts equality of the two successful run results. An execution error contradicts the success constructor. An undeclared oracle identity contradicts external validity.

For a generated family D_n, completeness means ∀n,∀r∈D_n, Verified(r). The explicit depth certificate supplies the base and successor implications

$$(\forall r \in D_0,\ V(r)), \qquad \forall n,\ (\forall r \in D_n,\ V(r)) \Longrightarrow (\forall r \in D_{n+1},\ V(r)).$$

Induction on n proves completeness. A held r∈D_n with ¬V(r) therefore contradicts either completeness or such a certificate. If every member of a finite sample passes, that refuted r cannot belong to the sample, by direct instantiation. If D_n⊆E_n for each n, completeness of E restricts to completeness of D by membership inclusion. These equations preserve the exact scope of the finite samples, declared domains and supplied induction certificates.

> Source, proof and provenance: V3 resource_ledger, sft/computation/operations.py:111; V3 run, sft/computation/operations.py:51; V3 source derivation, claims/SFT-COMP-CBL-HYPERCOMPUTATION-LIMIT-001/WHY_DERIVATION_CHECK.md:1. Native results: SFT.Computation.ComputationalLimit.fullDisclosure, ComputationalLimit.lean:20; SFT.Computation.ComputationalLimit.arbitraryDepth, ComputationalLimit.lean:70; SFT.Computation.ComputationalLimit.finiteEvidenceBoundary, ComputationalLimit.lean:83. Timeline: Turn 210, line 2257. Evidence: Completion TURN_210_HYPERCOMPUTATION_LIMIT_COMPLETION.

158. Exact generated input extent

The canonical encoding retains an exact codeword and decoder reconstruction for every held source input. Complete position cells retain occupied-position identities together with every encoded symbol. Their full symbol projection reconstructs the codeword. The original resource ledger retains its unique, nonempty occupied-support boundary; native reconstruction gives the full trace and exact position count.

Two complete records of the same codeword have equal size. A representation correspondence retains both actual encoding-table rows and both exact codeword lengths; unrelated representations are not silently assigned one invariant size. Every instance in a certified family retains its actual input and encoding record.

The singleton position supplies the base. Complete concatenated encodings have the exact sum of their recorded extents, and an admitted one-position extension increases the count by one. All 256 original Complexity candidates, the three original resource witnesses and complete native program fields and hashes agree with pinned V3, completing the Fold Input and Problem-Size Law.

An exact extent for codeword w consists of cells C with π₂(C)=w and resourceLedger(w,π₁(C))=ok(L). Map-length induction gives |π₁(C)|=|C|=|π₂(C)|=|w|. Substituting the original ledger reconstruction yields

$$\pi_2(L.timeTrace) = w, \quad L.spaceSupport = \pi_1(C), \quad L.timeCount = L.spaceCount = |w|.$$

The ledger guard supplies distinct occupied positions and nonempty support. Two exact records for the same w therefore have equal time and space counts. For two different encodings of an input x, lookup membership retains the actual rows (x,w₁) and (x,w₂); the respective extent equations give |w₁| and |w₂| without identifying them. A certified family supplies its held input's row by instantiating the family witness.

The singleton cell (position-1,held) gives both counts equal to one. For an admitted complete record of w₁++w₂, list-length induction gives

$$size(w_1 \mathbin{++} w_2) = |w_1| + |w_2| = size(w_1) + size(w_2).$$

Taking w₂ to be a singleton proves the admitted successor count increases by one. Exact finite encoding and decoder round-trip retain the original input reconstructed from each codeword.

> Source, proof and provenance: V3 resource_ledger, sft/computation/operations.py:111; V3 source derivation, claims/SFT-COMP-CPLX-INPUT-SIZE-001/WHY_DERIVATION_CHECK.md:1. Native results: SFT.Computation.InputExtent.completeExtent, InputExtent.lean:17; SFT.Computation.InputExtent.representationLedger, InputExtent.lean:46; SFT.Computation.InputExtent.compositionSize, InputExtent.lean:72; SFT.Computation.InputExtent.successorSize, InputExtent.lean:82. Timeline: Turn 211, line 2264. Evidence: Completion TURN_211_INPUT_SIZE_COMPLETION.

159. Time and retained-state resources

The original resource ledger reconstructs the complete ordered observation trace and exact occupied support. Its time count includes the initial observation. A terminal resource record retains the full halt trace, its exact action-plus-initial count and the exclusion of further transitions. These are exact host reports over generated traces.

Every finite nonempty sequence of configuration ledgers has an attained peak space-count class. Native reconstruction connects each measured bound to its actual occupied support. Equal complete time records recover equal histories; the original generated loop exhibits different histories ending at the same label, demonstrating why the terminal label alone does not suffice for recovery.

Ordered ledger composition retains shifted transition labels. An admitted observation/support successor preserves prior rows and adds one to each count. Original serial path composition counts the shared intermediate observation once. All 256 original candidates, resource witnesses and complete native program fields and hashes agree with pinned V3, completing the Fold Time and Space Resource Law.

An admitted ledger satisfies $\pi_2(timeTrace) = \tau$, $spaceSupport = O$, $timeCount = |\tau|$ and $spaceCount = |O|$. Therefore equal complete timeTrace fields imply equal histories by applying π₂. For a halt trace $\tau = b +\!+ [t]$, path length gives $timeCount = |actions| + 1$ and the halt certificate excludes every successful outgoing step at t.

An attained peak in a nonempty ledger list is constructed inductively. The singleton is its own peak. Given a peak p of the tail, compare the head's space count with p's. Select p when the head count is smaller or equal; otherwise select the head. Transitivity of the order relation proves the selected member bounds the whole list. Substitution of $spaceCount = |O|$ ties every bound to its actual occupied support.

Numbered observation rows satisfy, by induction on L,

$$timeRows_k(L \mathbin{++} R) = timeRows_k(L) \mathbin{++} timeRows_{k+|L|}(R).$$

An admitted extension $\tau +\!+ [next]$, $O +\!+ [fresh]$ consequently appends (transition-(1+|τ|),next) and increases both counts by one. For composed paths with action lists u,v, the retained trace is $(b +\!+ c) +\!+ [t]$, giving $timeCount = |u| + |v| + 1$. The shared interface is counted once. The concrete loop has

$run(held, []) = ok([held])$ and $run(held, [fold]) = ok([held, held])$; the histories differ despite their common terminal label.

> Source, proof and provenance: V3 TIME-SPACE; V3 resource_ledger; SFT.Computation.ExecutionResources.historyReconstructionUnique; SFT.Computation.ExecutionResources.peakSupport; SFT.Computation.ExecutionResources.serialObservationCount; Timeline: Turn 212, line 2269.

160. Circuit structural resources

The declared dependency order is exhausted by the original ordered-selection generator. Filtering those selections with the original graph-path check retains exactly the admissible ordered causal traces. Native maximum selection returns an actual longest trace whenever this domain is nonempty, and its length bounds every admitted trace in that dependency order. Each selected gate and adjacent wire is reconstructed from the registered graph.

Original graph admission preserves unique gate and wire identities and held endpoints. Gate size counts the canonical gate carrier, so changed wire incidence does not duplicate a shared component. Original circuit evaluation retains complete local traces and interfaces and establishes output support. The singleton gate supplies the base. Carrier extension, wire composition and dependency-order extension preserve their exact records and counts.

The resource theorem quantifies over every generated graph and every finite declared order. Its domain is exactly the ordered choices that pass the graph-path check. All 256 original candidates, resource witnesses and complete native program fields and hashes agree with pinned V3, completing the Fold Circuit Size and Depth Law.

For declared dependency order O, the complete causal domain is $C(G, O) = filter(pathIsValid(G), rawSelections(O))$. Ordered-selection completeness and filter membership yield

$$p \in C(G, O) \iff HeldChoice(O, p) \wedge pathIsValid(G, p) = true.$$

The longest-trace fold starts from an actual first member and compares it with each remaining path, choosing the new path when its length is at least the current length. Induction proves the selected path remains in the input list and bounds the length of every member. Thus any valid ordered witness makes the domain nonempty and yields an actual longest valid ordered path. Path validity reconstructs every selected gate in G.nodes and every adjacent wire in G.edges.

Gate count is |G.nodes| and wire count is |G.edges|. Original graph admission gives distinct carriers and held edge endpoints. Consequently changing the wire list alone leaves gate count unchanged; appending one gate raises its count by one, and concatenating wire lists adds their lengths. An old ordered path remains a held choice after a new position is appended, using the absent-choice constructor, so the old causal trace remains present. The one-node graph [held] with no wires has longest trace [held]. Evaluation and output-interface preservation use the exact composition derivation of Section 142.

Given any witness in this domain, `completeLongestTrace` proves nonemptiness of the filtered enumeration, then applies the attained-maximum induction to obtain a member bounding every admitted path. Appending a declared position preserves each earlier path: its ordered-choice proof takes the absent-position constructor and its graph-validity proof is unchanged. Thus the length bound is exact for the supplied order, and earlier admissible traces persist under order extension. completeLongestTrace; retainedOrderedTrace.

> Source, proof and provenance: V3 CIRCUIT-RESOURCE; V3 resource_ledger; SFT.Computation.CircuitResources.causalTraceMeaning; SFT.Computation.CircuitResources.completeLongestTrace; Timeline: Turn 213, line 2275.

161. Communication and query resources

Each query and answer record retains its registered interface identity. The ordered sent and received projections have the same exact cardinality as the source query ledger. Pairing those projections reconstructs every original query/answer record, and native proofs recover its declared alphabet and answer-relation membership. Serial composition retains both transcripts and adds their exact query counts; the singleton and successor constructions preserve every prior record.

The result certificate binds the decoded protocol identity to its registered executable request, its declared distributed input, the original protocol execution, the named oracle ledger and the local resource support. Its native proof reconstructs the actual transition path and complete terminal observation trace, with the exact action count and unchanged local knowledge. Unknown protocol or oracle identities are excluded by the original registry lookup. The declared oracle relation is retained without imposing single-valuedness or introducing an answer-producing premise.

All 256 original candidates, three resource-ledger witnesses and all native program fields and hashes agree with pinned V3, completing the Fold Communication and Query Complexity Law.

For flattened answer records R, define $sent(R) = [(identity, query)]$ and $received(R) = [(identity, answer)]$ in the same order. Map-length induction gives both lengths equal to |R|. Pairing corresponding projections and restoring the record proves by induction

$$restore(zip(sent(R), received(R))) = R.$$

The head reconstructs its original identity, query and answer; the tail uses the induction hypothesis. Valid-ledger provenance therefore recovers the selected declared interface for every restored record. Ledger append flattens to concatenation, so both projections concatenate and $queryCount(L + +K) = queryCount(L) + queryCount(K)$. A singleton has one record; appending it preserves all previous records and adds one query.

The protocol certificate retains $decode(code) = ok(name)$, $registry(name) = request$, $distributedInputs(participant) = request.initial$, the successful request run, a valid oracle ledger and an admitted local resource ledger. Expanding interpretation reproduces the protocol observation. Lookup membership retains both the protocol row and distributed-input row. Execution reconstruction gives $\tau = b + +[t]$, the complete path and $|b| = |actions|$; resource reconstruction returns τ and the unchanged local-knowledge list. An unknown decoded protocol name contradicts the selected registry lookup, while an unknown oracle identity contradicts ledger validity. No uniqueness of answers is required beyond the source relation.

> Source, proof and provenance: V3 COMMUNICATION-QUERY; V3 resource_ledger; SFT.Computation.CommunicationResources.completeQueryReconstruction; SFT.Computation.CommunicationResources.resultCertificate; Timeline: Turn 214, line 2281.

162. Scheduled unresolved branch resources

Every registered branch retains its label and complete deterministic execution request. Native schedule evaluation applies the original transition process to every branch, preserving order, multiplicity, successful traces and failure records. Successful runs reconstruct their actual paths and terminal observations. The terminal-output classes cover the complete declared branch support and are pairwise disjoint; failures remain explicitly classified.

A registered schedule description retains its source identity and exact encoding/decoding relation. Its complete encoded-position ledger supplies the exact description cost. Ordered composition and successor extension preserve every earlier execution record. The external-source alternative retains the original sealed measurement boundary, including complete rows and identified source; randomness is not introduced as a causal premise.

All 256 original candidates, resource witnesses and complete program fields and hashes agree with pinned V3, completing the Fold Randomness-Resource Law.

For branch $b = (label, request)$, $execute(b) = (label, run(request))$; the schedule result is its ordered map. Induction gives its exact length and label projection, while map membership gives every result an original branch witness. Successful runs reconstruct a path and terminal; failures retain their original error.

Let outcome(b) be failed(error) or returned(endLabel(trace)). The class for o is the source schedule filtered by $outcome(b) = o$, so

$$b \in class(o) \iff b \in schedule \wedge outcome(b) = o.$$

Each branch belongs to its own outcome class. Membership in two classes implies equality of their outcomes by transitivity, proving disjointness for distinct outcomes. A successful trace $b + +[t]$ has $endLabel = some(t)$, by induction on b. The classification keeps source order and multiplicity.

Map distribution yields $executions(L + +K) = executions(L) + +executions(K)$; the singleton and successor equations follow directly. Validated finite encoding supplies each registered schedule name's supported code and exact decoding. An exact position-cell record for that code w gives both description counts |w| and retains the actual registry and encoding rows. For the external-source alternative, acceptance is equivalent to $preSeal = false$, $mutation = false$, $allRows = true$ and $identified = true$, preserving the original sealed measurement conditions.

> Source, proof and provenance: V3 RANDOMNESS; V3 resource_ledger; SFT.Computation.RandomnessResources.outputClassesCover; SFT.Computation.RandomnessResources.outputClassesDisjoint; Timeline: Turn 215, line 2288.

163. Information retained for inverse execution

For each registered action and target configuration, the complete transition fibre contains exactly the held source configurations whose original transition reaches that target. Distinct merged predecessors cannot be represented by a unique predecessor record. Erasing their distinguishing labels prevents a single recovery function from recovering both.

Induction on the original execution path constructs a retained inverse ledger with one source/action/target row per transition. Every row remains an actual forward transition. Native inverse traversal checks the shared configurations and original transitions and reconstructs the initial configuration. Identity, serial composition and successor extension preserve all rows. The original resource ledger reconstructs the full inverse observation history, with one initial observation plus one per retained transition and unchanged occupied support.

This proves the retained inverse and its exact row count for every successful finite execution of the supplied transition system. All 256 original candidates, resource witnesses and complete native program fields and hashes agree with pinned V3, completing the Fold Reversibility-Cost Law.

For action a and target t, the original transition fibre is $F(a,t) = [s \in states : step(s,a) = ok(t)]$. Filter membership proves this equality of conditions exactly. If distinct x,y belong to the fibre, its predecessor record cannot be unique. A recovery function with $recover(t) = x$ and $recover(t) = y$ would give $x = y$ by transitivity, contradicting their distinction.

A retained inverse starts at the forward terminal and stores rows (source,action,target) in reverse traversal order. Its identity has no rows. Its successor checks the actual forward step and moves from the row's target to its source. Induction on a forward path constructs

$$P(s,u,b,t) \Longrightarrow \exists R,\ Inverse(t,R,s) \wedge |R| = |u|.$$

The forward successor case appends its first edge after the inverse of the remaining path. Inverse serial composition justifies this order. The undo operation checks $current = row.target$ and that the original source step reaches row.target, then recurses from row.source. Induction on Inverse therefore proves $undo(t, R) = ok(s)$, and membership induction proves every retained row is an actual forward transition. Its resource trace is t::map(source,R), so map-length preservation gives $timeCount = |R| + 1$ while the original ledger retains occupied support unchanged.

The quantifiers range over an arbitrary system, initial state and action list. The empty path gives an empty inverse. For a successor, first reverse the remaining path, then append its original head transition as a predecessor row. Consequently an execution with action list A constructs an executable inverse with exactly $|A|$ rows, independently of the lengths used in the source-comparison census. everyForwardPathHasRetainedInverse; nativeInverse.

> Source, proof and provenance: V3 REVERSIBILITY-COST; V3 resource_ledger; SFT.Computation.ReversibilityResources.everyForwardPathHasRetainedInverse; SFT.Computation.ReversibilityResources.inverseExecutes; Timeline: Turn 216, line 2295.

164. Independent and causally layered work

The native ready-set construction retains exactly the declared tasks not yet completed whose incoming predecessors are all completed. This condition proves that simultaneous ready tasks have no dependency edge between them. The original ordered-selection generator exhausts every layer choice over that ready set. Every synchronization row is an original incoming graph edge, and its predecessor belongs to the completed interface.

The layer schedule reconstructs the entire ordered work trace, preserving every task and synchronization interface under serial composition and successor extension. Full source coverage is transferred to the work trace without imposing the graph-carrier order on execution. The complete original causal-path enumeration and native maximum produce an attained span bounding every admissible causal trace in the declared execution order. Original resource ledgers retain exact work and support counts; no physical processor count or timing constant is introduced.

All 256 original candidates, resource witnesses and complete native program fields and hashes agree with pinned V3, completing the Fold Parallel-Complexity Law.

Readiness is the exact predicate

$$Ready(D, x) \iff x \in nodes \wedge x \notin D \wedge \forall p,\ (p, x) \in edges \Longrightarrow p \in D.$$

Boolean conjunction and edge-list membership establish equivalence with the executable ready test; filtering the node carrier therefore gives exactly Ready. If x and y are ready and (x,y) is an edge, readiness of y forces $x \in D$, contradicting readiness of x. Ordered selections of this complete ready set enumerate every declared layer choice. Synchronization filtering retains exactly original edges whose target lies in the layer, so each predecessor belongs to D.

A layer schedule has identity result D and successor state $D + +layer$. Induction gives $result = D + +flatten(layers)$. The same induction proves serial schedule composition, and list concatenation gives $flatten(L + +K) = flatten(L) + +flatten(K)$. A new admitted layer appends its complete work without changing previous rows. Starting from $D = []$, declared result coverage transfers directly to the flattened work trace, without requiring execution order to equal node-carrier order. The complete ordered causal-path maximum from Section 160, applied to this work order, supplies an attained span bounding every admitted path. Resource-ledger projection retains the complete work trace and its supplied occupied support with their exact counts.

> Source, proof and provenance: V3 PARALLEL; V3 resource_ledger; SFT.Computation.ParallelResources.readyIndependence; SFT.Computation.ParallelResources.fullSpan; Timeline: Turn 217, line 2302.

165. Finite resource comparison certificates

Constructive upper certificates retain the registered executable request, its complete native transition trace and original resource ledger. The resource measure selects the original observation count or occupied-support count. Native finite induction constructs an attained maximum over the complete declared family, retaining an actual verified member and proving that its cost bounds every member.

An adversarial lower witness retains two problem-family inputs with identical registered observations and different required answers. No response depending on that observation can correctly answer both. A lower certificate supplies such a witness for every below-bound member of its declared comparison family, excluding every such member from correctness. Problem-family extension and comparison-family composition retain the complete witnesses. Arbitrary asymptotic lower bounds remain separate claims with their own generated families.

All 256 original candidates, resource witnesses and complete native program fields and hashes agree with pinned V3, completing the Fold Lower- and Upper-Bound Law.

A verified resource record retains its registry lookup, successful native run and admitted resource ledger. These reconstruct the actual path and give $measured(time) = |observed|$ and $measured(space) = |occupied|$. Over a nonempty finite family, induction compares the head cost with an attained maximum of the tail, selects an actual member, and proves every cost is bounded by the selected cost. Instantiating verification at that member yields a verified upper witness.

For a lower witness, let x,y be held inputs with $observe(x) = observe(y)$ but $required(x) \neq required(y)$. Any correct response based on that observation would satisfy

$$required(x) = respond(observe(x)) = respond(observe(y)) = required(y),$$

a contradiction. A lower certificate supplies such a pair for each declared model whose measured cost is below B. Instantiation at any allegedly correct below-bound model excludes it, proving correctness implies not(cost<B) within that comparison family. Appending inputs preserves both held witnesses; composing comparison families splits model membership into the original two cases and retains the corresponding witness. The singleton upper bound is reflexive.

> Source, proof and provenance: V3 BOUNDS; V3 resource_ledger; SFT.Computation.BoundResources.verifiedUpper; SFT.Computation.BoundResources.lowerCertificate; Timeline: Turn 218, line 2309.

166. Resource-preserving reductions and completeness

A verified reduction retains its complete source domain, total encoded input translation and bidirectional answer decoder. Every source input has a registered executable overhead record satisfying the specified resource law. Native reconstruction recovers its original observation history, occupied support and exact counts. Across stages, every trace is retained and the sum of verified observation costs equals the length of the full concatenated trace.

Reduction chains preserve each original mapping, intermediate description and overhead witness under the same resource law. Hardness supplies a retained chain from every member of the supplied problem inventory; completeness additionally requires membership of the target. Composition transports hardness while retaining the full chain. Inventory extension requires the new certificate and preserves all prior certificates; the identity chain supplies the singleton base. The problem inventory and resource law are parameters of every reduction-chain judgment.

All 256 original candidates, resource witnesses and complete native program fields and hashes agree with pinned V3, completing the Fold Reduction and Completeness Law.

VerifiedReduction conjoins total input translation, bidirectional answer preservation, and—for every held source input x—a selected verified overhead record r satisfying the specified resource law. Instantiation supplies its target input, actual overhead lookup and complete answer equivalence. Resource reconstruction gives the overhead row's observation trace, occupied support and their exact counts.

For stage records R, define $trace([]) = []$, $trace(r :: R) = r.observed ++ trace(R)$, $cost([]) = 0$ and $cost(r :: R) = r.timeCount + cost(R)$. Induction substitutes each verified $r.timeCount = |r.observed|$ and obtains

$$cost(R) = |trace(R)|, \quad trace(L ++ R) = trace(L) ++ trace(R), \quad cost(L ++ R) = cost(L) + cost(R).$$

A reduction chain is identity or a verified first reduction followed by another chain. Induction on the first chain composes it with a second while retaining every map, intermediate problem and overhead law. Hard(I,t) means every $p \in I$ has such a chain to t; Complete(I,t) additionally requires $t \in I$. Composing each supplied chain transports hardness to a subsequent target. Membership in fresh::I splits the obligation into Chain(fresh,t) and Hard(I,t). Identity therefore supplies completeness for the singleton inventory.

For any registry $\mathcal{R}$, resource predicate L, problem inventory I and target t, the precise completeness judgment is

$$\mathrm{Complete}_{\mathcal{R},L}(I,t) \iff t \in I \wedge \forall p \in I,\ \mathrm{Chain}_{\mathcal{R},L}(p,t).$$

A chain is generated by identity and by a verified reduction followed by a chain. Induction on the first chain proves composition. Enlarging the inventory has the exact obligation

$$\mathrm{Hard}_{\mathcal{R},L}(p :: I,t) \iff \mathrm{Chain}_{\mathcal{R},L}(p,t) \wedge \mathrm{Hard}_{\mathcal{R},L}(I,t).$$

Hence arbitrary finite inventories are supported, while each added problem contributes its own retained reduction chain under the same resource predicate. chainComposition; completenessMeaning; inventorySuccessor.

> Source, proof and provenance: V3 REDUCTION-COMPLETENESS; V3 resource_ledger; SFT.Computation.CompletenessResources.exactStageCost; SFT.Computation.CompletenessResources.chainComposition; Timeline: Turn 219, line 2316.

167. Complete instance-family resource profiles

Each generated support cell retains its instance assignment and original resource record. Canonical-support validation, exact cell coverage and source lookups preserve distribution provenance and the explicit multiplicity of repeated instances. The exact aggregate sums every cell cost. Original positive-fraction normalization divides that sum by the full support-cell count, and native proofs establish exact value preservation without floating arithmetic or fitted weights. Original event weighting retains the exact support parts and its admission boundary.

The greatest-resource class contains every tied maximum. Native finite proofs construct a verified greatest member, bound every record and bound the aggregate by the complete support count times that resource value. Composition and successor extension preserve all costs and cells. The empirical-distribution alternative retains its original complete-row, identified-source and sealed measurement obligations.

All 256 original candidates, resource witnesses and complete native program fields and hashes agree with pinned V3, completing the Fold Average- and Worst-Case Law.

Complete support cells satisfy $map(identity, cells) = support$ and retain successful lookups from each cell identity to its instance name and from that name to its resource record. Lookup membership reconstructs both provenance rows. Map-length preservation gives $|map(record, cells)| = |support|$, including repeated instances represented by different cells.

Let m(r) select time or space count. Define $C([]) = 0$ and $C(r :: R) = m(r) + C(R)$. A verified record has positive m because the original ledger requires nonempty observation and occupied support. Thus a nonempty verified family has positive total. Normalizing C(R)/|R| to positive fraction p/q preserves the exact equation

$$p\,|R| = C(R)\,q.$$

For complete cells replace |R| by |support| using the count identity. The original exact-event weighting supplies support parts and propagates its original admission errors.

For an attained peak k, the greatest class is $[r \in R : m(r) = m(k)]$. Filter membership proves this condition. If r bounds every member, compare it with the peak; the peak also bounds r, so antisymmetry gives equal costs. Hence this class retains all and only tied maxima. Induction gives $C(L + +R) = C(L) + C(R)$ and, when every $m(r) \leq B$, $C(R) \leq |R|B$. Taking a singleton right list gives the exact successor aggregate. The empirical alternative retains $preSeal = false$, $mutation = false$, $allRows = true$ and $identified = true$ from the original measurement predicate.

> Source, proof and provenance: V3 AVERAGE-WORST; V3 resource_ledger; SFT.Computation.ProfileResources.attainedGreatestClass; SFT.Computation.ProfileResources.completeProfileAverage; Timeline: Turn 220, line 2323.

168. Exact approximation and retained objective distinction

The approximation construction retains the registered optimization carrier, preference relation and positive generated objective-cell traces. Native proofs derive the optimum bound from absence of a better incoming preference row and preserve equality of the objective values of tied optima. The exact approximation ledger uses the original ratio of optimum cells to candidate cells. Positive-fraction normalization preserves its value by exact cross multiplication.

The output certificate retains the original computation trace, time ledger and occupied support. Missing objective rows and invalid lower optima preserve the source failure boundary. Carrier extension preserves a retained optimum under the original absence of a better preference row, and the singleton construction supplies its base. No unregistered metric or irrational ratio is admitted. All 256 original candidates, resource witnesses, controls and native program fields and hashes agree with pinned V3, completing the Fold Approximation-Complexity Law.

Complete objective data supplies each feasible candidate x with a held positive cell list C_x. The registered objective order is $(x, y) \in preferred$ iff $|C_x| < |C_y|$. A retained optimum o has no incoming better preference row. If $|C_x| < |C_o|$, the order equivalence supplies precisely the excluded row (x,o). Therefore $|C_o| \leq |C_x|$. Applying this bound in both directions to two optima proves their objective lengths equal.

Put $b = |C_o|$ and $c = |C_x|$. Completeness supplies b>0 and the optimum argument gives $b \leq c$. The source approximation operation returns the normalized exact part p/q for b/c and proves

$$bq = pc.$$

A missing objective lookup contradicts completeness at that held candidate. With b>0 but not $b \leq c$, the original approximation operation returns its invalid-lower-optimum error. A verified computation ending in x retains $run(request) = ok(before + +[x])$, the full numbered observation projection and unchanged occupied support. Appending feasible candidates retains o whenever the original no-better-row condition still holds. The singleton with empty preference relation is its own optimum.

> Source, proof and provenance: V3 APPROXIMATION; V3 resource_ledger; SFT.Computation.ApproximationResources.optimumBound; SFT.Computation.ApproximationResources.exactApproximationCertificate; Timeline: Turn 221, line 2330.

169. Resource stratification by structural coordinates

A named structural coordinate is extracted from the retained fields of each declared input. Native proofs establish both source-row provenance and uniqueness of the extracted coordinate. Total extraction yields a complete family partition: every held input belongs to its extracted coordinate class, and classes with different coordinates cannot share an input. Missing coordinates fail the extraction condition. The construction introduces no fitted constant.

Every family member retains its original verified resource record, complete time trace, occupied support and exact counts. An answer-preserving reduction that preserves the extracted coordinate maps a source family into the corresponding target family while retaining its input encoding, answer decoding and verified resource overhead. Two-stage coordinate preservation and full-family composition, singleton and successor proofs retain all source rows. All 256 original candidates and complete native program records and hashes match pinned V3, completing the Fold Parameterized-Complexity Law.

For retained forms F, extraction first looks up input x, then the named coordinate k in its field list:

$$extract(F, k, x) = lookup(lookup(F, x), k).$$

A successful extraction yields both actual rows (x,fields) and (k,coordinate). Two successes on the same lookup have equal coordinates. Define $Family(c) = [x \in inputs : extract(F, k, x) = some(c)]$. Filter membership gives this exact condition; total extraction places every held input in its obtained class, while uniqueness makes classes with distinct coordinates disjoint. A missing extraction at a held input contradicts totality.

Filter distribution gives $\mathrm{Family}_{L++R}(c) = \mathrm{Family}_L(c) ++ \mathrm{Family}_R(c)$. The singleton and admitted successor follow by substituting the extracted coordinate. Complete resource lookup for a family member retains its actual resource row and the observation/support projections and counts.

A verified reduction supplies target y, its encoded input map, answer equivalence and verified overhead record. Coordinate preservation gives $extract(target, k, y) = extract(source, k, x)$; substitution of the source class equation places y in the corresponding target class. For two stages, transitivity through the extracted intermediate coordinate proves preservation across the composition, retaining both actual mappings.

> Source, proof and provenance: V3 PARAMETERIZED; V3 resource_ledger; SFT.Computation.ParameterizedResources.partitionCoverage; SFT.Computation.ParameterizedResources.partitionDisjoint; Timeline: Turn 222, line 2337.

170. Shortest exact descriptions at declared finite depth

The supplied alphabet generates every nonempty description through the declared depth. Filtering by the original decoding relation retains exactly those descriptions that reconstruct the target object. A native minimum construction returns a census member whose length bounds every other member. The minimal-description class retains every tied minimum. Each member decodes to the exact target and, through the registered interpreter interface, reconstructs the original computation request and its execution.

Translation retains its source encoding row and target decoder result. An actual translated description with an explicit overhead trace bounds the target minimum; concatenation preserves exact length accounting. Singleton and depth-extension proofs retain the complete finite construction. The depth is an arbitrary natural-number input. Its exact membership condition is positive word length at most that depth together

with successful decoding to the named object; a longer word fails that condition. The source distinction between finite minimum calculation and unrestricted shortest-description uncomputability is retained. All 256 original candidates and complete native program records and hashes match pinned V3.

The finite census is the original nonempty word layers through depth d filtered by exact decoding to object x. Thus

$$w \in Census_d(x) \iff (\exists n,\ 0 < n \le d \wedge w \in W_n) \wedge decode(w) = ok(x).$$

On a nonempty census, shortestFrom compares the next length with the current best and retains the shorter, choosing the next on a tie. Induction proves its result is a census member and has length no greater than every member. Filtering by equality to this attained minimum retains exactly all tied minima: the reverse direction follows by antisymmetry between a proposed minimum and the attained member. Every retained minimum therefore decodes to x; the selected registry request then reconstructs its unchanged execution by the interpretation equation.

If $d \le e$, the same width witness remains admitted at depth e. If |w|>d, word-length characterization contradicts the census width bound, excluding w. The singleton minimum is itself. A retained translation supplies its actual encoding row and decoder result; if a translated census member satisfies $|translated| \le |source| + |overhead|$, the target minimum satisfies that same upper bound by transitivity. Concatenation length is exactly additive. These are finite-depth minimum and explicit-overhead results.

For any depths $d \le D$, a description admitted at d remains admitted at D: keep its generated-word and decoding witnesses and compose its width bound with $d \le D$. Thus completeness is proved for every supplied depth, rather than for one fixed finite example. The attained minimum quantifies over that entire decoded census. completeDescriptions; retainedDepth; declaredDepthBoundary.

> Source, proof and provenance: V3 DESCRIPTIVE; V3 resource_ledger; SFT.Computation.DescriptiveResources.attainedMinimum; SFT.Computation.DescriptiveResources.minimumReconstructs; Timeline: Turn 223, line 2345.

171. Exact search on registered carriers

The translated search operation preserves the original admission order: repeated carrier identities are rejected before target membership is checked. A held target yields its original position label and exact identity. Native induction proves finite exhaustion, reconstructs the carrier as the unmatched prefix followed by the target and remaining suffix, and establishes that every earlier comparison differs from the target. Appending further carrier rows retains an already found position; the singleton base returns position-1. These search results combine with the exact ordering construction and complete native entry-program comparison presented below.

The scan equations are $scan(t, []) = none$ and

$$scan(t, x :: R) = \begin{cases} some(0), & x = t, \\ map(succ, scan(t, R)), & x \ne t. \end{cases}$$

For held t, induction on the carrier finds an index: the head match returns zero, otherwise membership lies in the tail and the inductive index is incremented. Induction on a successful scan also reconstructs

$$carrier = B +\!+ (t :: A), \qquad |B| = index, \qquad \forall x \in B,\ x \ne t.$$

The matching head gives $B = []$; a nonmatching head is prepended to the tail's unmatched prefix. Positional lookup at that index consequently returns t. With distinct carrier and held target, the executable result is (position-(index+1),t). Its guard order first rejects duplicate carriers, then rejects absence, retaining the original errors. A successful scan in L remains at the same index in $L +\!+ R$ by induction on L, so admitted extensions preserve the found position. The singleton returns (position-1,t).

> Source, proof and provenance: V3 SEARCH-ORDER; V3 exact_search; SFT.Computation.SearchTrace.scanTrace; SFT.Computation.SearchTrace.admittedSearch; Timeline: Turn 225, line 2360.

172. Exact ordering and complete recursion traces

The registered finite carrier generates its full arrangement census. Exact comparison rows select all arrangements satisfying the ordering invariant; native proofs establish that membership is precisely a full-carrier permutation satisfying every required comparison. Every duplicate-value class retains its original identities. The terminal certificate preserves the complete carrier, its length and the ordering invariant. Extension adds the new comparison rows while preserving the earlier invariant.

Structural recursion retains the initial value and the result of each exact registered successor step. Its trace contains one more value than successor positions, and every transition reconstructs from its prior value and position. Duplicate successor identities retain the original rejection. All three original search and recursion witnesses pass. The complete 256 algorithms-axis census and all native registration, decision, closure, control and hash outputs agree with pinned V3, completing the Fold Searching and Ordering Law.

The ordering predicate on x::R requires every pair (x,y), $y \in R$, in the registered relation and recursively requires ordering of R. Its empty case is true. Induction equates this Boolean with the corresponding empty/next invariant. Filtering the complete arrangement census therefore gives, for a nonempty distinct carrier C,

$$r \in orders(C, \rho) \iff Perm(r, C) \wedge Invariant(\rho, r).$$

Permutation preserves carrier membership and length. Combining membership with the class ledger gives x in class a of r iff $x \in C$ and $classify(x) = a$, retaining distinct identities in each duplicate-value class. To append fresh, require $(x, fresh) \in \rho$ for every old x. Induction on the invariant splits each comparison into an old element or fresh and preserves the earlier order.

For successor positions P and exact step f, recursion has $V(b, []) = [b]$ and $V(b, p :: P) = b :: V(f(b, p), P)$. Induction constructs the corresponding retained step certificate and proves $|V(b, P)| = |P| + 1$. Distinct successor identities make the original wrapper return this exact trace; duplicate identities retain its rejection.

> Source, proof and provenance: V3 SEARCH-ORDER; V3 structural_recursion; V3 run; SFT.Computation.OrderingTrace.completeOrders; SFT.Computation.OrderingTrace.exactRecursionTrace; Timeline: Turn 225, line 2360.

173. Exact arithmetic execution certificates

The arithmetic execution certificate retains each original registered operation, ordered input identities, intermediate exact part and bound output identity. Native proofs establish both directions of correspondence between this stepwise certificate and the original trace recorder. The recorded operation sequence is reconstructed exactly, with one row per operation. Every retained part remains positive, bounded through the structural One and in coprime normal form. Original fraction normalization preserves exact value by cross multiplication.

Terminal replay follows from the complete recorded execution. Serial composition retains the shared intermediate part and both row sequences; successor extension records the exact next operation and its inputs. The structural One supplies the base execution. Original operation failures propagate unchanged. All 256 original candidates, operational witnesses and native program fields and hashes agree with pinned V3, completing the Fold Exact Arithmetic-Algorithm Law.

Arithmetic Execution(a,ops,rows,b) has identity Execution(a,[],[],a). A successor requires $execute(op, a) = ok(c)$, then a certificate from c through the remaining operations, and prepends row(op,stepInputs(op,a),c). Induction on ops in the forward direction and on the certificate in reverse proves

$$recordFrom(a, ops) = ok(rows, b) \iff Execution(a, ops, rows, b).$$

The same successor structure gives $|rows| = |ops|$ and $map(operation, rows) = ops$. Each constructed row binds $outputHash = partIdentity(output)$. Its Part value carries $0 < n \leq d$ and Coprime(n,d). Original positive normalization additionally supplies $n \cdot d\prime = n\prime \cdot d$, preserving the exact fraction.

The existing generated-trace replay theorem applied to this correspondence gives $replayFrom(a, rows) = ok(b)$. Induction on a first certificate composes operations and rows by concatenation at its exact terminal/initial interface. Taking the second certificate to have one operation gives the successor row with its original ordered inputs. Structural One gives the identity base. If the first execute call fails, expanding recordFrom immediately propagates that same error.

> Source, proof and provenance: V3 ARITHMETIC; V3 structural_recursion; V3 run; SFT.Computation.ArithmeticTrace.completeTerminationTrace; SFT.Computation.ArithmeticTrace.terminalReconstruction; Timeline: Turn 226, line 2368.

174. Finite sequence matches and position reconstruction

For the registered subsequence alternative, complete finite scanning retains every skip/take choice that reconstructs the target sequence. Each match carries the source positions of its held symbols. Native proofs establish both directions between generated match ledgers and scan traces, preserving distinct occurrence positions even when symbols repeat. The match ledger contains exactly one position per target symbol.

Every recorded position and symbol reconstructs an exact source prefix, held symbol and suffix, with the position equal to its original offset. All positions lie inside the declared finite source interval. Appending source rows preserves earlier matches and their positions; a single held symbol supplies the base. No ungenerated infinite stream is treated as complete. All 256 original candidates, operational witnesses and complete native program fields and hashes agree with pinned V3, completing the Fold String and Sequence Algorithm Law.

For starting offset k, target x::N and source y::R, the complete match generator first emits all skip matches at k+1, then—only when $y = x$—prepends k to every take match for N in R. An empty target emits the single empty position list; a nonempty target in an empty source emits none. Induction establishes the equivalence

$$positions \in matchesFrom(k, N, S) \iff Match(k, N, S, positions),$$

where Match has exactly done, skip and take constructors. The forward direction splits membership between skip and take blocks; the reverse direction inserts each constructor into its corresponding generated block. Induction gives $|positions| = |N|$ and reconstructs N as a source-order selection.

For each $(i, x) \in zip(positions, N)$, the same induction gives $S = B + +x :: A$ and $i = k + |B|$. A skip prepends its source head to B; a take at the current head uses $B = []$; later takes apply the tail induction. Thus $k \leq i < k + |S|$. Repeated symbols retain their distinct offsets. Extending the source appends to the unused suffix in every constructor, preserving each prior position list. A singleton matching symbol has the singleton position list [k].

> Source, proof and provenance: V3 STRINGS-SEQUENCES; V3 structural_recursion; V3 run; SFT.Computation.SequenceTrace.completeMatchLedger; SFT.Computation.SequenceTrace.matchReconstructs; Timeline: Turn 227, line 2376.

175. Finite graph traversal certificates

The traversal certificate retains the complete frontier and visited ledger at each original search step. Empty-frontier termination, target detection, repeated-visit skipping and fresh-vertex expansion preserve the source branch order. A finite trace is constructed using the original decreasing unseen-support/frontier measure. Expansion appends every original outgoing unvisited target in edge order; no frontier cap or pruning changes the search.

Native replay yields the original reachability result. A successful rooted traversal provides the existing path witness, while exhausted-frontier failure excludes such a path. The same-node case retains original node-membership semantics. Existing path composition and graph extension proofs preserve the generated structure. Physical-network application retains the original empirical translation protocol. All 256 original candidates, witnesses and complete native program fields and hashes agree with pinned V3, completing the Fold Tree and Graph Algorithm Law.

Each traversal snapshot retains unseen support U, visited list V and complete frontier Q. Its branches, in original order, are: $Q = []$ returns false; a head equal to the target returns true; a different already-visited head is skipped; otherwise the fresh head h is expanded. Expansion sets

$$V' = V +\!+ [h], \qquad Q' = rest +\!+ outgoingUnvisited(edges, h, V').$$

The frontier invariant places every frontier node in U or V. A fresh h therefore belongs to U and its removal strictly reduces |U|. Skipping keeps U and shortens Q. Both branches decrease the lexicographic measure (|U|,|Q|), so well-founded induction constructs a finite snapshot trace without discarding frontier rows.

Induction on that trace applies the matching source search equation at each constructor, proving $searchLoop(initialSnapshot) = recordedAnswer$. For distinct origin and target, this is the original reachability result. Under graph admission, true reconstructs a complete path by reachability/path equivalence; false excludes any such path because that path would force true. The same-origin case retains the original test of node membership. The empirical translation condition remains $preSeal = false$, $mutation = false$, $allRows = true$ and $identified = true$.

> Source, proof and provenance: V3 TREES-GRAPHS; V3 structural_recursion; V3 run; SFT.Computation.TraversalTrace.finiteTraversal; SFT.Computation.TraversalTrace.rootedTraversalReconstructs; Timeline: Turn 228, line 2384.

176. Finite algebraic and geometric execution

An admitted closed operation table supplies every generated input pair with its exact carrier result. Native induction constructs a complete execution for every finite input trace, retaining every intermediate input/input/output row and proving that all intermediate and terminal identities remain in the carrier. Replay reconstructs the terminal result; row count and input projection preserve the complete input sequence. Serial composition and successor extension retain their exact shared identities and table rows, while table failures propagate unchanged.

For incidence geometry, each face coordinate is equivalent to its actual retained boundary row. The original cell-dimension algorithm is linked to its complete finite boundary-descent derivation, preserving admission and cycle rejection. Geometry remains finite and exact. All 256 original candidates, operational witnesses and complete native program records and hashes agree with pinned V3, completing the Fold Algebraic and Geometric Algorithm Law.

For an admitted closed table, total lookup on held a,x supplies a held next result b with $applyOperation(a, x) = ok(b)$. Induction on the finite input list repeats this construction, retaining row (a,x,b) before the recursive rows. The empty input has no rows and terminal a. Induction on the resulting Execution certificate proves replay, row count equal to input count, and projection of each row's right input equal to the original input list. Every row's two inputs and result remain in the carrier.

Serial composition joins two such certificates at their shared identity and concatenates both input and row lists. A one-operation second certificate appends exactly (current,right,next). A failed first lookup propagates its original error by expansion of the run definition.

For boundary relation B, filter/map membership gives

$$f \in faces(B, c) \iff (f, c) \in B.$$

A depth node requires c absent from the current visiting list and derivations for every face under $visiting + +[c]$. If their retained heights are H, the node height is heightMaximum(H)+1. Face derivations preserve every boundary member in order. Under the original incidence and held-cell admission, the exact algorithm satisfies $cellDimension(c) = ok(h)$ iff DepthDerivation(B,[],c,h). The visiting exclusion retains the original cycle boundary.

> Source, proof and provenance: V3 ALGEBRAIC-GEOMETRIC; V3 structural_recursion; V3 run; SFT.Computation.AlgebraGeometryTrace.closedExecution; SFT.Computation.AlgebraGeometryTrace.composition; Timeline: Turn 229, line 2392.

177. Exact subproblem reuse and reconstruction

The finite decomposition retains each subproblem identity and its complete ordered dependency list. Each dependency lookup reads an existing exact result, including repeated references to overlapping subproblems. A result is admitted only through its sole matching registered recurrence row. Executable table construction and its typed execution certificate imply one another, and each task adds one retained result. Its identity sequence is exactly the initial sequence followed by the schedule, preserving the source distinct-identity condition. Missing dependencies or unresolved recurrence rows halt the construction.

Every result constructed from the empty table has a finite derivation through the actual decomposition and recurrence rows. Reuse preserves the original dependency identity and value across later table extensions. Composition and successor preserve all earlier rows and their shared interface. A problem invariant follows by induction when its local recurrence preserves that invariant from the established dependency results. The decomposition and optimal-substructure conditions must be proved for each problem, as required by V3; no particular recurrence or optimum is supplied by the generic construction. All 256 original candidates, operational witnesses and complete native program fields and hashes agree with pinned V3, completing the Fold Dynamic-Programming Law.

Dependency reads traverse the ordered key list, looking up each value in the current table; failure of any lookup returns none. Their typed empty/next certificates are equivalent to successful inputs lookup by induction and preserve $|values| = |dependencies|$. Each zipped dependency/value pair is an actual table row.

The recurrence result is admitted only when the complete list of matching (key,values,result) rows has exactly one element. A task appends that key/result row and continues. Induction proves

$$run(rules, T, tasks) = some(U) \iff Execution(rules, T, tasks, U),$$

and yields $|U| = |T| + |tasks|$ and $map(key, U) = map(key, T) + +map(key, tasks)$. Thus distinctness of this combined identity sequence transfers to the final table. Missing dependency values or a non-singleton recurrence match halt the construction.

Given an invariant on initial rows and a local recurrence rule preserving it from every paired dependency result, induction on Execution proves the invariant for the terminal table. Taking the invariant to be a finite derivation through the registered decomposition and recurrence rows reconstructs every result produced from the empty table. Each execution extends T by a suffix; first lookup of an existing key remains unchanged under suffix extension, preserving repeated reuse. Serial certificates compose at the exact intermediate table, and a singleton final task gives successor extension.

> Source, proof and provenance: V3 DYNAMIC-PROGRAMMING; V3 structural_recursion; V3 run; SFT.Computation.DynamicProgrammingTrace.runExecution; SFT.Computation.DynamicProgrammingTrace.finalReconstruction; Timeline: Turn 230, line 2400.

178. Complete optimization traversal

The traversal retains every feasible candidate in source order and every exact incoming dominance row. Its terminal class is proved equal to the original complete undominated selection. Every eliminated feasible candidate has a retained, distinct feasible dominator; all undominated candidates survive. Ties and incomparable alternatives remain explicit, and uniqueness is asserted only for a singleton. Original feasibility and preference admission conditions are preserved without a score, metric, differentiability or convexity assumption.

The incumbent history contains its initial class and one state for each traversed candidate. Each prefix reconstructs its exact incumbent, and serial composition preserves that shared class. Extending both the candidate support and preference relation retains precisely the forms with neither an old nor a new dominator. An earlier optimum can disappear only through an added dominance witness. The singleton carrier supplies the base. All 256 original candidates, operational witnesses and complete native program fields and hashes agree with pinned V3, completing the Fold Optimization-Algorithm Law.

For candidate x, eliminators(P,x) filters every preference row whose target is x. The comparison ledger maps every feasible candidate to this full incoming-row list, so its identity projection is exactly the feasible list. Keep(x) means no preference row targets x; advance appends x to the incumbent precisely in that case. Induction gives

$$scan(P, I, F) = I +\!+ retained(F, P).$$

The kept branch appends its candidate and reassociates concatenation; the eliminated branch leaves the incumbent unchanged. With $I = []$, terminal membership is exactly $x \in F$ and no incoming dominance row. Under preference admission, each eliminated feasible x has a distinct held dominator recorded in eliminators(P,x). Ties and incomparable survivors are retained; classification is unique exactly when the complete survivor list is a singleton.

History begins with I and recurses after each advance, giving |F|+1 incumbent states. Serial scan first computes scan(P,I,L), then uses that exact incumbent for R. Its history equation removes the first history's last state before adjoining the second complete history, retaining the shared state once. With support $F +\!+ G$ and preference $P +\!+ Q$, survival requires membership in either support and absence of both old and new dominators. If an old survivor disappears, its elimination witness cannot be in P and must therefore be a new row in Q.

> Source, proof and provenance: V3 OPTIMIZATION; V3 structural_recursion; V3 run; SFT.Computation.OptimizationTrace.allOptima; SFT.Computation.OptimizationTrace.eliminationCertificate; Timeline: Turn 231, line 2408.

179. Scheduled algorithms and exact success support

Every registered schedule executes the same held deterministic kernel and initial form. The complete schedule ledger retains each identity and its execution trace or error, with one associated resource record. Exact outcomes define a covering, disjoint partition. Successful schedule identities are precisely the members of the registered target-outcome class; canonical schedule identities imply duplicate-free success support contained in the whole support.

The success part uses the original held-support/whole-support operation. Its positive result satisfies the exact count-ratio identity and lies through One; empty support selection retains the original empty-One marker. Resource reconstruction retains the full observation trace, its initial-observation-inclusive count and the supplied occupied-state support and count. Composition and successor preserve all previous

records. Apparent randomness remains an unresolved or externally measured deterministic schedule, with the original measurement boundary unchanged. All 256 original candidates, witnesses and complete native program fields and hashes agree with pinned V3, completing the Fold Randomized-Algorithm Law.

For each registered (name,actions), branches retains the same machine and initial form with that action list. Mapping to (execution,resource result) gives one record per schedule and projects back to the complete ordered execution list. Outcome classes from Section 162 cover every branch and are disjoint for different outcomes.

Success names are the identity projection of the target-outcome class. Map/filter membership gives

$$name \in Success(o) \iff \exists b \in Schedule,\ b.name = name \wedge outcome(b) = o.$$

Consequently success support is contained in the whole identity support. Induction on filtering preserves identity distinctness, so canonical schedules admit the exact event operation. For a positive success fraction p/q, the original weight theorem gives

$$|Success(o)|q = p\,|ScheduleNames|, \qquad p \le q.$$

An empty success list retains the original empty-One weight marker. Successful resource reconstruction gives the full trace, $timeCount = |actions| + 1$ and $spaceCount = |occupied|$; execution failure propagates the same error to the resource result. Map distribution proves composition of record lists and the exact one-record successor, preserving every previous execution and resource observation.

> Source, proof and provenance: V3 RANDOMIZED; V3 structural_recursion; V3 run; SFT.Computation.ScheduledAlgorithm.completeRecords; SFT.Computation.ScheduledAlgorithm.exactSuccessRatio; Timeline: Turn 232, line 2416.

180. Causally independent algorithm layers

Every task in an admitted ready layer reads the same prior retained result table. The dependency graph places each prerequisite before the layer and excludes dependencies between simultaneous tasks. Complete synchronization rows retain the corresponding prerequisite identities. Native layer execution and its typed trace imply one another, retaining each task identity and one exact recurrence result per task. Local invariants propagate from prior dependency results, and every generated output reconstructs through the original finite decomposition and recurrence rows.

Successive layers append their complete result rows to the shared table. The combined result identities equal the initial identities followed by the complete flattened layer trace. Complete task coverage therefore transfers directly to the work ledger, while the existing exhaustive path construction supplies the span certificate. Composition and successor preserve previous records and their shared interfaces. No physical speedup is claimed without an implementation experiment. All 256 original candidates, witnesses and complete native program fields and hashes agree with pinned V3, completing the Fold Parallel-Algorithm Law.

Let B be the retained result table and $L = [(k_i, d_i)]_{i=1}^{m}$ a layer of task identities and dependency lists. The source execution reads every dependency list against the same B:

$$\mathrm{layerRun}(R, B, []) = \mathrm{some}[], \qquad \mathrm{layerRun}(R, B, (k, d) :: L) = \mathrm{some}((k, v) :: W)$$

precisely when $\mathrm{inputs}(B, d) = \mathrm{some}\, u$, the selected recurrence gives $\mathrm{result}(R, k, u) = \mathrm{some}\, v$, and $\mathrm{layerRun}(R, B, L) = \mathrm{some}\, W$. A failed read or recurrence produces failure. Thus earlier rows in the same layer never become inputs to its later tasks. The native reconstruction and identity statements are

$$\mathrm{layerRun}(R, B, L) = \mathrm{some}\, W \iff \mathrm{LayerExecution}(R, B, L, W), \qquad \mathrm{ids}(W) = \mathrm{ids}(L), \quad |W| = |L|.$$

For an admitted ready layer, every graph prerequisite belongs to $\mathrm{ids}(B)$, and no edge joins two tasks in that layer. Under the retained local-rule hypothesis, validated dependency values imply a validated recurrence result; induction over the layer then validates every row of W. Instantiating this implication with the source derivation property reconstructs each result's derivation.

Writing $E(B, \mathcal{L})$ for execution of successive layers, its equations are

$$E(B, []) = \mathrm{some}\, B, \qquad E(B, L :: \mathcal{L}) = E(B +\!\!+ W, \mathcal{L}) \quad \text{when } \mathrm{layerRun}(R, B, L) = \mathrm{some}\, W.$$

The scheduled trace retains readiness at each intermediate table. Induction gives

$$\mathrm{ids}(B_{\mathrm{final}}) = \mathrm{ids}(B_{\mathrm{initial}}) +\!\!+ \mathrm{flatten}([\mathrm{ids}(L) : L \in \mathcal{L}]).$$

With an empty initial table, coverage of final node identities therefore yields coverage by the flattened work trace. The same layer induction propagates complete derivations, while concatenation of executions uses their shared retained table.

> Source, proof and provenance: V3 PARALLEL; V3 structural_recursion; V3 run; SFT.Computation.ParallelAlgorithm.completeReconstruction; SFT.Computation.ParallelAlgorithm.independentLayer; Timeline: Turn 233, line 2424.

181. Global replay of retained local process events

A retained event identifies its process, prior local state, action and resulting local state. The replay checker resolves the registered process kernel and verifies the unchanged local transition before updating that process in the global state table. Native certificates and successful replay imply one another for every finite event trace. Process identities are preserved, and every process absent from the trace retains its original local state. Serial composition and successor extension reconstruct through the shared intermediate state. These results establish the local-state replay component; the message and causal/fault certificates below complete its connection to distributed communication.

Write an event as $e = (\iota, p, b, a, c)$, with identity ι, process p, prior state b, action a, and resulting state c. For registry K and store S, the exact successful replay condition is

$$\begin{aligned} \mathrm{replayEvent}(K, S, e) = \mathrm{some}\, S' \iff \exists M :\ & \\ & \mathrm{lookupFirst}(K, p) = \mathrm{some}\, M \\ & \wedge\, \mathrm{lookupFirst}(S, p) = \mathrm{some}\, b \\ & \wedge\, \mathrm{step}(M, b, a) = \mathrm{ok}\, c \\ & \wedge\, S' = \mathrm{update}(S, p, c). \end{aligned}$$

Here update preserves the store's list structure and replaces the value of every row whose process identity equals p; lookup retains the original first-match convention. A missing process, wrong prior state, transition error, or mismatched resulting state rejects the event.

The full trace is generated by identity and successor rules:

$$\mathrm{GlobalTrace}(K, S, [], S), \qquad \frac{\mathrm{LocalStep}(K, S, e) \quad \mathrm{GlobalTrace}(K, \mathrm{update}(S, p, c), T, F)}{\mathrm{GlobalTrace}(K, S, e :: T, F)}.$$

Induction in both directions establishes

$$\mathrm{replay}(K,S,T) = \mathrm{some}\,F \iff \mathrm{GlobalTrace}(K,S,T,F), \qquad \mathrm{ids}(F) = \mathrm{ids}(S).$$

If $q \neq e.p$ for every event $e \in T$, each update preserves its lookup, hence

$$\mathrm{lookupFirst}(F,q) = \mathrm{lookupFirst}(S,q).$$

Two traces from S to M and from M to F compose into their concatenation from S to F; the successor case is the corresponding one-event extension.

> Source, proof and provenance: V3 DISTRIBUTED; V3 structural_recursion; V3 run; SFT.Computation.DistributedReplay.globalReplay; SFT.Computation.DistributedReplay.globalDerivation; Timeline: Turn 235, line 2440.

182. Exact distributed messages and causal reconstruction

Separate send and receive ledgers retain each message identity, payload, process and event identity. Every record binds to the registered alphabet and an actual process event. Every declared receipt has a retained send with the same message identity and payload at an earlier event position. Sends without a declared receipt remain visible; delivery is not assumed. Canonical event identities and exact positions make the causal relation transitive and irreflexive, excluding causal return cycles.

An original local transition error prevents global replay from continuing to a result. A completed certificate combines the full local-state trace, canonical event identities and verified message ledger, reconstructing the terminal global state while preserving all process identities. Trace extension retains prior port bindings and causal decisions; ledger composition retains both complete record collections. System implementation and physical network claims remain outside the formal theorem. All 256 original candidates, witnesses and complete native program fields and hashes agree with pinned V3, completing the Fold Distributed-Algorithm Law.

For a port $r = (m, \iota, p, v)$, let A be the payload alphabet and T the event list. The binding and causal predicates have the exact meanings

$$\mathrm{bound}(A,T,r) \iff v \in A \wedge \exists e \in T,\ e.\iota = \iota \wedge e.p = p,$$

$$\mathrm{before}_T(x,y) \iff \exists i, j,\ \mathrm{scan}(x, \mathrm{ids}(T)) = \mathrm{some}\, i \wedge \mathrm{scan}(y, \mathrm{ids}(T)) = \mathrm{some}\, j \wedge i < j.$$

Scan uses the source's first matching position. For send ledger S and receipt ledger R, validity is exactly

$$\big(\forall s \in S, \mathrm{bound}(A,T,s)\big) \wedge \big(\forall r \in R, \mathrm{bound}(A,T,r)\big) \wedge \big(\forall r \in R, \exists s \in S,\ s.m = r.m \wedge s.v = r.v \wedge \mathrm{before}_T(s.\iota, r.\iota)\big).$$

This quantifies over every receipt without requiring every send to have a receipt. An unpaired receipt contradicts the final conjunct. Determinacy of scan identifies the middle index in two causal comparisons, so $i < j$ and $j < k$ give $i < k$; $i < i$ is impossible. These establish transitivity, absence of self-cause, and absence of causal return.

The complete certificate conjoins distinct event identities, the global local-state trace, and this message-ledger validity:

$$C = \mathrm{Distinct}(\mathrm{ids}(T)) \wedge \mathrm{GlobalTrace}(K,I,T,F) \wedge \mathrm{valid}(A,T,S,R).$$

Consequently C implies $\mathrm{replay}(K, I, T) = \mathrm{some}\, F$ and $\mathrm{ids}(F) = \mathrm{ids}(I)$. Under the registered-machine and current-state premises, a local transition error yields a failed event replay, which yields failed replay for the trace beginning with that event. Appending events preserves existing bindings and successful scan positions; therefore it preserves this ledger's validity. Two valid send/receipt ledgers over the same event list compose by list concatenation, retaining witnesses from their respective original ledgers.

> Source, proof and provenance: V3 DISTRIBUTED; V3 structural_recursion; V3 run; SFT.Computation.DistributedMessages.completeReceiptProvenance; SFT.Computation.DistributedMessages.noCausalReturn; Timeline: Turn 235, line 2440.

183. Finite-prefix streaming and retained distinctions

Every generated input has a complete ordered prefix family, including its identity prefix. Native proofs establish both directions between prefix membership and exact input decomposition, giving one result record per prefix and a total of input length plus one records. Each successful result reconstructs its complete original execution path and terminal state summary. The original transition relation determines the next summary, and serial continuation uses the retained intermediate state.

The full ordered pair ledger retains exactly distinct supported inputs whose state summaries coincide. Inputs with different summaries are excluded from that loss ledger. Input extension preserves every earlier prefix, output and lost-distinction record. Each resource certificate retains the complete observation trace and supplied occupied support with their exact counts. The prefix and output theorems quantify over every finite input and every finite appended segment. All 256 original candidates, witnesses and complete native program fields and hashes agree with pinned V3, completing the Fold Online and Streaming Algorithm Law.

For a finite input x, the complete ordered prefix list is defined recursively:

$$P([]) = [[]], \qquad P(a :: x) = [] :: [a :: p : p \in P(x)].$$

Induction proves both its exact boundary and its count:

$$p \in P(x) \iff \exists s,\ x = p +\!\!+ s, \qquad |P(x)| = |x| + 1.$$

The output ledger is $O(x) = [(p, \mathrm{run}(M, i, p)) : p \in P(x)]$, so it retains exactly the original run for each prefix and has $|x| + 1$ rows. Its summary is

$$\sigma(p) = \begin{cases} \mathrm{failed}(r), & \mathrm{run}(M, i, p) = \mathrm{error}\, r, \\ \mathrm{returned}(\mathrm{endLabel}(t)), & \mathrm{run}(M, i, p) = \mathrm{ok}\, t. \end{cases}$$

For every successful prefix, native path reconstruction gives $x = p +\!\!+ s$, $t = b +\!\!+ [q]$, the original path from i through p to q, and $\sigma(p) = \mathrm{returned}(\mathrm{some}\, q)$. If that path extends by $\mathrm{step}(M, q, a) = \mathrm{ok}\, q'$, with the original initial-state admissibility, then

$$\sigma(p +\!\!+ [a]) = \mathrm{returned}(\mathrm{some}\, q').$$

Serial path composition gives the corresponding statement for an arbitrary admitted continuation.

For supplied finite support U, form the full ordered product list $U \times U$, retaining its source ordering and multiplicities, then filter it:

$$L(U) = \mathrm{filter}_{u \neq v \,\wedge\, \sigma(u) = \sigma(v)}(U \times U).$$

The exact membership theorem is

$$(u, v) \in L(U) \iff u \in U \wedge v \in U \wedge u \neq v \wedge \sigma(u) = \sigma(v).$$

Different summaries therefore exclude a pair. Since every old prefix remains a prefix after appending input, old output rows and pairs in $L(P(x))$ remain in the extended ledgers. Finally, for a successful run and its accepted resource record with supplied occupied support H,

$$\mathrm{map}(\mathrm{snd}, \ell.\,\mathrm{timeTrace}) = t, \quad \ell.\,\mathrm{spaceSupport} = H, \quad \ell.\,\mathrm{timeCount} = |p| + 1, \quad \ell.\,\mathrm{spaceCount} = |H|.$$

These equations retain the full finite observation trace as well as the exact support supplied to the recorder.

For arbitrary finite lists x, z, a previous prefix decomposition $x = p{+\!+}s$ gives

$$x{+\!+}z = p{+\!+}(s{+\!+}z).$$

The complete prefix-membership theorem therefore retains p, and the output-membership theorem retains exactly $(p, \mathrm{run}(M, i, p))$. This proves extension for any segment length; each execution still computes the finite prefix list supplied to it. prefixMeaning; prefixRetained; outputRetained.

> Source, proof and provenance: V3 ONLINE-STREAMING; V3 structural_recursion; V3 run; SFT.Computation.StreamingTrace.prefixMeaning; SFT.Computation.StreamingTrace.retainedSummary; Timeline: Turn 236, line 2448.

184. Exact oriented numerical-error records

For equal nonempty finite supports of exact positive fractions, the numerical error ledger retains one ordered cell record per paired input. Equal values carry the structural One marker and the equal orientation; unequal values carry their exact positive difference and the held-below or held-above orientation. Native arithmetic proofs identify each normalized difference by its cross-multiplied rational value. Nonpositive values and unequal or empty supports retain the original rejection behavior. These results establish the error-ledger component of the numerical formalisation.

For reference $x = n_x/d_x > 0$ and approximation $y = n_y/d_y > 0$, define the exact cross-products

$$A = n_y d_x, \qquad E = n_x d_y, \qquad D = d_x d_y > 0.$$

The source error operation is the following oriented positive-part record:

$$\mathrm{err}(x, y) = \begin{cases} (1, \mathrm{equal}), & A = E, \\ (\mathrm{normalizePositive}(E - A, D), \mathrm{held\text{-}below}), & A < E, \\ (\mathrm{normalizePositive}(A - E, D), \mathrm{held\text{-}above}), & E < A. \end{cases}$$

The equal case deliberately retains the structural One marker. For either unequal branch, with positive difference numerator Δ and normalized result n/d, preservation of exact value is proved by

$$\Delta d = nD, \qquad n > 0.$$

Thus normalization retains the exact rational difference and the orientation separately. The ordered row operation starts at index 1, emits $(\text{cell-}i, \operatorname{err}(x_i, y_i))$, and recurses at $i+1$. Its outer admission guard is

$$|X| = |Y| \ \wedge \ X \neq [],$$

while each paired element must have the positive constructor. A zero or negative constructor rejects the scientific value; a length mismatch or empty reference rejects the support. These are distinct original error branches, not numerical tolerances.

> Source, proof and provenance: V3 NUMERICAL; V3 exact_error_ledger; V3 run; SFT.Computation.NumericalError.belowValue; SFT.Computation.NumericalError.aboveValue; Timeline: Turn 240, line 2482.

185. Complete numerical error-ledger reconstruction

Successful numerical error-ledger execution is equivalent to a complete ordered certificate pairing each reference value with its approximation and exact oriented row. The certificate proves positivity of both input supports and every error value, equal support lengths, and one row per pair without omission. Conversely, every equal-length nonempty pair of positive supports constructs a successful ledger. Serial composition and successor extension preserve earlier rows while advancing the original cell labels. Independent finite expression comparisons exercise all 1,849 pairs of supports of widths up to two over six exact signed/zero/positive test values; their output and rejection records agree with native Lean execution.

Let $\mathcal{R}_i(X, Y, L)$ denote the retained ordered row certificate. Its constructors are

$$\mathcal{R}_i([], [], []), \qquad \frac{\mathcal{R}_{i+1}(X, Y, L)}{\mathcal{R}_i(x :: X, y :: Y, (\text{cell-}i, \operatorname{err}(x, y)) :: L)} \quad (x, y \text{ positive}).$$

Structural induction on these constructors reproduces the executable row operation. Conversely, inspection of each successful executable branch constructs the corresponding certificate. Therefore

$$\operatorname{rowsFrom}(i, X, Y) = \operatorname{ok} L \iff \mathcal{R}_i(X, Y, L),$$

$$\operatorname{exactErrorLedger}(X, Y) = \operatorname{ok} L \iff X \neq [] \wedge \mathcal{R}_1(X, Y, L).$$

Induction gives positivity of both supports and every error numerator, together with

$$|L| = |X| = |Y|.$$

Each row retains its paired positive values, index label, and exact error operation. Conversely, the recursively generated positive-row list provides a certificate for any two equal-length positive supports; nonemptiness then gives outer ledger admission.

For concatenated supports, the second certificate begins at the advanced cell index:

$$\mathcal{R}_i(X, Y, L) \wedge \mathcal{R}_{i+|X|}(X', Y', L') \implies \mathcal{R}_i(X +\!\!+ X', Y +\!\!+ Y', L +\!\!+ L').$$

Taking the second pair of supports to be singletons proves successor extension with label $\text{cell-}(i + |X|)$. The finite comparison family uses $1 + 6 + 6^2 = 43$ supports, including the empty support, so its complete

ordered pair family has $43^2 = 1,849$ cases. Rejected supports and signed or zero elements remain part of that comparison.

> Source, proof and provenance: V3 NUMERICAL; V3 exact_error_ledger; V3 run; SFT.Computation.NumericalLedger.exactAdmission; SFT.Computation.NumericalLedger.noOmittedRows; Timeline: Turn 240, line 2482.

186. Retained numerical update observations

Every state in a generated update trace resolves to its registered numerical carrier and a complete exact error ledger. The observation checker and the retained certificate imply one another; the ordered projection reproduces the entire state trace, and every numerical value is traced to its registered carrier row. Connecting these records to the original transition execution gives exactly one observation for each action plus the initial state. Composition retains all intermediate records. An admitted successor step appends its exact observation while preserving the earlier trace. Missing carrier values or rejected error ledgers cannot produce an accepted observation trace.

Let T be the registered state-to-value table and X the fixed reference support. The numerical observation recursion is

$$N(T, X, []) = \mathrm{some}[],$$

$$N(T, X, s :: S) = \mathrm{some}((s, Y, L) :: Q)$$

exactly when

$$\mathrm{lookupFirst}(T, s) = \mathrm{some}\, Y, \qquad \mathrm{exactErrorLedger}(X, Y) = \mathrm{ok}\, L, \qquad N(T, X, S) = \mathrm{some}\, Q.$$

Missing lookups, rejected ledgers, and failed suffix observations propagate failure. The corresponding inductive certificate retains these three witnesses at every row. Reconstruction and replay prove

$$N(T, X, S) = \mathrm{some}\, Q \iff \mathrm{Records}(T, X, S, Q).$$

Induction then yields

$$\mathrm{map}(\mathrm{fst}, Q) = S, \qquad |Q| = |S|,$$

and for every $(s, Y, L) \in Q$, the registered row $(s, Y) \in T$, nonempty reference, and full certificate $\mathcal{R}_1(X, Y, L)$.

If the original state execution satisfies $\mathrm{run}(M, i, a) = \mathrm{ok}\, S$, its path theorem gives $|S| = |a| + 1$. Combining this equality with the observation count gives

$$|Q| = |a| + 1.$$

An admitted transition from the retained terminal state to s', its registered value Y', and accepted error ledger L' extend both executions:

$$S' = S +\!\!+ [s'], \qquad Q' = Q +\!\!+ [(s', Y', L')].$$

The earlier records remain intact. Concatenation of observation certificates follows directly by induction over the first certificate.

> Source, proof and provenance: V3 NUMERICAL; V3 exact_error_ledger; V3 run; SFT.Computation.NumericalTrace.correspondence; SFT.Computation.NumericalTrace.nativeCount; Timeline: Turn 240, line 2482.

187. Finite numerical boundary certificates

The numerical carrier uses exact positive parts on finite generated cells. Its retained state updates reconstruct every oriented error row. Output enclosure is the complete held-order selection between registered bounds: every selected form belongs to the carrier and has both endpoint relations. Original exhaustive lower and upper bound results enclose each held output. A terminal certificate retains original state-constructor and order admission, the actual finite execution, all intermediate observations, its explicit target and its enclosure. The final numerical value and complete error ledger reconstruct from that certificate.

Projecting each original transition to its source and target connects executed paths to the admitted dynamics relation. The registered stability check holds exactly when every registered perturbation has a retained path to the target. Complete perturbation path families establish that check, and the target retains both enclosure relations. Successor construction requires the exact next transition, numerical record and enclosure evidence, retaining all earlier observations. This formalises the declared finite certificate boundary; it does not assert that every numerical procedure converges or import an ungenerated infinite limit.

The complete 256-candidate numerical program, its original operational witnesses, controls and all recorded hashes agree with pinned V3 under fresh native execution. Together with the carrier, update and boundary derivations, these results complete the Fold Numerical-Algorithm Law.

For generated carrier C, held order relation $\preceq$, and bounds l, u, the enclosure is the complete filtered list

$$I(C, \preceq, l, u) = [v \in C : l \preceq v \wedge v \preceq u].$$

Its membership theorem is exactly

$$v \in I(C, \preceq, l, u) \iff v \in C \wedge l \preceq v \wedge v \preceq u.$$

If the original lower- and upper-bound operations return L and U for held support H, then $l \in L$, $u \in U$, and $v \in H \cap C$ imply this enclosure: the lower-bound witness supplies $l \preceq v$, and the upper-bound witness supplies $v \preceq u$.

A terminal certificate retains all six conditions

$$\mathrm{create}(M) = \mathrm{ok}\, M, \quad \mathrm{isPartialOrder}(C, \preceq) = \mathrm{true}, \quad \mathrm{run}(M, i, a) = \mathrm{ok}\, S, \quad N(T, X, S) = \mathrm{some}\, Q,$$

$$S = B +\!\!+ [t] \text{ for some } B, \qquad t \in I(C, \preceq, l, u).$$

Induction over the observation trace ending at t produces

$$Q = Q_0 +\!\!+ [(t, Y, L)], \qquad \mathrm{lookupFirst}(T, t) = \mathrm{some}\, Y, \qquad X \neq [] \wedge \mathcal{R}_1(X, Y, L).$$

The terminal bounds are $l \preceq t \preceq u$, and the complete observation count remains $|Q| = |a| + 1$.

Project each original transition (s, a, s') to (s, s'), obtaining D_M. Induction over an executed path gives reachability in D_M. The registered stability equivalence is

$$\mathrm{stableUnderRegisteredPerturbations}(D_M, t, P) = \mathrm{true} \iff \forall p \in P, \mathrm{Reaches}(D_M, p, t).$$

Thus an explicit execution path from every registered perturbation constructs stability, and the retained terminal enclosure supplies the same two endpoint relations. The identity certificate uses the admitted initial state, its numerical record and enclosure. A successor certificate requires the actual next transition, accepted next record and next enclosure, and appends them to the prior trace; its derivation does not replace those required witnesses with a convergence assumption.

> Source, proof and provenance: V3 NUMERICAL; V3 exact_error_ledger; V3 run; SFT.Computation.NumericalBoundary.originalBoundsEnclose; SFT.Computation.NumericalBoundary.stableEnclosedTarget; Timeline: Turn 240, line 2482.

188. Exact symbolic transformations within the frozen grammar

The generated expression grammar contains positive exact values, held names and binary joins. Native evaluation preserves original binding uniqueness, literal admission and left-to-right error behavior. A successful result is equivalent to a complete evaluation derivation. Bound names retain their original raw Fraction values; the scientific normalization certificate separately carries the positive-binding boundary. Exact signed arithmetic is represented constructively, including cancellation and normalized positive differences.

A held tree occurrence is identified by its generated left/right address and surrounding context. Native proofs establish both directions of that representation and reconstruct the complete expression. Equivalence of successful exact values is preserved inside every context. Each rewrite record retains a rule in the supplied rule list, its held occurrence and its equivalence proof. All intermediate and final values are preserved along a complete trace; composition and successor retain earlier records. A normalization certificate includes the terminal absence of an admitted rewrite. Its final expression and exact positive scientific result reconstruct from the retained trace. The normalization judgment consists of the complete rewrite trace and the explicit terminal normality predicate.

All 256 original symbolic kernels, witnesses, controls and complete program fields and hashes agree with pinned V3. Independent evaluation of the source expressions agrees with Lean in all 7007 cases spanning 143 expressions and 49 environments, including invalid literals, missing and duplicate names, signed values and nested joins. These results complete the Fold Symbolic-Computation Law for the recursively generated value/name/join grammar.

Let ρ be the original binding list. Successful evaluation has the exact characterization

$$\mathrm{eval}(\rho, e) = \mathrm{ok}\, v \iff \mathrm{Distinct}(\mathrm{names}(\rho)) \wedge \mathrm{Evaluation}(\rho, e, v).$$

The evaluation derivation has positive-literal, registered-name and binary-join rules:

$$\frac{}{\rho \vdash \mathrm{value}(p) \Downarrow p}\ (p > 0), \qquad \frac{\mathrm{lookupFirst}(\rho, n) = \mathrm{some}\, v}{\rho \vdash \mathrm{name}(n) \Downarrow v}, \qquad \frac{\rho \vdash a \Downarrow u \quad \rho \vdash b \Downarrow v}{\rho \vdash \mathrm{join}(a, b) \Downarrow \mathrm{add}(u, v)}.$$

The executable operation checks binding uniqueness first and evaluates a join's left child before its right child; it propagates the first encountered error. The name rule retains its bound value without imposing the positive-literal restriction on that value.

For a generated context C, plugging and addressing satisfy

$$\mathrm{focus}(\mathrm{plug}(C, e), \mathrm{address}(C)) = \mathrm{some}\, e.$$

Conversely, successful focus supplies a context with that address and reconstructs the entire original expression by plugging. Define successful-value equivalence by

$$e \sim_\rho f \iff \forall v,\ \mathrm{eval}(\rho, e) = \mathrm{ok}\, v \iff \mathrm{eval}(\rho, f) = \mathrm{ok}\, v.$$

Induction on contexts, using the two join evaluations, proves

$$e \sim_\rho f \Longrightarrow \mathrm{plug}(C, e) \sim_\rho \mathrm{plug}(C, f).$$

Each admitted rewrite retains a registered rule, generated pattern and replacement, a generated context and the pattern/replacement equivalence. Contextual equivalence therefore proves each rewrite preserves successful values; induction on the complete rewrite trace preserves them at every source, target and terminal expression.

Normality is the generated-expression condition together with absence of any admitted rewrite whose source is that expression. A normalization certificate is a retained trace ending at such a normal expression. Hence

$$\mathrm{Normalized}(\rho, R, e, T, f) \wedge \mathrm{eval}(\rho, e) = \mathrm{ok}\, v \Longrightarrow \mathrm{eval}(\rho, f) = \mathrm{ok}\, v.$$

With the separately retained positive-binding condition, evaluation induction also gives $v = \mathrm{positive}(p)$ for some exact positive fraction p. Normality supplies the universal exclusion of an admitted outgoing rewrite at the terminal expression.

The grammar is inductive: positive values and names are base constructors, and joining any two generated expressions produces another. Contexts likewise have hole, left and right constructors, so contextual preservation applies at every finite depth. Rule-list extension is also constructive: if $R \subseteq R'$, trace induction replaces each rule-membership witness by its inclusion image and keeps the context, equivalence proof and endpoints. Hence

$$\mathrm{Trace}(\rho, R, e, T, f) \Longrightarrow \mathrm{Trace}(\rho, R', e, T, f).$$

Normality under R' requires its own exclusion of every newly admitted outgoing rewrite; the retained trace and that exclusion together form the extended normalization certificate. frozenRuleBoundary; Normal; Normalized; normalizationReconstructs.

> Source, proof and provenance: V3 SYMBOLIC; V3 evaluate_expression; SFT.Computation.SymbolicValues.sumValue; SFT.Computation.SymbolicValues.cancellation; SFT.Computation.SymbolicEvaluation.duplicateEnvironment; SFT.Computation.SymbolicEvaluation.replay; SFT.Computation.SymbolicContext.plugGenerated; SFT.Computation.SymbolicContext.sameIdentity; SFT.Computation.SymbolicTrace.admittedSource; SFT.Computation.SymbolicTrace.admittedTarget; Timeline: Turn 241, line 2490.

189. Exact approximation-bound admission and source rows

The inherited approximation operation is accepted exactly when the optimum cell count is positive and no greater than the candidate cell count. This equivalence preserves the original signed-integer input domain and its rejection behavior. Each successful part reconstructs the exact optimum-to-candidate ratio by cross multiplication; its numerator is positive and bounded by its denominator. A retained bound record

identifies both original objective rows. Extending those rows preserves the candidate, optimum and exact part without changing the objective or introducing a tolerance. These results establish the bound component of the approximation formalisation.

For the original signed integer inputs c, o, successful approximation is characterized without restricting its input domain:

$$\mathrm{exactApproximation}(c, o) = \mathrm{ok}\, p \iff \exists C, O \in \mathbb{N},\ c = C \wedge o = O \wedge 0 < O \leq C \wedge p = \mathrm{normalizePart}(O, C).$$

Here natural values are embedded in the original integer type, and the normalization carries the displayed positivity and bound witnesses. Negative inputs, zero optimum, or optimum greater than candidate retain the original rejection. For accepted $p = n/d$, normalization proves

$$Od = nC, \qquad 0 < n \leq d, \qquad C > 0.$$

Thus the accepted part is exactly the optimum-to-candidate ratio.

A bound record additionally retains objective rows (k, C) and (k_*, O), their membership in the original row list J, and the accepted approximation result. These witnesses give

$$(k, C) \in J, \quad (k_*, O) \in J, \quad 0 < O \leq C, \quad Od = nC.$$

Appending rows to J transports these membership witnesses while leaving k, k_*, C, O, p unchanged. This construction preserves the original objective values and ratio.

> Source, proof and provenance: V3 APPROXIMATE; V3 evaluate_expression; SFT.Computation.ApproximateBound.admission; SFT.Computation.ApproximateBound.reconstruct; Timeline: Turn 243, line 2506.

190. Approximation with complete objective and execution ledgers

The exact objective observation binds the feasible candidate and held optimum to their original cell counts. Approximation partitions every original objective distinction into retained or lost records; the partition is exhaustive and disjoint. Loss requires unequal original values and equal approximate values. Identity loses no original distinction, and an admitted downstream composition cannot restore a distinction it has closed. Preserving the original row images preserves every earlier decision under extension.

The same candidate and optimum counts generate the original exact oriented error row. A result certificate retains original feasibility and preference admission, complete objective rows, the actual state path, every registered numerical observation and the exact terminal lookup. Native reconstruction proves that its error belongs to that candidate and reference. The resource ledger reproduces the entire state trace, supplied occupied support and exact counts, including the initial observation. Identity, serial continuation and successor preserve the recorded interfaces; the successor may carry a different candidate with the same reference. These finite certificates introduce no tolerance, metric or conventional approximation algorithm. All 256 original candidates, witnesses, controls and complete native program fields and hashes agree with pinned V3, completing the Fold Approximation-Algorithm Law.

Let D be the original ledger of objective distinctions, and let f and g be the exact and approximate observations. The two filtered ledgers are

$$L = [(a, b) \in D : g(a) = g(b)], \qquad K = [(a, b) \in D : g(a) \neq g(b)].$$

Every member of D belongs to L or K, and no pair belongs to both. Since D already records exact differences,

$$(a,b) \in L \Longrightarrow f(a) \neq f(b) \wedge g(a) = g(b).$$

Taking $g = f$ gives no loss. Equality of the approximate images remains equality under the admitted downstream observation composition, so lost distinctions remain lost; consequently any distinction kept after composition was kept before it. Extension preserves either decision when the original distinction remains present and both approximate images remain unchanged.

For a retained bound record with optimum O and candidate C, define exact positive reference and candidate values by normalizing $O/1$ and $C/1$. Their singleton numerical ledger is

$$\mathrm{exactErrorLedger}([O],[C]) = \mathrm{ok}[(\text{cell-1}, \mathrm{err}(O,C))],$$

where the displayed numbers denote those normalized positive constructors. The result certificate retains feasibility and preference admission, objective coverage and domain, the uneliminated optimum, both admitted observations, the original state path and all numerical records. In particular, its exact observation satisfies

$$f(k) = \mathrm{some}(\mathrm{toString}\, C), \qquad f(k_*) = \mathrm{some}(\mathrm{toString}\, O).$$

If its state trace is $B +\!\!+ [t]$, terminal lookup appends precisely the candidate's value and error record to the prior observations. Therefore

$$\mathrm{map}(\mathrm{fst}, Q) = B +\!\!+ [t], \qquad |Q| = |a| + 1.$$

Its accepted resource ledger gives

$$\mathrm{map}(\mathrm{snd}, \ell.\,\mathrm{timeTrace}) = B +\!\!+ [t], \quad \ell.\,\mathrm{spaceSupport} = H, \quad \ell.\,\mathrm{timeCount} = |a| + 1, \quad \ell.\,\mathrm{spaceCount} = |H|.$$

Identity and concatenation follow from the original path and observation constructors. A changed-candidate successor additionally carries equality of the old and new reference values, the actual next transition, and the new terminal lookup; these witnesses give the new error row while retaining all previous observations.

> Source, proof and provenance: V3 APPROXIMATE; V3 evaluate_expression; SFT.Computation.ApproximateDistinctions.lossMeaning; SFT.Computation.ApproximateDistinctions.keptMeaning; SFT.Computation.ApproximateError.referenceRatio; SFT.Computation.ApproximateError.candidateRatio; SFT.Computation.ApproximateTrace.candidateFeasible; SFT.Computation.ApproximateTrace.optimumUneliminated; Timeline: Turn 243, line 2506.

191. Generated program syntax and complete occurrence records

The formation checker accepts exactly the generated value, name and binary join trees. Positive exact literal payloads and held names remain explicit. Rejection is equivalent to a constructive malformed derivation identifying an invalid literal, unsupported constructor or malformed child. Syntax membership imposes no binding environment: a well-formed name may remain unbound under execution.

The tree ledger retains the root and every child occurrence with its complete left/right address and exact subtree payload. Membership is equivalent to successful focus at that address, in both directions; one address identifies one subtree. Every occurrence of an accepted tree is formed and has a registered constructor. An invalid occurrence rejects the entire tree. The literal base has its identity record. Joining trees preserves both complete ledgers with their address prefixes, and generated contexts compose by retaining the held subtree. All 256 original candidates, the three exact semantics witnesses, controls and complete native program fields and hashes agree with pinned V3, completing the Fold Program-Syntax Law.

The formation recursion is

$$F(\mathrm{value}(p)) = \mathrm{true}\ (p > 0), \qquad F(\mathrm{name}(n)) = \mathrm{true}, \qquad F(\mathrm{join}(a,b)) = F(a) \wedge F(b).$$

Foreign, zero or negative literals and unsupported constructors return false. Induction on the expression and on the corresponding constructors proves both exact characterizations

$$F(e) = \mathrm{true} \iff \mathrm{Generated}(e), \qquad F(e) = \mathrm{false} \iff \mathrm{Malformed}(e).$$

A malformed join retains a malformed left or right child witness. Formation is independent of evaluation bindings: $F(\mathrm{name}(n)) = \mathrm{true}$, while evaluating that name in the empty environment returns the original unbound-name error.

For a leaf e, the occurrence ledger is $[([], e)]$. At a join it is

$$T(\mathrm{join}(a,b)) = [([], \mathrm{join}(a,b))] +\!\!+ \mathrm{lift}_L(T(a)) +\!\!+ \mathrm{lift}_R(T(b)),$$

where each lift prefixes the corresponding direction to every address. Induction proves the complete occurrence equivalence

$$(p, s) \in T(e) \iff \mathrm{focus}(e, p) = \mathrm{some}\, s.$$

Determinacy of focus implies that two rows with the same address have the same subtree. For a formed expression, contextual decomposition makes every retained subtree formed; its constructor is therefore value, name or join. Conversely, a retained malformed subtree contradicts whole-tree formation and yields rejection. The literal base and both lifted child ledgers remain explicit, and plugging a formed expression into a generated context preserves formation.

> Source, proof and provenance: V3 SYNTAX; V3 evaluate_expression; SFT.Computation.SyntaxFormation.membership; SFT.Computation.SyntaxFormation.completeTree; Timeline: Turn 244, line 2513.

192. Complete name occurrences and registered scope

Every name occurrence in the registered expression tree has a retained structural address, exact label and original environment lookup. Ledger membership is equivalent to a native occurrence derivation. The ledger contains exactly as many records as the tree contains names; free and bound records form an exhaustive disjoint partition. A bound occurrence reconstructs its exact environment row, and a free occurrence has no registered lookup. Preserving lookup results preserves the entire occurrence ledger.

Every successful evaluation resolves every name occurrence to its original value. Conversely, resolving every occurrence in a generated expression constructs its evaluation derivation. The original unique-binding admission remains part of executable evaluation. These results establish the name-occurrence and scope component of the binding formalisation without assigning a new binding grammar.

For environment ρ, address p, and expression e, the name ledger is generated by

$$N_\rho(p, \mathrm{value}(v)) = [], \qquad N_\rho(p, \mathrm{name}(x)) = [(p, x, \mathrm{lookupFirst}(\rho, x))],$$

$$N_\rho(p, \mathrm{join}(a, b)) = N_\rho(p +\!\!+ [L], a) +\!\!+ N_\rho(p +\!\!+ [R], b).$$

Unsupported constructors contribute no name rows. The inductive occurrence judgment has precisely the name, left-child and right-child constructors, giving

$$r \in N_\rho(p, e) \iff \mathrm{Located}(\rho, p, e, r), \qquad r.\,\mathrm{value} = \mathrm{lookupFirst}(\rho, r.\,\mathrm{label}).$$

Structural induction also gives $|N_\rho(p, e)| = \mathrm{nameCount}(e)$. Filtering by whether the lookup is some value or none partitions every occurrence into bound or free rows; the two conditions cannot both hold. A successful lookup supplies the original environment row, while a free row has lookup none. Pointwise equality of lookups in two environments gives equality of their entire occurrence ledgers.

Induction over successful evaluation proves every located name has a value. Conversely, induction over generated syntax, resolving each located name, constructs

$$\big(\mathrm{Generated}(e) \wedge \forall p, r, \mathrm{Located}(\rho, p, e, r) \Rightarrow \exists v,\ r.\,\mathrm{value} = \mathrm{some}\, v\big) \Longrightarrow \exists v, \mathrm{Evaluation}(\rho, e, v).$$

The distinct-binding guard is additionally required when replaying this derivation through the executable evaluator.

> Source, proof and provenance: V3 BINDING-SUBSTITUTION; V3 evaluate_expression; SFT.Computation.NameScope.completeLedger; SFT.Computation.NameScope.scopePartition; Timeline: Turn 248, line 2544.

193. Binding-extension formation and exact substitution behavior

The SEMX extension supplies value, name, join, let and four-branch same term shapes. Its formation decision is equivalent to a native derivation retaining the original bound-name list; formed terms have no free name relative to that list. Free-name set membership preserves the original initializer and body scopes. Value payloads are carried unchanged. Whole-tree renaming retains the source operation, and identity renaming preserves the complete term.

The original fresh-name search examines u, v, w and z in order, excluding exactly the free names of the body and replacement and the substituted name. Every selected name has that provenance and those exclusions. Exhaustion holds exactly when all four candidates are unavailable.

The translated substitution preserves literal values, exact name matches, ordered join and same recursion, the held-binder stop in the body, substitution into the initializer and conflicting-binder renaming before descent. Accumulated name mappings are proved equivalent to the source whole-body rename operation for arbitrary generated terms. Fresh-search failure precedes initializer substitution as in the source. The original let-y/name-x witness reconstructs its u-bound result. These are exact behavioral correspondence results; a general capture-preservation result is not inferred from the witness.

For bound-name list B, the SEMX formation equations include

$$W_B(\mathrm{value}(v)) = \mathrm{true}, \quad W_B(\mathrm{name}(x)) = [x \in B], \quad W_B(\mathrm{join}(a, b)) = W_B(a) \wedge W_B(b),$$

$$W_B(\mathrm{let}(x,a,b)) = W_B(a) \wedge W_{B +\!\!+ [x]}(b), \qquad W_B(\mathrm{same}(a,b,c,d)) = ((W_B(a) \wedge W_B(b)) \wedge W_B(c)) \wedge W_B(d).$$

Formation constructors mirror these equations, yielding $W_B(t) = \mathrm{true} \iff \mathrm{Formation}(B,t)$. The free-name recursion uses the same initializer/body scopes, with disjunction instead of conjunction; induction on formation gives $\mathrm{free}(B,x,t) = \mathrm{false}$ for every x.

Fresh selection scans $[u,v,w,z]$ in order with predicate

$$\mathrm{available}(b,r,x,c) \iff \neg\,\mathrm{free}([],c,b) \wedge \neg\,\mathrm{free}([],c,r) \wedge c \neq x.$$

A selected name belongs to this list and satisfies all three exclusions. The search returns the original exhaustion error exactly when every candidate fails availability.

Write $S^r_x(t)$ for the original partial substitution, with sequential error propagation. Literals remain fixed; a name becomes r exactly when its label equals x; join and same recurse in their original child order. For a let term, its branches are

$$S^r_x(\mathrm{let}(x,a,b)) = \mathrm{let}(x, S^r_x(a), b),$$

$$S^r_x(\mathrm{let}(y,a,b)) = \mathrm{let}(y, S^r_x(a), S^r_x(b)) \quad (y \neq x,\ \neg\,\mathrm{free}([],y,r)),$$

and, for a conflicting $y \neq x$ with selected fresh z,

$$S^r_x(\mathrm{let}(y,a,b)) = \mathrm{let}(z, S^r_x(a), S^r_x(\mathrm{rename}(y,z,b))).$$

These equations denote successful constructor assembly inside the source's sequential error operation. In the conflicting branch the fresh search runs before either recursive substitution. Composition of whole-tree name maps,

$$\mathrm{mapNames}(f, \mathrm{mapNames}(g,t)) = \mathrm{mapNames}(f \circ g, t),$$

and its substitution execution counterpart justify the accumulated-map implementation of the exact whole-body rename. No alternate renaming convention is introduced.

> Source, proof and provenance: V3 BINDING-SUBSTITUTION; V3 well_formed; V3 free_names; V3 rename_bound; V3 substitute; SFT.Computation.SemBindingTerms.formationMeaning; SFT.Computation.SemBindingTerms.formedHasNoFreeName; SFT.Computation.SemBindingTerms.freshBoundary; SFT.Computation.SemSubstitution.mapCompose; SFT.Computation.SemSubstitution.renamedExecution; Timeline: Turn 246, line 2528.

194. Capture preservation at the evaluator substitution boundary

The original evaluator enters let substitution only after the initializer is a value. This source guard is equivalent to the value-constructor shape. On that exact call domain, structural value replacement agrees with the original substitution operation for every generated body and cannot fail through fresh-name exhaustion. Formation under the discharged binding produces formation in the remaining scope, including shadowing and initializer recursion.

The complete binder ledger retains every original structural address and binder name unchanged. Substitution removes the discharged free name and preserves every other free name. Thus all evaluator substitution calls preserve the stated binder identities and free-name distinctions. The standalone helper

admits open replacements outside this evaluator call boundary; these results do not assert its unrestricted capture behavior.

The evaluator's ready-initializer guard is

$$\mathrm{isValue}(a) = \mathrm{true} \iff \exists v,\ a = \mathrm{value}(v).$$

For this replacement shape, define R_x^v structurally: values are unchanged, matching names become $\mathrm{value}(v)$, and join and same recurse through all children. Its let equation is

$$R_x^v(\mathrm{let}(y, a, b)) = \mathrm{let}\big(y, R_x^v(a), \mathrm{if}\ y = x\ \mathrm{then}\ b\ \mathrm{else}\ R_x^v(b)\big).$$

Because a value has no free name, the conflicting-binder branch cannot be entered. Structural induction proves the exact agreement and formation result

$$S_x^{\mathrm{value}(v)}(t) = \mathrm{ok}(R_x^v(t)), \qquad \mathrm{Formation}(x :: B, t) \Longrightarrow \mathrm{Formation}(B, R_x^v(t)).$$

The shadowing case keeps the body intact and transports its scope after removing the redundant discharged binding.

Let $\mathrm{binders}(t)$ retain every let binder with its full structural address. The same recursion proves

$$\mathrm{binders}(R_x^v(t)) = \mathrm{binders}(t), \qquad \mathrm{free}(B, x, R_x^v(t)) = \mathrm{false},$$

$$y \neq x \Longrightarrow \mathrm{free}(B, y, R_x^v(t)) = \mathrm{free}(B, y, t).$$

Combining value-shape extraction with these identities constructs a successful substitution result for every evaluator call satisfying the original guard, with all binder identities and all other free-name decisions preserved.

> Source, proof and provenance: V3 BINDING-SUBSTITUTION; V3 substitute; V3 is_value; V3 step; SFT.Computation.SemValueSubstitution.sourceAgreement; SFT.Computation.SemValueSubstitution.evaluatorCallSafety; Timeline: Turn 247, line 2536.

195. Canonical replacement in the registered expression grammar

Binding identity is the exact name admitted by the original environment uniqueness check. Successful admission of a new target binding proves its identity is absent from the surrounding environment; conversely, a fresh identity constructs an admitted extension. The new lookup returns its recorded value, while every other binding is unchanged.

Structural replacement has a complete native derivation for each value, name and join. The derivation and computed result reconstruct one another, preserving original payloads and complete child trees. For an arbitrary open replacement evaluated in the surrounding environment, substitution reconstructs the original evaluation under the discharged target binding. Every resulting occurrence resolves to that exact surrounding environment row. No binder or presentation alias is introduced by replacement. Formation, identity replacement, sequential composition and generated join successor are preserved. All 256 original candidates, witnesses, controls and complete native program fields and hashes agree with pinned V3, completing the registered Fold Binding and Substitution Law.

For the value/name/join grammar, canonical replacement obeys

$$[x := r]\,\mathrm{value}(v) = \mathrm{value}(v), \qquad [x := r]\,\mathrm{name}(y) = \begin{cases} r, & y = x, \\ \mathrm{name}(y), & y \neq x, \end{cases}$$

$$[x := r]\,\mathrm{join}(a, b) = \mathrm{join}([x := r]a, [x := r]b).$$

The executable operation and its value, matching-name, other-name and join derivation constructors reconstruct one another. Generated replacement and original syntax imply generated output. Identity replacement gives $[x := \mathrm{name}(x)]e = e$.

Under the original uniqueness admission, adjoining (x, v) requires $x \notin \mathrm{names}(\rho)$; uniqueness of ρ and this absence also suffice. Its lookup equations are

$$\mathrm{lookupFirst}((x, v) :: \rho, x) = \mathrm{some}\, v, \qquad y \neq x \Longrightarrow \mathrm{lookupFirst}((x, v) :: \rho, y) = \mathrm{lookupFirst}(\rho, y).$$

Induction on evaluation in the extended environment proves binding discharge:

$$\rho \vdash r \Downarrow v \ \wedge\ ((x, v) :: \rho) \vdash e \Downarrow w \Longrightarrow \rho \vdash [x := r]e \Downarrow w.$$

In the matching-name case the inserted derivation supplies v; other names use the unchanged lookup; the join case combines the two induction results. Replaying under the admitted environment gives the same implication for successful executable evaluations. The occurrence theorem then resolves every resulting name to its surrounding environment row, even when r itself contains names. Sequential replacement preserves a result when each replacement evaluates to its corresponding retained binding value.

> Source, proof and provenance: V3 BINDING-SUBSTITUTION; V3 evaluate_expression; SFT.Computation.NameReplacement.exactReplacement; SFT.Computation.NameReplacement.nativeDischarge; Timeline: Turn 248, line 2544.

196. Complete source evaluation traces

The evaluation record retains each entered expression and exact environment, every visited recursive child, and its terminal value or original failure. The native execution derivation preserves duplicate-binding rejection before descent and left-before-right evaluation. A first-child failure stops the join; a second-child failure retains the first completed trace. Successful joins retain both complete traces and their exact sum.

Computation and native derivation reconstruct one another. Every recorded result equals the original evaluator result, and two valid executions of the same expression in the same environment have identical results and complete event lists. Each accepted evaluation has such a trace. The literal base retains its identity record; join composition and successor preserve every visited child record, its syntax and its binding environment. All 256 original candidates, operational witnesses, controls and full program fields and hashes agree with pinned V3, completing the Fold Evaluation Law.

For environment ρ, expression e, child-event list C, and result r, define the complete frame

$$\mathrm{frame}(\rho, e, C, r) = [\mathrm{enter}(\rho, e)] +\!\!+\, C +\!\!+\, [\mathrm{leave}(\rho, e, r)].$$

Duplicate bindings return their original error with $C = []$ before any descent. Under distinct bindings, leaves use their original evaluator result and an empty child list. For a join, if the left run returns $(\mathrm{error}\, q, L)$, the parent retains only L; if it returns $(\mathrm{ok}\, a, L)$, the right run is evaluated and both lists $L +\!\!+ R$ are retained. A right error is propagated; two successful values yield $\mathrm{ok}(\mathrm{add}(a, b))$.

The native execution judgment has one constructor for each of these branches. Replay and structural reconstruction establish

$$\mathrm{Execution}(\rho, e, r, T) \iff \mathrm{run}(\rho, e) = (r, T), \qquad \mathrm{Execution}(\rho, e, r, T) \Longrightarrow \mathrm{eval}(\rho, e) = r.$$

Consequently two execution certificates for the same environment and expression satisfy

$$\mathrm{Execution}(\rho, e, r, T) \wedge \mathrm{Execution}(\rho, e, r', T') \Longrightarrow r = r' \wedge T = T'.$$

Every successful evaluation derivation under distinct bindings supplies this complete trace. Frame concatenation preserves every visited child event, including the completed left trace when the right child fails; it does not insert events for an unvisited child.

> Source, proof and provenance: V3 EVALUATION; V3 evaluate_expression; SFT.Computation.SymbolicEvaluation.duplicateEnvironment; SFT.Computation.SymbolicEvaluation.replay; SFT.Computation.EvaluationTrace.reconstruct; SFT.Computation.EvaluationTrace.replay; Timeline: Turn 249, line 2552.

197. Operational and compositional meaning correspondence

Constructor-indexed meaning records retain the original syntax, environment, exact result and ordered failure clauses. Soundness transforms each complete operational execution into its meaning record. Adequacy reconstructs a complete operational execution from every meaning record. The result map is equivalent to the original evaluator, and meaning is unique. Consequently no admitted execution and corresponding meaning can disagree on their result for every expression and environment of the source evaluator, including its original failure outcomes.

The positive literal supplies the base correspondence. Join assembly preserves both complete component traces and their exact sum. Equality of meaning observations is equivalent to equality of the original result, including failure. Identity, serial composition and every context generated by hole and left/right join insertion preserve this observation relation. Existing complete occurrence and execution ledgers preserve the original syntax and binding identities throughout assembly. All 256 candidates, source operational witnesses, controls and complete native program fields and hashes agree with pinned V3, completing the Fold Operational and Denotational Correspondence Law.

The meaning judgment uses the same duplicate-binding, leaf, left-failure, right-failure and successful-join clauses as the source execution. In particular, under distinct bindings,

$$\frac{\rho \models a \Downarrow \mathrm{ok}\, u \quad \rho \models b \Downarrow \mathrm{ok}\, v}{\rho \models \mathrm{join}(a, b) \Downarrow \mathrm{ok}(\mathrm{add}(u, v))}.$$

A left failure determines the parent result without a right premise; a right failure requires the successful left premise. Induction on execution gives soundness, and induction on these meaning constructors rebuilds the corresponding complete event frame, giving adequacy:

$$\mathrm{Meaning}(\rho, e, r) \iff \exists T, \mathrm{Execution}(\rho, e, r, T) \iff \mathrm{eval}(\rho, e) = r.$$

Two meanings therefore have equal results. An execution result and a meaning result for the same input must also agree.

Define observational equality over the complete result type, including errors:

$$e \equiv_\rho f \iff \forall r,\ \mathrm{Meaning}(\rho, e, r) \iff \mathrm{Meaning}(\rho, f, r).$$

The exact-meaning equivalence proves

$$e \equiv_\rho f \iff \mathrm{eval}(\rho, e) = \mathrm{eval}(\rho, f).$$

Identity and transitivity follow directly; case analysis of the ordered join results gives join congruence, and induction on a context gives

$$e \equiv_\rho f \Longrightarrow \mathrm{plug}(C, e) \equiv_\rho \mathrm{plug}(C, f).$$

Successful join assembly retains the concatenated complete child traces inside the parent frame, so correspondence preserves the same syntax and environments as operational evaluation.

The correspondence theorem quantifies over the expression and environment themselves. Contextual meaning is proved by induction on the context: the hole retains the given equality, and either join constructor combines the induction hypothesis with identity on the untouched sibling. Thus the context depth is arbitrary; the finite candidate census is a separate comparison of program forms. exactMeaning; contextualMeaning.

> Source, proof and provenance: V3 OPERATIONAL-DENOTATIONAL; V3 evaluate_expression; SFT.Computation.MeaningCorrespondence.soundness; SFT.Computation.MeaningCorrespondence.adequacy; SFT.Computation.EvaluationTrace.reconstruct; SFT.Computation.EvaluationTrace.replay; Timeline: Turn 250, line 2560.

198. Positive semantic-value typing and preservation

Within the registered value/name/join grammar, the typing derivation retains a complete premise tree for positive exact literals, exact positive name bindings and binary composition. The typing decision is equivalent to this derivation and implies generated syntax. Every typed expression evaluates to a positive exact value under the original unique-environment admission, and its complete operational trace reconstructs. Structural substitution preserves this typing relation; evaluating the substituted expression retains a positive result and complete trace.

Rejected typing checks have explicit mismatch derivations: invalid literal payloads, absent or nonpositive name bindings, unsupported constructors, or a mismatched child. Rejection and mismatch derivation are equivalent, and a typed expression cannot have such a mismatch. These judgments characterize the positive semantic-value type; the original evaluator and its raw signed environment behavior remain unchanged. They establish the positive typing and preservation component of the Type Theory formalisation.

The positive semantic-value judgment has exactly three rules:

$$\frac{p > 0}{\rho \vdash \mathrm{value}(p) : +}, \qquad \frac{\mathrm{lookupFirst}(\rho, x) = \mathrm{some}(\mathrm{positive}\, p)}{\rho \vdash \mathrm{name}(x) : +}, \qquad \frac{\rho \vdash a : + \quad \rho \vdash b : +}{\rho \vdash \mathrm{join}(a, b) : +}.$$

The decision procedure checks the positive constructor for literals and looked-up names and conjoins the two child decisions at a join. Structural reconstruction and replay give

$$\mathrm{checks}(\rho, e) = \mathrm{true} \iff \rho \vdash e : +.$$

Induction on typing supplies generated syntax and a positive evaluation derivation. With the original distinct-binding condition, this yields

$$\exists p > 0,\ \mathrm{eval}(\rho, e) = \mathrm{ok}(\mathrm{positive}\, p) \wedge \mathrm{Execution}(\rho, e, \mathrm{ok}(\mathrm{positive}\, p), (\mathrm{run}(\rho, e)).\mathrm{events}).$$

Typing substitution is the structural implication

$$\rho \vdash r : + \ \wedge \ ((x, v) :: \rho) \vdash e : + \Longrightarrow \rho \vdash [x := r]e : +.$$

Applying positive evaluation to this conclusion supplies the substituted expression's positive result and complete trace.

The mismatch constructors identify absent, zero or negative literals; missing, zero or negative name bindings; unsupported constructors; or a mismatched child. They satisfy

$$\mathrm{checks}(\rho, e) = \mathrm{false} \iff \mathrm{Mismatch}(\rho, e).$$

A typed term and a mismatch would force the same Boolean decision to be both true and false, so they are incompatible. This judgment leaves the original evaluator's signed name values unchanged.

> Source, proof and provenance: V3 TYPE; V3 evaluate_expression; SFT.Computation.StructuralTyping.checkMeaning; SFT.Computation.StructuralTyping.preservesEvaluation; Timeline: Turn 252, line 2576.

199. Generated type support and exact interface derivations

Types retain the original source-bound predicate on a finite generated carrier. Extending the carrier while preserving earlier verdicts retains the entire prior ordered type support and adds exactly the admitted new rows. Product extension retains the old joint support and every new pair. Sum extension retains the original held left/right labels and their order. These are generated source inputs, with no additional scientific parameter.

An interface derivation records every original composition call, its two arrows, the intermediate result or its first rejection. Replay and reconstruction establish equivalence with the source path fold. Successful composition requires exact interface identity; unequal interfaces produce the original rejection. Serial assembly preserves complete component traces, and a rejected prefix remains rejected under extension. The reconstructed flattened trace contains every earlier and appended component record. Together with positive typing, substitution and evaluation preservation, these results formalise the registered structural-admissibility requirements. All 256 original TYPE candidates, witnesses, controls and complete native program fields and hashes agree with pinned V3, completing the Fold Type-Theory Law.

For an admitted type support T selected from carrier C, extending the carrier by E and preserving every old admission decision gives the exact list equation

$$T' = T +\!\!+ \mathrm{filter}(\mathrm{allowed}_{\mathrm{new}}, E).$$

Thus earlier support members persist with their order. The generated product and held-label sum obey

$$\mathrm{product}(L +\!\!+ E, R) = \mathrm{product}(L, R) +\!\!+ \mathrm{product}(E, R),$$

$$\mathrm{sum}(L, R +\!\!+ E) = \mathrm{sum}(L, R) +\!\!+ [(\text{right-held}, e) : e \in E].$$

The singleton product extension includes exactly $[(n, r) : r \in R]$.

For a retained arrow h, interface execution over the remaining arrow list has the base rule $I(h, [], \mathrm{ok}\, h)$, a rejection rule when the next composition fails, and the successor rule

$$\frac{\mathrm{compose}(h, n) = \mathrm{ok}\, j \quad I(j, R, r)}{I(h, n :: R, r)}.$$

Replay and reconstruction yield

$$I(h, R, r) \iff \mathrm{foldRest}(h, R) = r.$$

A successful first composition requires $h.\,\mathrm{target} = n.\,\mathrm{source}$; inequality produces the original exact-interface rejection. Induction over successful execution gives

$$r.\,\mathrm{trace} = h.\,\mathrm{trace} +\!\!+ \mathrm{flatMap}(\mathrm{trace}, R).$$

Every original and appended component event is retained. Serial executions compose through the shared resulting arrow; a failed prefix stays failed under any appended suffix, because the rejected constructor performs no further composition.

> Source, proof and provenance: V3 TYPE; V3 evaluate_expression; SFT.Computation.TypeInterfaces.exactRun; SFT.Computation.TypeInterfaces.completeTrace; Timeline: Turn 252, line 2576.

200. Program equivalence on registered observations

For each input in the common finite support, the ledger retains both complete source executions and their registered observation labels. Ledger membership reconstructs the original input and both execution derivations. The equivalence certificate is true exactly when observations agree at every registered input. A failed certificate constructs an input with unequal observations.

Equal and unequal observation rows form exhaustive disjoint closed and retained distinction ledgers. Equivalence closes every observation row, while a retained distinction refutes equivalence. The complete underlying traces remain available in both cases. Identity, symmetry and serial equivalence composition preserve the declared observation relation. Support assembly and successor retain every earlier execution record; extending support requires agreement on its new inputs. A declared post-map preserves already equal observations. These results respect the original observation and input boundary. All 256 original candidates, witnesses, controls and complete native program fields and hashes agree with pinned V3, completing the Fold Program-Equivalence Law.

Let S be the registered input-environment list and o its observation map on complete executions. For programs a, b, each input row is

$$r_\rho = (\rho, \mathrm{run}(\rho, a), \mathrm{run}(\rho, b), o(\mathrm{run}(\rho, a)), o(\mathrm{run}(\rho, b))).$$

The complete ledger is $[r_\rho : \rho \in S]$, retaining ordering and repetitions. Its checker recursively conjoins observation equality:

$$C([], o, a, b) = \mathrm{true}, \qquad C(\rho :: S, o, a, b) = [o(\mathrm{run}(\rho, a)) = o(\mathrm{run}(\rho, b))] \wedge C(S, o, a, b).$$

Induction gives the exact equivalence

$$C(S, o, a, b) = \mathrm{true} \iff \forall \rho \in S,\ o(\mathrm{run}(\rho, a)) = o(\mathrm{run}(\rho, b)).$$

A false decision reconstructs a member of S where those observations differ. Filtering ledger rows by equality and inequality yields exhaustive, disjoint closed and retained ledgers, respectively. Every row still

reconstructs both complete execution judgments.

Equality supplies identity, symmetry and transitive composition on the same S, o. For support concatenation,

$$\mathrm{ledger}(S +\!\!+ T) = \mathrm{ledger}(S) +\!\!+ \mathrm{ledger}(T).$$

Agreement on both components therefore supplies agreement on the combined support. For any declared post-map h, equality of o-observations implies equality of $h \circ o$-observations by congruence.

> Source, proof and provenance: V3 PROGRAM-EQUIVALENCE; V3 evaluate_expression; SFT.Computation.ProgramEquivalence.exactCertificate; SFT.Computation.ProgramEquivalence.distinguishingInput; Timeline: Turn 253, line 2584.

201. Generated descent and terminal coverage

Every recursive call of the registered expression evaluator follows a strict child of its generated syntax tree. Structural induction constructs accessibility, and the retained node count decreases at each call. The call relation preserves the original duplicate-binding guard and admits the right child only after successful left evaluation. Every expression and input reconstructs a complete execution ending in its exact value or original failure.

For the original transition system, a retained descent certificate decreases on every transition. Each complete path has length bounded by its initial rank; native execution reconstructs both that bound and its exact state trace. Serial assembly and successor preserve the complete records and bound. A nonempty return cycle contradicts strict descent, and the registered loop provides an explicit counterexample to the existence of such a certificate. The structural evaluator conclusion applies to every expression in its constructor type. For the separate transition-system conclusion, the rank certificate supplies the required strict decrease on every transition. All 256 original candidates, witnesses, controls and complete native program fields and hashes agree with pinned V3, completing the Fold Termination Law.

The evaluator's structural measure is

$$n(\mathrm{value}(v)) = n(\mathrm{name}(x)) = n(\mathrm{outside}(t)) = 1, \qquad n(\mathrm{join}(a,b)) = n(a) + n(b) + 1.$$

Each child has strictly smaller measure. Structural induction constructs accessibility for the child relation. The actual call relation additionally requires distinct bindings, and a right call requires a successful left evaluation. Every actual call is a child step, so it also decreases n; duplicate bindings admit no recursive call. Complete execution reconstruction gives

$$\forall \rho, e,\ \exists r, C,\ \mathrm{Execution}(\rho, e, r, \mathrm{frame}(\rho, e, C, r)) \wedge \mathrm{eval}(\rho, e) = r.$$

For the original transition system M, a rank certificate is the explicit property

$$\mathrm{Ranked}(M, \nu) \iff \forall s, a, t,\ \mathrm{step}(M, s, a) = \mathrm{ok}\, t \Rightarrow \nu(t) < \nu(s).$$

Path induction yields, for a path from s to t with action list A,

$$\nu(t) \le \nu(s), \qquad |A| \le \nu(s),$$

and $|A| > 0$ gives $\nu(t) < \nu(s)$. The successor argument uses the next strict decrease and the bound on the remaining path. A nonempty return path would therefore imply $\nu(s) < \nu(s)$, excluding every such rank

certificate. The original loop supplies that counterexample. Serial and one-step path constructors retain the entire concatenated trace and the same initial-rank bound.

For any transition system and supplied rank ν satisfying Ranked, the path-bound theorem excludes a path with $|A| > \nu(s)$. This is the precise continuation exclusion obtained from the rank premise. The evaluator's complete terminal-coverage theorem instead follows from structural recursion and includes both returned values and original errors. terminalCoverage; noLongContinuation.

> Source, proof and provenance: V3 TERMINATION; V3 evaluate_expression; SFT.Computation.TerminationDescent.generatedWellFounded; SFT.Computation.TerminationDescent.nativeBound; Timeline: Turn 254, line 2592.

202. Correctness against the registered specification

The specification retains its exact precondition, postcondition on complete execution records, and partial or total termination declaration. Partial correctness establishes the postcondition for each retained execution of an admitted input. Total correctness additionally retains execution existence, supplied here by the finite registered evaluator. The complete-support certificate is equivalent to the declared correctness relation in either mode. Rejection reconstructs an admitted input whose postcondition fails.

Every input row retains its complete execution, specification decisions and termination declaration. Failure rows are exactly those carrying original runtime errors; their acceptability remains governed by the registered postcondition. Support assembly and successor preserve every earlier row. Specification consequence retains the same original execution under proved precondition and postcondition implications. Serial agreement connects original path certificates through their exact middle interface and retains both component traces. All 256 original candidates, witnesses, controls and complete native program fields and hashes agree with pinned V3, completing the Fold Program-Correctness Law.

For precondition P, postcondition Q on complete executions, program e, and registered support S, partial correctness is

$$\forall \rho \in S,\ P(\rho) = \mathrm{true} \Rightarrow \forall r, T,\ \mathrm{Execution}(\rho, e, r, T) \Rightarrow Q(\rho, (r, T)) = \mathrm{true}\,.$$

Total correctness conjoins existence of such an execution for every admitted input. Here that existence is supplied constructively by the finite evaluator's complete execution theorem, including its original error outcomes.

The certificate is

$$K([], P, Q, e) = \mathrm{true},$$

$$K(\rho :: S, P, Q, e) = (\mathrm{if}\ P(\rho)\ \mathrm{then}\ Q(\rho, \mathrm{run}(\rho, e))\ \mathrm{else}\ \mathrm{true}) \wedge K(S, P, Q, e).$$

Induction, together with uniqueness of complete execution, proves that $K = \mathrm{true}$ is equivalent to the correctness judgment selected by the specification's partial or total mode. A false certificate constructs

$$\rho \in S, \qquad P(\rho) = \mathrm{true}, \qquad Q(\rho, \mathrm{run}(\rho, e)) = \mathrm{false}\,.$$

The row ledger retains the input, complete execution, both decisions and mode. Its failure filter selects exactly rows whose result is an original error; the postcondition determines whether such a row is acceptable.

Consequence preserves an execution when the new precondition implies the old one on S, and the old postcondition implies the new one for that same execution. Serial path agreement uses the first path's established middle condition as the second path's premise and reconstructs the concatenated action and state records.

> Source, proof and provenance: V3 CORRECTNESS; V3 evaluate_expression; SFT.Computation.ProgramCorrectness.exactCertificate; SFT.Computation.ProgramCorrectness.failedCertificate; Timeline: Turn 255, line 2600.

203. Generated formal specification relations

A specification retains its generated input and output carriers and exact allowed relation under the original relation-admission guard. Membership proves both carrier identities. The complete observation certificate is true exactly when every observed row is allowed; falsification retains every observed row excluded from that relation. Refinement is allowed-row inclusion, preserving verified observations through identity and composition. Removing an allowed row produces an explicit falsification at the new boundary.

Invariant checking covers every state of the retained source trace and follows from the admitted initial invariant and transition preservation. A finite liveness witness retains original input admission, a complete executable path and its visited registered goal. A terminal obligation retains the original halted execution and normal terminal state. Both reconstruct native execution, and serial extension preserves the earlier goal and component records. The generated relation supplies the postcondition of the existing correctness checker; every verified execution therefore reconstructs an allowed pair in the original input and output carriers. All 256 candidates, witnesses, controls and full native program fields and hashes agree with pinned V3, completing the Fold Formal-Specification Law.

An admitted relation specification consists of input carrier I, output carrier O, and allowed rows R, with the original generated-relation admission. It supplies

$$(i, o) \in R \Longrightarrow i \in I \wedge o \in O.$$

For observed rows V, the checker and falsification ledger are

$$C_R(V) = \bigwedge_{v \in V} [v \in R], \qquad F_R(V) = [v \in V : v \notin R].$$

Thus $C_R(V) = \text{true} \iff \forall v \in V,\ v \in R$, and membership in $F_R(V)$ refutes acceptance. If $R_1 \subseteq R_2$, a certificate against R_1 supplies one against R_2. If a row is removed from R_2 to obtain R_1, its singleton observation remains accepted under R_2 and is an explicit falsification under R_1.

For an invariant J, trace checking is

$$\text{invariantChecks}(J, T) = \text{true} \iff \forall s \in T,\ J(s) = \text{true}\,.$$

An admitted initial J and preservation across every successful original transition give this statement by path induction. Finite liveness retains initial-state admission and a path whose full state trace contains a registered goal. A terminal witness additionally retains a halted path and normal terminal state in the goals. Both reconstruct actual execution; extending an existing path preserves goals already in its retained prefix.

For the original input and output observation maps ι, ω, the induced postcondition is

$$Q(\rho, (r, T)) = [(\iota(\rho), \omega(r, T)) \in R].$$

Applying the correctness certificate to an admitted input and its execution yields this exact allowed pair and hence both original carrier memberships.

> Source, proof and provenance: V3 SPECIFICATION; V3 evaluate_expression; SFT.Computation.FormalSpecification.invariantPreservation; SFT.Computation.FormalSpecification.terminalReconstruction; Timeline: Turn 256, line 2608.

204. Retained program transformation traces

A generated transformation retains the registered rule identity, source syntax, target syntax and exact contextual occurrence at every step. Semantic correspondence follows through arbitrary finite traces by contextual preservation and serial composition of the original evaluation relation. Every successful source evaluation reconstructs a complete native target execution with the same value. The proof retains every rule and both generated endpoints.

The recorded predecessor rows supply a reverse traversal of the complete transformation; a target without its record cannot recover distinct predecessors that share it. Resource evidence remains the original separate resource-ledger operation, retaining the initial observation, each rule event and occupied support. Semantic preservation alone asserts no resource improvement. The complete 256-member grammar, operational witnesses, controls and native program fields agree with pinned V3.

A transformation row retains a rule q and context C, with

$$\mathrm{source}(q, C) = \mathrm{plug}(C, q.\,\mathrm{source}), \qquad \mathrm{target}(q, C) = \mathrm{plug}(C, q.\,\mathrm{target}).$$

Admission retains rule membership, generated context and endpoints, and successful-value equivalence of the rule endpoints for every environment in the registered support. Contextual equivalence gives the same relation for the entire source and target expressions. Induction over the retained transformation trace therefore gives

$$\mathrm{Trace}(R, S, e, L, f) \wedge \rho \in S \Longrightarrow e \sim_{\rho} f.$$

A successful source evaluation then supplies a complete target execution with that same value. Induction also retains admission of every row and generated syntax of both endpoints. Serial traces concatenate through their exact common expression.

The reverse judgment walks recorded target-to-source rows. Induction on the forward trace constructs a reverse traversal returning to the original expression. Without the predecessor record, two distinct predecessors $a \neq b$ sharing output z cannot both be recovered by one function:

$$\neg\exists g,\ g(z) = a \wedge g(z) = b.$$

For initial identity i, the resource event trace is $i :: [q.\,\mathrm{identity} : q \in L]$. Its accepted ledger satisfies

$$\ell.\,\mathrm{timeCount} = |L| + 1, \qquad \ell.\,\mathrm{spaceCount} = |H|,$$

and reconstructs that entire event trace and supplied occupied support H. No resource inequality follows merely from semantic preservation.

> Source, proof and provenance: V3 TRANSFORMATION; V3 evaluate_expression; SFT.Computation.ProgramTransformation.correspondence; SFT.Computation.ProgramTransformation.retainedInverse; Timeline: Turn 257, line 2618.

205. Semantics-preserving compilation relations

The generated compiler relation retains both supplied language grammar carriers and every allowed source-to-target row. Its finite coverage check is equivalent to every source form having a target. A complete compiler certificate checks that coverage and every relation row against the declared runtime interface. It therefore derives a total verified translation relation, with source and target grammar exclusion and rejection of failed semantic checks.

Each observation row reconstructs both complete native executions. The result correspondence composes through an exact intermediate syntax and unchanged runtime input and observation interfaces. Identity and input-support extension preserve the certificate and complete observation records. Overhead is recorded by the original separate source and target resource ledgers, retaining every execution event and occupied support. All 256 grammar candidates and full native program outputs agree with pinned V3.

Let $R \subseteq I \times O$ be the admitted compiler relation. Its coverage check has the exact meaning

$$\mathrm{coverageCheck}(R) = \mathrm{true} \iff \forall s \in I, \exists t,\ (s,t) \in R.$$

The runtime interface retains input environments S, source and target expression maps E_s, E_t, and observation o. Each row check is

$$\mathrm{checked}(s,t) = C(S, o, E_s(s), E_t(t)),$$

using the complete program-equivalence checker. The complete compiler check conjoins coverage with every admitted row check:

$$\mathrm{compilerCheck} = \mathrm{coverageCheck}(R) \wedge \bigwedge_{(s,t)\in R} \mathrm{checked}(s,t).$$

Consequently it supplies, for every $s \in I$, a target $t \in O$ with both $(s,t) \in R$ and equal runtime observations for every input in S. Every observation row reconstructs both complete native executions.

Serial correspondence requires equal input supports, equal observation maps, and exact equality of the first target expression and second source expression at the intermediate form. Transitivity then gives the source-to-final-target equivalence. Support extension conjoins the new input checks and concatenates complete observation ledgers. Separate accepted resource records reconstruct each side's full labeled event list, its event count and its supplied space support; their values are retained independently.

The theorem takes the language relation and runtime interface as arbitrary parameters. Passing the complete checker supplies coverage and a semantic check for each relation row. For each source form, coverage selects a target row; the all-rows semantic conjunct applies to that same row and constructs Verified. Thus total translation follows for every source member of the supplied carrier, with the runtime's input and observation interfaces retained explicitly. verifiedTotal; completeCompilerCheck.

> Source, proof and provenance: V3 COMPILATION; V3 evaluate_expression; SFT.Computation.CompilationRelation.verifiedTotal; SFT.Computation.CompilationRelation.serialCorrespondence; Timeline: Turn 258, line 2626.

206. Machine-checkable program evidence

Program evidence retains the original proof-step grammar, registered premises and rules, specification, complete input boundary and bound program identity. The checker verifies both source and program identities, the complete native inference trace and the actual occurrence of the declared conclusion. The

registered interpretation supplies proved rule and premise soundness and the exact meaning of that conclusion for the retained program and specification. Native soundness then establishes correctness over the complete declared boundary.

Mismatched identities, invalid proof traces and missing conclusions cannot pass the checker. Identity proofs retain an already-held conclusion; serial proofs preserve every step through the exact intermediate proposition state. The complete 256-member grammar and native program comparisons agree with pinned V3.

A boundary B records source and program identities, program e, specification σ, input list S, premises P, rules R, and conclusion c. A proof object p supplies its identities and step list T. The exact checker theorem is

$$\mathrm{check}(B,p)=\mathrm{true} \iff p.\mathrm{sourceId}=B.\mathrm{sourceId} \wedge p.\mathrm{programId}=B.\mathrm{programId} \wedge \mathrm{AcceptedTrace}(R,P,T) \wedge c \in \mathrm{afterSteps}(P,T).$$

The registered interpretation μ retains the proof obligations

$$\forall r \in R,\ (\forall a \in r.\mathrm{premises}, \mu(a)) \Rightarrow \mu(r.\mathrm{conclusion}), \qquad \forall a \in P, \mu(a),$$

$$\mu(c) \Longrightarrow \mathrm{Correct}(S,\sigma,e).$$

Inference-trace induction transports these premise and rule witnesses to every proposition in the final state. Applying it to the retained conclusion gives $\mu(c)$, hence the declared program correctness and its complete-support certificate. The interpretation's soundness witnesses are explicit premises of this construction.

An identity mismatch contradicts the corresponding checker conjunct; an invalid trace or missing conclusion contradicts its own conjunct. With $c \in P$, the empty proof is accepted. For two accepted traces, the second begins at $\mathrm{afterSteps}(P,T_1)$; the final-state composition identity

$$\mathrm{afterSteps}(P,T_1 +\!\!+ T_2)=\mathrm{afterSteps}(\mathrm{afterSteps}(P,T_1),T_2)$$

then constructs the combined proof while retaining every earlier step.

> Source, proof and provenance: V3 VERIFICATION; V3 evaluate_expression; SFT.Computation.ProgramVerification.exactChecker; SFT.Computation.ProgramVerification.correctness; Timeline: Turn 259, line 2634.

207. Positive causal relation expansion

Causal expansion retains the original edges and adds exactly endpoints joined through a held intermediate event. A positive path contains at least one original edge; reflexive edges are not inserted by convention. Native proofs show that every finite expansion round preserves original edges and introduces only original positive paths. A closed result containing the original edges is their least transitive closure. Its self-edge exclusion is equivalent to exclusion of positive cycles. These results establish the relation-level expansion and cycle boundary; the complete Concurrency entry is verified below.

For held edge rows R and generated pair support U, one expansion is the complete filtered list

$$E_U(R)=[(a,c) \in U : (a,c) \in R \ \vee\ \exists b,\ (a,b) \in R \wedge (b,c) \in R].$$

Its membership equivalence follows from the exact two-row linking predicate. Original supported edges persist; each new edge is the concatenation of two held paths. Define a positive path by an original-edge constructor and a path-concatenation constructor, with no empty-path constructor. Induction over expansion rounds proves

$$(a, b) \in R_n \Longrightarrow \mathrm{Path}(R_0, a, b).$$

Original-edge containment persists whenever those edges belong to U.

If linked endpoint pairs are supported and a round adds no new members, its rows are transitive. More generally, any transitive relation T containing R_0 contains every positive R_0-path, by induction on the path constructors. Combining that leastness result with round soundness gives, for a closed round containing the original edges,

$$(a, b) \in R_n \iff \mathrm{Path}(R_0, a, b).$$

In particular,

$$\big(\forall a,\ (a, a) \notin R_n\big) \iff \big(\forall a,\ \neg\,\mathrm{Path}(R_0, a, a)\big).$$

This excludes exactly positive cycles. Reflexive edges arise only when justified by such a positive path; no reflexive closure is added by convention.

> Source, proof and provenance: V3 CONCURRENCY; V3 causal_closure; SFT.Computation.CausalClosure.pathLeast; SFT.Computation.CausalClosure.exactClosure; Timeline: Turn 263, line 2668.

208. Native causal fixed-point execution

Every growing expansion round fills at least one previously missing pair cell of the finite event support. This strict decrease establishes native termination without a cutoff. The executable result retains its complete expansion derivation and no-growth certificate. For carrier-supported original edges, its relation contains exactly the positive original paths; the final self-edge test is therefore exactly the positive-cycle boundary. Executed examples reconstruct the three-edge closure of a three-event chain, the complete cyclic relation of a two-event cycle, and empty closures. The complete Concurrency entry is verified below.

For finite pair support U and current rows R, define missing cells and growth by

$$M_U(R) = [e \in U : e \notin R], \qquad G_U(R) \iff \exists e \in U,\ \mathrm{linked}(R, e) \wedge e \notin R.$$

Expansion preserves every currently held supported cell, and a growth witness adds at least one previously missing cell. The strict filter-count theorem therefore gives

$$G_U(R) \Longrightarrow |M_U(E_U(R))| < |M_U(R)|.$$

The native recursion starts with the exact bound $|M_U(R)|$. In a growing round it expands and recurses with the reduced bound. At bound zero, growth would imply a natural number strictly less than zero and is impossible. A nongrowing round returns immediately. Thus the returned value carries both

$$\mathrm{Rounds}(U, R, \mathrm{closeRows}(U, R)) \quad \text{and} \quad \neg G_U(\mathrm{closeRows}(U, R)).$$

There is no branch returning an unfinished closure when the bound is exhausted.

Take U to be the complete ordered pair support of the event carrier. Supported original edges give supported path endpoints, so every potential linked pair is in U. Nongrowth then implies transitivity. Combining original-edge preservation, round soundness and leastness yields

$$(a, b) \in \mathrm{closeRows}(U, R) \iff \mathrm{Path}(R, a, b).$$

Self-edge exclusion is consequently equivalent to absence of positive return paths.

> Source, proof and provenance: V3 CONCURRENCY; V3 causal_closure; SFT.Computation.CausalTermination.nativeStable; SFT.Computation.CausalTermination.nativeExact; Timeline: Turn 263, line 2668.

209. Exact causal operation boundary and output

The native causal operation preserves the source guard order: repeated event identities are rejected first, followed by escaping edges and positive cycles. Repeated edges are retained as set input and produce no repeated output row. Successful output has exactly the original positive-path relation and is emitted in lexicographic string-pair order. The original three-event witness is established directly by kernel reduction. A further 1602 complete native output/error comparisons cover every three-event directed graph, reversed input order, duplicate edges, Unicode and empty labels, and guard precedence. The exact missing-pair bound supports structural recursion: at zero a growing round is contradictory, so no incomplete result can be returned.

Let $\mathrm{sortSet}$ remove repeated pairs and emit pairs using the source lexicographic string ordering. Its membership and distinctness statements are

$$e \in \mathrm{sortSet}(R) \iff e \in R, \qquad \mathrm{Distinct}(\mathrm{sortSet}(R)).$$

The public operation first checks event-identity uniqueness, then checks endpoints of $\mathrm{sortSet}(R)$, then computes the closure and checks self-edges. Writing $K = \mathrm{closeRows}(\mathrm{pairCells}(V), \mathrm{sortSet}(R))$, its exact successful-output condition is

$$\mathrm{causalClosure}(V, R) = \mathrm{ok}\, O \iff \mathrm{Distinct}(V) \wedge \neg\, \mathrm{escapes}(V, \mathrm{sortSet}(R)) \wedge \neg\, \mathrm{hasCycle}(K) \wedge O = \mathrm{sortSet}(K).$$

The escape predicate is false exactly when every retained edge has both endpoints in V; the cycle predicate is true exactly when some self-edge is held. Membership-preserving sorting transports paths in both directions, so accepted output satisfies

$$(a, b) \in O \iff \mathrm{Path}(R, a, b), \qquad \mathrm{Distinct}(O), \qquad \forall a, \neg\, \mathrm{Path}(R, a, a).$$

The original witness reduces to

$$\mathrm{causalClosure}([a, b, c], [(a, b), (b, c)]) = \mathrm{ok}[(a, b), (a, c), (b, c)].$$

The ordered rejection branches preserve precedence when more than one input defect is present.

> Source, proof and provenance: V3 CONCURRENCY; V3 causal_closure; SFT.Computation.CausalOperations.closureAdmission; SFT.Computation.CausalOperations.nativeSourceResult; Timeline: Turn 263, line 2668.

210. Complete concurrency schedules and local observations

The complete event schedule support retains exactly the permutations of the declared event carrier that respect every held causal relation row, including the empty identity schedule. Each retained event binds its exact process identity, local before-state, action and after-state to the original event and process registries. Native proofs establish that every scheduled event is an enabled original transition and that every registered process reconstructs its complete native observation through its terminal state.

Serial assembly preserves intermediate local states and every event record. Two enabled independent events on distinct processes may occur in either order with the same local endpoints; this is structural coexistence without a physical simultaneity assertion. The original causal kernel, three operational witnesses, complete 256-member grammar, four controls and every native program field agree with pinned V3, completing the Fold Concurrency Law.

Let $\mathrm{Before}(a, b, s)$ mean that scanning schedule s reaches a with b in the remaining suffix. The causal checker satisfies

$$\mathrm{causalCheck}(R, s) = \mathrm{true} \iff \forall (a, b) \in R, \mathrm{Before}(a, b, s).$$

Filtering the complete arrangement list, with the explicit empty-carrier case, gives

$$s \in \mathrm{allSchedules}(V, R) \iff \mathrm{Perm}(s, V) \wedge \forall (a, b) \in R, \mathrm{Before}(a, b, s),$$

under distinct event identities. For $V = []$, the empty schedule is retained exactly when the causal checker accepts it.

For each process p, project scheduled events to its actions A_p and prior states B_p. A local trace skips other processes' events and requires each own event to satisfy

$$\mathrm{step}(M_p, e.\,\mathrm{before}, e.\,\mathrm{action}) = \mathrm{ok}(e.\,\mathrm{after}).$$

Induction reconstructs the local path and, for an admitted initial state i_p,

$$\mathrm{run}(M_p, i_p, A_p) = \mathrm{ok}(B_p +\!\!+ [t_p]).$$

The admitted global schedule retains unique registries, a permutation of all event identities, original event rows, process binding, causal acceptance and a local trace for every registered process. These witnesses supply both enabledness of every event and the complete local observation for every process. Local traces compose through their shared middle state. Two enabled events with distinct process identities admit both local orderings, each with the same process-specific initial and final states.

> Source, proof and provenance: V3 CONCURRENCY; V3 causal_closure; SFT.Computation.ConcurrencySchedules.completeSchedules; SFT.Computation.ConcurrencySchedules.everyEventEnabled; Timeline: Turn 263, line 2668.

211. Computational causality and retained provenance

The event predecessor structure retains the original generated event carrier and the separate process-order and message-edge ledgers. A positive predecessor path is formed by an original row or by composing two such paths at their shared intermediate event. Native proofs equate the successful causal-closure output with precisely these retained source paths; every process-order and message row is preserved. The original event-uniqueness, endpoint and cycle guards remain unchanged.

Transitivity follows from path composition, while the admitted closure has neither self-predecessors nor reverse predecessor pairs. Incomparability is the absence of paths in both directions. Its complete generated pair support retains the event memberships without assuming a total global order. Empty edge ledgers have no predecessors. Assembly preserves both component ledgers and their interface paths, and every earlier causal output row remains in an admitted extension. These results concern computational predecessor structure; physical causal claims retain V3's later empirical-branch boundary. The original 256 candidates, operational witnesses, four controls and full native program outputs are verified against the pinned source, completing Fold Computational Causality.

For source $S = (V, P, M)$, keep process-order edges P and message edges M separate and execute closure on $P +\!\!+ M$. The provenance constructors are

$$\frac{(a,b) \in P}{a \prec_S b}, \qquad \frac{(a,b) \in M}{a \prec_S b}, \qquad \frac{a \prec_S b \quad b \prec_S c}{a \prec_S c}.$$

Induction in both directions establishes

$$a \prec_S b \iff \mathrm{Path}(P +\!\!+ M, a, b).$$

For an accepted output O, the closure theorem gives

$$(a,b) \in O \iff a \prec_S b.$$

Every original process and message edge is therefore retained. Path composition proves transitivity; cycle rejection gives $(a,a) \notin O$, and a reversed pair would compose with its forward pair to contradict that result.

The incomparability decision is precisely

$$\mathrm{incomparable}(O,a,b) \iff \neg(a \prec_S b) \wedge \neg(b \prec_S a).$$

Its complete pair ledger additionally requires $a, b \in V$. Appending two sources concatenates their event, process-order and message lists. Each component provenance proof transports into the combined source, and compatible paths compose at their common event. If both the old and combined source executions are admitted, every earlier output row persists. Empty process and message ledgers admit no positive predecessor proof.

> Source, proof and provenance: V3 CAUSALITY; V3 causal_closure; SFT.Computation.CausalProvenance.nativeProvenance; SFT.Computation.CausalProvenance.assemblyInterface; Timeline: Turn 264, line 2676.

212. Structural generator three: exact recurrence derivation

For every finite Fold depth, the recursively doubled support count equals the cardinality of the full generated Fold-word support. At positive depth this count exceeds One and has a unique positive predecessor. Taking the unit part with that predecessor as denominator gives the unit third at depth two and the unit seventh at depth three. Each equality is proved from the native support construction and exact part normalization.

The native Fold retains the complete first-return trace 1/3 → 2/3 → 1/3 and the complete trace 1/7 → 2/7 → 4/7 → 1/7. The intermediate states are proved distinct from the initial state, establishing first return with respective positive transition counts two and three. For every count strictly after two, three is no larger;

every count strictly after three is therefore nonminimal. This establishes existence and minimal positive-successor order without a bounded denominator search or measurement premise. The executable first-return routine, complete ten-axis program and original controls are verified below.

Let W_d be the complete generated Fold-word list after d extensions, and define its support count by

$$s_0 = 1, \qquad s_{d+1} = s_d + s_d.$$

The word-extension cardinality theorem gives $|W_d| = s_d$ by induction. The positive-depth count satisfies $s_d > 1$; its positive predecessor p_d is characterized uniquely by

$$p_d + 1 = s_d.$$

The unit part is $u_d = \mathrm{normalizePart}(1, p_d)$. In particular,

$$s_2 = 4, \quad p_2 = 3, \quad u_2 = \frac{1}{3}; \qquad s_3 = 8, \quad p_3 = 7, \quad u_3 = \frac{1}{7}.$$

The exact Fold transitions are

$$\frac{1}{3} \longmapsto \frac{2}{3} \longmapsto \frac{1}{3}, \qquad \frac{1}{7} \longmapsto \frac{2}{7} \longmapsto \frac{4}{7} \longmapsto \frac{1}{7}.$$

The transition lists omit the initial state and retain each successive state:

$$B = [2/3, 1/3], \qquad T = [2/7, 4/7, 1/7].$$

A first-return certificate requires a trace $L = L_0 +\!\!+ [u]$, a proof of every Fold transition from u, and $v \neq u$ for every $v \in L_0$. The displayed intermediate fractions satisfy those inequalities, proving first returns with $|B| = 2$ and $|T| = 3$.

For a natural count n, $2 < n$ implies $3 \leq n$. The witnessed ternary return therefore attains the least distinct positive successor of the binary count. If $3 < n$, the same witness satisfies $2 < 3 < n$, so n is nonminimal. Existence is supplied by the complete ternary orbit, rather than by a finite denominator search.

> Source, proof and provenance: V3 fold_part; V3 generator_period_three; SFT.Physics.GeneratorThreeDerivation.binaryFirstReturn; SFT.Physics.GeneratorThreeDerivation.ternaryFirstReturn; SFT.Physics.GeneratorThreeDerivation.leastDistinctSuccessor; Timeline: Turn 267, line 2702.

213. Native first-return execution

The original Fold first-return operation is implemented with its complete transition trace. It advances the exact positive part, tests equality with the original part, and then tests previous visitation. A prior visit before return produces the original recurrence error. A proof carried by the native execution records precisely these branches and connects every successful result to the retained first-return predicate.

Reduced denominators cannot increase under Fold. Consequently every executed part belongs to a finite carrier determined by the original denominator, and each fresh transition strictly reduces the remaining carrier. This proves termination without an arbitrary iteration bound or an additional scientific parameter. The native binary and ternary outputs and generator-period result are kernel-proved. Public Fraction guards and positive integer/Boolean count guards retain the original admission and rejection behavior. Complete-output comparisons for all 820 numerator/denominator pairs through denominator 40, including

repeated rational representations, returning and nonreturning parts, and four invalid inputs pass against independent exact-arithmetic reference traces. The finite test set supplements the general loop and first-return proofs.

For initial part u, current part x and seen-key list S, set $y = \mathrm{Fold}(x)$. The source loop tests, in order:

$$y = u : \ \mathrm{ok}[y]; \qquad y \neq u,\ \mathrm{key}(y) \in S : \ \mathrm{error}(\mathrm{recurrence});$$

otherwise it recurses from y with $\mathrm{key}(y) :: S$, prepending y to a successful returned trace. Its native run judgment retains exactly the return, repeated-visit and fresh-advance branches. Induction over a successful judgment reconstructs every transition and excludes the initial part from all preterminal states.

For the original reduced denominator D, Fold normalization proves

$$\mathrm{denominator}(\mathrm{Fold}(x)) \leq \mathrm{denominator}(x) \leq D.$$

The complete finite key carrier is

$$C_D = [(n, d) : n \in \mathrm{through}(D),\ d \in \mathrm{through}(D)],$$

where $\mathrm{through}(D) = [D, D-1, \ldots, 0]$. Every admitted part with denominator at most D belongs to this carrier because its positive numerator is no greater than its denominator. The loop invariant covers each such key by the remaining or seen list. A fresh key must therefore belong to the remaining list, and removing it strictly decreases that list's length. The recursion starts with $|C_D|$; an advancing branch at zero contradicts this strict decrease. This derives termination from the original denominator.

The checked public interface preserves Fraction admission and the original errors. Count inputs separately reject Booleans, nonpositive depths and predecessors at or below One. Kernel evaluation gives

$$\mathrm{firstReturn}(1/3) = \mathrm{ok}\, B, \qquad \mathrm{firstReturn}(1/7) = \mathrm{ok}\, T.$$

The generator operation takes $k = |B| + 1$, executes the unit-part return at depth k, and verifies $|B| = 2$ and $|T| = k$; its exact result is $\mathrm{ok}\, 3$.

> Source, proof and provenance: V3 first_return_trace; SFT.Physics.FoldReturnCarrier.keyInjective; SFT.Physics.FoldReturnCarrier.normalDenominatorBound; SFT.Physics.FoldFirstReturn.originalLoop; SFT.Physics.FoldFirstReturn.successfulSteps; Timeline: Turn 267, line 2702.

214. Complete Structural Generator Three entry

The ten registered binary structural axes generate all 1,024 candidate forms. Native proofs establish complete enumeration, exact decision correspondence and the sole preserving form, with each rejected coordinate violating its original registered requirement. The three operational witnesses evaluate the native unit-part first-return routines and the derived generator-period operation. Their outputs reproduce the complete binary and ternary traces and the exact successor result three.

The full StructuralPhysicsProgram reproduces every registered candidate, identifier-based decision, complete closure record, original control field and associated hash against pinned V3. The full-entry result therefore includes the Fold-support construction, positive predecessor, first-return existence, minimal successor proof, original executable guards and all 1,024 forms. This completes the Structural Generator Three entry with no axiom dependencies or free scientific parameters.

The registered candidate is the ordered ten-coordinate tuple of carrier, action, recurrence, binary anchor, successor, unit construction, orbit record, order closure, measurement direction and extension rule. Each coordinate contains its original rejected and admitted alternative, so the complete product list has

$$|\mathcal{C}| = \underbrace{2 \cdot 2 \cdots 2}_{10 \text{ factors}} = 1,024, \qquad \forall c : \mathrm{Candidate},\ c \in \mathcal{C}.$$

Let c_* have the admitted alternative in every coordinate. The native decision equivalences are

$$\mathrm{survives}(c) = \mathrm{criteriaDecision}(c), \qquad \mathrm{survives}(c) = \mathrm{true} \iff \bigwedge_{i=1}^{10} (c_i = \mathrm{admitted}) \iff c = c_*.$$

Each rejected-coordinate theorem returns false for the original decision. These criteria retain the exact positive-period carrier, double-and-cast action, complete first return, depth-two anchor, least positive successor, support-predecessor unit part, every orbit transition, positive-count closure, derivation-before-measurement direction and absence of an extra rule.

The three executable witness checks retain

$$\mathrm{map}(\mathrm{key}, B) = [(2,3),(1,3)], \qquad \mathrm{map}(\mathrm{key}, T) = [(2,7),(4,7),(1,7)], \qquad \mathrm{generatorPeriod} = \mathrm{ok}\,3.$$

These evaluations accompany the support, predecessor, transition and minimality derivations above. Full program comparison additionally retains every candidate decision, closure and original control record with its source hash; the classification does not replace those derivations.

> Source, proof and provenance: V3 generator_period_three; SFT.Physics.GeneratorOperations.publicSupport; SFT.Physics.GeneratorOperations.publicPredecessor; SFT.Physics.GeneratorThreeWitnesses.binaryCheckPassed; SFT.Physics.GeneratorThreeWitnesses.ternaryCheckPassed; Timeline: Turn 267, line 2702.

215. Spatial Dimension Three

The complete ordered pair support of the two held Fold fibre roles contains four cells. The original stability constraint retains a positive independent-direction count strictly above the fibre boundary and strictly below pair-cell saturation. Native enumeration retains every positive count in this open window, preserving ascending order and excluding both endpoints. For every generated count, these two strict inequalities imply equality to three; the complete native result is the singleton[3].

The independently completed Fold-recurrence operation also returns three. The native spatial-dimension operation checks equality between the complete stability census and the singleton recurrence result, establishing the original cross-lock. Exact integer and Boolean guards are retained. All 1,024 original grammar forms, three operational witnesses, four controls and complete program fields/hashes are verified against pinned V3, completing Spatial Dimension Three. Measurement remains downstream of the sealed structural count.

For ordered fibre-label lists L, R, the generated pair support satisfies

$$|\,\mathrm{pairCells}(L, R)| = |L|\,|R|.$$

The original Fold label support has length two, hence its full self-pair support has four cells. Positive counts are enumerated in ascending order by

$$P_0 = [], \qquad P_{n+1} = P_n \mathbin{+\!\!+} [n+1], \qquad k \in P_n \iff 0 < k \le n.$$

For fibre count b, the stability list is

$$S_b = [k \in P_{b^2} : b < k \wedge k < b^2].$$

Its exact membership theorem is $k \in S_b \iff b < k < b^2$, excluding both endpoints. At $b = 2$,

$$2 < k < 4 \Longrightarrow 3 \leq k \leq 3 \Longrightarrow k = 3, \qquad S_2 = [3].$$

The public stability operation admits integer fibre counts at least two and preserves rejection of Booleans, negative values, zero and One.

Finally the spatial operation computes S_2, obtains the independently derived generator count g, and checks $S_2 = [g]$. With $\mathrm{generatorPeriod} = \mathrm{ok}\,3$, the exact cross-lock is

$$\mathrm{spatialDimension} = \mathrm{ok}\,3 = \mathrm{generatorPeriod}\,.$$

Thus the full stability enumeration and the retained first-return recurrence identify the same count before downstream measurement.

> Source, proof and provenance: V3 stable_spatial_counts; V3 spatial_dimension_three; SFT.Physics.SpatialStability.onlySpatialCount; SFT.Physics.SpatialStability.completeSpatialDerivation; Timeline: Turn 268, line 2709.

216. Boundary Rank Two

Holding the source-normal direction in the forced three-direction carrier leaves exactly all nonnormal independent directions. Their count followed by the held normal reassembles the carrier, establishing the unique positive predecessor two. Native execution retains the original predecessor and reassembly check.

At any generated positive scale, the two independent coordinate supports form the complete ordered pair product. Membership is equivalent to membership of both coordinates; each pair occurs once and the cardinality is the product of the coordinate counts. Extension of either coordinate retains all previous pairs and adds its pair with every coordinate on the other direction. Equal scale r therefore yields r paired with r. Exact rational growth preserves this product by normalized multiplication, with the original strict above-One input boundary. The scale-two and scale-three witnesses yield four and nine.

All 2,048 original candidate forms, three operational witnesses, four controls and complete native program fields and hashes match pinned V3. This completes Boundary Rank Two without a measured exponent, conventional surface formula or continuum premise.

Let the independent spatial carrier be $A +\!\!+ [n] +\!\!+ B$, with held normal n distinct from the remaining directions. The boundary list is $A +\!\!+ B$, with exact membership

$$d \in \partial C \iff d \in C \wedge d \neq n, \qquad |\partial C| + 1 = |C|.$$

For $|C| = 3$, successor injectivity gives $|\partial C| = 2$, and any rank r satisfying $r + 1 = 3$ equals two. The native boundary operation computes the positive predecessor of the spatial dimension and verifies this reassembly.

For coordinate supports L, R, the complete pair operation satisfies

$$(a, b) \in L \times R \iff a \in L \wedge b \in R, \qquad |L \times R| = |L|\,|R|.$$

Extending the left coordinate by a adds its full row of pairs with R; extending the right coordinate adds one pair for every member of L, retaining the source order. Equal coordinate counts r therefore give r^2 cells. For exact positive scale $q = n/d > 1$, normalized multiplication returns u/v with

$$u\,d^2 = n^2 v.$$

The rank-two growth operation returns this product q^2, retaining the strict above-One guard. Its original witnesses are $2^2 = 4$ and $3^2 = 9$.

> Source, proof and provenance: V3 boundary_rank_two; V3 boundary_growth; SFT.Physics.BoundarySupport.completeBoundaryRank; SFT.Physics.BoundaryGrowth.exactGrowthValue; SFT.Physics.BoundaryPairs.completeEquivalentSupport; Timeline: Turn 269, line 2716.

217. Inverse fine-structure cover construction

The native binary count is the positive predecessor of the independently completed generator-three recurrence. Positive powers retain the source repeated-multiplication operation and its exact count domain. For every admitted carrier, the binary-cover search returns a positive depth whose complete Fold support covers the carrier; every earlier positive depth has strictly smaller support than the carrier. Its finite execution budget follows from the carrier and increasing positive support and does not truncate the search. The support count is proved equal to the cardinality of the complete Fold-word enumeration at every depth.

For generator volume three cubed and its successor volume, the least depths are five and seven. These independently agree with the source binary/generator decompositions. Executing the original typed block construction yields binary two, generator three, down depth five, up depth seven, tower 128, boundary 9 and cover 250. These verified cover and block results supply the complete finite promotion ladder and exact rational assembly described below.

The generator count $g = 3$ supplies binary predecessor $b = 2$. Positive powers execute repeated multiplication, with the native identity

$$\mathrm{powerFrom}(a, k, z) = z\,a^k.$$

For carrier size N, the covering search starts at depth $d = 1$ and support $s = 2$, stopping when $N \le s$, otherwise replacing (d, s) by $(d + 1, 2s)$. Its invariant $s = 2^d$ and retained failed comparisons prove

$$d_N > 0, \qquad N \le 2^{d_N}, \qquad 0 < k < d_N \Longrightarrow 2^k < N.$$

The finite recursion carries $N \le s + f$; positivity of s gives
$N \le 2s + (f - 1)$ at an advancing step. At zero remaining budget the invariant already proves coverage. Thus the budget cannot truncate an uncovered search.

The complete Fold support has cardinality 2^d, connecting the numerical comparisons to generated words. The original generator volumes are $3^3 = 27$ and $3^4 = 81$, so

$$2^4 < 27 \le 2^5, \qquad 2^6 < 81 \le 2^7, \qquad d_{\downarrow} = 5, \quad d_{\uparrow} = 7.$$

The independent decomposition checks are $5 = 2 + 3$ and $7 = 3 + (3 + 1)$. The exact typed blocks follow:

$$T = b^{d_{\uparrow}} = 2^7 = 128, \qquad B = g^b = 3^2 = 9, \qquad C = b\,d_{\downarrow}^{g} = 2 \cdot 5^3 = 250.$$

> Source, proof and provenance: V3 minimal_binary_cover; V3 fine_structure_blocks; SFT.Physics.AlphaCovers.binaryCountTwo; SFT.Physics.AlphaCovers.powerFromValue; SFT.Physics.AlphaBlocks.forcedBlocks; SFT.Physics.AlphaBlocks.downMinimal; Timeline: Turn 271, line 2731.

218. Terminal inverse fine-structure ratio

The complete three-direction promotion object begins with three down-depth factors. Each successor replaces one remaining interchangeable direction by an up-depth factor, preserving the total count of directions. Its remaining-direction count decreases strictly and reaches structural absence after three promotions. The generated stages are distinct and exhaustive, with complete rung supports 125,175,245,343. No further successor is admitted once all directions are promoted.

The native continued assembly reads a positive rung count from one through four. It forms the descending chain of each rung plus the returning One divided by the following chain, then adds the reciprocal chain to cover 250. Exact normalized positive-rational arithmetic retains the typed tower 128 plus boundary 9 multiplied by the covered-return dilation. All four finite orders are represented; the default is the terminal fourth order. The leading ratio is 34259/250 and the terminal ratio is 503846395469/3676744786. Both identities are kernel-proved with zero axiom dependencies.

The first rung, 125, supplies the base cover $250 = 2 \times 125$ and hence the separate leading-order result; the terminal fourth-order assembly adds the reciprocal continuation built from the remaining rungs 175, 245 and 343.

Writing the terminal effective cover as D, the exact assembly is:

$$D = 250 + \cfrac{1}{175 + \cfrac{1}{245 + \cfrac{1}{343}}}, \qquad \alpha^{-1} = 128 + 9\frac{D+1}{D} = \frac{503846395469}{3676744786}.$$

All 1,024 original candidate forms, three native operational witnesses, four controls and full program outputs and hashes agree with pinned V3. This completes the registered inverse fine-structure entry. Measurement remains excluded from the construction and can only enter its separately registered downstream comparison.

A promotion stage is (p, r), recording promoted and remaining directions. Its successor is

$$(p, r+1) \longmapsto (p+1, r), \qquad p + (r+1) = (p+1) + r = 3,$$

and $(p, 0)$ has no successor. The complete generated stage list is

$$[(0,3), (1,2), (2,1), (3,0)].$$

At promoted count p, the rung is $5^{3-p}7^{p}$, with the source's explicit identity factor when either exponent is absent. Hence

$$r_0 = 125, \quad r_1 = 175, \quad r_2 = 245, \quad r_3 = 343.$$

The four admitted effective covers are exactly

$$D_1 = 250, \qquad D_2 = 250 + \frac{1}{175},$$
$$D_3 = 250 + \frac{1}{175 + \dfrac{1}{245}}, \quad D_4 = 250 + \frac{1}{175 + \dfrac{1}{245 + \dfrac{1}{343}}}.$$

The leading rung supplies $250 = 2r_0$; subsequent orders use the remaining continuation. For each admitted order j, normalized positive arithmetic assembles

$$A_j = T + B\frac{D_j + 1}{D_j} = 128 + 9\frac{D_j + 1}{D_j}.$$

Evaluating from the innermost positive denominator outward makes the terminal arithmetic explicit:

$$245 + \frac{1}{343} = \frac{84036}{343}, \qquad 175 + \frac{343}{84036} = \frac{14706643}{84036}, \qquad D_4 = 250 + \frac{84036}{14706643} = \frac{3676744786}{14706643}.$$

Substitution into the common assembly gives

$$128 + 9\frac{D_4 + 1}{D_4} = \frac{128(3676744786) + 9(3676744786 + 14706643)}{3676744786} = \frac{503846395469}{3676744786}.$$

Every denominator is positive, so each reciprocal is admitted and the equalities follow by exact integer cross multiplication. Consequently

$$A_1 = \frac{34259}{250}, \qquad A_4 = \frac{503846395469}{3676744786}, \qquad \alpha = \frac{3676744786}{503846395469}.$$

The default chooses the complete rung-list length, four. Every addition, reciprocal and multiplication preserves its rational value by the corresponding cross-multiplication identity, so the displayed terminal ratio is obtained without floating arithmetic.

> Source, proof and provenance: V3 promotion_rungs; V3 effective_cover; V3 inverse_fine_structure; SFT.Physics.AlphaPromotion.stageCount; SFT.Physics.AlphaPromotion.successorConservesDirections; SFT.Physics.AlphaRatio.leadingRatio; SFT.Physics.AlphaRatio.terminalRatio; SFT.Physics.AlphaRatio.automaticTerminal; Timeline: Turn 271, line 2731.

219. Charged-lepton invariant operations

The complete colour support retains every word of each generated depth exactly once. Its cardinality is three to that depth; the two held Fold-fibre copies of the fifth-power support therefore contain 486 cells. The original predecessor operation returns 485, whose successor reassembles that complete support, and no other positive predecessor has this property.

Exact positive take admits a positive part strictly below the held whole. Its normalized remainder preserves the rational difference, and adding the retained part reconstructs the whole at the common denominator. The native charged-lepton operation pairs the two fibres with generator three and applies the original neighbouring channel census. Channels two, three and four yield 2/969,3/1454 and 4/1939 respectively; only three has the identity of the admitted generator carrier. The constructed invariant tuple is 1,1/6,1/485,3/1454, without forming an irrational root. These native operation and support results are connected to the complete symmetric carrier below; the registered charged-lepton cubic entry is verified.

Let A be the three original colour labels and C_d all length-d words over A. Complete word enumeration gives

$$w \in C_d \iff |w| = d \wedge \forall x \in w,\ x \in A, \qquad |C_d| = 3^d,$$

with no duplicate words. The two fibre-tagged copies of C_5 therefore contain

$$N = 2 \cdot 3^5 = 486.$$

The original positive predecessor h satisfies $h + 1 = N$, hence uniquely $h = 485$.

For positive fractions $w = n_w/d_w$ and $p = n_p/d_p$, positive take requires

$$n_p d_w < n_w d_p$$

and normalizes the remainder

$$\frac{n_w d_p - n_p d_w}{d_w d_p}.$$

Its exact reassembly is

$$(n_w d_p - n_p d_w) + n_p d_w = n_w d_p.$$

For each original neighbouring channel $c \in [2, 3, 4]$, the sharpened product is the reciprocal of the retained $485 - 1/c$:

$$P_c = \frac{1}{485 - \frac{1}{c}} = \frac{c}{485c - 1}.$$

Thus $P_2 = 2/969$, $P_3 = 3/1454$, and $P_4 = 4/1939$. Carrier identity requires $c = g = 3$, yielding the invariant tuple

$$(S_1, S_2, P_{\text{lead}}, P_{\text{sharp}}) = \left(1, \frac{1}{2 \cdot 3}, \frac{1}{485}, \frac{3}{1454}\right).$$

The channel census and original integer/Boolean rejection boundary remain explicit.

> Source, proof and provenance: V3 positive_take; V3 charged_lepton_invariants; SFT.Physics.LeptonTake.heldPartReassembles; SFT.Physics.LeptonSupport.nativeLeadingDerivation; SFT.Physics.LeptonInvariants.exactInvariants; Timeline: Turn 273, line 2747.

220. Complete charged-lepton symmetric carrier

The three-root carrier is represented by formal position identities, retaining exact symmetric coefficients without producing numerical roots. The complete distributive expansion contains every selection of the three formal factors exactly once. Grouping by selected degree yields the three singleton terms, all three distinct pair products and the single triple product, together with the leading One. The recursive expansion and the generated ledger are proved equal. Appending a fresh generated term retains every earlier term and records the new term once.

The native generator fixes the carrier count at three. The exact construction assigns root sum 1, pair sum 1/6, leading product 1/485 and the carrier-identity sharpened product 3/1454. All 2,304 original grammar forms, three native witnesses, four controls and complete program outputs and hashes match pinned V3. This completes the charged-lepton cubic entry while preserving its exact-rational invariant boundary and exclusion of measured or fitted root values.

Let x_1, x_2, x_3 denote formal positions. Each Boolean selection of those positions appears once in the complete product ledger:

$$[[], [3], [2], [2,3], [1], [1,3], [1,2], [1,2,3]].$$

The recursive distributive expansion is

$$E([]) = [[]], \qquad E(p :: R) = E(R) +\!\!+ [p :: t : t \in E(R)].$$

For the three positions this equals the displayed ledger. Filtering by degree gives three singleton terms, three pair terms and one triple term; selection injectivity proves no duplicate term. In conventional formal-polynomial notation, the same grouping reads

$$\prod_{i=1}^{3}(z - x_i) = z^3 - (x_1 + x_2 + x_3)z^2 + (x_1x_2 + x_1x_3 + x_2x_3)z - x_1x_2x_3.$$

The native invariant carrier assigns

$$x_1 + x_2 + x_3 = 1, \qquad x_1x_2 + x_1x_3 + x_2x_3 = \frac{1}{6},$$

with leading product $1/485$ and sharpened product $3/1454$. These are coefficients on formal positions, not assigned numerical root values. The generator count equals the singleton-ledger length. Appending a fresh generated term preserves all earlier terms and, under freshness, retains uniqueness.

> Source, proof and provenance: V3 charged_lepton_invariants; SFT.Physics.LeptonLedger.ledgerExhaustive; SFT.Physics.LeptonLedger.cubicExpansion; SFT.Physics.LeptonLedger.normalizedInvariantCarrier; Timeline: Turn 273, line 2747.

221. Terminal charged-lepton self-coupling

The terminal operation uses the native generator three, fine-structure cover depths five and seven, and reciprocal terminal inverse fine-structure ratio. It attaches one alpha factor to every formal root, divides the resulting cubic coupling by the complete 27-cell colour carrier, and weights it by 5+(7/3)alpha. The correction is exactly 376926193013708290190758164816402371555272/52200858161852702444325842762753846686284139832 01. Holding this positive correction from 3/1454 yields 156597093978711688014438154664545109568370005841157590004776733382935404977537704409308185713931574254. Native kernel proofs establish that the correction is strictly smaller than the admitted product and the retained product remains positive.

This result preserves the V3 observational-derivation provenance: the executable relation takes no measured lepton value, while empirical admission additionally requires the two complete post-seal CODATA comparisons and their custody controls. The exact native operation, registered structural-program

comparison and native empirical validation are complete, including both post-seal CODATA comparisons and their custody controls; the Charged-Lepton Terminal entry is complete within its original empirical and observational-derivation boundaries.

The terminal inverse fine-structure ratio fixes

$$\alpha = \frac{3676744786}{503846395469}.$$

Coupling one factor to each formal root is defined by an empty-product One and multiplication at each retained position. Induction gives

$$\mathrm{coupleRoots}(\alpha, R) = \alpha^{|R|},$$

hence the three-position carrier contributes α^3. The complete depth-three colour support has $3^3 = 27$ cells. With cover depths $5, 7$, the original correction is therefore

$$\delta = \frac{\alpha^3}{27}\left(5 + \frac{7}{3}\alpha\right) = \frac{37692619301370829019075816481640237155527 2}{52200858161852702444325842762753846686284139832 01}.$$

The terminal product is the exact positive take

$$P_* = \frac{3}{1454} - \delta = \frac{1565970939787116880144381546645451095683700058411 5}{7590004776733382935404977537704409308185713931574254}.$$

For $\delta = n_\delta / d_\delta$, the proved strict inequality is

$$1454 n_\delta < 3 d_\delta.$$

Thus $3d_\delta - 1454 n_\delta > 0$; normalizing it over $1454 d_\delta$ yields $P_* > 0$, and adding back the held numerator reconstructs $3d_\delta$. This derivation uses only the retained structural outputs; the separately registered post-seal comparisons retain their own observational admission.

> Source, proof and provenance: V3 terminal_self_coupling_correction; V3 terminal_product_invariant; SFT.Physics.LeptonTerminal.exactCorrection; SFT.Physics.LeptonTerminal.exactTerminal; SFT.Physics.LeptonTerminal.heldCorrectionReassembly; Timeline: Turn 278, line 2791.

222. Terminal grammar and exact root brackets

The complete 1,024-form terminal grammar preserves all ten registered axes, original decisions and unique survivor. Fresh native program execution reproduces every original census, closure, control and digest, including empirical evidence mode and observational-derivation provenance. The three operational witnesses are computed from the native correction, cover depths and retained product.

Post-seal evaluation translates the exact signed cubic and all original bisection branches. The dyadic isolation procedure enumerates every cell at each visited depth and retains exact lower roots or opposite-sign cells according to the source rule. A constructive run certificate links the complete native search to the original stopping rule: exactly three positive, nondegenerate brackets. For the native terminal product the search starts at depth one, rejects earlier stopping points and succeeds at depth seven with brackets (1/128,1/64), (3/16,25/128) and (101/128,51/64). No numerical irrational root is constructed. The rational refinement, source parsing, custody and all-row validation are connected below.

The signed rational polynomial used by post-seal isolation is

$$f(x) = x^3 - x^2 + \frac{1}{6}x - P_*.$$

At depth d, enumerate every cell

$$I_{d,i} = \left[\frac{i}{2^d}, \frac{i+1}{2^d}\right], \qquad 0 \le i < 2^d.$$

For endpoints l, u, the source cell rule retains (l, l) if $f(l) = 0$; otherwise it retains (l, u) if $f(l)f(u) < 0$, and retains no row otherwise. The stopping predicate requires exactly three retained brackets and, for every one, $0 < l < u$. The native isolation judgment records a rejected stopping predicate at every earlier visited depth and the accepted predicate at the final depth.

Starting at depth one, the actual terminal execution returns at depth seven:

$$\left(\frac{1}{128}, \frac{1}{64}\right), \qquad \left(\frac{3}{16}, \frac{25}{128}\right), \qquad \left(\frac{101}{128}, \frac{51}{64}\right).$$

The successful bounded execution supplies the complete isolation judgment from depth one, rather than treating exhaustion as acceptance. All cell evaluations and sign products use exact signed rational arithmetic. The separate ten-axis terminal candidate product has $2^{10} = 1,024$ forms; its program comparison retains the original empirical evidence mode and observational-derivation provenance alongside these executable witnesses.

> Source, proof and provenance: V3 isolate_three_roots; SFT.Physics.LeptonIntervals.exactBracketRetained; SFT.Physics.LeptonIntervals.midpointRootRetained; SFT.Physics.LeptonIsolation.terminalSearchComplete; SFT.Physics.LeptonIsolation.terminalNativeRun; Timeline: Turn 278, line 2791.

223. Exact terminal mass-ratio refinements

The squared-ratio operation retains the source positivity checks and forms its lower bound from the numerator lower endpoint divided by the denominator upper endpoint, and its upper bound from the numerator upper endpoint divided by the denominator lower endpoint, squaring both exactly. Refinement bisects every retained root bracket together and stops on the original disjointness or containment condition. A constructive run theorem proves that every successful native refinement follows those transitions and retains every bracket.

Starting from the native terminal product and its complete dyadic isolation, the muon-electron computation reaches its exact registered interval after 26 refinements and the muon-tau computation after 14. Kernel proofs establish the full runs, exact final rational endpoints, containment and overlap with both registered source intervals. Independent exact-fraction comparisons reproduce all 42 successive prediction intervals alongside division, positivity and indexing checks. These results use the exact registered source endpoints; native fixed-width source-row parsing and the post-seal custody/adverse-control gate are connected below.

For positive numerator and denominator brackets $N = [n_l, n_u]$ and $D = [d_l, d_u]$, the original squared-ratio operation returns

$$Q(N, D) = \left[\left(\frac{n_l}{d_u}\right)^2, \left(\frac{n_u}{d_l}\right)^2\right].$$

It retains the original positive-lower-endpoint and division guards. For predicted interval $[a, b]$ and registered source interval $[c, d]$, the stopping decisions are

$$\text{disjoint} \iff b < c \vee d < a, \qquad \text{contained} \iff c \le a \wedge b \le d.$$

The loop stops on their disjunction. Otherwise it bisects every root bracket, preserving their number and ordering. With $m = (l + u)/2$, bisection keeps an exact point bracket unchanged, returns (m, m) if $f(m) = 0$, selects (l, m) if $f(l)f(m) < 0$, and selects (m, u) in the remaining branch.

The complete terminal refinement runs yield

$$Q_{\mu/e} = \left[\frac{2760006749688854416}{13348308265544641}, \frac{2760006753011508025}{13348308034474884}\right] \subseteq \left[\frac{2067682781}{10000000}, \frac{2067682873}{10000000}\right],$$

$$Q_{\mu/\tau} = \left[\frac{164508926409}{2766726569104}, \frac{164509737604}{2766723242409}\right] \subseteq \left[\frac{118919}{2000000}, \frac{23787}{400000}\right].$$

The native calls allow 27 and 15 prediction checks, respectively, accounting for the initial prediction and the 26 or 14 refinement transitions. Their run certificates retain each computed interval and every earlier rejected stopping decision; exact containment and overlap are kernel-proved for both final results. These endpoint comparisons remain downstream of the sealed terminal product.

> Source, proof and provenance: V3 bisect_root; V3 squared_ratio_interval; V3 predicted_ratio_interval; SFT.Physics.LeptonRatios.refinementSound; SFT.Physics.LeptonRefinement.muonElectronRun; SFT.Physics.LeptonRefinement.muonTauRun; SFT.Physics.LeptonRefinement.bothRowsOverlap; Timeline: Turn 278, line 2791.

224. Exact source rows and complete comparison record

The native decimal parser retains positive finite decimal records, exact exponent scaling and the original rejection conditions. Fixed-width selection retains the complete snapshot row census, requires exactly one row with each requested quantity name, and parses the central and uncertainty fields without floating arithmetic. The uncertainty must be strictly positive and below the central value. Both complete parsed interval triples are kernel-proved equal to the registered rational bounds and central values. Fresh native file execution confirms the complete row sequence and both parsed results from the pinned snapshot.

The native comparison record combines those parsed triples with the actual terminal construction and interval refinements. It retains both predicted intervals, both source intervals, both central values, each overlap outcome and their conjunction. The complete record is proved successful and independently reproduced field by field. The source-row and numerical comparison components feed the original post-seal held-label prediction, custody and adverse-control procedure.

For a finite decimal with positive concatenated digit integer n, p fractional places and integer exponent e, the parser constructs the normalized exact value

$$v = \begin{cases} n10^{e}/10^{p}, & e \ge 0, \\ n/10^{p+(-e)}, & e < 0. \end{cases}$$

The original digit, whitespace, exponent and finite-positive-record checks precede this construction. Fixed-width selection requires a row of at least 110 characters, whose stripped first 60 characters equal the requested quantity name. Exactly one such row must exist. The next 25 characters supply the central value c, and the following 25 supply uncertainty u. Admission requires $0 < u < c$, returning

$$(c - u, c, c + u).$$

For the pinned snapshot the two exact triples are

$$\left(\frac{2067682781}{10^7}, \frac{2067682827}{10^7}, \frac{2067682873}{10^7}\right), \qquad \left(\frac{118919}{2000000}, \frac{118927}{2000000}, \frac{23787}{400000}\right).$$

The comparison row retains the predicted interval, source endpoints, central value and overlap decision. For $P = [a, b]$ and $S = [c, d]$,

$$\mathrm{overlap}(P, S) = [d \geq a] \wedge [b \geq c].$$

The complete record's acceptance is the conjunction of both overlap decisions. Native execution of the parser and both terminal refinements returns that record, whose conjunction is proved true.

> Source, proof and provenance: V3 source_interval; V3 comparison_record; V3 exact_decimal; SFT.Physics.LeptonDecimal.exactDecimal; SFT.Physics.LeptonSource.muonElectronParsed; SFT.Physics.LeptonSource.muonTauParsed; SFT.Physics.LeptonComparison.nativeComparison; SFT.Physics.LeptonComparison.bothRowsPassed; Timeline: Turn 278, line 2791.

225. Registered terminal prediction and empirical validation

The registered predictor reads the sealed derivation premise, constructs the physical-observation label, retains the premise/label pair, and emits the label. Native proofs establish the output, retention of the bound premise and all four trace positions for every admitted nonblank premise. Source data-only instruction fields are retained. Complete before/after protected-tree records are compared across prediction execution. The target vault commits both original external rows, permits a single release only after the matching envelope is sealed, and verifies the experiment, custodian, seal, support, target content and release identity.

Both released rows are compared without omission. The original unfavorable control changes the external-observation label and is rejected. The numeric comparison and label comparison must agree; a disagreement produces the original terminal validation failure. The native wrapper is connected to the kernel-proved complete source-row and refinement result. Fresh execution reproduces every original empirical-validation field, including the prediction trace, isolation certificate, custody certificate and measurement-receipt identity. Host metadata in the source receipt is retained explicitly as metadata, independently of native Lean execution. The Charged-Lepton Terminal entry is complete within its original empirical and observational-derivation boundaries.

For a nonblank derivation seal s, the registered four-instruction program performs

$$\mathrm{input}(s), \quad \mathrm{label}(\text{physical-observation}, L), \quad \mathrm{pair}(s, L), \quad \mathrm{emit}(L).$$

Its output theorem gives the original label L; its retained bound-result is the premise/label pair, and its complete position ledger is $[1, 2, 3, 4]$. This is the registered held-label program; the numeric construction is connected by the separate agreement gate below.

The protected-tree comparison requires equality of root identity, the complete ordered path/hash list, and snapshot hash. The data-only audit also checks the registered field set and absence of executable-source fields. A vault release requires that it has not already released, that the prediction experiment matches, and that the envelope hash matches its commitment. Release marks the vault used. Verification requires an empty violation list covering experiment, custodian, prediction seal, envelope, complete target support, committed content and release-payload identity. The native rejection theorems exclude access before a seal, replacement of an existing seal, and a second release.

For both released target rows i, the label check is $[L = L_i]$. Its combined result is

$$V = \left(\bigwedge_i [L = L_i]\right) \wedge [L \neq \text{changed-external-observation}].$$

The original two target comparisons are true and the unfavorable changed-label control is rejected. For numeric result N, the wrapper requires

$$N.\,\text{allRowsPassed} = V.$$

A discrepancy returns the original validation error. The native terminal record has $N.\,\text{allRowsPassed} = \text{true}$; combining its complete parser/refinement execution with a passing custody-and-label validation constructs the successful terminal wrapper. Host identifiers remain receipt metadata.

> Source, proof and provenance: V3 ChargedLeptonTerminalExternalValidator; SFT.Physics.LeptonPrediction.registeredOutput; SFT.Physics.LeptonPrediction.premiseRetained; SFT.Physics.LeptonExperiment.changedSourceRejected; SFT.Physics.LeptonCustody.noPreSealAccess; SFT.Physics.LeptonCustody.cannotReseal; SFT.Physics.LeptonValidation.tamperedControlRejected; SFT.Physics.LeptonValidation.terminalConnection; Timeline: Turn 278, line 2791.

226. Electroweak Fold-fibre mixing

The native binary count generates the half-One balance image. At binary depth two, the complete support is the ordered sequence 1/4,1/2,3/4,1. Filtering that entire support by exact Fold return yields precisely 1/4 and 3/4. Both parts are proved to return to 1/2, they are distinct, and their positive sum is exactly One. A completeness theorem excludes every other depth-two preimage. The source construction therefore returns $unified = 1/2$, $\text{sin_squared} = 1/4$ and $\text{cos_squared} = 3/4$.

The enumeration preserves the original rational-image and positive-depth guards and every generated support position. Native membership equivalence identifies each retained part exactly with membership in the original support together with the required Fold image. The complete 256-form grammar, unique preserving form, three operational witnesses and four controls reproduce all pinned V3 program fields and identities. Measurement remains a later sealed claim, outside this bare exact squared-channel construction.

For support count m, enumerate every exact positive part in source order:

$$E_m = [1/m, 2/m, \ldots, m/m], \qquad |E_m| = m.$$

Filtering by Fold image y has the exact membership property

$$p \in \text{returning}(y, E_m) \iff p \in E_m \wedge \text{Fold}(p) = y.$$

With native binary count two and binary depth two, $m = 2^2 = 4$, so

$$E_4 = [1/4, 1/2, 3/4, 1].$$

Kernel evaluation and the membership theorem give

$$\text{returning}(1/2, E_4) = [1/4, 3/4],$$

$$\mathrm{Fold}(1/4) = 1/2, \qquad \mathrm{Fold}(3/4) = 1/2, \qquad 1/4 \neq 3/4, \qquad 1/4 + 3/4 = 1.$$

Thus every depth-two returning member equals one of these two parts. The native operation also checks that the preimage count equals the binary count and that the sum partitions One, yielding

$$(\mathrm{unified}, \sin^2, \cos^2) = (1/2, 1/4, 3/4).$$

The original guard sequence is retained: zero depth fails the preimage guard, while Boolean true reaches and fails the positive-power count guard.

> Source, proof and provenance: V3 electroweak_mixing; SFT.Physics.ElectroweakMixing.completeFibre; SFT.Physics.ElectroweakMixing.noOtherDepthTwoPreimage; SFT.Physics.ElectroweakMixing.nativeMixing; Timeline: Turn 279, line 2804.

227. Joint synchronization and retained readiness

The synchronization carrier retains each held participant, its source transition system and readiness state, and the common rendezvous label. The complete observation ledger records every local transition outcome. Its blocked subledger contains exactly the rows whose original transition operation fails. No early failure discards another participant's observation.

Native proofs establish that a joint release exists exactly when every held participant has an enabled source transition. Every released transition belongs to its original transition relation; participant identities and source systems are preserved. A blocked participant excludes release. The finite joint trace retains its exact intermediate readiness states, one for each rendezvous label. Identity, serial trace composition, parallel participant assembly and successor extension retain every prior participant and trace record. These statements use the registered process-transition boundary without introducing a timing or lock implementation.

The complete 256-form synchronization grammar, three source operational witnesses, four controls and every original program output field and hash agree with pinned V3. Together with the general readiness, release, blocked-row and retained-composition derivations, this completes the registered Fold Synchronization entry.

For rendezvous label a and participant $p = (i, M, s)$, the full observation row is

$$O_a(p) = (p, \mathrm{step}(M, s, a)).$$

The ledger maps this operation over every participant; filtering its error outcomes gives exactly the blocked rows. Its participant projection recovers the original complete list.

Joint release has the empty-list identity and a successor for each enabled transition:

$$\frac{\mathrm{step}(M, s, a) = \mathrm{ok}\, t \quad \mathrm{Joint}(a, P, Q)}{\mathrm{Joint}(a, (i, M, s) :: P, (i, M, t) :: Q)}.$$

Replay and reconstruction prove

$$\mathrm{release}(a, P) = \mathrm{some}\, Q \iff \mathrm{Joint}(a, P, Q),$$

$$\exists Q, \mathrm{release}(a, P) = \mathrm{some}\, Q \iff \forall p \in P, \exists t,\ \mathrm{step}(p.M, p.s, a) = \mathrm{ok}\, t.$$

A held failed transition contradicts this condition. Induction preserves participant and system identity lists, and reconstructs every enabled transition's original source row.

For a finite label trace, each successor retains the prior readiness list before its joint release. The state-list count therefore equals the label-list count. Serial composition concatenates both labels and retained states through the exact middle participant list. Parallel assembly combines the two joint derivations, and its observation ledger is the concatenation of their complete ledgers.

> Source, proof and provenance: V3 SYNCHRONIZATION; V3 causal_closure; V3 run; SFT.Computation.Synchronization.jointAssembly; SFT.Computation.Synchronization.exactReadiness; SFT.Computation.Synchronization.blockedRowsComplete; Timeline: Turn 284, line 2855.

228. Communication and exact record transfer

A communication record retains the sender and receiver, message identity, both endpoint events and the actual channel relation row. The native construction enumerates the complete held event/channel product. A delivery is retained exactly when its events and relation row belong to their source ledgers and its endpoint, content and message coordinates agree. Missing send, receive or channel records exclude a purported delivery. No reliability, ordering or delay assumption follows from these records.

Delivery traces retain every intermediate event and channel row. Serial composition requires the same intermediate event; parallel assembly retains both complete component ledgers. Message identity persists through every admitted finite trace. Every distinct pair of sent contents is classified by whether the received contents remain distinct or coincide. The retained and lost ledgers are exhaustive and disjoint; their records retain both source deliveries. Extending the held ledgers preserves earlier deliveries, and extending a delivery trace preserves its earlier pair distinctions. Recorded transformations connect to the existing information-conservation ledger and its downstream loss-preservation theorem.

The complete 256-form Communication grammar, original operational witnesses, controls and every program output field and hash reproduce pinned V3. With the native delivery and distinction derivations, this completes the registered Fold Communication entry.

For send event s, receive event t, and channel row r, compatibility is the conjunction

$$s.\mathrm{process} = r.\mathrm{sender}, \quad t.\mathrm{process} = r.\mathrm{receiver}, \quad s.\mathrm{content} = r.\mathrm{input}, \quad t.\mathrm{content} = r.\mathrm{output}, \quad s.\mathrm{message} = t.\mathrm{message}\,.$$

The complete candidate list is the ordered product of all sends, receives and channel rows. Filtering it gives

$$(s,t,r) \in \mathrm{deliveries}(S,T,R) \iff s \in S \wedge t \in T \wedge r \in R \wedge \mathrm{compatible}(s,t,r).$$

Thus absence of any one source record excludes the delivery. Trace composition requires the exact shared intermediate event, and induction preserves message identity from first to last event.

For a delivery list $D = d :: R$, its ordered occurrence-pair enumeration is

$$\mathrm{pairs}(d :: R) = [(d,e) : e \in R] +\!\!+ \mathrm{pairs}(R), \qquad \mathrm{pairs}([]) = [].$$

Among these pairs with unequal sent contents, retained rows have unequal received contents and lost rows have equal received contents. Equality case analysis proves exhaustiveness; equality and inequality cannot both hold, proving disjointness. Both endpoints reconstruct as original deliveries. Appending

delivery records preserves earlier pairs and their classifications. Mapping each delivery to its sent/received content pair supplies the recorded information transformation; its admitted downstream composition preserves distinctions already closed by that map.

> Source, proof and provenance: V3 COMMUNICATION; V3 causal_closure; V3 run; SFT.Computation.Communication.deliveryMeaning; SFT.Computation.Communication.serialComposition; SFT.Computation.Communication.deliveryDistinctionExhaustion; Timeline: Turn 287, line 2876.

229. Distributed partial orders and retained event identities

The distributed event ledger retains the event carrier, strict causal closure and a separate reflexive identity row for every held event. Native proofs derive reflexivity, antisymmetry and transitivity from the original admitted causal closure. Every nonidentity row reconstructs from its original process or message provenance. Incomparability is exactly the absence of both directions in the reflexive relation, and its endpoints are distinct held events.

The linear-extension support enumerates every permutation satisfying all strict causal rows. It does not choose a total global order for incomparable events: the two-event independent carrier retains both orders. Admitted source assembly preserves earlier identity and causal rows. Exhaustive execution on all 512 relations over three events agrees with independent closure,cycle rejection,identity,incomparability and linear-extension computations.

For event carrier V and admitted strict causal output C, retain the reflexive relation separately:

$$I_V = [(v, v) : v \in V], \qquad R = I_V +\!\!+ C.$$

Membership is exactly

$$(a, b) \in R \iff (a \in V \wedge a = b) \vee (a, b) \in C.$$

Reflexivity follows from the identity rows. Two strict opposite directions would form a positive cycle, so antisymmetry follows from causal admission. For transitivity, identity cases reduce to the other premise, and the strict/strict case uses causal path composition. Every nonidentity row therefore retains the original process/message provenance.

The incomparable-pair ledger contains precisely

$$a, b \in V, \qquad (a, b) \notin R, \qquad (b, a) \notin R.$$

Its endpoints must be distinct because each held identity is in R. The linear-extension list retains all permutations respecting every strict edge:

$$s \in \text{extensions} \iff \text{Perm}(s, V) \wedge \forall (a, b) \in C, \text{Before}(a, b, s).$$

For independent events a, b, both $[a, b]$ and $[b, a]$ are retained. Admitted source extension preserves both prior identity rows and prior causal proofs.

Fold distributed partial-order law is complete: the native derivations,complete 256-form grammar,original witnesses and controls,and all program outputs and hashes agree with pinned V3.

> Source, proof and provenance: V3 PARTIAL-ORDER; V3 causal_closure; V3 run; SFT.Computation.DistributedOrder.antisymmetric; SFT.Computation.DistributedOrder.transitive; SFT.Computation.DistributedOrder.completeLinearExtensions; Timeline: Turn 288, line 2879.

230. Consensus under a declared communication and fault boundary

A consensus record retains the process inputs,proposal values,decision states,complete message deliveries,communication model,fault model and termination declaration. Native acceptance is equivalent to complete process coverage,a common terminal value drawn from the original proposals,admitted message provenance and explicit boundary and termination declarations. Every process has that terminal decision; two held decision rows therefore agree. A pending decision state excludes terminal acceptance. The termination field is an explicit Boolean declaration checked alongside the complete terminal-decision rows.

Composition checks equality of the declared communication and fault boundaries and retains both input,decision and message ledgers. Compatible accepted records with the same terminal value assemble to an accepted record. The one-process identity record is accepted under its named boundary,and adjoining a process with that common proposal preserves acceptance and every earlier input/message record. Six hundred concrete acceptance cases agree with independent evaluation of the same conditions.

Let P be the process identities projected from the input rows, D the decision rows, and v the proposed common terminal value. Coverage requires

$$\forall p \in P, \exists d \in D,\ d.\,\mathrm{process} = p, \qquad \forall d \in D,\ d.\,\mathrm{process} \in P.$$

The exact acceptance condition conjoins this coverage with nonempty communication and fault names, a true termination declaration, v belonging to the original proposals,

$$\forall d \in D,\ d.\,\mathrm{decision} = \mathrm{some}\, v,$$

and source admission of every retained message delivery. Consequently every process has row $(p, \mathrm{some}\, v)$, every two decisions agree, and the common value reconstructs from an original input. A row (p, none) contradicts the terminal condition.

The public composition operation first requires equality of the two communication-model names and fault-model names. With that compatibility and acceptance of both records at the same v, concatenating their inputs, decisions, sends, receives, channels and messages preserves acceptance; their termination flags are conjoined. A one-process record with its own proposal and terminal decision is accepted under named boundaries. Appending such a record preserves the earlier input and message ledgers. The theorem applies to any record satisfying these conjuncts; every-process decision follows by selecting its coverage row and substituting the common terminal value into that row.

Fold consensus law is complete: the native derivations,complete 256-form grammar,original witnesses and controls,and all program outputs and hashes agree with pinned V3.

The proof separates two consequences of acceptance. `terminationExplicit` projects the stored declaration. `everyProcessDecides` uses coverage to obtain an actual decision row for each process and uses the all-terminal check to identify its decision with the common value. Thus the latter conclusion reconstructs the retained terminal decisions, while the former reports exactly the declared termination field. acceptanceMeaning; everyProcessDecides; terminationExplicit.

> Source, proof and provenance: V3 CONSENSUS; V3 causal_closure; V3 run; SFT.Computation.Consensus.acceptanceMeaning; SFT.Computation.Consensus.agreement; SFT.Computation.Consensus.everyProcessDecides; Timeline: Turn 288, line 2879.

231. Agreement impossibility from indistinguishable predecessors

The carrier retains two executions,their predecessor identities,complete traces,local observations,required decisions and schedules. The declared grammar retains its asynchronous and fault boundary and complete scheduler support. A contradiction witness exists exactly when both executions belong to that support,their local observations agree,and their predecessor identities and required decisions differ. The native pair enumeration includes every such supported witness.

An observation-only decision rule must return the same value on identical local observations. If it satisfied both supported executions,it would therefore equate their different required decisions. Native Lean proves the contradiction and excludes a correct observation-only solver for any grammar containing that witness. This is relative to the held grammar: an absent schedule rejects the witness,and transfer to a different grammar requires new support evidence for both executions.

Extending the source support preserves earlier witnesses and execution records. Extending both executions by the same observed event preserves observation equality and the hidden predecessor distinction;the contradiction persists when the resulting schedules are admitted. An execution paired with itself cannot be a contradiction witness. All 1,024 concrete support,observation,predecessor and decision checks agree with independent evaluation of the same conditions.

An execution e retains predecessor identity p_e, trace T_e, observation o_e, required decision d_e and schedule s_e. For grammar G, support means

$$e \in G.\,\mathrm{executions} \wedge s_e \in G.\,\mathrm{schedulerSupport}\,.$$

The contradiction-witness condition is exactly

$$W_G(a,b) \iff \mathrm{supported}_G(a) \wedge \mathrm{supported}_G(b) \wedge o_a = o_b \wedge p_a \neq p_b \wedge d_a \neq d_b.$$

The complete ordered execution product, filtered by this condition, contains (a,b) exactly when $W_G(a,b)$ holds.

Suppose an observation-only rule f were correct on every supported execution, so $f(o_e) = d_e$. Congruence applied to $o_a = o_b$ gives

$$d_a = f(o_a) = f(o_b) = d_b,$$

contradicting the witness's different required decisions. Therefore

$$W_G(a,b) \Longrightarrow \neg\exists f,\ \forall e, \mathrm{supported}_G(e) \Rightarrow f(o_e) = d_e.$$

An absent schedule rejects support and hence the witness. Transfer to another grammar requires supportedness there for both original executions. Extending execution and scheduler lists preserves old support. Appending the same observed event to both traces preserves observation equality and leaves predecessor and required-decision identities unchanged, so the contradiction persists whenever both extended executions are supported. Pairing an execution with itself cannot satisfy $d_e \neq d_e$.

Fold agreement-impossibility law is complete: its native derivations,full256 grammar,original witnesses/controls and every program output field/hash agree with pinned V3.

> Source, proof and provenance: V3 AGREEMENT-IMPOSSIBILITY; V3 causal_closure; V3 run; SFT.Computation.AgreementImpossibility.enumerationComplete; SFT.Computation.AgreementImpossibility.contradiction;

> SFT.Computation.AgreementImpossibility.noObservationOnlySolver; Timeline: Turn 289, line 2884.

232. Replication,consistency observations and retained update order

A replica record retains its identity and current source state. Each update retains its identity,sender,receiver and source transition label. An admitted update belongs to the registered update carrier and propagation relation,holds its sender,and changes a recipient through the original transition operation. Native proofs preserve all replica identities and their number,retain unaffected copies,and reconstruct the changed state from an original transition row.

A replay retains every update and intermediate replica state. Serial composition matches the complete intermediate copy ledger;parallel assembly retains the other copies. Appending an admitted update extends both the event history and state trace while preserving earlier records. Consistency is the explicit agreement of every retained state with the observed value. State classes contain exactly their matching copies,are complete and disjoint,and distinct observed states reject a common consistency value. No platform,wall clock or merge policy is introduced.

The order-loss ledger retains two different update histories that reach identical terminal copies. The native theorem proves that no function of those terminal copies alone can reconstruct both histories. Two concrete source-bound replay orders establish such a witness: each reaches the same two updated copies,while both event orders remain in the trace record. 726 exact consistency/class outputs and all three replay/order-loss proofs pass with zero axiom dependencies.

An update u is admitted only when it belongs to the registered update list, its sender/receiver pair belongs to the propagation relation, and its sender is held in the replica list. Its change constructor selects a replica with identity $u.\mathrm{receiver}$ and applies

$$\mathrm{step}(M, s, u.\mathrm{label}) = \mathrm{ok}\, t.$$

It replaces that selected replica's state s by t, retaining its identity and all other list positions. Induction over the selection gives

$$\mathrm{ids}(R') = \mathrm{ids}(R), \qquad |R'| = |R|,$$

and preserves every held copy with a different identity. The changed state reconstructs from the original transition row $(s, u.\mathrm{label}, t)$.

A replay successor retains its update and prior replica list before continuing from the changed list. Consequently the update and prior-state lists have equal lengths, and serial replay concatenates both through the exact middle replica list. Consistency and state-class membership are

$$\mathrm{consistent}(R, v) \iff \forall r \in R,\ r.\mathrm{state} = v, \qquad r \in \mathrm{class}(R, v) \iff r \in R \wedge r.\mathrm{state} = v.$$

Each copy belongs to its own state class; different classes cannot share a copy. Two held unequal states contradict consistency at any common v.

If two retained replays have histories $U \neq V$ but the same terminal copies F, any proposed history recovery h would have to satisfy

$$U = h(F) = V,$$

a contradiction. Keeping the complete histories supplies the distinction that their common terminal observation omits.

Fold replication and consistency law is complete: its native derivations,full256 grammar,original witnesses/controls and every program output field/hash agree with pinned V3.

> Source, proof and provenance: V3 REPLICATION-CONSISTENCY; V3 causal_closure; V3 run; SFT.Computation.Replication.consistencyMeaning; SFT.Computation.Replication.nativeOrderLossCannotRecover; Timeline: Turn 290, line 2892.

233. Declared computational fault support and recovery

The fault carrier retains nominal transition/observation rows,generated fault actions,the affected component and its observed outcome,and the declared recovery relation. Native proofs identify a deviation exactly by generated membership and exclusion from the nominal transition relation. Detection distinguishes observations absent from nominal support;identical observations remain indistinguishable to an observation-only decision.

Classification first checks generated support. An ungenerated action is outside the guarantee even when a matching recovery row exists. A generated action is uncorrectable exactly when its matching recovery ledger is empty;otherwise every matching declared row is retained. Matching preserves both the affected component and the post-fault recovery source state. The global recovery guarantee is proved equivalent to the existence of a matching row for every generated fault,without extending it to ungenerated actions.

Recovery traces retain every declared row and exact intermediate state interface. Identity and serial composition preserve the complete trace. Source assembly and successor extension retain earlier fault/recovery records;two generated-support guarantees compose on the combined declared model. All 3,328 concrete classification,detection,deviation and recovery-guarantee checks agree with independent evaluation.

For nominal transition/observation rows N, generated faults G, and recovery rows R, define

$$\mathrm{deviation}(f) \iff f \in G \wedge f.\mathrm{action} \notin \mathrm{fst}(N), \qquad \mathrm{detected}(f) \iff f.\mathrm{observation} \notin \mathrm{snd}(N).$$

The recovery list is exactly

$$R_f = [r \in R : r.\mathrm{component} = f.\mathrm{component} \wedge r.\mathrm{before} = f.\mathrm{after}].$$

Classification first tests $f \in G$: absence gives outside; membership with $R_f = []$ gives uncorrectable; membership with $R_f \neq []$ returns the whole recoverable list R_f. Thus a recovery row alone does not admit an ungenerated fault.

The complete guarantee is

$$\mathrm{allGeneratedCorrectable} = \mathrm{true} \iff \forall f \in G, \exists r \in R_f.$$

Each matching row gives a one-step recovery trace from the fault's resulting state on the same component. Serial recovery concatenates through the shared intermediate state. Appending declared models preserves both components' generated faults and recovery witnesses, so their two complete guarantees imply the combined guarantee. Detection remains observation-relative: if a fault observation equals a nominal observation, every function of that observation gives the same answer on both.

Fold computational fault-model law is complete: its native derivations,full256 grammar,original witnesses/controls and every program output field/hash agree with pinned V3.

> Source, proof and provenance: V3 FAULT-MODEL; V3 causal_closure; V3 run; SFT.Computation.FaultModel.recoveryMeaning; SFT.Computation.FaultModel.exactGuarantee; Timeline: Turn 291, line 2897.

234. Distributed knowledge from local observations

The admitted carrier retains finite global execution support and complete local observation maps. An individual knowledge ledger contains exactly the propositions true at every supported execution with the same local observation. Native proofs establish truth at the actual execution, invariance under equal observations, and rejection by an indistinguishable counterexample.

Common knowledge iterates the union of registered local indistinguishability relations, including each supported identity, to its finite fixed point. The construction terminates by exhausting missing supported pairs and has no imposed iteration cutoff. The resulting relation is proved equivalent to retained paths and closed under composition. Common knowledge implies every registered observer's individual knowledge. Decision admission requires validated observation maps and a supported actual execution. The concrete checks include a two-observer chain in which individual knowledge holds but common knowledge fails.

For supported executions E, observer map o, actual execution a, and truth support T, define

$$[a]_o = [e \in E : o(a) = o(e)], \qquad K_o(a,T) \iff \forall e \in [a]_o,\ e \in T.$$

The actual supported execution belongs to its own class, so $K_o(a,T)$ implies $a \in T$. Equal observations give equal class membership conditions and therefore the same knowledge decision. An indistinguishable $e \notin T$ rejects that decision.

The common-knowledge base relation contains every supported identity and every pair identified by at least one registered observer:

$$B = \{(e,e) : e \in E\} \cup \{(a,b) \in E \times E : \exists o,\ o(a) = o(b)\}.$$

Its finite native closure B^+ is equivalent to retained positive paths in B; the explicit identity rows provide reflexivity. The missing-pair decrease proves termination and transitivity. Common knowledge is

$$K_{\text{common}}(a,T) \iff \forall e \in E,\ \text{Path}(B,a,e) \Rightarrow e \in T.$$

Every individual indistinguishability edge is a base path, so common knowledge implies each registered observer's individual knowledge. The public decision additionally requires the original execution-support and observation-map admission and actual membership $a \in E$. A chain of different observers' edges may reach a false execution outside one observer's immediate class, explaining the retained distinction between individual and common knowledge.

Fold distributed-knowledge law is complete: its native derivations,full256 grammar,original witnesses/controls and every program output field/hash agree with pinned V3.

> Source, proof and provenance: V3 DISTRIBUTED-KNOWLEDGE; V3 causal_closure; V3 run; SFT.Computation.DistributedKnowledge.individualMeaning; SFT.Computation.DistributedKnowledge.commonMeaning; SFT.Computation.DistributedKnowledge.commonTruth; Timeline: Turn 292, line 2907.

235. Locality from registered communication neighborhoods

The network carrier retains each process identity and every directed communication row. A process belongs to a finite-radius neighborhood exactly when a retained registered trace reaches it within that radius. The radius bounds the length of the communication trace; it is not a fitted scientific parameter. The proof includes the identity case and every generated successor. All trace endpoints and communication rows remain registered, and serial composition and network extension preserve earlier traces.

Local views retain each reachable process identity together with its input observation. Identical local views force equal outputs for any decision depending only on that view. Consequently, two locally indistinguishable instances requiring different outputs provide a lower witness. An unregistered starting process is rejected, and a trace using an omitted communication row cannot be admitted. The concrete checks exhaust the 64 directed three-process graphs without self-edges at four successive radii, alongside outside-carrier and local-view distinction cases: 834 kernel checks in total.

For process carrier V and directed rows R, the finite-radius neighborhood recurrence is

$$N_0(p) = [q \in V : q = p],$$

$$N_{r+1}(p) = [q \in V : q = p \vee \exists b \in V,\ (p,b) \in R \wedge q \in N_r(b)].$$

A retained reachability trace has either the supported identity or one registered edge followed by a trace from its endpoint. Induction in both directions proves, for $p \in V$,

$$q \in N_r(p) \iff \exists T, \mathrm{Reach}(p, T, q) \wedge |T| \leq r.$$

Every trace endpoint belongs to V and every edge belongs to R. Serial traces concatenate, while inclusion of process and edge supports transports each earlier trace unchanged.

The local view is the ordered list

$$\mathrm{view}(p, r) = [(q, \mathrm{image}(I, q)) : q \in N_r(p)].$$

Every view row reconstructs its reachable process. If two views A, B are equal, congruence gives $f(A) = f(B)$ for any view-only decision f. Thus $A = B$ and required outputs $x \neq y$ imply

$$\neg\exists f,\ f(A) = x \wedge f(B) = y.$$

The query rejects $p \notin V$, and a missing edge contradicts trace provenance.

Fold locality law is complete: its native derivations,full256 grammar,original witnesses/controls and every program output field/hash agree with pinned V3.

> Source, proof and provenance: V3 LOCALITY; V3 causal_closure; V3 run; SFT.Computation.Locality.exactNeighborhood; SFT.Computation.Locality.indistinguishableOutput; SFT.Computation.Locality.lowerWitness; Timeline: Turn 293, line 2916.

236. Computation over generated communication networks

The network carrier retains registered local machines, link channels, schedule support, local actions, resources and faults. Each admitted action is tied to an actual transition row of its registered local machine. Each message retains both endpoint events and the exact channel transformation, with registered endpoint processes. An execution retains its initial form, admitted action schedule, every intermediate state, complete messages and aggregate result. Every output process is proved to come from a registered local machine.

Resource and fault ledgers preserve the ordered action records, and every ledger item is proved to belong to its declared support. Identity and serial execution preserve traces and process identities; parallel component changes retain the other component. Componentwise extension preserves earlier machine, message, schedule, resource and fault records and carries the complete earlier execution into the extended network. A concrete two-process execution checks state changes, message delivery, resource and fault records, plus rejection of missing transitions, channels, sends and incompatible message identities. Operating systems, cloud platforms and physical networks remain downstream validation domains.

A registered network action a carries process, before-state, label, after-state, resource list and fault list. Admission retains

$$a \in A, \quad \exists m \in M,\ m.\mathrm{process} = a.\mathrm{process} \wedge \mathrm{step}(m.\mathrm{system}, a.\mathrm{before}, a.\mathrm{label}) = \mathrm{ok}(a.\mathrm{after}),$$

and membership of every action resource and fault in their declared network supports. A delivery additionally requires the original communication admission and registered machines at both endpoint processes.

The state-change constructor replaces a selected node $(a.\mathrm{process}, a.\mathrm{before})$ by $(a.\mathrm{process}, a.\mathrm{after})$. Induction preserves the process-identity list. A complete execution retains a registered action schedule, the full state trace, all admitted messages and registered initial processes. Therefore every final process reconstructs from an original local machine.

The complete resource and fault ledgers are

$$\mathcal{R} = \mathrm{flatMap}(\mathrm{resources}, A_{\mathrm{run}}), \qquad \mathcal{F} = \mathrm{flatMap}(\mathrm{faults}, A_{\mathrm{run}}).$$

Trace provenance supplies admission of each action, so every member of these ledgers belongs to its declared support. Componentwise network inclusion transports machine, event, channel, action, schedule, resource and fault witnesses. The resulting extended execution has exactly the same actions, intermediate states, messages and final nodes.

Fold network-computation law is complete: its native derivations,full256 grammar,original witnesses/controls and every program output field/hash agree with pinned V3.

> Source, proof and provenance: V3 NETWORK-COMPUTATION; V3 causal_closure; V3 run; SFT.Computation.NetworkComputation.traceProvenance; SFT.Computation.NetworkComputation.messageSource; SFT.Computation.NetworkComputation.resultProcessSource; Timeline: Turn 294, line 2925.

237. Resource-relative one-wayness and exact security partitions

The forward carrier retains a finite source support and a total forward map. Each inverse fibre contains exactly the supported predecessors of its observed image. Every forward image has a retained predecessor, so unrestricted finite inversion is not declared impossible. Inverse success means that a proposed supported predecessor maps exactly to the observed value.

An adversary attempt retains its identity, observed challenge, proposed predecessor, machine actions and complete state trace. Admission requires membership in the registered process grammar, exact execution of the held machine, and compliance with its declared resource bound. The complete success ledger retains every registered attempt. A native equivalence connects computed failure over all admitted bounded attempts to the absence of any admitted successful inverse. The concrete resource example fails at its smaller registered bound and succeeds when the larger bound admits the successful retained trace. This is a resource-relative result, not an imported hardness assumption.

The original security partition operation requires duplicate-free message support and exactly one observation for every message. It returns lexically ordered observation classes with lexically ordered members. The empty support retains its original empty result. Native proofs establish the exact validation condition, class membership, predecessor membership and disjointness of distinct observation classes. The 1,305 concrete checks include duplicate, missing, extra and reordered records, equal observations, distinct observations, inverse proposals and resource-bound outcomes.

For supported forward map F, its inverse fibre is

$$F^{-1}(y) = \{x \in S : \mathrm{image}(F, x) = \mathrm{some}\, y\}.$$

Totality on S supplies such a fibre containing every original input. An attempt a succeeds exactly when

$$a.\,\mathrm{proposed} \in S \wedge \mathrm{image}(F, a.\,\mathrm{proposed}) = \mathrm{some}(a.\,\mathrm{observed}).$$

Admission is separately

$$a \in G.\,\mathrm{attempts} \wedge \mathrm{run}(G.M, G.i, a.\,\mathrm{actions}) = \mathrm{ok}(a.\,\mathrm{states}) \wedge |a.\,\mathrm{actions}| \leq G.\,\mathrm{bound}\,.$$

Filtering by the latter conditions yields exactly the admitted attempts. Conjoining failure across that complete filtered list gives

$$\mathrm{allBoundedFail}(F, G) = \mathrm{true} \iff \forall a, \mathrm{admitted}(G, a) \Rightarrow \neg\, \mathrm{success}(F, a),$$

hence excludes an admitted successful inverse. The resource bound belongs to the retained process grammar; increasing it may admit an already retained successful trace.

The separate security partition succeeds exactly when message support is distinct, observation keys match that support, and observation-row count equals support count. Its class for view v is the lexically ordered fibre $F^{-1}(v)$; class labels are the ordered distinct observed labels. Membership therefore retains both supported predecessor and exact image. If one message belongs to two classes, lookup determinacy equates their labels, proving disjointness of differently labeled classes.

Fold one-wayness law is complete: its native derivations,full256 grammar,original witnesses/controls and every program output field/hash agree with pinned V3.

> Source, proof and provenance: V3 ONE-WAYNESS; V3 exact_security_partition; SFT.Computation.OneWayness.exactBoundedFailure; SFT.Computation.OneWayness.computedNoAdmittedSuccess; SFT.Computation.SecurityPartition.partitionMeaning; Timeline: Turn 295, line 2934.

238. Observation-relative secrecy and disclosure

The carrier retains message support, key or transformation support, exact ciphertext rows and the registered adversary observation. A possible message is reconstructed from a particular supported transformation whose ciphertext has the observed value. The complete ordered pair ledger retains every

supported pair; its closed ledger contains exactly those pairs with distinct messages and equal adversary observations. Closed and separated distinctions exhaust the supported distinct-message pairs and are disjoint.

A closed pair supplies two distinct supported predecessor messages. No decision using only their common registered observation can recover both. Subsequent deterministic processing of that same observation preserves its equality. This conclusion remains relative to the specified adversary observation: an explicitly additional disclosure that distinguishes the pair also distinguishes the combined view. Missing transformation rows or keys are rejected. Source extension preserves earlier support, transformations and observation values, and therefore preserves earlier possible messages and closed distinctions. The 1,729 concrete checks cover held and missing message/key/transformation support, equal and unequal observations, the complete closed ledger and additional disclosure.

For transformation rows $r = (m, k, c)$, admission requires registered row, supported message and key, and a defined adversary observation $o(c)$. The possible-message ledger has exact membership

$$m \in \mathrm{possible}(v) \iff \exists r,\ r \in R \wedge r.m = m \wedge r.m \in M \wedge r.k \in K \wedge o(r.c) = \mathrm{some}\, v.$$

The complete ordered product of admitted rows is filtered into closed and separated pairs. A closed pair satisfies

$$\mathrm{admitted}(a) \wedge \mathrm{admitted}(b) \wedge a.m \neq b.m \wedge o(a.c) = o(b.c);$$

a separated pair uses inequality of those observations. Equality case analysis is exhaustive and disjoint on supported pairs with distinct messages.

For a closed pair, any observation-only decoder recovering both messages would give

$$a.m = \mathrm{decode}(o(a.c)) = \mathrm{decode}(o(b.c)) = b.m,$$

contradicting distinctness. Deterministic post-processing preserves observation equality by congruence. In contrast, an additional disclosure d with $d(a) \neq d(b)$ separates the combined views $(o(a.c), d(a))$ and $(o(b.c), d(b))$. Extension preserves earlier ambiguity when it retains the message/key/row supports and the original observation on each retained row.

Fold secrecy law is complete: its native derivations, full 256-form grammar, original witnesses and controls, and every program output field and hash agree with pinned V3.

> Source, proof and provenance: V3 SECRECY; V3 exact_security_partition; SFT.Computation.Secrecy.noObservationRecovery; SFT.Computation.Secrecy.closedLedgerMeaning; SFT.Computation.Secrecy.disclosureBoundary; Timeline: Turn 296, line 2943.

239. Integrity of protected records

The carrier retains message identities, protected records, allowed transformations, adversarial changes and the exact verification relation. Acceptance requires a registered change, its protected source record and the corresponding before/after verification row. The accepted and rejected ledgers exhaust the supported change domain and are disjoint. A separate false-accept ledger retains exactly the accepted changes outside the allowed transformation support.

The computed integrity check is proved equivalent to authorization of every accepted change. Its successful result excludes every unauthorized accepted change in the registered support; an actual unauthorized acceptance witnesses failure of that check. The verifier is not assumed sound. Retained execution traces reconstruct each accepted change with matching record interfaces. Under the checked

integrity result every trace change is authorized. Identity, serial composition and source extension preserve the actual trace and its source records. The 1,026 concrete checks include incomplete support and verification relations, complete verdict ledgers, a two-step execution and a verifier that accepts an unauthorized change. Integrity alone makes no secrecy or authorship claim.

For protected records R, generated changes C, allowed changes A, and verification relation V, exact acceptance is

$$\mathrm{accept}(c) \iff c \in C \wedge c.\mathrm{before} \in R \wedge (c.\mathrm{before}, c.\mathrm{after}) \in V.$$

Supported changes with the verification row form the accepted ledger; supported changes without it form the rejected ledger. These exhaust the supported domain and are disjoint. The false-accept ledger is

$$F = [c \in C : \mathrm{accept}(c) \wedge c \notin A].$$

The computed integrity condition satisfies

$$\mathrm{integrity} = \mathrm{true} \iff \forall c, \mathrm{accept}(c) \Rightarrow c \in A.$$

Thus any actual accepted $c \notin A$ is a concrete refutation, rather than an excluded possibility assumed away.

A trace constructor retains each accepted change's exact before/after interface and continues from that after-record. Induction reconstructs acceptance of every trace row. Applying a successful integrity check then proves authorization of every row. Identity and serial trace constructors preserve the original interfaces. Inclusion of protected records, changes, allowed rows and verification rows transports earlier acceptance and complete traces into the extended model; it does not by itself assert integrity for newly added changes.

Fold integrity law is complete: its native derivations, full 256-form grammar, original witnesses and controls, and every program output field and hash agree with pinned V3.

> Source, proof and provenance: V3 INTEGRITY; V3 exact_security_partition; SFT.Computation.Integrity.exactIntegrity; Timeline: Turn 297, line 2952.

240. Authentication relative to registered identity support

The carrier retains principal identities, credential ownership, challenge records, exact response rows and adversary attempts. Acceptance is proved equivalent to a complete retained verification trace: the attempt, claimed principal and challenge are registered; a held credential matches the claimed owner; and its exact credential/challenge/response row is present. Unsupported attempts, challenges and responses are rejected.

Credential acceptance is not assumed to establish the actual actor. The complete impersonation ledger contains exactly the accepted attempts whose actor differs from the claimed principal. The computed authenticity check is equivalent to equality of actor and claim for every accepted supported attempt. Its success excludes impersonation in that registered support, while an accepted impersonation witnesses failure. Componentwise source extension preserves earlier verification traces and accepted records. The 4,100 concrete checks include missing support, incorrect credential ownership, absent response rows, a retained verification trace and an explicitly accepted impersonation. Identity claims therefore retain V3’s exact credential and adversary boundary.

Write A, P, C, Q, R for the registered attempts, principals, credentials, challenges and response rows. For an attempt a, the exact acceptance predicate is

$$\mathrm{Accept}(a) \iff a \in A \wedge a_{\mathrm{claimed}} \in P \wedge a_{\mathrm{challenge}} \in Q \wedge \exists c \in C\,(c_{\mathrm{identity}} = a_{\mathrm{credential}} \wedge c_{\mathrm{principal}} = a_{\mathrm{claimed}}) \wedge (a_{\mathrm{credential}}, a_{\mathrm{challenge}}, a_{\mathrm{response}}) \in R.$$

Filtering C by the two credential equalities gives the witness list. Nonemptiness supplies its first witness; conversely any matching member proves nonemptiness. Together with the three support checks and response membership, this proves the equivalence with VerificationTrace. Thus the authenticity derivation is

$$I = [a \in A \mid \mathrm{Accept}(a) \wedge a_{\mathrm{actor}} \neq a_{\mathrm{claimed}}], \qquad \mathrm{Authentic} = \mathrm{true} \iff \forall a,\ \mathrm{Accept}(a) \Rightarrow a_{\mathrm{actor}} = a_{\mathrm{claimed}}.$$

An element of I contradicts the right-hand equality directly. Componentwise extension preserves every conjunct of Accept and therefore the complete trace; it does not remove the requirement to check authenticity over any newly added attempts.

Fold authentication law is complete: its native derivations, full 256-form grammar, original witnesses and controls, and every program output field and hash agree with pinned V3.

> Source, proof and provenance: V3 AUTHENTICATION; V3 exact_security_partition; SFT.Computation.Authentication.exactAuthenticity; SFT.Computation.Authentication.acceptedImpersonationFails; Timeline: Turn 298, line 2961.

241. Hashing fibres, collisions and domain separation

The finite carrier retains message support, a deterministic map and its complete declared output support. The preimage ledger retains every supported message in its exact image fibre. The collision ledger contains every and only supported distinct-message pair with equal output.

For duplicate-free message support larger than the declared output support, native finite counting proves that the map cannot be injective. The complete collision ledger is therefore nonempty and supplies an actual supported pair with distinct messages and equal output. This derives the collision witness without assuming a conventional hashing model or collision-resistance premise. Deterministic downstream composition preserves an already equal image. Held domain labels distinguish domain-separated output records, while equal labels preserve precisely the underlying output equality.

Source extension preserves earlier messages, outputs and map values, hence earlier predecessor and collision witnesses. Concrete checks cover all eight maps from three messages to two declared outputs, their full fibres and collision ledgers, empty support and domain labels. Each of the eight maps also instantiates the general larger-domain collision theorem. Collision resistance retains its source resource boundary; the existence of a finite collision is not denied.

Let M be the message list, Y the declared output list and h the map, with $x \in M \Rightarrow h(x) \in Y$. Ordered filters give

$$F_y = [x \in M \mid h(x) = y], \qquad C = [(a, b) \in M \times M \mid a \neq b \wedge h(a) = h(b)].$$

The preimage ledger is $[(y, F_y) \mid y \in Y]$; every source message belongs to its own $F_{h(x)}$. If M has no duplicates and $|Y| < |M|$, injectivity on M would inject its distinct members into the declared finite output list, contradicting the finite cardinality bound. Hence

$$|Y| < |M|,\ \mathrm{distinct}(M) \Longrightarrow C \neq [] \Longrightarrow \exists a, b \in M,\ a \neq b \wedge h(a) = h(b).$$

The last implication extracts a pair from the complete nonempty ledger. For any downstream map g, equality substitution gives $g(h(a)) = g(h(b))$. Domain separation is the pair construction $h_d(x) = (d, h(x))$:

unequal first coordinates force unequal outputs, while $(d, u) = (d, v) \iff u = v$. An extension retaining map values on old messages consequently retains their fibres and collisions.

Fold hashing law is complete: its native derivations, full 256-form grammar, original witnesses and controls, and every program output field and hash agree with pinned V3.

> Source, proof and provenance: V3 HASHING; V3 exact_security_partition; SFT.Computation.Hashing.fibreMeaning; SFT.Computation.Hashing.largerDomainCollision; Timeline: Turn 299, line 2970.

242. Commitment binding, hiding and retained phases

The carrier retains message and opening support, exact commit and verification relations, and the declared observation of each commitment. The complete accepted-pair ledger supplies the computed binding check. Its successful result is proved equivalent to message agreement for every verified pair sharing a commitment. The collision ledger contains exactly the verified equal-commitment pairs with different messages and gives a concrete failure witness.

The hiding observation classes retain the exact generated commit rows. A class containing distinct messages prevents their simultaneous recovery from that observation alone. Binding and hiding are not declared unconditionally incompatible: the native boundary specifies when equality of observations forces equality of commitments, yielding conflicting verified openings. A checked example retains binding for distinct commitments while a coarser registered observation hides their messages.

Commit and opening traces retain actual source rows and their shared commitment interface. Under the checked binding result, two accepted openings of the same retained commitment agree on the message. Source extension preserves the generation and verification rows, phase interfaces and earlier observation classes. The 2,313 concrete outcomes cover finite commit, verification and observation supports, explicit missing message/opening/source rows, collisions and the compatible binding-and-hiding example.

For a row $r = (m, o, c)$, put $S(r) \iff m \in M \wedge o \in O$, $G(r) \iff S(r) \wedge r \in C$, and $V(r) \iff S(r) \wedge r \in V_0$, where C, V_0 are the commit and verification lists. The accepted list is V_0 filtered by S. Exhaustive pairing derives

$$\mathrm{Binding} = \mathrm{true} \iff \forall a, b,\ V(a) \wedge V(b) \wedge a_c = b_c \Rightarrow a_m = b_m,$$

$$B = [(a, b) \mid V(a) \wedge V(b) \wedge a_c = b_c \wedge a_m \neq b_m].$$

Here the comprehension for B traverses the ordered accepted-pair list. Each member refutes Binding by applying its equality conclusion to the unequal messages. An opening trace adds $V(r)$ and $r_c = c_{\mathrm{committed}}$; two such equalities yield the shared-commitment premise and hence equal opened messages.

For observation image O_{obs}, the hiding class is

$$H_v = [r \in C \mid S(r) \wedge O_{\mathrm{obs}}(r_c) = \mathrm{some}(v)].$$

If $a, b \in H_v$ and $a_m \neq b_m$, a decoder recovering both would imply $a_m = d(\mathrm{some}(v)) = b_m$, a contradiction. A binding failure follows only with the additional verified-opening premises and the stated implication from equal observations to equal commitments. The same row memberships and unchanged observation images transport these traces and classes through the registered extension.

Fold commitment law is complete: its native derivations, full 256-form grammar, original witnesses and controls, and every program output field and hash agree with pinned V3.

> Source, proof and provenance: V3 COMMITMENT; V3 exact_security_partition; SFT.Computation.Commitment.exactBinding; SFT.Computation.Commitment.hiddenMessagesCannotRecover; Timeline: Turn 300, line 2979.

243. Signature correctness and exact adversary support

The signature carrier retains message support, key identities, signing and verification relations, accepted signatures and the registered forgery adversary. The computed correctness check is proved equivalent to successful verification of every supported signing row. A successful signing trace therefore produces a verification trace carrying the identical key, message and signature.

The complete forgery ledger contains precisely the registered adversary rows that verify but are absent from the signing relation. The computed unforgeability check is proved equivalent to signing-relation membership for every verified row in that adversary support. Its successful result excludes all such forgery-ledger members. The domain remains the registered adversary support; no number-theoretic hardness premise or unrestricted computational claim is introduced.

Source extension preserves signing and verification rows and their retained traces. Missing keys and rows outside the adversary support have explicit exclusion proofs. All 2,048 concrete outcomes across the finite signing, verification and adversary supports pass, together with the retained native correctness trace.

For $r = (k, m, \sigma)$, let $S(r) \iff k \in K \wedge m \in M$, $G(r) \iff S(r) \wedge r \in G_0$, and $V(r) \iff S(r) \wedge r \in V_0$. The complete checks and forgery ledger are

$$\mathrm{Correct} = \mathrm{true} \iff \forall r,\ G(r) \Rightarrow V(r), \qquad F = [r \in A \mid V(r) \wedge r \notin G_0],$$

$$\mathrm{Unforgeable} = \mathrm{true} \iff \forall r \in A,\ V(r) \Rightarrow r \in G_0.$$

Correctness converts a signing trace $(r, G(r))$ into $(r, V(r))$, retaining k, m, σ exactly. If $r \in F$, the unforgeability implication produces $r \in G_0$, contradicting the ledger's exclusion. The quantifier is over the registered adversary list A. A missing key prevents $S(r)$; a row outside A cannot enter F. Componentwise inclusion preserves the positive signing and verification memberships and their traces, without asserting that a newly enlarged adversary list automatically passes the old security check.

Fold signature law is complete: its native derivations, full 256-form grammar, original witnesses and controls, and every program output field and hash agree with pinned V3.

> Source, proof and provenance: V3 SIGNATURE; V3 exact_security_partition; SFT.Computation.Signature.exactCorrectness; SFT.Computation.Signature.exactUnforgeability; Timeline: Turn 301, line 2988.

244. Proof of knowledge and source-bound extraction

The carrier retains statement support, the statement/witness relation, prover and verifier processes, transcript support, acceptance, the extractor relation and registered adversary support. The computed extraction ledger contains precisely the registered extraction rows whose transcript agrees and whose witness belongs to the statement relation. Its nonempty result is proved equivalent to existence of such a row.

The computed knowledge check covers every accepted transcript in the registered adversary support. Its successful result is proved equivalent to the existence of an exact witness-bearing extraction row for each such transcript. The failure ledger contains exactly the admitted accepted transcripts with no extraction. Knowledge is therefore expressed by the registered extraction relation.

An interaction retains actual prover and verifier runs and their shared terminal/initial interface. Both transition traces reconstruct from their source rows. An accepted interaction under the checked knowledge result yields an actual extractor row and a witness of its statement. Source extension preserves accepted rows, extraction witnesses and earlier interactions. All 1,024 finite support checks pass, together with an explicit interacting prover/verifier and its extracted witness.

For a transcript t, statement support Σ, transcript list T, accepted list A, witness relation W and extractor rows E, define

$$\mathrm{Accept}(t) \iff t_s \in \Sigma \wedge t \in T \wedge t \in A, \qquad E_t = [e \in E \mid e_t = t \wedge (t_s, e_w) \in W].$$

Inspection of the filtered list proves $E_t \neq [] \iff \exists e \in E,\ e_t = t \wedge (t_s, e_w) \in W$. Applying that equivalence at every registered adversary transcript gives

$$\mathrm{Knowledge} = \mathrm{true} \iff \forall t \in D,\ \mathrm{Accept}(t) \Rightarrow \exists e \in E,\ e_t = t \wedge (t_s, e_w) \in W.$$

The failure ledger is $[t \in D \mid \mathrm{Accept}(t) \wedge E_t = []]$. An interaction separately carries

$$\mathrm{run}(P, i, u) = \mathrm{ok}(\tau_P), \quad \mathrm{last?}(\tau_P) = \mathrm{some}(j), \quad \mathrm{run}(V, j, v) = \mathrm{ok}(\tau_V).$$

Run admission reconstructs both transition executions. For an accepted interaction in D, Knowledge supplies the extractor row and its witness relation membership. Extension preserves these exact runs, row memberships and shared interface, so the same transcript and extraction witness remain available.

Fold proof-of-knowledge law is complete: its native derivations, full 256-form grammar, original witnesses and controls, and every program output field and hash agree with pinned V3.

> Source, proof and provenance: V3 PROOF-KNOWLEDGE; V3 exact_security_partition; SFT.Computation.ProofKnowledge.exactKnowledge; SFT.Computation.ProofKnowledge.acceptedExtraction; Timeline: Turn 302, line 2997.

245. Zero knowledge, simulation and closed witness distinctions

Real transcripts retain their statement and witness supports. Simulator rows record a statement and transcript; a retained SimulationTrace additionally certifies the actual process execution and its terminal transcript. The exact observation relation determines the views of each transcript. A matching simulator row is proved to have the identical statement and the same admitted observations as its real transcript. The computed coverage check is equivalent to existence of such a registered simulator row for every supported real row. The failure ledger enumerates every uncovered real row.

Acceptance correctness is proved equivalent to acceptance of every supported real transcript. For distinct witnesses of a common statement, a shared matching simulator view closes their witness distinction: the observed view cannot simultaneously recover both witnesses. This result is relative to the registered observation relation and simulator support.

The complete ordered-pair enumeration supplies closed and retained distinction ledgers. Exact membership theorems establish that these ledgers are disjoint and exhaustive for same-statement, distinct-witness pairs. Source extension preserves real rows, the registered observations and simulator executions. The 3,584 concrete outcomes cover observation relations, simulator support, acceptance, coverage failures and both distinction ledgers, alongside an actual simulator execution and a native closed-witness proof.

Write R for the real rows whose statements and statement/witness pairs are supported, S for the registered simulator rows and O for the observation relation. Define

$$\mathcal{V}(t) = \{v \mid (t, v) \in O\}, \qquad r \sim s \iff r_{\text{statement}} = s_{\text{statement}} \wedge \mathcal{V}(r_{\text{transcript}}) = \mathcal{V}(s_{\text{transcript}}).$$

The equality here is equality of membership, implemented by checking inclusion in both directions; observation-list ordering or multiplicity is not asserted equal. Filtering S by this predicate and testing nonemptiness proves

$$\text{Covered} = \text{true} \iff \forall r \in R,\ \exists s \in S,\ r \sim s.$$

A SimulationTrace additionally proves the actual simulator run and $\text{last?}(\tau_s) = \text{some}(s_{\text{transcript}})$. Those execution fields are separate from the matching-row check.

For same-statement rows $a, b \in R$ with different witnesses, a common matching simulator row gives $\mathcal{V}(a_t) = \mathcal{V}(b_t)$ by transitivity. For any common observed v, simultaneous decoding implies $a_w = d(v) = b_w$, contradicting $a_w \neq b_w$. The ordered pair list $R \times R$, filtered by equal statement and unequal witness, splits according to the Boolean same-views check into disjoint closed and retained ledgers. Coverage failures are exactly the real rows with no matching simulator row; correctness separately checks $r_t \in A$ for every $r \in R$.

Fold zero-knowledge law is complete: its native derivations, full 256-form grammar, original witnesses and controls, and every program output field and hash agree with pinned V3.

> Source, proof and provenance: V3 ZERO-KNOWLEDGE; V3 exact_security_partition; SFT.Computation.ZeroKnowledge.noViewRecovery; SFT.Computation.ZeroKnowledge.exactSimulation; SFT.Computation.ZeroKnowledge.exactCoverage; Timeline: Turn 303, line 3003.

246. Multiparty computation and explicit corruption boundaries

The joint carrier retains party identities, private input rows, the joint output relation, local views and an explicit communication process. Every admitted execution names a corruption set from the registered boundary and retains its actual transition trace. Undeclared corruption sets are excluded.

For each coalition, exact projection retains its private inputs and local views. The complete ordered-pair ledger identifies leakage when two joint rows agree in output and corrupted inputs while their coalition views differ. The computed privacy check is proved equivalent to view agreement for every such pair and excludes every leakage-ledger member. The whole registered-security check applies this property to every declared corruption set.

Source extension preserves earlier joint rows, communication executions and leakage witnesses. All 768 concrete checks over private-input support, joint relations, local views and corruption sets pass. An explicit source-bound communication execution also satisfies the checked registered privacy theorem.

Let J be the joint-relation rows whose private-input owners and local-view owners are registered, with each input itself in the private-input support. For coalition C, retain the source list order in the projections

$$I_C(r) = [i \in r_{\text{inputs}} \mid i_{\text{party}} \in C], \qquad V_C(r) = [v \in r_{\text{views}} \mid v_{\text{party}} \in C].$$

The leakage ledger filters $J \times J$ by

$$a_{\text{output}} = b_{\text{output}} \wedge I_C(a) = I_C(b) \wedge V_C(a) \neq V_C(b).$$

Consequently the all-pairs check has the exact meaning

$$\text{Private}(C) = \text{true} \iff \forall a, b \in J,\ [a_{\text{output}} = b_{\text{output}} \wedge I_C(a) = I_C(b)] \Rightarrow V_C(a) = V_C(b).$$

Each leakage member contradicts this implication. Whole-support security is $\forall C \in \mathcal{C}, \mathrm{Private}(C) = \mathrm{true}$, where $\mathcal{C}$ is the declared corruption list. An execution supplies $C \in \mathcal{C}$ and an actual communication run, so this universal statement applies to its own coalition. Extension preserves the row, coalition membership and run; since its projections are unchanged, an earlier leakage witness also remains a witness.

Fold multiparty-computation law is complete: its native derivations, full 256-form grammar, original witnesses and controls, and every program output field and hash agree with pinned V3.

> Source, proof and provenance: V3 MULTIPARTY; V3 exact_security_partition; SFT.Computation.Multiparty.exactPrivacy; SFT.Computation.Multiparty.exactSecurity; Timeline: Turn 304, line 3012.

247. Adversarial computation and complete outcome ledgers

The carrier retains the system process, registered adversary strategies, resource law, interaction schedules, security states and stopping states. Exact admission requires a generated schedule, a registered adversary and compliance with the declared resource bound. Undefined strategies and over-budget schedules are excluded by native proofs.

The favorable and unfavorable ledgers contain every admitted row according to its terminal security state. Their membership theorems establish an exhaustive and disjoint partition. The computed security check is equivalent to security of every admitted outcome; it excludes all unfavorable rows, while any unfavorable member refutes that checked result.

Actual executions retain the complete communication steps and terminal state. A stopped execution additionally satisfies the registered stopping rule. Source extension preserves earlier schedules, both outcome ledgers and actual stopped traces. All 2,048 concrete checks pass over strategy, schedule, resource and security supports, together with an explicit stopped execution and its native security proof.

For registered schedules L, strategies D, resource bound B and secure terminal states Q, write

$$J = [s \in L \mid s_{\mathrm{strategy}} \in D \wedge |s_{\mathrm{actions}}| \leq B], \qquad J_{+} = [s \in J \mid s_{\mathrm{final}} \in Q], \quad J_{-} = [s \in J \mid s_{\mathrm{final}} \notin Q].$$

Deciding the terminal membership yields the exhaustive disjoint partition of J. The all-rows computation proves

$$\mathrm{Secure} = \mathrm{true} \iff \forall s \in J,\ s_{\mathrm{final}} \in Q.$$

A member of J_{-} refutes this statement. An Execution additionally requires

$$\mathrm{run}(P, s_{\mathrm{initial}}, s_{\mathrm{actions}}) = \mathrm{ok}(s_{\mathrm{trace}}), \qquad \mathrm{last?}(s_{\mathrm{trace}}) = \mathrm{some}(s_{\mathrm{final}});$$

a StoppedExecution adds membership in the stopping-state list. Thus applying Secure to the execution's source row establishes security of its actual recorded terminal state. Extension increases the resource allowance, preserves schedules and runs, and preserves secure-state membership in both directions; these conditions retain both outcome ledgers and the stopped trace.

Fold adversarial-computation law is complete: its native derivations, full 256-form grammar, original witnesses and controls, and every program output field and hash agree with pinned V3.

> Source, proof and provenance: V3 ADVERSARIAL; V3 exact_security_partition; SFT.Computation.Adversarial.outcomeExhaustion; SFT.Computation.Adversarial.exactSecurity; Timeline: Turn 305, line 3021.

248. Observation-based and resource-bounded security

The carrier retains supported secrets, the observation relation, registered adversaries, attempted guesses and their exact costs. The complete success ledger contains precisely the admitted attempts with a correct guess. Its resource-bounded part contains exactly those successful attempts whose cost is within the declared bound.

Whole-support failure and bounded failure have separate computed checks and equivalence theorems. Failure across the complete registered attempt support implies bounded failure, and a checked larger resource bound implies the corresponding smaller-bound result. The converse is not asserted: a native example exhibits bounded failure alongside a successful higher-cost attempt.

Full observation closure retains every supported secret in each admitted observed class. Its computed condition is equivalent to that relation property and prevents simultaneous recovery of distinct secrets from a common view. A checked example has closed observations and a successful guess, keeping observation-based non-recovery distinct from a claim that every guess fails.

Reduction certificates retain source admission, successful guesses and resource costs; bounded failure transfers through a checked certificate. Identity and serial composition preserve the reduction interfaces. Source extension retains previous attempts and success witnesses. All 1,024 concrete outcomes and both distinguishing native examples pass.

Let J contain precisely the registered attempts with a supported secret, an admitted secret/view pair and a registered adversary. Define

$$S = [a \in J \mid a_{\text{guess}} = a_{\text{secret}}], \qquad S_B = [a \in S \mid a_{\text{cost}} \le B].$$

The two all-failure checks satisfy

$$\text{AllFail} \iff \forall a \in J,\ a_{\text{guess}} \neq a_{\text{secret}}, \quad \text{Fail}_B \iff \forall a \in J,\ a_{\text{cost}} \le B \Rightarrow a_{\text{guess}} \neq a_{\text{secret}}.$$

Thus AllFail implies Fail_B; if $b \le B$, the latter implies Fail_b by transitivity of the cost inequality. A reduction $f : J_1 \to J_2$ preserves success and satisfies $\text{cost}(f(a)) \le \text{cost}(a)$. A successful source attempt costing at most B would therefore be a successful target attempt within B, proving

$$\text{Fail}_B(J_2) \Rightarrow \text{Fail}_B(J_1).$$

Identity and composition satisfy the same conditions, with the composed cost bound obtained by transitivity.

Full observation closure is

$$s \in X \wedge (s, v) \in O \Rightarrow \forall t \in X,\ (t, v) \in O.$$

For distinct s, t in a common view class, simultaneous recovery requires $s = d(v) = t$, a contradiction. This conclusion concerns recovering both identities from one view; it does not imply that every individual guess in J fails.

Fold information-theoretic and computational-security law is complete: its native derivations, full 256-form grammar, original witnesses and controls, and every program output field and hash agree with pinned V3.

> Source, proof and provenance: V3 SECURITY-DEFINITION; V3 exact_security_partition; SFT.Computation.SecurityDefinition.exactBoundedFail; SFT.Computation.SecurityDefinition.reductionBoundedFailure; Timeline: Turn 306, line 3030.

249. Classical security under an enlarged interface boundary

The carrier retains the classical scheme, declared oracle identities and capabilities, exact query rows, query counts and resource costs. The admitted ledger is proved equivalent to source membership for each query and compliance with both limits.

A declared quantum-capable interface remains distinct from an operational classical ledger. Every query in a classical ledger is proved to carry classical capability; a quantum-interface query cannot inhabit that operational ledger. Actual classical execution traces reconstruct from the scheme transitions. No quantum operation is derived by this boundary construction.

The exact missing-dependency ledger identifies unfulfilled reduction dependencies. A reduction use retains evidence for every registered dependency and an actual source- and success-preserving resource certificate; bounded failure transfers through that certificate. Migration preserves earlier interfaces, queries, resource allowances and actual classical traces. All 770 concrete checks pass, including admitted declarations that do not qualify as classical executions.

For query list $q_1, \ldots, q_n$, interface I, registered query rows Q, count limit N and resource limit B, admission is exactly

$$\left(\forall i,\ q_i \in Q \wedge (q_i)_{\mathrm{oracle}} \in I\right) \wedge n \le N \wedge \sum_{i=1}^{n} (q_i)_{\mathrm{cost}} \le B.$$

The cost recursion is $c([]) = 0$, $c(q :: r) = q_{\mathrm{cost}} + c(r)$. A ClassicalLedger adds

$$\forall q \in L,\ q_{\mathrm{oracle.capability}} = \mathrm{classical}.$$

A member with capability quantumInterface contradicts the distinct capability constructors. A ClassicalExecution separately supplies $\mathrm{run}(P, i, a) = \mathrm{ok}(\tau)$, from which the transition execution is reconstructed.

For dependencies D and available records A, the exact missing ledger is $[d \in D \mid d \notin A]$. A ReductionUse provides $\forall d \in D, d \in A$, eliminating every member of that ledger, together with the source-, success- and cost-preserving reduction of the preceding section. Its bounded-failure transfer follows from that certificate. Migration preserves old query membership, increases both limits and preserves actual runs, so it transports the same classical execution without changing its queries or capability evidence.

Fold post-quantum security-boundary law is complete: its native derivations, full 256-form grammar, original witnesses and controls, and every program output field and hash agree with pinned V3.

> Source, proof and provenance: V3 POST-QUANTUM-BOUNDARY; V3 exact_security_partition; SFT.Computation.PostQuantumBoundary.classicalOnly; SFT.Computation.PostQuantumBoundary.certifiedBoundedTransfer; Timeline: Turn 307, line 3039.

250. Inference from generated hypotheses and exact evidence

The native inference operation preserves the pinned source program: hypothesis names must be unique, examples and hypothesis support must be nonempty, and retained names follow the original order. A hypothesis survives precisely when its relation contains every registered evidence row. The output theorem connects each returned name to that exact source membership.

Retained and eliminated ledgers are proved exhaustive and disjoint over the registered hypotheses. Each retained hypothesis yields a proof trace carrying its source record and compatibility with every evidence row. The uncertainty ledger enumerates all ordered retained pairs with distinct identities.

Adding evidence cannot introduce a previously eliminated candidate. Extending hypothesis support preserves earlier compatible records. These derivations introduce neither a prior distribution nor a pretrained weight. All 194 concrete checks pass, including executable expected outputs regenerated from the pinned source function, empty-support boundaries and duplicate-name rejection priority.

Write a hypothesis as $h = (n_h, R_h)$, with evidence list E and ordered hypothesis list H. The exact survival rule is

$$\mathrm{Fit}_E(h) \iff \forall e \in E,\ e \in R_h, \qquad H_E = [h \in H \mid \mathrm{Fit}_E(h)].$$

The executable checks unique names first, then rejects an empty E or H, otherwise returning $[n_h \mid h \in H_E]$. Hence a returned name n has an actual source witness $h \in H$ with $n_h = n$ and $E \subseteq R_h$. Filtering H by Fit and its Boolean negation partitions its members, and the retained proof trace is precisely $(h, h \in H, E \subseteq R_h)$.

For evidence extension $E \subseteq E'$,

$$E' \subseteq R_h \Rightarrow E \subseteq R_h, \qquad H_{E'} \subseteq H_E.$$

For hypothesis extension $H \subseteq H'$, an old retained record remains retained against the same evidence. The uncertainty ledger is the full ordered list

$$U_E = [(a, b) \in H_E \times H_E \mid n_a \neq n_b].$$

Its membership theorem follows by the two list traversals and the final identity comparison, retaining each source record and its original order.

Fold inference law is complete: its native derivations, full 256-form grammar, original witnesses and controls, and every program output field and hash agree with pinned V3.

> Source, proof and provenance: V3 INFERENCE; V3 consistent_hypotheses; SFT.Computation.Inference.exhaustive; SFT.Computation.Inference.executableOutput; Timeline: Turn 308, line 3049.

251. Sealed classification, prediction and target custody

The carrier retains supported inputs and labels, the hypothesis relation and training evidence. A sealed hypothesis carries proof of retention against that evidence. Prediction returns a label exactly when the supported output relation contains a single label; source membership and all training rows reconstruct from the sealed record.

Targets are kept in a separate evaluation structure. Its held-apart condition proves that target rows are absent from training evidence. Matching and falsification ledgers retain exact evaluation results. Falsification requires an actual predicted label unequal to the target label; absent or ambiguous predictions are recorded separately as nonmatches.

Extension preserves the earlier sealed hypothesis and its predictions under the stated source-interface conditions. All 1,536 concrete input, label, rule, prediction and falsification checks pass. These formal results preserve the registered boundary: natural-data performance requires the separate empirical constitution.

For sealed hypothesis h, supported inputs X and labels Y, prediction uses the ordered list

$$L_h(x) = [y \mid (x, y) \in R_h,\ x \in X,\ y \in Y], \qquad \mathrm{predict}_h(x) = \mathrm{ok}(y) \iff L_h(x) = [y].$$

This is a singleton-list condition: repeated rows are not silently deduplicated. The sealed record carries $h \in H_E$, so every training row belongs to R_h. Evaluation then separates

$$M = [t \in T \mid \mathrm{predict}_h(t_x) = \mathrm{ok}(t_y)],$$

$$F = [t \in T \mid \exists y,\ \mathrm{predict}_h(t_x) = \mathrm{ok}(y) \wedge y \neq t_y].$$

An error outcome is a nonmatch but cannot supply the successful prediction required for F. Match and falsification membership are therefore disjoint. Custody requires $\forall t \in T, \forall e \in E,\ t_x \neq e_x$, implying $(t_x, t_y) \notin E$. The registered extension preserves input and label lists, retains h, and admits no new evidence outside the old evidence; it therefore preserves the same output list and exact prediction.

Fold classification and prediction law is complete: its native derivations, full 256-form grammar, original witnesses and controls, and every program output field and hash agree with pinned V3.

> Source, proof and provenance: V3 CLASSIFICATION-PREDICTION; V3 consistent_hypotheses; SFT.Computation.ClassificationPrediction.predictionSource; SFT.Computation.ClassificationPrediction.evaluationExhaustion; Timeline: Turn 309, line 3058.

252. Representation grammar and preserved distinctions

The representation carrier retains source forms, generated codeword grammar, the exact encoding relation and a task relation. Each transformation trace carries its source identity, admitted word and actual encoding row. Task traces additionally retain the exact task row. The complexity ledger records the generated word length for every encoding row.

The construction reuses the previously verified encoding laws. A shared codeword closes the corresponding distinction and cannot recover two distinct source identities simultaneously. Distinct words retain the distinction. The complete closed ledger has exact common-class membership, and the retained ledger identifies distinct-image source pairs.

Composition retains source identities and proves each resulting row comes through an actual intermediate encoding and interface row. Source extension preserves prior transformation and task traces. All 518 concrete checks pass, including missing-interface rejection, complete closed/retained ledgers, task outputs and composition, together with native transformation and closure fixtures. No opaque transformation is admitted without its exact source trace.

A transformation is a source row with explicit membership evidence

$$x \in X, \qquad w \in G, \qquad (x, w) \in E,$$

and a task trace adds $(w, y) \in T$. The complete task and complexity ledgers satisfy

$$y \in \mathrm{tasks}(w) \iff (w, y) \in T, \qquad (x, n) \in C_E \iff \exists w,\ (x, w) \in E \wedge |w| = n.$$

Under the registered encoding-validation check, equal transformation words give a closed distinction and unequal words give a retained distinction. If $x \neq x'$ and both transformations have word w, recovering both would require $x = d(w) = x'$, which is impossible. Equal words also imply equal task lists by substitution.

For an admitted interface composition $E; I$, exact row correspondence is

$$(x, z) \in E; I \iff \exists w,\ (x, w) \in E \wedge (w, z) \in I.$$

The source projection of the resulting list equals that of E, preserving the original source order. The closed ledger uses complete source pairs sharing an encoding class; the retained ledger uses differing lookup images. Componentwise source, grammar, encoding and task inclusion transports the original transformation and task trace with the identical word and result.

Fold representation-learning boundary law is complete: its native derivations, full 256-form grammar, original witnesses and controls, and every program output field and hash agree with pinned V3.

> Source, proof and provenance: V3 REPRESENTATION; V3 consistent_hypotheses; SFT.Computation.RepresentationLearning.noRecoveryFromClosed; SFT.Computation.RepresentationLearning.distinctionExhaustion; Timeline: Turn 310, line 3067.

253. Generalization and exhaustive target-family certificates

The generalization carrier retains the hypothesis class and development evidence, sealed predictor, withheld inputs and complete target rows. Every withheld input must have an explicit target row, and the held-apart condition excludes those target inputs from development evidence. The computed pass check is equivalent to correct prediction for every held target row; target completeness extends that checked statement to every withheld input.

The failure ledger retains exact adverse targets and each such member refutes the whole-target pass check. Extension preserves earlier targets and failures when predictions remain sealed. A check over enlarged target support entails the old checks; success on the old targets does not itself entail success on new targets.

A structural certificate carries a generated target family, its base proof and a successor proof. Induction derives correctness for every generated member. Transferring that result to the target support additionally requires explicit exhaustive coverage. Generalization retains the source boundary: it is empirical unless the structural successor proof exhausts the target family. All 64 concrete held-target checks and a native structural-family fixture pass.

Let W be withheld inputs, $T_W = [t \in T \mid t_x \in W]$, and p the sealed predictor. The exact finite check is

$$\mathrm{Pass} \iff \forall t \in T_W,\ p(t_x) = \mathrm{ok}(t_y).$$

The supplied completeness condition $\forall x \in W, \exists t \in T,\ t_x = x$ places such a witness in T_W; applying Pass proves a checked target value for every withheld input. The failure ledger filters T_W by unsuccessful agreement, so any member contradicts Pass. With unchanged predictions, extending the target support preserves old failures, and passing the enlarged support implies passing its old members.

For the structural case, a certificate supplies a generated target $g(n)$, the base $P(0)$, and $\forall n, P(n) \Rightarrow P(n+1)$, where

$$P(n) \equiv p(g(n)_x) = \mathrm{ok}(g(n)_y).$$

Natural-number induction gives $\forall n, P(n)$. The additional coverage premise $\forall t \in T_W, \exists n, g(n) = t$ then substitutes each target into this family theorem and proves Pass. This last coverage step is explicit; correctness on the generated family alone does not identify it with the target support.

Fold generalization law is complete: its native derivations, full 256-form grammar, original witnesses and controls, and every program output field and hash agree with pinned V3.

> Source, proof and provenance: V3 GENERALIZATION; V3 consistent_hypotheses; SFT.Computation.Generalization.exactPass; SFT.Computation.Generalization.structuralCoversTargets; Timeline: Turn 311, line 3076.

254. Sample complexity and constructive teaching witnesses

The sample carrier retains complete hypothesis support, possible examples and selected evidence. The teaching check is equivalent to supported evidence, target membership, target compatibility and closure of all compatible hypotheses to the target identity. Its complete ambiguity ledger contains no distinct-name pair when that certificate holds.

For every distinct rejected alternative in the hypothesis support, the constructive witness theorem returns an actual selected example present in the target relation and absent from that alternative. The proof traverses the selected evidence; it does not introduce a probabilistic or asymptotic sample bound. The resource ledger retains each selected example and its unit contribution, with sample size equal to the complete selected list length.

A larger supported sample preserves the teaching certificate when the target fits the added evidence and hypothesis support remains unchanged. All 1,024 concrete support, sample, relation and ambiguity checks and the native distinguishing-witness fixture pass. The source boundary is retained: each family requires a generated certificate; no asymptotic formula is imported.

For selected sample S, possible examples E, hypotheses H and target t, the teaching condition is

$$S \subseteq E, \quad t \in H, \quad S \subseteq R_t, \quad \forall h \in H,\ S \subseteq R_h \Rightarrow n_h = n_t.$$

It follows that every retained pair has equal names, so the distinct-name ambiguity ledger has no member. For an alternative $a \in H$ with $n_a \neq n_t$, compatibility would contradict the final teaching condition. Traversing the finite sample therefore yields

$$\exists e \in S,\ e \in R_t \wedge e \notin R_a.$$

Constructively, failure of the all-members test is located at a head element that fails membership or recursively in the tail; target compatibility supplies membership in R_t. The resource ledger is $[(e, 1) \mid e \in S]$, and the sample size is $|S|$. If $S \subseteq S'$, the hypothesis list is unchanged, $S' \subseteq E'$, and the target fits S', every hypothesis fitting S' also fits S. The original name-closure implication thus proves teaching for the enlarged sample.

Fold sample-complexity law is complete: its native derivations, full 256-form grammar, original witnesses and controls, and every program output field and hash agree with pinned V3.

> Source, proof and provenance: V3 SAMPLE-COMPLEXITY; V3 consistent_hypotheses; SFT.Computation.SampleComplexity.exactTeaching; SFT.Computation.SampleComplexity.teachingWitness; Timeline: Turn 312, line 3085.

255. Exact optimization over evidence-compatible hypotheses

The learning optimization carrier retains the generated hypothesis support, evidence and exact objective relation. Feasible identities correspond to actual supported hypotheses containing every evidence row. Complete traversal retains every identity without a better objective-relation row. Under the admitted relation conditions, each eliminated candidate has a distinct feasible witness and an exact elimination row.

Tied undominated identities remain together; selection does not introduce a tie breaker. The update trace reconstructs the complete scan and retains its initial state and each traversal step. Serial traversal composes exactly. Extension over enlarged feasible support and appended objective rows requires survival against both the original and added relation ledgers.

All 512 concrete evidence, hypothesis-relation and objective outputs pass, together with native trace and elimination-witness proofs. Gradient, differentiability and fitted parameter spaces are not assumed.

Let

$$F = [n_h \mid h \in H,\ E \subseteq R_h], \qquad O = \mathrm{scan}_{\prec}([], F).$$

The scan theorem gives the exact survivor certificate

$$n \in O \iff n \in F \wedge \forall b,\ (b, n) \notin \prec.$$

Under the source preference-validity check, each omitted $n \in F$ has an actual $b \in F$, $b \neq n$, with (b, n) in its elimination ledger. Two feasible names without such incoming rows both survive; the algorithm adds no tie-breaking order.

The recorded history contains its initial state and one state per candidate:

$$|\,\mathrm{history}_{\prec}([], F)| = |F| + 1, \qquad \mathrm{scan}_{\prec}([], F_1 +\!\!+ F_2) = \mathrm{scan}_{\prec}(\mathrm{scan}_{\prec}([], F_1), F_2).$$

For $F' = F +\!\!+ N$ and $\prec' = \prec +\!\!+ A$, extension has the exact form

$$n \in O' \iff (n \in F \vee n \in N) \wedge \forall b,\ (b, n) \notin \prec \wedge \forall b,\ (b, n) \notin A.$$

Thus appended objective rows are checked explicitly against every retained identity, and the complete trace reconstructs the same traversal.

Fold optimization-in-learning law is complete: its native derivations, full 256-form grammar, original witnesses and controls, and every program output field and hash agree with pinned V3.

> Source, proof and provenance: V3 LEARNING-OPTIMIZATION; V3 consistent_hypotheses; SFT.Computation.LearningOptimization.exactOptima; SFT.Computation.LearningOptimization.comparedEveryCandidate; Timeline: Turn 313, line 3094.

256. Computational induction and exact successor tests

The induction carrier retains observation rows, candidate rule grammar, exact consistency relation and minimality. Retained alternatives are exactly the supported rules containing every observation. Minimal rule identities retain the evidence-compatible selection and its complete objective relation.

Adding an observation retains exactly those previous alternatives whose relations also contain that new row. An absent row excludes the rule from the enlarged evidence support. Enlarging the rule grammar preserves earlier consistent alternatives when observations remain fixed.

A structural continuation certificate carries generated evidence, a checked base and a successor proof. Induction establishes every generated row from that certificate. Finite agreement alone does not prove unrestricted continuation: a concrete checked example retains a rule on existing observations and rejects it on the next observation. All 1,026 concrete outputs and a native certified-continuation fixture pass.

For observations E, rule list H and rule relation R_h, put

$$A(E) = [h \in H \mid \forall e \in E,\ e \in R_h].$$

The appended-evidence derivation splits list membership in $E{+}{+}[e]$:

$$h \in A(E{+}{+}[e]) \iff h \in A(E) \wedge e \in R_h.$$

Thus a missing successor row removes precisely that rule; no new retained rule appears merely by adding evidence. The minimal-rule list further filters feasible names by absence of an incoming row in the registered objective relation:

$$n \in M(E) \iff n \in [n_h \mid h \in A(E)] \wedge \forall b,\ (b, n) \notin \prec.$$

For a SuccessorCertificate with generated evidence g, its fields are
$g(0) \in R_h$ and $\forall n,\ g(n) \in R_h \Rightarrow g(n+1) \in R_h$.
Induction applies the base at zero and the supplied successor implication at each next index, proving $\forall n, g(n) \in R_h$. This continuation theorem uses the certificate's actual successor proof; the finite filter $A(E)$ alone supplies only the evidence memberships it tested. Enlarging H with E fixed preserves each old retained record by the same conjunction.

Fold computational-induction law is complete: its native derivations, full 256-form grammar, original witnesses and controls, and every program output field and hash agree with pinned V3.

> Source, proof and provenance: V3 INDUCTION; V3 consistent_hypotheses;
> SFT.Computation.ComputationalInduction.nextAlternatives;
> SFT.Computation.ComputationalInduction.generatedContinuation; Timeline: Turn 314, line 3103.

257. Search and planning through exact execution traces

The planning carrier retains generated states, actions and transition rows, the initial state, exact goal class and complete plan support. A selected plan is precisely a supported action sequence whose actual execution succeeds and terminates in the goal class. Its proof constructs the full transition execution from the admitted initial state.

The success and failure ledgers are disjoint and exhaust every supported plan. The complete execution ledger retains actual traces or errors, and the resource ledger records each plan with its exact action count. Missing or ambiguous transitions and initial states outside the carrier remain explicit failures.

Serial plan composition preserves the shared state interface and reaches the composed goal. Admitted extension of state, transition, goal and plan support preserves earlier successful executions. All 1,536 concrete outputs and native composition and execution proofs pass. The scope remains the generated environment model.

For initial state s, goal list G, supported plans Π and transition system P, selection has the exact form

$$\pi \in \mathrm{Selected} \iff \pi \in \Pi \wedge \exists \tau,\ \mathrm{run}(P, s, \pi) = \mathrm{ok}(\tau) \wedge \mathrm{terminal}(s, \tau) \in G.$$

Terminal evaluation obeys $t(s, []) = s$ and $t(s, x :: r) = t(x, r)$.
Run admission yields $s \in P_{\mathrm{states}}$ and the typed execution for every selected plan. Filtering Π by this Boolean acceptance and its negation gives exhaustive disjoint success and failure ledgers. The other two ledgers are exactly

$$[(\pi, \mathrm{run}(P, s, \pi)) \mid \pi \in \Pi], \qquad [(\pi, |\pi|) \mid \pi \in \Pi].$$

For paths $s \xrightarrow{u} m$ and $m \xrightarrow{v} z$, composition supplies the complete execution of $u{+}{+}v$. If this concatenated plan belongs to Π and $z \in G$, the selection equivalence admits it. Extension uses an admitted larger transition system, the same initial state, and retained transition, goal and plan rows to transport that successful execution.

Fold search and planning law is complete: its native derivations, full 256-form grammar, original witnesses and controls, and every program output field and hash agree with pinned V3.

> Source, proof and provenance: V3 SEARCH-PLANNING; V3 consistent_hypotheses; SFT.Computation.SearchPlanning.selectedExecution; SFT.Computation.SearchPlanning.ledgerComplete; Timeline: Turn 315, line 3112.

258. Reinforcement computation through generated interaction traces

Interaction admission retains the exact current state, supported source and target states, generated action and outcome label, environment row and policy relation. Replay is equivalent to a typed execution of those rows. The retained and rejected trajectory ledgers are disjoint and exhaust the supplied trajectory support.

Every registered policy has a complete evaluation record containing its retained trajectories. Outcome labels remain in their original order. Policy comparison uses an explicit exact relation; no external reward scale, fitted value function or stochastic cause is introduced.

Serial trace composition preserves the intermediate state and concatenates outcome records. Enlarged source supports preserve earlier admitted rows and trajectories. All 1,024 concrete outputs, covering 21 trajectories per model, and native component/composition proofs pass.

Write an interaction as $r = (s, a, t, o)$. At current state x, admission requires

$$s = x, \quad s, t \in S, \quad a \in A, \quad o \in O, \quad r \in E, \quad (s, a) \in R_p,$$

where R_p is the selected policy relation. Replay is the recursion

$$\mathrm{replay}_p(x, []) \iff x \in S, \qquad \mathrm{replay}_p(x, r :: u) \iff \mathrm{admit}_p(x, r) \wedge \mathrm{replay}_p(t, u).$$

Induction over the row list identifies this Boolean recursion with the base and next constructors of Trace. Retained trajectories are therefore exactly the supplied trajectories with a Trace from the initial state; the complementary filter supplies rejected trajectories. Policy evaluation is the ordered map $p \mapsto (p, \mathrm{retained}(p))$ over all registered policies.

For admitted traces u and v sharing the final/initial state, induction on u proves admission of $u{+}{+}v$, and list mapping proves

$$\mathrm{labels}(u{+}{+}v) = \mathrm{labels}(u){+}{+}\,\mathrm{labels}(v).$$

The policy-name comparison retains n exactly when it is registered and no row (b, n) occurs in the given comparison relation. Componentwise support extension preserves each conjunct of row admission and hence every previous Trace.

Fold reinforcement-computation law is complete: its native derivations, full 256-form grammar, original witnesses and controls, and every program output field and hash agree with pinned V3.

> Source, proof and provenance: V3 REINFORCEMENT; V3 consistent_hypotheses; SFT.Computation.Reinforcement.replayMeaning; SFT.Computation.Reinforcement.exhaustion; SFT.Computation.Reinforcement.traceComposition; Timeline: Turn 316, line 3121.

259. Multi-agent computation and returning joint traces

Joint-row admission retains generated source and target states, exact outcome classes, agent identities and supplied action rows. Local observations are selected by their exact agent and state coordinates. Communication reconstructs from the complete admitted transition execution.

Joint traces compose through a shared state interface. Executable replay is equivalent to the typed trace and its final state. The cycle check is equivalent to a trace returning to its initial state; the cycle ledger retains exactly those returning traces in the supplied path support. Source extension preserves earlier admitted rows and traces.

All 2,208 concrete support, observation and cycle outputs and native cycle and communication proofs pass. Game-theoretic correspondence remains downstream and requires exact preference relations.

For a joint row $r = (s, t, A_r, o)$, membership in the admitted relation is

$$r \in J \iff r \in J_0 \wedge s, t \in S \wedge o \in O \wedge \forall (a, u) \in A_r,\ a \in \mathcal{A} \wedge (a, u) \in U.$$

The local observation list for agent a at state s is the original observation list filtered by those two coordinates. JointTrace has the constructors

$$s \in S \Rightarrow \mathrm{Trace}(s, [], s), \qquad r \in J \wedge \mathrm{Trace}(r_t, u, z) \Rightarrow \mathrm{Trace}(r_s, r :: u, z).$$

Induction on the first trace proves serial composition. Replay checks the current source equality, row membership and the recursive tail, so

$$\mathrm{replay}(s, u) = \mathrm{true} \iff \mathrm{Trace}(s, u, \mathrm{final}(s, u)).$$

Adding $\mathrm{final}(s, u) = s$ proves the cycle-check equivalence with $\mathrm{Trace}(s, u, s)$; filtering the supplied path list retains exactly these cycles. The communication run separately reconstructs its own admitted transition execution. Support inclusion transports joint traces by preserving each constructor's memberships, without adding an unrecorded action for any agent.

Fold multi-agent computation law is complete: its native derivations, full 256-form grammar, original witnesses and controls, and every program output field and hash agree with pinned V3.

> Source, proof and provenance: V3 MULTIAGENT; V3 consistent_hypotheses; SFT.Computation.MultiAgent.jointComposition; SFT.Computation.MultiAgent.replayMeaning; Timeline: Turn 317, line 3130.

260. Trace-bound adaptation with preserved constitutional identity

Adaptation retains the initial process, observation history and exact admitted update grammar. Requests are checked against the constitutional engine and claim-law identities. A changed identity stops the update. Successful receipts preserve the observation history, generated update trace and original identities; their execution theorem reconstructs each update from the source transition relation.

The performance relation remains an explicit exact relation. Membership establishes only the comparison supplied by that relation; no external score or automatic improvement is introduced. Update failures retain their original error boundary.

Serial update composition preserves the complete intermediate trace. Admitted extension preserves earlier successful receipts when the initial process and constitutional and law identities remain fixed. All 640 concrete outputs and native composition and execution proofs pass. Adaptation cannot silently alter the constitutional engine or claim law.

For model identities c, ℓ, initial process s, update system P and request (u, c', ℓ'), the identity guard is

$$I \iff c' = c \wedge \ell' = \ell.$$

The update returns an identity error when I fails. Otherwise it forwards the exact run error or constructs

$$\mathrm{run}(P, s, u) = \mathrm{ok}(\tau) \Longrightarrow \mathrm{adapt}(u, c, \ell) = \mathrm{ok}(u, \tau, c, \ell).$$

Conversely every successful receipt proves I, the successful run, unchanged observations and both original identities. Run admission then reconstructs the actual update execution.

For paths from s through m to z, with action lists u, v and stored path prefixes β, γ, serial composition gives the receipt

$$(u +\!\!+ v,\ (\beta +\!\!+ \gamma) +\!\!+ [z],\ c, \ell).$$

Its performance predicate remains exactly
$\mathrm{improves}(b, w) = \mathrm{true} \iff (b, w) \in \prec$.
An admitted extension retains the initial state and identities and preserves source transition rows; it consequently returns the identical earlier trace and observations.

Fold adaptation law is complete: its native derivations, full 256-form grammar, original witnesses and controls, and every program output field and hash agree with pinned V3.

> Source, proof and provenance: V3 ADAPTATION; V3 consistent_hypotheses; SFT.Computation.Adaptation.admittedReceipt; SFT.Computation.Adaptation.updateComposition; Timeline: Turn 318, line 3139.

261. Learning limits from indistinguishable evidence

Observed evidence is the exact restriction of each hypothesis relation to the supplied observed inputs. The paired construction retains identical observed rows, an unobserved target and different target requirements, with both requirements explicitly present in their respective source hypotheses.

Every fixed learner produces the same target decision from those identical evidence rows. Consequently it cannot satisfy both differing requirements. The constructive counterexample retains the failed requirement, its source row and the unobserved-target boundary. Deterministic preprocessing also preserves indistinguishability.

Source extension preserves this result when it retains the observed evidence and original target rows. All 64 concrete observed-row outputs and native same-decision and failed-requirement proofs pass. These limits remain relative to the exact evidence, hypothesis and target grammars.

Let

$$O_X(h) = [(x, y) \in R_h \mid x \in X].$$

The paired construction supplies $O_X(h_0) = O_X(h_1)$, a target $z \notin X$, source rows $(z, y_i) \in R_{h_i}$, and $y_0 \neq y_1$. For any fixed learner L, equality substitution gives

$$L(O_X(h_0), z) = L(O_X(h_1), z) = v.$$

If $v = y_0$, then $v \neq y_1$; otherwise $v \neq y_0$ already witnesses failure. This explicit equality decision constructs

$$L(O_X(h_0), z) \neq y_0 \ \vee \ L(O_X(h_1), z) \neq y_1,$$

with the corresponding source target row and unobserved-target evidence retained. For any deterministic preprocessing f, congruence yields $f(O_X(h_0)) = f(O_X(h_1))$, so the same argument applies after preprocessing. Extension requires equality of the old and new observed lists on each side and retention of the target rows; these premises reconstruct the paired witness and its counterexample.

Fold computational limits-of-learning law is complete: its native derivations, full 256-form grammar, original witnesses and controls, and every program output field and hash agree with pinned V3.

> Source, proof and provenance: V3 LEARNING-LIMITS; V3 consistent_hypotheses; SFT.Computation.LearningLimits.notBothCorrect; SFT.Computation.LearningLimits.processingPreservesIndistinguishability; Timeline: Turn 319, line 3148.

262. Interpretability through exact verified decision certificates

The certificate binds the model description, initial input, action sequence, complete execution trace, premise ledger and decision. Verification succeeds exactly when these fields match the source model and actual execution, and the terminal decision belongs to the supplied relation. Verified execution reconstructs every transition.

Changing the actual trace, premise ledger or model description causes rejection; an unsupported decision is also rejected. The complete counterfactual ledger retains each supplied alternate input and action sequence with its actual execution result, including failures.

Serial composition produces a certificate for the complete composed path. Admitted source extension preserves verified certificates while retaining model identity, premises and prior transition and decision support. All 1,536 concrete outputs and native certificate and execution proofs pass. A narrative explanation or score cannot replace the exact trace.

For certificate $c = (d, s, u, \tau, P_c, y)$, model description d_m, premises P_m and decision relation D, verification is equivalent to

$$d = d_m \wedge \mathrm{run}(M, s, u) = \mathrm{ok}(\tau) \wedge P_c = P_m \wedge (\mathrm{terminal}(s, \tau), y) \in D.$$

The successful-run conjunct reconstructs every transition. If the actual run returns $\tau' \neq \tau$, the certificate cannot satisfy that conjunct; unequal description or premise lists likewise refute their respective equality checks. Missing terminal/decision membership refutes the final conjunct.

For counterfactual support C, the ledger is

$$[((s, u), \mathrm{run}(M, s, u)) \mid (s, u) \in C],$$

including error outcomes. Serial path composition from s through m to z constructs the full trace $(\beta +\!\!+ \gamma) +\!\!+ [z]$; with $(z, y) \in D$, the four verification conjuncts establish its certificate. Admitted extension

preserves model identity and premises and transports the actual run and decision row, thereby preserving the original certificate.

Fold interpretability and verification law is complete: its native derivations, full 256-form grammar, original witnesses and controls, and every program output field and hash agree with pinned V3.

> Source, proof and provenance: V3 INTERPRETABILITY-VERIFICATION; V3 consistent_hypotheses; SFT.Computation.Interpretability.verificationMeaning; SFT.Computation.Interpretability.verifiedExecution; Timeline: Turn 320, line 3157.

263. Complete classical learning reports

The classical learning report retains the actual inference result, sealed hypothesis, complete held-target support, prediction records, evaluation decision and resource ledger. Exact inference returns precisely the compatible registered hypothesis identities. The sealed hypothesis retains every training example.

Evaluation passes exactly when every held target prediction matches its registered label. The complete prediction ledger covers every withheld input, and target custody excludes those inputs from training evidence. Resource records retain exact counts of examples, hypotheses, withheld inputs and target rows.

Identical source training, hypothesis and target data yield identical reports, independently of proof witnesses. Source extension preserves earlier prediction and failure records and carries checks over enlarged support back to the earlier targets. All 64 report-field outputs and native inference and held-target proofs pass. Quantum learning operations remain entirely outside this branch.

Fold classical-learning model law is complete: its native derivations, full 256-form grammar, original witnesses and controls, and every program output field and hash agree with pinned V3.

With evidence E, hypotheses H, withheld inputs W, filtered targets T_W and sealed predictor p, the report is the explicit tuple

$$\big(\mathrm{infer}(E,H),\ T_W,\ [(x,p(x)) \mid x \in W],\ \mathrm{all}_{t\in T_W}[p(t_x)=\mathrm{ok}(t_y)],\ [(\mathrm{examples},|E|),(\mathrm{hypotheses},|H|),(\mathrm{withheld},|W|),(\mathrm{targets},|T_W|)]\big).$$

Under the original unique-name and nonempty-support guards,
$\mathrm{infer}(E,H)=\mathrm{ok}([n_h \mid h\in H,\ E\subseteq R_h])$.
The sealed hypothesis supplies both source membership and $E\subseteq R_h$. The report's pass field is therefore equivalent to exact agreement on every $t\in T_W$; complete target coverage supplies such a checked target for every $x\in W$. Custody gives $(x,y)\notin E$ for all withheld x.

Equal training data, sealed hypothesis, withheld list and target list give equal report fields and hence equal reports; proof witnesses do not enter those computed fields. With unchanged predictions, extension preserves old prediction records and failed targets, while a successful enlarged evaluation implies the old target checks.

> Source, proof and provenance: V3 CLASSICAL-LEARNING; V3 consistent_hypotheses; SFT.Computation.ClassicalLearning.completePredictionLedger; SFT.Computation.ClassicalLearning.reportReproducible; Timeline: Turn 321, line 3166.

264. Exact values for capability-closed prediction

The prediction language has 15 registered opcodes. Its value representation retains structural empty-One identities, positive counts, positive numerator/denominator pairs, held labels, words, pairs and tables. Structural equality preserves the source fields; in particular, distinct ratio representations are not identified merely because their reduced fractions coincide.

Native equality is proved equivalent to structural equality. Label admission retains the source whitespace boundary, and table admission requires positive support and distinct key hashes. Canonical serialization preserves the original dataclass fields. All 505 concrete equality, serialization and admission checks pass; all 14 freshly computed value hashes match pinned V3. These results cover the value layer; interpreter execution and document admission remain separate components.

The exact value grammar is

$$V ::= \mathrm{EmptyOne}(i) \mid \mathrm{Count}(n) \mid \mathrm{Ratio}(n,d) \mid \mathrm{Label}(f,l) \mid \mathrm{Word}([V]) \mid \mathrm{Pair}(V,V) \mid \mathrm{Table}([(V,V)]),$$

where count fields are positive and both label fields pass the registered nonblank test. Structural comparison recursively checks matching constructors and matching fields:

$$\mathrm{equal}(a,b) = \mathrm{true} \iff a = b.$$

In particular,
$\mathrm{Ratio}(n,d) = \mathrm{Ratio}(n',d') \iff n = n' \wedge d = d'$;
cross-product equality alone does not establish structural identity. For words and tables, the recursive equality checks list length, order and every cell or pair.

Canonical serialization recursively emits the corresponding registered field objects. Let $H(v)$ be the canonical identity of that serialization. Table admission is exactly

$$E \neq [] \wedge \mathrm{distinct}([H(k) \mid (k,v) \in E]).$$

This retains the source hash-key guard itself, without replacing it by an assumed injectivity theorem for hashes. Count admission proves $n > 0$; label admission preserves the original family and label strings and their nonblank evidence.

> Source, proof and provenance: V3 PositiveRatio; V3 FoldWord; V3 FoldPair; V3 FoldTable; V3 validate_fold_value; SFT.Physics.CapabilityValues.countPositive; SFT.Physics.CapabilityValues.tableAdmission; Timeline: Turn 325, line 3203.

265. Admission and exact generated operations

Instruction admission preserves the registered destination and argument requirements. Program admission requires a nonblank identity, positive instruction support and exactly one final emission. All 858 source-derived admission cases pass.

The register-consuming operations preserve words and pairs, admit tables only through their original key-identity guard, and require exactly one structurally matching lookup image. Junction, product and quotient use positive integer arithmetic followed by exact fraction reduction. The reduction proof preserves the original numerator-to-denominator ratio. Structural assertions accept identical values and reject distinct values.

All 644 operation cases agree with the pinned interpreter, including arity and type rejection, lookup, unreduced ratio inputs and exact arithmetic outputs. The native declarations and proofs have no axiom dependencies. Literal parsing, the execution loop and document loading remain distinct components of the complete prediction runtime.

An instruction (o,d,a) is admitted when its argument list is nonempty and, if o produces a value, d is nonblank. Program admission adds

$$\mathrm{nonblank}(id), \quad I \neq [], \quad |[i \in I \mid i_o = \mathrm{emit}]| = 1, \quad \mathrm{lastOpcode}(I) = \mathrm{emit}.$$

The executable preserves the guard order: destination before argument support, then program identity/support before terminal-emission checks.

For positive ratio operands $a/b, c/d$, the generated operations are

$$\text{junction} = \text{reduce}(ad + cb, bd), \quad \text{product} = \text{reduce}(ac, bd), \quad \text{quotient} = \text{reduce}(ad, bc).$$

Positivity of every product and numerator supplies the reduction premises. If reduction of (n, d) returns (n', d'), its proof establishes $nd' = n'd$, preserving the represented ratio exactly. Lookup filters entries by structural key equality and accepts precisely a singleton image list. Table construction first requires a positive even list of values, pairs adjacent values, and applies the original table guard. Structural assertion succeeds for $a = b$ and rejects $a \neq b$, by the equality equivalence of the preceding section.

> Source, proof and provenance: V3 FoldInstruction; V3 FoldProgram; V3 _execute_instruction; SFT.Physics.CapabilityAdmission.instructionExact; SFT.Physics.CapabilityAdmission.programExact; SFT.Physics.CapabilityOperations.reductionValue; SFT.Physics.CapabilityLiterals.registeredInput; Timeline: Turn 325, line 3203.

266. Complete finite prediction execution and trace accounting

Execution dispatches each of the 15 registered opcodes in sequence. Registered-input reads remain separate from held-register reads. Count and ratio literals retain the source decimal and positive-value checks, including the explicit ASCII-leading-zero rule. The retained host comparison profile identifies the Unicode decimal table used by the source runtime.

Each value-producing instruction writes a fresh register; attempting to overwrite a held register halts. Emission and assertion preserve the register mapping. Every successful instruction appends one trace row containing its positive position, opcode, argument identities and output identity. Induction over the actual execution loop proves that a successful run adds exactly as many trace rows as there are instructions.

Eight complete source comparisons preserve every returned field and hash, including an execution exercising all 15 opcodes, the original registered-count witness, Unicode decimal input and rejection of overwrites, unheld reads, missing inputs and invalid manifests. All 912 decimal and literal checks pass. The integrated execution definitions and trace-accounting proofs have zero axiom dependencies.

The closed opcode list is

$$[\text{input}, \text{emptyOne}, \text{label}, \text{count}, \text{ratio}, \text{word}, \text{pair}, \text{table}, \text{lookup}, \text{junction}, \text{product}, \text{quotient}, \text{same}, \text{assertSame}, \text{emit}].$$

For state (R, e, T), dispatch computes output v and its argument values. Commit appends $r = (k + 1, o, [H(a_i)], H(v))$ to T. Emission returns $(R, v, T{+\!+}[r])$; assertion returns $(R, e, T{+\!+}[r])$; a value-producing instruction returns $(R{+\!+}[(d, v)], e, T{+\!+}[r])$ exactly when d is unoccupied. An occupied destination is rejected.

The loop's base returns its state unchanged. Its successor dispatches, commits one row and recurses over the remaining instructions. Thus induction gives

$$\text{run}(s, k, I) = \text{ok}(s') \Longrightarrow |T_{s'}| = |T_s| + |I|.$$

Decimal decoding obeys $D(a, []) = a$ and $D(a, c :: u) = D(10a + \delta(c), u)$, with δ obtained from the registered Unicode decimal ranges. Literal admission separately rejects empty input and ASCII leading zero, and requires a positive decoded result.

Registered input reads consult the manifest; other value references consult held registers. Successful execution retains the program, manifest, output and complete trace identities computed from their canonical forms.

> Source, proof and provenance: V3 execute; SFT.Physics.CapabilityExecution.commitTrace; SFT.Physics.CapabilityExecution.runTraceLength; Timeline: Turn 325, line 3203.

267. Data-only prediction documents and the complete capability grammar

The program loader admits only the registered schema, exact field sets, string argument traces and known opcodes. The value loader recursively preserves tagged structural empty-One, positive counts, ratios, held labels, words, pairs and tables. Recursive decoding follows the finite document structure without an imposed depth bound; table admission retains the original unique-key identity condition. All 317 program-document and 131 recursive value-document comparisons pass.

The original eight binary axes generate exactly 256 candidates. Native enumeration covers every coordinate combination and preserves source order, identifiers, exact forms and rejection reasons. Exactly one candidate retains all eight requirements. The operational witness executes the original registered-count input and emission program; its completed output and closed opcode surface are proved directly. These grammar results accompany the native interpreter and trace proofs.

Capability-closed exact Fold prediction form is complete: its native derivations, full 256-form grammar, original witnesses and controls, and every program output field and hash agree with pinned V3.

For document fields F and required names N, the implemented exact-key test is

$$(\forall (k,v) \in F,\ k \in N) \wedge (\forall k \in N,\ \exists v,\ (k,v) \in F).$$

The program loader requires $N = [\mathrm{schema}, \mathrm{program_id}, \mathrm{instructions}]$, schema identity `sft-v3-fold-program/1`, and positive instruction support. Each instruction uses its three registered fields, a known opcode, a string destination and an argument list consisting entirely of strings, before instruction and program admission are applied.

Recursive value preparation traverses children before assembling the parent result. A word gathers the decoded cells in order; a pair requires two object-valued cells and both decoded results; a table gathers decoded pair entries and then applies table admission. Counts require positive integer documents, with Boolean documents explicitly excluded. Each branch preserves its own exact field and host-type rejection.

For the eight binary grammar axes,

$$\mathcal{C} = \prod_{j=1}^{8} [\mathrm{rejected}, \mathrm{admitted}], \quad |\mathcal{C}| = 2^8 = 256, \quad \mathrm{survives}(c) \iff \bigwedge_{j=1}^{8} c_j = \mathrm{admitted}.$$

Complete nested enumeration establishes membership of every candidate; the conjunction identifies the unique all-admitted survivor. This classification is accompanied by the actual loader and execution derivations, not used in their place.

> Source, proof and provenance: V3 fold_program_from_mapping; V3 fold_value_from_mapping; SFT.Physics.CapabilityDocuments.opcodeRoundTrip; SFT.Physics.CapabilityGrammar.generatedCardinality; SFT.Physics.CapabilityGrammar.uniqueSurvivor; Timeline: Turn 325, line 3203.

268. Exact reference forms and bounded approximations

The comparison report retains the exact reference, approximation and complete orientation-labelled error ledger. Reconstruction establishes nonempty matched support and the original positive-fraction inputs for every cell. Changing the approximation cannot replace the retained exact reference. Equality retains the source One marker; held-below and held-above rows retain positive error magnitudes.

The error fractions reconstruct the corresponding exact numerator-to-denominator difference. Output intervals derive from the order of the reference and approximation, and both values lie within their bounds. Cell identities and original fractions supply the precision provenance. Appending an admitted comparison preserves every earlier row and adds the next exact ledger entry.

All 125 original calculation cases and 81 interval cases pass. The complete 256-candidate grammar and three original scientific-computation witnesses pass native verification with zero axiom dependencies. Approximate values remain downstream of the exact reference.

Fold exact and approximate calculation law is complete: its native derivations, full 256-form grammar, original witnesses and controls, and every program output field and hash agree with pinned V3.

Let the reference and approximation cells be positive fractions $x = a/b$ and $y = c/d$. Cross multiplication determines the orientation:

$$cb < ad:\ x - y = \frac{ad - cb}{bd}, \qquad ad < cb:\ y - x = \frac{cb - ad}{bd}.$$

If the stored reduced error is u/v, the below case satisfies

$$ubd + cbv = adv,$$

and the above case satisfies $ubd + adv = cbv$. These are the native reconstruction identities; they avoid floating subtraction. When $ad = cb$, the ledger retains $(\mathrm{One}, \mathrm{equal})$, an equality marker rather than a positive error magnitude.

The output stores the ordered endpoint pair (y, x) when $cb \le ad$, otherwise (x, y). These endpoints define the closed interval $[y, x]$ or $[x, y]$, respectively; the order test proves the inclusive inequalities placing both values in that interval. An admitted comparison report reconstructs its unchanged reference, approximation and Rows trace from index 1, with one row per reference cell. Appending a positive pair appends precisely its error row labelled $1 + |R|$, retaining the earlier list. Thus every row retains both original fractions and its exact position.

> Source, proof and provenance: V3 EXACT-APPROXIMATE; V3 exact_error_ledger; SFT.Computation.ExactApproximate.reportReconstruction; SFT.Computation.ExactApproximate.belowReconstructsReference; SFT.Computation.ExactApproximate.aboveReconstructsApproximation; Timeline: Turn 326, line 3212.

269. Exact perturbations and registered stability

A registered perturbation retains the reference and changed initial forms and their operation sequences. Both traces execute in the admitted transition system. Exact numerical support supplies the input and terminal output error ledgers, preserving cell identities, positive fractions and orientation labels. Reconstruction proves that every admitted observation contains precisely those two execution traces and two ledgers.

The amplification relation records the pair of input and output ledger forms. Stable observations are exactly its registered members; observations outside the relation are unstable, and unavailable traces or supports reject the observation. No floating epsilon or norm is introduced. Extending the registered perturbations, amplification relation or admitted transition system preserves the corresponding earlier results.

All 864 paired-execution and ledger comparisons and three native classification checks pass. The complete 256-candidate grammar, original three witnesses and four controls pass fresh native verification with zero axiom dependencies.

For registered perturbation $p = (s, s', u, u')$, admission requires the two executions

$$\mathrm{run}(P, s, u) = \mathrm{ok}(\tau), \qquad \mathrm{run}(P, s', u') = \mathrm{ok}(\tau').$$

The value table's first matching records supply exact supports for the initial pair and for $t = \mathrm{terminal}(s, \tau)$, $t' = \mathrm{terminal}(s', \tau')$. Their admitted comparisons produce L_{in} and L_{out}. Consequently a successful observation is exactly

$$(\tau, \tau', L_{\mathrm{in}}, L_{\mathrm{out}})$$

with the registered-perturbation, two-run and two-ledger premises recoverable from it.

Let $F(L)$ retain every row's cell identity, numerator, denominator and orientation. Classification uses

$$A(p) = (F(L_{\mathrm{in}}), F(L_{\mathrm{out}})), \qquad \mathrm{stable}(p) \iff A(p) \in \mathcal{R}.$$

Missing admission yields rejected; an admitted observation outside $\mathcal{R}$ yields unstable. Enlarging perturbation support preserves old observations; enlarging $\mathcal{R}$ preserves old stable memberships. An admitted transition-system extension preserves both runs and hence the identical observation when the value table is fixed.

Fold numerical-stability law is complete: its native derivations, full 256-form grammar, original witnesses and controls, and every program output field and hash agree with pinned V3.

> Source, proof and provenance: V3 STABILITY; V3 exact_error_ledger; SFT.Computation.Stability.stableMeaning; SFT.Computation.Stability.stableClassification; SFT.Computation.Stability.unstableClassification; Timeline: Turn 327, line 3221.

270. Local error transport and terminal provenance

Every dependency edge retains its identity and original reference and approximation supports. Transport computes the original orientation-labelled error ledger for each edge and halts at the first inadmissible comparison. Reconstruction recovers every local ledger and its complete edge sequence. Path composition retains endpoint interfaces and all component identities; adjoining an admitted edge preserves the earlier records.

The terminal ledger compares the retained initial and final supports directly. Paths with identical endpoints have identical terminal ledgers. A closed single-cell path retains the original equal marker, while all intermediate local ledgers remain available as cancellation provenance. All 686 source-derived local and terminal comparisons pass, including empty, mismatched and nonpositive support rejection. These native results have zero axiom dependencies. The generated-scale accumulated bound is established below.

For edge $e = (id, R, A)$, let $\mathcal{E}(R, A)$ be its exact comparison ledger. Transport is

$$\mathcal{T}([]) = \mathrm{ok}([]), \qquad \mathcal{T}(e :: r) = \begin{cases} \mathrm{ok}((e, L) :: K), & \mathcal{E}(R, A) = \mathrm{ok}(L),\ \mathcal{T}(r) = \mathrm{ok}(K), \\ \mathrm{error}, & \text{at the first failed computation.} \end{cases}$$

Induction on the edge list proves equivalence with the base and step constructors of Trace, and proves that projecting its retained edges returns the entire original edge list. Two successful traces concatenate their edge and local-ledger lists.

Path independently requires each edge's approximation to be the next interface reference. A Propagation supplies this Path, the local Trace, and the terminal comparison

$$L_{\mathrm{terminal}} = \mathcal{E}(R_{\mathrm{initial}}, A_{\mathrm{final}}).$$

Equal endpoint supports therefore force equal terminal ledgers by determinism, regardless of route. For a closed positive single cell, the result is exactly $[(\mathrm{cell}-1, \mathrm{One}, \mathrm{equal})]$. This endpoint equality leaves every intermediate local ledger in the retained transport trace, preserving cancellation provenance.

> Source, proof and provenance: V3 ERROR-PROPAGATION; V3 exact_error_ledger; SFT.Computation.ErrorPropagation.exactTransport; SFT.Computation.ErrorPropagation.terminalReconstruction; Timeline: Turn 329, line 3236.

271. Generated accumulated bounds for finite exact traces

A positive common denominator is generated from the actual finite fraction trace. Every coordinate satisfies the exact cross-product identity with its original fraction. Extending the trace rescales the earlier coordinates without changing their represented values. No numerical scale is fitted or supplied independently.

The orientation-labelled local error rows determine exact positive differences; equal rows retain the original One marker. Internal coordinate differences count unmatched support, with an empty difference contributing no support to the sum. The triangle bound and induction over the finite trace establish that its terminal difference is bounded by the accumulated local differences. Exact endpoint scaling connects that bound to the original initial and terminal fractions. Cancellation does not erase the local provenance.

All 340 generated accumulated-bound comparisons agree with independent exact rational calculations. Together with the 686 local and terminal source comparisons, the full 256-candidate grammar, three witnesses and four controls, the native derivations pass with zero axiom dependencies.

For positive fractions $p_i = a_i/b_i$, scale generation begins with $D([]) = 1$, $N([]) = []$. Adding a head $p = a/b$ to an already generated tail (D, N) gives

$$D' = bD, \qquad N' = aD :: [bn \mid n \in N].$$

The head satisfies $(aD)b = a(bD)$; each prior identity $n_i b_i = a_i D$, multiplied by b, establishes the rescaled identity. Thus every coordinate satisfies $n_i b_i = a_i D'$, and positivity of all denominators proves positivity of the generated scale.

Define

$$g(a, b) = \begin{cases} b - a, & a \le b, \\ a - b, & b < a, \end{cases} \qquad A(n, []) = 0, \quad A(n, m :: r) = g(n, m) + A(m, r).$$

The triangle inequality $g(a, c) \le g(a, b) + g(b, c)$, followed by induction over the trace, proves

$$g(n_0, n_k) \le \sum_{i=0}^{k-1} g(n_i, n_{i+1}).$$

For fractions $p = a/b, q = c/d$ with generated coordinates n, m, the coordinate identities give

$$g(n, m)\, bd = g(ad, cb)\, D.$$

Together with the exact oriented-error reconstruction, this connects each nonzero local difference to its ledger magnitude and the endpoint difference to the original endpoint fractions. Equal neighbours contribute $g(n, n) = 0$ to accumulation while their stored ledger retains One with the equal orientation. The bound numerator and positive denominator are generated from the finite trace itself; no independent numerical scale is supplied.

Fold error-propagation law is complete: its native derivations, full 256-form grammar, original witnesses and controls, and every program output field and hash agree with pinned V3.

> Source, proof and provenance: V3 ERROR-PROPAGATION; V3 exact_error_ledger; SFT.Computation.ErrorBound.completeScaling; SFT.Computation.ErrorBound.generatedAccumulatedBound; SFT.Computation.ErrorBound.generatedEndpointGap; Timeline: Turn 329, line 3236.

272. Finite discretization and refinement provenance

The discretization retains its declared finite source relation, generated cells and admitted coverage map. Every source form has a retained cell image. Every declared source edge therefore yields a local operator, and reconstruction preserves both endpoint images and the complete ordered source relation. Uncovered endpoints reject an operator.

The boundary ledger records all source distinctions closed by shared cells; the retained ledger records source pairs with distinct images. A registered projection from refined cells to coarse cells proves that refinement preserves earlier distinctions and that every finer closed class lies within a coarse class. Serial assembly and adjoining a source edge preserve all earlier operator records.

All 513 source encoding, loss, operator and exact projection comparisons pass. The complete 256-candidate grammar, three original witnesses and four adverse controls pass native verification with zero axiom dependencies. The construction uses finite generated forms throughout.

Let $q : X \to C$ denote the admitted first-lookup coverage map. For each declared edge $(x, y) \in R$, total coverage supplies both images, so the local operator is

$$\mathrm{op}(x, y) = ((x, y), q(x), q(y)).$$

Traversing R yields the ordered operator list, whose source-edge projection is exactly R. Empty traversal returns the empty list; adjoining an edge appends its operator after all earlier records. A failed endpoint lookup rejects that local operator.

The distinction ledgers use the complete source-pair support:

$$L_{\mathrm{closed}} = [(x, y) \mid q(x) = q(y)], \qquad L_{\mathrm{retained}} = [(x, y) \mid q(x) \neq q(y)].$$

For coarse and fine coverage q_c, q_f, registered projection π supplies $\pi(q_f(x)) = q_c(x)$ for their retained source images. If $q_f(x) = q_f(y)$, substitution gives $q_c(x) = q_c(y)$. Contraposition therefore yields

$$q_c(x) \neq q_c(y) \Rightarrow q_f(x) \neq q_f(y).$$

Thus finer equality implies coarser equality on the corresponding retained source support, and earlier distinctions survive refinement.

Fold discretization law is complete: its native derivations, full 256-form grammar, original witnesses and controls, and every program output field and hash agree with pinned V3.

> Source, proof and provenance: V3 DISCRETIZATION; V3 exact_error_ledger; SFT.Computation.Discretization.completeSourceRelation; SFT.Computation.Discretization.refinementRetainsDistinction; Timeline: Turn 330, line 3242.

273. Registered convergence traces and successor certificates

Each refinement step retains the exact reference and obtains both reports from registered outputs. Admission requires the declared refinement pair, the registered comparison pair and nested unresolved error classes. Every unresolved cell is traced to its original non-equal error-ledger row. Reconstruction recovers every executable transition; nesting composes over arbitrary finite traces, so a closed cell cannot reappear as an unresolved error.

A separate checked certificate establishes strict decrease of the unresolved-class count. For traces satisfying that certificate, the number of successor steps plus the terminal unresolved count is bounded by the initial unresolved count. The executable certifiedDescent check tests strict class descent on every retained report pair. Target acceptance additionally requires the declared terminal identity and an admitted exact report.

All 793 source-derived step, trace, target and class-descent comparisons pass. The complete 256-candidate grammar, three original witnesses and four adverse controls pass native verification with zero axiom dependencies.

For an admitted exact report r, define its ordered unresolved-class list by

$$U(r) = [\mathrm{cell}(\rho) \mid \rho \in r_{\mathrm{errors}},\ \rho_{\mathrm{orientation}} \neq \mathrm{equal}].$$

Nesting is $\forall c \in U(r'), c \in U(r)$. A registered step additionally requires its named pair in both the refinement and comparison relations and successful exact reports at both names. The recursive step runner and the Steps constructors agree by induction, with one retained report pair per successor name.

Nesting composes by membership implication. Hence an initial absence $c \notin U(r_0)$ implies $c \notin U(r_k)$. Strict descent adds $|U(r')| < |U(r)|$. For a certified strict trace of k steps,

$$k + |U(r_k)| \leq |U(r_0)|.$$

The base is equality at $k = 0$. For a successor, strict natural-number decrease gives $1 + |U(r_1)| \leq |U(r_0)|$; combining it with the inductive tail bound proves the displayed inequality. Target acceptance still separately requires a registered terminal name and its admitted report. Neither nesting alone nor target-name membership is substituted for the strict-descent certificate.

Fold convergence-certificate law is complete: its native derivations, full 256-form grammar, original witnesses and controls, and every program output field and hash agree with pinned V3.

`stepsStrict` connects the executable pair list to the bound: the all-pairs check supplies the head decrease, its tail restriction supplies the recursive premise, and equality of the registered report lookups identifies the adjoining reports. This constructs a StrictTrace for the actual run. Applying `strictTraceBound`

then yields the displayed inequality at arbitrary trace length. stepsStrict; strictTraceBound.

> Source, proof and provenance: V3 CONVERGENCE; V3 exact_error_ledger; SFT.Computation.Convergence.stepsReconstruct; SFT.Computation.Convergence.strictTraceBound; SFT.Computation.Convergence.runSource; Timeline: Turn 331, line 3252.

274. Typed scientific symbolic traces and normalization

The model retains its exact environment, registered rewrite rules and generated expression syntax. Each admitted rewrite preserves the source and target type interfaces and the exact evaluator value. The evaluator-derived literal form retains the original returned value or rejection; for typed expressions it is a positive exact value and evaluating that literal reproduces the same result.

Typed rewrite traces retain every original record and registered rule identity. Induction proves type, value and normalized-result preservation through the complete trace. Serial composition and adjoining a rewrite preserve all earlier records. Rule normality is the explicit predicate excluding every admitted rewrite whose source equals the terminal expression. Physical interpretation remains outside formal symbolic closure.

All 320 original evaluator comparisons and two concrete typed-rewrite/normalization proofs pass. The complete 256-candidate grammar, three witnesses and four controls pass native verification with zero axiom dependencies.

For environment Γ, let $E_\Gamma(e)$ be exact evaluation and define

$$N_\Gamma(e) = E_\Gamma(e).\,\mathrm{map}(v \mapsto \mathrm{Literal}(v)).$$

Thus $E_\Gamma(e) = \mathrm{ok}(v)$ gives
$N_\Gamma(e) = \mathrm{ok}(\mathrm{Literal}(v))$.
For a typed expression in an environment with distinct names, evaluation supplies a positive exact fraction v, and replay gives
$E_\Gamma(\mathrm{Literal}(v)) = \mathrm{ok}(v)$.

Each Rewrite contains its registered record, admission proof, and typed source and target. Its value-preservation theorem implies

$$N_\Gamma(\mathrm{source}(r)) = N_\Gamma(\mathrm{target}(r)).$$

The identity TypedTrace retains its typed expression. At each successor, transitivity composes this equality with the tail equality, proving equal normal-form results and the same positive evaluation at both endpoints. The trace also retains membership of every rule in the registered rule list. Concatenating traces preserves these results. A Normalized certificate additionally carries the terminal predicate Normal relative to those rules; this is distinct from the evaluator's literal result.

Fold scientific symbolic-computation law is complete: its native derivations, full 256-form grammar, original witnesses and controls, and every program output field and hash agree with pinned V3.

The terminal certificate is constructed by `normalizationBoundary`: project the typed trace to its original source Trace and pair it with the terminal Normal proof. In symbols,

$$\mathrm{TypedTrace}(e, T, f) \wedge \mathrm{Normal}(f) \Longrightarrow \mathrm{Normalized}(e, T, f).$$

The separate evaluator-derived literal is determined by the exact returned value; the rewrite-normality predicate quantifies over the supplied rules and contexts. normalizationBoundary.

> Source, proof and provenance: V3 SYMBOLIC; V3 exact_error_ledger; SFT.Computation.ScientificSymbolic.typedNormalization; SFT.Computation.ScientificSymbolic.traceEvaluation; Timeline: Turn 332, line 3258.

275. Registered simulation and complete observation traces

Simulation executes the original admitted state system from a registered initial state under its declared action schedule. Each visited state is mapped to a retained observation. Reconstruction recovers the exact state execution and every observation lookup. There is one observation per visited state, including the initial form; an omitted visited observation halts the report.

Serial execution preserves the state and observation interfaces. Appending an admitted transition adds its exact next state and observation while preserving earlier records. Extending an admitted model with retained transition and observation rows preserves its earlier reports. Comparison acceptance records only membership in the declared relation between the simulated and external observation traces. The result remains a consequence of its registered model; empirical natural-law status is not derived by simulation.

All 1,280 source-execution and observation comparisons pass, covering valid traces, missing transitions, invalid actions, outside initial states and omitted observations. The complete 256-candidate grammar, three original witnesses and four controls pass native verification with zero axiom dependencies.

For state observation map O, observation recursion is

$$\mathrm{obs}([]) = \mathrm{some}([]), \qquad \mathrm{obs}(s :: u) = \mathrm{some}(v :: w)$$

exactly when $\mathrm{lookup}(O, s) = \mathrm{some}(v)$ and $\mathrm{obs}(u) = \mathrm{some}(w)$. Simulation combines

$$\mathrm{run}(P, s, a) = \mathrm{ok}(\tau), \qquad \mathrm{obs}(\tau) = \mathrm{some}(\omega).$$

Reconstruction yields the state Execution and the complete ObservationRows trace. Induction on observation rows gives $|\omega| = |\tau|$; the execution trace includes the initial state, so

$$|\omega| = |a| + 1.$$

For an appended admitted action leading to s', its registered observation v' extends the report by $\tau +\!\!+ [s']$ and $\omega +\!\!+ [v']$. Serial paths use the retained intermediate interface and concatenate their observation segments without losing the visited states. Model extension preserves old transition rows and exact observation lookups. External comparison remains the exact membership test $(\omega, \omega_{\mathrm{external}}) \in C$ in the supplied comparison relation.

Fold simulation law is complete: its native derivations, full 256-form grammar, original witnesses and controls, and every program output field and hash agree with pinned V3.

> Source, proof and provenance: V3 SIMULATION; V3 exact_error_ledger; SFT.Computation.Simulation.executionCertificate; SFT.Computation.Simulation.exactSimulation; Timeline: Turn 333, line 3267.

276. Exact computational dynamics and orbit certificates

An admitted computational dynamics model retains its complete state support and held update relation. Every accepted orbit reconstructs all visited states, registered transitions, successor time labels and the first-return trace. The identity orbit retains empty One as its clock. No deterministic-update restriction is

imposed on the source relation.

A fixed form requires its explicit self-transition. The first returned cycle preserves the earliest repeated state and all intervening update rows; absence of a return is equivalent to distinctness of the trajectory. Any registered ledger closed under the update relation contains every state of an orbit that starts within it. Stability for a registered perturbation support is equivalent to a finite transition derivation from each registered start to the specified terminal form.

Serial composition joins equal state interfaces. Appending a registered transition extends its orbit and time trace by that exact pair. Retaining states and relation rows in an admitted extension preserves earlier reports and predecessor distinctions. Physical time and continuous dynamics retain the source empirical correspondence boundary.

All 2,745 comparisons with the original dynamics routines pass, including every two-state transition relation, valid and invalid trajectories, fixed forms, predecessor records, earliest returns and ordered perturbation searches. The complete 256-candidate grammar, three original witnesses and four controls pass native verification with zero axiom dependencies.

For admitted update relation R on states S, an orbit trajectory $\tau = [s_0, \ldots, s_k]$ requires every state in S, every adjacent pair in R, and an admitted transition-time result. The identity trajectory has

$$\mathrm{orbit}([s]) = ([s], \mathrm{EmptyOneClock}, \mathrm{none}).$$

For an accepted orbit, no return is equivalent to distinctness of τ. A returned cycle has the exact decomposition

$$\tau = (u{+\!+}[t]{+\!+}v){+\!+}[t]{+\!+}w, \qquad \mathrm{cycle} = [t]{+\!+}v{+\!+}[t],$$

with $u{+\!+}[t]{+\!+}v$ distinct and $t \notin u$. These conditions identify the first repeated state and retain the intervening registered transitions.

If a ledger L contains s_0 and is closed under R, induction on adjacent pairs proves $\forall s \in \tau, s \in L$. Fixed-form membership is exactly $(s, s) \in R$; registered perturbation stability is exactly reachability of the terminal from every supplied start. Appending $(s_k, s_{k+1}) \in R$ appends precisely that pair to the time trace. Orbit composition joins the shared state once, and inclusion of state and update supports preserves the same admitted orbit.

Fold computational-dynamics law is complete: its native derivations, full 256-form grammar, original witnesses and controls, and every program output field and hash agree with pinned V3.

> Source, proof and provenance: V3 COMPUTATIONAL-DYNAMICS; V3 exact_error_ledger; SFT.Computation.ComputationalDynamics.orbitReconstruction; SFT.Computation.ComputationalDynamics.returnedOrbit; SFT.Computation.ComputationalDynamics.orbitInvariant; Timeline: Turn 334, line 3279.

277. Inverse problems and retained predecessor fibres

An inverse problem retains the complete generated source and observation supports and their registered forward relation. The relation may have several images for a source or several sources for an observation. Its inverse fibre lists every possible predecessor in the original source order. Reconstruction selects a source exactly when this fibre is a singleton; distinct possible predecessors remain unresolved. An unregistered observation cannot yield a reconstructed source.

An exclusion ledger retains its declared and applied exclusions with their registration certificate. Every remaining predecessor is an original possible source absent from the applied exclusions. An exclusion that has not been registered cannot remove a possible source. Reconstruction after registered exclusions requires the complete remaining fibre to be a singleton and supplies its exact forward-relation witness.

Relational composition retains the intermediate witness in both directions. The structural single-source base reconstructs its registered observation. Extension retains previous possible predecessors, and the successor rule appends precisely the newly generated source when its forward pair is present. No fitted regularizer or unregistered prior selects among unresolved fibres.

All 1,984 source-admitted fibre, registered-exclusion and exact relational-composition cases pass. The complete 256-candidate grammar, three original witnesses and four controls pass native verification with zero axiom dependencies.

For source support X, observation support Y and admitted relation $R \subseteq X \times Y$, the inverse fibre is

$$F(y) = [x \in X \mid (x, y) \in R], \qquad \mathrm{reconstruct}(y) = \mathrm{some}(x) \iff F(y) = [x].$$

A singleton fibre supplies the exact source and forward-row witness. Two distinct fibre members contradict singleton reconstruction, so ambiguity returns none. Any fibre member also proves $y \in Y$; consequently an unregistered observation cannot reconstruct a source.

For applied exclusions A with certificate $A \subseteq D$, the remaining list is

$$F_A(y) = [x \in F(y) \mid x \notin A].$$

An undeclared possible source $x \notin D$ cannot belong to A and remains in $F_A(y)$. Successful excluded reconstruction requires $F_A(y) = [x]$, yielding both its original forward witness and uniqueness among remaining members. Composition retains the intermediate source:

$$(x, z) \in R;Q \iff \exists y,\ (x, y) \in R \wedge (y, z) \in Q.$$

For a fresh appended source x', with old relation decisions retained, the new fibre is $F(y) +\!\!+ [x']$ if (x', y) is present, and $F(y)$ otherwise.

Fold inverse-problem law is complete: its native derivations, full 256-form grammar, original witnesses and controls, and every program output field and hash agree with pinned V3.

> Source, proof and provenance: V3 INVERSE-PROBLEM; V3 exact_error_ledger; SFT.Computation.InverseProblem.reconstructionExactly; SFT.Computation.InverseProblem.exclusionReconstructionCertificate; Timeline: Turn 335, line 3290.

278. Computational statistics from exact support traces

A statistical computation retains complete canonical sample support and its total statistic map. The exact summary preserves every class label, all source samples in that class, the reduced positive class-to-whole fraction and every unresolved within-class distinction. Classes cover the complete support without overlap. Singleton certainty retains structural empty One.

The classification certificate reconstructs each source lookup in order and contains exactly one output label per sample. Certificates compose by concatenating their sample and result traces. Two-stage statistic composition retains the actual intermediate lookup; distinctions closed by an upstream statistic cannot be restored by its downstream deterministic map.

A fresh sample extends the exact classification trace, changes only its corresponding class membership, and adds precisely its pairs with the prior members of that class. Exact class fractions are updated against the extended support count. The structural single-sample base yields one complete class. Probability retains its deterministic-support interpretation unless an external distribution is measured.

All 608 checks pass: 488 comparisons with the original exact entropy-ledger routine, including invalid-input errors and fresh-sample extensions, and 120 complete classification-trace equations. The complete 256-candidate grammar, three original witnesses and four controls pass native verification with zero axiom dependencies.

For canonical sample support X and total statistic f, each class is

$$C_y = [x \in X \mid f(x) = y].$$

Coverage follows by taking $y = f(x)$; if a sample belongs to both C_y and C_z, uniqueness of its statistic image gives $y = z$. Each summary row retains C_y, its unresolved within-class pairs and a positive exact fraction a_y/b_y satisfying

$$|C_y|\, b_y = a_y |X|, \qquad a_y \le b_y.$$

ClassificationTrace records every lookup in order. Its base has two empty lists, and its successor adds one sample and label; hence label count equals sample count, and concatenation composes certificates.

For fresh $x' \notin X$ assigned v,

$$C'_y = C_y +\!\!+ \begin{cases} [x'], & v = y, \\ [], & v \neq y, \end{cases} \qquad |C'_y|\, b'_y = a'_y(|X| + 1).$$

The new classification trace appends v; only the matching class gains the fresh sample and its unresolved pairs. Under a second deterministic statistic g, equal f-images have equal $g(f(\cdot))$-images, so closed distinctions remain closed and retained downstream distinctions were already retained upstream.

Fold computational-statistics law is complete: its native derivations, full 256-form grammar, original witnesses and controls, and every program output field and hash agree with pinned V3.

> Source, proof and provenance: V3 COMPUTATIONAL-STATISTICS; V3 exact_error_ledger; SFT.Computation.ComputationalStatistics.exactStatisticClasses; SFT.Computation.ComputationalStatistics.classificationComplete; Timeline: Turn 336, line 3296.

279. Complete coordinate products and high-dimensional computation

Generated coordinate carriers determine a complete ordered product support. Membership is equivalent to a retained choice from every coordinate carrier, and every row has the same number of positions as the coordinate trace. The number of generated rows is exactly the product of the carrier counts. Canonical carriers produce duplicate-free rows without an enumeration cutoff.

Sparse relations retain selected pairs from that full support; every admitted endpoint has a complete coordinate trace. Factored relations materialize all combinations of their local relation choices while preserving coordinate order. An executable edge trace requires each registered pair and an exact source-target interface at every step. Replay reconstructs the complete execution, and admitted execution retains a generated terminal row.

The resource ledger records the exact numbers of generated product rows, retained relation rows and executed transition rows. Coordinate and execution composition preserve their constituent traces; adding a coordinate appends its actual selected value. A declared projection retains selected positions and rejects unavailable positions. If two distinct full rows yield the same projection, no decoder of that projected result can reconstruct both original rows. Dimension remains a generated coordinate trace.

All 1,320 literal-product, factored-relation, original relation-admission, retained-execution and projection cases pass. The complete 256-candidate grammar, three original witnesses and four controls pass native verification with zero axiom dependencies.

For coordinate lists $A_1, \ldots, A_d$, complete support is generated recursively:

$$P([]) = [[]], \qquad P(A :: R) = \mathrm{concat}_{a \in A}[a :: r \mid r \in P(R)].$$

The corresponding CoordinateTrace chooses one member from each list. Induction proves

$$r \in P(A_1, \ldots, A_d) \iff \text{the row has that complete coordinate trace}, \quad |r| = d, \quad |P| = \prod_{i=1}^{d} |A_i|.$$

Distinct coordinate carriers give distinct rows because different heads separate branches and induction separates equal-head tails. Factored relations use the same recursion on local source/target pairs, retaining a FactorTrace for every materialized pair.

Replay of edges checks the current source equality and relation membership at each step; it is equivalent to the typed Execution. Sparse-relation admission places both endpoints in P, so induction preserves generated terminal support. The resource tuple is $(|P|, |R|, |\tau|)$. Projection recursively selects the declared indices and returns exactly that many values if all indices exist. If distinct rows a, b both project to p, simultaneous decoding would imply $a = d(\mathrm{some}(p)) = b$, a contradiction.

Fold high-dimensional computation law is complete: its native derivations, full 256-form grammar, original witnesses and controls, and every program output field and hash agree with pinned V3.

> Source, proof and provenance: V3 HIGH-DIMENSIONAL; V3 exact_error_ledger; SFT.Computation.HighDimensional.completeProduct; SFT.Computation.HighDimensional.exactProductCardinality; SFT.Computation.HighDimensional.productDistinct; Timeline: Turn 337, line 3308.

280. Many-body joint support and interaction records

A many-body computational model retains distinct body identities, their complete canonical local state carriers and a registered interaction graph. Every interaction endpoint belongs to the body carrier. The joint support contains every local-state combination once, with one coordinate per body and an exact product cardinality.

An admitted update relation contains only generated joint endpoints. Execution reconstructs every retained update and preserves the complete terminal state. Serial execution and appending an admitted update preserve all prior transition records. The structural single-body base retains its local states; adding a body appends its exact local coordinate to each generated joint row.

A configuration is a canonical held selection of the declared joint support. Its correlation ledger records the presence of every declared combination, while the missing-combination ledger retains the exact absent rows. Factorization against the declared local product holds precisely when the selection contains every generated joint row. A missing combination provides an explicit rejection witness. Empty held support retains its original structural empty-One presentation.

The resource ledger records body identities, interaction edges, generated joint states, admitted update rows and executed transitions. These are consequences of retained finite structures; no physical many-body law or quantum amplitude is imported. All 1,716 original graph-admission and source-admitted selection, relation and execution checks pass. The complete 256-candidate grammar, three original witnesses and four controls pass native verification with zero axiom dependencies.

For bodies $B_1, \ldots, B_n$ with local carriers S_i, the joint support is

$$J = P(S_1, \ldots, S_n), \qquad |J| = \prod_i |S_i|, \qquad x \in J \Rightarrow |x| = n.$$

Graph admission proves distinct body identities and membership of both endpoints of every interaction edge. Canonical local carriers give duplicate-free J. An admitted update relation lies in $J \times J$; execution retains its actual rows and a terminal member of J. Composition concatenates traces, and a successor appends its registered update pair.

For held configuration $H \subseteq J$, the exact missing list and correlation ledger are

$$M = [x \in J \mid x \notin H], \qquad C = [(x, [x \in H]) \mid x \in J].$$

The unrestricted factorization check tests equality of support membership in both directions; for an admitted configuration, $H \subseteq J$ is already certified, so

$$\mathrm{factorized}(H) \iff \forall x \in J,\ x \in H.$$

Any $x \in M$ therefore rejects factorization. Adding a body extends an old row x by each admitted new coordinate s, producing $x{+}{+}[s]$. Resource counts retain the numbers of bodies, interaction edges, joint rows, update rows and executed rows separately.

Fold many-body computation law is complete: its native derivations, full 256-form grammar, original witnesses and controls, and every program output field and hash agree with pinned V3.

> Source, proof and provenance: V3 MANY-BODY; V3 exact_error_ledger; SFT.Computation.ManyBody.completeJointSupport; SFT.Computation.ManyBody.jointCardinality; SFT.Computation.ManyBody.configurationFactorizationExactly; Timeline: Turn 338, line 3317.

281. Prediction seals and registered target access

The source empirical boundary separates prediction from target opening. An unsealed prediction can be sealed once; an existing seal cannot be replaced. Its identity binds the experiment identifier, prediction hash, trace hash and envelope hash through the original canonical hash record.

After sealing, the supplied target identities must coincide with the registered identity set. The native membership-equivalence proof preserves this set comparison, including repeated registered identities. Successful opening retains both the existing seal and the target material; premature access and mismatched identities are rejected. The canonical-value comparison protocol also preserves the source operand-returning conjunction and rejects measurement-driven changes to the law.

These boundary results pass 124 checks against the pinned source, including identity-set cases, access ordering, replacement rejection and seal hashes. These seal and target-access results form the empirical-boundary component of the completed mathematical-modelling entry. Registration declarations are audited separately from the actual Lean theorem dependencies.

The seal identity is the canonical hash of the exact object

$$\{\text{experiment_id} : e,\ \text{prediction_hash} : p,\ \text{trace_hash} : t,\ \text{envelope_hash} : v\}.$$

Sealing is a two-case operation: an empty seal slot returns the fresh seal; an occupied slot rejects replacement. Target opening first requires an existing seal s, then checks supplied and registered identity lists by

$$\text{match}(A, R) \iff (\forall a \in A,\ a \in R) \wedge (\forall r \in R,\ r \in A) \iff \forall n,\ n \in A \leftrightarrow n \in R.$$

Thus order and repeated identities do not alter the source set-comparison condition. When it holds, opening returns exactly $(s, \text{material})$, preserving both; otherwise it rejects.

For Boolean protocol fields, admission is

$$\neg\text{targetBeforeSeal} \wedge \neg\text{measurementChangesLaw} \wedge \text{allRows} \wedge \text{sourceIdentified}.$$

The canonical-value operation retains the source operand-returning conjunction. If the first two operands are false in truth value and allRows is true in truth value, its result is the original source operand; a truthy law-mutation operand yields Boolean false. This preserves the source's value-level behavior as well as the Boolean boundary.

> Source, proof and provenance: V3 MATHEMATICAL-MODELLING; V3 seal_prediction; SFT.Computation.MathematicalModelling.unsealedAccepts; SFT.Computation.MathematicalModelling.resealRejected; SFT.Computation.MathematicalModelling.exactTargetOpening; Timeline: Turn 340, line 3335.

282. Formal structures, observations and retained model comparisons

The mathematical-modelling carrier records a disclosed phenomenon, the admitted model registration, finite formal and observation supports, and their exact registered relation. The relation need not be total or single-valued. Registration entails classified provenance, no declared axioms or free parameters, and admitted dependency receipts; the actual Lean dependency audit is separate.

Execution begins in declared support and reconstructs every retained relation row. Identity execution supplies the base. Serial composition, one-step extension and inclusion-preserving model extension preserve the complete operation trace.

Each comparison retains its formal input, observation and admitted comparison protocol. The complete ledger records every agreement or disagreement with the declared relation. Its length equals the number of supplied comparisons; concatenation and successor addition preserve earlier rows. The disagreement filter contains precisely the supplied rows absent from that relation. Revision records retain separate earlier and later model ledgers, preserving the original evidence.

These results preserve the source boundary that fitting observations or producing an opaque prediction does not establish the model law. The 1,164 source-admitted execution, comparison and empirical-boundary checks pass, together with the complete 256-candidate grammar, three exact-error witnesses and four controls. All native audited declarations have zero axiom dependencies.

A correspondence supplies finite formal support X, observation support Y, relation $R \subseteq X \times Y$, and admitted registration. Registration proves

$$\text{declaredAxioms} = [], \quad \text{freeParameters} = [], \quad \text{missingDependencies} = [],$$

together with nonempty classified provenance. These are registration-field statements; the separate Lean axiom audit concerns actual proof dependencies.

OperationTrace has identity and registered-edge successor constructors. Its replay satisfies

$$\mathrm{execute}(x,\tau)=\mathrm{some}(z)\iff x\in X\wedge \mathrm{OperationTrace}(x,\tau,z).$$

Every trace edge belongs to R; induction proves serial composition and transport under relation inclusion. For supplied comparison rows C, each with the admitted protocol and supported endpoints, the complete ledger is

$$L(C)=[((c_x,c_y),[(c_x,c_y)\in R])\mid c\in C].$$

Consequently $|L(C)|=|C|$ and $L(C{+\!\!+}D)=L(C){+\!\!+}L(D)$. Falsifying rows are exactly those supplied rows failing membership in R. Revision stores $(L_{\mathrm{old}},L_{\mathrm{new}})$; projection recovers the unchanged old ledger instead of overwriting prior comparison evidence.

Fold mathematical-modelling law is complete: its native derivations, full 256-form grammar, original witnesses and controls, and every program output field and hash agree with pinned V3.

> Source, proof and provenance: V3 MATHEMATICAL-MODELLING; V3 seal_prediction; SFT.Computation.MathematicalModelling.comparisonAgreementExactly; SFT.Computation.MathematicalModelling.executionExactly; Timeline: Turn 340, line 3335.

283. Reversible gates and exact phase-labelled support

A reversible gate has a nonempty finite word map with distinct sources and distinct targets, together with a nonempty phase-successor trace. Inversion swaps each source and target while retaining row order and the phase trace. The inverse satisfies the same admission conditions; applying the row inversion twice recovers the original gate.

An admitted state has nonempty branch support, distinct words and a distinct phase cycle containing at least two labels. Every branch phase belongs to that cycle. A nonempty phase action shifts the registered phase index by the trace length modulo the cycle length. The native preservation proof establishes that the action retains every word in its original order, retains the phase cycle and returns another admitted state.

Complete support generation uses every word of the declared position count over the registered alphabet, in literal product order. The source permits a singleton alphabet; that domain is preserved. Generated-support membership is equivalent to the exact word length and membership of every label in the declared alphabet.

These components pass 818 comparisons against the pinned V3 operations, including rejected inputs and exact phase results. All 38 declarations in this component audit have zero axiom dependencies. Gate application, the complete inverse state trace and the other reversible-model components are included in the completed reversible-model entry.

For gate map G and successor trace u, admission is

$$G\neq[],\quad \mathrm{distinct}(\pi_1 G),\quad \mathrm{distinct}(\pi_2 G),\quad u\neq[].$$

Inversion maps $(x,y)\mapsto(y,x)$, swaps the two uniqueness conditions, retains u, and satisfies $(G^{-1})^{-1}=G$ in the original row order. A state (B,C) requires nonempty branch list B, distinct branch words, distinct phase cycle C with $|C|\geq 2$, and every branch phase in C.

For C_i the phase at index i, phase action is

$$(w, C_i) \longmapsto (w, C_{(i+s) \bmod d}), \qquad d = |C|, \quad s = |u| \bmod d.$$

Phase lookup and the modulo bound prove that the result remains in C. Mapping this action over branches retains the exact word list, and therefore its nonemptiness and distinctness; state admission is preserved. Complete support over alphabet A and n registered positions satisfies

$$w \in \text{support} \iff |w| = n \wedge \forall a \in w,\ a \in A.$$

The original positive, distinct alphabet guard permits $|A| = 1$.

> Source, proof and provenance: V3 REVERSIBLE-MODEL; V3 FoldQuantumState; V3 ReversibleGate; SFT.QuantumComputation.ReversibleModel.inverseTwice; SFT.QuantumComputation.ReversibleModel.inverseAdmission; SFT.QuantumComputation.ReversibleModel.completeSupportExactly; Timeline: Turn 349, line 3395.

284. Exact inverse phase and branch reconstruction

For a phase-cycle length d, an original phase index below d and a shift s below d, the source inverse shift is zero when s is zero and d−s otherwise. Native constructive arithmetic proves that the forward modular shift followed by this inverse returns the original index. Distinct phase labels then identify the original phase exactly.

The complete branch transformation combines a registered word lookup with this phase action. When target words are distinct, reversing the word-map rows recovers each source word. The branch-level inverse theorem restores every original word and phase pair in order, including all retained branches.

The implementation passes 3,264 additional comparisons against original V3 gate application and inverse-state results. The 17 additional declarations have zero axiom dependencies. Wrapper admission and domain preservation are established by the integrated round-trip derivation below.

For $d > 0$, $i < d$ and $s < d$, define the exact inverse shift

$$\bar{s} = \begin{cases} 0, & s = 0, \\ d - s, & s \neq 0. \end{cases}$$

The modular reconstruction is

$$((i + s) \bmod d + \bar{s}) \bmod d = i.$$

When $s = 0$, reduction returns i. Otherwise $s + (d - s) = d$; moving the first remainder through addition reduces the expression to $(i + d) \bmod d = i$. Distinct phase-cycle labels make phaseIndex of a selected label recover its unique original index, so this arithmetic identity restores the original phase.

For the word map, a forward lookup $G(w) = v$ and distinct target words imply $G^{-1}(v) = w$. Combining the two inverses gives

$$(w, C_i) \xrightarrow{G,s} (v, C_{(i+s) \bmod d}) \xmapsto{G^{-1},\bar{s}} (w, C_i).$$

Induction over the branch list restores each head pair and then the tail, proving equality of the entire original ordered branch list.

> Source, proof and provenance: V3 REVERSIBLE-MODEL; V3 apply_gate; SFT.QuantumComputation.ReversiblePhase.inverseIndex; SFT.QuantumComputation.ReversiblePhase.phaseInverse; Timeline: Turn 349, line 3395.

285. Admitted gate round trips

For every admitted state and reversible gate whose source domain equals the complete state support, forward gate application produces an admitted result with the same number of branches. Every output word comes from a retained input branch through the registered word map. Distinct target words preserve distinct support; phase actions preserve membership in the registered cycle.

The output support is exactly the inverse gate's source domain. This equality is derived from the forward operation and gate admission. Applying the inverse gate with the source inverse-phase action therefore restores the complete original state, including branch order, word identities, phase labels and phase cycle. The round-trip theorem needs no additional inverse-domain assumption.

The constructive proofs have zero axiom dependencies. The underlying operation definitions are unchanged from the implementation checked against 3,264 original V3 application and inverse-state cases. Joint composition, observation records and the other reversible-model components are included in the completed entry.

The application-domain check is equality of complete support membership:

$$\forall w,\ w \in \pi_1 G \leftrightarrow w \in \mathrm{support}(B).$$

With admitted state and gate, every source lookup and phase shift therefore succeeds. Distinct gate targets preserve distinct output words, and the transformation preserves branch count and phase-cycle membership, giving an admitted output state B'.

The transformed-support theorem gives

$$v \in \mathrm{support}(B') \iff \exists (w,\phi) \in B,\ G(w) = v.$$

Domain equality and gate admission identify this support with $\pi_2 G = \pi_1 G^{-1}$. Hence the inverse-domain condition is derived from the forward application. Applying the branch inverse theorem yields the full round trip

$$\mathrm{apply}(B, G, \mathrm{false}) = \mathrm{ok}(B') \quad \Longrightarrow \quad \mathrm{apply}(B', G^{-1}, \mathrm{true}) = \mathrm{ok}(B).$$

The existence theorem additionally supplies B', its admission and $|B'| = |B|$; no separate inverse-domain premise is needed.

> Source, proof and provenance: V3 REVERSIBLE-MODEL; V3 apply_gate; SFT.QuantumComputation.ReversibleApplication.applicationInverse; SFT.QuantumComputation.ReversibleApplication.admittedRoundTrip; Timeline: Turn 349, line 3395.

286. Complete joint-state composition

Joint-state composition enumerates the literal Cartesian product of the two branch lists, preserving the original nested order. Each joint word contains the left word, the registered joint separator and the right word. Its phase is the cycle label at the sum of the two source indices modulo the common cycle length.

The native proofs establish both exact output-word membership and the full product branch count. Every pair of input branches contributes its registered row. For admitted inputs with a shared phase cycle, all phase lookups succeed; final admission follows the original state constructor's distinct-word check. The source guard and its rejection behavior are preserved.

The implementation matches V3 in 2,809 composition cases: 1,003 accepted results and 1,806 rejections. All 19 composition definitions and theorems have zero axiom dependencies. These results establish the joint-composition component of the completed reversible-model entry.

For a shared phase cycle C of length d, joint construction sends

$$((u, C_i), (v, C_j)) \longmapsto (u +\!\!+ [\text{joint}] +\!\!+ v,\ C_{(i+j) \bmod d}).$$

Its outer traversal follows the left branch list and its inner traversal follows the right branch list. Thus the complete joint word projection is

$$\text{concat}_{l \in B_L}[\, l_w +\!\!+ [\text{joint}] +\!\!+ r_w \mid r \in B_R\,], \qquad |B_{\text{joint}}| = |B_L||B_R|.$$

Every generated word has exactly the stated pair-of-input-branches membership witness. Admitted phases make every index lookup and modular shift succeed. The final state constructor still checks distinct joint words: admitted inputs and equal cycles do not bypass this source guard. Accordingly composition returns the generated state when that word-list check passes and the original duplicate-word error otherwise; unequal phase cycles reject before joint construction.

> Source, proof and provenance: V3 REVERSIBLE-MODEL; V3 compose; SFT.QuantumComputation.ReversibleComposition.jointMembership; SFT.QuantumComputation.ReversibleComposition.compositionCount; Timeline: Turn 349, line 3395.

287. Complete observation and predecessor records

An admitted classification assigns exactly one label to each word in the full state support. The observation record retains every original word, phase and assigned label. Its projection reconstructs the original branch list, and its length equals the original branch count.

Selection retains precisely the branches assigned the requested label, in their original order. A nonempty selected class remains an admitted state with the original phase cycle; an empty class receives the source rejection. The complete observation record remains available independently of the selected class.

Predecessor merging groups all classified branches under the source's sorted target labels. Each original predecessor retains its word and phase. The factorization decision holds precisely when every combination of a retained left coordinate and retained right coordinate belongs to the registered joint support, with the source nonempty and uniqueness guards preserved.

These operations pass 8,537 comparisons against original V3 observation, predecessor-merge and factorization outputs. All 28 native definitions and theorems have zero axiom dependencies.

A classification f is admitted when its source words are distinct and have exactly the state's support membership. Complete recording traverses the original branches:

$$R = [((w, \phi), f(w)) \mid (w, \phi) \in B], \qquad \pi_1 R = B, \quad |R| = |B|.$$

For requested label l, selection is

$$B_l = [(w, \phi) \in B \mid f(w) = l].$$

This filter preserves original order, distinct words and phase membership. A nonempty B_l therefore forms an admitted state with the original cycle; an empty B_l returns the registered rejection. The complete record R remains separate from the selected class.

Predecessor merging sorts the distinct observed labels and returns $[(l, B_l)]$ in that order; each original branch belongs to its corresponding group with word and phase retained. For a nonempty, duplicate-free joint support J, the factorization check is

$$\mathrm{factorable}(J) \iff \forall (a,b) \in J,\ \forall (c,d) \in J,\ (a,d) \in J.$$

The nested all-members traversal proves exactly this closure over retained left and right coordinates.

> Source, proof and provenance: V3 REVERSIBLE-MODEL; V3 predecessor_merge; V3 observe; SFT.QuantumComputation.ReversibleObservation.recordMembership; SFT.QuantumComputation.ReversibleObservation.observationReconstruction; Timeline: Turn 349, line 3395.

288. Complete repetition fault-mask enumeration

For a generated fault-depth trace of length d, the repetition encoder retains 2d + 1 copies of the held label. The fault census enumerates every position selection of size zero through d, in the original combination order, and records the changed word and its decoded label.

The native enumeration theorem establishes that a mask occurs exactly when it is a source-order sublist of the full position range with length at most d. It applies to arbitrary widths and depths. The decoder preserves the original rejection conditions: empty words, labels outside held/returned, and tied counts.

All 18 translated definitions and proved results have zero axiom dependencies. The implementation passes 1, 322 original-source comparisons, including all 168 census rows for widths 3, 5 and 7 across both held labels. Arbitrary-depth correction is established in Section 289.

For ordered position list P, the combinations recursion is

$$C(P, 0) = [[]], \qquad C([], k+1) = [],$$

$$C(p :: P, k+1) = [p :: m \mid m \in C(P,k)] +\!\!+ C(P, k+1).$$

The first branch includes the current position and the second omits it, preserving the original combination order. Induction on P proves

$$m \in C(P,k) \iff m \text{ is a source-order sublist of } P \wedge |m| = k.$$

Concatenating these lists for $k = 0, \ldots, d$ over $P = [0, \ldots, w-1]$ therefore gives

$$m \in \mathrm{Masks}(w,d) \iff m \text{ is a sublist of } [0,\ldots,w-1] \wedge |m| \le d.$$

An admitted depth trace has nonempty distinct labels and length d; encoding returns $w = 2d+1$ copies of the selected held/returned label. Changing a word tests each successive position against m, preserving word length. The decoder first checks nonempty support and allowed labels, then rejects equal held/returned counts and otherwise returns the strictly more numerous label.

> Source, proof and provenance: V3 REVERSIBLE-MODEL; V3 exhaustive_fault_census; SFT.QuantumComputation.ReversibleFaults.combinationExactly; SFT.QuantumComputation.ReversibleFaults.faultMasksExactly; Timeline: Turn 349, line 3395.

289. Arbitrary-depth repetition correction

For every admitted repetition encoding with fault depth d, every mask in the complete census changes at most d positions among 2d + 1 retained labels. The opposite-label count is bounded by the mask length, and the two label counts sum to the full word length. The original label therefore has a strict majority.

The native correction theorem applies to both held and returned labels at arbitrary finite depth. The complete census returns precisely its source-ordered changed words, each paired with the original decoded label; no mask is omitted. All 14 correction definitions and theorems have zero axiom dependencies.

The eight original operational witnesses are also reproduced: complete support, phase action, joint composition, predecessor merging, nonfactorability, observation with retained record, inverse recovery, and exhaustive repetition correction at widths 3, 5 and 7. Their complete named output agrees with V3. All 16 witness declarations have zero axiom dependencies.

Let r be the opposite-label count and c the original-label count after applying a census mask m. The change recursion proves

$$r \le |m| \le d, \qquad c + r = 2d + 1.$$

For the count bound, a changed head consumes one occurrence of its position from the mask; removing that position does not affect any later position. Induction bounds the remaining opposite labels. The second identity follows because each word position contains exactly one of the two labels.

If $c \le r$, then $c + r \le 2r \le 2d$, contradicting $c + r = 2d + 1$. Hence $r < c$, and the original decoder returns the original label for both encodings. Induction over the complete ordered mask list now gives

$$\mathrm{Census}(a, D) = \mathrm{ok}\big([(\mathrm{change}(m, \bar{a}, a^{\,2d+1}), a) \mid m \in \mathrm{Masks}(2d+1, d)]\big).$$

Here $a^{\,2d+1}$ denotes list replication, $\bar{a}$ the other held label, and $d = |D|$. The result covers every admitted finite depth and every enumerated mask, including the empty mask.

> Source, proof and provenance: V3 REVERSIBLE-MODEL; V3 repetition_encode; V3 repetition_decode; V3 exhaustive_fault_census; SFT.QuantumComputation.ReversibleFaults.majorityFromBudget; SFT.QuantumComputation.ReversibleFaults.exhaustiveCorrection; Timeline: Turn 349, line 3395.

290. Reversible-model grammar and operational controls

The eight registered binary axes generate all 256 source candidates. Native exhaustion and uniqueness proofs establish the sole all-preserving member and reject each alternative coordinate. Every candidate identifier, exact form, decision and reason agrees with the pinned V3 census.

All four original controls retain their exact tests, descriptions and receipt hashes. These outputs agree with V3. The registration, census, decision lookup and closure representations preserve the original canonical record structure. Their 70 native definitions and theorems have zero axiom dependencies.

Observation-record extension preserves the complete earlier record and appends the newly classified branch. Native append and successor theorems establish both exact order and retention of every earlier record row.

Pair-cell and phase extension obey the same exact append laws: complete earlier rows remain in their original order, followed by every newly generated row. Position extension uses the established Fold-word successor identity. All seven successor theorems have zero axiom dependencies.

The grammar is the ordered product of the support, unit, composition, phase, transformation, observation, generality and addition axes:

$$\mathcal{G} = \prod_{i=1}^{8} [\text{rejected}, \text{admitted}], \qquad |\mathcal{G}| = 256, \qquad \text{survives}(g) \iff \bigwedge_i g_i = \text{admitted}.$$

The source identifier comparison equals this coordinate criterion. Complete enumeration and the conjunction therefore establish the single all-admitted survivor.

Writing s for its identifier, s_- for the incomplete-support identifier, h for the supplied source hash, S for the enumerated survivor identifiers, E for the exclusions and W for the original witness Booleans, the four original control predicates are exactly

$$s_- \neq s, \qquad H(\{\text{changed} : h\}) \neq h, \qquad |S| = 1, \qquad E \neq [] \wedge \bigwedge W.$$

These are the retained source tests; the artifact control checks the literal survivor count.

For admitted observation records, recursion proves

$$\text{record}(B +\!\!+ C) = \text{record}(B) +\!\!+ \text{record}(C).$$

A newly classified branch thus appends exactly its branch/label row. The same list induction gives the pair-row, joint-row and phase-row append laws. Position extension retains the full Fold-word successor identity $W_{n+1} = \text{extendWords}(A, W_n)$.

> Source, proof and provenance: V3 REVERSIBLE-MODEL; V3 operational_witnesses; SFT.QuantumComputation.ReversibleGrammar.candidatesComplete; SFT.QuantumComputation.ReversibleGrammar.uniqueSurvivor; SFT.QuantumComputation.ReversibleControls.falsePremisePassed; SFT.QuantumComputation.ReversibleControls.boundaryPassed; Timeline: Turn 349, line 3395.

291. Complete finite reversible execution and interpretation

A finite execution trace retains each admitted gate application and its predecessor state. Serial composition preserves every earlier row. Applying the inverse gates in exact reverse order restores the initial state. The predecessor-record length equals the number of gate applications, and the complete branch count is preserved throughout execution.

The parameterized description interface retains each registered initial state and gate sequence. Its decoder recovers the exact request, and evaluation invokes only the source gate operation. Every registered description has its corresponding interpretation; admitted traces preserve their complete record, reverse execution and resource counts. Every finite registry and code support satisfying encoding validation, exact decoding and unique request names is covered by the same interpreter theorem.

All 18 execution and interpretation declarations have zero axiom dependencies. The implementation agrees with 4, 165 traces formed by repeated applications of V3 operations. The registered build, axiom audit and fresh candidate comparisons verify the complete reversible-model implementation against pinned V3.

Complete Fold reversible-computation model complete: native source derivations, full 256-candidate enumeration, all eight original witnesses and fault censuses, all four controls, and every original program output field and hash are verified against pinned V3.

Forward execution has an identity constructor and a successor retaining the current state, admitted gate and exact application result:

$$B_0 \xrightarrow{G_1} B_1 \xrightarrow{G_2} \cdots \xrightarrow{G_k} B_k, \qquad P = [B_0, \ldots, B_{k-1}].$$

Each gate's round-trip theorem supplies its inverse step. Inductively invert the tail first, then the head, yielding

$$B_k \xrightarrow{G_k^{-1}} \cdots \xrightarrow{G_1^{-1}} B_0, \qquad |P| = k, \qquad |B_k| = |B_0|.$$

Every inverse step uses the registered inverse phase action. Concatenation of forward traces concatenates both gate and predecessor lists.

The interpreter decodes a codeword to a name, looks up that name's request $(B_0, [G_i])$, and runs this exact gate evaluator. Under unique registry names, admitted encoding and an exact decoder, each registered request has an admitted codeword c satisfying

$$\mathrm{interpret}(c) = \mathrm{some}(\mathrm{evaluate}(B_0, [G_i])).$$

For an admitted Forward trace, evaluation returns precisely (P, B_k); inverse execution and both counts above therefore transfer to that interpretation.

The registry and code support are theorem parameters. Encoding validation and exact decoding supply, for every registry member, a supported codeword decoding to its name. Unique registry names make lookup recover that particular request. `exactRequest` then substitutes the decoder and lookup equations into the interpreter; induction on Forward substitutes the successful gate evaluation. Neither the registry size nor the gate-list length is fixed by the operational comparison census. completeFrozenGrammar; forwardExecutes; interpretedInverse.

> Source, proof and provenance: V3 REVERSIBLE-MODEL; V3 apply_gate;
> SFT.QuantumComputation.ReversibleExecution.backwardSerial;
> SFT.QuantumComputation.ReversibleExecution.completeInverse;
> SFT.QuantumComputation.ReversibleInterpretation.forwardExecutes;
> SFT.QuantumComputation.ReversibleInterpretation.exactRequest; Timeline: Turn 349, line 3395.

292. Fold quantum information unit

An admitted one-position support over the two held labels contains exactly the two corresponding one-label words. Every generated position retains both distinct alternatives, with its complete earlier and later word support. The registered finite phase-label carrier remains unchanged.

Observation selects only the originally classified alternatives. Its complete two-row record reconstructs the original branches and retains the phase cycle. These properties use the original support and observation operations, without importing a conventional qubit or complex-vector representation.

The native unit proofs and the complete unchanged 256-candidate grammar pass the registered build, zero-axiom audit and fresh V3 candidate comparison. The shared phase, gate, composition, observation and fault-correction operations retain their verified source behavior.

For admitted complete support over two distinct labels a, b and one position,

$$\operatorname{support}(B) = [[a], [b]], \qquad |B| = 2.$$

Given generated prefix u and suffix v, the complete product includes both $u{+\!\!+}[a]{+\!\!+}v$ and $u{+\!\!+}[b]{+\!\!+}v$.
They have the same generated length and alphabet membership; equality would force $a = b$ at the inserted coordinate, contradicting distinctness.

For an accepted observation with complete record R, reconstruction gives

$$\pi_1 R = B, \qquad |R| = 2, \qquad C_{\text{selected}} = C_{\text{original}}.$$

Every selected branch has word $[a]$ or $[b]$ and its classification lookup equals the selected label. The two alternatives and phase carrier are therefore retained by the original complete-support and observation operations.

Fold quantum information-unit law complete: native source derivations, full 256-candidate enumeration, all eight original witnesses and fault censuses, all four controls, and every original program output field and hash are verified against pinned V3.

> Source, proof and provenance: V3 INFORMATION-UNIT; V3 complete_support; V3 observe; SFT.QuantumComputation.InformationUnit.twoLabelUnit; SFT.QuantumComputation.InformationUnit.everyPositionRetainsBoth; Timeline: Turn 350, line 3404.

293. Joint-state provenance and marginal reconstruction

Every generated joint branch has precisely the provenance of an original left and right branch pair. Its word is the original concatenation with the joint separator; its phase is selected by the sum of the component phase indices modulo the common cycle. The complete row order and product branch count are retained.

Every pair of admitted component branches contributes its cell. Projecting the retained component provenance reconstructs both original marginals. These proofs preserve the original word domain and state-constructor guards.

For a registered nonempty unique pair support, the original nonfactorable decision holds exactly when a cross-pair formed from its projected coordinates is missing. The positive decision is exactly closure under every such cross-pair. The ten native composition declarations have zero axiom dependencies; the shared original composition and factorization comparisons remain applicable.

Define a component-provenance cell by

$$\operatorname{Cell}(l, r, z) \iff l \in B_L \wedge r \in B_R \wedge \operatorname{jointBranch}(C, l, r) = \operatorname{some}(z).$$

Successful composition establishes

$$z \in B_J \iff \exists l, r, \operatorname{Cell}(l, r, z).$$

Every admitted pair l, r supplies a cell. Since both component states have nonempty branch lists, choosing a branch on the other side proves the marginal equivalences

$$l \in B_L \iff \exists r, z,\ z \in B_J \wedge \operatorname{Cell}(l, r, z),$$

with the symmetric formula for $r \in B_R$. These projections use retained provenance, not an assumed parser for the joint-word separator. Each cell separately proves its concatenated word and modular sum of phase indices.

For nonempty duplicate-free pair support J, failure of the nested cross-pair check supplies an actual failed member at each traversal:

$$\mathrm{factorable}(J) = \mathrm{ok}(\mathrm{false}) \iff \exists l, r \in J,\ (l_1, r_2) \notin J.$$

Conversely any such pair witnesses failure of the complete closure test.

Fold quantum state-composition law complete: native source derivations, full 256-candidate enumeration, all eight original witnesses and fault censuses, all four controls, and every original program output field and hash are verified against pinned V3.

> Source, proof and provenance: V3 STATE-COMPOSITION; V3 compose; SFT.QuantumComputation.StateComposition.compositionRowsExactly; SFT.QuantumComputation.StateComposition.leftMarginalExactly; SFT.QuantumComputation.StateComposition.rightMarginalExactly; Timeline: Turn 351, line 3410.

294. Complete generated support and phase retention

For an admitted alphabet of size b and generated position trace of length k, the source support contains exactly b^k branches. Every branch records a word of length k over the registered alphabet and the initial held phase label. The exact membership theorem includes every generated word.

Every nonempty phase action preserves the entire word support, its exact branch count and its distinctness. The construction retains one word-and-phase record per branch without introducing an amplitude magnitude, normalization constant or stochastic selection.

The five native support theorems and the claim-specific complete 256-candidate grammar pass the registered build, zero-axiom audit and fresh source comparison. The shared original operation checks remain applicable because the executable operations are unchanged.

For alphabet A, k registered positions and initial phase ϕ, complete generation returns

$$B = [(w, \phi) \mid w \in W_k(A)], \qquad (w, \psi) \in B \iff |w| = k \wedge (\forall a \in w,\ a \in A) \wedge \psi = \phi.$$

The word recursion has $|W_0| = 1$ and $|W_{k+1}| = |A|\,|W_k|$. Induction therefore yields

$$|B| = |A|^k.$$

For every admitted nonempty phase-action trace, the mapped word projection remains exactly $W_k(A)$. The original phase-action preservation theorem gives admitted output, and equality of word projections gives unchanged branch count and distinct support. Thus the generated count and membership are preserved together, not inferred from a sample of branches.

Superposition-equivalent Fold support law complete: native source derivations, full 256-candidate enumeration, all eight original witnesses and fault censuses, all four controls, and every original program output field and hash are verified against pinned V3.

> Source, proof and provenance: V3 SUPERPOSITION; V3 complete_support; V3 phase_action; SFT.QuantumComputation.Superposition.fullBranchCount; SFT.QuantumComputation.Superposition.completeRowMembership; Timeline: Turn 352, line 3419.

295. Exact phase period and predecessor interference

A generated phase trace whose length equals the registered cycle length restores the entire original state. An inverse action with the source inverse shift restores every original branch and phase while retaining the same cycle.

Each reported interference fibre contains exactly the original branches assigned to its image. Their held phase labels are preserved. Branches assigned the same image remain together in that complete fibre; no predecessor is discarded. This is the source interpretation of interference as exact predecessor merging.

All seven phase and interference theorems have zero axiom dependencies. The claim-specific full 256-candidate grammar passes the registered build, audit and fresh V3 comparison; the unchanged shared operational comparisons remain applicable.

For phase-cycle length d, a trace of length d has shift $d \bmod d = 0$. Phase-index reconstruction proves $\mathrm{shift}(C, 0, \phi) = \phi$ for each registered phase. Applying this equality to every branch gives

$$\mathrm{phaseAction}(B, u) = \mathrm{ok}(B) \quad \text{when } |u| = d.$$

For a general admitted action with shift s, any nonempty reverse trace whose length modulo d is the source inverse shift $\bar{s}$ restores the same state by the modular inverse identity.

If predecessor merging returns groups G, then for every retained group (l, F_l),

$$b \in F_l \iff b \in B \wedge \mathrm{lookup}(f, b_{\mathrm{word}}) = \mathrm{some}(l).$$

The branch b includes its original phase. Two branches with the same registered image therefore inhabit the same exact fibre, both retained; no phase or predecessor is replaced by the class label.

Fold phase and interference operation law complete: native source derivations, full 256-candidate enumeration, all eight original witnesses and fault censuses, all four controls, and every original program output field and hash are verified against pinned V3.

> Source, proof and provenance: V3 PHASE-INTERFERENCE; V3 phase_action; V3 predecessor_merge; SFT.QuantumComputation.PhaseInterference.exactPhasePeriod; SFT.QuantumComputation.PhaseInterference.predecessorFibreExactly; SFT.QuantumComputation.PhaseInterference.phaseActionInverse; Timeline: Turn 353, line 3433.

296. Exact component correlations and retained observation provenance

For the admitted nonempty, unique finite pair support, nonfactorability is equivalent to the presence of retained pairs with a missing cross pair. The two retained pairs necessarily differ in both component coordinates. The original source factorability decision therefore has a constructive correlation witness.

After pair-cell composition and observation, a selected branch corresponds exactly to an original component pair whose joint branch has the selected observation label. The complete record retains every original component pair, including those outside the selected class. This establishes the source correlation and observation statements without removing component provenance.

All four entanglement theorems have zero axiom dependencies. The complete 256-candidate grammar passes the registered build, audit and fresh V3 comparison. The shared composition and observation operations retain their original verified source correspondence.

For nonempty unique joint support J, a missing cross pair gives retained $l, r \in J$ with $(l_1, r_2) \notin J$. If $l_1 = r_1$, this cross pair is $r \in J$; if $l_2 = r_2$, it is $l \in J$. Both contradict its absence. Hence

$$\text{nonfactorable}(J) \Rightarrow \exists l, r \in J,\ l_1 \neq r_1 \wedge l_2 \neq r_2 \wedge (l_1, r_2) \notin J.$$

The converse failure witness is the same absent cross pair in the original factorability test.

For successful composition followed by observation label v, exact membership composes to

$$z \in B_{\text{selected}} \iff \exists l, r, \text{Cell}(l, r, z) \wedge f(z_{\text{word}}) = v.$$

Without the final label condition, every component cell still belongs to the branch projection of the complete observation record. Thus selecting a class changes selected support while preserving the record of all original component pairs.

Fold entangling-composition law complete: native source derivations, full 256-candidate enumeration, all eight original witnesses and fault censuses, all four controls, and every original program output field and hash are verified against pinned V3.

> Source, proof and provenance: V3 ENTANGLEMENT; V3 compose; V3 is_factorable; V3 observe; SFT.QuantumComputation.Entanglement.retainedCorrelationWitness; SFT.QuantumComputation.Entanglement.selectedComponentProvenance; Timeline: Turn 354, line 3439.

297. Exact measurement selection and complete reconstruction

Every branch in the pre-observation state has an exact registered outcome in the complete observation record. A branch assigned to another outcome is absent from the selected state but remains in that record with its original word, phase and outcome. Selected branches retain their original order.

Projecting the record onto its original branch rows and retaining the unchanged phase cycle reconstructs the entire original state. Selection and reconstruction use the original finite classification relation; no stochastic collapse postulate or outcome-frequency premise is introduced. Outcome frequencies retain the source empirical boundary.

All four measurement theorems have zero axiom dependencies. The complete 256-candidate grammar passes the registered build, audit and fresh V3 comparison. The unchanged observation operation retains its original source-comparison evidence.

For an accepted observation, classification totality supplies for each $b \in B$ a registered outcome o with $(b, o) \in R$. If $o \neq v$, exact selected-class membership gives

$$b \notin B_v, \qquad (b, o) \in R.$$

The first conclusion concerns the selected state; the second preserves the omitted branch in the complete record. The filter equation is

$$B_v = [b \in B \mid f(b_{\text{word}}) = v],$$

which retains order. Since $\pi_1 R = B$ and the selected phase cycle equals the original cycle C, complete reconstruction is the structural equality

$$(\pi_1 R, C_{\text{selected}}) = (B, C).$$

Every original word and phase is restored by projection, including branches outside the selected class.

Fold quantum measurement-semantics law complete: native source derivations, full 256-candidate enumeration, all eight original witnesses and fault censuses, all four controls, and every original program output field and hash are verified against pinned V3.

> Source, proof and provenance: V3 MEASUREMENT; V3 observe; SFT.QuantumComputation.Measurement.reconstructEntireState; SFT.QuantumComputation.Measurement.selectedClassPreservesOrder; Timeline: Turn 355, line 3451.

298. Exact reversible gate branch correspondence

Every output branch of an accepted gate application corresponds to an original input branch through the registered word map and exact held phase action. Conversely, every such input branch contributes its corresponding output branch. The result support equals the complete registered image support, and admitted distinct source words cannot merge to one target word.

The original inverse gate and inverse phase shift recover the entire original state. Serial composition preserves the order of the registered gate applications and reconstructs predecessors in reverse order. These statements retain the finite source-bound gate descriptions; physical gate fidelity remains the source empirical engineering boundary.

All four gate-specific theorems have zero axiom dependencies. The complete 256-candidate grammar passes the registered build, audit and fresh V3 comparison. The unchanged shared phase, application and serial-evaluation operations retain their source-comparison evidence.

For gate G acting with shift s on cycle C, the exact branch correspondence is

$$z \in B' \iff \exists b \in B,\ \mathrm{lookup}(G, b_{\mathrm{word}}) = \mathrm{some}(z_{\mathrm{word}}) \wedge \mathrm{shift}(C, s, b_{\mathrm{phase}}) = \mathrm{some}(z_{\mathrm{phase}}).$$

Induction on the branch transformation proves both directions: a produced head supplies its source row, while tail membership uses the induction hypothesis. With gate admission, the image support is exactly the target projection of G. If two source words have the same lookup image, uniqueness of target rows in the inverse map gives

$$G(x) = y = G(x') \Rightarrow x = x'.$$

The admitted inverse application restores the complete original state. For a gate list, serial composition retains causal order and the inverse proof uses precisely the reversed list with inverse word maps and inverse phase shifts.

Fold reversible transformation and gate law complete: native source derivations, full 256-candidate enumeration, all eight original witnesses and fault censuses, all four controls, and every original program output field and hash are verified against pinned V3.

> Source, proof and provenance: V3 GATE; V3 apply_gate; SFT.QuantumComputation.GateLaw.exactGateBranch; SFT.QuantumComputation.GateLaw.branchMapInjective; Timeline: Turn 356, line 3457.

299. Exact circuit evaluation, inverse and observation ledger

Successful evaluation of an admitted circuit yields its complete causal trace of original gate applications. The trace determines an inverse circuit in exact reverse gate order, retains one predecessor state per gate, and preserves the complete branch count and held phase cycle.

The final observation record retains the original branch count. Its branch projection together with the unchanged initial phase cycle reconstructs the entire terminal state, including the distinctions outside the selected observation class. Hardware timing and noise remain external to the source circuit semantics.

All five circuit-specific theorems have zero axiom dependencies. The registered build, audit and fresh comparison preserve the full 256-candidate grammar. Shared gate, serial-evaluation and observation operations retain their original source-comparison evidence.

Suppose the initial state and every listed gate are admitted and evaluation returns (P, B_k). Induction over the evaluator uses admission preservation for its head application, then reconstructs the tail, proving Forward for the entire run. The resulting circuit certificate gives

$$\mathrm{Backward}(B_k, \mathrm{reverseOrder}(G), B_0), \quad |P| = |G|, \quad |B_k| = |B_0|, \quad C_k = C_0.$$

The last equality follows by composing the phase-cycle equality from every application.

For an accepted terminal observation with complete record R, observation reconstruction and these invariants imply

$$|R| = |B_0|, \qquad (\pi_1 R, C_0) = (B_k, C_k).$$

The record reconstructs the entire terminal state, rather than only its selected branch class. All identities refer to the actual successful evaluation and original gate operations.

Fold quantum circuit syntax and semantics law complete: native source derivations, full 256-candidate enumeration, all eight original witnesses and fault censuses, all four controls, and every original program output field and hash are verified against pinned V3.

> Source, proof and provenance: V3 CIRCUIT; V3 observe; V3 apply_gate; SFT.QuantumComputation.Circuit.evaluationHasExactTrace; SFT.QuantumComputation.Circuit.circuitInverseAndLedger; Timeline: Turn 357, line 3467.

300. Exact simulation of the frozen circuit-description grammar

The common interpreter decodes a registered description and executes its exact finite circuit. Successful interpretation determines the original decoded name, source-bound request and complete causal trace. Every registered description in the validated encoding relation has a corresponding code whose interpretation preserves that trace, its reverse-order inverse, the held phase cycle and the gate-count ledger.

Observation after interpreted execution retains the original branch count. The complete record and unchanged phase cycle reconstruct the entire terminal state. These results establish exact simulation for every request in any registry satisfying the encoding and decoder hypotheses.

All four universality-specific theorems have zero axiom dependencies. The registered build, audit and fresh V3 comparison preserve the complete 256-candidate grammar and unchanged shared source operations.

Successful interpretation has a recoverable source chain

$$\mathrm{decode}(c) = \mathrm{ok}(n), \quad \mathrm{lookup}(\mathcal{R}, n) = \mathrm{some}(q), \quad \mathrm{evaluate}(q_{\mathrm{initial}}, q_{\mathrm{gates}}) = \mathrm{ok}(P, B).$$

For any validated encoding with unique registry names and an exact decoder, every registered request with an admitted Forward execution has some admitted codeword satisfying this chain. Substituting the

request into the circuit theorems supplies its exact inverse execution, unchanged phase cycle and $|P| = |q_{\text{gates}}|$.

An accepted observation after that interpretation has

$$|R| = |q_{\text{initial}}.\text{branches}|, \qquad (\pi_1 R, q_{\text{initial}}.\text{phaseCycle}) = B.$$

Thus encoding coverage, request recovery and exact circuit execution are separate proved links. Registry, code support, encoding, decoder and admitted circuit trace are all parameters of this theorem.

Fold quantum universality law complete: native source derivations, full 256-candidate enumeration, all eight original witnesses and fault censuses, all four controls, and every original program output field and hash are verified against pinned V3.

Write $N = \text{map}(\text{fst}, \mathcal{R})$. The coverage premises are

$$\text{distinct}(N) = \text{true}, \quad \text{validateEncoding}(N, S, E) = \text{true}, \quad \text{exactDecoder}(N, E) = \text{some}(D).$$

For each $(n, q) \in \mathcal{R}$ and each
$\text{Forward}(q_{\text{initial}}, q_{\text{gates}}, P, B)$,
`completeFrozenGrammar` yields $c \in S$ with $E(n) = \text{some}(c)$ and the exact interpreted evaluation. `forwardExecutes` replaces that evaluation by the retained run. `exactInverseTrace`, `circuitPhaseCycle` and `retainedPredecessorCount` then supply inverse execution, phase-cycle equality and the exact gate count. This proves coverage for arbitrary finite registries and arbitrary admitted circuit lengths; the 256-form candidate census separately checks the claim's eight-axis program grammar. everyEncodedCircuitSimulated.

> Source, proof and provenance: V3 UNIVERSALITY; V3 apply_gate; SFT.QuantumComputation.Universality.everyEncodedCircuitSimulated; SFT.QuantumComputation.Universality.interpretedObservationPreserved; Timeline: Turn 358, line 3474.

301. Exact algorithm branch derivations and decoded outcomes

A final algorithm branch is present exactly when an original input branch reaches it through the registered sequence of word lookups and held phase actions. This correspondence records each gate in causal order and retains both the branch word and phase.

A selected observation branch has exactly such an input-to-output trace together with the selected classification label. Each interference group similarly contains all and only traced branches assigned to its image. The original complete support and predecessor records are preserved. Named speedups retain the source requirement for their own exact problem family and resource proof.

The branch-trace representation and all three algorithm-specific theorems have zero axiom dependencies. The registered build, audit and fresh V3 comparison preserve the full 256-candidate grammar and unchanged shared operations.

For fixed cycle C, define the branch relation recursively:

$$T_C([], b, z) \iff b = z,$$

$$T_C(g :: G, b, z) \iff \exists m,\ \text{lookup}(g, b_w) = \text{some}(m_w) \wedge \text{shift}(C, s_g, b_\phi) = \text{some}(m_\phi) \wedge T_C(G, m, z), \qquad s_g = |g_{\text{successors}}| \bmod |C|.$$

Induction on the full Forward trace, using the gate branch correspondence at each head, proves

$$z \in B_{\text{terminal}} \iff \exists b \in B_{\text{initial}},\ T_C(G, b, z).$$

Exact observation membership appends $f(z_w) = v$ to this condition for a selected class. Exact predecessor-fibre membership appends the same condition for an interference group. Both therefore retain an actual initial branch and every intermediate word lookup and phase action in causal order.

Fold quantum algorithm-family law complete: native source derivations, full 256-candidate enumeration, all eight original witnesses and fault censuses, all four controls, and every original program output field and hash are verified against pinned V3.

> Source, proof and provenance: V3 ALGORITHMS; V3 predecessor_merge; V3 observe; V3 apply_gate; SFT.QuantumComputation.Algorithms.everyBranchHasExactTrace; SFT.QuantumComputation.Algorithms.selectedBranchCorrectness; Timeline: Turn 359, line 3487.

302. Exact live-support and measurement resource counts

Every intermediate state of an admitted circuit retains the complete initial branch count. The live-support ledger contains one such count per gate, and serial composition adds the exact gate counts. For a generated alphabet of size b and a position support of size n, each live state and the terminal state retain b^n branches. The complete measurement record also has b^n rows.

Canonical input and occupied-position records, causal-depth witnesses, gate and wire supports, and query/communication rows retain the inherited exact source resource definitions. Lower and upper claims remain tied to their registered witnesses; no separation of conventional complexity classes follows without a separate certificate.

All five quantum-resource theorems have zero axiom dependencies. The registered build, audit and fresh comparison preserve the complete 256-candidate grammar and unchanged shared source operations.

Let $n = |B_0|$ and $k = |G|$. Each gate preserves branch count, so induction on Forward proves the same count for every predecessor state in P, as well as for the terminal state:

$$[\,|B| \mid B \in P\,] = \text{replicate}(k, n), \qquad |B_{\text{terminal}}| = n.$$

Serial execution concatenates the two predecessor lists; their lengths add to the combined gate count. For complete support over alphabet size b and position count m, substitution of the generated cardinality gives

$$[\,|B| \mid B \in P\,] = \text{replicate}(k, b^m), \qquad |B_{\text{terminal}}| = b^m, \qquad |R_{\text{observation}}| = b^m.$$

The last equality follows from complete observation-record reconstruction, not from counting selected branches. These are exact finite support and gate ledgers for the admitted execution.

Fold quantum complexity law complete: native source derivations, full 256-candidate enumeration, all eight original witnesses and fault censuses, all four controls, and every original program output field and hash are verified against pinned V3.

> Source, proof and provenance: V3 COMPLEXITY; V3 complete_support; V3 observe; SFT.QuantumComputation.Complexity.generatedLiveSupportCost; SFT.QuantumComputation.Complexity.generatedMeasurementCost; Timeline: Turn 360, line 3497.

303. Exact communication provenance and complete joint records

The complete joint observation record reconstructs both the sender and receiver supports through their retained component-pair provenance. A local sender transformation preserves the exact connection from each original sender branch through its registered word map and phase action into the joint branch.

The complete record has exactly the product of the sender and receiver branch counts. Correlations and branches outside the selected observation class remain represented in the source records. The inherited finite channel relation, closed-distinction and resource ledgers retain their exact source semantics. Physical transmission capacity and security require the registered media experiments specified by V3.

All four communication-specific theorems have zero axiom dependencies. The registered build, audit and fresh V3 comparison preserve the complete 256-candidate grammar and unchanged shared operations.

For sender state S, receiver state T, their accepted joint state and complete observation record R, component provenance gives

$$t \in T \iff \exists s, z,\ z \in \pi_1 R \wedge \mathrm{Cell}(S, T, s, t, z),$$

and symmetrically for $s \in S$. This follows from the complete marginal theorem and $\pi_1 R = B_{\mathrm{joint}}$, so branches outside the selected observation class remain represented.

If a local sender gate first produces S', each joint branch has the stronger chain

$$z \in B_{\mathrm{joint}} \iff \exists s, s', t,\ s \in S \wedge G(s_w) = s'_w \wedge \mathrm{shift}(C, s_G, s_\phi) = s'_\phi \wedge \mathrm{Cell}(S', T, s', t, z),$$

where the lookup and phase equalities denote their successful Option results. This combines exact gate provenance with exact joint-cell membership. Finally, complete observation and product cardinality give

$$|R| = |S|\,|T|.$$

The count and provenance describe the retained finite communication construction; the physical correspondence remains attached to its registered media evidence.

Fold quantum communication law complete: native source derivations, full 256-candidate enumeration, all eight original witnesses and fault censuses, all four controls, and every original program output field and hash are verified against pinned V3.

> Source, proof and provenance: V3 COMMUNICATION; V3 compose; V3 observe; SFT.QuantumComputation.Communication.receiverSupportRetained; SFT.QuantumComputation.Communication.completeCommunicationRecordCost; Timeline: Turn 361, line 3507.

304. Exact coding reconstruction and disjoint error supports

The original redundant held and returned codewords have disjoint supports under the declared bounded error actions. Every mask generated by the original fault family decodes to its original logical label. The empty error action leaves the codeword unchanged, and the original decoder is a left inverse of the admitted encoder.

The complete error enumeration, generated fault-depth trace and source decoder domain are retained. The coding result applies to the registered generated error family, preserving the source boundary without introducing an additional noise model.

All four coding-specific theorems have zero axiom dependencies. The registered build, audit and fresh V3 comparison preserve the complete 256-candidate grammar and the unchanged fault-operation comparisons, including all original width-three, width-five and width-seven census rows.

For depth d, let E_h and E_r be the words obtained from the held and returned repetition encodings by masks of size at most d. Arbitrary-depth correction gives

$$x \in E_h \Rightarrow \mathrm{decode}(x) = \mathrm{ok}(\mathrm{held}), \qquad x \in E_r \Rightarrow \mathrm{decode}(x) = \mathrm{ok}(\mathrm{returned}).$$

If one word belonged to both supports, the same decoder result would equal two distinct successful labels. Constructor injectivity contradicts this, establishing $E_h \cap E_r = \varnothing$.

The empty-mask recursion satisfies $\mathrm{change}([], w) = w$. Applying correction to that mask yields the left-inverse identity

$$\mathrm{encode}(a, D) = \mathrm{ok}(w) \Rightarrow \mathrm{decode}(w) = \mathrm{ok}(a).$$

For a nonempty declared mask, the complete mask-membership theorem supplies the size bound used by correction. Thus both reconstruction and disjointness are consequences of the retained decoder on the specified error family.

Fold quantum coding law complete: native source derivations, full 256-candidate enumeration, all eight original witnesses and fault censuses, all four controls, and every original program output field and hash are verified against pinned V3.

> Source, proof and provenance: V3 CODING; V3 repetition_encode; V3 repetition_decode; SFT.QuantumComputation.Coding.declaredErrorSupportsDisjoint; SFT.QuantumComputation.Coding.everyDeclaredErrorReconstructed; Timeline: Turn 362, line 3517.

305. Complete error census and successor recovery

Every admitted encoding has exactly the original repetition width 2t + 1. A row occurs in the actual fault census exactly when an ordered mask drawn from the full generated width, containing at most t positions, produces that recorded word and logical label. Every census row decodes to its recorded original label.

Extending an admitted fault-depth trace preserves recovery for all previously declared errors. The complete bounded-mask enumeration and arbitrary finite-depth recovery proof retain the original family and decoder domain, including every original width-three, width-five and width-seven census row.

All four error-correction-specific theorems have zero axiom dependencies. The registered build, audit and fresh V3 comparison preserve the complete 256-candidate grammar and unchanged shared source operations.

For an admitted encoding with $t = |D|$, list replication gives

$$|w| = 2t + 1.$$

The complete census theorem identifies each actual row by

$$(u, a) \in \mathrm{CensusRows} \iff \exists m,\ m \text{ is a sublist of } [0, \ldots, 2t] \wedge |m| \leq t \wedge u = \mathrm{change}(m, \bar{a}, w).$$

Correction applied to this witness proves $\mathrm{decode}(u) = \mathrm{ok}(a)$ for every row, while the exhaustive map supplies every permitted mask's row in source order.

If $D' = D{+}{+}[d']$ is admitted, then its budget is $t+1$. An old mask still satisfies $|m| \le t \le t+1$, so applying the same correction theorem to the enlarged encoding recovers a. This preserves recovery for each earlier mask without omitting the new depth's additional masks.

Fold quantum error-correction law complete: native source derivations, full 256-candidate enumeration, all eight original witnesses and fault censuses, all four controls, and every original program output field and hash are verified against pinned V3.

> Source, proof and provenance: V3 ERROR-CORRECTION; V3 exhaustive_fault_census; SFT.QuantumComputation.ErrorCorrection.completeCensusRows; SFT.QuantumComputation.ErrorCorrection.everyCensusRowRecovers; Timeline: Turn 363, line 3533.

306. Recovery at every contained circuit location

The original decoder followed by the original encoder restores each encoded word under its declared component faults. Applying these operations to every annotated output branch preserves the entire branch list, its order and held phases. After a contained original gate application, recovery therefore restores the exact gate output.

Induction over the complete causal circuit trace establishes recovery after every location whose encoded output support and component fault masks satisfy the registered containment conditions. Undecodable faults retain the exact source error. No hardware threshold constant or recovery guarantee outside the declared fault model is introduced.

All seven fault-tolerance theorems, three operation compositions and the recovered-execution representation have zero axiom dependencies. The registered build, audit and fresh V3 comparison preserve the full 256-candidate grammar and unchanged source gate and error operations.

Recovery is the original composition

$$\mathrm{recover}_D(u) = \mathrm{decode}(u)\ \mathrm{bind}\ (a \mapsto \mathrm{encode}(a, D)).$$

For encoded w and a declared mask, decoding gives the original label and re-encoding gives exactly w. Therefore repeated declared fault locations satisfy

$$\mathrm{recoverLocations}_D([m_1, \ldots, m_k], w) = \mathrm{ok}(w),$$

by induction on the location list.

For annotated branch rows $((w, \phi), m)$, containment supplies an admitted encoding of w and a declared mask. Componentwise recovery then returns exactly the branch projection of the annotated list, preserving phase and order. If that projection equals a gate's complete output support, recovery restores that entire output. Induction over Forward, with complete contained annotations for every retained state and terminal state, constructs RecoveredExecution for all circuit locations. A decoder error is forwarded unchanged by recover; the theorem requires the stated containment premises at each location.

Fold quantum fault-tolerance law complete: native source derivations, full 256-candidate enumeration, all eight original witnesses and fault censuses, all four controls, and every original program output field and hash are verified against pinned V3.

> Source, proof and provenance: V3 FAULT-TOLERANCE; V3 apply_gate; V3 repetition_encode; V3 repetition_decode; SFT.QuantumComputation.FaultTolerance.recoveryAfterContainedGate; SFT.QuantumComputation.FaultTolerance.completeRecoveredExecution; Timeline: Turn 364, line 3549.

307. Exact simulator execution and observation correspondence

Successful interpreted execution is equivalent, in both directions, to the complete admitted circuit trace of the registered target request. Evaluation errors are likewise preserved exactly. The simulator retains the same gate-count ledger, complete branch count and held phase cycle.

Every selected observation branch and every interference-group branch corresponds exactly to an original input branch, its complete word-and-phase trace, and its registered classification label. This connects the encoded target description to its operational outputs and retained provenance. Simulation derives consequences of the registered model; it does not by itself establish a natural quantum law.

All five simulation-specific theorems have zero axiom dependencies. The registered build, audit and fresh V3 comparison preserve the full 256-candidate grammar and unchanged shared source operations.

For an admitted decoded request q, with all gates admitted, the interpreter and circuit trace satisfy

$$\mathrm{interpret}(c) = \mathrm{some}(\mathrm{ok}(P, B)) \iff \mathrm{Forward}(q_0, q_G, P, B).$$

The forward implication reconstructs the actual evaluator trace; the reverse replays it through the same decoded request. Error correspondence is equally exact:

$$\mathrm{interpret}(c) = \mathrm{some}(\mathrm{error}(e)) \iff \mathrm{evaluate}(q_0, q_G) = \mathrm{error}(e).$$

The successful trace implies $|P| = |q_G|$, $|B| = |q_0|$ and unchanged phase cycle. Combining branch-trace correspondence with observation membership gives

$$z \in B_{\mathrm{selected}} \iff [\exists b \in q_0,\ T_{C_0}(q_G, b, z)] \wedge f(z_w) = l.$$

The same formula characterizes membership in the interference group labelled l. Thus decoded simulation retains exactly the target request's execution, errors, resources and classified branch provenance.

Fold quantum simulation law complete: native source derivations, full 256-candidate enumeration, all eight original witnesses and fault censuses, all four controls, and every original program output field and hash are verified against pinned V3.

> Source, proof and provenance: V3 SIMULATION; V3 observe; V3 apply_gate; SFT.QuantumComputation.Simulation.exactExecutionCorrespondence; SFT.QuantumComputation.Simulation.simulatedObservationCorrespondence; Timeline: Turn 365, line 3562.

308. Complete quantum verification challenge schedules

Branchwise acceptance is equivalent to checking every original input-to-output word-and-phase trace admitted by the observation decoder. This equivalence extends to every challenge in the registered finite schedule. A failing challenge on one accepted branch rejects the complete schedule. Excluded branches remain reconstructible in the full measurement record.

Claim and witness supports retain their complete pair-cell construction. Verifier execution preserves its full causal circuit trace, inverse ledger and exact resources. The inherited registered prover and verifier action schedules, statement/witness membership, extraction records and source/program binding controls remain explicit. Security and correctness apply within the registered prover and resource grammar.

All four verification-specific theorems have zero axiom dependencies. The native build and fresh source comparison retain the complete 256-candidate grammar and unchanged operational witnesses and controls.

Let Q be the registered list of Boolean branch challenges and let

$$A(z) \iff \exists b \in B_0,\ T_C(G,b,z) \wedge f(z_w) = l$$

be exact selected-branch membership after the actual circuit execution. The complete challenge computation is

$$\bigwedge_{q \in Q} \bigwedge_{z \in B_{\text{selected}}} q(z) = \text{true} \iff \forall q \in Q,\ \forall b \in B_0,\ \forall z,\ T_C(G,b,z) \wedge f(z_w) = l \Rightarrow q(z) = \text{true}.$$

This follows by expanding both finite all-members checks and substituting the branch-membership equivalence. A supplied challenge q with such a traced selected branch z and $q(z) = \text{false}$ contradicts the universal result and rejects the schedule. If instead its outcome is $o \neq l$, the branch is absent from the selected state but (z, o) remains in the complete record. The schedule quantifies over all registered challenges and all accepted traced branches.

Fold quantum verification law complete: native source derivations, full 256-candidate enumeration, all eight original witnesses and fault censuses, all four controls, and every original program output field and hash are verified against pinned V3.

> Source, proof and provenance: V3 VERIFICATION; V3 observe;
> SFT.QuantumComputation.Verification.completeChallengeSchedule;
> SFT.QuantumComputation.Verification.failedChallengeRejects; Timeline: Turn 366, line 3575.

309. Joint hypothesis-search provenance and retained alternatives

Each selected hypothesis-search branch corresponds, in both directions, to an original sample/hypothesis pair cell, its exact word-and-phase circuit trace and its observation label. The same correspondence holds for interference groups. The complete measurement record retains every alternative and its exact trace, including alternatives outside the selected class.

The complete record contains exactly the product of the original sample and hypothesis support sizes. Circuit and branch ledgers retain the original gate and support resources. The inherited classical inference and evaluation laws retain consistent hypotheses, sealed predictions, all withheld evaluation targets and their separation from training examples. No pretrained quantum model, fitted amplitude or unexplained speedup is introduced.

All four learning-specific theorems have zero axiom dependencies. The registered build and fresh candidate comparison preserve the full 256-candidate grammar, original witnesses and controls.

For sample support E, hypothesis support H, their admitted joint state J, circuit G and selected label l, exact membership factors through both constructions:

$$z \in B_{\text{selected}} \iff \exists e, h, b,\ \text{Cell}(E,H,e,h,b) \wedge T_{C_J}(G,b,z) \wedge f(z_w) = l.$$

The same conjunction describes an interference-fibre member. For the complete observation record, the label is unrestricted:

$$(z,o) \in R \iff [\exists b \in J,\ T_{C_J}(G,b,z)] \wedge f(z_w) = o.$$

Thus every terminal alternative, including unselected outcomes, retains an initial joint branch and its complete circuit trace. Composition gives $|J| = |E||H|$; gate execution preserves this count, and complete observation gives

$$|R| = |E||H|.$$

This cardinality concerns the full retained record, not merely the selected hypothesis class.

Fold quantum learning law complete: native source derivations, full 256-candidate enumeration, all eight original witnesses and fault censuses, all four controls, and every original program output field and hash are verified against pinned V3.

> Source, proof and provenance: V3 LEARNING; V3 compose; V3 observe; SFT.QuantumComputation.Learning.selectedJointExample; SFT.QuantumComputation.Learning.allAlternativesRecorded; Timeline: Turn 367, line 3582.

310. Bidirectional classical and quantum operational correspondence

Projecting each original gate action onto its word map gives exactly the classical word trace. Conversely, every such trace lifts constructively from any held phase to the original phase-labelled execution. Actual terminal-word results and decoded observation results agree in both directions. The full quantum trace retains its phase information.

When every original phase action is a complete-cycle shift, the word trace embeds with unchanged held phase. A singleton word and held phase satisfy the original state constructor exactly when the registered phase cycle is distinct and nontrivial. The singleton embedding retains one branch at every circuit location and at the terminal state; the predecessor ledger has exactly one entry per original gate. No phase-sensitive joint trace is identified with a phase-erased model.

All eight correspondence-specific theorems and the word-trace representation have zero axiom dependencies. The registered build and fresh source comparison retain the complete 256-candidate grammar, witnesses and controls.

WordTrace keeps each registered word lookup while BranchTrace additionally retains the phase update. Projection removes only those phase fields:

$$T_C(G, (x, \phi), (y, \psi)) \Rightarrow W(G, x, y).$$

Conversely, for a nonempty cycle C and held $\phi \in C$, induction on WordTrace chooses each next phase using the registered shift and gives

$$W(G, x, y) \iff \exists \psi,\ T_C(G, (x, \phi), (y, \psi)).$$

With an actual Forward execution, this is equivalent to terminal-word membership; observation adds the same registered label test on y.

If every gate shift is zero modulo $|C|$, each chosen phase remains ϕ, so the lifted trace ends at (y, ϕ). A singleton state is admitted when its phase is held in a distinct cycle of length at least two. Its circuit ledger then satisfies

$$|P| = |G|, \quad |B_{\text{terminal}}| = 1, \quad [|B| \mid B \in P] = \text{replicate}(|G|, 1).$$

Phase-sensitive information remains in the lifted execution even though its word projection has classical correspondence.

Full operational classical-quantum correspondence law complete: native source derivations, full 256-candidate enumeration, all eight original witnesses and fault censuses, all four controls, and every original program output field and hash are verified against pinned V3.

> Source, proof and provenance: V3 CLASSICAL-CORRESPONDENCE; V3 apply_gate; SFT.QuantumComputation.ClassicalCorrespondence.operationalWordCorrespondence; SFT.QuantumComputation.ClassicalCorrespondence.decodedWordCorrespondence; Timeline: Turn 368, line 3589.

311. Registered self-description and quantum computation limits

The original complementary verdict is represented by an admitted singleton phase-labelled quantum request whose registered interpreter output is proved exactly. If a predictor answers on the same decoded description selecting that complementary execution, correctness contradicts the original complement operation. Total prediction is therefore excluded when the registered grammar retains the corresponding admitted self-description support.

The inherited halting boundary quantifies over every predictor with the registered code and verdict interface, including quantum implementations. Its actual stopping and non-stopping witnesses, complement records and same-description conditions remain unchanged. Finite quantum execution retains its decoded registry, complete gate trace, original errors, exact resource ledger and reconstructible observation record. The contradiction is quantified over a predictor and registry satisfying the explicit supported self-description, totality and correctness premises below.

All three limits-specific theorems have zero axiom dependencies. The registered build and fresh source comparison preserve the complete 256-candidate grammar, original operational witnesses and controls.

Let c decode to the registered name selecting a singleton request with complementary verdict $\bar{v}$, held phase ϕ, admitted cycle C and empty gate list. Its interpreter result is exactly

$$\mathrm{interpret}(c) = \mathrm{some}(\mathrm{ok}([], ([[[\bar{v}], \phi)], C))).$$

If a predictor answers v on this same c, the source complement operation supplies $\bar{v} \neq v$. Correctness on that decoded request would require $v = \bar{v}$, a contradiction.

For the total-prediction theorem, the registered support premise first supplies such a decoded self-description and a complementary admitted request for each answer there. Totality supplies an answer in the source Verdict domain. Substitution into the exact interpreter equation and correctness produces the contradiction above. The supported self-description is the premise that binds the predictor answer to the complementary request at the same decoded codeword.

Limits of Fold quantum computation law complete: native source derivations, full 256-candidate enumeration, all eight original witnesses and fault censuses, all four controls, and every original program output field and hash are verified against pinned V3.

More explicitly, supported supplies c, n with $D(c) = \mathrm{ok}(n)$, and for every answer v returned by the predictor at c, an admitted singleton request at n whose output $\bar{v}$ satisfies the source complement equation. Totality at this decoded c supplies such a v in Verdict. The singleton interpreter equation produces $\bar{v}$; correctness at the same code gives $v = \bar{v}$. Rewriting the complement equation by that equality contradicts `complementHasNoFixedPoint`. These steps establish

$$\mathrm{Supported}(\mathcal{R}, D, P) \wedge \mathrm{Total}(D, P) \wedge \mathrm{Correct}(\mathcal{R}, D, P) \Longrightarrow \bot,$$

where the three predicates denote precisely these retained hypotheses. selfPredictionContradiction; totalInternalQuantumPredictionExcluded.

> Source, proof and provenance: V3 LIMITS; V3 apply_gate; V3 self_negating_boundary; SFT.QuantumComputation.Limits.selfPredictionContradiction; SFT.QuantumComputation.Limits.totalInternalQuantumPredictionExcluded; Timeline: Turn 369, line 3596.

312. Complete target-package custody checks

For canonicalised target packages, the vault constructor retains the original experiment, custodian, complete sorted target support, nonce-bound content identity and preregistered envelope identity. Successful release preserves the complete package and nonce, binds it to the supplied prediction seal and changes only the one-use release state. Any later release is rejected.

Exchange acceptance is equivalent to all seven original checks passing: experiment identities, custodian, prediction seal, envelope, complete target support, committed content identity and release payload identity. Combined failures retain the original ordered violation list. These are identity checks; no assumption of mathematical hash injectivity is introduced.

Eight native theorems have zero axiom dependencies. Eleven constructor cases and thirty exchange-tampering cases reproduce the pinned source commitments, release records, hashes and errors. The complete target-custody entry is verified against the pinned source, including its full grammar, operational witness, controls and program output identities.

For sorted named target package V and nonce n, the committed content identity is

$$h_T = H([(\text{name}, n, \text{value}) \mid (\text{name}, \text{value}) \in \text{sort}(V)]).$$

The vault retains V, n, sorted target identities, experiment e, custodian k, expected envelope h_E, and initially false released state. Release checks that state first, then matching seal experiment and envelope. Success returns the unchanged package and nonce, binds the release to the seal hash, and sets only the vault's released flag to true; another release consequently rejects.

Exchange verification collects ordered violations $V_1 +\!\!+ \cdots +\!\!+ V_7$. Each V_i is empty exactly when its check passes, so

$$\text{verify} = \text{ok}() \iff \bigwedge_{i=1}^{7} \text{check}_i \,.$$

The checks compare experiment identities, custodian, seal, envelope, sorted complete names, both recomputed and declared content identities, and the recomputed release-payload identity. This is an equality-of-record-identities theorem, with no hash-injectivity premise.

> Source, proof and provenance: V3 __init__; V3 release; V3 verify; SFT.Physics.TargetCustody.releasePreservesPackage; SFT.Physics.TargetCustody.exchangeChecksExactly; Timeline: Turn 371, line 3625.

313. Named-target admission and portable custody grammar

The source string-key check preserves every admitted named target and its canonical payload in order. Conversely, every complete named package passes this representation step. The original constructor still rejects empty or blank support and malformed experiment, custodian, nonce and envelope identities in their source order.

The original commitment-release-verification witness retains every computed identity and check. Its protocol proof applies to every valid envelope identity and follows exact record and payload preservation. Fresh execution checks the concrete witness and identities. The full eight-axis grammar retains all 256 candidates, its sole preserving form and every original control.

Fifteen constructor/input cases and thirty exchange-tampering cases match the pinned source. The named-entry proofs, general witness protocol and complete grammar are audited with zero axiom dependencies. Custody remains a prerequisite for empirical testing; it does not assert that a physical relation has passed an empirical test.

Portable target commitment and post-seal custody complete: native source derivations, full 256-candidate enumeration, original operational witnesses, all four controls, and every original program output field and hash are verified against pinned V3.

The named-target representation follows

$$N([]) = \mathrm{some}([]), \qquad N((\mathrm{text}(s), v) :: r) = \mathrm{some}((s, v) :: N_r)$$

when $N(r) = \mathrm{some}(N_r)$; any nontext key returns none. Induction proves both preservation and reconstruction:

$$N(E) = \mathrm{some}(V) \Rightarrow [(\mathrm{text}(s), v) \mid (s, v) \in V] = E,$$

$$N([(\mathrm{text}(s), v) \mid (s, v) \in V]) = \mathrm{some}(V).$$

Payloads and order are unchanged. The constructor retains its original identity guards before this conversion and its named-support and envelope guards afterward.

For the original witness package, any syntactically valid envelope identity permits construction; the matching prediction record then permits release. Retained package, nonce, identities and recomputed release hash make all seven verification violations empty, and the released prediction-seal hash equals the supplied seal hash. This proves the witness protocol for every such admitted envelope identity, alongside the concrete source execution.

> Source, proof and provenance: V3 __init__; V3 release; V3 verify; SFT.Physics.TargetCustodyInputs.completeTargetEntries; SFT.Physics.TargetCustodyInputs.everyNamedPackageAccepted; SFT.Physics.TargetCustodyGrammar.generatedCardinality; SFT.Physics.TargetCustodyGrammar.candidatesComplete; Timeline: Turn 371, line 3625.

314. Data-only package admission and protected-tree retention

The hostile-package auditor preserves the original parser and its failure conditions. Successful admission implies that the protected snapshots agree in every field and that the certificate binds the exact parsed program and source document. A changed snapshot cannot pass. Every supplied executable source is rejected, with the original distinction between absent or blank source and nonempty source.

Protected-tree snapshots retain all enumerated census and admitted-model files, in source traversal order, with raw-byte file hashes and the complete canonical snapshot identity. The native retention proofs apply to the complete ordered file inputs supplied by the filesystem adapter.

The original 256-form grammar, sole survivor, source-rejection witness and four controls are preserved. Sixty document-audit cases, ten source-rejection cases and four filesystem cases reproduce the pinned operations, certificate fields, hashes and errors. All registered native declarations have zero axiom dependencies. The full original program-output comparison passes; the hostile-package entry is complete.

Hostile empirical-package and authority protection complete: native source derivations, full 256-candidate enumeration, original operational witnesses, all four controls, and every original program output field and hash are verified against pinned V3.

After the original document parser succeeds, let E indicate a supplied executable-source field, A an additional field, and S_0, S_1 the protected snapshots. The audit decision is

$$\text{pass} \iff \neg E \wedge \neg A \wedge S_0 = S_1.$$

A successful certificate therefore yields snapshot equality and binds

$$h_{\text{package}} = H(\text{document}), \qquad h_{\text{program}} = H(\text{canonical}(\text{parsedProgram})).$$

A changed snapshot contradicts the equality conjunct; parser errors propagate unchanged. Executable-source rejection returns an error for every supplied value, distinguishing a nonblank text source from missing or invalid source material.

For enumerated file lists F_c, F_a, snapshot construction returns the ordered rows

$$\text{hashFiles}(F_c) +\!\!+ \text{hashFiles}(F_a), \quad \text{hashFiles}(F) = [(p, \text{sha256}(\text{bytes})) \mid (p, \text{bytes}) \in F].$$

Their path projection equals the complete input path projection. The snapshot identity hashes the retained root and this full ordered row list; missing authority directories preserve the original rejection.

> Source, proof and provenance: V3 snapshot_protected_tree; V3 audit_program_document; V3 reject_executable_prediction_source; SFT.Physics.HostilePackage.acceptedCertificateBound; SFT.Physics.HostilePackage.snapshotRetainsEveryFile; Timeline: Turn 372, line 3635.

315. Physical observation, reference pairing and complete comparisons

An observation retains a named phenomenon/reference pairing and its comparison record. The One construction contains one pairing. Appending any generated pairing preserves every earlier pairing and increases the support by exactly one. The native successor proof applies to arbitrary finite prior support.

The registered prediction sequence reads the sealed derivation premise, constructs the held physical-observation label, binds the premise to that prediction and emits the prediction. It has one registered input and one terminal emission; successful execution retains all four trace rows.

The observation-carrier experiment preserves both registered BIPM rows and their source locators. Comparison uses the released observation labels, retains every row in order and rejects an absent row. Acceptance requires every row to pass and the deliberately changed control to be rejected. The original 256-form grammar has one preserving form; all original forms, witnesses, controls and source boundaries remain present.

Twelve comparison cases and the complete experiment registration reproduce the pinned source operations. These results concern the original registered structural-label comparison; they do not treat a reported decimal inscription as a proof value. The complete original program-output comparison passes. The complete native blind-validator comparison preserves all original fields and hashes, including the 1,979,202-byte source snapshot, and closes the observation-carrier entry.

The observation carrier starts with one phenomenon/reference pair and extends by

$$P' = P +\!\!+ [(o, r)], \qquad \text{take}_{|P|}(P') = P, \qquad |P'| = |P| + 1.$$

The prediction program has exactly four instructions: read the registered premise, construct the physical-observation label, pair premise and prediction in a held register, then emit the prediction label. It has one input read and one final emission; successful execution has four trace rows.

For target row t and released label v_t, comparison retains target identity, source identity and source locator and sets

$$\mathrm{pass}(t) \iff p = v_t.$$

Recursive comparison preserves the complete ordered target-identity projection and rejects a missing release row. Acceptance is

$$\left(\bigwedge_{t \in T} p = v_t\right) \wedge p \neq \texttt{changed-external-observation}.$$

The second conjunct is the original deliberately changed-label control. These equations describe the registered structural-label experiment; the emitted label is the registered program label, while the premise is retained in its bound trace.

> Source, proof and provenance: V3 prediction_program_document; V3 validate; V3 validate; SFT.Physics.ObservationCarrier.successorRetainsPrior; SFT.Physics.ObservationPrediction.onlyRegisteredInput; SFT.Physics.ObservationPrediction.oneTerminalEmission; SFT.Physics.ObservationComparison.completeRowsRetained; Timeline: Turn 480, line 4102.

316. Source-bound observation validation and prediction sealing

The native observation validator retains the source sequence: validate the registered specification and source snapshot, construct the input and preregistered custody envelope, execute the capability-closed prediction, seal its output and trace, audit the protected state, release and verify custody, and compare every registered row. Its certificates retain the original executor, interpreter, program, input, target, comparison and seal identities. The final receipt binds the complete comparison record and deliberately changed control to the derivation seal and source snapshot.

A changed snapshot is rejected by the complete validator. The prediction seal binds the exact execution output, trace and preregistered envelope; it cannot be replaced. An empty held derivation premise is rejected. The audited program document equals the registered data-only document. These native results have zero axiom dependencies.

All 256 original observation-program lookups and their census, decision, closure, control and registration fields and hashes reproduce V3. The complete native blind-validator comparison includes the original 1,979,202 source bytes and matches every original validation field and hash.

The validator is the ordered composition

$$\mathrm{specification} \to \mathrm{sourceCheck} \to \mathrm{parse} \to \mathrm{inputs/vault} \to \mathrm{execute} \to \mathrm{seal} \to \mathrm{protectedAudit} \to \mathrm{release/verify} \to \mathrm{openTargets} \to \mathrm{compare}.$$

Each failing operation returns its original error before later stages execute. The seal's retained fields obey

$$(e, h_p, h_t, h_E) = (\mathrm{experimentId}, \mathrm{execution.outputHash}, \mathrm{execution.traceHash}, H(\mathrm{envelope}(\mathrm{derivationSeal}))).$$

A prior seal rejects replacement; an empty premise rejects input construction. The protected audit additionally requires the parsed program's canonical identity to equal the execution's program identity.

The final receipt hashes the experiment registration, derivation seal, prediction seal, source snapshot, complete comparison rows, changed-label control and execution trace identity. Final acceptance is the all-row/control predicate of the preceding section. The retained native pipeline definitions, proved component bindings and complete source-output comparison establish the registered observation-carrier result.

> Source, proof and provenance: V3 validate; V3 validate; SFT.Physics.ObservationValidation.resealRejected; SFT.Physics.ObservationValidation.sourceMismatchRejects; SFT.Physics.ObservationValidation.sealBindsExecution; Timeline: Turn 480, line 4102.

317. Half-One vacuum floor and complete finite oscillator spectrum

Complete binary support retains precisely the half-One under self-pairing to the One. The native proof also establishes uniqueness among arbitrary admitted positive parts, self-antipodality and the one-Fold return. At positive depth k, the complete spectrum contains 2^k parts (2j−1)/2^(k+1), for j=1,…,2^k. Each part lies in the positive unit domain. The ordered construction is injective; adjacent raw numerators differ by two, while successor depth doubles cardinality and doubles the common denominator. The paired successor numerators lie one half-spacing on either side of their retained parent coordinate.

The general native derivation and all four original operational witnesses have zero axiom dependencies. Fresh native evaluation agrees with all 510 ordered source values at depths 1 through 8. The complete 256-form grammar is retained and kernel checked. All 256 actual candidate lookups, original program fields, hashes and controls reproduce V3. The implementation-distinct V3 validator accepts the native output and rejects an omitted candidate, an additional survivor and a failed control. The Half-One vacuum-floor entry is complete.

The complete binary part support contains $1/2$ and One. Self-pairing to One selects only $1/2$, and the general positive-part theorem strengthens this to

$$\mathrm{selfJunction}(p) = \mathrm{One} \iff p = \tfrac{1}{2}.$$

The same part is its own complement and folds to One. For positive depth k, let $N = 2^k$. The complete spectrum is

$$p_i = \frac{2i+1}{2N}, \qquad 0 \le i < N.$$

Normalization retains the cross-product identity with this raw fraction. Since $0 < 2i+1 < 2N$, every level is in the positive unit domain. Equal normalized levels imply equal raw numerators and hence equal indices, proving injectivity.

The successor doubles support and denominator. Writing $n_i = 2i+1$, its paired numerators satisfy

$$n_{2i} = 2n_i - 1, \qquad n_{2i+1} = 2n_i + 1,$$

so the two successor values are

$$p'_{2i} = p_i - \frac{1}{4N}, \qquad p'_{2i+1} = p_i + \frac{1}{4N}.$$

Adjacent raw numerators differ by two, and complete cardinality is 2^k at every generated depth.

The source grammar tests carrier, dependency, relation, enumeration, minimality, measurement, record and extension. Its ordered coordinate product has $2^8 = 256$ forms. The half-One uniqueness and successor spectrum identities above supply the relation coordinate; the remaining coordinates retain its dependency

trace, full enumeration, omission controls, seal and source records. Write D_i for each ordered source coordinate list and c_i^* for its admitted value. The nested product construction and acceptance predicate give

$$G = \prod_{i=1}^{8} D_i, \qquad c \in G \iff \bigwedge_{i=1}^{8} (c_i \in D_i), \qquad |G| = \prod_{i=1}^{8} |D_i| = 2^8,$$

$$A(c) \iff \bigwedge_{i=1}^{8} (c_i = c_i^*) \iff c = c^*.$$

Each coordinate type has exactly its listed constructors. The native membership proof chooses the arbitrary candidate's constructor at each nested product stage; the acceptance proof then fixes every field to its admitted constructor. Thus the product omits no typed candidate and the acceptance conjunction has exactly one survivor. candidatesComplete, VacuumFloorGrammar.lean:L118; uniqueSurvivor, VacuumFloorGrammar.lean:L161.

Additional proof bindings for the preceding conclusions: Lean completeCardinality, VacuumSpectrum.lean:L68; Lean levelInjective, VacuumSpectrum.lean:L111; Lean successorLowerHalfSpacing, VacuumSpectrum.lean:L121; Lean successorUpperHalfSpacing, VacuumSpectrum.lean:L126.

Grammar source: V3 vacuum_axes, vacuum_lineage_laws_v1.py:L142; V3 VACUUM_FLOOR_SPEC, vacuum_lineage_laws_v1.py:L155.

> Source, proof and provenance: V3 vacuum_floor, vacuum_lineage_laws_v1.py:L36; V3 oscillator_levels, vacuum_lineage_laws_v1.py:L49; Lean arbitrarySelfPair, VacuumSpectrum.lean:L25; Lean floorReturns, VacuumSpectrum.lean:L30; Timeline: Turn 377, line 3670.

318. Exact cosmic-ratio continuations and outward source intervals

The generation volume 27 has least binary cover depth 5. The complete cover has 32 cells, giving baryon share 5/32, dark share 27/32 and leading dark-to-baryon ratio 27/5. The unit part over 31 has ordered first-return trace (2,4,8,16,1)/31; the source continuation gives 279/52. For the Hubble calibration construction, matter and vacuum shares are 1/3 and 2/3. The leading correction is 1/12 and the leading ratio 13/12. The depth-seven orbit over 127 has trace (2,4,8,16,32,64,1)/127; its refined correction is 257/3048 and its ratio 3305/3048. Native evaluation preserves all 19 fields of these two original constructions, including both complete ordered traces.

External source intervals are used only after structural forcing. The pinned Planck density rows give the complete outward dark-to-baryon interval [238/45,1210/223]. The pinned Planck and SH0ES route records give [720/679,3704/3345] for late divided by early. Both exact predictions lie within their respective interval. Native proofs retain all six original comparison witnesses. The source checks retain both uncertainties, source identities, the density confidence boundary, both Hubble route qualifications and matching units; no central-value-only tolerance replaces these intervals.

With forced binary and generator values $b = 2, c = 3$, the volume $c^c = 27$ has least cover $2^5 = 32$. The source ratios are

$$f_b = \frac{5}{32}, \quad f_d = \frac{27}{32}, \quad R_0 = \frac{27}{5}, \quad R_1 = \frac{27}{5 + 1/(32-1)} = \frac{279}{52}.$$

The unit part over 31 has the complete five-step return trace already displayed. For the Hubble construction,

$$f_m = \frac{1}{3}, \quad f_v = \frac{2}{3}, \quad T = 2^3 = 8, \quad \delta_0 = \frac{2/3}{8} = \frac{1}{12},$$

$$\delta_1 = \frac{2/3 + 1/(2^7 - 1)}{8} = \frac{257}{3048}, \qquad R_1^H = 1 + \delta_1 = \frac{3305}{3048}.$$

The depth-seven orbit supplies the original complete return over 127.

For post-seal numerator and denominator rows $n \pm u_n, d \pm u_d$, with $0 < u_n < n$ and $0 < u_d < d$, the source comparison uses

$$I = \left[\frac{n - u_n}{d + u_d}, \frac{n + u_n}{d - u_d}\right].$$

Substitution of the pinned density rows yields $[238/45, 1210/223]$; the pinned Hubble rows yield $[720/679, 3704/3345]$. Exact rational comparison accepts both leading and refined ratios in their respective intervals. The route, confidence, source-identity and unit guards precede this comparison.

> Source, proof and provenance: V3 dark_baryon_structure, cosmology_prior_value_laws.py:L22; V3 hubble_calibration_structure, hubble_calibration_law.py:L27; V3 hubble_ratio_interval, hubble_calibration_validation.py:L30; V3 DARK_BARYON_EMPIRICAL_SPEC, cosmology_prior_value_validation.py:L44; V3 density_ratio_interval, cosmology_prior_value_validation.py:L25; Lean darkFirstReturn, CosmicRatios.lean:L106; Lean hubbleFirstReturn, CosmicRatios.lean:L112; Lean originalHubbleWitnesses, CosmicIntervals.lean:L75; Lean originalDensityWitnesses, CosmicIntervals.lean:L76; Timeline: Turn 483, line 4144; Timeline: Turn 487, line 4237.

319. Dual quark cubics and first complete dyadic separation

The role-exchanged quark invariant triples are (1,1/8,1/383) and (1,1/12,1/3071). The native search follows every original dyadic-support successor from depth one and stops at the first complete three-interval census. Its exact down-type intervals are [(1/64,1/32),(7/64,1/8),(27/32,55/64)]; its exact up-type intervals are [(1/256,1/128),(11/128,23/256),(29/32,233/256)]. Kernel proofs establish these complete search outputs, three positive separated intervals in each channel, and correspondence of the search recursion to the original stopping rule. The representation retains exact polynomial invariants and rational enclosures.

The two invariant routes agree exactly:

$$s_u = \frac{1}{b(c+c)} = \frac{1}{b^2 c} = \frac{1}{12}, \qquad s_d = \frac{1}{b(c+1)} = \frac{1}{b^2 b} = \frac{1}{8} \quad (b = 2, c = 3).$$

The role-exchanged reaches use depth seven for the down product and seven plus three for the up product:

$$q_d = \frac{1}{3 \cdot 2^7 - 1} = \frac{1}{383}, \qquad q_u = \frac{1}{3 \cdot 2^{10} - 1} = \frac{1}{3071}.$$

Thus the cubic equations are $x^3 + s_d x = x^2 + q_d$ and $x^3 + s_u x = x^2 + q_u$. The executable compares their positive and counter sides exactly, retaining which is larger and the positive difference.

At dyadic depth k, it traverses all adjacent positive endpoints

$$\left(\frac{i+1}{2^k}, \frac{i+2}{2^k}\right), \qquad 0 \le i < 2^k - 1.$$

An exact lower endpoint yields a singleton interval; otherwise an exact upper endpoint or opposite endpoint hands retains the bracket. Search starts at depth one with the source fuel 32, returns at the first census of three brackets, and otherwise advances one depth. SearchRun records every failed count before the successful depth. Exact evaluation gives the displayed down and up lists at depths six and eight, with three positive separated intervals in each channel.

The quark-channel invariant entry is complete: both independent constructions yield 1/12 and 1/8, the cover guards give depths seven and five, and all 256 source forms, program fields, hashes and controls agree with fresh native execution. The original implementation-distinct validator accepts that output and rejects omitted enumeration, an additional survivor and a failed control.

The quark-cubic entry is complete. Fresh native execution reproduces every one of its 256 candidate lookups and all original fields and hashes. The V3 implementation-distinct validator accepts that output and rejects an omitted candidate, an additional survivor and a failed control. The exact cubic invariants and complete first dyadic root separation are retained as established above. Timeline: quark-cubic completion, Turn 379.

> Source, proof and provenance: V3 quark_channel_invariants, matter_flavour_laws_v1.py:L55; V3 quark_cubic_invariants, matter_flavour_laws_v1.py:L75; V3 quark_root_brackets, matter_flavour_laws_v1.py:L138; Lean exactCubics, QuarkInvariants.lean:L74; Lean exactChannels, QuarkInvariants.lean:L40; Lean searchCorrespondence, QuarkIsolation.lean:L55; Lean midpointExactRetained, QuarkIsolation.lean:L80; Timeline: Turn 378, line 3682.

320. Squared Planck/proton support and terminal transport

Write c=3 for the derived generator and b=2 for the derived binary count. The successor volume and its least cover are

$$V = c^{c+1} = 3^4 = 81$$

d = min{k ∈ ℕ : k>0 and 81 ≤ 2^k} = 7.
Indeed 2^6=64<81≤128=2^7; monotonicity gives 2^k≤64 for every positive k<7. The complete support is N=b^d=128. Its positive predecessor m satisfies m+1=N, hence m=127 uniquely. Consequently the squared hierarchy is H²=b^m=2^{127}, and the corresponding unsquared exponent is m/2=127/2. These are successive operations on the derived counts; no exponent is independently selected.

For terminal transport let α=3676744786/503846395469 and g_c=2/3. The removed share and retained complement are

$$g_c\alpha = 7353489572/1511539186407$$

$$t = 1 - g_c\alpha = 1504185696835/1511539186407$$

The inequalities 0<7353489572<1511539186407 imply 0<t<1. Multiplication of the bare squared carrier by this complement gives

$$H^2_{\text{terminal}} = 2^{127}t$$

$$= 25592393460381748800840516069019941843257249497088 0/1511539186407$$

Thus 0<H_terminal²<H². The terminal source entry is complete, including the independent colour-coupling dependency and its exact correspondence to the hierarchy share.

Generator-three successor volume is 81, whose least binary cover depth is 7. Complete support has 128 positions and its unique positive predecessor has 127 massive positions. The retained unsquared exponent is 127/2; the exact squared comparison carrier is 2^{127}. Native proofs retain the least-cover exclusion, predecessor uniqueness and both exact exponent records.

Terminal transport uses the already derived electromagnetic share 3676744786/503846395469 and the colour share 2/3. The retained positive One-complement multiplies the bare hierarchy once, producing 25592393460381748800840516069019941843257249497088 0/1511539186407. The native result preserves the strict-positive-retained-share guard and the lower squared hierarchy.

> Source, proof and provenance: V3 proton_planck_terminal_transport, precision_value_laws_v1.py:L106; V3 terminal_proton_planck_squared_ratio, precision_value_laws_v1.py:L116; V3 massive_state_count, lineage_particle_laws.py:L92; V3 proton_planck_squared_ratio, lineage_particle_laws.py:L97; Lean terminalTransportStrict, PlanckHierarchy.lean:L92; Lean squaredHierarchyExact, PlanckHierarchy.lean:L35; Timeline: Turn 381, line 3706; Timeline: Turn 496, line 4450.

321. Local vacuum support and separately typed cosmic magnitude

Volume 27 is first covered at depth 5. Two held labels per covering level give boundary-record depth 10, amplitude floor 1/1024 and self-composed energy floor 1/1048576. All four original support-scope and observable-composition candidates remain explicit; only the complete boundary with energy self-composition survives. The separate terminal cosmic share 11/16 gives normalized cosmological magnitude 33/16. Its scale transport is (33/16) times the held rate squared divided by the held limiting speed squared.

For complete support n, the native integer numerator identity establishes the sum of the first n odd numerators as n squared. The executable finite ledger retains every rational mode and accumulates its exact total before division by support. Fresh native/source comparisons agree on all 510 modes across depths 1 through 8, every total and mean, all four local candidates and all 27 original scale-covariance comparisons. These checks retain the distinction between the local energy floor and the cosmic density share.

The complete rational mode ledger is also verified at arbitrary positive depth: at depth k+1 its original ordered accumulator has total 2^(k+1)/2, mean 1/2 and a true closure result. This follows from normalization-preserving rational addition and the complete odd-numerator identity. For every admitted positive rate, speed and common rescaling carrier, exact scale transport is unchanged by multiplying both rate and speed by that carrier. The original nonpositive rate and speed exclusions are retained.

The terminal vacuum-density grammar retains all 4,096 forms from its twelve binary relation axes. Its complete candidate blocks, combined grammar, seven operational witnesses, controls and evidence modules compile; all 603 definitions and theorems in that component audit have zero axiom dependencies. The full source entry is complete: its source-bound completion receipt retains the complete program comparison, original independent controls and prerequisite closure.

The local construction follows the counted chain

$$3^3 = 27, \qquad 2^4 < 27 \le 2^5, \qquad d_{\mathrm{boundary}} = 2 \cdot 5 = 10, \qquad A_{\mathrm{local}} = 2^{-10}, \qquad E_{\mathrm{local}} = A_{\mathrm{local}}^2 = 2^{-20}.$$

In source order, the complete pair of binary choices gives the amplitude depth, observable depth and acceptance triples

$$(5,5,\mathrm{false}),\quad (5,10,\mathrm{false}),\quad (10,10,\mathrm{false}),\quad (10,20,\mathrm{true}).$$

For a supplied positive depth k, put N=2^k. The complete ordered modes are e_i=(2i+1)/(2N), for 0≤i<N. If S_N denotes their numerator sum, S_0=0 and S_(N+1)=S_N+2N+1. Substituting S_N=N² gives S_(N+1)=(N+1)²; finite induction therefore proves

$$\sum_{i=0}^{N-1} e_i = \frac{N^2}{2N} = \frac{N}{2}, \qquad \frac{1}{N}\sum_{i=0}^{N-1} e_i = \frac{1}{2}.$$

The actual normalized rational accumulator preserves common-denominator addition at each appended term, so this identity applies to the executed ledger as well as its integer numerator sum.

The separately supplied terminal cosmic share gives 3·(11/16)=33/16. For positive exact carriers H=n/d and v=m/e, scale transport is

$$\Lambda(H,v) = \frac{33H^2}{16v^2} = \frac{33n^2e^2}{16d^2m^2}.$$

For every positive common carrier s, cancelling s² from numerator and denominator gives Λ(sH,sv)=Λ(H,v). The positivity guards ensure the divisions and cancellation are admitted. These equations keep the local self-composed floor and cosmic scale transport as distinct source quantities.

The source grammar tests volume, cover, boundary, observable, floor, radiative, cosmic, geometry, typing, transport, measurement and extension. Its ordered coordinate product has $2^{12} = 4096$ forms. The counted local floor, complete mode ledger, separate cosmic share and squared-rate transport above supply the admitted values; treating the amplitude as energy or identifying the local floor with cosmic share fails their corresponding coordinates. The coordinate-membership and acceptance argument of Section 317 applies to this product, with the following native completeness and uniqueness proofs. candidatesComplete, VacuumDensityGrammarBase.lean:L162; uniqueSurvivor, VacuumDensityGrammar.lean:L781.

Grammar source: V3 SPEC, vacuum_density_scale_terminal_law_v1.py:L134.

> Source, proof and provenance: V3 boundary_record_depth, vacuum_density_scale_terminal_law_v1.py:L34; V3 local_vacuum_energy_floor, vacuum_density_scale_terminal_law_v1.py:L44; V3 finite_zero_point_ledger, vacuum_density_scale_terminal_law_v1.py:L49; V3 local_floor_candidates, vacuum_density_scale_terminal_law_v1.py:L75; V3 normalized_cosmological_constant, vacuum_density_scale_terminal_law_v1.py:L93; V3 cosmological_constant_scale_transport, vacuum_density_scale_terminal_law_v1.py:L99; Lean completeLocalCandidateDepths, VacuumDensity.lean:L101; Lean completeOddSum, VacuumDensity.lean:L67; Lean completeTotalNumerators, VacuumLedgerProofs.lean:L166; Lean everyPositiveLedgerCloses, VacuumLedgerProofs.lean:L227; Lean everyPositiveReferenceCovariant, VacuumScaleProofs.lean:L121; Lean nonpositiveRateRejected, VacuumScaleProofs.lean:L129; Timeline: Turn 515, line 5097.

322. Complete prime-sector construction

Using the least cover d=7 derived above, define

$$P = \{p \in \{2,3,4,5,6,7\} : \forall a \in \{2,\dots,p-1\},\ a \nmid p\}$$

The proper-divisor range for p=2 is empty. For p=3, 2 does not divide 3; for p=5, none of 2,3,4 divides 5; for p=7, none of 2,3,4,5,6 divides 7. The excluded counts have explicit divisors: 4=2·2 and 6=2·3. Therefore P={2,3,5,7}, with every count in the source range decided. For each admitted p, complete squared support and its positive predecessor give

$$M(p)+1=p^2, M(p)=p^2-1$$

$$g(p)=(p-1)/p$$

Evaluation in source order yields

$$(M(2),M(3),M(5),M(7))=(3,8,24,48)$$

$$(g(2),g(3),g(5),g(7))=(1/2,2/3,4/5,6/7)$$

Although 11 is prime, 11>7, so it is outside this derived sector support.

The independently generated successor-cover ceiling is seven. The native construction tests every proper divisor of every count from two through that ceiling and retains precisely (2,3,5,7). Applying the original positive-predecessor operation to each complete squared sector support gives mediator counts (3,8,24,48); applying it to each sector before exact division gives couplings (1/2,2/3,4/5,6/7). Eleven is prime and excluded by the same finite ceiling. Kernel-checked exact results and fresh evaluation agree with the pinned construction, including all 34 primality decisions from zero through 33. These results add no selected sector or fitted coupling.

Additional proof bindings for the preceding conclusions: Lean completeLadder, PrimeSectorLadder.lean:L43; Lean completeTests, PrimeSectorLadder.lean:L48; Lean exactCouplings, PrimeSectorLadder.lean:L74.

> Source, proof and provenance: V3 prime_sector_ladder, lineage_particle_laws.py:L68; Lean nextPrimeExcluded, PrimeSectorLadder.lean:L51; Lean booleanSectorRejected, PrimeSectorLadder.lean:L81; Timeline: Turn 381, line 3706.

323. Exact Fold return and binary multiplicative order

For q>0, let h_q(x) be the held representative of x modulo q: h_q(x)=x mod q when that remainder is nonzero, and h_q(x)=q otherwise. The executed recurrence is

$$r_0=p, r_{k+1}=h_q(2r_k)$$

Induction gives r_k ≡ p·2^k (mod q): the initial step is p≡p·1, and doubling the induction identity gives r_(k+1)≡2p·2^k=p·2^(k+1). For a reduced input, gcd(p,q)=1. Because held representatives are unique in {1,...,q},

$$r_k=p \iff p\cdot 2^k\equiv p \pmod{q}$$

$$\iff q \text{ divides } p(2^k-1)$$

$$\iff q \text{ divides } 2^k - 1$$

$$\iff 2^k \equiv 1 \pmod{q}$$

The cancellation uses gcd(p,q)=1. Taking the least positive k on both sides identifies first Fold return with first binary multiplicative order whenever that return exists. For odd q>1, existence follows from the finite-carrier argument below; the even-denominator transient is treated separately.

For every supplied positive reduced part p/q through the One, native repeated double-and-cast execution satisfies rank_k modulo q = p times 2^k modulo q. The complete-denominator representative is retained when a remainder vanishes. Reducedness permits cancellation of p: an exact return to the original held rank occurs if and only if 2^k modulo q equals 1 modulo q. The first-positive-return and first-positive-order predicates therefore have the same least index. These identities hold at arbitrary supplied finite depth; they do not use a bounded prime list. All 25 definitions and supporting theorems have zero Lean axiom dependencies.

Successful finite witness replays preserve the original order-loop and orbit-loop guard, update and stopping branches. The native correspondence proofs connect returned replay records to those source loop relations. An unfinished replay cannot pass a witness; its internal execution budget supplies no scientific value or survivor selector.

> Source, proof and provenance: V3 _fold_rank, lineage_laws.py:L21; V3 _order_two, lineage_laws.py:L26; V3 ORBIT_NUMBER_THEORY, lineage_laws.py:L147; Lean foldRankResidue, OrbitOrderIdentity.lean:L109; Lean iterateBounds, OrbitOrderIdentity.lean:L179; Timeline: Turn 381, line 3706.

324. Counted binary transient and complete orbit support

Let the supplied reduced denominator be 2^a times an odd positive core m. Native execution has reduced denominator m at depth a and at every later depth. Before that depth, at depth k with a=k+r and positive r, its denominator is exactly 2^r times m and is even. Thus a is exactly the first depth with an odd denominator. The proof includes the source complete-denominator representative, normalization and greatest-common-divisor cancellation; it concerns actual held-rank execution.

For any first binary order, distinct positions before that first return cannot collide. The complete indexed cycle has exactly that many positions, and every admitted reduced numerator repeats with that period. All four original finite witnesses also pass: the five displayed order values, every reduced rank through odd denominator 99, the ten prime-period divisibility cases and the denominator 73 tiling count. Fresh native execution reproduces all 2,006 original ordered traces, containing 57,842 retained positions. The current component batch contains 96 audited definitions and theorems with zero axiom dependencies.

Write D_k for the denominator of the actual normalized executed state, with q=2^a m and gcd(p,q)=1. Remainder preservation and normalization give

$$D_k = \frac{q}{\gcd(p2^k, q)} = \begin{cases} 2^{a-k}m, & k < a, \\ m, & k \geq a. \end{cases}$$

For k<a, removing the shared binary power leaves gcd(p,2^(a−k)m)=1, so the removed common divisor is exactly 2^k. At k=a+j, oddness of m makes gcd(2^j,m)=1, so the removed divisor remains exactly 2^a. The first case is even and the second odd. This proves the exact first odd-denominator depth a for the executed states, including the complete-denominator representative.

If ℓ is the first positive binary return, an equality r_i=r_j at 0≤i<j<ℓ cancels the reduced starting rank and the invertible power 2^i modulo the odd denominator, giving 2^(j−i)≡1. This contradicts minimality of ℓ. Consequently the full list [r_0,…,r_(ℓ−1)] has ℓ distinct positions and r_(k+ℓ)=r_k for every finite k.

> Source, proof and provenance: V3 _fold_rank, lineage_laws.py:L21; V3 _orbit, lineage_laws.py:L37; V3 ORBIT_NUMBER_THEORY, lineage_laws.py:L147; Lean exactBinaryTransient, OrbitBinaryDepth.lean:L163; Lean firstOddDenominatorAtDepth, OrbitBinaryDepth.lean:L177; Lean noEarlierCollision, OrbitCycleSupport.lean:L7; Lean positionsInjective, OrbitCycleSupport.lean:L29; Timeline: Turn 381, line 3706.

325. Complete cyclotomic counting and source-loop termination

For odd q>1, gcd(2,q)=1. If two powers at indices i<j have the same residue, cancellation of 2^i gives 2^(j−i)≡1 (mod q). The q+1 residues at indices 0,…,q cannot all be distinct in a carrier of q residues, so a positive return exists. Let ℓ be the least such index. For a reduced starting rank p, an equality r_i=r_j with 0≤i<j<ℓ would imply 2^(j−i)≡1, contradicting minimality. Hence its complete cycle C(p) has exactly ℓ members.

For any cycles C(p), C(s), suppose r_i(p)=r_j(s). Each cycle has period ℓ, so advancing within the cycle by the complementary index ℓ−j expresses s as an iterate of p. The same reasoning expresses p as an iterate of s. Thus C(p)=C(s). Every reduced rank lies in its own cycle, and distinct cycles are disjoint. If R_q= {r:1≤r<q, gcd(r,q)=1} and C is the number of cycles, successive removal of complete cycles therefore gives

$$|R_q| = \ell C$$

For prime q, each 1≤r<q is coprime to q, so |R_q|=q−1 and
q−1 = ℓ C, hence ℓ divides q−1.
This is a finite partition derivation for each admitted denominator, independent of the finite witness table.

For every admitted odd denominator above the One, the finite carrier forces a first binary return. If no return occurred through the carrier bound, one more executed position than there are held ranks would have to remain distinct, contradicting the exact finite count. The original order loop therefore terminates, and its returned count is the first binary multiplicative order. For every reduced numerator, the original orbit loop returns a complete ordered trace of exactly this length.

The executed cycles preserve reducedness. Two cycles that share a rank have identical members; every reduced rank belongs to its own cycle. Removing complete cycles successively from the full reduced-residue support proves that the residue count equals orbit length times orbit count. For a prime denominator, every positive rank below the prime is reduced, so the period divides one less than the prime. These are arbitrary supplied finite-denominator results, not conclusions extrapolated from the witness census.

The orbit loop also terminates for positive held parts with even denominators: each fresh step adds a distinct held rank, and the finite carrier bounds the number of additions before repetition. Combined with the counted binary-depth result, this retains the transient prefix before the odd recurrent core. The new derivation batch has 71 audited definitions and theorems with zero axiom dependencies; the original 256-form program batch has a further 108.

> Source, proof and provenance: V3 _order_two, lineage_laws.py:L26; V3 _orbit, lineage_laws.py:L37; V3 ORBIT_NUMBER_THEORY, lineage_laws.py:L147; Lean overlappingCyclesCoincide, OrbitCyclotomic.lean:L60; Lean iterateReduced, OrbitCyclotomic.lean:L7; Lean countClosedCyclesAux, OrbitTilingCount.lean:L165; Lean reducedResiduesClosed, OrbitTilingCount.lean:L275; Lean noCollisionBeforeReturn, OrbitOrderExistence.lean:L54; Lean firstOrderExists, OrbitOrderExistence.lean:L77; Lean orbitReplayCompletes, OrbitLoopCompletion.lean:L73; Lean originalOrbitOrderIdentity, OrbitLoopCompletion.lean:L99; Timeline: Turn 381, line 3706.

326. Completed orbit, prime-sector and base hierarchy verification

The orbit-number-theory entry is complete, including the general return/order identity, disjoint cyclotomic counting, prime-period divisibility, exact binary transient and original source-loop termination. Every one of its 256 original program cases and all original records and hashes reproduce V3. Its implementation-distinct validator accepts the complete native product and rejects each retained adverse variant.

With that dependency closed, the prime-sector ladder and base proton/Planck hierarchy entries are also complete. The generated ceiling seven admits exactly sectors two, three, five and seven. Their mediator counts are three, eight, twenty-four and forty-eight, with couplings one-half, two-thirds, four-fifths and six-sevenths. The successor volume 81 has least binary cover seven; its support 128 has unique massive predecessor 127. The squared hierarchy is 2^127, with the unsquared exponent retained exactly as 127/2. Both entries reproduce their complete 256-case programs and pass their original independent validators.

The separate terminal hierarchy has its own completed source-bound transport and verification, given in Sections 320 and 440.

The source grammar tests domain, reduction, transition, period, partition, transient, generality and addition. Its ordered coordinate product has $2^8 = 256$ forms. The return/order identity, counted binary transient and disjoint-cycle partition of Sections 323–325 supply the period, transient and partition criteria at every admitted denominator. The coordinate-membership and acceptance argument of Section 317 applies to this product, with the following native completeness and uniqueness proofs. candidatesComplete, OrbitGrammarBase.lean:L118; uniqueSurvivor, OrbitGrammar.lean:L61.

Grammar source: V3 ORBIT_NUMBER_THEORY, lineage_laws.py:L147.

> Source, proof and provenance: V3 ORBIT_NUMBER_THEORY, lineage_laws.py:L147; Lean orbitReplayCompletes, OrbitLoopCompletion.lean:L73; Lean originalOrbitOrderIdentity, OrbitLoopCompletion.lean:L99; Lean nextPrimeExcluded, PrimeSectorLadder.lean:L51; Lean booleanSectorRejected, PrimeSectorLadder.lean:L81; Timeline: Turn 381, line 3706.

327. Exact positive-ratio composition

Let N(a,b) denote division of a positive numerator and denominator by their greatest common divisor. Its normalized pair (u,v) satisfies av=ub. For an arbitrary reduced positive ratio x=(n,d), define its reciprocal y=N(d,n) and composition x·y=N(ny_n,dy_d). Value preservation for y gives dy_d=y_n n=ny_n. The composite therefore has equal positive raw numerator and denominator, and normalization gives (1,1). Every admitted positive ratio composed with its reciprocal returns the One, including ratios above the One.

For positive counts a,b,c, the raw product of the views a/b and b/c is N(ab,bc)=(u,v). Value preservation gives abv=ubc. Associativity and commutativity place the common positive factor b on the same side; applying finite division by b and the identity (bk)/b=k gives av=uc. Thus this composite represents a/c exactly. For any positive m, the raw product of m/1 and 1/m is (m·1,1·m); commutativity makes the sides equal, and normalization yields (1,1). This reconstructs the One from m corresponding equal parts.

These native component results have thirteen audited definitions and theorems with zero axiom dependencies. The full source entry additionally includes shorter separation, complement, local transport and finite-family cycle composition.

The same telescoping law holds for actual composition after normalizing both input views. Let x=N(a,b), y=N(c,d), and z=compose(x,y). Value preservation gives ax_d=x_n b, cy_d=y_n d, and (x_n y_n)z_d=z_n(x_d y_d). Put e=x_d y_d>0. Multiplying and rearranging the first two identities gives (ac)e=(x_n y_n)(bd). Multiplication by z_d and substitution of the third identity therefore yield e(ac z_d)=e(z_n

bd). Exact positive cancellation gives ac z_d=z_n bd. Specializing the second view to N(b,c) gives ab z_d=z_n bc; cancellation of b gives a z_d=z_n c. Thus normalization of either input and the composite preserves the telescoping relation itself.

> Source, proof and provenance: V3 EXACT_RELATIONS, lineage_laws.py:L101; Lean telescopeNormalized, ExactRatioRelations.lean:L86; Lean reciprocalReturnsOne, ExactRatioRelations.lean:L32; Timeline: Turn 395, line 3760.

328. Unique positive holding complement

For a generated count m>1, write m=k+2 and define h=N(k+1,k+2), the exact (m−1)-of-m part. A positive ratio x=n/d is the holding complement of the equal part 1/m precisely when x+1/m=1. Clearing the positive denominator dm gives the entirely positive-side equation

$$nm + d = dm$$

Expanding dm=d(k+1)+d and removing the same terminal d from both sides forces nm=(k+1)d. Normalization of h gives h_n m=(k+1)h_d, so h satisfies the holding equation. If any other positive ratio x satisfies it, multiply its forced equality by h_d and the equality for h by d. Rearranging gives m(nh_d)=m(h_n d). Cancellation by positive m yields nh_d=h_n d, establishing uniqueness of the exact rational value. At m=2, h=N(1,2)=(1,2), the half-One witness. The condition m>1 is precisely the positive-complement domain: at m=1 the holding equation would require n+d=d, which admits no positive numerator. These seven native definitions and theorems have zero axiom dependencies; the source entry's other relation and cycle requirements remain separate.

> Source, proof and provenance: V3 EXACT_RELATIONS, lineage_laws.py:L101; Lean holdingForcesValue, HoldingComplement.lean:L23; Lean holdingUniqueValue, HoldingComplement.lean:L35; Timeline: Turn 395, line 3760.

329. Exact-relations operational witnesses and complete grammar

The five original witnesses evaluate the same exact operations as the source. Reciprocal composition gives (3/2)(2/3)=6/6=1. Equal-part reconstruction gives 8(1/8)=8/8=1. Relative views give (5/3)(3/2)=15/6=5/2. The binary holding complement is (2−1)/2=1/2. For the supplied periods 2,3,4, the first common return is 12: it is divisible by all three, and the only positive multiples of 4 below 12 are 4 and 8, neither divisible by 3. Equivalently lcm(lcm(2,3),4)=lcm(6,4)=12. These are operational witnesses; the finite-family return theorem concerns every supplied positive finite family separately.

The registered grammar has eight binary axes: provenance, relation, orientation, threshold, transport, composition, generality and addition. Its ordered Cartesian product therefore has 2^8=256 forms. A form survives exactly when every coordinate is its admitted value; hence the all-admitted coordinate exists and is unique. Replacing any admitted coordinate by its rejected alternative falsifies that conjunct. This enumeration retains all forms, while the underlying relation derivations supply the scientific statements represented by those criteria.

Explicitly, for the eight-coordinate source product G and its admitted coordinate a,

$$G = \prod_{j=1}^{8} \{a_j, b_j\}, \qquad |G| = 256, \qquad \mathrm{accept}(x) \iff \bigwedge_{j=1}^{8} (x_j = a_j).$$

Thus the survivor list is [a]; changing coordinate j to b_j makes its jth acceptance conjunct false. This statement describes the complete registered grammar alongside, rather than in place of, the ratio and return derivations.

The retained artifact control tests both the survivor count and distinct candidate identifiers:

$$|\,\mathrm{survivors}(G)| = 1 \quad \wedge \quad |\,\mathrm{eraseDups}(\mathrm{IDs}(G))| = |G|.$$

The original boundary control is the conjunction of a nonempty exclusion list and all five passing operational witnesses. The mathematical content of the exclusions is supplied by the typed domains and the individual derivations; the control record retains its exact executable predicate.

> Source, proof and provenance: V3 EXACT_RELATIONS, lineage_laws.py:L101; Lean reciprocalReturnsOne, ExactRatioRelations.lean:L32; Lean telescopeNormalized, ExactRatioRelations.lean:L86; Timeline: Turn 395, line 3760.

330. Positive local separation before casting

After a common finite refinement, let the held ordered positions be l/d and u/d, where 0<l<u≤d. Their positive directed gap is N(u−l,d). If this normalized pair is (g_n,g_d), then (u−l)g_d=g_n d, preserving the exact common-refinement value. The uncast Fold step doubles both held numerators. Its gap numerator is therefore

$$(u+u)-(l+l) = 2u-2l = 2(u-l) = (u-l)+(u-l)$$

Normalizing the two equal positive numerators against the same denominator gives identical positive Fraction objects. For the resulting transported pair (t_n,t_d),

$$2(u-l)t_d = t_n d$$

Thus the uncast local gap transports by the generated binary factor. This statement concerns the local gap before whole-turn casting; it does not identify every directed gap with the shorter circular separation or omit a change of cast branch. The component's seven audited declarations have no axiom dependencies.

For two positions on the same Fold branch, the actual casting operation preserves this doubled gap. If 2u≤d, then also 2l≤d; both cast numerators are unchanged, and their difference is 2(u−l). If d<2l, then d<2u and both doubled numerators are at most 2d. Casting therefore removes exactly d from each. The retained difference is (2u−d)−(2l−d)=2u−2l=2(u−l). The shared-removal identity follows by restoring d to both retained numerators and cancelling the common added term. Thus both same-branch cases are connected to actual complete-One removal, not only to uncast arithmetic. Branch-crossing shorter-separation classification is a separate part of the relation derivation. The local component now has eleven audited declarations without axiom dependencies.

> Source, proof and provenance: V3 EXACT_RELATIONS, lineage_laws.py:L101; Lean upperBranchGap, LocalSeparation.lean:L69; Lean lowerBranchGap, LocalSeparation.lean:L60; Timeline: Turn 395, line 3760.

331. Least common return of every positive finite cycle family

For supplied positive periods a,b, let g=gcd(a,b). The actual native least-common-multiple operation is L=ab/g. Since g divides both a and b, exact division yields L=a(b/g)=b(a/g). Thus a divides L, b divides L, and L>0. Also gL=ab.

To prove minimality, let c>0 be any shared return, so a divides c and b divides c. Then ab divides both ac and bc. The constructive greatest-common-divisor property gives ab dividing gcd(ac,bc). Positive scaling gives gcd(ac,bc)=c·gcd(a,b)=cg. Therefore cg=abk=gLk for a finite count k. Cancelling the positive g gives c=Lk, so L divides every positive shared return and L≤c. This establishes the first common return, not merely one common multiple.

For a nonempty supplied finite family, retain its first period p and the remaining finite list P. Define J(p,[])=p and J(p,q::P)=J(lcm(p,q),P). Induction proves positivity and the exact characterization, for every positive n,
J(p,P) divides n ⇔ p divides n and every period in P divides n.
The empty-tail case is the single cycle. At a successor, divisibility by lcm(p,q) is equivalent to simultaneous divisibility by p and q, by the two-period result; the induction hypothesis handles the remaining list. Applying the characterization to n=J(p,P) shows that every supplied period returns there. Applying it to any other positive shared return n shows J(p,P) divides n and hence J(p,P)≤n. The original witness is J(2,[3,4])=12. All sixteen native definitions and theorems in this derivation have zero axiom dependencies; the construction uses no completed infinite family or fixed period table.

> Source, proof and provenance: V3 EXACT_RELATIONS, lineage_laws.py:L101; Lean jointCharacterization, LeastCommonReturn.lean:L90; Lean originalJointWitness, LeastCommonReturn.lean:L115; Timeline: Turn 395, line 3760.

332. Held orientation and exact shorter-part selection

For two supplied generated positive parts a=n_a/d_a and b=n_b/d_b, compare the positive cross-products n_a d_b and n_b d_a. Define s=a when n_a d_b≤n_b d_a, and s=b otherwise. In the first case s≤b and s=a; in the second case n_b d_a<n_a d_b, so s≤a and s=b. Hence s belongs to the supplied pair, remains positive and is no longer than either member. The held side is retained as a separate label. Changing that label changes neither the cross-products nor s, so orientation introduces no signed magnitude. In the tie case n_a d_b=n_b d_a, either presentation has the same exact ratio. The native shorter-part component proves these statements without axioms. The combined exact-relations component audit covers 26 modules and 179 declarations, all with zero axiom dependencies. Its complete-program comparison and independent validation are recorded in Section 333.

> Source, proof and provenance: V3 EXACT_RELATIONS, lineage_laws.py:L101; Lean tiePreservesExactValue, HeldShorterPart.lean:L49; Lean selectedFromGenerated, HeldShorterPart.lean:L22; Timeline: Turn 395, line 3760.

333. Completed exact-relations verification

The exact-relations entry is complete. Its general positive-ratio, reciprocal, telescoping, holding-complement, held shorter-part, uncast local transport and finite-family least-common-return results accompany the complete original 256-form grammar. Fresh Lean execution reproduces every original program field and hash. The original independent validator accepts this output and rejects omitted enumeration, an additional survivor and a failed control. The entry-specific audit additionally includes its declared types and covers 179 declarations.

> Source, proof and provenance: V3 EXACT_RELATIONS, lineage_laws.py:L101; Lean reciprocalValue, ExactRatioRelations.lean:L22; Lean reciprocalReturnsOne, ExactRatioRelations.lean:L32; Lean originalJointWitness, LeastCommonReturn.lean:L115; Lean leftFactor, LeastCommonReturn.lean:L8; Timeline: Turn 395, line 3760.

334. Positive finite refinement and exact counted return

Define r_0=One and r_(n+1)=lowerPreimage(r_n), using the existing exact normalized preimage operation. It halves the represented positive part, so induction gives r_n=N(1,2^n): the base is N(1,1), and the successor doubles the whole denominator, 2^n+2^n=2^(n+1). Every r_n remains positive and admitted. The native Fold identity F(lowerPreimage(x))=x gives F(r_(n+1))=r_n. Repeated application therefore gives F^n(r_n)=One. More generally F^k(r_(j+k))=r_j for all supplied finite j,k.

Every successor rung lies on or below half-One: its numerator doubled is at most its denominator. It cannot equal One, since that equality would require 2≤1. Hence F^k(r_((j+1)+k))=r_(j+1)≠One. Together with the counted return this excludes every earlier positive or zero step before the designated depth. Each supplied

finite depth n has the lawful larger depth n+1; the proof does not install an ungenerated terminal depth. These eleven native declarations have zero axiom dependencies. The source entry's separate finite-difference, nested-bound and object-boundary results remain part of its full verification.

> Source, proof and provenance: V3 LIMIT_CONTINUUM, lineage_laws.py:L192; Lean dyadicRepresentation, FiniteRefinement.lean:L55; Lean countedReturn, FiniteRefinement.lean:L18; Timeline: Turn 408, line 3808.

335. Exact finite-difference identities

Put q=2^k and h=1/q at a positive generated depth k. For the centered quadratic construction, clearing the denominator q² gives

$$(q+1)^2+(q-1)^2=2q^2+2$$

Thus (1+h)²+(1−h)²=2+2h², and the centered second difference divided by h² is exactly 2. The native proof establishes the denominator identity for q=m+1 by distributivity and exact addition; it is not inferred from the finite witness list.

For the forward cubic construction, clearing q³ gives

$$(q+2)^3+q^3=2(q+1)^3+6q+6$$

Equivalently (1+2h)³+1=2(1+h)³+6h²+6h³. Its forward second difference divided by h² is therefore 6+6h, with exact positive error E_k=6/2^k. The native error carrier is the normalized positive fraction N(6,2^k); its numerator and denominator satisfy 6E_d=E_n2^k. The denominator successor is 2^(k+1)=2^k+2^k. These seven audited component declarations have zero axiom dependencies.

For normalized E_k=(a_n,a_d) and E_(k+1)=(b_n,b_d), value preservation gives 6a_d=a_n2^k and 6b_d=b_n(2^k+2^k). Multiplying these identities by the opposite denominators and cancelling the positive 2^k yields (b_n+b_n)a_d=a_n b_d. Therefore 2E_(k+1)=E_k exactly. Since b_n a_d>0, it follows that b_n a_d<a_n b_d, proving strict decrease in the actual normalized representation.

There is also a finite certificate for every supplied positive rational tolerance a/d. Induction gives n+1≤2^n: at the successor, n+2≤2(n+1)≤2·2^n. Choose the finite depth K=6d. Then 2^K≥K+1>6d, so E_K=6/2^K<1/d≤a/d. The native proof multiplies through by the positive denominators and cancels only positive factors, establishing E_n d<a E_d for the normalized error pair. K is a sufficient generated certificate depth computed from the supplied tolerance; it is not a fitted scientific parameter or a claim of optimal depth. Thus the error can be certified below every supplied positive rational tolerance without introducing zero as an attained error or an ungenerated limit value. The finite-difference component now has eleven audited declarations with zero axiom dependencies.

> Source, proof and provenance: V3 LIMIT_CONTINUUM, lineage_laws.py:L192; Lean finiteErrorCertificate, FiniteDifferenceBounds.lean:L76; Lean quadraticCentered, FiniteDifferenceBounds.lean:L12; Timeline: Turn 408, line 3808.

336. Complete finite dyadic closure

At depth n, express the ordered terms 1/2,…,1/2^n at the common denominator 2^n. The numerator list is generated by W₀=[] and W_(n+1)=map(w↦w+w,W_n)++[1]. Doubling each retained numerator compensates for doubling the denominator; appending 1 supplies the new final rung. Let T_n be the finite sum of this complete list. Finite-list induction gives T_(n+1)=2T_n+1. The boundary rung has numerator 1 at the same denominator.

The base has T₀+1=1=2^0. If T_n+1=2^n, then T_(n+1)+1=2T_n+2=2(T_n+1)=2^(n+1). Thus the complete finite ledger satisfies T_n+1=2^n at every supplied depth. Its exact normalized ratio N(T_n+1,2^n) is therefore One. For n=5 the ordered numerators are [16,8,4,2,1], summing to 31; adding the boundary numerator 1 gives 32/32=One. Equivalently 1/2+1/4+1/8+1/16+1/32+1/32=1. The ten native declarations for the ordered ledger, accumulation and normalized closure have zero axiom dependencies.

> Source, proof and provenance: V3 LIMIT_CONTINUUM, lineage_laws.py:L192; Lean originalClosure, DyadicClosure.lean:L56; Lean completeLedger, DyadicClosure.lean:L37; Timeline: Turn 408, line 3808.

337. Exact rational execution of the finite-refinement witnesses

The normalized rational operations evaluate the source expressions at each original supplied depth. For h=1/2^k, the centered quadratic quotient is ((1+h)²−2+(1−h)²)/h²=2. The forward cubic quotient is ((1+2h)³−2(1+h)³+1)/h²=6+6h, so its error is E_k=6/2^k and 2E_(k+1)=E_k. The complete five-term dyadic sum with its boundary term is (16+8+4+2+1+1)/32=1. These calculations use exact normalized signed rational arithmetic for intermediate subtraction while retaining positive generated rungs and error bounds.

Fresh native execution reproduces 57 exact rational rows: fourteen rungs, ten quadratic quotients, nine cubic quotients, nine errors, nine doubled successor errors, five ordered ledger terms and the complete ledger. The four original source witness expressions also evaluate to true. These finite execution comparisons accompany the arbitrary-depth derivations in Sections 334–336; they do not replace them. The component audit covers 171 definitions and theorems without axiom dependencies.

> Source, proof and provenance: V3 LIMIT_CONTINUUM, lineage_laws.py:L192; Lean closurePassed, LimitComputations.lean:L57; Lean finiteRungsPassed, LimitComputations.lean:L54; Timeline: Turn 408, line 3808.

338. Complete finite traces and the typed object boundary

Let r_n be the generated rung from Section 334. Define the ordered trace T₀=[r₀] and T_(n+1)=T_n++[r_(n+1)]. At the base, |T₀|=1. Since appending one record adds one to list length, induction gives |T_n|=n+1. The append construction retains every prior record and contains its new terminal rung.

Membership in T_(n+1) splits into membership in T_n or equality with r_(n+1). Induction therefore gives p∈T_n⇒∃k≤n, p=r_k. Conversely, if k≤n+1, either k=n+1, in which case the appended terminal record supplies r_k, or k≤n, in which case induction and retention supply it. Thus

$$p \in T_n \iff \exists k \le n, p = r_k$$

Every record is a positive exact Part, so p∈T_n implies p_numerator>0. Moreover |T_n|=n+1<n+2=|T_(n+1)|: every reached finite trace has a strictly longer generated successor. This is an arbitrary supplied finite-depth construction, with no largest-stage object.

The original typed grammar's acceptance equivalence fixes all eight admitted coordinates. Projecting that equality to its object, continuum and cardinality coordinates gives finite-stage, unbounded-process-boundary and finite-bound-count respectively. A candidate with completed-infinity, ambient-real-line or completed-comparison in those coordinates is rejected by the corresponding proved coordinate exclusion. The finite traces above provide the explicit generated-stage construction, while these exclusions state the source object-language boundary. The ten declarations in this component have zero axiom dependencies.

The trace induction and typed acceptance can be stated together without identifying the trace with a completed infinite collection:

$$|T_n| = n+1, \qquad p \in T_n \iff \exists k \le n,\ p = r_k, \qquad T_{n+1} = T_n +\!\!+ [r_{n+1}].$$

The list on the right is generated for each supplied finite n. If A(c) is the grammar's acceptance predicate, its coordinate characterization gives

$$A(c) \Longrightarrow \left(c_{\text{object}} = \text{finiteStage}\right) \wedge \left(c_{\text{continuum}} = \text{unboundedProcessBoundary}\right) \wedge \left(c_{\text{cardinality}} = \text{finiteBoundCount}\right).$$

Replacing any of these coordinates by its excluded source alternative contradicts the corresponding equality. The original boundary-control Boolean additionally checks that the exclusion list is nonempty and all four source witnesses pass; that Boolean and the coordinate-exclusion theorems retain their separate roles.

> Source, proof and provenance: V3 LIMIT_CONTINUUM, lineage_laws.py:L192; Lean acceptedObjectBoundary, FiniteRefinementTrace.lean:L71; Lean traceCount, FiniteRefinementTrace.lean:L11; Timeline: Turn 408, line 3808.

339. Completed finite-refinement and continuum-boundary verification

The finite-refinement entry is complete. Its positive dyadic rungs, exact counted returns, finite-difference identities, shrinking positive error certificates, complete finite ledger and finite trace construction are derived in Sections 334–338. For every supplied finite n the trace has n+1 records and a lawful next trace; the source's typed grammar excludes completed-infinity, ambient-real-line and completed-cardinality objects. The complete original 256-form program reproduces all V3 records and hashes. Its original independent validator accepts the fresh output and rejects omitted enumeration, an extra survivor and a failed control.

This entry adds 98 theorems and 171 audited definitions and theorems across 26 modules; its 181-declaration component audit also includes declared types. The full finite witness calculation reproduces 57 exact rational rows and the four original witness expressions.

The source grammar tests object, successor, refinement, convergence, continuum, cardinality, generality and addition. Its ordered coordinate product has $2^8 = 256$ forms. The base and successor constructions in Sections 334–338 supply finite stages, always-next extension, positive rungs, nested bounds and finite counts; these are the retained object-language constructors. The coordinate-membership and acceptance argument of Section 317 applies to this product, with the following native completeness and uniqueness proofs. candidatesComplete, LimitContinuumGrammarBase.lean:L118; uniqueSurvivor, LimitContinuumGrammar.lean:L61.

Grammar source: V3 LIMIT_CONTINUUM, lineage_laws.py:L192.

> Source, proof and provenance: V3 LIMIT_CONTINUUM, lineage_laws.py:L192; Lean dyadicRepresentation, FiniteRefinement.lean:L55; Lean countedReturn, FiniteRefinement.lean:L18; Lean acceptedObjectBoundary, FiniteRefinementTrace.lean:L71; Lean everyTraceMemberPositive, FiniteRefinementTrace.lean:L65; Timeline: Turn 408, line 3808.

340. Terminal electroweak support from the promotion object

The admitted alpha promotion object contains the ordered rungs [125,175,245,343]. Its length is four, equal to the successor of the generated direction count three. The terminal binary support therefore has 2^4=16 words; the complete Fold word constructor independently supplies all sixteen records at that depth. The native support result equals the length of this complete generated list.

Holding the three generator directions uses repeated positive predecessor steps, 16→15→14→13. For arbitrary finite host counts r,t, the same recursion satisfies predecessorSteps(r+t+1,t)=r+1: the base performs no step, and the successor removes one predecessor and applies the induction hypothesis. At the admitted counts this fixes the running level thirteen. The binary predecessor of three is two, so the charged axis count is 13+2=15 and the neutral axis count is 13+1=14. The four neutral pair channels

come from $2^2=4$. Consequently the charged squared support is $15^2=225$, the neutral support is $4\cdot14^2=784$, and their combined count is $225+784=1009$. The complete support successor fixes the return divisor $16+1=17$.

These sixteen audited definitions and theorems have zero axiom dependencies. They establish the support and channel counts used by the terminal rational-share construction; the separate full-entry validation includes that rational construction and its complete candidate program.

> Source, proof and provenance: V3 terminal_promotion_count, precision_value_laws_v1.py:L54; V3 terminal_binary_support, precision_value_laws_v1.py:L61; V3 electroweak_running_level, precision_value_laws_v1.py:L65; V3 terminal_return_divisor, precision_value_laws_v1.py:L84; Lean countsAsSquares, ElectroweakTerminalSupport.lean:L83; Lean channelCountsExact, ElectroweakTerminalSupport.lean:L69; Timeline: Turn 488, line 4245.

341. Exact terminal electroweak share and positive complement

At an admitted positive running level l, the source constructs the charged and neutral supports as $(l+2)^2$ and $4(l+1)^2$. Normalizing their positive ratio gives $R(l)=(l+2)^2/[4(l+1)^2+(l+2)^2]$. The derived level $l=13$ therefore gives $R(13)=225/(784+225)=225/1009$. Boolean, zero and negative level inputs retain their original rejection; the level is supplied by the support derivation, not selected from measurement.

The already derived inverse fine-structure ratio is 503846395469/3676744786. Its exact reciprocal is alpha=3676744786/503846395469, a strict positive part of the One. The sole self-return crosses the derived divisor seventeen once, giving alpha/17=3676744786/8565388722973. The terminal share is thus

$$S = 225/1009 + 3676744786/8565388722973$$

$$= (225 \cdot 8565388722973 + 3676744786 \cdot 1009)/(1009 \cdot 8565388722973)$$

$$= 1930922298157999/8642477221479757$$

Its numerator is positive and smaller than its denominator. Taking this positive part from the One gives

$$C = (8642477221479757 - 1930922298157999)/8642477221479757$$

$$= 6711554923321758/8642477221479757$$

The exact normalized sum satisfies S+C=8642477221479757/8642477221479757=1. Native evaluation uses the normalized positive-fraction addition, reciprocal, division and positive-take operations; fifteen component declarations have zero axiom dependencies. These rational results accompany the complete source grammar and validation required for full entry registration.

Additional proof bindings for the preceding conclusions: Lean runningShareExact, ElectroweakTerminalRatio.lean:L54; Lean terminalAlphaExact, ElectroweakTerminalRatio.lean:L56; Lean sinSquaredExact, ElectroweakTerminalRatio.lean:L60; Lean completeShareOne, ElectroweakTerminalRatio.lean:L69.

> Source, proof and provenance: V3 terminal_alpha, precision_value_laws_v1.py:L34; V3 electroweak_running_share, precision_value_laws_v1.py:L71; V3 terminal_electroweak_sin_squared, precision_value_laws_v1.py:L88; V3 terminal_electroweak_cos_squared, precision_value_laws_v1.py:L99; Lean negativeRejected,

> ElectroweakTerminalRatio.lean:L77; Lean booleanRejected, ElectroweakTerminalRatio.lean:L71; Timeline: Turn 488, line 4245.

342. Positive-take domain and arbitrary positive running levels

The count-taking operation first admits an integer count c≥2 and an integer take depth t≥1, excluding Boolean inputs. It then performs the original repeated positive predecessor operation. For every supplied finite r,t, the native recursion proves take(r+t+2,t+1)=r+1. The actual terminal inputs therefore give take(16,3)=13. The guard does not silently require t<c: if a further predecessor is requested after reaching the One, the original predecessor error propagates. Thus initial domain admission and successful completion of the repeated take remain distinct source conditions.

For every positive integer level l=n+1, evaluation of the generated binary and pair counts yields the exact normalized fraction

$$R(n+1) = N((n+3)^2, 4(n+2)^2 + (n+3)^2)$$

The charged numerator is positive because n+3>0, and the denominator is positive because it contains that positive charged term. This identity is proved uniformly in n, while the support construction fixes the terminal instance n=12. Thirteen audited declarations establish the original count-taking domain, its recursion, terminal instance and arbitrary positive-level running-share identity without axiom dependencies.

Fresh execution preserves the source function domain over all 625 pairs of counts drawn from integers −2 through 20 and both Boolean values, including both initial guard failures and predecessor exhaustion. It also reproduces all 25 running-share results on that input set and nine derived support/ratio outputs. Each output, including the exact error text, matches execution of the unchanged pinned V3 functions and their original dependency functions. These comparisons supplement the arbitrary-positive-level and predecessor-induction proofs above.

> Source, proof and provenance: V3 positive_take, precision_value_laws_v1.py:L43; V3 electroweak_running_share, precision_value_laws_v1.py:L71; Lean runningShareAllPositive, ElectroweakTerminalDomain.lean:L57; Lean oneCountRejected, ElectroweakTerminalDomain.lean:L43; Timeline: Turn 488, line 4245.

343. Complete labelled electroweak channel supports

Generate the finite positive position list P_0=[] and P_(n+1)=P_n++[n+1]. Induction gives |P_n|=n and x∈P_n iff 0<x≤n. Every previous position is at most n, so n+1 is fresh; induction therefore proves that P_n is duplicate-free. These are labels for the counted support positions.

For finite lists X,Y, generate their complete product by concatenating [(x,y):y∈Y] for each x∈X in order. A pair belongs to this list exactly when its two coordinates belong to X and Y. Prepending each new first-coordinate block adds |Y| records, giving |X×Y|=|X|·|Y| by induction on X. Within a block, pair equality implies equality of second coordinates; between blocks, it implies equality of first coordinates. Thus distinct X and Y give a duplicate-free product.

The charged support is the full product P_{15}×P_{15}, with 15·15=225 distinct records. The neutral support is P_4×(P_{14}×P_{14}), with 4·14·14=784 distinct records, retaining each of the four neutral-channel labels and both axis positions. The native proof identifies these list lengths with the charged/neutral pair returned by the original source-count calculation. The two supports therefore supply the previously derived 225/(225+784)=225/1009 share through complete labelled enumeration. Sixteen component declarations have zero axiom dependencies.

The source grammar tests carrier, terminal, support, relation, correction, sign, enumeration, minimality, measurement and extension. Its ordered coordinate product has $2^{10} = 1024$ forms. The promotion termination, support 16, active level 13 and sole alpha/17 return in Sections 340–342 determine the relation and correction coordinates. The coordinate-membership and acceptance argument of Section 317 applies to this product, with the following native completeness and uniqueness proofs. candidatesComplete, ElectroweakTerminalGrammarBase.lean:L140; uniqueSurvivor, ElectroweakTerminalGrammar.lean:L205.

Grammar source: V3 precision_axes, precision_value_laws_v1.py:L128; V3 ELECTROWEAK_TERMINAL_SPEC, precision_value_laws_v1.py:L143.

> Source, proof and provenance: V3 terminal_binary_support, precision_value_laws_v1.py:L61; V3 electroweak_running_share, precision_value_laws_v1.py:L71; Lean sourceChannelCounts, ElectroweakChannels.lean:L101; Lean positionsUnique, ElectroweakChannels.lean:L38; Timeline: Turn 488, line 4245.

344. General odd-denominator vacuum recurrence

For any supplied denominator q > 1 with q mod 2 = 1, let r_k be the held rank obtained after k executions of the original binary Fold from unit rank r_0 = 1. The existing exact rank calculation gives

$$0 < r_k \le q, \qquad r_k \equiv 2^k \pmod{q}, \qquad \gcd(r_k, q) = 1.$$

The first inequality follows from the positive held-rank transition. Reducedness follows from gcd(1,q) = 1 and coprimality of every binary power with odd q. Exact held-rank recovery identifies return with the binary order condition:

$$r_k = 1 \iff 2^k \bmod q = 1 \bmod q.$$

If no positive return occurred through q steps, cancellation of the common binary factor would make all q+1 positions r_0,...,r_q distinct. They lie in the complete q-member positive rank carrier, contradicting its finite cardinality. Ascending search therefore supplies a first period p with

$$0 < p \le q, \qquad r_p = 1, \qquad \forall j,\ 0 < j < p \Longrightarrow r_j \neq 1.$$

The executed transition then gives r_(p+k) = r_k. Repeated removal of a complete period yields

$$r_k = r_{k \bmod p} \qquad (k \ge 0).$$

Thus positivity and exact recurrence persist at every finite continuation, without a cutoff on denominators or Fold depth. Each generated odd successor q = 2n+3 satisfies q > 1 and q mod 2 = 1 by the exact remainder identity (3+2n) mod 2 = 3 mod 2 = 1, so the same first-return theorem applies for every supplied n. Five native theorems establish these statements with no axiom dependencies. The executable odd-denominator wrapper and its complete registered entry checks are separate from these general recurrence theorems.

> Source, proof and provenance: V3 odd_denominator_orbit, vacuum_lineage_laws_v1.py:L59; Lean unitFirstReturn, VacuumOddRecurrence.lean:L19; Lean unitPerpetualCycle, VacuumOddRecurrence.lean:L30; Timeline: Turn 435, line 3916.

345. Exact odd-denominator orbit interface and returned traces

The executable recurrence interface retains the original guard order. Boolean inputs and integer denominators d ≤ 1 receive the positive-denominator error. On the remaining domain the derived binary count is two, and the next test is exactly

$$\lfloor d/2 \rfloor\, 2 = d.$$

Equality rejects the denominator as even; otherwise the source unit part 1/d is passed to the original first-return loop. Its result retains every successive Fold, with the returned source included as the last member. The final guard rejects an empty trace or an element outside the positive One domain. For every represented Part p, its stored inequalities give 0 < numerator(p) ≤ denominator(p); induction on a trace therefore proves that its outside-domain test is false. Consequently every nonempty successful trace is preserved unchanged by this final guard.

The exact computed traces include

$$d = 3 : (2/3, 1/3),$$
$$d = 5 : (2/5, 4/5, 3/5, 1/5),$$
$$d = 7 : (2/7, 4/7, 1/7),$$
$$d = 9 : (2/9, 4/9, 8/9, 7/9, 5/9, 1/9).$$

Their lengths are respectively 2,4,3,6, with each ordered equality proved by native computation. Twenty-one interface definitions and theorems have no axiom dependencies. Fresh Lean execution agrees with the unchanged pinned V3 function and its original dependencies on 36 cases: both Boolean inputs and every integer from −2 through 31, comparing the full ordered rational traces and exact rejection outcomes. This finite comparison complements the general recurrence derivation of section 344; it does not replace that derivation.

> Source, proof and provenance: V3 odd_denominator_orbit, vacuum_lineage_laws_v1.py:L59; Lean nineTrace, VacuumOddOrbit.lean:L78; Lean exactBinaryGuard, VacuumOddOrbit.lean:L38; Timeline: Turn 435, line 3916.

346. Exact Fraction and held-rank recurrence correspondence

Let c_q(n) denote the original positive cast: repeatedly subtract q while the retained numerator is greater than q. Each subtraction preserves the remainder, so induction on the actual removal trace proves

$$c_q(n) \bmod q = n \bmod q.$$

For positive n and q, the cast lies in 1,...,q. The held-rank Fold is

$$F_q(n) = \begin{cases} q, & 2n \bmod q = 0, \\ 2n \bmod q, & \text{otherwise.} \end{cases}$$

It lies in the same held-rank interval and has remainder 2n mod q. Two ranks in 1,...,q with equal remainders are equal: a nonzero remainder names the rank itself, while zero names the retained whole q. Hence

$$c_q(n+n) = F_q(n).$$

Write N(n,q) for the original normalized positive Fraction n/q. The already derived normalization and Fold-cast identity now yields the exact state equation

$$\mathrm{Fold}(N(n,q)) = N(c_q(n+n), q) = N(F_q(n), q).$$

For the unit orbit define x_k = N(r_k,q), where r_k is the actual held-rank iteration from section 344. Then Fold(x_k) = x_(k+1). The rank return r_p = 1 supplied for every odd q > 1 therefore gives x_p = x_0. This is equality of the actual normalized Fraction states, including their positive-cast convention, rather than only equality of modular residues. Eight native declarations establish this correspondence and Fraction-return existence with no axiom dependencies. The original loop's first-return branch and successful output for arbitrary odd q follow from the visited-state argument below.

> Source, proof and provenance: V3 odd_denominator_orbit, vacuum_lineage_laws_v1.py:L59; Lean fractionCycleExists, VacuumOddTrace.lean:L56; Lean castMatchesRank, VacuumOddTrace.lean:L17; Timeline: Turn 435, line 3916.

347. General success of the original odd-denominator first-return loop

For odd q, reducedness gives gcd(r_k,q) = 1. Normalization therefore leaves the numerator unchanged:

$$\mathrm{numerator}(x_k) = r_k / \gcd(r_k, q) = r_k.$$

Equality of Fraction states implies equality of their held ranks. Before the first binary period p, the rank-cancellation proof excludes every collision at distinct positions, hence

$$0 \le i < j < p \quad \Longrightarrow \quad x_i \neq x_j.$$

At loop position k < p, retain the invariant that the current state is x_k and every stored visited key is the key of some x_i with i ≤ k. The actual next Fold is x_(k+1). If k+1 = p, this state is x_0, so the original return-to-source branch succeeds before testing visited keys. If it differs from the source, k+1 < p. A visited-key match would then give x_(k+1) = x_i with i ≤ k, contradicting the displayed collision exclusion. Thus the non-source-repeat error branch is impossible. Advancing adds exactly the next key and preserves the invariant. Induction over the terminating original Run proves successful output.

The retained first-return theorem supplies the complete trace structure

$$\mathrm{firstReturn}(x_0) = \mathrm{ok}(T), \qquad T = B \mathbin{+\!\!+} [x_0], \qquad \mathrm{Steps}(x_0, T), \qquad \forall x \in B,\ x \neq x_0.$$

This also proves T is nonempty. For odd q, the even-denominator guard cannot hold: if floor(q/2)·2 = q, taking remainders gives q mod 2 = 0, contradicting q mod 2 = 1. The wrapper therefore reaches the original loop, and its final positive-domain guard preserves T by section 345. For every supplied n with (n+2) mod 2 = 1,

$$\exists T, \quad \mathrm{oddDenominatorOrbit}(n+2) = \mathrm{ok}(T) \wedge \mathrm{FirstReturn}(1/(n+2), T).$$

Nine native theorems establish the Fraction-state collision exclusion, actual loop success, guard passage and complete wrapper result with no axiom dependencies. The theorem is quantified over the full admitted odd-denominator domain.

> Source, proof and provenance: V3 odd_denominator_orbit, vacuum_lineage_laws_v1.py:L59; Lean runSuccessful, VacuumOddLoop.lean:L39; Lean originalWrapperSucceeds, VacuumOddLoop.lean:L108; Timeline: Turn 435, line 3916.

348. Original vacuum recurrence witnesses

The four original witness records use the actual odd-denominator orbit function. The first requires the ordered trace (2/3,1/3); the next two require trace lengths four and three for denominators five and seven. The fourth checks every denominator d = 2n+3 for n = 0,...,14 and requires that the final returned Fraction is 1/d:

$$\bigwedge_{n=0}^{14} \mathrm{last}(\mathrm{oddDenominatorOrbit}(2n+3)) = 1/(2n+3).$$

All four predicates evaluate to true in native Lean, retaining their original names and statements. The complete prefix gives the following denominator/period pairs:

$$(3,2),(5,4),(7,3),(9,6),(11,10),$$
$$(13,12),(15,4),(17,8),(19,18),(21,6),$$
$$(23,11),(25,20),(27,18),(29,28),(31,5).$$

These periods are lengths of the full ordered traces, whose final elements are respectively 1/d. Fresh execution matches those traces against the original V3 function. Ten witness definitions and theorems have no axiom dependencies. The general result for arbitrary odd denominators is the theorem of section

347; this complete finite prefix preserves the source's additional concrete witnesses.

> Source, proof and provenance: V3 odd_denominator_orbit, vacuum_lineage_laws_v1.py:L59; Lean originalWrapperSucceeds, VacuumOddLoop.lean:L108; Lean runSuccessful, VacuumOddLoop.lean:L39; Timeline: Turn 435, line 3916.

349. Completed odd-denominator vacuum recurrence

The odd-denominator recurrence entry is complete. The general rank return, normalized FractionFold bridge and original first-return-loop derivation are given in Sections 344–347; the four original witnesses and ordered denominator 3..31 return traces are given in Section 348. For every admitted odd denominator q>1, the actual source wrapper produces a nonempty positive trace ending at its original unit part, with no earlier return. The exact Boolean/integer rejection domain is retained.

All 256 original candidate decisions and all registration, closure, control and digest fields match fresh native execution. The pinned independent validator accepts this output and rejects omitted enumeration, an extra survivor and a failed control. The component adds 92 native theorems and 151 audited definitions/theorems across 25 modules, all with zero axiom dependencies. The cumulative completed census contains 308 of 2,781 entries, with 17,849 audited theorems and 38,713 audited definitions and theorems.

The source grammar tests carrier, dependency, relation, enumeration, minimality, measurement, record and extension. Its ordered coordinate product has $2^8 = 256$ forms. The general termination and return proof in Sections 344–347 supplies the admitted odd-denominator recurrence, including the complete trace rather than a finite prefix. The coordinate-membership and acceptance argument of Section 317 applies to this product, with the following native completeness and uniqueness proofs. candidatesComplete, VacuumOddGrammarBase.lean:L118; uniqueSurvivor, VacuumOddGrammar.lean:L61.

Grammar source: V3 vacuum_axes, vacuum_lineage_laws_v1.py:L142; V3 VACUUM_RECURRENCE_SPEC, vacuum_lineage_laws_v1.py:L182.

> Source, proof and provenance: V3 odd_denominator_orbit, vacuum_lineage_laws_v1.py:L59; Lean unitPerpetualCycle, VacuumOddRecurrence.lean:L30; Lean unitFirstReturn, VacuumOddRecurrence.lean:L19; Lean originalWrapperSucceeds, VacuumOddLoop.lean:L108; Lean runSuccessful, VacuumOddLoop.lean:L39; Timeline: Turn 435, line 3916.

350. Complete reference-equivalence classes and retained records

Let R be the generated kind-preserving reference equivalence of the physical quantity construction, and write Q_a(o) for R(a,o.carrier). The equivalence properties and the separate observation record are those of the quantity carrier itself. Every observation belongs to the class anchored at its own carrier, since R(o.carrier,o.carrier).

If an observation o lies in both Q_a and Q_b, symmetry gives R(o.carrier,b), and transitivity with R(a,o.carrier) gives R(a,b). For any observation x, transitivity in both directions then yields

$$Q_a(o) \wedge Q_b(o) \Longrightarrow R(a,b), \qquad R(a,b) \Longrightarrow \big(Q_a(x) \iff Q_b(x)\big).$$

Conversely, suppose membership agrees for all observations. Apply the membership equivalence to the observation with carrier b and any held record. Reflexivity gives membership in Q_b, hence R(a,b). Therefore

$$\big(\forall o,\ Q_a(o) \iff Q_b(o)\big) \iff R(a,b).$$

Disconnected anchors consequently have no common member. The classes cover all admitted observations and are either disjoint or have identical membership, without constructing a quotient that erases records. If x and y belong to Q_a, kind preservation gives kind(x.carrier)=kind(a)=kind(y.carrier). Replacing only an observation's record leaves its membership unchanged; distinct records remain separately held by the underlying observation constructor. Eight native theorems establish these class consequences with zero axiom dependencies. They operate on the source's reference-equivalence carrier; they do not replace it with a numerical-value classifier.

> Source, proof and provenance: V3 QUANTITY_CARRIER, measurement_laws.py:L99; Lean classIdentityIff, QuantityClasses.lean:L34; Lean sameClassConnects, QuantityClasses.lean:L28; Timeline: Turn 480, line 4102.

351. Exact same-dimension reference transport

A reference retains its name and canonical dimension form. An admitted comparison from reference a to reference b carries both the equality D(a)=D(b) and a positive normalized ratio x. The dimension form is a held form, independent of the ratio's numerical value. If D(a)≠D(b), an admitted comparison would supply the contradictory equality, so no such comparison can be constructed.

For x=n/d with n,d positive, reversing the comparison exchanges the reference endpoints, reverses their dimension equality and uses the already derived normalized reciprocal. Writing N for exact normalization,

$$x^{-1} = N(d, n), \qquad x\,x^{-1} = N(1, 1).$$

For comparisons a→b with ratio x and b→c with ratio y, transitivity gives D(a)=D(c), and the composite ratio is

$$z = N(n_x n_y, d_x d_y), \qquad (n_x n_y) d_z = n_z (d_x d_y).$$

The cross-product identity is the normalization-preserving composition theorem, valid at arbitrary admitted positive counts. For views N(n,m) and N(m,k), its existing exact cancellation theorem gives

$$n_z\, k = n\, d_z,$$

so the intermediate reference cancels in value while both input comparisons remain available in the finite chain.

A chain begins with one comparison and extends by appending a comparison whose starting reference is the current endpoint. Its evaluated comparison is defined by

$$C([a \to b]) = C_{ab}, \qquad C(P \mathbin{+\!\!+} [b \to c]) = C(P) \circ C_{bc}.$$

The constructor retains the prior chain and the appended comparison. Each collapse uses the positive ratio product and transitive dimension identity; hence every supplied finite chain has a positive ratio and preserves the dimension of its endpoints. Thirteen native theorems establish exclusion of unlike dimensions, positive ratios, reciprocal return, exact composition, telescoping and the finite-chain base and successor equations, with zero axiom dependencies. This component formalizes transport on admitted references. Canonical dimension generation is derived in Sections 352–353; the complete source-program and empirical comparisons are reported in Section 355.

> Source, proof and provenance: V3 UNIT_COMPARISON, measurement_laws.py:L125; Lean everyFiniteChainPreservesDimension, ReferenceTransport.lean:L81; Lean reverseReturnsOne, ReferenceTransport.lean:L55; Timeline: Turn 480, line 4102.

352. Finite canonical dimension composition

The registered dimension statement describes canonical finite products and exact quotients of independent quantity kinds. Represent a generated description by a kind label, a product of two descriptions, or an exact quotient of two descriptions. Its support is the finite list of kind labels appearing in that description, retaining both operands at every composition. No measured value enters this construction.

For a held kind k, retain a numerator count n and denominator count d. A matching kind contributes (1,0), and an absent kind contributes (0,0). Products add like-side counts; quotients exchange the second operand's sides:

$$C_k(ab) = (n_a + n_b, d_a + d_b), \qquad C_k(a/b) = (n_a + d_b, d_a + n_b).$$

These are counts of occurrences in the finite description. Their zero denotes absence of a factor, not a physical zero magnitude. Cancellation gives the canonical residual pair

$$R(n, d) = (n - d, d - n).$$

At least one residual side is absent: if n≤d, n−d=0; otherwise d≤n and d−n=0. Induction on the common appended count c proves

$$R(n + c, d + c) = R(n, d).$$

Thus a common numerator/denominator factor cancels exactly without choosing a numerical scale or using a signed physical magnitude.

Write F_k(p)=R(C_k(p)) for the held form of description p at kind k. Descriptions share a dimension precisely when their forms agree at every kind. Reflexivity, symmetry and transitivity follow from equality of the residual pairs. Distinct primitive labels cannot be identified: at label k, the description k has pair (1,0), whereas a different primitive q has pair (0,0). This retains the independence of the kind carriers.

Finite count addition proves product commutation and association. Exchanging denominator orientation proves

$$(a/b)/c \sim a/(bc), \qquad a/(b/c) \sim (ac)/b.$$

Here ∼ means equality of all held kind forms. For (ab)/b, the counts at each kind are n_a+n_b+d_b and d_a+d_b+n_b. Reordering the same appended support and applying common-factor cancellation gives

$$(ab)/b \sim a, \qquad a/a \sim 1.$$

The final notation denotes absence of every residual factor, not an extra primitive quantity. Likewise, the counts of (a/b)(b/c) contain the common addition n_b+d_b on both sides. Removing that addition proves

$$(a/b)(b/c) \sim a/c.$$

These identities hold for every supplied finite description, including composite denominator descriptions.

> Source, proof and provenance: V3 DIMENSION_COMPOSITION, measurement_laws.py:L112; Lean sharedKindCancellation, DimensionComposition.lean:L119; Lean outsideSupport, DimensionComposition.lean:L139; Timeline: Turn 480, line 4102.

353. Complete finite decision of held dimension identity

Let S(p) be the complete generated support of a description p. Induction on its construction proves that k∉S(p) implies C_k(p)=(0,0), hence F_k(p)=(0,0). For products and quotients, absence from the appended support implies absence from both operand supports, and the count equations preserve the absent pair.

The executable comparison checks every label in S(a)++S(b):

$$B(a,b) = \bigwedge_{k \in S(a) +\!+ S(b)} [F_k(a) = F_k(b)].$$

If B is true, finite-list induction supplies equality at each listed kind. Any unlisted kind is absent from both supports and has pair (0,0) on both sides. Therefore the finite comparison establishes equality at every kind. Conversely, equality at every kind supplies every conjunction member, so finite-list induction makes B true:

$$B(a,b) = \text{true} \iff \forall k,\ F_k(a) = F_k(b).$$

Duplicate occurrences and support order do not omit a kind: the comparison retains the complete operand supports, and repeated equality checks preserve the result. Distinct primitive kind labels are rejected by the finite comparison because its soundness would otherwise contradict primitive independence. The construction, cancellation identities and complete finite decision are established by 31 native theorems; all 39 declarations in this component have zero axiom dependencies.

The original measurement grammar remains a separate complete product of eight binary source coordinates: carrier, relation, provenance, prediction access, measurement record, row retention, finite generality and extra rule. All 256 forms are retained. A form survives exactly when each coordinate has its admitted source value, giving one survivor and an explicit rejected coordinate for every alternative. The dimension-specific relation is the original canonical-dimension-path relation. The accompanying evidence retains the original registration, dependencies, two operational pairing witnesses, closure records and four executable controls. These grammar and evidence components add 20 native theorems. Their candidate decisions supplement the finite dimension construction above.

The source constructor `empirical_dimensions` retains seven fixed requirements: complete Fold carrier, source-bound proof trace, capability-closed prediction, separate measurement record, complete registered rows, one-successor closure and no extra rule. Its eighth requirement here is the canonical dimension path derived above. Each requirement has one named rejected alternative. `candidate_rows` nests these eight two-element lists in source order. For an arbitrary typed candidate, coordinate case analysis supplies membership in each list; nested `flatMap`/`map` membership then proves that the entire candidate occurs. The native decision equals the eight-coordinate conjunction, and the conjunction fixes the unique all-admitted tuple. The same construction is instantiated by the later empirical Physics grammars with their own derived relation coordinate. candidatesComplete, DimensionGrammar.lean:L119; acceptanceIffExactCriteria, DimensionGrammar.lean:L171.

Grammar source: V3 empirical_dimensions, generated_empirical_law.py:L123; V3 candidate_rows, generated_empirical_law.py:L136.

> Source, proof and provenance: V3 DIMENSION_COMPOSITION, measurement_laws.py:L112; Lean finiteComparisonSound, DimensionComposition.lean:L209; Lean sharedKindCancellation, DimensionComposition.lean:L119; Timeline: Turn 480, line 4102.

354. Reference transport on equivalent generated descriptions

Same-dimension reference transport can now use the generated paths themselves. A reference retains a name and a finite dimension description. Its comparison carries an exact positive ratio and the proved relation ∼ from Sections 352–353, rather than requiring identical written descriptions. The finite decision and its soundness theorem admit a comparison exactly on the proved dimension relation. A failed decision excludes every such comparison, independently of which numerical ratio is proposed.

Reversal uses symmetry of ∼ and the normalized reciprocal; composition uses transitivity and normalized ratio multiplication:

$$(a \xrightarrow{x} b)^{-1} = b \xrightarrow{x^{-1}} a, \qquad (a \xrightarrow{x} b \xrightarrow{y} c) = a \xrightarrow{xy} c.$$

Both resulting reference pairs pass the complete finite dimension comparison. Their ratio equations and reciprocal return are the exact positive-ratio identities of Section 351. Replacing a comparison's ratio cannot change its independently retained dimension relation.

The constructors admit, for example, the descriptions pq and qp by product commutation, and (pq)/q and p by common-factor cancellation. They therefore preserve equivalence of generated descriptions instead of imposing identity of their unreduced syntax. Nine native theorems and the comparison constructors establish this connection with zero axiom dependencies.

For the dimension entry, fresh compiled Lean execution reproduces all 256 V3 candidate lookups, registration fields, census and proof hashes, closure, controls and unknown-identifier rejection. The unchanged V3 independent validator accepts the complete output and rejects omitted enumeration, an additional survivor and a failed control. The translated experiment preserves the original two BIPM target rows, source identity, registered prediction label, and post-seal comparison protocol. Four experiment theorems establish its source-hash rejection branch, complete two-row count, both retained observation labels and registered specification validity. The full native byte-level empirical validator also reproduces the pinned V3 output, including all 1,979,202 source bytes, both target rows and every receipt field and hash.

> Source, proof and provenance: V3 DIMENSION_COMPOSITION, measurement_laws.py:L112; V3 UNIT_COMPARISON, measurement_laws.py:L125; Lean ratioCannotChangeDimension, DimensionReferenceTransport.lean:L41; Lean finiteAdmissionRetainsRatio, DimensionReferenceTransport.lean:L46; Timeline: Turn 480, line 4102.

355. Completed measurement carrier, dimension and unit entries

Observation retains the phenomenon, held reference and exact comparison record. Quantity membership is determined by the kind-preserving reference relation, while the measured record remains separately held. Sections 350–354 give the class equivalence, finite canonical dimension construction and exact positive reference transport. The four entries—observation carrier, quantity carrier, dimension composition and unit comparison—are complete against their pinned source obligations.

For each entry, the native program reproduces all 256 original candidate lookups, their ordered forms and decisions, census and proof hashes, registration, closure, controls and rejection of an unknown identifier. The unit grammar retains the same-dimension-reference-ratio relation. Its full candidate product is

$$G = C \times R \times P \times A \times M \times H \times F \times E, \qquad |G| = 2^8 = 256,$$

where the coordinates are the original carrier, relation, provenance, prediction access, measurement record, row retention, finite generality and extra-rule alternatives. Acceptance is the conjunction of the

eight original admitted coordinates. Every rejected tuple has at least one explicitly rejected coordinate, and the single all-admitted tuple survives. This classification accompanies the unrestricted finite transport derivations of Sections 351 and 354.

The original blind empirical validator runs the registered prediction before opening the held target. The native comparison retains both BIPM rows and the complete source snapshot of 1,979,202 bytes. For all four entries, every empirical output field and hash agrees with the pinned V3 record, including execution, prediction sealing, custody, isolation certificates, controls and receipt. Host platform and implementation metadata are supplied from that record. The protected-tree before/after snapshots are supplied identically, preserving the original equality test. These are explicit adapter inputs; the source-byte digest is computed from the complete snapshot during each native run.

The cumulative formalisation contains 308 completed census entries, 17,849 audited theorems and 38,713 audited definitions and theorems, with zero axiom dependencies in the retained audits. Complete candidate comparisons cover 142,354 records across 308 completed entries. The four derivation components in Sections 356–359 are accompanied by completed source-program and empirical translations, as reported in Section 362.

> Source, proof and provenance: V3 QUANTITY_CARRIER, measurement_laws.py:L99; V3 DIMENSION_COMPOSITION, measurement_laws.py:L112; V3 UNIT_COMPARISON, measurement_laws.py:L125; Lean sameClassConnects, QuantityClasses.lean:L28; Lean classIdentityIff, QuantityClasses.lean:L34; Lean finiteComparisonSound, DimensionComposition.lean:L209; Lean unlikeKindsRejected, DimensionComposition.lean:L228; Lean retainsDimension, ReferenceTransport.lean:L25; Lean differentDimensionExcluded, ReferenceTransport.lean:L28; Timeline: Turn 480, line 4102.

356. Operational realization with a complete retained trace

A realization step from reference a to reference b contains the admitted same-dimension comparison, its exact positive ratio x, source identity σ and observation record ρ. A finite process begins with one step and extends only by a step whose starting reference is the current endpoint. For a process P and appended step s, define its resulting comparison and trace by

$$R([s]) = s.\mathrm{comparison}, \qquad R(P \mathbin{+\!\!+} [s]) = R(P) \circ s.\mathrm{comparison},$$

$$T([s]) = [\mathrm{row}(s, x)], \qquad T(P \mathbin{+\!\!+} [s]) = T(P) \mathbin{+\!\!+} [\mathrm{row}(s, R(P \mathbin{+\!\!+} [s]).\mathrm{ratio})].$$

Each row holds the from-reference, to-reference, source identity, observation record, local factor and cumulative ratio. The row constructor preserves both endpoints and both source fields exactly. The composition operation is the normalized positive ratio composition of Section 351; hence, writing x=n_x/d_x and y=n_y/d_y,

$$\mathrm{ratio}(R(P \mathbin{+\!\!+} [s])) = N(n_x n_y, d_x d_y),$$

where x is the prior cumulative ratio and y the appended comparison ratio.

Let H(P) be the ordered list of source-identity/record pairs and let L(P) count steps. The first-step equations are H([s])=[(σ_s,ρ_s)] and L([s])=1; extension appends that pair and increases the count by one. The finite-list identities

$$\mathrm{map}(f, X \mathbin{+\!\!+} [x]) = \mathrm{map}(f, X) \mathbin{+\!\!+} [f(x)], \qquad |X \mathbin{+\!\!+} [x]| = |X| + 1$$

follow by induction on X. Applying them at each process extension proves

$$\mathrm{map}((\sigma,\rho),T(P)) = H(P), \qquad |T(P)| = L(P) > 0.$$

Thus the trace is nonempty, retains every comparison record in order and preserves its entire prior prefix on extension. The resulting comparison carries D(a)~D(b), and its normalized ratio has positive numerator. Twelve native theorems establish these statements with zero axiom dependencies. Source identity here is the retained identity field; the trace theorem preserves supplied records and does not manufacture empirical observations.

> Source, proof and provenance: V3 REFERENCE_REALIZATION, measurement_laws.py:L138; Lean allRecordsRetained, ReferenceRealization.lean:L58; Lean successorRetainsPrior, ReferenceRealization.lean:L89; Timeline: Turn 481, line 4110.

357. Measured values as separate records

The source measured-value record is the ordered triple

$$V = (r, s, p),$$

with held reference identity r, textual numerical inscription s and observation provenance p. Replacing the inscription by t gives

$$\mathrm{replace}(V, t) = (r, t, p).$$

Projection therefore proves reference retention, exact replacement of the inscription and provenance retention. These three facts do not depend on the text being parsed as a proof-domain scalar.

Each coordinate remains distinguishable. If two records V and W were equal, applying any coordinate projection would give equality of that coordinate. Taking the contrapositive separately yields

$$r_V \neq r_W \Rightarrow V \neq W, \qquad s_V \neq s_W \Rightarrow V \neq W, \qquad p_V \neq p_W \Rightarrow V \neq W.$$

Conversely, rebuilding a record from its three held projections reproduces it exactly:

$$(V.r, V.s, V.p) = V.$$

The seven native proofs retain the source distinction between a quantity's structural carrier and its externally reported numerical inscription, with zero axiom dependencies.

> Source, proof and provenance: V3 VALUE_RECORD, measurement_laws.py:L151; Lean exactReconstruction, MeasuredValueRecord.lean:L34; Lean referenceRetained, MeasuredValueRecord.lean:L13; Timeline: Turn 481, line 4112.

358. Complete retention of measurement uncertainty

A source-bound alternative consists of its relation, source identity and the recorded verdict of whether it remains admissible. Let a(x) denote that verdict. The uncertainty support is the complete ordered retention operation

$$U([]) = [], \qquad U(x :: X) = \begin{cases} x :: U(X), & a(x) = \mathrm{true}, \\ U(X), & a(x) = \mathrm{false}. \end{cases}$$

The admissibility verdict is part of the supplied measurement record. It is not a fitted choice made by this retention operation.

Define a valid retention derivation by three rules: the empty input retains the empty output; an admitted head must be retained; a rejected head must be removed. Induction on X constructs such a derivation for U(X). Induction on any valid derivation proves its output equals U(X). Therefore two valid retention derivations for the same input cannot select different supports.

List induction also proves both directions of the exact support statement:

$$x \in U(X) \iff x \in X \ \wedge \ a(x) = \text{true}.$$

The forward direction establishes source membership and admissibility; the reverse establishes that no admitted source alternative is omitted. A rejected alternative cannot belong to U(X), since membership would make its Boolean verdict both true and false. Finally, induction on X proves

$$U(X +\!\!+ Y) = U(X) +\!\!+ U(Y).$$

Thus the construction preserves all admissible alternatives in source order, including their retained identities. Eight native theorems establish construction, exactness, unique selection, complete membership and concatenation with zero axiom dependencies.

> Source, proof and provenance: V3 UNCERTAINTY, measurement_laws.py:L164; Lean allAdmissibleRetained, MeasurementUncertainty.lean:L59; Lean retentionConstructed, MeasurementUncertainty.lean:L21; Timeline: Turn 481, line 4114.

359. Calibration with aligned comparison and uncertainty histories

A calibration comparison record contains an operational step and its complete supplied alternatives ledger. A nonempty calibration chain has the same first/append structure as the operational process. Erasing only the ledger wrapper yields its operational process O(P); the steps, comparison ratios and observation records remain unchanged. Let L(P) be its ordered list of ledgers and define its uncertainty history by

$$U_C(P) = \text{map}(U, L(P)).$$

For the initial comparison record c,

$$L([c]) = [c.\text{alternatives}],$$

and extension satisfies

$$L(P +\!\!+ [c]) = L(P) +\!\!+ [c.\text{alternatives}],$$

$$U_C(P +\!\!+ [c]) = U_C(P) +\!\!+ [U(c.\text{alternatives})].$$

These equations retain every intermediate ledger and every admitted alternative within it. Induction proves one ledger per comparison. The map-length identity and Section 356 then give

$$|U_C(P)| = |L(P)| = L_O(O(P)) = |T(O(P))|,$$

where L_O counts operational steps. No comparison is omitted from the aligned history.

The retained source pairs satisfy

$$\mathrm{map}((\sigma, \rho), T(O(P))) = H(O(P)).$$

On extension, the actual cumulative ratio is the normalized composition of the previous cumulative ratio with the new step ratio. Its numerator remains positive, and the endpoint dimension relation follows by transitivity at each composition. Ten native theorems establish the history alignment, complete source-record retention, prefix retention, exact ratio update, positivity and dimension preservation, with zero axiom dependencies. Together Sections 356–359 provide 37 native derivation theorems.

> Source, proof and provenance: V3 CALIBRATION, measurement_laws.py:L177; Lean oneLedgerPerComparison, Calibration.lean:L33; Lean completeHistoryAlignment, Calibration.lean:L41; Timeline: Turn 481, line 4116.

360. Exact source-byte hashing equivalence

The empirical source check computes SHA-256 over the complete held byte array. Its block-local lookup is related to the original absolute lookup by the finite-list identity

$$\mathrm{get}(\mathrm{drop}(B, o), j) = \mathrm{get}(B, o + j).$$

Induction on o proves the identity for every byte list B and index j, including the original out-of-range lookup behavior. Applying it to each of the four byte positions in every initial schedule word makes all 16 initial words identical. The remaining 48 schedule words use the same recurrence, so the entire block schedule agrees. Identical schedules and unchanged rounds imply identical block state updates. Induction over the block-offset list then gives

$$\forall B : \mathrm{ByteArray}, \quad \mathrm{digest}_{\mathrm{window}}(B) = \mathrm{digest}_{\mathrm{original}}(B).$$

Padding, constants, rounds and digest formatting remain the original operations. Six native theorems establish the lookup, schedule, block and whole-digest equivalence without axiom dependencies. Consequently each measurement experiment's source-match predicate equals its original predicate for every supplied source byte array. The complete snapshot is still hashed; the equivalence does not replace the bytes with a stored digest.

> Source, proof and provenance: V3 hash_file, source.py:L25; Lean digestPreserved, SHA256Window.lean:L79; Lean schedulePreservesEveryByte, SHA256Window.lean:L33; Timeline: Turn 481, line 4110.

361. Dimensional consistency at every finite comparison

The source dimensional-consistency statement requires canonical path identity at each compared or joined pair, and restricts conversions to reversible exact same-dimension reference maps. The path construction and complete finite decision are those of Sections 352–353. For the full ordered list P of comparison pairs, define

$$A([]) = \mathrm{true}, \qquad A((p, q) :: P) = B(p, q) \wedge A(P),$$

where B is the proved finite comparison of canonical dimension forms. Define lawfulness by

$$\mathcal{L}(P) \iff \forall (p,q) \in P,\ \forall k,\ F_k(p) = F_k(q).$$

Induction on the entire list proves both directions. If A is true, its first conjunct supplies canonical equality for the head pair, while the induction hypothesis supplies it for every tail pair. Conversely, lawfulness supplies the first finite comparison and the induction hypothesis supplies the remaining conjunction. Hence

$$A(P) = \text{true} \iff \mathcal{L}(P).$$

No comparison is selected or dropped. If a retained pair has unequal canonical forms, assuming A(P) true contradicts that pair's inequality; therefore A(P) is false. If P and Q are each lawful, induction on P proves lawfulness of P++Q, preserving the full comparison list.

An admitted conversion from reference a to reference b retains both D(a)~D(b) and its exact positive normalized ratio x. Its inverse is the original reverse comparison b→a with reciprocal ratio. Symmetry of canonical path identity supplies the inverse dimension relation. The exact normalized ratio theorem gives

$$a \xrightarrow{x} b \xrightarrow{x^{-1}} a, \qquad N(x\,x^{-1}) = \frac{1}{1}.$$

Here the displayed equality means the actual composed normalized fraction has numerator 1 and denominator 1. It follows from the previously proved reciprocal-return identity, rather than a numerical approximation. For a→b and b→c, transitivity of the retained dimension relations yields a→c, and soundness/completeness of B makes its finite dimension check pass. Unlike endpoint forms exclude an admitted conversion because such a conversion would carry the contradictory dimension relation.

For a full calibration chain P:a→b, the composition theorem in Section 359 supplies D(a)~D(b); applying finite-comparison completeness gives B(D(a),D(b))=true. Eleven native theorems establish finite lawfulness, exclusion, concatenation, exact reversible conversions and calibration endpoint consistency, with zero axiom dependencies.

> Source, proof and provenance: V3 DIMENSIONAL_CONSISTENCY, measurement_laws.py:L190; Lean allSameSound, DimensionalConsistency.lean:L14; Lean allSameComplete, DimensionalConsistency.lean:L28; Timeline: Turn 481, line 4118.

362. Completed measurement closure

Reference realization, measured-value records, uncertainty, calibration and dimensional consistency are complete against their pinned V3 source obligations. The mathematical derivations are given in Sections 356–359 and 361. Each original eight-axis product contains all 256 candidate forms, with acceptance defined by the same eight-coordinate conjunction in Section 355. The admitted relation coordinate is respectively definition-to-operation-trace, held-reference-number-provenance-record, retained-unresolved-observation-support, complete-composed-reference-chain and canonical-path-identity-and-reversible-conversion. Each is paired with the same complete carrier, source-bound provenance, closed prediction access, separate measured record, all-row retention, successor closure and no-extra-rule coordinates. Thus the all-admitted tuple is the sole survivor in each product, and every other tuple has an explicit rejected coordinate. These exact finite exclusions accompany the general derivations rather than replacing them. Fresh compiled native execution reproduces every candidate lookup, ordered census and decision, registration field, closure, control and digest; an unknown candidate identifier is rejected. For each entry the unchanged pinned independent validator accepts the original output and rejects omitted enumeration, an additional survivor and a failed control.

The full blind empirical comparisons reproduce the original two-row BIPM protocol, retaining capability-closed prediction, prediction sealing, post-seal target opening, complete row comparison and changed-row rejection. Every field and hash matches its pinned record, including source, custody, isolation and receipt fields. All 1,979,202 snapshot bytes are hashed natively. As in Section 355, original host metadata and equal before/after protected-tree snapshots are supplied explicitly to the adapter.

The complete measurement chain now runs from observation through quantity, canonical dimension, unit comparison, reference realization, measured record, uncertainty and calibration to dimensional consistency. The cumulative census contains 308 completed entries out of 2,781. There are 17,849 audited theorems and 38,713 audited definitions and theorems, all with zero axiom dependencies. Full candidate comparisons cover 142,354 records across 308 completed source entries. The five entries contribute 161 newly registered theorems and 533 newly audited definitions/theorems; the 37 realization, record, uncertainty and calibration component theorems reported earlier are not counted twice.

> Source, proof and provenance: V3 CALIBRATION, measurement_laws.py:L177; V3 DIMENSIONAL_CONSISTENCY, measurement_laws.py:L190; Lean calibrationEndpointLawful, DimensionalConsistency.lean:L80; Lean allSameSound, DimensionalConsistency.lean:L14; Lean everyRecordHeld, Calibration.lean:L54; Lean successorRetainsSupport, Calibration.lean:L49; Timeline: Turn 481, line 4116.

363. Complete observed events and retained change

The mechanics source follows the completed measurement chain. A physical event retains the complete observed source state; a change retains both endpoint observations and the source-bound action connecting them. The construction below is uniform in the held state carrier and uses the already defined observation record. It does not replace that carrier with a finite answer classifier or a reported numerical inscription.

Write an observed event as E=(s,ρ), where s is the complete supplied carrier state and ρ is its held observation record. A transition from a to b holds source identity σ and action record A. Its retained row is

$$R(a \to b) = (a, b, \sigma, A).$$

Both complete events remain present, including both observation records. Projecting a row recovers each of these coordinates. If either endpoint, source identity or action record differs, equality of the two rows would imply equality of that coordinate by projection, a contradiction. Reconstructing the row from its four projections returns the exact original row.

An event path begins at one observed event and extends by one transition from its current endpoint. Its event sequence E(P), transition rows T(P) and source/action ledger H(P) satisfy

$$E([a]) = [a], \quad T([a]) = [], \quad H([a]) = [],$$

$$E(P \mathbin{+\!\!+} [b \to c]) = E(P) \mathbin{+\!\!+} [c],$$

$$T(P \mathbin{+\!\!+} [b \to c]) = T(P) \mathbin{+\!\!+} [R(b \to c)],$$

$$H(P \mathbin{+\!\!+} [b \to c]) = H(P) \mathbin{+\!\!+} [(\sigma_{bc}, A_{bc})].$$

The endpoint indices ensure that the appended transition starts at the previous endpoint. Let n(P)=|T(P)|. The append-one length identity proves n(P+[b→c])=n(P)+1. Induction on the path then proves

$$|E(P)| = n(P) + 1, \qquad E(P) \neq [], \qquad \mathrm{map}((\sigma, A), T(P)) = H(P).$$

Thus every extension preserves all earlier events and transitions, appends exactly one generated change, and retains every source/action record in order. Fifteen native theorems establish these base, successor, retention and distinguishability properties without axiom dependencies.

> Source, proof and provenance: V3 EVENT_CHANGE, mechanics_laws.py:L68; Lean allSourceRecordsRetained, MechanicalEvents.lean:L79; Lean completeEventCount, MechanicalEvents.lean:L55; Timeline: Turn 482, line 4126.

364. Duration from generated recurrence

A held reference recurrence contains its starting observation, returning observation and complete event path C. It retains the equality of the returning and starting carrier states, while the observation records remain separately held. Its generated transition count m=n(C) is positive. No absolute background time coordinate enters the construction.

For a measured event path P, let n=n(P). The source distinction between absent change and positive duration is represented by separate constructors:

$$\Delta(P; C) = \begin{cases} \mathrm{EmptyOne}, & n = 0, \\ \mathrm{Positive}(N(n, m)), & n > 0, \end{cases}$$

where N is the previously derived exact positive fraction normalization. The zero in the branch condition counts the absence of transitions; it is not a numerical physical duration. The positive constructor carries a fraction whose numerator is proved positive and whose denominator is nonzero.

For one observed event there is no transition, so Δ([a];C)=EmptyOne. Conversely, if Δ(P;C)=EmptyOne, a positive transition count would put the result in the distinct positive constructor. Therefore

$$\Delta(P; C) = \mathrm{EmptyOne} \iff n(P) = 0.$$

Appending a transition gives n+1 and excludes the Empty One case. For n>0, writing N(n,m)=u/v, exact normalization proves

$$nv = um, \qquad u > 0.$$

The reference recurrence compared with itself gives N(m,m)=1/1: both normalized numerator and denominator are exactly 1.

Changing the held reference from a recurrence of m transitions to one of k transitions composes the generated ratios. If z=N(n,m)\circ N(m,k), the exact telescoping theorem gives

$$z_{\mathrm{num}}\, k = n\, z_{\mathrm{den}}.$$

The intermediate recurrence cancels in value while both source paths remain available. The event-count identity |E(P)|=n(P)+1 connects duration to the retained ordered path. Eleven native theorems establish absence, positive duration, exact normalization, reference return, reference-unit One and reference-change telescoping with zero axiom dependencies.

> Source, proof and provenance: V3 DURATION, mechanics_laws.py:L77; Lean absenceExactly, MechanicalDuration.lean:L43; Lean referenceChangeTelescopes, MechanicalDuration.lean:L85; Timeline: Turn 482, line 4128.

365. Reference-relative location and held displacement orientation

A location retains its reference identity, complete supplied support and observation record:

$$L = (r, S, \rho).$$

Changing either the reference identity or the support distinguishes the location, because equality would preserve each projection. Rebuilding L from its three projections returns L exactly. This is a reference-relative record; no unobserved absolute position is added.

A displacement holds two such locations, their admitted exact positive separation x and which endpoint holds the orientation:

$$d = (L_1, L_2, x, h), \qquad h \in \{\text{first}, \text{second}\}.$$

The separation is the supplied exact quantity relation, not a fitted coefficient. Both endpoint reference identities and both observation records remain held. Define opposite(first)=second and opposite(second)=first. The two finite cases establish opposite(opposite(h))=h and opposite(h)≠h.

Reversing the comparison orientation is

$$\text{reverse}(d) = (L_1, L_2, x, \text{opposite}(h)).$$

It preserves the positive magnitude and the entire source pair, changes the held orientation, and returns d when applied twice. Orientation reversal introduces no negative physical length.

Relabelling the written endpoint order is a separate description operation:

$$\text{swap}(d) = (L_2, L_1, x, \text{opposite}(h)).$$

Checking the two held-endpoint cases proves that this operation preserves the actual held location. It also preserves x and is involutive. The distinction between reversing a comparison and exchanging the written endpoint order keeps orientation and description identity explicit. Fourteen native theorems establish the record, positivity, retention and orientation properties without axiom dependencies.

> Source, proof and provenance: V3 LOCATION_DISPLACEMENT, mechanics_laws.py:L86; Lean pairRetainsReferences, MechanicalLocation.lean:L51; Lean pairRetainsObservations, MechanicalLocation.lean:L56; Timeline: Turn 482, line 4130.

366. Exact speed and oriented velocity

The source speed relation divides the retained exact separation by a positive generated duration. Velocity retains that rate together with the displacement orientation. For separation x=n_x/d_x and duration t=n_t/d_t, define

$$q = N(n_x d_t, d_x n_t).$$

Both products are positive because x and t are admitted positive fractions. Normalization supplies the exact cross-product identity

$$(n_x d_t) q_{\mathrm{den}} = q_{\mathrm{num}} (d_x n_t), \qquad q_{\mathrm{num}} > 0.$$

This constructs the quotient from the two retained quantity relations without an added numerical scale.

The native velocity record retains the displacement, both endpoint events, the complete event path, the reference recurrence and the proof that the path contains a transition. Its speed is the quotient above with t=N(n(P),n(C)); its orientation is the held displacement endpoint. Reversing that orientation leaves the displacement magnitude, event path and reference recurrence unchanged. Hence

$$\mathrm{speed}(\mathrm{reverse}(v)) = \mathrm{speed}(v),$$

$$\mathrm{orientation}(\mathrm{reverse}(v)) = \mathrm{opposite}(\mathrm{orientation}(v)), \qquad \mathrm{reverse}(\mathrm{reverse}(v)) = v.$$

The duration trace and reference are retained by equality of the complete corresponding fields, rather than by equality of a reported number alone.

The dimension description is the generated quotient of the separation and duration descriptions. For each kind k, if their numerator/denominator occurrence counts are (a,b) and (c,d), its counts are

$$C_k(D_{\mathrm{speed}}) = (a + d, b + c).$$

This is the exact quotient construction of Section 352, retaining both dimension carriers. Equality of two unreduced quotient descriptions implies equality of each supplied operand description; their canonical comparisons still use the proved reduction relation. Eleven native theorems establish the exact rate, positivity, orientation, retained records and dimension-description properties with zero axiom dependencies.

> Source, proof and provenance: V3 SPEED_VELOCITY, mechanics_laws.py:L95; Lean speedExact, MechanicalVelocity.lean:L42; Lean quotientExact, MechanicalVelocity.lean:L34; Timeline: Turn 482, line 4132.

367. Completed initial mechanics chain

The event/change, duration, location/displacement and speed/velocity entries are complete against the pinned V3 model. Their general structural derivations are given in Sections 363–366. Each original eight-axis grammar contains all 256 forms, retaining carrier, relation, provenance, prediction access, measured record, row retention, finite generality and extra-rule coordinates. The admitted relation is respectively state-bound-event-transition, transition-to-reference-recurrence-ratio, reference-separation-with-held-orientation and separation-per-recurrence-with-orientation. The full conjunction admits one tuple and gives a rejected coordinate for every other tuple.

The original operational witnesses remain executable: extending [event-a] to [event-a,event-b] retains its first event, and changing the held orientation of the pair (orientation,1) preserves its magnitude coordinate. Each program reproduces every candidate lookup, census and decision hash, registration, closure, control and unknown-identifier rejection. The original independent validator accepts the native output and rejects an omitted candidate, an extra survivor and a failed control.

The four full blind empirical validators reproduce the pinned CODATA protocol. They retain each original pair of target rows, the sealed prediction, post-seal source opening, complete row comparison and changed-row control. Every output field and hash agrees with V3, including execution, custody, isolation and receipt fields. The complete 40,801-byte CODATA snapshot is hashed natively. Original host metadata and equal before/after protected-tree snapshots are explicit adapter inputs, as in the preceding measurement comparisons.

The registered formalisation contains 308 completed entries out of 2,781, with 17,849 audited theorems and 38,713 audited definitions and theorems, all with zero axiom dependencies. Registered candidate comparisons cover 142,354 records across 308 completed entries. The terminal Planck entry is complete, including its 1,024-form native comparison, original independent-validator checks and source-derived colour-coupling dependency.

> Source, proof and provenance: V3 SPEED_VELOCITY, mechanics_laws.py:L95; Lean speedExact, MechanicalVelocity.lean:L42; Lean retainsOrientation, MechanicalVelocity.lean:L49; Timeline: Turn 482, line 4132.

368. Canonical identity under the proved byte-hash equivalence

For a held canonical value v, source identity is computed as

$$I(v) = \texttt{sha256:} \parallel H(\mathrm{UTF8}(\mathrm{render}(v))),$$

where rendering retains the original canonical ordering and serialization. Section 360 establishes equality of the original and block-window digest for every byte array. Substitution at the exact rendered byte array therefore proves

$$\forall v, \quad I_{\mathrm{window}}(v) = I(v).$$

The corresponding isolated native identity theorem has zero axiom dependencies. It preserves all canonical data constructors, ordering and rendering; the scientific computations and complete candidate grammars are unchanged. This implementation equivalence is separate from the full source-output comparisons required for entry completion.

> Source, proof and provenance: V3 sha256_identity, canonical.py:L36; Lean identityOriginal, Canonical.lean:L59; Lean digestPreserved, SHA256Window.lean:L79; Lean blockPreserved, SHA256Window.lean:L59; Timeline: Turn 482, line 4126.

369. Completed dark-to-baryon fraction

The dark-to-baryon entry is complete against its pinned V3 source. Its structural inputs are the independently generated binary b=2, generator c=3 and least cover depth d=5. The construction of Section 318 gives

$$V = c^c = 27, \quad T = b^d = 32, \quad f_b = \frac{d}{T} = \frac{5}{32}, \quad f_d = \frac{V}{T} = \frac{27}{32}, \quad R_0 = \frac{V}{d} = \frac{27}{5}.$$

The least-cover proof checks $27 \leq 2^5$ and proves $2^k < 27$ for every positive k<5. The return floor is $T - 1 = 31$. The exact Fold recurrence from 1/31 produces

$$\frac{2}{31}, \frac{4}{31}, \frac{8}{31}, \frac{16}{31}, \frac{1}{31}.$$

The first four terms differ from the initial part and the fifth returns to it. This is the complete first-return trace, not a truncated orbit. The source continuation is therefore

$$R_1 = \frac{V}{d + 1/(T-1)} = \frac{27}{5 + 1/31} = \frac{27 \cdot 31}{156} = \frac{279}{52}.$$

The sealed prediction contains both R_0 and R_1. Only afterwards does the evaluator open the density rows and construct the outward interval of Section 318. Exact cross-multiplication proves

$$\frac{238}{45} \le \frac{27}{5} \le \frac{1210}{223}, \qquad \frac{238}{45} \le \frac{279}{52} \le \frac{1210}{223}.$$

The complete ten-axis grammar contains all 1,024 forms. Native source comparisons preserve every lookup decision, canonical census and decision hash, registration field, closure record, original control and unknown-identifier rejection. The original independent validator accepts the complete result and rejects an omitted candidate, an additional survivor and a failed control. The full blind empirical comparison preserves all 743 source bytes, both density rows, confidence and unit guards, custody, isolation, certificates and receipt. The ten registered modules contain 50 theorems and 158 audited definitions and theorems, with no axiom dependencies. The Hubble calibration entry is separately completed with its full grammar, controls and empirical boundary in Section 387.

The source grammar tests carrier, volume, cover, partition, leading, orbit, deepening, target, provenance and extension. Its ordered coordinate product has $2^{10} = 1024$ forms. The least cover, depth/volume partition and own-depth return fix the admitted carrier, normalization and continuation; the target and provenance coordinates keep the density readings outside that construction. The coordinate-membership and acceptance argument of Section 317 applies to this product, with the following native completeness and uniqueness proofs. candidatesComplete, DarkBaryonGrammar.lean:L140; uniqueSurvivor, DarkBaryonGrammar.lean:L189.

Grammar source: V3 DARK_BARYON_SPEC, cosmology_prior_value_laws.py:L43.

> Source, proof and provenance: V3 dark_baryon_structure, cosmology_prior_value_laws.py:L22; V3 DARK_BARYON_SPEC, cosmology_prior_value_laws.py:L43; Lean darkFirstReturn, CosmicRatios.lean:L106; Lean hubbleFirstReturn, CosmicRatios.lean:L112; Lean completeRowsRetained, DarkBaryonComparison.lean:L29; Lean missingRowRejects, DarkBaryonComparison.lean:L66; Timeline: Turn 483, line 4144.

370. Source-bound inertial continuation

The inertia construction retains the complete velocity carrier v, including displacement, orientation, generated path and reference recurrence. A transfer record is a triple t=(source identity, action record, resulting carrier). The V3 provenance rule has two admitted continuation forms: retain v when no transfer is present, or retain the result of the supplied source-bound transfer. With the absence marker denoted by none,

$$S(v, \mathrm{none}) = v, \qquad S(v, \mathrm{some}(t)) = t.\mathrm{result}.$$

The inductive admissibility relation has exactly these retained and transferred constructors. Elimination of its constructors proves that every admitted continuation w satisfies $w = S(v, s)$. Consequently, if an admitted w differs from v, then there exists a retained transfer t with $s = \mathrm{some}(t)$ and $w = t.\mathrm{result}$. The absence marker is a structural alternative; it is not a zero-valued motion carrier.

For a finite ordered transfer-option list L, define the complete succession and source ledger recursively:

$$F(v, []) = v, \qquad F(v, s :: L) = F(S(v, s), L),$$

$$Q([]) = [], \quad Q(\text{none} :: L) = Q(L), \quad Q(\text{some}(t) :: L) = t :: Q(L).$$

Induction on L establishes $Q(L) = [] \Rightarrow F(v, L) = v$. The empty list is reflexive. An absent-transfer successor reduces to the induction hypothesis; a present-transfer successor gives a nonempty ledger and cannot satisfy the premise. This proves preservation of the complete velocity, and hence preservation of its exact speed and held orientation. The same recursion proves

$$F(v, L +\!\!+ R) = F(F(v, L), R), \qquad Q(L +\!\!+ R) = Q(L) +\!\!+ Q(R).$$

Ten native component theorems establish these statements with zero axiom dependencies. The original 256-form program, complete blind empirical method and four independent-validator cases also agree with V3. The inertia entry is complete following its acceleration dependency.

> Source, proof and provenance: V3 INERTIA, mechanics_laws.py:L113; Lean closedPath, MechanicalInertia.lean:L49; Lean successorAdmissible, MechanicalInertia.lean:L30; Timeline: Turn 484, line 4160.

371. Complete inertia–velocity pair cells

Let P be the ordered, nonempty inertial-part support and v its complete velocity carrier. Momentum assigns the same complete motion carrier to every source occurrence:

$$C(P, v) = \text{map}(p \mapsto (p, v), P).$$

Induction on P proves

$$\text{map}(\pi_1, C(P, v)) = P, \qquad |C(P, v)| = |P|,$$

$$(p, w) \in C(P, v) \iff p \in P \ \wedge \ w = v,$$

$$C(P +\!\!+ Q, v) = C(P, v) +\!\!+ C(Q, v).$$

These identities retain the order and multiplicity of every supplied occurrence. No output cell can introduce an unregistered inertial part or a different motion label. Projection recovers the complete input support; the first cell of a nonempty support recovers v, so neither source coordinate is erased.

For the held exact positive inertial content m=n_m/d_m and the already derived speed u=n_u/d_u, the magnitude is the original normalized exact product

$$p = \text{normalizePositive}(n_m n_u, d_m d_u).$$

The normalization theorem proves

$$(n_m n_u) d_p = n_p (d_m d_u), \qquad n_p > 0.$$

The orientation is retained directly from the complete velocity carrier. The pair-cell record retains that carrier as well as m and P; no fitted coefficient or signed magnitude is introduced. Ten native component theorems establish the cell and product properties with zero axiom dependencies. The full 256-form program, complete 40,801-byte blind empirical comparison and four independent-validator cases agree with V3. The momentum entry is complete following acceleration and inertia in source dependency order. The two entries contribute 80 registered theorems, with no axiom dependencies.

> Source, proof and provenance: V3 MOMENTUM, mechanics_laws.py:L122; Lean cellMembership, MechanicalMomentum.lean:L22; Lean productExact, MechanicalMomentum.lean:L75; Timeline: Turn 484, line 4162.

372. Exact oriented scalar changes

The admitted scalar operations used here are the existing V3 operations in sft/mathematics/calculator/values.py. Their domain consists of structural Empty One and an exact positive rational magnitude paired with either forward-held or counter-held orientation. In particular, a stored numerical zero or negative magnitude is not added to the domain. Reversal exchanges the two orientation labels and leaves the positive magnitude unchanged; reversing twice returns the same complete scalar.

For positive magnitudes x=p/q and y=r/s, define

$$A = ps, \qquad B = rq, \qquad D = qs > 0.$$

When the orientations agree, addition retains that orientation and normalizes (A+B)/D. When the orientations differ, the complete comparison has three branches:

$$A > B: \quad (o_x, \mathrm{normalizePositive}(A - B, D)),$$

$$B > A: \quad (o_y, \mathrm{normalizePositive}(B - A, D)),$$

$$A = B: \quad \mathrm{EmptyOne}.$$

Strict comparison proves the retained numerator positive in each nonempty branch. For the left-held remainder z=n_z/d_z, normalization proves

$$(A - B)d_z = n_z D,$$

while exact removal reconstructs the original common support:

$$Bd_z + n_z D = Ad_z.$$

The right-held branch gives the symmetric identities. The result is Empty One exactly when the cross-products coincide. Equality, left dominance and right dominance therefore exhaust the original positive-rational domain. Empty One is an additive identity, and the source subtraction is

$$x \ominus y = x \oplus \mathrm{reverse}(y).$$

In particular, x⊖x is Empty One for every admitted scalar, including Empty One itself. Both the operation and its result retain the original held-orientation convention.

For an admitted positive forward duration t=a/b, the original division first constructs the exact reciprocal b/a and then multiplies. If the numerator magnitude is m=n/d, the resulting reduced magnitude z satisfies

$$(nb)d_z = n_z(da), \qquad n_z > 0.$$

Division preserves the numerator's held orientation; an Empty One numerator remains Empty One. The positive-duration type excludes division by structural absence, matching the source boundary. These statements are universal native proofs over the complete admitted scalar domain. A separate fresh execution replay compares 18,258 results with the unchanged pinned V3 operations: 51 reversals, 2,601 additions, 2,601 subtractions and 13,005 positive-duration rates, including both held orientations and exact cancellation. This finite replay checks execution correspondence; the universal arithmetic identities are established by the kernel proofs.

> Source, proof and provenance: V3 add_scalar, values.py:L158; V3 subtract_scalar, values.py:L172; V3 multiply_scalar, values.py:L176; V3 divide_scalar, values.py:L189; Lean opposedEqual, FoldScalar.lean:L112; Lean opposedEmptyIff, FoldScalar.lean:L118; Timeline: Turn 484, line 4158; Timeline: Turn 485, line 4176.

373. Acceleration from the complete velocity comparison

An acceleration record has two complete observed velocity endpoints E_before and E_after, a source-bound generated path between those very endpoints, and a held reference recurrence. The endpoint velocities retain the displacement support, exact separation, original motion interval, reference recurrence, observation and held orientation defined in Sections 363–366. The comparison does not replace them with an answer-only rate.

The two displacement orientation labels are renamed bijectively as forward-held and counter-held for the original scalar operations. Both round-trip identities are proved. Reversing the displacement orientation corresponds exactly to scalar reversal, with the speed unchanged.

Let N>0 be the generated comparison-path transition count and K>0 the held reference recurrence count. The duration τ=n_τ/d_τ is obtained by the original exact normalization:

$$\tau = \mathrm{normalizePositive}(N, K), \qquad N d_\tau = n_\tau K.$$

The acceleration value is then computed, rather than supplied as an independent field:

$$\Delta v = v_{\mathrm{after}} \ominus v_{\mathrm{before}}, \qquad a = \Delta v \,/\, \tau.$$

The difference is the complete oriented operation of Section 372. If it is Empty One, division retains Empty One. If it is the held positive magnitude m=n_m/d_m, the reduced rate n_a/d_a retains its orientation and satisfies

$$(n_m d_\tau) d_a = n_a (d_m n_\tau).$$

Together with Nd_τ=n_τK and positivity, this is exactly the generated comparison m/(N/K), without a fitted scale. An unchanged complete velocity gives Empty One by universal self-subtraction. Different endpoint carriers or different retained source/action ledgers distinguish the complete acceleration records even when a numerical projection happens to agree. The original path keeps all source-bound transitions and endpoint observations.

The acceleration entry is complete against V3. Its full 256-form program, all original controls and hashes, complete 40,801-byte empirical comparison and four independent-validator cases pass. Its mathematical construction uses the same positive rational and held-orientation domain as the source, and its native declarations have zero axiom dependencies.

> Source, proof and provenance: V3 ACCELERATION, mechanics_laws.py:L104; Lean unchangedIsEmptyOne, MechanicalAcceleration.lean:L72; Lean exactChangeRate, MechanicalAcceleration.lean:L62; Timeline: Turn 484, line 4158.

374. Force from source-bound momentum transfer

The momentum carrier of Section 371 retains every inertial occurrence with its complete velocity. For inertial content m and speed u, its exact magnitude is their normalized product, while its orientation remains held by the velocity. A force record retains two complete observed momentum endpoints P_before and P_after, the actual generated path joining them, every source/action record on that path and the reference recurrence.

For its generated positive duration τ=N/K, force is computed using the same source operations:

$$\Delta p = p_{\text{after}} \ominus p_{\text{before}}, \qquad F = \Delta p \,/\, \tau.$$

Thus an unchanged complete momentum gives structural Empty One. For a nonempty transfer magnitude r=n_r/d_r and τ=n_τ/d_τ, the normalized force n_F/d_F retains the transfer orientation and obeys

$$(n_r d_\tau) d_F = n_F (d_r n_\tau), \qquad N d_\tau = n_\tau K.$$

These identities retain the full momentum difference and generated interval. Source provenance remains in the complete path, not in an independent interaction coefficient. The before and after momentum records retain their inertial supports and complete motion labels; changing either endpoint or its source/action ledger distinguishes the force record.

The force entry is complete against V3. The complete 256-form source program reproduces every decision, hash, control and registration field, and the complete 40,801-byte blind empirical validator retains both source rows and every output field. The original independent validator accepts the native result and rejects an omitted candidate, an additional survivor and a failed control. All native definitions and theorems used for these results have zero axiom dependencies.

> Source, proof and provenance: V3 FORCE, mechanics_laws.py:L131; Lean unchangedIsEmptyOne, MechanicalForce.lean:L50; Lean positiveChangeRateExact, MechanicalForce.lean:L57; Timeline: Turn 484, line 4164.

375. Mechanical work and retained energy transfer

The source composes transferred force with the complete held displacement. Let F be the force carrier of Section 374 and X the displacement carrier of Section 365. The work record retains both F and X. Its value is computed by the existing V3 oriented product:

$$W = F \otimes X.$$

For two positive magnitudes f=n_f/d_f and x=n_x/d_x, normalization gives

$$w = \text{normalizePositive}(n_f n_x, d_f d_x), \qquad (n_f n_x) d_w = n_w (d_f d_x), \qquad n_w > 0.$$

The held-orientation rule is the original multiply_scalar rule: matching orientations give forward-held, and differing orientations give counter-held. Its four cases are exhaustive. Reversing one factor reverses the product orientation without changing its positive magnitude; reversing both factors preserves the product. If either factor is structural Empty One, the product is Empty One. The same admitted source arithmetic therefore handles the absence boundary without introducing a numerical zero or negative magnitude.

The composition also retains the complete interaction path. Let L be the ordered list of source-bound transfer rows in F's generated path. Pairing each row with X gives

$$C(L, X) = \mathrm{map}(\ell \mapsto (\ell, X), L).$$

The native pair-cell identities prove

$$\mathrm{map}(\pi_1, C(L, X)) = L, \qquad |C(L, X)| = |L|,$$

$$(\ell, Y) \in C(L, X) \iff \ell \in L \wedge Y = X.$$

Thus every supplied transfer row retains the complete displacement, with its references, support, observation and orientation. The work record also retains the full force endpoints and source/action ledger. Changing the force or displacement distinguishes the complete work records even when their projected values agree.

The source's energy transfer is this retained work/change record. Its computed energy change is W; its force and displacement reconstruct the original complete record. No independent energy answer or fitted interaction coefficient is supplied to the construction.

The energy-dimension closure follows from the already derived product and quotient paths. For arbitrary held dimension descriptions M, L and T,

$$D_F = (M \cdot (L/T))/T, \qquad D_W = D_F \cdot L,$$

$$D_E = ((M \cdot L) \cdot L)/(T \cdot T).$$

For every generated kind, the numerator and denominator occurrence counts of D_W and D_E agree. The positive-side calculation reduces to

$$(a + (b + c)) + c + b = (a + b) + b + (c + c),$$

where a is the positive M count, b the positive L count and c the negative T count. The negative-side calculation uses the corresponding opposite counts and the identical associativity/commutativity identity. Hence the full description closes in the energy dimension, conventionally written ML²/T², while retaining both original descriptions. These occurrence counts are bookkeeping for finite dimension paths, not new physical scalars.

The work/energy entry is complete against pinned V3. Its complete 256-form program, original controls and hashes, full 40,801-byte blind empirical method and four independent-validator cases pass. The native scalar-product and work declarations report no axiom dependencies. A separate fresh arithmetic

replay checks all 20,859 outputs against unchanged V3, including 2,601 products covering both orientations, Empty One and every supplied numerator/denominator pair from one through five. Universal multiplication and composition identities are proved independently of that finite replay.

> Source, proof and provenance: V3 WORK_ENERGY, mechanics_laws.py:L140; Lean positiveCompositionExact, MechanicalWorkEnergy.lean:L46; Lean energyDimensionClosure, MechanicalWorkEnergy.lean:L91; Timeline: Turn 485, line 4176.

376. Power as the rate of the retained energy transfer

Power retains the complete energy-transfer record of Section 375 and the generated interaction interval belonging to its force path. Let N>0 be that path's transition count and K>0 the held reference recurrence count. The exact duration τ=n_τ/d_τ satisfies

$$\tau = \mathrm{normalizePositive}(N, K), \qquad N d_\tau = n_\tau K.$$

For energy transfer E, power is computed by the original positive-duration division:

$$P = E/\tau.$$

If E is a held positive magnitude e=n_e/d_e, the original reciprocal/product operations give the reduced rate n_P/d_P with

$$(n_e d_\tau) d_P = n_P (d_e n_\tau), \qquad n_P > 0.$$

The energy orientation is preserved by division by the positive forward duration. These identities, together with the generated-duration identity, establish the exact transfer per recurrence without an additional scale. Structural absence of energy transfer remains Empty One.

The power carrier retains the complete force, displacement, ordered transition path, endpoint observations and held recurrence. Its force and displacement reconstruct the entire original transfer record. This binds the numerator and duration to the same interaction and preserves the source's ordered energy-transfer provenance.

The power entry is complete in source dependency order after work/energy. Its complete 256-form source program reproduces every original decision, output field, hash, control and registration. Its full 40,801-byte blind empirical comparison and all four original independent-validator cases pass. The native quotient, duration and complete-record proofs have no axiom dependencies.

> Source, proof and provenance: V3 POWER, mechanics_laws.py:L149; Lean positiveEnergyRate, MechanicalPower.lean:L25; Lean exactEnergyRate, MechanicalPower.lean:L15; Timeline: Turn 485, line 4178.

377. Angular motion: exact phase in a retained closed recurrence

Let C be a finite observed recurrence with N > 0 source-bound transitions, returning to the same complete carrier. Its traversal retains a held orientation h. A position is an observed event together with an initial path P and remaining path Q satisfying P joined to Q = C. Consequently,

$$\mathrm{changes}(P) + + \mathrm{changes}(Q) = \mathrm{changes}(C)$$

k + r = N, k = count(P), r = count(Q), k ≤ N.

This decomposition retains every transition and its order. The phase magnitude at a present position is the normalized exact part k/N; its orientation is h. An initial path with no transition has structural Empty One. The terminal position has phase magnitude N/N = 1, while its observed carrier equals the starting carrier. Thus returning to the same carrier does not discard the completed traversal.

If normalize(k,N) = (u,v), Lean proves k v = u N and u ≤ v. No irrational angular unit enters this construction. For two ordered positions, the connecting observed interval I must satisfy P_before joined to I = P_after. The full interval ledger is retained, and k_before + count(I) = k_after. With a held clock recurrence of M transitions,

tau = count(I)/M,

$$\Delta\phi = \phi_{\text{after}} \ominus \phi_{\text{before}}$$

$$\omega = \text{dividePositive}(\Delta\phi, \tau)$$

The subtraction and division are the previously derived exact held-orientation scalar operations. For a present phase change of magnitude a/b and positive duration c/d, the normalized rate p/q satisfies (a d) q = p (b c); its held change orientation is preserved. The mathematical construction covers every finite admitted path decomposition, independently of the finite candidate grammar.

Here $x \ominus y = x \oplus \text{reverse}(y)$ is the oriented FoldScalar operation of §372; equality yields EmptyOne, and division retains that case.

> Source, proof and provenance: V3 ANGULAR_MOTION, mechanics_laws.py:L158; Lean phaseIsPart, MechanicalAngularMotion.lean:L96; Lean closedCarrierIdentity, MechanicalAngularMotion.lean:L108; Timeline: Turn 486, line 4192.

378. Angular momentum and source-bound torque

The angular-momentum carrier retains the complete lever separation r, linear momentum p and cyclic traversal. The actual linear-momentum trajectory is included in that same closed path through an explicit leading-path/trajectory/trailing-path decomposition. Its entire ordered transition list is therefore recoverable from the retained cycle.

Writing oriented quantities as (h,x), the exact composition is

$$L = \text{multiply}(r, p)$$

$$|L| = \text{normalize}(r_n p_n, r_d p_d)$$

The orientation follows the original scalar product table: equal held orientations give forward-held composition; unequal held orientations give counter-held composition. The cyclic traversal orientation is retained in the complete carrier. The product identity is (r_n p_n) L_d = L_n (r_d p_d). Neither the momentum record nor the reference-relative separation is replaced by the result's magnitude.

Torque retains two observed angular-momentum carriers, their complete connecting source/action trace and a held clock recurrence. For a present interval,

tau = count(interval)/count(clock),

$$\Delta L = L_{\text{after}} \ominus L_{\text{before}}$$

$$T = \mathrm{dividePositive}(\Delta L, \tau)$$

Exact quotient identities hold for every present change. An unchanged angular-momentum carrier gives structural Empty One. Changes to either endpoint or to the source/action ledger distinguish the interaction records even when a scalar result agrees.

Here $x \ominus y = x \oplus \mathrm{reverse}(y)$ is the oriented FoldScalar operation of §372; equality yields EmptyOne, and division retains that case.

> Source, proof and provenance: V3 ANGULAR_MOMENTUM, mechanics_laws.py:L167; Lean exactComposition, MechanicalAngularMomentum.lean:L29; Lean exactChangeRate, MechanicalAngularMomentum.lean:L84; Timeline: Turn 486, line 4194.

379. Complete constituent weighting and the composite centre

Each constituent retains its complete momentum carrier, positive inertial content m_i and reference-relative oriented separation x_i. Every weighted term is generated by the original scalar product,

$$w_i = \mathrm{multiply}(m_i, x_i)$$

$$W = w_1 \oplus (w_2 \oplus (\cdots \oplus w_n))$$

$$M = m_1 + (m_2 + (\ldots + m_n))$$

$$X = W / M$$

The constituent list is nonempty and every m_i is positive, so M is positive. Each normalized weighted product obeys its cross-product identity. Each mass-addition step normalizes (a d + c b)/(b d), retaining (a d + c b) q = p (b d) for its normalized result p/q. The final centre retains the exact positive-division identity and the orientation of W. Cancellation of the weighted support yields structural Empty One without introducing a zero-valued scientific magnitude.

The weighting ledger stores (constituent_i,w_i) for every occurrence. Projecting its first coordinates recovers the original constituent list exactly. Composite momentum retains the disjoint ordered junction of all constituent momentum records, including repeated-valued occurrences.

A grouping is a finite tree of constituents whose junction joins both complete leaf lists. For any groups A, B and C,

$$\mathrm{leaves}(\mathrm{join}(\mathrm{join}(A, B), C)) = (\mathrm{leaves}(A) + + \mathrm{leaves}(B)) + + \mathrm{leaves}(C)$$

$$= \mathrm{leaves}(A) + + (\mathrm{leaves}(B) + + \mathrm{leaves}(C))$$

$$= \mathrm{leaves}(\mathrm{join}(A, \mathrm{join}(B, C)))$$

The assembled complete carrier is equal under this regrouping; its weighted centre and momentum junction are consequently equal. Evaluation retains the complete leaf order throughout, so regrouping cannot omit, duplicate or silently reorder a constituent.

> Source, proof and provenance: V3 COMPOSITE_CENTRE, mechanics_laws.py:L176; Lean exactWeightedPart, MechanicalCompositeCentre.lean:L43; Lean centreExact, MechanicalCompositeCentre.lean:L90; Timeline: Turn 486, line 4196.

380. Closed mechanical transfer and exact carrier conservation

A closed interaction ledger records each transfer occurrence as (source, destination, carrier, source identity, action record). Its carrier is the entire derived momentum or energy-transfer record. Distinct occurrences remain distinct even when their carrier values agree.

For a ledger indexed by its finite occurrences I, let c_i be the retained carrier. The departure and arrival supports are

$$C_{\text{out}} = [c_i \mid i \in I], \qquad C_{\text{in}} = [c_i \mid i \in I].$$

Each source occurrence is paired with its destination occurrence, and the reverse pairing returns that source occurrence. Both inverse identities are proved for every finite occurrence. Projection from the full departure and arrival ledgers proves

$$C_{\text{out}} = C_{\text{in}}$$

$$\text{length}(C_{\text{out}}) = \text{length}(C_{\text{in}})$$

$$F(C_{\text{out}}) = F(C_{\text{in}})$$

for every aggregate F on the complete retained carriers. This establishes conservation of the complete support, not merely equality of a selected scalar total. A successor transfer appends its source, destination and full carrier without altering the earlier ledger. The result applies within V3's declared closed-transfer boundary: every recorded outgoing carrier has its recorded incoming counterpart. Source, destination and provenance remain separately distinguishable.

> Source, proof and provenance: V3 CONSERVATION, mechanics_laws.py:L185; Lean completeConservation, MechanicalConservation.lean:L37; Lean changedProvenanceDistinguished, MechanicalConservation.lean:L78; Timeline: Turn 486, line 4198.

381. Reversible restoration, finite recurrence and exact period

A restoring step retains its before/after observations and its source-bound action. Its generated response reverses the held displacement orientation while retaining the complete separation and reference supports. Let R denote this reversal. The constructive identities are

$$R(R(d)) = d, R(d) \neq d$$

Thus two consecutive restoring reversals return the full carrier, while the intermediate held orientation differs. For the observed path d to R(d) to d, Lean retains all three observations, both source/action records and exactly two transitions. Relative to a clock recurrence of M transitions, its exact period is 2/M. If its normalized period is p/q, then 2q = pM.

The construction also covers arbitrary finite source-bound restoring paths. Induction over appended transitions proves that the endpoint carrier after k transitions is R iterated k times on the starting carrier. For every natural count n,

$$R \text{ iterated } (2n) \text{ times on } d = d$$

Every present path with 2n transitions therefore supplies a closed observed recurrence. Its period is the exact transition-count/reference-count ratio, with the corresponding cross-product identity. All intermediate observations and actions remain in the path; closure never discards them. The angular-phase construction applies to this retained recurrence, giving exact held phase parts without an imported angular scalar or fitted period.

> Source, proof and provenance: V3 CONSTRAINT_OSCILLATION, mechanics_laws.py:L194; Lean phaseUsesRetainedRecurrence, MechanicalConstraintOscillation.lean:L81; Lean finiteClosedRestoration, MechanicalConstraintOscillation.lean:L121; Timeline: Turn 486, line 4200.

382. Complete source-bound field support

A localized source retains its complete observed carrier and its generated location. Each response retains an accessible cell identity, a complete observed response and an ordered propagation path from that source location to the response location. For a finite support list S and response ledger R, admissibility retains the exact identity

$$\mathrm{map}(cell, R) = S$$

Consequently every generated cell is represented, no ungenerated cell is inserted, and length(R) = length(S). The construction applies to every finite generated support and source-bound response assignment that preserves those cell identities. Repeated-valued observations remain separate occurrences.

Appending a response r extends the complete carrier by

$$S' = S + +[\mathrm{cell}(r)]$$

$$R' = R + +[r]$$

$$\mathrm{map}(cell, R') = S'$$

Earlier response and propagation records are unchanged. Projection of the transition ledger recovers every source identity and action record. An ordered propagation path with k transitions retains k + 1 observed events. Changes to a response observation or its support cell distinguish the complete records even when a projected value agrees.

> Source, proof and provenance: V3 SOURCE_RESPONSE, field_laws.py:L41; Lean sourceTraceRetained, FieldSourceResponse.lean:L62; Lean responseCount, FieldSourceResponse.lean:L33; Timeline: Turn 487, line 4212.

383. Geometric dilution and the unique conserved share

Let the positive source support be s = a/b, and let its complete equivalent boundary contain N > 0 generated cells. The response assigned to every cell is the reduced exact ratio

$$u = \mathrm{normalize}(a, bN)$$

If u = p/q, normalization proves

$$aq = pbN$$

The complete response ledger pairs each boundary cell with u. Projecting the cell coordinates recovers the entire boundary, and the response count is N. Every retained response has the same share; choosing a different profile cannot satisfy that complete equal-cell assignment.

Uniqueness follows from conservation. If a second positive reduced share v = r/t satisfies a t = r b N, combining that identity with a q = p b N gives

$$(bN)rq = (bN)pt$$

Since bN is positive, cancellation gives r q = p t. Reduced positive-fraction identity then yields v = u. The gcd and positive-cancellation steps are proved constructively. No fitted boundary profile enters this result.

For every positive generated count k, common replication also preserves the reduced relation:

$$\mathrm{normalize}(ka, kb) = \mathrm{normalize}(a, b)$$

The same exact cancellation supplies the reference-count invariance used by normalized field responses.

> Source, proof and provenance: V3 GEOMETRIC_DILUTION, field_laws.py:L47; Lean noSelectedProfile, FieldGeometricDilution.lean:L30; Lean shareUnique, FieldGeometricDilution.lean:L56; Timeline: Turn 487, line 4214.

384. Weak relational gravitation and complete inertial-source response

At the registered weak relational boundary, the source is the complete inertial momentum carrier, including its positive inertial content m. Its field retains every generated accessible response cell and the full propagation trace connecting that cell to the source. The geometric source share at each of N equivalent boundary cells is m/N, with the exact conservation and uniqueness identities of the preceding section.

Each observed response is a complete free-motion change carrier: two observed velocities, their source-bound connecting interval and a held clock recurrence. Its generated duration and change rate are

tau = count(interval)/count(clock),

$$\Delta v = v_{\mathrm{after}} \ \mathrm{minus} \ v_{\mathrm{before}}$$

$$a = \Delta v / \tau$$

All such response records remain paired with the same retained inertial source and its generated geometric share. The construction is universal over the complete accessible support. An additional description label leaves the physical source-response carrier unchanged; it does not select a separate charge-family coupling. The geometric source assignment and observed free-motion change remain distinct, fully retained parts of the same weak-boundary record.

The registered post-seal comparison preserves both GWOSC source/detector locators, their propagation identities and the original changed-row rejection. Those external records do not supply the source carrier or select its geometric relation.

> Source, proof and provenance: V3 GRAVITATIONAL_INTERACTION, field_laws.py:L53; Lean everyResponseHasSourceTrace, FieldGravitation.lean:L45; Lean responseIsObservedMotionChange, FieldGravitation.lean:L37; Timeline: Turn 487, line 4216.

385. Conserved electric distinction and held orientation

The electric carrier is a positive exact amount q together with one of the two held Fold labels. Both fibres are retained, they are distinct, and they exhaust the admitted orientation type. Reversal exchanges the labels while preserving the positive amount:

$$\mathrm{reverse}(\mathrm{reverse}(q,h)) = (q,h)$$

$$\mathrm{amount}(\mathrm{reverse}(q,h)) = q > 0$$

The two interaction classes distinguish equal and opposite held labels. Their complete case table is

first / first -> equal-held,
first / second -> opposite-held,
second / first -> opposite-held,
second / second -> equal-held.

Reversing one carrier exchanges the classes; reversing both preserves the class. The classification never changes the positive quantity support. For a closed source/destination transfer ledger, complete electric carriers are conserved occurrence by occurrence. Projecting their held labels therefore gives the same outgoing and incoming label lists. This retains exact conservation of both amount and orientation.

> Source, proof and provenance: V3 ELECTRIC_DISTINCTION, field_laws.py:L59; Lean closedHeldLabelConservation, FieldElectricDistinction.lean:L60; Lean twoFibresExhaust, FieldElectricDistinction.lean:L24; Timeline: Turn 487, line 4218.

386. Charge-normalized electric field and potential difference

An electric interaction retains its held charge carrier Q, complete force/work transfer and both reference-relative displacement endpoints. V3 division takes the reciprocal of the right carrier and composes it with the left carrier. For positive magnitude q_n/q_d, the reciprocal retains the charge orientation and has magnitude q_d/q_n. Thus

$$E = \mathrm{force}/Q$$

$$\Delta V = \mathrm{work}/Q$$

$$\mathrm{work} = \mathrm{multiply}(\mathrm{force}, \mathrm{heldDisplacement})$$

If the transferred magnitude is a/b, the normalized quotient p/t satisfies

$$(aq_d)t = p(bq_n)$$

Its orientation is the original scalar-product orientation of the transferred carrier and held charge. Reversing the charge reverses the normalized response. Structural Empty One transfer remains Empty One, while the divisor is the retained positive charge amount. The complete source-bound work, force and endpoint records remain available alongside the resulting value.

A common generated replication of the two reference comparison counts cancels exactly, normalize(kn,kd) = normalize(n,d). Consequently the normalized relation is unchanged by that common reference replication. The implementation uses V3's reciprocal-and-product operation directly; a fresh

comparison covers all 2,550 combinations of Empty One or the two held orientations of numerator/denominator pairs from 1 through 5, divided by every corresponding nonempty held charge. General normalization and reversal identities are established separately by native Lean proofs.

The next magnetic source/motion component uses the same held-orientation composition r(q,v) = q composed with v. Its native proofs establish r(reverse(q),v) = reverse(r(q,v)), r(q,reverse(v)) = reverse(r(q,v)), and r(reverse(q),reverse(v)) = r(q,v). With the original forward/forward unit convention, these two reversal laws determine all four orientation pairs uniquely. The complete transported-source and response-ledger construction is given in Section 388.

> Source, proof and provenance: V3 ELECTRIC_POTENTIAL, field_laws.py:L65; Lean changedWorkDistinguished, FieldElectricPotential.lean:L77; Lean normalizationPositive, FieldElectricPotential.lean:L15; Timeline: Turn 487, line 4220.

387. Completed Hubble calibration from the generated partition and first return

The generator-three carrier and binary Fold distinction give the complete formed/open partition

$$matter = 1/3, vacuum = 2/3, matter + vacuum = 1$$

The binary tower over the generator is 2^3 = 8. Dividing the open share by that tower yields the leading correction and calibration ratio

$$delta_0 = (2/3)/8 = 1/12$$

$$R_0 = 1 + delta_0 = 13/12$$

The next complete support has 3^4 = 81 members. Its least binary cover has depth 7: 81 ≤ 2^7, while every positive smaller depth has insufficient support. The resulting recurrence floor is 2^7 - 1 = 127. Starting from 1/127, the complete first-return orbit is

2/127, 4/127, 8/127, 16/127, 32/127, 64/127, 1/127.

Lean proves the ordered first-return property, including absence of an earlier return. The single-recurrence deepening therefore supplies

$$delta_1 = ((2/3) + (1/127))/8 = 257/3048$$

$$R_1 = 1 + delta_1 = 3305/3048$$

The structural construction takes no measured value as input. Its generated partition, least cover, exact ordered recurrence and rational assembly are verified before the separate empirical comparison.

The original post-seal boundary compares the late/early ratio from the registered Planck and SH0ES abstract records. It retains the respective route qualifications, common units and uncertainty inscriptions. With early value E and uncertainty u_E, and late value L and uncertainty u_L, the outward interval is

[(L - u_L)/(E + u_E), (L + u_L)/(E - u_E)].

The source requires positive uncertainty bounds before taking these quotients. For the pinned inscriptions E = 67.4, u_E = 0.5, L = 73.04 and u_L = 1.04, the complete interval is [720/679, 3704/3345], and both sealed ratios 13/12 and 3305/3048 lie inside it. These decimal inscriptions are parsed only in the measured-record comparison; they do not enter structural forcing.

The completed entry retains every one of its 2,048 candidate forms, original decisions and proof hashes, complete controls, and all 1,316 source bytes in the empirical validator. The independent validator accepts the original full result and rejects an omitted candidate, an additional survivor and a failed control. The source route, uncertainty and custody qualifications remain part of the stated result.

The source grammar tests carrier, occupancy, partition, support, leading, depth, orbit, deepening, target, provenance and extension. Its ordered coordinate product has $2^{11} = 2048$ forms. The one-third/two-thirds partition, depth-three support and depth-seven first-return floor fix the admitted leading and deepened ratios; both external routes remain unavailable until sealing. The coordinate-membership and acceptance argument of Section 317 applies to this product, with the following native completeness and uniqueness proofs. candidatesComplete, HubbleGrammarBase.lean:L151; uniqueSurvivor, HubbleGrammar.lean:L397.

Grammar source: V3 HUBBLE_CALIBRATION_SPEC, hubble_calibration_law.py:L57.

> Source, proof and provenance: V3 hubble_calibration_structure, hubble_calibration_law.py:L27; V3 hubble_ratio_interval, hubble_calibration_validation.py:L30; Lean hubbleFirstReturn, CosmicRatios.lean:L112; Lean darkFirstReturn, CosmicRatios.lean:L106; Lean containedBounds, CosmicIntervals.lean:L78; Lean originalHubbleWitnesses, CosmicIntervals.lean:L75; Timeline: Turn 487, line 4237.

388. Oriented electric transport and the magnetic response record

The magnetic construction retains a complete electric charge q and an observed motion v, including both endpoints, positive separation, clock comparison and the entire source/action path. The held orientation takes two values, forward and counter, with reversal j. In particular, j(j(h)) = h. The response orientation is the same two-fibre composition used by the exact Fold scalar carrier:

$$\text{forward composed with forward} = \text{forward}$$

$$\text{forward composed with counter} = \text{counter}$$

$$\text{counter composed with forward} = \text{counter}$$

$$\text{counter composed with counter} = \text{forward}$$

Writing r(q,v) for this composition of held charge and held motion gives

$$r(j(q), v) = j(r(q, v))$$

$$r(q, j(v)) = j(r(q, v))$$

$$r(j(q), j(v)) = r(q, v)$$

These identities follow by all four orientation cases. Conversely, the forward/forward convention and the two separate reversal identities determine the response on every pair: the forward/counter value follows by motion reversal, the counter/forward value by charge reversal, and the counter/counter value by both. No fitted magnitude enters this determination.

The source distinguishes the along-motion role from the transverse response role. The magnetic response record retains the latter together with the transported charge, full motion and observed response. The roles remain distinct, so the response cannot be relabeled as the along-motion record. This is the original held relational description; no numerical angle or imported metric field equation enters the construction. Reversing either held orientation preserves the positive charge amount and the complete propagation path while reversing the response orientation.

For a finite response ledger R = [r_1,...,r_n], the generated ledger retains each tuple (transport_i, observation_i, role_i, held_i). Projection recovers [transport_1,...,transport_n] exactly. Appending one accessible response appends its complete tuple and preserves every preceding record. For each retained path p, projection of its complete change rows onto source/action pairs equals sourceRecords(p); the number of transitions is positive by the transported-motion boundary.

> Source, proof and provenance: V3 MAGNETIC, field_laws.py:L71; Lean uniqueTwoReversalComposition, FieldMagneticOrientation.lean:L22; Lean reversingBoth, FieldMagneticOrientation.lean:L17; Lean chargeReversal, FieldMagnetic.lean:L44; Lean motionReversal, FieldMagnetic.lean:L52; Timeline: Turn 488, line 4245.

389. Linked magnetic change and complete cyclic electric work

A linked magnetic change retains the before and after response records, source identity and original change record. The closed generated boundary is an ordered recurrence p with an explicitly held traversal. Its terminal carrier equals its initial carrier. Every transition s of p retains its full observed mechanical work transfer W(s), including force and displacement, rather than only an extracted number.

If L denotes the complete linked change and h the held traversal, the cyclic record is

C(L,p,W) = [(L,s,h,W(s)) : s in changes(p)].

Projection onto the transition component yields exactly changes(p). Therefore every original boundary occurrence is present, no additional transition is admitted, and the complete source/action projection equals sourceRecords(p). Projection onto the linked-change component retains L at every occurrence, preserving the common change provenance around the entire cycle.

The induced electric interaction for each transition uses the charge q carried by the after-response of L and the complete work W(s):

$$E_L(s) = (q, W(s))$$

$$\Delta V_L(s) = \mathrm{normalizeCharge}(\mathrm{energyChange}(W(s)), q)$$

Thus no additional electric source is introduced by the cyclic construction. Both endpoints and the force/work trace remain available in W(s), and the potential operation is the previously established exact charge normalization. A different linked change or a different complete boundary gives a distinct cyclic record. The construction preserves the source's relational work correspondence without importing a differential flux equation.

> Source, proof and provenance: V3 INDUCTION, field_laws.py:L77; Lean inducedPotentialUsesCompleteWork, FieldInduction.lean:L76; Lean completeSourceProvenance, FieldInduction.lean:L58; Timeline: Turn 488, line 4247.

390. Common electric-magnetic source and propagation closure

The joint carrier retains the complete electric interaction, magnetic response and linked cyclic induction record. Their source correspondence is explicit:

$$q_{\mathrm{magnetic}} = q_{\mathrm{electric}}$$

$$\mathrm{magneticAfter}(\mathrm{induction}) = \mathrm{magnetic}$$

The electric observation projection retains the full electric interaction together with its charge-normalized field and potential. The magnetic observation projection retains the full transported source/motion response, transverse role and held orientation. Applying induction's electric-response construction to any boundary transition therefore returns the same conserved charge as the electric projection.

The opposite construction transports the original electric charge with the observed post-transfer velocity already retained in its force record. Its magnetic projection preserves both that charge and that exact motion record. The transverse response remains the original source-defined role of the oriented transport.

For the complete common propagation list P = [s_1,...,s_n], retain both response legs at every occurrence:

$$J(P) = \mathrm{concat}_{i=1}^{n}[(s_i, e \to m), (s_i, m \to e)].$$

Here $e \to m$ and $m \to e$ name the electric-to-magnetic and magnetic-to-electric response roles. Projecting the electric legs recovers P, and projecting the magnetic legs also recovers P. Both projections consequently recover the original complete closed boundary trace. Structural induction gives |J(P)| = |P| + |P|. The base list is empty; appending a source occurrence adds its two retained legs. The construction records the two response families on one common propagation history, retaining their different observation roles. A changed electric interaction or changed magnetic record is distinguished by the joint carrier.

Additional proof bindings for the preceding conclusions: Lean everyElectricLeg, FieldElectromagnetic.lean:L49; Lean everyMagneticLeg, FieldElectromagnetic.lean:L55; Lean completeCycleCount, FieldElectromagnetic.lean:L71.

> Source, proof and provenance: V3 ELECTROMAGNETIC_COMPOSITION, field_laws.py:L83; Lean changingMagneticReturnsSameElectricSource, FieldElectromagnetic.lean:L87; Lean electricTransportRetainsMotion, FieldElectromagnetic.lean:L102; Timeline: Turn 488, line 4249.

391. Gauge equivalence through complete observable records

A description consists of an observable record O and an unobserved internal-label sequence lambda. The observable contains the observed source record, complete ordered observations and complete ordered transfers:

$$O = (S, [o_1, ..., o_m], [t_1, ..., t_n])$$

$$D = (O, \lambda)$$

The original physical equivalence is

D_a ~ D_b if and only if O_a = O_b.

Equality of observable records immediately gives reflexivity, symmetry and transitivity. Arbitrarily replacing lambda preserves O and therefore preserves equivalence. Conversely, equality of all three observable components is sufficient to reconstruct equality of O; retaining only selected observations or transfers is not sufficient for this construction.

The canonical projection pi(D) = O has the exact kernel

pi(D_a) = pi(D_b) if and only if D_a ~ D_b.

Every observable is represented by (O,[]). Consequently a canonical representative requires neither a chosen privileged internal labeling nor a quotient axiom. A changed observation sequence or changed transfer sequence excludes equivalence. All of these statements preserve order and multiplicity, including repeated equal-valued observations.

> Source, proof and provenance: V3 GAUGE_EQUIVALENCE, field_laws.py:L89; Lean changedTransferExcluded, FieldGaugeEquivalence.lean:L56; Lean completeRecordsSuffice, FieldGaugeEquivalence.lean:L42; Timeline: Turn 488, line 4251.

392. Generated adjacency and completed causal accessibility

Let A(a,b) be the generated adjacency relation on observed events. Source-bound propagation is constructed by the following rules:

one(a) : a reaches a,
append(p,c,A(b,d)) : a reaches d,

where p reaches b from a and c is the complete recorded change from b to d. Erasing only the adjacency certificates yields the existing full mechanical event path. Structural induction proves that every retained change row has adjacent before and after events. For a noninitial destination, case analysis on the final propagation constructor supplies its actual adjacent predecessor. A remote change cannot be admitted through an unrecorded predecessor in this representation.

Composition joins two compatible paths, and its erased trace equals the complete mechanical join. Thus composition retains the ordered source/action records of both pieces. For a finite list of completed arrivals C, accessibility at that event is

accessible(C) = [destination(r) : r in C].

Membership yields an actual arrival occurrence in C with that destination and its retained adjacent propagation proof. Conversely, a destination outside this complete list has no arrival occurrence in C. Appending one completed arrival gives

$$\mathrm{accessible}(C + +[r]) = \mathrm{accessible}(C) + +[\mathrm{destination}(r)]$$

This is the completed-path boundary of the source claim. It does not replace completed accessibility by all paths that might be generated at a later event.

The radiative carrier retains the full source identity, energy and momentum records, recurrence and held polarization while local source contact is absent. For an observation sequence R, its detached ledger is [(carrier, none, r) : r in R]. Projection recovers R exactly, the complete carrier remains at every occurrence, and appending an observation preserves all prior records. The exact recurrence frequency relative to a retained reference is count(reference)/count(recurrence), with positivity and normalized cross-product identity proved. The complete joint electromagnetic recurrence and frequency correspondence is developed in Section 394.

> Source, proof and provenance: V3 LOCALITY_CAUSALITY, field_laws.py:L95; Lean everyTransitionAdjacent, FieldLocality.lean:L29; Lean successorPreservesAccessibility, FieldLocality.lean:L84; Timeline: Turn 488, line 4253.

393. Terminal on-shell electroweak share

The terminal alpha construction has four rungs, generated by complete exhaustion of the three direction promotions. Let c = 3 be the independently forced generator and b = c - 1 = 2 its positive predecessor. The complete binary support of the four-rung object is

$$N = b^4 = 16$$

The native support construction generates all sixteen binary words. Holding the three generator directions applies exactly three positive predecessor steps to N, giving the running level

$$\ell = 16 - 3 = 13$$

The subtraction is the source's repeated positive-predecessor operation. Its general identity is predecessorSteps(r + k + 1,k) = r + 1. Boolean, nonpositive and exhausted inputs retain the original rejection boundaries; no signed proof magnitude is introduced.

At positive level ell, the charged support is the complete product of ell + b positions with itself, while the neutral support has b^b pair channels, each with the complete product of ell + 1 positions:

$$C(\ell) = (\ell + 2)^2$$

$$N_{\mathrm{neutral}}(\ell) = 4(\ell + 1)^2$$

$$s_{\mathrm{running}}(\ell) = C(\ell)/(C(\ell) + N_{\mathrm{neutral}}(\ell))$$

For ell = 13 these give C = 15^2 = 225 and N_neutral = 4 x 14^2 = 784. The two counts sum to 1,009, so

$$s_{\mathrm{running}}(13) = 225/1009$$

These are complete generated channel counts. The position construction contains each positive index through its bound exactly once. Its product has cardinality |A||B|, contains precisely the pairs from A and B and preserves distinctness. Consequently the charged and neutral counts arise from the complete support products rather than selected representative channels.

The sole terminal self-return crosses the complete support successor N + 1 = 17 once. With the already derived terminal alpha, the correction and on-shell share are

$$\alpha = 3676744786/503846395469$$

$$\delta_{\mathrm{EW}} = \alpha/17 = 3676744786/8565388722973$$

$$\sin^2(\theta_W) = 225/1009 + \delta_{\mathrm{EW}}$$

$$= 1930922298157999/8642477221479757$$

The positive complement gives

$$\cos^2(\theta_W) = 6711554923321758/8642477221479757$$

$$\sin^2(\theta_W) + \cos^2(\theta_W) = 1$$

Native Lean verifies the exact reduced rational identities, the strict positive-part boundaries and the complete channel construction. The terminal entry retains all 1,024 original candidate forms, control outcomes and closure records. Its registered evidence mode is formal: no measured value, uncertainty or

best-fit table enters these derivations.

Additional proof bindings for the preceding conclusions: Lean runningShareExact, ElectroweakTerminalRatio.lean:L54; Lean terminalAlphaExact, ElectroweakTerminalRatio.lean:L56; Lean sinSquaredExact, ElectroweakTerminalRatio.lean:L60; Lean completeShareOne, ElectroweakTerminalRatio.lean:L69.

> Source, proof and provenance: V3 terminal_electroweak_sin_squared, precision_value_laws_v1.py:L88; V3 terminal_electroweak_cos_squared, precision_value_laws_v1.py:L99; Lean negativeRejected, ElectroweakTerminalRatio.lean:L77; Lean booleanRejected, ElectroweakTerminalRatio.lean:L71; Lean sourceChannelCounts, ElectroweakChannels.lean:L101; Lean allPositions, ElectroweakChannels.lean:L30; Timeline: Turn 488, line 4245.

394. Detached radiative recurrence with its complete field carrier

A radiative carrier retains its source identity, complete energy-transfer and momentum records, closed recurrence and held polarization. The joint electric-magnetic carrier is retained explicitly. Its electric work record is the radiative energy record; its post-transfer momentum is the radiative momentum record; and its complete induction boundary is the radiative recurrence. These correspondences prevent an independently chosen energy, momentum or clock history from replacing the original field carrier.

For a recurrence path p, the generated response word is the complete two-leg word J(changes(p)) of Section 390. Its electric projection recovers changes(p), and its magnetic projection recovers the same changes(p). Both therefore recover the joint field's original propagation history. Repeating the word is finite successor generation:

$$W_0 = []$$

$$W_{n+1} = W_n + +J(\text{changes}(p))$$

$$|W_n| = n|J(\text{changes}(p))|$$

The length identity follows by induction: the empty word has no occurrences, and each successor appends one complete recurrence word. The recurrence's terminal carrier equals its initial carrier. Each appended cycle retains the same source-bound field projections without a new local source attachment.

For a complete observation sequence R, the detached observation ledger is

D(R) = [(C,none,r) : r in R],

where C is the complete retained carrier and none records absence of continued local source contact. Projection recovers R exactly. Carrier, source identity, full energy, full momentum and polarization remain present at every observation occurrence. Appending a record appends one complete tuple and preserves all earlier records. Absence of local contact does not erase the carrier's original source provenance.

With positive recurrence count N and positive held-reference count M, the exact frequency is normalize(M,N). Its reduced numerator a and denominator b satisfy M b = a N. No measured frequency enters this operation. The source's separate spectral-record comparison retains its original complete row and source-custody boundary.

> Source, proof and provenance: V3 RADIATION, field_laws.py:L101; Lean noContinuedLocalContact, FieldRadiation.lean:L44; Lean frequencyExact, FieldRadiation.lean:L85; Timeline: Turn 489, line 4268.

395. Interaction classes from complete source, carrier and response traces

The physical signature of an interaction description is the complete ordered triple

$$\Sigma = (\text{conserved source labels},\ \text{propagation carriers},\ \text{observable response traces})$$

Its descriptive name remains separate. Two descriptions belong to the same class exactly when their complete signatures are equal. Names alone cannot distinguish classes: (Sigma,name_a) and (Sigma,name_b) have the same canonical signature for every pair of names.

Native equality gives reflexivity, symmetry and transitivity. Equality of the complete signatures preserves each source, propagation and response sequence. Conversely, equality of all three sequences reconstructs equality of the signature. If any of the three complete sequences differs, equivalence is excluded.

For a finite input ledger [d_1,...,d_n], the canonical occurrence ledger is [Sigma(d_1),...,Sigma(d_n)]. Every original occurrence has its canonical class present, and every output occurrence comes from an original description. Equal signatures identify the same class even when multiple original occurrences or names remain in the ledger. The construction does not discard duplicate observations or infer new interaction kinds from names. The registered PDG comparison retains the original observed-class boundary; the generic trace theorem does not assign a universal class count to arbitrary inputs.

> Source, proof and provenance: V3 INTERACTION_CLASSES, field_laws.py:L107; Lean changedCompleteTraceDistinguishes, FieldInteractionClasses.lean:L52; Lean namesAloneDoNotDistinguish, FieldInteractionClasses.lean:L19; Timeline: Turn 489, line 4270.

396. Exact interior change and boundary flow

Let I(x) record whether a generated support node lies inside the held region. Every original transfer t retains its named source s(t), destination d(t), complete carrier, source identity and action record. The complete outward and inward ledgers are selected by

O = [t : I(s(t)) = true and I(d(t)) = false],
J = [t : I(s(t)) = false and I(d(t)) = true].

The membership theorems are biconditionals: a transfer occurs in O precisely when it occurs in the original network and satisfies the outward conditions; the corresponding statement holds for J. Thus no crossing is omitted or invented. Inward selection is outward selection for the complementary region, with both endpoints retained.

Let D be the number of transfers departing from an interior node and A the number arriving at an interior node. Complete finite-network induction gives

$$D + |J| = A + |O|$$

The proof checks all four endpoint-membership cases. An interior-to-interior transfer adds one to both D and A. An interior-to-exterior transfer adds one to D and |O|. An exterior-to-interior transfer adds one to A and |J|. An exterior-to-exterior transfer changes none of the four counts. This equality expresses continuity without introducing a negative proof magnitude.

The result retains complete carriers as well as counts. Interior losses are source-side records of O, and boundary outflow is the recipient-side record of the same O. Projection onto carriers gives exactly equal ordered lists. Interior gains and boundary inflow similarly project to the same complete carrier list from J. Source identities, actions and distinct transfer occurrences remain available throughout; equal-valued transfers are not silently merged.

> Source, proof and provenance: V3 CONSERVED_SOURCE, field_laws.py:L113; Lean inwardAsComplement, FieldSourceContinuity.lean:L67; Lean outwardMember, FieldSourceContinuity.lean:L35; Timeline: Turn 489, line 4272.

397. Action and reaction as paired held transfer records

For every complete transfer t, construct the source-loss and recipient-gain records

$$L(t) = (\text{source}(t), \text{carrier}(t), first, \text{sourceIdentity}(t), \text{action}(t))$$

$$G(t) = (\text{destination}(t), \text{carrier}(t), second, \text{sourceIdentity}(t), \text{action}(t))$$

The two records retain the same full carrier and provenance, while their held roles are distinct and opposite:

$$\text{carrier}(L(t)) = \text{carrier}(G(t))$$

$$\text{held}(G(t)) = \text{opposite}(\text{held}(L(t)))$$

Both named endpoints are retained. When the carrier is the previously derived full momentum record, the magnitude at each end is the same positive fraction. Opposite held roles therefore do not require a negative momentum proof quantity.

For a complete network T, the paired ledger [(L(t),G(t)) : t in T] has exactly |T| entries. Projection onto source-side carriers equals projection onto recipient-side carriers, occurrence by occurrence. Appending a transfer appends its complete pair and preserves the earlier paired history. This is the original reciprocal closed-transfer construction, including the retained motion and source records inside each momentum carrier.

> Source, proof and provenance: V3 ACTION_REACTION, field_laws.py:L119; Lean positiveMomentumAtBothEnds, FieldActionReaction.lean:L53; Lean heldRolesDistinct, FieldActionReaction.lean:L26; Timeline: Turn 489, line 4274.

398. Period and frequency from complete recurrence counts

A wave retains a closed, oriented recurrence and a complete held-reference recurrence. Let N > 0 be the wave recurrence's transition count and M > 0 the reference count. The exact period is

$$T = \text{normalize}(N, M)$$

and frequency is its positive reciprocal f. If T has reduced numerator p and denominator q, then N q = p M. If f has reduced numerator a and denominator b, reciprocal normalization gives q b = a p. Combining the two identities and cancelling the positive p yields

$$Mb = aN$$

$$f = \text{normalize}(M, N)$$

Consequently the exact normalized product T f is One: its numerator and denominator are both 1. The proof holds for every positive generated recurrence/reference pair. Both period and frequency remain positive; no floating or irrational cycle value is introduced.

The recurrence's last carrier equals its first carrier. A held phase position retains an initial and remaining path whose concatenation is the complete original recurrence. Therefore the phase record preserves the full source/action history as well as the count ratio.

> Source, proof and provenance: V3 PERIOD_FREQUENCY, wave_laws.py:L33; Lean periodExact, WavePeriodFrequency.lean:L17; Lean frequencyIsReverseCountRatio, WavePeriodFrequency.lean:L39; Timeline: Turn 489, line 4276.

399. Adjacent propagation retaining source, frequency and phase

A propagation record retains the original source observation, wave recurrence, source event, generated adjacency relation and complete finite arrival list. Every arrival carries its actual source-to-cell adjacent path. Its phase is a full position in the same source recurrence, including both the initial and remaining path components.

For each arrival a, the generated observation tuple is

(source, wave, destination(a), phase(a)).

Projection onto destinations yields exactly the completed accessible support. Projection onto source observations gives the original source at every occurrence. Projection onto frequency gives the same exact source frequency f at every occurrence. Every transition in an arrival path satisfies the generated adjacency relation, and every phase decomposition reconstructs the complete source recurrence.

If A is the completed arrival list and a is a new completed arrival, the generated ledger obeys

$$\mathrm{records}(A + +[a]) = \mathrm{records}(A) + +[(source, wave, \mathrm{destination}(a), \mathrm{phase}(a))]$$

This retains every preceding source, phase and response record. Propagation uses the actual generated finite paths; it does not substitute a completed infinite wave or an imported continuum propagation equation.

> Source, proof and provenance: V3 PROPAGATION, wave_laws.py:L39; Lean completeSupport, WavePropagation.lean:L18; Lean originalCarrierAtEveryCell, WavePropagation.lean:L28; Timeline: Turn 489, line 4278.

400. Exact wavelength, period and propagation speed

One spatial repeat retains its full two-endpoint displacement and positive separation lambda, paired with the original temporal wave recurrence and held clock reference. The source quotient is the exact reciprocal-and-product operation:

$$v = \lambda / T = \lambda f$$

If lambda has numerator l_n and denominator l_d, T has numerator t_n and denominator t_d, and the reduced speed has numerator v_n and denominator v_d, the verified cross-product identity is

$$(l_n t_d) v_d = v_n (l_d t_n)$$

All factors belong to the admitted positive domain. The reduced positive-ratio uniqueness theorem establishes that any candidate satisfying this same exact relation equals v. Thus the three-carrier closure is unique for the retained spatial and temporal repeat; it does not select a fitted speed.

The full displacement endpoints and temporal recurrence remain in the construction. A changed endpoint pair, changed spatial repeat or changed temporal wave record gives a distinct paired record, even when an extracted scalar happens to coincide. The separate complete spectral-source comparison retains the original wavelength, wavenumber and frequency-conversion boundary.

> Source, proof and provenance: V3 SPEED_LENGTH_FREQUENCY, wave_laws.py:L45; Lean speedUnique, WaveSpeedRelation.lean:L34; Lean wavelengthOverPeriod, WaveSpeedRelation.lean:L19; Timeline: Turn 489, line 4280.

401. Complete labelled wave superposition

The source composition retains every independently generated wave trace on shared support. Write L=[l_1,...,l_m] and R=[r_1,...,r_n], where each record may carry its complete source, frequency, phase and support. The disjoint junction is

$$J(L,R) = [\mathrm{inl}(l_1), ..., \mathrm{inl}(l_m)] + +[\mathrm{inr}(r_1), ..., \mathrm{inr}(r_n)]$$

The two projections select their own branch and remove only its injection label. Recursion on the lists gives

$$pi_L(J(L,R)) = L, pi_R(J(L,R)) = R$$

$$|J(L,R)| = |L| + |R|$$

The injections are individually injective and have disjoint images: inl(l) cannot equal inr(r). Consequently coincident extracted values do not identify distinct source occurrences. If J(L,R)=J(L',R'), applying the two projections gives L=L' and R=R'. These reconstruction identities establish complete preservation and uniqueness of the labelled composition. No constituent is discarded, duplicated or replaced by a fitted amplitude.

> Source, proof and provenance: V3 SUPERPOSITION, wave_laws.py:L51; Lean noBranchCollision, WaveSuperposition.lean:L51; Lean leftInjective, WaveSuperposition.lean:L54; Timeline: Turn 490, line 4294.

402. Common-cell phase-labelled predecessor merging

Each predecessor retains its observation cell, full incoming trace, held orientation and positive exact support. Merging is defined at a shared observation cell and retains the ordered pair of complete predecessors alongside the resulting support. The support operation is the previously established exact held-scalar addition.

For positive fractions x=a/b and y=c/d, matching held labels produce

$$x + y = (ad + cb)/(bd)$$

reduced by the exact normalization operation. The normalized numerator N and denominator D satisfy (ad+cb)D=Nbd. For opposed labels the comparison is between the positive cross-products ad and cb. Equality closes the support distinction to EmptyOne; ad>cb retains the first held orientation with positive remainder (ad-cb)/(bd); ad<cb retains the opposite orientation with positive remainder (cb-ad)/(bd). These subtractions denote the corresponding admitted positive remainder branches, not negative scientific quantities.

Thus exact opposed closure is equivalent to ad=cb. In either unequal branch, recomposing the remainder with the smaller support reconstructs the larger support. Closure of the observed distinction does not erase either predecessor: projecting the merged observation recovers both original records and their common cell. Distinct source traces remain distinct records even when the merged support is equal.

> Source, proof and provenance: V3 INTERFERENCE, wave_laws.py:L57; Lean equalOpposedSupportCloses, WaveInterference.lean:L33; Lean closureExactlyEqualSupport, WaveInterference.lean:L39; Timeline: Turn 490, line 4296.

403. Aperture selection and complete reachable-support regeneration

Let P be the complete finite list of incident propagation paths, A(p) the registered aperture decision and S(p) the generated successor list for path p. The retained paths and regenerated output are

$$R_A(P) = [p \in P \mid A(p) = \text{true}],$$
$$G_A(P) = \text{concat}_{p \in R_A(P)}[(p, q) \mid q \in S(p)].$$

Induction on P establishes exact aperture membership:

p in R_A(P) iff p in P and A(p)=true.

The full flattened successor generation then gives both directions of

(p,q) in G_A(P) iff p in P, A(p)=true and q in S(p).

Every registered reachable successor therefore appears, and every generated output has a retained incident origin. If A(p)=false, no pair (p,q) can occur. The list construction preserves occurrence order and multiplicity, including different incident paths arriving at the same output cell.

For actual local propagation records, each output carries its complete generated path. Every transition in that path satisfies the registered adjacency relation, by induction over the propagation constructors. This supplies the locality of the regenerated support without imposing a continuum field solution or deleting source paths after their arrival cells coincide.

> Source, proof and provenance: V3 DIFFRACTION, wave_laws.py:L63; Lean reachableOutputIsLocal, WaveDiffraction.lean:L62; Lean excludedPathCannotContribute, WaveDiffraction.lean:L54; Timeline: Turn 490, line 4298.

404. Held transverse polarization

A polarized recurrence retains the complete radiation carrier, including its joint electromagnetic response, energy, momentum, recurrence and held polarization label. Reversal changes the held polarization label and retains the other fields. With the two held orientations denoted h_1 and h_2,

$$\text{opp}(h_1) = h_2, \text{opp}(h_2) = h_1$$

$$\text{opp}(\text{opp}(h)) = h, \text{opp}(h) \neq h$$

For the full carrier reversal R this yields R(R(p))=p. Its energy carrier, momentum carrier and recurrence are unchanged:

$$E(R(p)) = E(p), M(R(p)) = M(p), C(R(p)) = C(p)$$

Because frequency is derived from that same recurrence and reference clock, f(R(p))=f(p). The transverse role extracted from the joint magnetic response is likewise unchanged. The orientation distinction therefore preserves the complete transverse response geometry and all positive carrier magnitudes; it introduces no continuous angle or fitted polarization coefficient.

> Source, proof and provenance: V3 POLARIZATION, wave_laws.py:L69; Lean frequencyUnchanged, WavePolarization.lean:L28; Lean transverseRoleUnchanged, WavePolarization.lean:L32; Timeline: Turn 490, line 4300.

405. Recurrence-labelled support response and dispersion

A registered support response assigns each retained recurrence label its complete spatial repeat, temporal wave, causal arrival path and phase decomposition. For a finite label list L and registered response rule r, the complete response ledger is

D(L,r) = [(ell,r(ell),v(r(ell))) | ell in L].

Projecting its first field recovers L exactly; projecting its response field recovers map(r,L). Consequently the transition rule is applied to every original label without conflating equal extracted values. Each speed is the established exact ratio

$$v_\ell = \lambda_\ell / T_\ell = \lambda_\ell f_\ell$$

Every arrival path retains adjacency at each transition, and the initial and remaining phase paths concatenate to the whole original recurrence. The registered support response, rather than an added universal correction, determines the label dependence.

For a repeated registered support segment, let c_a and c_b be the generated transition counts for two responses. After n positive repetitions their accumulated counts are n c_a and n c_b. Positive multiplication cancellation proves

c_a≠c_b and n>0 imply n c_a≠n c_b.

Thus distinct registered propagation timings persist through repeated support. The statement uses the timing differences supplied by the actual support rule; it does not assert a difference when that rule assigns equal timings. A changed exact speed also distinguishes the complete response record, while every appended label preserves the earlier ledger.

> Source, proof and provenance: V3 DISPERSION, wave_laws.py:L75; Lean phaseRetainsWholeRecurrence, WaveDispersion.lean:L39; Lean timingSeparationPersists, WaveDispersion.lean:L48; Timeline: Turn 490, line 4302.

406. Exact recurrence matching and resonance

Retain the full driver and supported recurrence paths on their common generated refinement, with positive counts a and b and their held phase labels phi_d and phi_s. The least common return is

$$L = \operatorname{lcm}(a,b) = a(b/\gcd(a,b)) = b(a/\gcd(a,b))$$

Both a and b divide L and L is positive. Exact phase-aligned matching at a generated return n is

Matched(n) iff n>0, a divides n, b divides n and phi_d=phi_s.

If the phases agree, L is a matched return. Every matched n is divisible by L, hence L≤n. Any positive matched return that is least among all matched returns must equal L, by the two inequalities. This proves uniqueness of the least exact refinement without a tolerance band.

If phi_d≠phi_s, Matched(n) is false for every n under that retained phase pairing. This is an exclusion of unmatched phases from coherent transfer, not erasure of the driver or supported traces. For a list N of matched returns the transfer ledger is

T(N) = [(n,driver,supported,phi_d,phi_s) | n in N].

Its length equals |N|, every row has equal held phase labels, and appending a matched return appends exactly one complete transfer record while preserving all prior records. The original recurrence paths remain in every row, so repeated aligned transfer carries its full source correspondence.

> Source, proof and provenance: V3 RESONANCE, wave_laws.py:L81; Lean commonReturnMatches, WaveResonance.lean:L23; Lean allMatchedReturnsRefine, WaveResonance.lean:L35; Timeline: Turn 490, line 4304.

407. Wave energy and held momentum along a causal path

A transported wave retains its full radiation carrier K, source event s, remote arrival event r, complete adjacent propagation path P, held propagation orientation h and source action record. The momentum orientation is the retained path orientation. The closed transfer is

(s,r,K,sourceIdentity,actionRecord).

Its source-loss and remote-gain projections have the same full carrier. Therefore their exact work and momentum agree:

$$E_{\mathrm{loss}} = E_{\mathrm{gain}}, W_{\mathrm{loss}} = W_{\mathrm{gain}}, M_{\mathrm{loss}} = M_{\mathrm{gain}}$$

$$\mathrm{orientation}(M_{\mathrm{gain}}) = h$$

The accounting roles are opposed while the transported momentum remains the original held carrier. Every transition q along P is recorded with (q,K,h). Projecting these records recovers the complete transition list of P, and each row retains K and h. Every path transition satisfies the original adjacency relation.

For an arbitrary finite network of these transports, the outgoing and incoming carrier lists are equal occurrence by occurrence:

$$\mathrm{outgoingSupport}(network) = \mathrm{incomingSupport}(network)$$

This conservation identity retains the full energy and momentum records at both ends and along the causal path. It does not replace them with independently assigned endpoint scalars. Remote work is thereby paired with its corresponding source transfer and transported recurrence.

> Source, proof and provenance: V3 ENERGY_MOMENTUM, wave_laws.py:L87; Lean eachPathRecordRetainsCarrier, WaveEnergyMomentum.lean:L47; Lean sameWorkAtBothEnds, WaveEnergyMomentum.lean:L29; Timeline: Turn 490, line 4306.

408. Physical quantum support and preparation

The physical state retains the admitted Fold state: a finite list of distinct words with held phase labels, together with the original nontrivial phase cycle. It also retains the source identity and complete preparation record. Admission checks the original word uniqueness and phase-cycle membership; no amplitude list replaces these records.

For generated alphabet A and position list P, the original complete-support operation supplies

support(state) = words(A,|P|).

A word belongs to this support exactly when it has |P| positions and each of its labels belongs to A. The original generation and admission conditions remain in force. Every branch (w,phi) retains phi in the registered cycle.

For source identity s and preparation trace H, the physical record ledger is

R = [((w,phi),s,H) | (w,phi) in state.branches].

Projection onto the first field recovers state.branches exactly. Projection of any row onto its source and preparation fields yields s and H. A changed source identity or preparation trace distinguishes the complete physical state even when an extracted word support agrees. These identities retain both generated alternatives and their physical preparation provenance.

> Source, proof and provenance: V3 PHYSICAL_STATE, quantum_physics_laws.py:L90; Lean completeBranchesRecovered, QuantumPhysicalState.lean:L17; Lean everyPhaseHeld, QuantumPhysicalState.lean:L28; Timeline: Turn 491, line 4318.

409. Closed reversible physical evolution

Closed evolution uses the original admitted gates and their successful applications. A gate carries its finite one-to-one word map and generated phase-successor trace. Let G=[g_1,...,g_n] be the actual gate schedule, S_0 the prepared state, S_n the terminal state, and H the retained predecessor-state list.

Induction over the successful application trace constructs the backward derivation using the same gates in reverse order and their original inverse operations:

Forward(S_0,G,H,S_n) implies Backward(S_n,reverse(G),S_0).

Each inverse application reconstructs the preceding full word-and-phase state. The same induction proves

$$|H| = |G|, \qquad |\operatorname{branches}(S_n)| = |\operatorname{branches}(S_0)|$$

The complete branch correspondence is stronger than a count identity:

b in S_n.branches iff there exists a in S_0.branches with BranchTrace(cycle,G,a,b).

Thus every final branch has an actual original predecessor trace, and every branch generated by that trace is retained. The physical evolution record additionally holds the original source identity and full preparation. Reversibility does not discard phase action or reconstruct an answer-only support.

The registered empirical protocol for this entry uses its separate exact action/energy relation. For positive input intervals A=[a_l,a_u] and E=[e_l,e_u], the capability-closed program predicts

$$T = [a_l/e_u, a_u/e_l]$$

The eight original instructions read the four endpoints, form the two ordered quotients, pair them and emit the interval. The external action and energy inscriptions are parsed as exact rational measurement records; an official decimal prefix is bounded by its next decimal place and the original uncertainty is retained. The duration target remains committed until the prediction is sealed.

The current pinned comparison yields

$$T = \left[\frac{81120909}{335364978631600000000000}, \frac{527285909}{217987236110060000000000000}\right]$$

$$T_{\text{external}} = \left[\frac{12094421632919}{500000000000000000000000000000}, \frac{2418884326589}{1000000000000000000000000000000}\right]$$

The exact overlap test is not(t_u < o_l or o_u < t_l). It accepts this released target interval and rejects the displaced control [t_u+1,t_u+2]. All input, program, output, trace, custody and receipt hashes retain the original source protocol. These measured correspondence inputs remain separate from the preceding native reversible-evolution derivation.

Measured-comparison source, executable relation and exact input binding: V3 measured VALUE_SPECS, quantum_physics_laws.py:L209; Lean EvolutionMeasured.program:L18; Lean EvolutionMeasured.validate:L88; Lean EvolutionMeasured.sealBindsExactInputs:L149.

> Source, proof and provenance: V3 EVOLUTION, quantum_physics_laws.py:L97; Lean everyBranchHasSource, QuantumPhysicalEvolution.lean:L27; Lean exactlyReversible, QuantumPhysicalEvolution.lean:L15; Timeline: Turn 491, line 4322.

410. Observables, selected fibres and retained records

The original observable is a total classification of admitted physical word support. For prepared state S, classification M and observed label ell, a successful observation returns selected state S_ell and the complete record list R. Its selected fibre satisfies

(w,phi) in S_ell.branches iff (w,phi) in S.branches and M(w)=ell.

Every original branch receives an outcome record. If M(w)=k with k≠ell, the branch is excluded from the selected state but its record ((w,phi),k) remains in R. Consequently a closed alternative is recorded rather than erased.

The complete state is reconstructible from the record projection and retained cycle:

$$\mathrm{State}(\mathrm{map}(first, R), S_\ell.phaseCycle) = S$$

The selected branch list preserves the original order. The physical observation record also retains source identity, preparation trace, classification and observed label, so the partition and its setting remain attached to the result. The model distinguishes selecting a record class from deleting the support needed to reconstruct its alternatives.

> Source, proof and provenance: V3 OBSERVABLE, quantum_physics_laws.py:L104; Lean closedClassNotErased, QuantumPhysicalObservable.lean:L27; Lean everySourceClassRecorded, QuantumPhysicalObservable.lean:L21; Timeline: Turn 491, line 4324.

411. Exact quantum branch-support weights

The predecessor census retains the held phase/action records of the prepared state. For a successfully observed class let n be its selected branch count and N the complete original branch count. Original admission gives n>0 and N>0. The source weight is the exact reduced positive part

$$w = \mathrm{normalize}(n, N)$$

If w has numerator p and denominator q, normalization preserves

$$nq = pN$$

The full record list has N rows. The selected list is the original branch list filtered by the registered classification, so induction over that list proves n≤N. Each retained branch contributes one occurrence; absent classes are rejected by the original observation admission and are not replaced by a numerical-zero weight row.

For any admitted reduced positive fraction p'/q' satisfying nq'=p'N, reduced-ratio uniqueness gives p'/q'=w. This establishes the sole exact weight for the complete selected and whole counts. The numerator is positive, and the denominator is inherited from the complete census through reduction. No measured frequency, fitted amplitude or stochastic choice enters this ratio.

> Source, proof and provenance: V3 WEIGHT, quantum_physics_laws.py:L111; Lean onlyExactWeight, QuantumPhysicalWeight.lean:L27; Lean selectedCountAtMostWhole, QuantumPhysicalWeight.lean:L42; Timeline: Turn 491, line 4326.

412. Record-preserving joint refinement and incompatibility

Let S be the complete generated source support, A(s) and B(s) the two complete observation records, including their held phase/action information, and F the original generated family of candidate joint refinements. A refinement r supplies a generated joint record J_r(s) and its two record projections L_r and R_r.

Its exact preservation condition is

P(r,s) iff L_r(J_r(s))=A(s) and R_r(J_r(s))=B(s),
Preserves(r) iff for every s in S, P(r,s).

The finite Boolean preservation computation is proved equivalent to this full row-wise condition. A failed computation constructs an actual source occurrence s for which at least one of the two complete records differs; failure is not merely an indirect candidate-count test.

The incompatibility boundary is then established exactly over the registered family:

Incompatible(A,B;S,F)
iff every r in F fails Preserves(r)
iff there is no r in F preserving both complete records on every source occurrence.

Suppose a candidate resolution r in F preserves A on all of S. The constructive failed-row theorem supplies an occurrence whose A or B record differs. The first alternative contradicts preservation of A, so a required B record must differ. Interchanging A and B proves the symmetric statement. Resolving either side therefore closes a distinction required to preserve the other whenever the registered common-refinement condition fails.

The ledger [(s,J_r(s),L_r(J_r(s)),R_r(J_r(s))) | s in S] projects back to S exactly. Thus the exclusion retains complete source provenance. It introduces neither an irrational uncertainty bound nor a replacement family of admissible refinements.

> Source, proof and provenance: V3 INCOMPATIBILITY, quantum_physics_laws.py:L118; Lean resolutionClosesRequiredRecord, QuantumPhysicalIncompatibility.lean:L76; Lean resolutionClosesRequiredLeftRecord, QuantumPhysicalIncompatibility.lean:L86; Timeline: Turn 491, line 4328.

413. Physical spin as finite held cyclic action

Spin uses the original generated phase action on an admitted physical constituent. Let C be its complete held cycle and T its generated orientation-successor trace. The action shifts each held branch label by

$$k = |T| mod |C|$$

$$\mathrm{index}(phi') = (\mathrm{index}(phi) + k) mod |C|$$

The existing finite lookup construction proves that every admitted starting label has a resulting label in C. Successful action retains the original cycle and the complete word support:

C'=C, support(S')=support(S).

For a full generated orientation recurrence, |T|=|C|, the exact phase action returns the entire original state:

$$\mathrm{phaseAction}(S,T) = S$$

This is a statement about every admitted finite cycle and its actual generated trace, not only a two-label example. Distinct held readings remain distinct branch records: phi_1≠phi_2 implies (w,phi_1)≠(w,phi_2). They are labels, not signed or imaginary proof magnitudes.

The physical action record retains the complete original preparation, source identity, orientation trace and resulting state. The separately registered external spin classifications are compared only after sealing; they do not select the cyclic construction.

> Source, proof and provenance: V3 SPIN, quantum_physics_laws.py:L125; Lean cyclicLabelAction, QuantumPhysicalSpin.lean:L14; Lean fullOrientationRecurrenceReturns, QuantumPhysicalSpin.lean:L24; Timeline: Turn 491, line 4330.

414. Indistinguishability through complete exchange traces

Physical identity in the V3 construction is exchange equivalence of the complete observable transition and composition record. Let p=(a,b) be a pair of constituents and E(p)=(b,a) its exchange. The operation retains both constituents and satisfies

$$E(E(p)) = p$$

For the complete registered observation family O=[o_1,...,o_n], retain the ordered ledger

$$L_O(p) = [o_1(p), ..., o_n(p)]$$

The physical indistinguishability condition is

I_O(p) iff for every o in O, o(E(p))=o(p).

Induction on the complete observation list gives the exact equivalence

$$I_O(p) \iff L_O(E(p)) = L_O(p)$$

The forward induction rewrites each head observation and retains the induction hypothesis for the remaining ledger. In the reverse direction, equality of the two lists gives equality of their heads and tails, recovering every registered observation. No constituent or observation is discarded by this quotient.

Exchange is an involution, so I_O(p) implies I_O(E(p)). Conversely, a held observation o in O with o(E(p))≠o(p) distinguishes the constituents. Extending the registered family preserves the exact condition:

I_(o::O)(p) iff o(E(p))=o(p) and I_O(p).

Thus indistinguishability does not erase a physical distinction that appears in the complete transition or composition trace. The original empirical identity classifications remain separately sealed and compared after the derivation.

> Source, proof and provenance: V3 INDISTINGUISHABILITY, quantum_physics_laws.py:L132; Lean exchangeTwice, QuantumPhysicalIdentity.lean:L10; Lean fullLedgerExactly, QuantumPhysicalIdentity.lean:L54; Timeline: Turn 492, line 4344.

415. Exclusion in the alternating exchange class

The alternating composition class carries one of two opposite held orientations. Write J for orientation reversal. These are distinct labels in the positive exact representation, with

J(h)≠h.

For an orientation record H on constituent pairs, alternating exchange means

$$H(E(p)) = J(H(p))$$

If exchange fixes a pair, E(p)=p, substitution would require H(p)=J(H(p)), contradicting the distinction of the two held orientations. Hence

E(p)=p implies that p is excluded from the alternating class.

A constituent occupant retains its complete word w and its one-constituent observation cell c. For a=(w_a,c_a) and b=(w_b,c_b), equality of both fields gives

w_a=w_b and c_a=c_b implies a=b,

$$E(a,b) = (a,b)$$

Combining these identities with the alternating-orientation condition gives the same-cell exclusion derivation:

w_a=w_b and c_a=c_b implies not alternating(a,b).

Equivalently, any admitted alternating pair with identical constituent words must satisfy c_a≠c_b. The contradiction uses the actual exchanged record and the retained orientation distinction. It introduces no signed proof magnitude, occupation fit or additional exclusion rule. The statement applies to the original alternating exchange class, and its external occupation classifications retain their original empirical boundary.

> Source, proof and provenance: V3 EXCLUSION, quantum_physics_laws.py:L139; Lean admittedSameWordRequiresDistinctCell, QuantumPhysicalExclusion.lean:L35; Lean identicalSameCellExcluded, QuantumPhysicalExclusion.lean:L29; Timeline: Turn 492, line 4346.

416. Tunnelling from complete generated support

Let S be an admitted prepared physical Fold state. A successful reversible gate sequence G generates terminal state S', retains every predecessor state, and supplies an exact branch trace from each source branch to its terminal branch. The existing evolution construction establishes

b in branches(S') iff there exists a in branches(S) with BranchTrace(G,a,b),

and the inverse gate sequence reconstructs S exactly.

Let K(b) be the held phase-compatibility test for the generated barrier record, and C(b) the corresponding coarse classical observation. The complete traversal support is the ordered filter

T=[b in branches(S') : K(b)=true and C(b)=false].

The filter membership proof gives both directions of this equation for every generated branch. Consequently every observed b in T has an original source a and its exact successful gate trace. The branch was generated by the complete reversible evolution; it was not added to compensate for the coarse observation.

The coarse support is P=[b in branches(S') : C(b)=true]. If b belongs to both T and P, the retained coarse record would satisfy C(b)=false and C(b)=true. Therefore

b in T implies b not in P.

Conversely, every generated branch with the held compatible phase record and a closed coarse record belongs to T. This establishes the V3 distinction: complete support retains the lawful path that the coarse observation closes. The full preparation, gate sequence, predecessor ledger and terminal state remain beside the traversal observation. The underlying evolution stays exactly reversible after the traversal subset has been recorded.

> Source, proof and provenance: V3 TUNNELLING, quantum_physics_laws.py:L146; Lean completeSupportDoesNotDeletePath, QuantumPhysicalTunnelling.lean:L46; Lean traversalExactly, QuantumPhysicalTunnelling.lean:L12; Timeline: Turn 492, line 4348.

417. Discrete spectra from exact boundary recurrence

For an admitted prepared state S and a generated boundary trace t, let A(S,t) be the original finite cyclic phase action. Successful action preserves the complete branch support and original phase cycle. A boundary class is closed precisely when the resulting state equals the initial state:

$$closed_S(t) \iff A(S,t) = S$$

The executable closure test compares both complete state fields, branches and phase cycle, after successful action. Equality of the two fields reconstructs equality of the state; conversely equality of the states makes both comparisons true. Failure of the original action is rejected.

For the complete finite family B of generated boundary traces, the bound-state spectrum is

Sigma(S,B)=[t in B : closed_S(t)].

The native membership proof establishes

t in Sigma(S,B) iff t in B and A(S,t)=S.

Every generated boundary trace is tested, and every retained class closes exactly. If the held cycle has length m and a generated trace has length m, the cyclic-action derivation gives A(S,t)=S, so that trace is retained in the spectrum. The original empty-action guard excludes the empty trace; the full-cycle result uses the admitted nontrivial phase cycle.

A transition record retains two distinct generated closed trace classes u and v. Its obligations give

u in Sigma(S,B), v in Sigma(S,B), u≠v,
A(S,u)=S and A(S,v)=S.

The transition retains the complete preparation, boundary family and both trace records. Discreteness follows from the finite generated recurrence classes and their exact closure test. No continuum eigenvalue equation is imported to select the classes.

> Source, proof and provenance: V3 DISCRETE_SPECTRA, quantum_physics_laws.py:L153; Lean closureExactly, QuantumPhysicalSpectra.lean:L11; Lean fullRecurrenceAdmitted, QuantumPhysicalSpectra.lean:L46; Timeline: Turn 492, line 4350.

418. Entanglement as nonfactorable complete joint support

Retain the common preparation and the full original branch beside its two constituent observations. For projections l and r on the complete prepared support, the joint ledger contains

(b,l(b),r(b)) for every b in branches(S).

Projecting each ledger row back to b reconstructs branches(S) in its original order. The joint support J consists of the corresponding pairs (l(b),r(b)); the preparation and source identity remain held separately from the projections.

Independent constituent supports L and R factor the joint support exactly when

(x,y) in J iff x in L and y in R.

Such a factorization forces cross-word closure: if (x,y) and (x',y') belong to J, then (x,y') also belongs to J. The proof takes x from the first pair's left membership and y' from the second pair's right membership and applies the reverse direction of the product condition.

The converse follows from the complete marginals. If J is cross-closed, any x in the left marginal and y in the right marginal have source rows (x,a) and (b,y) in J. Cross closure supplies (x,y). Hence

J has an independent-support factorization iff J is cross-closed.

Entanglement is the failure of this factorization while common preparation remains retained. In particular,

(x,y) in J, (x',y') in J, (x,y') not in J

excludes every independent product L×R, not only one proposed product. Each individual constituent reading occurs, but their missing cross word prevents a reconstruction from independent marginals. The original successful Fold composition separately establishes its complete product-support relation through jointWord(l,r); that existing construction supplies the independent-composition side of the distinction. Neither the correlation proof nor the composition projection drops the full source branch or common preparation.

> Source, proof and provenance: V3 ENTANGLEMENT, quantum_physics_laws.py:L160; Lean missingCrossWordExcludesEveryIndependentProduct, QuantumPhysicalEntanglement.lean:L39; Lean entanglementExactlyNonclosed, QuantumPhysicalEntanglement.lean:L45; Timeline: Turn 492, line 4352.

419. Bell correlations and the omitted-dependence boundary

A complete generated event retains common preparation p, both measurement settings s_L and s_R, and both output records a and b. Its deterministic input is

$$I(e) = (p(e), s_L(e), s_R(e))$$

The full ledger stores each event beside this input and its two outputs. Projecting the ledger back to its source events recovers the entire census in order. The physical ledger additionally retains the admitted Fold preparation and every original branch beside its event interpretation.

The local factorized record considered by V3 retains a record h(e) and proposes functions f and g satisfying, for every generated event,

$$f(h(e), s_L(e)) = a(e)$$

$$g(h(e), s_R(e)) = b(e)$$

Suppose the complete census contains e and e' for which

$$h(e) = h(e'), s_L(e) = s_L(e'), \quad \text{but } a(e) \neq a(e')$$

Function congruence forces

$$f(h(e), s_L(e)) = f(h(e'), s_L(e'))$$

whereas faithful reproduction would identify these equal values with the two distinct held outputs. This contradiction excludes every such factorized reproduction. The same derivation applies to the right side when its retained record and local setting agree but its generated output differs.

The excluded factorization is therefore specified by the exact generated dependence it omits. The complete preparation-and-settings record remains part of the deterministic state throughout. Applying the contradiction to events obtained from two actual held physical branches connects the boundary directly to the complete Fold support. This preserves V3's setting-inclusive correlation statement and its original independent causal boundary; no additional numerical Bell bound or replacement hidden-record premise is introduced.

> Source, proof and provenance: V3 BELL, quantum_physics_laws.py:L167; Lean physicalOmissionExcludesLocalFactorization, QuantumPhysicalBell.lean:L75; Lean omittedLeftDependenceExcludesFactorization, QuantumPhysicalBell.lean:L19; Timeline: Turn 492, line 4354.

420. Contextuality and complete observation provenance

A successful physical observation retains the admitted preparation S, its complete compatible classification C, the selected class q and the complete branch record R. The original observation construction proves

$$\mathrm{map}(first, R) = \mathrm{branches}(S)$$

$$|R| = |\,\mathrm{branches}(S)|$$

Selection does not erase the other original branches. The observation record retains (S,C,q,R), so distinct complete contexts remain distinct recorded observations.

Let r(i,C) denote the held reading of an item i in context C. If two generated compatible contexts give

r(i,C_1)≠r(i,C_2),

no context-erasing assignment a(i) can reproduce both. Such an assignment would require

$$a(i) = r(i, C_1), a(i) = r(i, C_2)$$

and transitivity would identify the two distinct readings. The same contradiction excludes an assignment proposed for every item and every context in the complete registered families: specialize it to the held item and the two held contexts. This derives the precise V3 contextuality boundary from the changed generated reading, while retaining the entire compatible partition alongside each observation.

> Source, proof and provenance: V3 CONTEXTUALITY, quantum_physics_laws.py:L174; Lean noUniversalContextErasingAssignment, QuantumPhysicalContextuality.lean:L30; Lean everyOriginalBranchHeld, QuantumPhysicalContextuality.lean:L18; Timeline: Turn 493, line 4366.

421. Decoherence through reversible environmental records

Let an admitted preparation S evolve reversibly to S'. A complete terminal branch is b=(w,phi), with physical word w and held phase label phi. Environmental distribution retains an environmental word e(b) and a phase reading d satisfying, for every original terminal branch,

$$d(e(b)) = phi$$

The global record and its recovery map are

$$G(b) = (w, e(b))$$

$$U(w, e) = (w, d(e))$$

Substitution gives U(G(b))=b for every held branch. Therefore equal global records imply equal original branches. The global encoding cannot merge two distinct held branches or erase their phase distinction.

In particular, if phi_a≠phi_b, then e(a)≠e(b): equality of the environmental words would give equality of their decoded phases. Applying recovery to every row of the complete global ledger reconstructs branches(S') in its original order.

The reduced observation omits the inaccessible environmental component. Its projection is exactly

map(first,map(G,branches(S')))=support(S').

Branches with the same system word consequently have the same reduced observation even when their distinct phases remain recoverable from distinct environmental words. The original successful reverse gate trace still reconstructs S from S'. The local closure of phase distinctions is thus a projection of retained global records, with no stochastic deletion of the source support.

> Source, proof and provenance: V3 DECOHERENCE, quantum_physics_laws.py:L181; Lean globalEncodingCannotMergeHeldBranches, QuantumPhysicalDecoherence.lean:L24; Lean fullSupportReconstructed, QuantumPhysicalDecoherence.lean:L40; Timeline: Turn 493, line 4368.

422. The environment-distinguished classical quotient

On the declared macroscopic boundary, let k(b) identify the relevant predecessor class and e(b) its environmental record. The classical-limit condition is that environmental records distinguish these classes:

e(a)=e(b) implies k(a)=k(b)

for all branches in the complete terminal support. Equivalently for a held distinction, k(a)≠k(b) forces e(a)≠e(b).

For a macroscopic reading M, retain the joint observation

$$J(b) = (M(b), e(b))$$

If two different predecessor classes had the same joint observation, projecting to the environmental component would identify their environmental records and hence their classes. Therefore

k(a)≠k(b) implies J(a)≠J(b).

Each nonempty environmental fibre consequently contains exactly one represented predecessor class: choose a member to establish existence, and apply the class-equality implication to any two members for uniqueness. Different interfering predecessor classes cannot merge within that fibre, even when their retained macroscopic readings coincide. The complete ledger still contains (b,J(b)) for every source branch, and projecting to b reconstructs the entire original support. Equal environmental records also give equal decoded phase labels through the environmental reconstruction law. These identities establish the original record-stabilized classical observation quotient on its declared scale without deleting the underlying reversible branch process.

> Source, proof and provenance: V3 CLASSICAL_LIMIT, quantum_physics_laws.py:L188; Lean closedCrossClassMerging, QuantumPhysicalClassicalLimit.lean:L27; Lean everyEnvironmentalFibreHasOnePredecessorClass, QuantumPhysicalClassicalLimit.lean:L17; Timeline: Turn 493, line 4370.

423. Complete remote fibres and no-signalling

Represent the complete joint observation support as an ordered list of pairs (l,r). A generated change of remote partition is a bijective relabelling f with inverse g, satisfying g(f(r))=r and f(g(r'))=r'. Apply it to every joint row:

$$T_f(l, r) = (l, f(r))$$

The complete relabelling is reversible row by row, and its local projection is unchanged:

$$\mathrm{map}((l, r') -> (l, g(r')), \mathrm{map}(T_f, J)) = J$$

$$\mathrm{map}(first, \mathrm{map}(T_f, J)) = \mathrm{map}(first, J)$$

For any registered local observation test P, the exact local support count is

$$N_P(J) = |\,\mathrm{filter}(P, \mathrm{map}(first, J))|$$

Applying the same filter to the equal local projections proves

$$N_P(\mathrm{map}(T_f, J)) = N_P(J)$$

The total support count is unchanged as well, so both the numerator and denominator of the exact local observation weight remain identical. Every remote-fibre row is retained; a remote partition change cannot alter that local count by omitting a branch.

The independent causal boundary remains the original generated-support rule. A noninitial admitted arrival has a generated adjacent predecessor. A destination outside the completed causal support has no completed arrival record. The count-preservation result and this source-bound causal condition give the V3 no-signalling boundary: remote relabelling preserves the complete local observation, while a changed causal response requires its generated transfer trace.

> Source, proof and provenance: V3 NO_SIGNALLING, quantum_physics_laws.py:L195; Lean completeRemoteRelabellingReversible, QuantumPhysicalNoSignalling.lean:L8; Lean changedArrivalRequiresGeneratedPredecessor, QuantumPhysicalNoSignalling.lean:L43; Timeline: Turn 493, line 4372.

424. Mass-energy from two propagation transfers

A retained inertial preparation carries exact positive content m. Its held limiting propagation carrier has exact speed c and retains its complete spatial, temporal and source records. The original conversion composes that inertial content with two copies of the same propagation-speed carrier:

$$E = m(cc) = mc^2$$

The number of transfers is forced by the already derived energy dimension. Starting with the mass kind, let D_n be the dimension after n speed transfers:

$$D_0 = M$$

$$D_{n+1} = D_n(L/T)$$

Induction gives the length-kind exponent of D_n as n, with no length-kind denominator. The transferred-work energy dimension is ML^2/T^2. Equality of the dimension forms therefore requires n=2. Conversely, expanding the products and quotients for D_2 gives exactly the energy dimension for every kind. Thus two transfers are both necessary and sufficient within the original generated conversion construction.

Write the exact positive carriers as m=m_n/m_d and c=c_n/c_d. Normalized Fold composition yields a positive reduced energy carrier E_n/E_d satisfying

$$(m_n c_n^2) E_d = E_n (m_d c_d^2)$$

The reduced-positive-fraction identity theorem proves uniqueness: any admitted positive reduced carrier satisfying this cross-product equation equals E. No further conversion coefficient is present. The full record retains the original inertial preparation, source trace and both complete propagation transfers.

The original numerical validation separately supplies the CODATA electron-mass interval and exact vacuum-speed carrier. The ten-instruction prediction computes the lower and upper speed squares and then the corresponding mass products:

$$E_{\text{lower}} = m_{\text{lower}} c_{\text{lower}}^2$$

$$E_{\text{upper}} = m_{\text{upper}} c_{\text{upper}}^2$$

For the pinned source, the exact prediction interval is

[204677644636298392091678595 1/250000000000000000000000000000000000000000000000, 204677644762124117114833064 7/250000000000000000000000000000000000000000000000].

The separately committed mass-energy target interval is

[40935528927/500000000000000000000000, 40935528953/500000000000000000000000].

The prediction is sealed before the target is released. Their exact rational intervals overlap, while the deliberately displaced unfavorable interval is rejected. These observed input and target records retain the source's empirical role; they do not select the structural two-transfer law.

Measured-comparison source, executable relation and exact input binding: V3 measured VALUE_SPECS, matter_nuclear_laws.py:L130; Lean MassEnergyMeasured.program:L18; Lean MassEnergyMeasured.validate:L90; Lean MassEnergyMeasured.sealBindsExactInputs:L151.

> Source, proof and provenance: V3 MASS_ENERGY, matter_nuclear_laws.py:L49; Lean forcedEnergyUnique, MatterMassEnergy.lean:L33; Lean agreesWithTransferredWorkDimension, MatterMassEnergy.lean:L62; Lean twoTransfersHaveTheRequiredDimension, MatterMassEnergyDimension.lean:L31; Lean energyDimensionForcesExactlyTwoTransfers, MatterMassEnergyDimension.lean:L23; Timeline: Turn 493, line 4374.

425. Particle-antiparticle reversal of held labels

A complete constituent retains its physical Fold word and preparation, positive inertial support, finite spin action, complete transition grammar and ordered conserved-label record. For a held orientation h, let J(h) be its opposite. The existing held-label construction gives

$$J(J(h)) = h$$

J(h)≠h.

Charge conjugation reverses every oriented label while retaining its identity:

(name,h) -> (name,J(h)).

Applying this map to the complete ordered label list defines the reversed constituent. Induction on the list and the orientation involution give

$$\mathrm{reverseLabels}(\mathrm{reverseLabels}(L)) = L$$

$$\mathrm{reverse}(\mathrm{reverse}(p)) = p$$

Every original label has its reversed row in the new record. Projecting the reversed list onto label names gives the original name list, and its length is unchanged. Thus no conserved label is added, omitted or replaced by an unrecorded label.

The reversal leaves the complete physical word, inertial carrier, spin action and transition grammar unchanged. In particular, the reversed constituent retains the same strictly positive inertial magnitude. The distinguishing operation acts on held orientations, not on a negative mass or signed proof magnitude. The original particle/antiparticle property and charge-conjugate decay records remain separate empirical comparisons at the registered boundary.

> Source, proof and provenance: V3 PARTICLE_ANTIPARTICLE, matter_nuclear_laws.py:L55; Lean everyOrientedLabelReversed, MatterAntiparticle.lean:L35; Lean labelReversalTwice, MatterAntiparticle.lean:L16; Timeline: Turn 493, line 4376.

426. Spin action and exchange-composition classes

The finite spin action retains the prepared physical word, its complete phase cycle and the generated orientation trace. Let s be the prepared state, A its admitted orientation action, and r a held-orientation reading of its generated result. For the two held orientations h and J(h), the exchange alternatives are exhaustive and exclusive:

$$r(A(s)) = r(s) \text{ or } r(A(s)) = J(r(s))$$

J(h)≠h.

The first equation defines the preserving class and the second the alternating class. The construction classifies the actual generated spin result; it does not assign the class by an external particle name. When an exchange observation identifies its initial and final held readings with those of the generated action, its exchange class equals this spin-action class. A full phase-cycle trace returns the original state,

$$A_{\mathrm{period}}(s) = s$$

and therefore has preserving held reading. These statements retain the full action trace and the original finite cycle rather than replacing either with a continuously adjustable spin value.

For two identical words occupying the same cell, exchanging the pair leaves the complete pair unchanged. Every held reading of that pair must consequently agree before and after exchange. Alternation would require h=J(h), which the two held constructors exclude. Thus an alternating composition cannot contain that duplicate one-cell pair, while the preserving composition admits it. Both classifications are derived from the same exchange operation, whose square returns the original pair.

> Source, proof and provenance: V3 FERMION_BOSON, matter_nuclear_laws.py:L60; Lean preservingSameCellAdmitted, MatterExchange.lean:L44; Lean alternatingSameCellExcluded, MatterExchange.lean:L35; Lean fullSpinPeriodPreserving, MatterExchangeSpin.lean:L21; Lean spinExchangeCorrespondence,

> MatterExchangeSpin.lean:L28; Timeline: Turn 494, line 4392.

427. Complete transitionwise conservation of physical labels

A complete transition ledger pairs each input label occurrence with either an output carrier or a named boundary carrier. Labels are counted as occurrences, so equal inscriptions do not erase distinct transfers. For any exact label test T and a complete ledger L, define I_T(L), O_T(L) and B_T(L) as its input, output and named-boundary counts. The empty ledger satisfies

$$I_T([]) = O_T([]) = B_T([]) = 0$$

Appending an accepted output row increases I_T and O_T by one; appending an accepted boundary row increases I_T and B_T by one. Rejected rows change none of these counts. Induction on every row therefore gives

$$I_T(L) = O_T(L) + B_T(L)$$

The stronger ordered statement is also retained: projection of the complete input ledger onto its labels equals projection of the destination ledger onto its labels. Every destination occurrence has its original input, and every original occurrence has a destination. These results apply to each complete generated transition and to any exact label selection; neither a missing channel nor an unnamed boundary can be inserted into the pairing silently.

> Source, proof and provenance: V3 CONSERVED_LABELS, matter_nuclear_laws.py:L65; Lean exactCompleteLabelContinuity, MatterConservedLabels.lean:L35; Lean everyTargetHasItsOriginalInput, MatterConservedLabels.lean:L68; Timeline: Turn 494, line 4394.

428. Minimal recurrent trace classes and the observed particle spectrum

Let R be any finite list of recurrent constituent descriptions, and let the observable-trace type have decidable equality. The classification theorem quantifies over every such R and trace type; the registered measurement census is one application. Each description retains the complete constituent, a generated full-period return, its observable transition signature τ, and the source's list D of proper recurrent constituent decompositions. Particle names are descriptive fields. They do not enter τ or the minimality test.

Within this complete generated domain, a description is minimal exactly when its proper-decomposition list is empty:

$$\mathrm{minimal}(d) \iff D(d) = \emptyset$$

Let E(R) select the minimal descriptions from the finite source census R. The selection theorem proves both directions:

d∈E(R) ⇔ d∈R and D(d)=∅.

Any recorded proper recurrent decomposition therefore excludes its parent from E(R). The trace-class criterion is

$$d_1 \sim d_2 \iff \tau(d_1) = \tau(d_2)$$

Changing only a particle name preserves the class; changing the complete observable trace distinguishes it. Applying constructive duplicate removal to the complete selected signature list gives the spectrum

$$S(R) = \mathrm{unique}([\tau(d) : d \in E(R)])$$

The resulting list has no repeated signature, and its exact membership theorem is

τ∈S(R) ⇔ there exists d∈R with D(d)=∅ and τ(d)=τ.

Thus no minimal source class is omitted and no class lacking a minimal source representative is introduced. The source census supplies the generated decompositions and complete transition signatures; an external particle-table inscription cannot replace those structures or choose the equivalence law. The registered PDG rows remain the separate empirical membership boundary. Each retained constituent also preserves its original generated recurrence: an orientation trace of the complete phase-cycle length returns its prepared state exactly.

The proof proceeds through three exact list operations. Structural induction on R proves that filtering retains a description precisely when it occurs in R and has no recorded proper decomposition. Mapping the retained descriptions to their signatures therefore has exactly those signature preimages. Duplicate removal proceeds recursively: if a head signature occurs in the retained tail, discard that head; otherwise retain it. In either branch membership is unchanged, and induction proves distinctness. Combining these two membership equivalences yields the displayed spectrum equivalence for every R. Appending descriptions changes the source list supplied to the same theorem; there is no cardinality cutoff in its proof. elementaryExactly, MatterSpectrum.lean:L27; uniqueMembership, MatterSpectrum.lean:L90; uniqueDistinct, MatterSpectrum.lean:L115.

Additional proof bindings for the preceding conclusions: Lean spectrumExactly, MatterSpectrum.lean:L127; Lean spectrumNonEquivalent, MatterSpectrum.lean:L140.

> Source, proof and provenance: V3 PARTICLE_SPECTRUM, matter_nuclear_laws.py:L70; Lean recurrentSourceRetained, MatterSpectrum.lean:L143; Lean properCompositeExcluded, MatterSpectrum.lean:L54; Timeline: Turn 494, line 4396.

429. Boundary-closed composite hadrons

A joint hadron record retains its recurrent whole word, at least two complete constituents, its internal transfer ledger, and the observable total carriers and positive inertial support. If the constituent list is

$$C = [c_1, c_2] + +R$$

then |C|=|R|+2. Every internal transfer has both endpoints in the finite constituent index type Fin(|C|); the internal ledger therefore contains no endpoint outside the joint boundary.

Write departures(N) and arrivals(N) for the complete carrier projections of the internal network N. The original transfer pairing gives the ordered identity

$$\mathrm{departures}(N) = \mathrm{arrivals}(N)$$

Consequently, for every aggregate F of those complete carriers,

$$F(\mathrm{departures}(N)) = F(\mathrm{arrivals}(N))$$

The external observation retains the total-carrier record and the whole word's strictly positive inertial carrier:

$$\mathrm{external}(j) = (\mathrm{totalCarriers}(j), \mathrm{inertia}(j))$$

Changing internal transfer arrangements while holding those external carriers fixed leaves this observation unchanged. More generally, every external reading f obeys

$$\mathrm{external}(j_1) = \mathrm{external}(j_2) \implies f(\mathrm{external}(j_1)) = f(\mathrm{external}(j_2))$$

An internal label therefore cannot be distinguished separately by a reading of the same external totals. The generated proper decomposition [C] is retained in the joint description, so the composite-exclusion theorem removes that description from the elementary census. The whole joint word still has its complete

generated phase recurrence. This distinguishes boundary-closed composition from an elementary trace class without importing an external hadron taxonomy.

> Source, proof and provenance: V3 COMPOSITE_HADRONS, matter_nuclear_laws.py:L75; Lean externalRetainsTotalAndInertia, MatterHadrons.lean:L40; Lean closedCompositeExcludedFromElementary, MatterHadrons.lean:L52; Timeline: Turn 494, line 4398.

430. Complete scattering channels and exact cross-section measure

A scattering channel retains its incoming and outgoing constituent words, preparation record, detector conditions, source identity and observed success flag. Let L be the complete registered channel ledger and H(L) its exact successful-channel selection. The native selection theorem is

c∈H(L) ⇔ c∈L and successful(c)=true.

Induction on L gives |H(L)|≤|L|. In the source's positive observed-support domain, k=|H(L)|>0, hence n=|L|>0. The exact normalized support ratio w=w_n/w_d satisfies

$$kw_d = w_n n$$

For the registered positive geometry carrier g=g_n/g_d, the cross section is the existing exact composition

$$\sigma = w \cdot g$$

Its reduced positive representation satisfies the full source-normalized equation

$$(kg_n)\sigma_d = \sigma_n(ng_d)$$

The uniqueness theorem for positive reduced fractions proves that any admitted positive carrier satisfying this equation equals σ. No fitted multiplier is introduced. The geometry's recorded inscription does not select or alter the mathematical carrier used in that composition.

The complete incoming-to-outgoing map retains one row per original channel and preserves every preparation, detector and source field. Its length is n, and every original channel occurs in it. The empirical collision-event and reconstructed-outgoing-object records are checked separately against the sealed relation. The positive-support construction retains its registered domain rather than asserting a positive measured cross section when no successful event is present.

> Source, proof and provenance: V3 SCATTERING, matter_nuclear_laws.py:L80; Lean onlyExactCrossSection, MatterScattering.lean:L70; Lean everyChannelRetained, MatterScattering.lean:L80; Timeline: Turn 494, line 4400.

431. Conserved particle decay transitions

A decay record retains the recurrent input constituent, the lower-recurrence joint output, a nonempty list of product words, and the complete pairing of conserved labels and mechanical carriers. The mechanical-carrier type retains the original energy-transfer and momentum records; these are not replaced by untyped particle-name tokens. Each carrier is assigned to a generated product index or a named boundary carrier, together with its source trace.

Let L be this complete pairing ledger. Releasing and receiving its carriers gives the ordered identity

$$[(carrier, trace) : \mathrm{released}(L)] = [(carrier, trace) : \mathrm{received}(L)]$$

Every aggregate F of the full label and mechanical-carrier records consequently satisfies

$$F(\mathrm{released}(L)) = F(\mathrm{received}(L))$$

The occurrence-level statements hold in both directions: every received carrier has its source occurrence, and every source occurrence has its recorded destination. This is the exact closure that excludes an unrecorded source or an unpaired disappearance.

The generated product list has the form [p₁] ++ P, so its length is strictly positive. The retained finite recurrence comparison is

period(outputJoint)<period(input),

which excludes an unchanged self-return as the decay result. The source and product words each retain their own complete phase-cycle return traces:

$$A_{\text{input-period}}(\text{input}) = \text{input}$$

$$A_{\text{output-period}}(\text{outputJoint}) = \text{outputJoint}$$

The structural transition therefore retains both recurrent words, their ordered product support and every carrier crossing the boundary. The registered lifetime, decay-mode and branching records remain empirical comparisons to this sealed transition structure.

> Source, proof and provenance: V3 DECAY, matter_nuclear_laws.py:L85; Lean notUnchangedSelfReturn, MatterDecay.lean:L77; Lean everyOutputCarrierHasInput, MatterDecay.lean:L54; Timeline: Turn 494, line 4402.

432. State mixing as cyclic overlap of observation refinements

The common support is the complete prepared Fold state, including every word and held phase label. Let P(b) and I(b) be the exact preparation and interaction selections on a branch b. Their joint observed support in state s is

O(P,I;s)=[b∈branches(s) : P(b) and I(b)],

$$N(P, I; s) = |O(P, I; s)|$$

Selection is proved in both directions: a branch belongs to O exactly when it belongs to the complete original support and both selections accept it. Thus

0≤N(P,I;s)≤|branches(s)|.

The two selections retain their separate roles. A branch on which P and I disagree witnesses that they are different refinements, rather than two names for the same selection.

An admitted finite spin action advances held phases through the original phase cycle. For a generated action A with orientation trace a and cycle length m, the complete resulting branch sequence is determined by the original shift with residue |a| mod m. Two admitted actions with equal residues therefore give the same overlap support and exact count for the fixed observation refinements. The original word support is preserved:

support(A(s))=support(s).

A complete cycle returns the state and both observation quantities:

$$A_m(s) = s$$

$$O(P, I; A_m(s)) = O(P, I; s)$$

$$N(P, I; A_m(s)) = N(P, I; s)$$

A difference between overlap counts at intermediate generated actions necessarily distinguishes their observed supports. The mixing readout is this exact cyclic overlap, with the constituent support retained throughout. No additional constituent or stochastic source is introduced by a changed observed label. The separate registered mixing and oscillation records are evaluated against the sealed structural relation.

> Source, proof and provenance: V3 MIXING, matter_nuclear_laws.py:L90; Lean exactPhaseResidueDeterminesOverlap, MatterMixing.lean:L72; Lean fullCycleRestoresOverlap, MatterMixing.lean:L58; Timeline: Turn 495, line 4416.

433. Nuclear binding and the positive released carrier

Let the separated inertial carrier be u=u_n/u_d and the bound joint-word carrier be b=b_n/b_d. The source's binding domain has strictly greater separated support,

$$L = u_n b_d > R = b_n u_d$$

The positive reduction Δ is the existing normalized Fold remainder of these carriers. Its exact reduced representation satisfies

$$(L - R)\Delta_d = \Delta_n u_d b_d$$

Δ_n>0.

This is an oriented positive remainder, not a negative mass. Constructive remainder reconstruction proves the complete balance

$$L\Delta_d = \Delta_n u_d b_d + R\Delta_d$$

The positive reduced representation of Δ is unique: any admitted carrier satisfying the same exact cross-product equation equals this normalized reduction.

The bound joint record retains the full recurrent word, its complete constituents, source trace and named destination for the released carrier. Applying the already derived two-propagation conversion to Δ gives the released energy E=Δc². With c=c_n/c_d, the native equation is

$$(\Delta_n c_n^2)E_d = E_n(\Delta_d c_d^2)$$

The release therefore pairs the exact observed separation of inertial support with the corresponding positive energy carrier. The destination and complete source trace remain in the released record. The bound word retains its generated full-cycle return, and the registered nuclear mass, level and binding records retain their separate empirical role.

> Source, proof and provenance: V3 NUCLEAR_BINDING, matter_nuclear_laws.py:L95; Lean completeMassBalance, NuclearBinding.lean:L29; Lean uniqueReduction, NuclearBinding.lean:L37; Timeline: Turn 495, line 4418.

434. Finite nuclear recurrence classes and carrier transitions

A nuclear level retains its boundary-closed joint word, complete recurrence trace and observed class signature τ. Names do not select the class. For the complete finite source list L, the level census is

$$C(L) = \mathrm{unique}([\tau(l) : l \in L])$$

The constructive census has no duplicate signature and satisfies

$$\tau \in C(L) \iff \text{there exists } l \in L \text{ with } \tau(l) = \tau$$

Thus every represented recurrence class has its original level record, and no source class is lost when equivalent names are merged. Each joint word retains its generated complete phase-cycle return.

The energy carrier of a level is obtained from its original whole-word inertial support and the registered limiting-propagation carrier through the existing mass-energy construction. For initial and final positive energies x=x_n/x_d and y=y_n/y_d, write

$$L = x_n y_d$$

$$R = y_n x_d$$

If L>R, the transition emits the positive forward-held remainder with value (L−R)/(x_d y_d). If L<R, the positive remainder (R−L)/(x_d y_d) is counter-held, identifying absorption. In either case the exact normalized cross-product identity is retained. For the emitted remainder q, complete carrier reconstruction is

$$Rq_d + q_n x_d y_d = Lq_d$$

An unchanged level has an empty transfer, rather than an invented signed magnitude. The emitted or absorbed carrier is therefore the exact difference between the two named recurrence classes, with its orientation and full source records preserved.

> Source, proof and provenance: V3 NUCLEAR_LEVELS, matter_nuclear_laws.py:L100; Lean emittedDifference, NuclearLevels.lean:L42; Lean absorbedDifference, NuclearLevels.lean:L48; Timeline: Turn 495, line 4420.

435. Radioactivity and complete deterministic path weights

The source domain retains a nonempty complete list H of generated evolution paths from the prepared nuclear support. Every path contains its gate sequence, all predecessor states and its terminal state. At the registered observed boundary, unstable support has no stable self-return:

for every h∈H, observed(terminal(h))≠observed(initial).

Consequently terminal(h)≠initial for every retained path: an unchanged state could not produce the stipulated different observed reading. This is the exact transition exclusion associated with the absence of a closed observed return.

The original reversible-evolution proofs retain the stronger source statements. Every terminal branch has an original source branch and its complete generated branch trace; each path preserves the full number of branches. The predecessor-state list also has exactly one row per generated gate. Thus a transition in the observed nuclear support does not erase the underlying deterministic path record or create an unrecorded source.

For an event label e, select the complete event fibre

$$H_e = [h \in H : \text{observed}(\text{terminal}(h)) = e]$$

The selection theorem proves exact membership in both directions. For a represented event with k=|H_e|>0 and n=|H|>0, the event weight is the unique positive reduced carrier w satisfying

$$kw_d = w_n n$$

This weight is determined by the complete observer-relative path census. It introduces no ontic random choice or fitted probability. The registered decay, half-life and branching records remain the source's empirical comparison boundary.

> Source, proof and provenance: V3 RADIOACTIVITY, matter_nuclear_laws.py:L105; Lean exactEventSelection, NuclearRadioactivity.lean:L29; Lean forcedObservedTransition, NuclearRadioactivity.lean:L15; Timeline: Turn 495, line 4422.

436. Complete nuclear reaction channels

A nuclear reaction retains its complete incident and product joint words. Every transfer has a source and target identified by a finite word index or an explicitly named boundary. Its carrier is either a complete material joint word or an original held-label, energy or momentum carrier; provenance remains attached to the same occurrence.

For a complete channel ledger L, write S(L) and T(L) for its source and target records. Projection onto the carrier and its provenance gives the ordered identity

$$[(carrier, provenance) : S(L)] = [(carrier, provenance) : T(L)]$$

This retains more than equal aggregate totals. Each original channel has both its source row and its target row, every target row has an original channel preimage, and both ledger lengths equal |L|. Consequently every exact aggregate F of the complete carrier records satisfies

$$F(S(L)) = F(T(L))$$

The incident and product words retain their individual generated phase-cycle returns. The complete map therefore preserves the material, energy, momentum and held-label records while distinguishing each named boundary crossing. The registered reaction-channel, cross-section and product-yield records remain separate empirical checks of this closed map.

> Source, proof and provenance: V3 NUCLEAR_REACTIONS, matter_nuclear_laws.py:L110; Lean everyTargetFromOriginalChannel, NuclearReactions.lean:L61; Lean everyChannelHasSourceAndTarget, NuclearReactions.lean:L50; Timeline: Turn 496, line 4442.

437. Fission as conserved multiple-product decomposition

The fission refinement has exactly one incident bound joint word. Its product list is

$$P = [p_1, p_2] + +R$$

so |P|=|R|+2≥2. Each product retains its own complete constituent record and generated recurrence:

A_period(p)(p)=p for every p∈P.

The multiple-product condition excludes an internal excitation represented by a single output word: P cannot equal [q] for any q, because their list lengths differ. This distinguishes separately recurrent product composition from a change confined to one joint word.

The complete reaction ledger remains present. Its ordered material, label and mechanical-carrier projection is identical on the source and target sides, including the named carriers accompanying decomposition. Fission therefore preserves the same carrier identities and source traces as the general reaction law while imposing its particular one-to-multiple composition.

> Source, proof and provenance: V3 FISSION, matter_nuclear_laws.py:L115; Lean notSingleInternalExcitation, NuclearFission.lean:L28; Lean everyProductSeparatelyRecurrent, NuclearFission.lean:L41; Timeline: Turn 496, line 4444.

438. Fusion and the exact positive release

The fusion refinement retains multiple incident joint words and exactly one bound recurrent product. For inputs [i₁,i₂] ++ R, define their complete separated inertial support by the original positive Fold sum:

$$M([i]) = m(i)$$

M([i] ++ J)=m(i)+M(J) for nonempty J.

The separated carrier in the binding record is exactly this complete input sum; it is not an independently selected magnitude. The single output word is the same joint word held by that binding record. Its positive binding condition gives

M(inputs)>m(bound).

The normalized positive reduction Δ is obtained from this exact separation, and the released carrier is the original two-propagation energy conversion E=Δc². Its exact reduced identity is

$$(\Delta_n c_n^2) E_d = E_n (\Delta_d c_d^2)$$

E_n>0.

The release retains its named destination and source trace. The complete fusion record includes every input, the bound output, the original reaction channels and this exact release record. Material, held-label, energy and momentum carrier closure is inherited from the complete reaction ledger. The single product retains its generated full-cycle return. Thus the bound composition and released positive carrier remain linked to the same full incident support.

> Source, proof and provenance: V3 FUSION, matter_nuclear_laws.py:L120; Lean wholeInputExceedsBoundSupport, NuclearFusion.lean:L39; Lean namedPositiveRelease, NuclearFusion.lean:L49; Timeline: Turn 496, line 4446.

439. Exact colour coupling from complete Fold support

The admitted Fold recurrence gives generator period c=3. The original binary-count operation takes its unique positive predecessor, yielding b=2. Transport retains both distinct held Fold labels over all generator classes, so the normalized coupling is

$$g_c = b/c = 2/3$$

The native construction evaluates the original binary-count and generator-period operations before forming the positive reduced ratio. It yields numerator two and denominator three; no particle table, nuclear closure list or measured interaction strength enters the construction.

Complete replication does not change this ratio. For every positive whole replication count k,

$$(2k)/(3k) = 2/3$$

$$(2k) \cdot 3 = 2 \cdot (3k)$$

The constructive normalization theorem proves equality of the reduced positive carriers for every k, not only a fixed list of examples. Both held labels remain present and distinct. Moreover, any admitted positive reduced carrier q satisfying

$$2q_d = 3q_n$$

is exactly g_c. These identities supply the base and successor closure of the source construction.

The complete registered product contains 128 forms across carrier, participating numerator, normalization denominator, held orientation, exact ratio, target custody and extension. Exactly the form retaining the complete generator carrier, both held labels, the forced normalized ratio and closed target access survives. The original source, false-premise, artifact and boundary controls remain part of that complete product check.

The source grammar tests carrier, numerator, denominator, orientation, normalization, target and extension. Its ordered coordinate product has $2^7 = 128$ forms. The generator-three carrier and both held Fold labels force the numerator 2 and denominator 3; the label orientation is retained through normalization. The coordinate-membership and acceptance argument of Section 317 applies to this product, with the following native completeness and uniqueness proofs. candidatesComplete, ColourCouplingGrammar.lean:L107; uniqueSurvivor, ColourCouplingGrammar.lean:L147.

Grammar source: V3 COLOUR_COUPLING_SPEC, atomic_constants.py:L299.

> Source, proof and provenance: V3 colour_coupling, atomic_constants.py:L145; Lean forcedFromFoldAndGenerator, ColourCouplingDerivation.lean:L26; Lean binaryCountTwo, ColourCouplingDerivation.lean:L11; Timeline: Turn 496, line 4448.

440. Closed charged-colour proton-to-Planck hierarchy

The terminal hierarchy uses the same source-derived colour coupling g_c=2/3. Native correspondence identifies the existing hierarchy's positive colour-share construction with the independent complete-label-over-generator construction; it is not a separately adjustable coefficient.

The generated successor volume is 3^4=81. Its least binary cover is depth seven because

$$2^6 = 64 < 81 \leq 128 = 2^7$$

The complete support has 128 positions, whose unique positive predecessor is 127. The bare squared hierarchy is therefore H²=2^{127}, with the unsquared exponent retained as 127/2.

For the terminal electromagnetic share

$$\alpha = 3676744786/503846395469$$

the complete charged-colour removal and positive retained complement are

$$g_c\alpha = 7353489572/1511539186407$$

$$t = 1 - g_c\alpha = 1504185696835/1511539186407$$

The exact inequalities 0<7353489572<1511539186407 imply 0<t<1. Applying this retained transport once gives

$$H^2_{\text{terminal}} = 2^{127}t$$

$$= 255923934603817488008405160690199418432572494970880/1511539186407$$

Thus the terminal carrier is positive and strictly below the bare squared hierarchy. Both the lower hierarchy and the omitted-carrier alternatives remain explicit in the source's complete 1,024-form product. The full native program comparison retains every candidate, decision, witness, closure, control and source identity.

Independent validation accepts the complete original result and rejects candidate omission, an extra survivor and a failed control. The base hierarchy, electromagnetic share and independently derived colour coupling are all completed source dependencies of this terminal entry.

The source grammar tests carrier, terminal, support, relation, correction, sign, enumeration, minimality, measurement and extension. Its ordered coordinate product has $2^{10} = 1024$ forms. The lower-law hierarchy and its positive One complement fix the admitted single terminal transport; the target cannot choose its orientation or correction. The coordinate-membership and acceptance argument of Section 317 applies to this product, with the following native completeness and uniqueness proofs. candidatesComplete, PlanckTerminalGrammarBase.lean:L140; uniqueSurvivor, PlanckTerminalGrammar.lean:L205.

Grammar source: V3 precision_axes, precision_value_laws_v1.py:L128; V3 PROTON_PLANCK_TERMINAL_SPEC, precision_value_laws_v1.py:L178.

> Source, proof and provenance: V3 proton_planck_terminal_transport, precision_value_laws_v1.py:L106; V3 terminal_proton_planck_squared_ratio, precision_value_laws_v1.py:L116; V3 proton_planck_squared_ratio, lineage_particle_laws.py:L97; Lean terminalTransportStrict, PlanckHierarchy.lean:L92; Lean terminalAlphaExact, PlanckHierarchy.lean:L52; Lean existingShareRetainsBothLabelsOverGenerator, PlanckColourCorrespondence.lean:L10; Lean existingShareIsForcedCoupling, PlanckColourCorrespondence.lean:L5; Timeline: Turn 381, line 3706; Timeline: Turn 496, line 4450.

441. Exact charged and neutral source-count running

The bare row retains one source. Each colour-feedback successor appends both held Fold labels. The binary count is obtained as the positive predecessor of the independently generated period three, so b=3−1=2. Writing k for the number of successor operations, the source loop is

$$C_0 = 1, \qquad C_{k+1} = C_k + b = C_k + 2$$

Induction gives the complete result

$$C_k = 1 + 2k \qquad (k \geq 0).$$

The base is C_0=1+2·0=1. If C_k=1+2k, the next source row is C_{k+1}=1+2k+2=1+2(k+1). Consequently $C_{k+1}-C_k$=2 and $C_{k+1}>C_k$. The public positive level ℓ=k+1 therefore carries C(ℓ)=1+2(ℓ−1). Every finite prefix retains every generated position in its original order; the four-level prefix is (1,3,5,7).

A neutral carrier appends no feedback label. Its recurrence is $N_0 = 1$ and $N_{k+1} = N_k$, hence $N_k = 1$ at every level. Its seven-level prefix is (1,1,1,1,1,1,1). The source domains require positive generated integer levels and lengths; zero, negative integers and Boolean inputs are rejected. These are structural source counts and an exact successor slope, with the original exclusion of a measured renormalized beta coefficient retained.

The source grammar tests carrier, dependency, relation, enumeration, minimality, measurement, record and extension. Its ordered coordinate product has $2^8 = 256$ forms. The charged source successor and neutral hold above provide the admitted relation; imported beta functions and fitted slopes occupy the rejected relation coordinate. The coordinate-membership and acceptance argument of Section 317 applies to this product, with the following native completeness and uniqueness proofs. candidatesComplete, StrongRunningGrammar.lean:L118; uniqueSurvivor, StrongRunningGrammar.lean:L161.

Additional proof bindings for the preceding conclusions: Lean loopCorrespondence, StrongRunning.lean:L18; Lean exactCount, StrongRunning.lean:L26; Lean exactSuccessor, StrongRunning.lean:L33; Lean neutralEveryLevel, StrongRunning.lean:L84.

Grammar source: V3 common_axes, lineage_particle_laws.py:L164; V3 STRONG_RUNNING_SPEC, lineage_particle_laws.py:L262.

> Source, proof and provenance: V3 self_sourced_level_count, lineage_particle_laws.py:L114; V3 colour_running_prefix, lineage_particle_laws.py:L124; V3 neutral_running_prefix, lineage_particle_laws.py:L130; Lean levelZeroRejected, StrongRunning.lean:L90; Lean levelNegativeRejected, StrongRunning.lean:L92; Timeline: Turn 497, line 4468.

442. Exact vacuum screening and one-Fold exposure

The complete binary-support selection retains exactly the self-pairing half-One h=1/2, with h+h=1. The screening construction calls this selection and then applies the admitted Fold operation F. The previously derived return identity gives

$$s = h = 1/2, e = F(h) = 1$$

The ordered positive-fraction comparison is checked without approximation:

$$s_n e_d = 1 \cdot 1 = 1 < 1 \cdot 2 = e_n s_d$$

Thus s<e, and the exposed numerator and denominator coincide, e_n=e_d=1. These are the two guards in the original support construction. Every successful returned pair is exactly (h,1), so every returned pair satisfies the same strict increase. The result fixes the half-One screened endpoint, the exposed One and the direction of one complete screening-layer removal. Additional physical-scale correspondence remains the separately sealed empirical relation specified by V3; the two endpoints are not used to insert a fitted running curve or measured beta coefficient.

The source grammar tests carrier, dependency, relation, enumeration, minimality, measurement, record and extension. Its ordered coordinate product has $2^8 = 256$ forms. The exact half-One screening and One exposure above supply the admitted running direction; a fitted curve or reversed direction fails the relation coordinate. The coordinate-membership and acceptance argument of Section 317 applies to this product, with the following native completeness and uniqueness proofs. candidatesComplete, VacuumPolarizationGrammar.lean:L118; uniqueSurvivor, VacuumPolarizationGrammar.lean:L161.

Grammar source: V3 vacuum_axes, vacuum_lineage_laws_v1.py:L142; V3 VACUUM_POLARIZATION_SPEC, vacuum_lineage_laws_v1.py:L208.

> Source, proof and provenance: V3 vacuum_polarization_support, vacuum_lineage_laws_v1.py:L72; Lean everyReturnedDirection, VacuumPolarization.lean:L37; Lean anyReturnedSupport, VacuumPolarization.lean:L31; Timeline: Turn 497, line 4466.

443. Self-similar support and exact finite-tail convergence

Let m denote the retained label count. The empty host accumulator has support C_0=1, and each appended label position multiplies the support by m:

$$C_0 = 1, \qquad C_{d+1} = mC_d, \qquad C_d = m^d$$

The induction is exact at every finite depth: m·m^d=m^(d+1). For three labels the first three supports are 3, 9 and 27. Appending either held binary label increases word length by exactly one. The appended words are distinct and both return to the original word under the Fold return. The transported local separation doubles; the source witness is 2·(2/35)=4/35. Within the original exponent grammar {1,2,3,4}, the equation 2^e=2 retains exactly e=1.

At binary depth d the source's adjacent-carrier gap is

$$a_d = (2 + 2^d)/(3 + 2^d) - (1 + 2^d)/(2 + 2^d)$$

$$= 1/[(2 + 2^d)(3 + 2^d)]$$

Writing D_d=(2+2^d)(3+2^d), one has a_d=1/D_d. Both factors strictly increase with d because 2^(d+1)=2·2^d>2^d. Therefore D_(d+1)>D_d. Also D_d≥2^d, giving the exact majorant a_d≤2^(−d).

For the actual finite partial sum S(d,n)=Σ_(j=0)^(n−1) a_(d+j), retain an integer numerator and positive denominator, S(d,n)=N_(d,n)/Q_(d,n). Their recurrence is

$$N_{d,0} = 0, Q_{d,0} = 1$$

$$N_{d,n+1} = N_{d,n} D_{d+n} + Q_{d,n}$$

$$Q_{d,n+1} = Q_{d,n} D_{d+n}$$

The zero numerator here denotes the empty host accumulator. Every nonempty term is positive. Induction using D_(d+n)≥2^(d+n) proves

$$N_{d,n} 2^{d+n} + 2Q_{d,n} \leq 2^{n+1} Q_{d,n}.$$

or, equivalently,

$$S(d,n) + \frac{2}{2^{d+n}} \leq \frac{2}{2^d}.$$

The base is equality at n=0. For the successor, multiply the preceding inequality by the new positive denominator, insert the recurrence, and use 2^(d+n)≤D_(d+n) to bound the newly added numerator. This proves the bound for the actual summed gap ledger at every depth and count. The explicit positive remainder 2/2^(d+n) halves with every successor, so every finite tail satisfies S(d,n)<2/2^d. No measured tail or imported limiting value selects the bound.

The original fourteen successive denominator comparisons and eleven-term rational bracket are retained, including Σ_(d=0)^10 1/[(2+2^d)(3+2^d)]<11/60. The complete 256-form source grammar, closure and controls agree with fresh native execution, and all exported declarations have empty Lean axiom dependencies.

The source grammar tests scale, separation, information, power, support, series, generality and addition. Its ordered coordinate product has $2^8 = 256$ forms. The local doubling, one-label return, support recurrence and rational-tail induction above supply the scale, information, support and series criteria; the separate four-exponent census supplies the source power witness. The coordinate-membership and acceptance argument of Section 317 applies to this product, with the following native completeness and uniqueness proofs. candidatesComplete, SelfSimilarGrammarBase.lean:L118; uniqueSurvivor, SelfSimilarGrammar.lean:L61.

Here “every finite tail” quantifies over all natural depths d and counts n: neither is bounded by the fourteen source comparisons or eleven-term example. Likewise the support proof quantifies over all natural label counts m and depths d. The source exponent witness has the distinct four-entry domain $\{1,2,3,4\}$: its exact filter keeps 1. Thus the source witness domains are explicit, while support and rational-tail closure follow from the unrestricted successor inductions above. completeSupport, SelfSimilarConvergence.lean:L13; originalExponentCensus, SelfSimilarConvergence.lean:L28; everyFiniteTailBound, SelfSimilarConvergence.lean:L147.

Additional proof bindings for the preceding conclusions: Lean completePartialSumBudget, SelfSimilarConvergence.lean:L127; Lean everyFiniteTailBound, SelfSimilarConvergence.lean:L147.

Grammar source: V3 SELF_SIMILAR_CONVERGENCE, lineage_laws.py:L492.

> Source, proof and provenance: V3 SELF_SIMILAR_CONVERGENCE, lineage_laws.py:L492; Lean originalBracketWitnessPassed, SelfSimilarConvergence.lean:L88; Lean geometricMajorant, SelfSimilarConvergence.lean:L46; Timeline: Turn 498, line 4487.

444. Source-bound relational spacetime events

An event relation retains the original source observation s, the adjacent transition e→e′, its source identity and action record, one generated recurrence clock C, one oriented spatial relation x, and the complete causal predecessor path P:s→e. The complete source-to-event path is the append

$$P\prime = P \cdot (e \to e\prime)$$

Every edge of P′ belongs to the held adjacency relation. Its event ledger and transition ledger are exactly

$$\mathrm{Events}(P\prime) = \mathrm{Events}(P) + +[e\prime]$$

$$\mathrm{Changes}(P\prime) = \mathrm{Changes}(P) + +[\mathrm{record}(e \to e\prime)]$$

$$n(P\prime) = n(P) + 1$$

Thus the new event retains every predecessor and introduces exactly its generated transition. Mapping the complete transition ledger to source/action pairs reproduces the full original source trace. No causal predecessor is replaced by a numerical summary.

The recurrence clock has a positive closed transition count c=n(C). The complete path also has positive count n=n(P)+1. Its exact temporal relation is the reduced positive fraction t=n/c. Writing t=N/D, the normalization theorem gives

$$nD = Nc, N > 0, D > 0$$

The clock's recurrence closes on its original carrier. The spatial relation separately retains both observed supports, their reference identities, the positive exact separation and its held orientation. Numerical duration and separation do not replace the source observations or causal records.

Appending a further adjacent event retains the source and clock and extends both complete ledgers by exactly one record. A change in the held clock, spatial relation or source observation remains a distinguishable event relation. Reconstructing the relation from all of its retained fields gives the original relation exactly. These results formalize the source's operational event construction without an external background coordinate manifold.

The source's complete 256-form grammar and its original GWOSC source/detector target rows remain separate from these structural derivations. The empirical method seals the event relation before target release and retains the complete custody, detector-row and tampering controls.

The eight-axis construction of Section 353 applies here with relation coordinate `event-as-clock-space-causal-relation`, justified by the preceding derivation. candidatesComplete, SpacetimeEventGrammar.lean:L120 proves membership of every typed alternative; acceptanceIffExactCriteria, SpacetimeEventGrammar.lean:L172 identifies acceptance with this relation and the seven shared carrier, provenance, custody, record, row, successor and extension requirements.

> Source, proof and provenance: V3 EVENT_RELATION, spacetime_gravity_laws.py:L43; Lean fullSourceTrace, SpacetimeEvent.lean:L72; Lean completePredecessorChanges, SpacetimeEvent.lean:L47; Timeline: Turn 498, line 4489.

445. Canonical interval under reference relabeling

The interval construction retains an exact temporal relation t, positive spatial separation x, held orientation o and complete causal record c. Write their joint separation as s=(t,x,o,c). A reference description is d=(s,L), where L contains the description labels. Relabeling replaces only L:

$$R_{L\prime}(s,L)=(s,L\prime),\kappa(s,L)=s$$

The original equivalence is d~e exactly when κ(d)=κ(e). Equality of the canonical record is reflexive, symmetric and transitive. More precisely,

$$d\sim e \iff R_{\mathrm{labels}(e)}(d)=e$$

The forward direction substitutes the equal separation record and retains e's labels. The reverse direction projects the separation record from the relabeling equality. Thus equivalence changes reference labels only. Temporal relation, spatial separation, held orientation and causal records are individually preserved, and equality of all four fields suffices for equivalence.

Every separation has the representative rep(s)=(s,[]), with κ(rep(s))=s. For any result f that is invariant under every reference relabeling, define its induced interval result by g(s)=f(rep(s)). Then

$$g(\kappa(d))=f(R_{[]}(d))=f(d)$$

If another function h satisfies h(κ(d))=f(d) for every description, substitution of d=rep(s) gives h(s)=f(rep(s))=g(s). This is pointwise uniqueness of the induced result. It establishes the canonical interval's universal property without a quotient axiom or a chosen preferred reference label.

For a generated event relation r, the separation is constructed from its exact elapsed recurrence ratio, its retained positive spatial separation and orientation, and its complete causal transition ledger. The canonical projection preserves each of these source fields. A changed temporal, spatial or causal record cannot be removed by reference relabeling. The complete original 256-form grammar and the source's two CODATA correspondence rows retain their sealed prediction, custody and adverse-row checks.

The eight-axis construction of Section 353 applies here with relation coordinate `reference-invariant-event-separation-class`, justified by the preceding derivation. candidatesComplete, SpacetimeIntervalGrammar.lean:L120 proves membership of every typed alternative; acceptanceIffExactCriteria, SpacetimeIntervalGrammar.lean:L172 identifies acceptance with this relation and the seven shared carrier, provenance, custody, record, row, successor and extension requirements.

> Source, proof and provenance: V3 INTERVAL, spacetime_gravity_laws.py:L48; Lean fullSeparationSuffices, SpacetimeInterval.lean:L65; Lean factorUniqueAtEverySeparation, SpacetimeInterval.lean:L85; Timeline: Turn 499, line 4507.

446. Causal accessibility on finite recurrence support

Let A→B denote an admitted adjacent transition. A generated propagation path P from a to b is either the retained event at a=b or a predecessor path followed by one admitted adjacent transition. Its recurrence count n(P) is the length of its complete transition ledger. For held finite support R, define

$$a\preccurlyeq_R b \iff \text{there exists } P:a\to b \text{ with } n(P)\le R$$

The complete causal cone is C_R(a)={b | a≼_R b}. This definition includes every generated path within the retained support. If b lies outside C_R(a), no generated path from a to b can satisfy n(P)≤R. Every transition on a path witnessing membership retains its original adjacency proof and source/action record.

The retained event has no transition and witnesses a≼_R a. For paths P:a→b and Q:b→c, concatenation preserves their complete ordered ledgers and satisfies

$$n(P \cdot Q) = n(P) + n(Q)$$

Consequently a≼_R b and b≼_S c imply a≼_(R+S) c. Increasing the available support preserves any existing witness. Appending one admitted adjacent transition extends accessibility from R to R+1 and adds exactly that transition to the record.

The original antisymmetry statement concerns irreversible records. Such records retain a finite history H(a), and each irreversible adjacent extension strictly increases its length. Appending one record gives |H++[r]|=|H|+1>|H|. Induction on a generated path therefore gives |H(a)|≤|H(b)|; every nonempty irreversible path gives a strict increase. If both a≼_R b and b≼_S a hold, a nonempty forward path would imply

|H(a)|<|H(b)|≤|H(a)|,

which is impossible. The forward path must be the retained event, giving a=b. This preserves the source's distinction between irreversible record order and a reversible recurrence; no signed time or external continuum is introduced. The complete 256-form grammar and all original GWOSC correspondence, custody and adverse-row checks are retained.

The eight-axis construction of Section 353 applies here with relation coordinate `limiting-path-reachability-order`, justified by the preceding derivation. candidatesComplete, SpacetimeCausalOrderGrammar.lean:L120 proves membership of every typed alternative; acceptanceIffExactCriteria, SpacetimeCausalOrderGrammar.lean:L172 identifies acceptance with this relation and the seven shared carrier, provenance, custody, record, row, successor and extension requirements.

> Source, proof and provenance: V3 CAUSAL_ORDER, spacetime_gravity_laws.py:L53; Lean everyReachableStepAdjacent, SpacetimeCausalOrder.lean:L47; Lean pathHistoryMonotone, SpacetimeCausalOrder.lean:L65; Timeline: Turn 499, line 4509.

447. Complete prime-sector force and particle inventory

The previously derived closed prime ladder is P=(2,3,5,7). For each p∈P, the complete fibre has p held charge kinds. Its ordered pair cells number p²; removing the unique colourless return leaves p²−1 mediators. Holding every fibre position except the returning One gives the share (p−1)/p. Thus the full sector table is

p 2 3 5 7
charge kinds 2 3 5 7
mediators 3 8 24 48
holding share 1/2 2/3 4/5 6/7

At any admitted positive whole support R, the source running construction gives g_p(R)=(p+R−1)/(p+R). For p<q, common-denominator subtraction yields

g_q(R)−g_p(R)

$$= [(q+R-1)(p+R) - (p+R-1)(q+R)]/[(p+R)(q+R)]$$

$$= (q-p)/[(p+R)(q+R)] > 0$$

The numerator identity holds at every finite support. The original witness checks all six ordered pairs from P at R=8, retaining each exact fraction. A complete p-member fibre closes the sector singlet, while an antipodal pair has two constituents in every sector. The three-generation diagonal and adjacent-band shares are

$$A_p = (3p-1)/(3p), B_p = (2p-1)/(3p)$$

For p=(2,3,5,7), these give A=(5/6,8/9,14/15,20/21) and B=(1/2,5/9,3/5,13/21). The shortfall from the holding share is 1−(p−1)/p=1/p. Dividing this mass-part by the binary electron mass-part 1/2 gives 2/p, so the complete ratios are (1,2/3,2/5,2/7).

The finite gauge count is 3+8+24+48=83. The penta/hepta contribution is 24+48=72. Their two charge kinds across two new sectors and three generations give 2·2·3=12 Smithions. The complete named inventory is therefore 83+3+12=98, where the three additional names are photon, graviton and Higgs. The prime-sector ceiling excludes eleven as the first subsequent prime; no fifth sector is appended.

These are the original formal inventory and its retained predictions. The unobserved penta/hepta consequences remain in the record, with no measured coupling or known-particle table selecting the formulas. All 4,096 alternatives in the twelve-axis grammar are retained, together with the complete source guards, five witness families, closure and controls.

The source grammar tests sector, charge, mediator, coupling, running, singlet, mixing, mass, inventory, measurement, record and extension. Its ordered coordinate product has $2^{12} = 4096$ forms. The derived prime ceiling, pair-cell counts, coupling shortfalls and full singlet, mixing and mass records above supply the admitted inventory coordinates, including the retained penta/hepta predictions. The coordinate-membership and acceptance argument of Section 317 applies to this product, with the following native completeness and uniqueness proofs. candidatesComplete, SectorInventoryGrammarBase.lean:L162; uniqueSurvivor, SectorInventoryGrammar.lean:L205.

Grammar source: V3 axes, sector_inventory_law_v1.py:L85.

> Source, proof and provenance: V3 running_coupling, sector_inventory_law_v1.py:L26; V3 running_gap, sector_inventory_law_v1.py:L32; V3 fermion_mass_part, sector_inventory_law_v1.py:L57; V3 gauge_carrier_total, sector_inventory_law_v1.py:L65; V3 undiscovered_gauge_carrier_count, sector_inventory_law_v1.py:L69; V3 smithion_kind_count, sector_inventory_law_v1.py:L73; V3 complete_named_particle_count, sector_inventory_law_v1.py:L80; Lean exactElectronRatioRecord, SectorInventory.lean:L169; Lean runningAtEveryPositiveSupport, SectorInventory.lean:L79; Timeline: Turn 499, line 4511.

448. Exact source-boundary growth discrimination

The structural boundary-growth law is derived and sealed before the experiment. The observed compatible family cannot supply or alter that law. The boundary-growth method receives an exact distance ratio r>1 and a complete ordered response interval 1<L≤U after the prediction method is sealed. It starts with exponent one and predicted ratio r. Each successor multiplies the current prediction by the same held ratio. Every exponent whose prediction belongs to [L,U] is retained; the method stops at the first strict excess above U.

Write r=a/b with a>b>0 and U=u/v with v>0. These are representations of the held observations, not adjustable scientific constants. The positive-power recurrence is r^(n+1)=r^n·r. A constructive lower bound valid for every finite n is

b·a^n ≥ b^n(b+n).

The base n=0 is equality. For the successor, a≥b+1 and

$$(b+n)(b+1) = b(b+n+1) + n \geq b(b+n+1)$$

Multiplying the induction hypothesis by b+1 and then using b+1≤a establishes the bound at n+1. Thus the stopping depth can be derived from the actual input. Set N=bu+1. Since ub<b+N, the bound gives

u·b^N<a^N≤v·a^N,

so r^N>U. Every later power remains above U: multiplying a strict excess by b and using a>b preserves strict excess at the successor. Consequently every compatible positive exponent lies strictly below N. This is a proved termination bound derived from the observed rational inputs; it is not a fixed search cutoff.

The numerical selection is performed by exact whole cross-products:

$$L \le r^n \iff l \cdot b^n \le l_d \cdot a^n$$

$$r^n \le U \iff v \cdot a^n \le u \cdot b^n$$

where L=l/l_d. The source stop-scan is proved equal to the complete finite interval census: after the first strict excess, every remaining candidate in that census is rejected by the successor theorem. The first-excess index itself has a proved above-endpoint certificate, and its next power is strictly larger. No logarithm, fitted exponent, regression or imported physical dimension enters the method.

The original synthetic calls retain exponent three for r=3 and L=U=27, and for r=2 with [L,U]=[7,9]. For r=5/2 and U=40, the first strict-excess exponent is five, and the original monotonic-stop certificate is true. The public exact-fraction, distance-above-One and complete ordered response-interval guards are retained. The full ten-axis grammar contains 1,024 forms and retains the original source-bound, post-seal measurement boundary.

The source grammar tests source, distance, response, scale, candidates, selection, stopping, target, record and extension. Its ordered coordinate product has $2^{10} = 1024$ forms. The positive ratio recurrence, interval test and first strict excess prove the admitted candidate, selection and stopping coordinates for each supplied interval; a fixed search depth or first-match selector fails those criteria. The coordinate-membership and acceptance argument of Section 317 applies to this product, with the following native completeness and uniqueness proofs. candidatesComplete, BoundaryMeasurementGrammarBase.lean:L140; uniqueSurvivor, BoundaryMeasurementGrammar.lean:L205.

Grammar source: V3 SPEC, source_boundary_growth.py:L70.

> Source, proof and provenance: V3 compatible_positive_exponents, source_boundary_growth.py:L22; V3 monotonic_stop_certificate, source_boundary_growth.py:L52; Lean stopScanRetainsCompleteFamily, BoundaryGrowthDiscriminator.lean:L122; Lean originalMethodAlwaysCloses, BoundaryGrowthDiscriminator.lean:L191; Timeline: Turn 499, line 4515.

449. Generated common support and finite convergence witnesses

The common running coordinate starts at One and applies binary succession. At positive level n, write k=n−1. Induction on the exact succession loop gives

$$R_n = 2^{n-1}, R_{n+1} = 2R_n$$

The complete sector ladder P=(2,3,5,7) is held at every level. For p∈P and R>0, the inventory construction gives g_p(R)=(p+R−1)/(p+R). Its positive shortfall and binary-successor gain are

$$1 - g_p(R) = 1/(p+R)$$

$$g_p(2R) - g_p(R) = R/[(p+R)(p+2R)] > 0$$

For the latter identity, the common numerator is

$$(p+2R-1)(p+R)-(p+R-1)(p+2R)=R$$

For p<q, the exact pair gap is Δ_(p,q)(R)=(q−p)/[(p+R)(q+R)]. Both denominator factors strictly increase under R↦2R, while the positive numerator remains q−p. Hence

0<Δ_(p,q)(2R)<Δ_(p,q)(R).

Let the finite positive tolerance be 1/d. Starting at support One, the original binary-cover procedure stops at a generated support R≥(q−p)d. Its retained succession record identifies R=2^k and level k+1. Because p,q are positive,

(q−p)d≤R<(p+R)(q+R),

and therefore Δ_(p,q)(R)<1/d. This is a finite witness for each positive whole denominator, with no completed-infinite proof value. The source certificate retains all six ordered sector pairs and all six denominators (1,2,3,5,7,11), giving 36 witnesses; each satisfies both the binary-support bound and the strict gap comparison. The initial six levels retain supports (1,2,4,8,16,32). For example, required support 55 is covered at level seven with support 64.

The source's physical orientation record distinguishes two already derived carriers. Increasing retained self-source support corresponds to greater range and lower transfer scale; increasing exposed electromagnetic support after screening removal corresponds to a closer probe and higher transfer scale. Both formal holding shares increase with their generated support. A single imported energy-direction sign is not imposed, and measurements do not select either orientation.

The nine-axis grammar contains seven three-way choices—sector domain, scale generation, coupling form, successor law, pair-gap law, convergence law and physical translation—and two binary choices for target custody and extension. Thus its complete product has $3^7 2^2 = 8748$ forms. Its decision is the conjunction of the nine computed `candidateFacts`: the source ladder, generated support, coupling and gap observations, positive successor takes, convergence certificates, carrier translation, sealed targets and empty extension. The all-`c0` tuple alone satisfies them. candidatesComplete, RunningConvergenceGrammarBase.lean:L113; uniqueSurvivor, RunningConvergenceGrammar.lean:L29.

Grammar source: V3 source domains and predicates, coupling_running_convergence_terminal_law_v1.py:L68.

> Source, proof and provenance: V3 generated_scale_support, coupling_running_convergence_terminal_law_v1.py:L122; V3 generator_indexed_coupling, coupling_running_convergence_terminal_law_v1.py:L135; V3 coupling_successor_take, coupling_running_convergence_terminal_law_v1.py:L154; V3 generator_pair_gap, coupling_running_convergence_terminal_law_v1.py:L163; V3 convergence_witness, coupling_running_convergence_terminal_law_v1.py:L187; Lean allOriginalToleranceWitnesses, RunningConvergence.lean:L262; Lean successorHoldingNumerator, RunningConvergence.lean:L102; Timeline: Turn 500, line 4543.

450. Inertial-frame correspondence from complete invariant records

An inertial reference assigns a name to each complete generated record and recovers that same record from its name. Write N_A:R→L_A and D_A:L_A→R for naming and recovery in reference A. Completeness and faithful labeling give

$$D_A(N_A(r))=r, N_A(D_A(x))=x$$

For references A and B, the correspondence is therefore

$$T_{AB}(x)=N_B(D_A(x))$$

Recovery in B gives D_B(T_AB(x))=D_A(x). The inverse follows directly:

$$T_{BA}(T_{AB}(x)) = N_A(D_B(N_B(D_A(x)))) = N_A(D_A(x)) = x$$

Thus T_AB is injective, and each y in L_B has preimage T_BA(y), proving surjectivity. Identity and composition are forced by the same recovery equations:

$$T_{AA}(x) = x, T_{BC}(T_{AB}(x)) = T_{AC}(x)$$

The complete invariant record retains the event-coincidence identity, canonical interval class and limiting causal-path record. Equality of those three fields gives equality of the whole record. If a proposed relabeling F preserves all three fields, then D_B(F(x))=D_A(x), and applying N_B forces F(x)=T_AB(x). This is pointwise uniqueness of the source correspondence. A different coordinate assignment at any held record therefore fails at least one retained invariant.

Uniform recurrence and translation are retained through their generated successor S:R→R. The reference description of that successor is S_A=N_A∘S∘D_A. Substitution gives

$$T_{AB}(S_A(x)) = S_B(T_{AB}(x))$$

Induction on the finite recurrence count yields T_AB(S_A^n(x))=S_B^n(T_AB(x)) for every generated n. Event coincidence, interval class and causal accessibility are preserved throughout the complete successor record. The original 256-form grammar, two CODATA reference rows, source custody and adverse control remain attached to this correspondence.

The eight-axis construction of Section 353 applies here with relation coordinate `interval-preserving-frame-bijection`, justified by the preceding derivation. candidatesComplete, SpacetimeInertialGrammar.lean:L120 proves membership of every typed alternative; acceptanceIffExactCriteria, SpacetimeInertialGrammar.lean:L172 identifies acceptance with this relation and the seven shared carrier, provenance, custody, record, row, successor and extension requirements.

> Source, proof and provenance: V3 INERTIAL_TRANSFORMATION, spacetime_gravity_laws.py:L58; Lean threeInvariantsForceUniqueCorrespondence, SpacetimeInertial.lean:L107; Lean coordinateDistortionExcluded, SpacetimeInertial.lean:L88; Timeline: Turn 500, line 4539.

451. Post-seal running comparison and exact measurement intervals

The running-law prediction is fixed from the generated supports and complete sector certificate before the registered target package is released. Its ordered Fold word contains 21 inputs: four support values, the base and fourth-level pair gaps, a finite tolerance and its witness gap, five held relation labels, and the base and fourth-level shares of each of the four sectors. The prediction program reads each registered input in that order, forms the complete word and emits it. The measurement package retains all seven QCD Table 9.1 rows, every reported averaging column, nine Figure 9.4 measurement classes, two explicit multiscale strong-coupling rows, five current inverse-alpha rows, eleven hadronic Delta-alpha rows and both explicit prior electromagnetic endpoints.

For each positive central value x and uncertainty u<x, the exact measured interval is

$$I(x,u) = [x-u, x+u]$$

The low-scale strong-coupling record at Q=137/100 GeV is 0.445±0.114, giving I_low=[331/1000,559/1000]. The m_Z record is 0.1180±0.0009, giving I_Z=[1171/10000,1189/10000]. The scale record m_Z=91.1876±0.0021 GeV gives [182371/2000,911897/10000] GeV. Exact cross-products establish

137·2000 < 182371·100,
1189·1000 < 331·10000.

The scales are ordered and the upper high-energy coupling endpoint is below the lower low-energy endpoint. The reversed-direction control would require 559·10000<1171·1000, which is false.

The retained prior Thomson inverse-alpha record is 137.035999084±0.000000021, giving

$$I_0 = [137035999063/1000000000, 27407199821/200000000]$$

The explicit prior Z-scale record is 127.930±0.008, giving I_Z^em=[63961/500,63969/500]. Their separation is the exact integer inequality

63969·1000000000 < 137035999063·500.

Inverse alpha decreases, so positive electromagnetic alpha increases toward the Z scale. The current adopted inverse-alpha interval is retained separately as [34258999789/250000000,68517999599/500000000]; it does not replace the explicitly registered prior endpoint pair. Every hadronic interval and every reported strong-coupling averaging interval retains its positive lower endpoint.

The two observed energy directions agree with the previously derived carrier-specific orientation records: retained self-source support and electromagnetic screening exposure have opposite physical energy directions on the common generated support axis. The full vector figure remains identified by its immutable source hash; unprinted coordinates are not supplied by plot digitization. Removing one QCD row leaves six rather than seven, and removing one hadronic Delta-alpha row leaves ten rather than eleven, so both omission controls fail the complete-row conditions. These observational comparisons retain the source distinction between measured running directions and the formal common-support convergence prediction.

> Source, proof and provenance: V3 carrier_translation_record, coupling_running_convergence_terminal_law_v1.py:L214; V3 target_boundary_is_forced, coupling_running_convergence_terminal_law_v1.py:L379; V3 qcd_analysis, coupling_running_convergence_terminal_validation_v1.py:L144; V3 electromagnetic_analysis, coupling_running_convergence_terminal_validation_v1.py:L196; V3 exact_measurement_analysis, coupling_running_convergence_terminal_validation_v1.py:L244; Lean uncertaintyRetainsExactEndpoints, RunningConvergenceMeasurement.lean:L128; Lean wrongClaimRejected, RunningConvergenceMeasurementProofs.lean:L27; Lean strongLowExact, RunningConvergenceMeasurementProofs.lean:L11; Timeline: Turn 500, line 4543.

452. Elementary causal support and the invariant limiting ratio

An elementary source-bound event retains its adjacent causal transition, spatial support and recurrence clock. If the predecessor path has no transition, adjoining the elementary transition gives N=0+1=1. For a reference clock containing C positive recurrence units, the normalized duration t=n_t/d_t obeys N d_t=n_t C, hence d_t=n_t C in the elementary case. Let its positive spatial separation be l=n_l/d_l. The limiting ratio is the exact positive quotient

$$v_* = l/t, n_l d_t d_v = n_v d_l n_t$$

No speed is assigned to an absent recurrence: the EmptyOne temporal case has no quotient. Equality of the complete interval record preserves both spatial separation and positive duration, and therefore preserves their quotient. An inertial correspondence reads back the same elementary record, so v_*(b.read(b.name(a.read(x))))=v_*(a.read(x)). This is the boundary invariance inherited from the already derived exact frame correspondence.

For an accessible path making n elementary support advances during R retained recurrence units, causal accessibility supplies n≤R. Writing one elementary support-to-recurrence ratio as A/B, positivity gives

$$(nA)B \leq (RA)B = A(RB)$$

Thus nA/(RB)≤A/B. One elementary advance in one recurrence attains equality. This bound uses the complete retained causal path and its recurrence budget; it introduces no independent propagation coefficient.

The separately registered empirical relation uses all required CODATA Planck-length and Planck-time records. For their positive uncertainty intervals L=[L_−,L_+] and T=[T_−,T_+], the source program fixes the predicted limiting-speed interval before releasing the target:

$$V = [L_-/T_+, L_+/T_-]$$

The program reads the four endpoints, computes these two quotients, pairs them and emits the interval. The released target is the CODATA speed of light in vacuum. Its interval is compared with V; the adverse interval [V_++1,V_++2] is disjoint and must be rejected. This observational interval calculation retains the source distinction between the structural elementary boundary ratio and the external measurement records.

> Source, proof and provenance: V3 LIMIT_SPEED, spacetime_gravity_laws.py:L63; Lean inertialBoundaryRatioInvariant, SpacetimeLimitSpeed.lean:L49; Lean elementaryDurationExact, SpacetimeLimitSpeed.lean:L23; Timeline: Turn 500, line 4545.

453. Relative clock accumulation on complete causal paths

Let P and Q be complete causal paths between the same first and last events. Their positive retained recurrence counts are n=N(P) and m=N(Q). The source clock-rate correspondence is the normalized positive ratio

$$r = n/m, nd_r = n_r m, n_r > 0$$

Reversing the reference paths exchanges n and m; reversing twice returns the original complete path record. The correspondence therefore compares generated recurrence accumulations without introducing an absolute external time carrier.

The value is forced by the retained counts. If another positive reduced candidate q satisfies n d_q=n_q m, multiplication and cancellation give

$$m(n_q d_r) = (nd_q)d_r = (nd_r)d_q = m(n_r d_q)$$

Since m>0, n_q d_r=n_r d_q. Thus every count-preserving candidate has the same rational value as r; no independent clock-rate coefficient remains. For a third positive reference count k, exact normalized composition telescopes:

$$(n/m)(m/k) = n/k$$

An inertial naming correspondence reads back the same complete causal-path pair. Applying the recurrence-count ratio to the recovered pair therefore returns the original ratio. Common endpoints are retained in the path types, and the already established event, interval and limiting-boundary records remain unchanged under this naming correspondence. The empirical boundary remains the registered GWOSC timing and source-frame/detector-frame records.

> Source, proof and provenance: V3 CLOCK_RATE, spacetime_gravity_laws.py:L68; Lean inertialNamingPreservesRate, SpacetimeClockRate.lean:L40; Lean reverseRatioExact, SpacetimeClockRate.lean:L34; Timeline: Turn 500, line 4567.

454. Complete simultaneous support and relative spatial extent

Let U be the retained whole spatial support and let π assign each support observation its held clock label. The simultaneous slice at label τ retains exactly

$$S_\tau = \{s \in U : \pi(s) = \tau\}$$

Both directions of membership are retained: every member of S_τ belongs to U and has clock label τ; every member of U at τ belongs to S_τ. If τ≠σ, no support observation belongs to both slices, since that would give τ=π(s)=σ. The positive count of a slice is its complete list length. For two positive slice counts n and m, the relative extent is the normalized positive ratio a=n/m, satisfying n d_a=n_a m. A slice compared with itself has normalized numerator and denominator both One.

The paired change of extent and clock records is expressed in the original support-per-recurrence boundary. Let t and u be the respective positive recurrence counts. Preserving that boundary gives nu=mt. Put b=t/u. Normalization gives n d_a=n_a m and t d_b=n_b u. The retained pairing then forces

$$(mu)(n_a d_b) = (nd_a)(ud_b)$$

$$= (nu)(d_a d_b)$$

$$= (mt)(d_a d_b)$$

$$= (md_a)(td_b)$$

$$= (mu)(n_b d_a)$$

Since mu>0, n_a d_b=n_b d_a, so the extent and clock ratios have the paired exact value required by the preserved boundary. No coordinate metric or independent transformation coefficient is introduced by this construction. Inertial naming reads back the same complete slice, including all its spatial observations. The source empirical boundary retains the registered CODATA length/time relations and invariant-speed carrier.

> Source, proof and provenance: V3 LENGTH_RELATION, spacetime_gravity_laws.py:L73; Lean sliceMembershipExactly, SpacetimeLengthRelation.lean:L37; Lean sameSliceComparedWithItself, SpacetimeLengthRelation.lean:L71; Timeline: Turn 501, line 4573.

455. Universal free response and local inertial–gravitational equivalence

The source equivalence concerns complete local event traces under universal free propagation. Let P denote their common causal path class, and let G(c) and R(c) be the gravitational and uniform-reference descriptions for a held constituent label c. Universal response retains

$$G(c) = P, R(c) = P$$

Consequently G(c)=R(c). For every local observation F of the complete trace,

$$F(G(c)) = F(P) = F(R(c))$$

The same statement holds for different constituent labels c and d: F(G(c))=F(R(d)). Thus no held constituent label distinguishes the two local descriptions within this universal response. Conversely, a local observation with F(G(c))≠F(R(c)) excludes that universal-equivalence record. The equivalence does not erase the source condition that the complete traces lie in the same causal path class.

A common generated successor T preserves the construction: G′(c)=T(G(c)), R′(c)=T(R(c)), and P′=T(P). Applying T to the original equalities gives G′(c)=P′=R′(c). Iterating this construction retains equivalence after every finite number of generated successors. The common-path construction is explicit, and every local readout is handled pointwise without a function-extensionality axiom. The original external comparison remains the registered GWOSC free-propagation and common-detector-response records.

> Source, proof and provenance: V3 EQUIVALENCE, spacetime_gravity_laws.py:L78; Lean changedReadoutExcludesUniversalEquivalence, GravityEquivalence.lean:L35; Lean everyLocalReadoutAgrees, GravityEquivalence.lean:L19; Timeline: Turn 501, line 4575.

456. Source-bound transport and relational curvature

Each generated adjacent relation retains its first event, last event and source identity. A complete path is either the retained event or an existing path followed by one adjacent edge. Let T_e transport the held orientation across edge e. Transport along a path obeys

$$T_{\mathrm{empty}}(o) = o$$

$$T_{p\cdot e}(o) = T_e(T_p(o))$$

Its source record obeys Src(empty)=[] and Src(p·e)=Src(p)++[source(e)]. Thus every appended transition retains all earlier source identities and appends exactly its own source.

For two generated paths p and q with the same endpoints and initial held orientation o, the source curvature distinction is the nonclosure

T_p(o)≠T_q(o).

A path cannot witness nonclosure against itself. Any nonclosing comparison therefore distinguishes two paths. If transported orientation is path-independent on the held support, T_p(o)=T_q(o) for every such pair, no pair witnesses curvature there. This gives the source distinction beyond flat support entirely through adjacent source-bound relations, without a background embedding.

Changing the held orientation names by an injective map h preserves the distinction exactly:

$$h(T_p(o)) \neq h(T_q(o)) \iff T_p(o) \neq T_q(o)$$

The forward implication follows because equal orientations have equal names; the reverse follows from injectivity. The source identity and complete transport trace remain attached to the nonclosing comparison. The empirical boundary retains the original GWOSC compact-source waveform, propagation and polarization records.

> Source, proof and provenance: V3 CURVATURE, spacetime_gravity_laws.py:L83; Lean pathIndependentCannotWitnessCurvature, GravityCurvature.lean:L48; Lean injectiveOrientationNamesPreserveNonclosure, GravityCurvature.lean:L53; Timeline: Turn 501, line 4577.

457. Complete inertial recurrence along an unforced curved path

A local free step retains its source-curved adjacent event relation, incoming word w, outgoing word w′ and the established inertial recurrence T. The local condition is w′=T(w), and its list of extra force carriers is empty. Concatenation requires the next incoming word to equal the previous outgoing word; thus it preserves the entire propagating word rather than merely a transition count.

For the retained event path, N=0 and the word is unchanged. Appending one local step gives

$$N(p \cdot e) = N(p) + 1$$

$$w_{\text{final}}(p \cdot e) = T(w_{\text{final}}(p))$$

Induction therefore yields w_final(p)=T^N(p)(w_initial). Likewise, the complete extra-force list satisfies F(empty)=[] and F(p·e)=F(p)++F(e); since every local F(e)=[], F(p)=[] for every finite free path. Projecting the local steps to their adjacent source-curved relations retains a complete GravityCurvature path with the same event endpoints.

A minimal free path p between the fixed endpoint events from the held initial word satisfies N(p)≤N(q) for every competing complete free path q in that source relation, retaining its own resulting word. If both p and q satisfy this condition, the two inequalities give N(p)=N(q). Conversely, a competing q with N(q) <N(p) excludes p from the minimal class. These statements retain the distinction between equality of the forced minimum transition count and identity of entire path records.

The generated-support selection uses the existing V3 finite geometry path-distance construction. For admitted distinct endpoint labels and any generated adjacent trace p, it returns a generated trace s with |s|≤|p|. The shortest-path theorem quantifies over every competing generated trace, including longer traces with repeated visits; it does not truncate competitors to a chosen depth. A returned absence is equivalent to the absence of any generated trace between the admitted endpoints. The construction introduces neither a new metric nor a selected path-length parameter.

> Source, proof and provenance: V3 GEODESIC, spacetime_gravity_laws.py:L88; Lean everyTransitionUnforced, GravityGeodesic.lean:L46; Lean completeRecurrencePreserved, GravityGeodesic.lean:L55; Timeline: Turn 502, line 4597.

458. Realization and minimality of the complete geodesic trace

The geometry selection and locally unforced propagation describe the same registered support paths. A supported free path retains each adjacent endpoint pair in the generated curved-support relation. Its complete event list E(p) satisfies

$$E(empty_a) = [a]$$

$$E(p \cdot e) = E(p) + +[\text{last}(e)]$$

$$|E(p)| = N(p) + 1$$

Induction on the free path establishes that E(p) is a generated geometry trace. Conversely, induction on any generated geometry trace constructs a complete free realization. The retained event begins with the initial word. At an adjacent step from a to b, the construction retains the held source identity, takes the current word w to T(w), appends an empty extra-force record and joins the words exactly. Every generated trace therefore has a free-path realization with precisely its complete event list, unchanged source provenance, empty extra-force list and final word T^N(w_initial).

Let s be the trace returned by the existing shortest-path construction for admitted distinct endpoints. Its free realization p has E(p)=s. For every supported competing free path q, the converse construction gives a generated trace E(q). The previously proved geometry inequality therefore gives

$$N(p) + 1 = |s| \leq |E(q)| = N(q) + 1$$

hence N(p)≤N(q). This connects actual selection to minimal free propagation, rather than merely declaring a path minimal. It ranges over every supported competing path and does not fix their resulting words or impose a selected depth cutoff. All original source, recurrence and force records remain attached to p.

The original source boundaries are retained. At an admitted identical endpoint, the geometry result is EmptyOne, while the complete retained-event record has no transition and leaves the initial word unchanged. An endpoint outside the generated carrier is rejected. For distinct admitted endpoints, absence of a selected result is equivalent to the absence of a generated path. The empirical comparison remains the registered GWOSC orbital, propagation and source-trajectory records.

> Source, proof and provenance: V3 GEODESIC, spacetime_gravity_laws.py:L88; Lean everyReachableSupportHasSelectedPath, GravityGeodesicSelection.lean:L11; Lean selectedPathIsShortest, GravityGeodesicSelection.lean:L23; Lean selectedTraceHasCompleteFreeRealization, GravityGeodesicBridge.lean:L52; Lean selectedFreePathIsMinimal, GravityGeodesicBridge.lean:L63; Timeline: Turn 502, line 4597.

459. Complete inertial-energy source and gravitational boundary closure

A registered gravitational relational change retains its originating inertial-energy event, destination event, complete adjacent propagation trace and both source-curved comparison paths between those endpoints. The source energy is the already derived positive value E=mc² of the originating conversion record. Its preparation identity and complete action record are retained with the transfer; no independent source coefficient enters the construction.

For a complete list of changes C=[c₁,...,c_n], construct the transfer ledger by

$$L(C) = [(\mathrm{first}(c), \mathrm{last}(c), c, \mathrm{source_identity}(c), \mathrm{action_record}(c)) : c \in C]$$

Projecting the carrier field returns C exactly. Thus no comparison path, propagation trace, energy source or constituent record is discarded in forming the ledger. Every propagation step remains adjacent, every distinct arrival has an adjacent predecessor, and projecting source/action pairs from the complete trace recovers every original source record.

The complete source/destination accounting gives the equality of whole carrier lists

$$\mathrm{Outgoing}(L(C)) = \mathrm{Incoming}(L(C))$$

Applying the source-energy reading to both sides preserves equality of the exact energy-reading lists. The result retains each full carrier and its position in the ledger, rather than checking only the number of transfers.

For a held interior predicate I, an outward transfer belongs to the ledger, begins inside and ends outside. Conversely, every ledger transfer satisfying those two endpoint conditions belongs to the outward record. The existing boundary-conservation derivation then gives

$$\mathrm{InteriorLossCarriers}(I, L(C)) = \mathrm{BoundaryOutflowCarriers}(I, L(C))$$

Applying the exact energy reading gives the corresponding equality of interior-loss and outward-energy lists. Hence an outward field transfer cannot appear without its complete registered source change, and an admitted outward change cannot be omitted from the boundary account. The external boundary retains the original GWOSC source mass-energy, radiated-energy and detector-strain records.

> Source, proof and provenance: V3 FIELD_SOURCE, spacetime_gravity_laws.py:L93; Lean noOmittedOutwardChange, GravityFieldSource.lean:L81; Lean interiorChangeExactlyOutwardTransfer, GravityFieldSource.lean:L62; Timeline: Turn 502, line 4599.

460. Gravitational-wave propagation

Let Δ be a source-curvature change with its complete adjacent causal trace, and let R be a recurrence whose initial carrier is Δ. Recurrence closure and the initial-carrier equation give

$$R.last.carrier = R.first.carrier = \Delta$$

The transverse support is taken from the already derived three-direction support S with one held normal n. The boundary construction retains exactly

$$B = \{d \in S : d \neq n\}$$

$$|B| + 1 = |S| = 3$$

so |B| = 2. A retained polarization p ∈ B therefore satisfies p ≠ n; no second nonnormal direction is discarded. This applies to the source's transverse-recurrence condition rather than imposing a further empirical polarization fit.

For the elementary boundary relation issuing from the source change, complete causal counting gives N = 1. If separation and recurrence duration are represented by s = a/b and t = c/d, the limiting ratio v = u/w obeys

$$(ad)w = u(bc), a, b, c, d, u, w > 0$$

The result reuses the established elementary causal-boundary ratio. The source energy is the actual positive inertial-energy carrier E(Δ), and the transported momentum is retained as a complete momentum carrier whose content agrees with that source. The detached observation construction is

$$O_j = (W, none, r_j)$$

where W contains Δ, recurrence, boundary relation, energy source, momentum and polarization. For every finite list of detector records, projection recovers W at every position and therefore preserves (E(W), P(W), p(W)) exactly, while the continuing local-source-contact projection is none. The complete trace also retains every adjacent transition and every source/action pair.

The original 256-form product and its controls are retained in full. The GWOSC boundary uses the original sealed relation, timing/polarization rows and complete original validator, without introducing a new fitted wave coefficient or extending its empirical scope.

> Source, proof and provenance: V3 GRAVITY_WAVE, spacetime_gravity_laws.py:L98; Lean noContinuingContact, GravityWave.lean:L70; Lean elementaryPropagation, GravityWave.lean:L42; Timeline: Turn 503, line 4620.

461. Causal horizon boundary

Let G = (V,E) be an admitted finite directed causal-support graph and o the observer. Write P_G(x,o) for a valid generated path from x to o. The existing complete directed reachability construction establishes

$$reachable_G(x, o) = true \leftrightarrow P_G(x, o)$$

Define the accessible and confined classes by

$$A_o(x) : \Longleftrightarrow \ reachable_G(x, o) = true$$

$$H_o(x) : \Longleftrightarrow \ reachable_G(x, o) = false$$

Boolean case analysis classifies every event into exactly one class, and path completeness gives H_o(x) ↔ ¬ P_G(x,o).
This is a typed path exclusion, not a test of descriptive wording.

Every outward continuation remains confined: if H_o(x) and P_G(x,y), then H_o(y). Otherwise P_G(y,o), concatenated with P_G(x,y), would construct P_G(x,o), contradicting confinement. Thus

$$H_o(x) \wedge P_G(x, y) \implies H_o(y)$$

For o ∈ V, the identity path gives A_o(o). The finite census

$$C_o = [(x, reachable_G(x, o)) : x \in V]$$

satisfies map(first,C_o) = V, so no registered event is omitted. Direction is preserved throughout; an outward path is never manufactured by reversing an edge. These results establish the source's observer-relative accessibility boundary and exclusion of information transfer without a generated causal trace.

The original complete 256-form grammar, GWOSC compact-object and ringdown boundary, and all original controls remain unchanged.

The eight-axis construction of Section 353 applies here with relation coordinate `outward-causal-path-accessibility-boundary`, justified by the preceding derivation. candidatesComplete, GravityHorizonGrammar.lean:L120 proves membership of every typed alternative; acceptanceIffExactCriteria, GravityHorizonGrammar.lean:L172 identifies acceptance with this relation and the seven shared carrier, provenance, custody, record, row, successor and extension requirements.

> Source, proof and provenance: V3 HORIZON, spacetime_gravity_laws.py:L103; Lean confinedExactlyNoOutwardTrace, GravityHorizon.lean:L17; Lean everyOutwardContinuationRemainsConfined, GravityHorizon.lean:L43; Timeline: Turn 503, line 4622.

462. Propagation redshift relation

Retain the emitted wave, observed wave, both clock recurrences, source observation and complete source-to-observer causal path. If an emitted cycle takes n_e transitions against m_e clock transitions, and the observed cycle takes n_o against m_o, the established period/frequency construction gives

$$T_e = n_e/m_e, \nu_e = m_e/n_e$$

$$T_o = n_o/m_o, \nu_o = m_o/n_o$$

Every count is positive because these are actual recurrences. The emitted-to-observed recurrence-frequency ratio is therefore

$$R = \nu_e/\nu_o = (m_e n_o)/(n_e m_o)$$

For normalized frequency fractions ν_e = a/b and ν_o = c/d, its reduced representation R = u/w satisfies

$$(ad)w = u(bc)$$

Positive normalization proves uniqueness: any positive reduced fraction satisfying that same exact cross-product equation equals R. This positive ratio is the recurrence comparison itself; it is not an identification with the potentially signed conventional offset z.

The record retains both observation-clock carriers, emission and observation events, and every adjacent path transition and source/action record. Changing source or detector names alone leaves R unchanged. This separates an actual change of the paired recurrence reading from relabeling while preserving the full source-to-observer provenance. The original complete 256-form grammar and GW231123 source/detector boundary are retained; no fitted redshift parameter is introduced into the derivation.

The eight-axis construction of Section 353 applies here with relation coordinate `emission-observation-recurrence-ratio`, justified by the preceding derivation. candidatesComplete, CosmoRedshiftGrammar.lean:L120 proves membership of every typed alternative; acceptanceIffExactCriteria, CosmoRedshiftGrammar.lean:L172 identifies acceptance with this relation and the seven shared carrier, provenance, custody, record, row, successor and extension requirements.

> Source, proof and provenance: V3 REDSHIFT, cosmological_boundary_laws.py:L39; Lean forcedRatioUnique, CosmoRedshift.lean:L30; Lean exactFrequencyRatio, CosmoRedshift.lean:L22; Timeline: Turn 503, line 4624.

463. Relational expansion observation

For each registered source observation, let L be the complete finite separation-support count and C the observation-clock recurrence count. Both are positive. Its exact relative support is q = L/C, with normalized representation q = a/b satisfying

$$Lb = aC$$

A before/after source-bound pair retains the same source identity and both complete observation records. Its change direction is determined by comparing
a_after b_before and a_before b_after.
A greater first product is growth, equality is unchanged relative support, and a smaller first product is contraction. No signed proof magnitude or external spatial container is required.

For a declared finite census with at least two distinct registered sources, coherence in direction d is defined recursively by

$$\mathrm{coherent}(d, []) = True$$

$$\mathrm{coherent}(d, x :: xs) \leftrightarrow \mathrm{direction}(x) = d \wedge \mathrm{coherent}(d, xs)$$

Induction proves that every registered member has direction d. Conversely, any registered discordant member excludes coherence. The successor equation states exactly when adding a source preserves the shared direction; earlier source and path records remain present. Thus a selected local change cannot stand in for the declared multi-source census.

A common finite refinement k>0 preserves the relative support:

$$\mathrm{normalize}(kL, kC) = \mathrm{normalize}(L, C)$$

Indeed, Lb=aC implies (kL)b=a(kC), and uniqueness of positive normalization gives the equality. Coherent expansion therefore uses exact source-separation support relative to clock recurrence without choosing an absolute scale. The original complete candidate product, observational scope and NASA-LAMBDA boundary are preserved.

The eight-axis construction of Section 353 applies here with relation coordinate `coherent-source-separation-clock-change`, justified by the preceding derivation. candidatesComplete, CosmoExpansionGrammar.lean:L120 proves membership of every typed alternative; acceptanceIffExactCriteria, CosmoExpansionGrammar.lean:L172 identifies acceptance with this relation and the seven shared carrier, provenance, custody, record, row, successor and extension requirements.

> Source, proof and provenance: V3 EXPANSION, cosmological_boundary_laws.py:L44; Lean everyRegisteredSourceAgrees, CosmoExpansion.lean:L72; Lean everySourceHasCommonOrientation, CosmoExpansion.lean:L35; Timeline: Turn 503, line 4626.

464. Background-radiation field relation

Let D and F be the declared finite direction and frequency supports. The complete observation support is their ordered product

$$D \times F = concat_{d \in D}[(d, f) : f \in F]$$

The cell census retains exactly this product, together with each cell's complete source-bound observation and foreground/background contributions. If G and B extract the foreground and background contributions from a retained sequence R, structural recursion gives

$$|G(R)| + |B(R)| = |R|$$

x ∈ B(R) ↔ background(x) ∈ R.
The complete tagged sequence remains available, so foreground separation does not replace the original observation history.

For the retained thermodynamic class θ, frequency consistency is

$$\forall r \in cells, \mathrm{class}(r) = \theta$$

The cell census is nonempty. If both θ and θ′ satisfy this equation, selecting any retained cell gives θ = class(r) = θ′. A discordant retained cell excludes consistency. Together with the full declared direction/frequency product, this distinguishes the source's background class from a favorable selection of cells. These are statements over the registered finite observations, preserving the Physics-to-cosmology boundary.

The original independent FIRAS comparison uses two separately registered snapshots. The input record 2.725 ± 0.002 K gives

$$I = [2723/1000, 2727/1000]K$$

The withheld monopole record 2725 ± 1 mK gives

$$J = [681/250, 1363/500]K$$

The exact four-instruction program reads the input lower and upper endpoints, pairs them and emits that pair before the independent target is released. Its invariant prediction is I. The comparison is

$$\mathrm{overlap}([a,b],[c,d]) \iff \neg(b < c \vee d < a)$$

Here J lies inside I, so overlap holds. The original adverse control displaces both endpoints above the predicted upper bound:

$$U = [3727/1000, 4727/1000]K$$

$$\mathrm{overlap}(I,U) = false$$

Both parsed intervals, the overlap and rejection have direct kernel proofs with no axiom dependencies. These temperatures are registered empirical observations; they are not introduced as adjustable parameters in the structural derivation. The full native validator also reproduces the original source hashes, sealed input program, independent target custody, hostile-package audit, certificates and receipt.

> Source, proof and provenance: V3 BACKGROUND, cosmological_boundary_laws.py:L49; Lean discordantCellExcludesBackgroundClass, CosmoBackground.lean:L85; Lean noDeclaredDirectionFrequencyOmitted, CosmoBackground.lean:L78; Lean sourceMismatchRejected, CosmoBackgroundMeasured.lean:L164; Lean independentIntervalsOverlap, CosmoBackgroundMeasured.lean:L181; Timeline: Turn 505, line 4640.

465. Gravitational lensing relation

Each retained image path has the same emitter e and observer o and carries a complete generated adjacent causal trace p:e→o. Its source-curved trace is constructed recursively:

$$C(\mathrm{one}(e)) = \mathrm{empty}(e)$$

$$C(\mathrm{append}(p,\delta)) = \mathrm{append}(C(p), \mathrm{edge}(\mathrm{sourceIdentity}(\delta)))$$

Consequently

$$\mathrm{sources}(C(p)) = \mathrm{map}(sourceIdentity, \mathrm{changes}(p))$$

Every curving source therefore comes from the original causal trace. No additional interaction is inserted into the path construction.

For the complete declared finite path census P, let d(p), t(p) and S(p) denote the retained direction record, causal arrival count and observed support. The full observation projection is

$$I(P) = [(p, d(p), t(p), S(p)) : p \in P]$$

Structural induction gives

$$\mathrm{map}(first, I(P)) = P$$

$$|I(P)| = |P|$$

$$\mathrm{map}(observedSupport, I(P)) = [S(p) : p \in P]$$

Thus image multiplicity and the complete observed-support projection retain every registered path. Arrival comparison keeps the ordered pair (t(p),t(q)), preserving both recurrences rather than discarding one endpoint in a delay label.

The existing source-curvature transport gives a further distinction. If transporting a held direction along C(p) and C(q) does not close, then C(p) ≠ C(q), hence p ≠ q. The same emitter/observer pair can therefore retain distinct source-curved paths with distinct directional observations. The original complete candidate grammar and NASA-LAMBDA observational boundary remain unchanged.

The eight-axis construction of Section 353 applies here with relation coordinate `multiple-curved-source-observer-paths`, justified by the preceding derivation. candidatesComplete, CosmoLensingGrammar.lean:L120 proves membership of every typed alternative; acceptanceIffExactCriteria, CosmoLensingGrammar.lean:L172 identifies acceptance with this relation and the seven shared carrier, provenance, custody, record, row, successor and extension requirements.

> Source, proof and provenance: V3 LENSING, cosmological_boundary_laws.py:L54; Lean everyCurvingSourceRetained, CosmoLensing.lean:L12; Lean nonclosingDirectionsRequireDifferentPaths, CosmoLensing.lean:L60; Timeline: Turn 505, line 4642.

466. Causal distance and observation-time relation

For a source-bound observation r, define its retained support signature

$$\Sigma(r) = (source, emissionEvent, observationEvent, completeCausalChanges, emittedRecurrence, observedRecurrence)$$

The distance-support equivalence is exactly

$$r\ s \iff \Sigma(r) = \Sigma(s)$$

Equality supplies reflexivity, symmetry and transitivity. Its exact class is retained as the predicate

$$[r](s) \iff \Sigma(r) = \Sigma(s)$$

without introducing a quotient axiom. Both recurrence carriers and every causal change remain explicit components of Σ.

Distance observations retain their declared projection kind: luminosity, propagation, or simultaneous separation. Two projections are equivalent only when

$$\mathrm{kind}(p) = \mathrm{kind}(q) \wedge \Sigma(\mathrm{record}(p)) = \Sigma(\mathrm{record}(q))$$

Different kinds therefore cannot be conflated, even when they concern the same source and observer. Source/detector relabeling changes neither the support signature nor its equivalence class. The construction preserves the original finite causal support and observation-time conditioning, together with the GW231123 luminosity-distance and source-frame boundary; it does not introduce an additional cosmological fit.

> Source, proof and provenance: V3 DISTANCE, cosmological_boundary_laws.py:L59; Lean namingDoesNotAlterSupportClass, CosmoDistance.lean:L39; Lean bothRecurrenceCarriersRetained, CosmoDistance.lean:L45; Timeline: Turn 505, line 4644.

467. Finite structure growth and retained histories

Let S be the registered finite support of source-density network states and let F be its admitted single-valued transition map. The exact density records enumerate the registered nodes; the gravity and expansion records retain the transition-source support. A history of n steps has the form h=(x_0,...,x_n), with F(x_i)=x_{i+1} at each appended step. The native history construction retains both the state list and every ordered transition record.

The retained-history base and successor equations are

$$\mathrm{states}(\mathrm{start}(x_0)) = [x_0]$$

$$\mathrm{states}(\mathrm{append}(h, x)) = \mathrm{states}(h) + +[x]$$

$$\mathrm{transitions}(\mathrm{append}(h, x)) = \mathrm{transitions}(h) + +[(\mathrm{last}(h), x)]$$

Consequently |states(h)|=n+1. If h and h' have the same initial state and depth, their endpoints and complete state lists coincide. Induction uses uniqueness of the registered successor at each step: F(x)=y and F(x)=z imply y=z. It does not introduce an independently selected cosmological history or a stochastic transition.

An unresolved observation fibre retains its constituent histories. For the separately supplied finite nonempty observation support H and a represented nonempty fibre $B \subseteq H$, its exact support weight is $|B|/|H|$; the realization retains the class label, exact support part and every member. Thus the weight records unresolved distinctions rather than changing the underlying deterministic evolution. The initial density support and complete registered interaction/expansion provenance remain explicit.

The entry preserves the original 256-form grammar and original external-observation boundary. Its NASA source rows remain empirical records compared through the original sealed validator; no external cosmic census or historical fit is promoted to a premise of the finite law.

The eight-axis construction of Section 353 applies here with relation coordinate `finite-density-network-dynamical-growth`, justified by the preceding derivation. candidatesComplete, CosmoStructureGrowthGrammar.lean:L120 proves membership of every typed alternative; acceptanceIffExactCriteria, CosmoStructureGrowthGrammar.lean:L172 identifies acceptance with this relation and the seven shared carrier, provenance, custody, record, row, successor and extension requirements.

Additional proof bindings for the preceding conclusions: Lean completeHistoryLength, CosmoStructureGrowth.lean:L53; Lean finiteHistoryDeterministic, CosmoStructureGrowth.lean:L80.

> Source, proof and provenance: V3 STRUCTURE_GROWTH, cosmological_boundary_laws.py:L64; Lean unresolvedWeightsRetainTheirMembers, CosmoStructureGrowth.lean:L94; Lean sourceStatement, CosmoStructureGrowth.lean:L101; Timeline: Turn 514, line 5058.

468. Physics-to-astronomy/cosmology handoff

Write P for the retained physical evolution and source-bound observation relations, and M=(C,I,T) for the external object census, historical initial state and inferred chronology. The handoff is the typed pair H=(P,M). Replacing the measured input record gives H[M']=(P,M'), so

$$\mathrm{physics}(H[M']) = \mathrm{physics}(H)$$

$$\mathrm{observations}(H[M']) = \mathrm{observations}(H)$$

$$\mathrm{evolution}(H[M']) = \mathrm{evolution}(H)$$

Each of C, I and T remains an explicit measured-input field. None is silently used to select or redefine P.

For every retained observation o, its physical projection keeps the source, emitter, observer, complete causal support and both recurrence carriers. Applying this projection to all observations commutes with replacement of M because the entire observation list is retained. Finite histories under the retained evolution continue to satisfy the determinism theorem of Section 467.

This establishes the source-declared disciplinary boundary: universal physical relations remain within Physics; astronomical object census and historical inference are passed explicitly to their subsequent applications. The original 256 alternatives, controls and empirical validator are retained with this handoff.

The eight-axis construction of Section 353 applies here with relation coordinate `law-versus-historical-state-boundary`, justified by the preceding derivation. candidatesComplete, CosmoBranchBoundaryGrammar.lean:L120 proves membership of every typed alternative; acceptanceIffExactCriteria, CosmoBranchBoundaryGrammar.lean:L172 identifies acceptance with this relation and the seven shared carrier, provenance, custody, record, row, successor and extension requirements.

> Source, proof and provenance: V3 BRANCH_BOUNDARY, cosmological_boundary_laws.py:L69; Lean everyMeasuredRoleRemainsExplicit, CosmoBranchBoundary.lean:L36; Lean allObservationRecordsRetained, CosmoBranchBoundary.lean:L39; Timeline: Turn 514, line 5060.

469. Exact partition closure and spatial flatness

Let an admitted partition be a nonempty finite list p_i=n_i/d_i of strictly positive exact fractions, with d_i>0. Its total is evaluated by the exact recurrence

$$N_0 = 0, D_0 = 1$$

$$N_{k+1} = N_k d_{k+1} + n_{k+1} D_k$$

$$D_{k+1} = D_k d_{k+1}$$

Induction gives N_k/D_k=sum_(i=1)^k p_i. Consequently the source condition sum_i p_i=One is exactly N_m=D_m. The closed_partition operation rejects an empty list, a non-Fraction input, a nonpositive part or

a total unequal to One. An admitted closed partition returns EMPTY_ONE, the source's structural-absence scalar. The arithmetic accumulator is an exact representation of the sum; it does not replace that returned scalar by a physical numerical-zero object.

For a selected parent p=n/d, write its two proposed children as a=u/v and b=w/z, with a,b>0. The refinement is admitted exactly when the index identifies an existing part and

$$a + b = p$$

$$d(uz + wv) = nvz$$

The resulting list is prefix ++ [a,b] ++ suffix. All untouched parts and their order are retained. In particular, refine_partition imposes its positivity conditions on the two children; it does not add a new positivity guard on untouched parts. Its original Boolean-index interpretation is preserved.

Let (N,D) and (N',D') be the exact total pairs before and after this replacement. Distributing the common prefix and suffix factors through the accumulator gives

$$dN' = vzN$$

$$dD' = vzD$$

Since d>0, N=D implies N'=D'. Thus every admitted positive exact split of a closed positive partition preserves closure to EMPTY_ONE. For a finite refinement sequence P_0 -> P_1 -> ... -> P_k, the base is the initial closure and the successor is this same replacement identity; induction yields

$$\mathrm{closed_partition}(P_0) = \mathrm{EMPTY_ONE}$$

$$\Longrightarrow \mathrm{closed_partition}(P_k) = \mathrm{EMPTY_ONE}$$

for every finite k and every admitted sequence. No preferred split ratio or refinement depth is introduced. This is the source's depth-independent flatness derivation.

The external curvature comparison retains its empirical status. The registered Planck TT,TE,EE+lowE+lensing+BAO record has central orientation positive, magnitude 0.0007 and complete 68-percent uncertainty 0.0019. In the registered signed observational coordinate its interval is [(7-19)/10000, (7+19)/10000]

$$= [-0.0012, 0.0026]$$

which contains the structural-absence prediction at the coordinate origin. The uncertainty, confidence region, data combination and equation-47b source locator remain part of the measured record. They are neither derived constants nor premises of partition closure.

The native representation retains all eleven original grammar axes and all 2,048 alternatives, with the original elimination decisions, three operational witnesses, controls and source-bound validation method.

The source grammar tests carrier, component, normalization, completeness, remainder, absence, refinement, scope, target, provenance and extension. Its ordered coordinate product has $2^{11} = 2048$ forms. The common-total partition and parent-preserving split prove the admitted normalization, empty remainder and all-finite-repartitions coordinates by the induction above. The coordinate-membership and acceptance argument of Section 317 applies to this product, with the following native completeness and uniqueness proofs. candidatesComplete, CosmoSpatialFlatnessGrammarBase.lean:L151; uniqueSurvivor, CosmoSpatialFlatnessGrammar.lean:L205.

Grammar source: V3 SPATIAL_FLATNESS_SPEC, spatial_flatness_law.py:L41.

> Source, proof and provenance: V3 closed_partition, spatial_flatness_law.py:L16; V3 refine_partition, spatial_flatness_law.py:L26; V3 planck_interval_contains_absence, spatial_flatness_validation.py:L25; Lean closedExactly, CosmoSpatialFlatness.lean:L51; Lean refineRetainsPrefix, CosmoSpatialFlatness.lean:L72; Lean refinementPreservesClosedPartition, CosmoSpatialFlatnessProofs.lean:L170; Lean allFiniteRefinementsPreserveAbsence, CosmoSpatialFlatnessProofs.lean:L198; Lean completeIntervalContainsAbsence, CosmoSpatialFlatnessIntervals.lean:L29; Lean completeRowsRetained, CosmoSpatialFlatnessComparison.lean:L29; Lean missingRowRejects, CosmoSpatialFlatnessComparison.lean:L66; Timeline: Turn 515, line 5069.

470. Complete generated cosmic budget

The previously derived generator g=3 has spatial volume g^g=27. Its least complete binary cover has depth D=5 and support N=2^D=32. A complete boundary pair retains two states per depth. Writing P(D) for their count, P(0)=0 and P(D+1)=P(D)+2, so induction gives P(D)=2D. At the derived depth this yields

$$P = 2D = 10, F = N - P = 22$$

$$f_m = P/N = 10/32 = 5/16$$

$$f_v = F/N = 22/32 = 11/16$$

The support partition P+F=N therefore gives f_m+f_v=1.

The independently generated within-matter split retains D states against g^g states:

$$f_{b|m} = D/N = 5/32$$

$$f_{cd|m} = g^g/N = 27/32$$

$$f_{b|m} + f_{cd|m} = (5 + 27)/32 = 1$$

Composition with the total matter share gives

$$f_b = f_m f_{b|m} = (5/16)(5/32) = 25/512$$

$$f_{\mathrm{cd}} = f_m f_{cd|m} = (5/16)(27/32) = 135/512$$

Consequently f_b+f_cd=160/512=5/16=f_m, and f_b+f_cd+f_v=1. The complete ten-field construction is (D,N,P,F,f_m,f_v,f_(b|m),f_(cd|m),f_b,f_cd)

$$= (5, 32, 10, 22, 5/16, 11/16, 5/32, 27/32, 25/512, 135/512)$$

Each field is generated by these count, complement and composition operations; the displayed tuple is their evaluated result.

The registered empirical comparison retains the Planck-plus-BAO baseline record and complete 68-percent uncertainties. Its direct intervals are
f_m: 0.3111 +/- 0.0056,
f_v: 0.6889 +/- 0.0056.
For the baryon and cold-dark readings, let H=67.66 +/- 0.42 in the record's conventional Hubble units and h=H/100. Then
h in [0.6724,0.6808],
h^2 in [0.45212176,0.46348864].
The physical-density records are omega_b=0.02242 +/- 0.00014 and omega_cd=0.11933 +/- 0.00091. Positive-denominator interval division gives
f_b in [0.02228/0.46348864,0.02256/0.45212176],
f_cd in [0.11842/0.46348864,0.12024/0.45212176].
The derived shares lie in their respective registered intervals. The source's displaced leading shares 2/3, 1/3, 5/96 and 9/32 are rejected in the corresponding vacuum, matter, baryon and cold-dark comparisons. These observations test the derived budget; they do not determine its support or supply a fitted partition.

All ten returned fields, the original eleven grammar axes and 2,048 alternatives, four operational witnesses, controls and full source-bound empirical validator are retained.

The source grammar tests support, boundary, pinning, free, global, internal, composition, closure, target, provenance and extension. Its ordered coordinate product has $2^{11} = 2048$ forms. The least cover, two-state boundary count, unpinned complement and composed internal shares above determine the global, composition and exact-closure coordinates. The coordinate-membership and acceptance argument of Section 317 applies to this product, with the following native completeness and uniqueness proofs. candidatesComplete, CosmoCompleteBudgetGrammarBase.lean:L151; uniqueSurvivor, CosmoCompleteBudgetGrammar.lean:L205.

Grammar source: V3 COSMIC_BUDGET_SPEC, cosmic_budget_law.py:L47.

> Source, proof and provenance: V3 cosmic_budget_structure, cosmic_budget_law.py:L20; V3 planck_budget_intervals, cosmic_budget_validation.py:L33; V3 all_inside, cosmic_budget_validation.py:L52; Lean inheritedWithinMatter, CosmoCompleteBudget.lean:L68; Lean exactTenFields, CosmoCompleteBudget.lean:L40; Lean originalCompleteComparison, CosmoCompleteBudgetIntervals.lean:L66; Lean completeRowsRetained, CosmoCompleteBudgetComparison.lean:L29; Lean registeredComparisons, CosmoCompleteBudgetComparison.lean:L73; Timeline: Turn 515, line 5073.

471. Common finite scale axis and terminal electroweak transport

Let the first positive level be l=1. The inherited binary successor gives

$$R_l = 2^{l-1}, R_{l+1} = 2R_l$$

$$s_l = 1/R_l, s_{l+1} = s_l/2$$

$$T_l = R_l$$

Here s_l is the exact spacing of the One partition and T_l is its complete traversal count. These identities hold at every positive finite level. The prime-sector vector uses the previously derived sectors p in {2,3,5,7} on that same support:

$$g_p(R) = (p + R - 1)/(p + R)$$

A common change of positive rational reference cancels: for a,b,u>0,

$$(au)/(bu) = a/b$$

The certificate's unit witness is therefore (7/5)/(11/3)=21/55, unchanged when both carriers are multiplied by 13/17. The unit reference supplies no coefficient to either running law.

The electroweak source at binary support R is R+2. Its internal square anchor requires R+2 itself to be a binary support. Writing R=2^k, the complete classification is
2+2^k=2^m => k=1 and m=2.
For k=0 the source is three. For k>=2, 2^k<2^k+2<2^(k+1), excluding every binary support between successive levels. Thus support two, at level l=2, is the unique anchor; its source is four. The finite enumerator returns this anchor over the original registered depth range, while the proof establishes its uniqueness at arbitrary finite depth.

At support R the charged carrier is (R+1)/(R+2) and the neutral carrier is 1/2. Their exact squared-support composition gives

$$W(R) = (1/2)^2/[((R+1)/(R+2))^2 + (1/2)^2]$$

$$= (R+2)^2/[4(R+1)^2 + (R+2)^2]$$

This equals the already derived electroweak running share. For R=1,2,4,8 the exact values are
9/25, 4/13, 9/34, 25/106.
Put D(R)=4(R+1)^2+(R+2)^2. Direct expansion and factorization give
(R+2)^2 D(2R)-(2R+2)^2 D(R)

$$= 4R(4R^2 + 9R + 4)$$

Every factor on the right is positive when R>0; both denominators are positive. Consequently

$$W(R) - W(2R) = 4R(4R^2 + 9R + 4)/[D(R)D(2R)] > 0$$

The source's successive-level descent follows at every positive finite level, including the complete registered finite descent check.

The terminal electroweak chain uses four promotion rungs, obtained by three successive direction promotions, to complete support 16, holds the three generator directions, and retains active level 13. Its base share and return are

$$W(13) = 225/1009$$

$$\text{return divisor} = 16 + 1 = 17$$

$$\alpha = 3676744786/503846395469$$

$$\text{returned share} = \alpha/17 = 3676744786/8565388722973$$

$$\text{terminal share} = 225/1009 + \alpha/17$$

$$= 1930922298157999/8642477221479757$$

Equivalently, the returned share is terminal share minus base share, as in the original assembly. The eight returned fields retain promotion count, complete support, held directions, active level, base share, return divisor, returned share and terminal share.

The eight landmarks preserve both support and kind:
One origin: complete support 1;
electroweak internal square: complete support 2;
three-space step: complete support 8;
terminal electroweak support: complete support 16;
terminal electroweak active level: held support 13;
down-sector cover: complete support 32;
up-sector cover: complete support 128;
proton-Planck massive predecessor: held support 127.
The nine-field common-axis certificate retains supports, spacings and traversals at levels 1 through 8; prime-sector vectors and leading weak shares at levels 1 through 5; the anchor enumeration through level 12; the complete terminal chain; all eight landmarks; and the exact unit witness.

The measured comparison retains distinct electroweak schemes. The four registered central values and standard uncertainties are on-shell 0.22342 +/- 0.00009, effective lepton angle 0.23154 +/- 0.00006, MS at Z 0.23122 +/- 0.00006, and MS non-decoupled 0.23140 +/- 0.00006. Exact interval construction is [c-u,c+u], with u>0 and c>u. The terminal share lies in the on-shell interval [0.22333,0.22351]. The complete low-transfer vector retains NuTeV 0.2277 +/- 0.0016, Jefferson Lab eDIS 0.2299 +/- 0.0043, SLAC E158 0.2403 +/- 0.0013, and cesium APV 0.2349 +/- 0.0018 with their original scheme labels.

The leading support-eight reading 25/106 lies in the APV interval [0.2331,0.2367]. The E158 interval [0.2390,0.2416] and APV interval both lie strictly above the MS-at-Z interval [0.23116,0.23128]. The eDIS interval [0.2256,0.2342] overlaps it and remains non-discriminating. The NuTeV interval [0.2261,0.2293] lies above the complete terminal on-shell interval and remains an explicit adverse row. The W-threshold minimum and sign change bound the physical interpretation of the descending branch. All eight figure measurement classes and the complete figure snapshot are retained, together with the source-bound complete strong/electromagnetic vectors, terminal electroweak receipt and proton-Planck receipt. The seven formal prediction inputs seal before this measured vector is released. The original disjoint-interval control changes the on-shell target to [0.23999,0.24001] and rejects the terminal correspondence.

The native representation retains all thirteen original grammar axes and all 12,288 alternatives, eleven operational witnesses, and the original controls, closure and registration behavior. No measured value selects a support level, curve coefficient or terminal return.

The source grammar tests origin, successor, spacing, transport, sectors, weak curve, terminal, anchor, units, placement, target, provenance and extension. Its ordered coordinate product has $3 \cdot 2^{12} = 12288$ forms. The weak-curve coordinate has three alternatives: linear channel ratio, target-assigned angle and the admitted neutral square over the sum of charged and neutral squares; the other twelve coordinates are binary. The support successor, unique square anchor and squared-carrier derivation above determine the admitted axis and weak-curve choices. The coordinate-membership and acceptance argument of Section 317 applies to this product, with the following native completeness and uniqueness proofs. candidatesComplete, CommonScaleAxisGrammarBase.lean:L176; uniqueSurvivor, CommonScaleAxisGrammar.lean:L1166.

Grammar source: V3 SPEC, common_scale_axis_terminal_law_v1.py:L173.

> Source, proof and provenance: V3 scale_spacing, common_scale_axis_terminal_law_v1.py:L48; V3 common_rational_rescaling_ratio, common_scale_axis_terminal_law_v1.py:L60; V3 internal_square_anchor_levels, common_scale_axis_terminal_law_v1.py:L87; V3 leading_electroweak_share_at_support, common_scale_axis_terminal_law_v1.py:L95; V3 terminal_electroweak_chain, common_scale_axis_terminal_law_v1.py:L112; V3 leading_curve_strictly_descends, common_scale_axis_terminal_law_v1.py:L149; V3 authoritative_record, common_scale_axis_terminal_validation_v1.py:L99; V3 exact_measurement_analysis, common_scale_axis_terminal_validation_v1.py:L170; V3 formal_prediction_inputs, common_scale_axis_terminal_validation_v1.py:L206; Lean originalChainFields, CommonScaleAxis.lean:L159; Lean traversalsAreSupport, CommonScaleAxis.lean:L148; Lean binarySourceHasOnlyOneAnchor, CommonScaleAxisProofs.lean:L63; Lean everyPositiveUnitCancels, CommonScaleAxisProofs.lean:L24; Lean leadingSupportFormula, CommonScaleAxisAlgebra.lean:L64; Lean squaredSupportFormula, CommonScaleAxisAlgebra.lean:L20; Timeline: Turn 515, line 5089.

472. Terminal cosmic component transport and expansion

The inherited spatial generator has three directions. Retained matter therefore transports with the three volume factors of a positive exact past stretch r. Radiation retains those factors and one further wave-recurrence factor; an unchanged vacuum Fold retains the One:

$$M(r) = r * r * r = r^3$$

$$R(r) = M(r) * r = r^4$$

V(r)=1, for every exact rational r>0.
The complete three-component table retains matter with empty-pressure-One-form, radiation with outward-third-One, and vacuum with tension-One. Their conventional correspondences are respectively w=0 with a^-3, w=1/3 with a^-4, and w=-1 with a^0. The signs in these conventional coordinates are represented by typed orientations in the formal construction.

Composition with the generated present budget gives the late matter-vacuum squared-rate carrier

$$E^2(r) = 11/16 + (5/16)r^3 = (11 + 5r^3)/16$$

For any positive whole counts n,d with r=n/d, the native rational derivation yields

$$E^2(n/d) = (11d^3 + 5n^3)/(16d^3)$$

$$f_m(n/d) = 5n^3/(11d^3 + 5n^3)$$

$$f_v(n/d) = 11d^3/(11d^3 + 5n^3)$$

The common denominator is positive. Consequently f_m(r)+f_v(r)=1 at every admitted stretch, and E^2(1)=1. Radiation's relative transport is r^4; the registered late-time squared-rate carrier comprises matter and vacuum and does not supply a measured present radiation normalization.

Equality of the matter and vacuum terms gives

$$(5/16)r^3 = 11/16 \Longrightarrow r^3 = 11/5$$

The accelerating separation compares vacuum tension with half the matter term:

$$(5/32)r^3 = 11/16 \Longrightarrow r^3 = 22/5$$

At the present stretch its positive accelerating magnitude is

$$11/16 - (1/2)(5/16) = 17/32$$

Both threshold cubes are exact rational outputs; the derivation does not select a decimal cube root or introduce a fitted threshold.

The dimensionless law seals before the measured Hubble reference is released. The registered reference is H0=67.66 +/- 0.42 km s^-1 Mpc^-1, with a complete 68-percent interval [67.24,68.08]. For a row with measured redshift z, central rate H and reported uncertainty u, put

$$r = 1 + z$$

$$I_{\mathrm{pred}} = [67.24^2 E^2(r), 68.08^2 E^2(r)]$$

L=H-2u when H>2u, and L=0 at the typed empty measurement boundary otherwise,

$$I_{\mathrm{obs}} = [L^2, (H + 2u)^2]$$

The exact comparison is interval overlap:
lower(I_pred)<=upper(I_obs) and lower(I_obs)<=upper(I_pred).
It uses squared rational rates throughout. The complete registered CCH vector and its generated E^2 values are:

Row	z	H	reported u	exact E^2(1+z)
01	0.07	69.0	19.6	$\frac{3425043}{3200000}$
02	0.09	69.0	12.0	$\frac{3495029}{3200000}$

Row	z	H	reported u	exact E^2(1+z)
03	0.12	68.6	26.2	$\frac{56327}{50000}$
04	0.17	83.0	8.0	$\frac{3801613}{3200000}$
05	0.1791	78.0	6.2	$\frac{3839275386671}{3200000000000}$
06	0.1993	78.0	6.9	$\frac{3924977763657}{3200000000000}$
07	0.2	72.9	29.6	$\frac{491}{400}$
08	0.27	77.0	14.0	$\frac{4248383}{3200000}$
09	0.28	88.8	36.6	$\frac{67143}{50000}$
10	0.3519	85.5	15.7	$\frac{4670777877359}{3200000000000}$
11	0.3802	86.2	14.6	$\frac{603651850701}{400000000000}$
12	0.4	95.0	17.0	$\frac{309}{200}$
13	0.4004	79.9	11.4	$\frac{77286760501}{50000000000}$
14	0.4247	90.4	12.8	$\frac{5091813447223}{3200000000000}$
15	0.4497	96.3	14.4	$\frac{5246733141473}{3200000000000}$
16	0.47	89.0	49.6	$\frac{5376523}{3200000}$
17	0.4783	83.8	10.2	$\frac{5430633786687}{3200000000000}$
18	0.48	97.0	62.0	$\frac{21257}{12500}$
19	0.5929	107.0	15.5	$\frac{6241713610089}{3200000000000}$
20	0.6797	95.0	10.5	$\frac{6939092293573}{3200000000000}$
21	0.75	98.8	33.6	$\frac{2419}{1024}$
22	0.7812	96.5	12.5	$\frac{122674467677}{50000000000}$
23	0.8754	124.5	17.4	$\frac{1099502065633}{400000000000}$
24	0.88	90.0	40.0	$\frac{69099}{25000}$
25	0.9	117.0	23.0	$\frac{9059}{3200}$
26	1.037	133.5	17.6	$\frac{10652264653}{3200000000}$
27	1.3	168.0	17.0	$\frac{14367}{3200}$
28	1.363	160.0	33.8	$\frac{15394446147}{3200000000}$

Row	z	H	reported u	exact E^2(1+z)
29	1.43	177.0	18.0	$\frac{16548907}{3200000}$
30	1.53	140.0	14.0	$\frac{18394277}{3200000}$
31	1.75	202.0	40.0	$\frac{7359}{1024}$
32	1.965	186.5	50.6	$\frac{226127857}{25600000}$

Every one of these 32 rows passes the registered two-uncertainty overlap comparison. The complete vector retains the four one-standard-uncertainty departures rather than selecting rows or changing the registered comparison boundary.

The source's present-budget intervals are matter [0.3055,0.3167] and vacuum [0.6833,0.6945]. Positive outward division gives
vacuum/matter in [6833/3167,1389/611],
which contains 11/5. Doubling the same interval gives
[13666/3167,2778/611],
which contains the acceleration-onset cube 22/5. The measured accelerating magnitude 0.60 +/- 2(0.10) gives [2/5,4/5], containing 17/32. The measured transition redshift 0.80 +/- 2(0.10) gives r in [1.6,2.0], hence
r^3 in [512/125,8],
containing 22/5. The conventional constant-w record -1.013 with upper uncertainty 0.038 and lower uncertainty 0.043 maps to the tension-magnitude interval

$$[1.013 - 0.038, 1.013 + 0.043] = [39/40, 132/125]$$

which contains the One.

The DESI DR2 model-extension preference remains an explicit adverse record: its 3.1-sigma and 2.8-to-4.2-sigma statements are retained with the original model-extension-tension status. It neither selects the static law nor disappears from the target vector. The complete protocol contains the 32 CCH targets and five additional registered rows for the Planck ratio, accelerating magnitude, transition cube, tension magnitude and adverse DESI evidence. All six source files remain hash-bound, and the deliberately changed observation is rejected by the original sealed comparison control.

The native construction retains all twelve original grammar axes, all 4,096 alternatives and the five original operational witnesses. Its generic positive-rational transport and closure proofs accompany the original complete measured comparison.

The source grammar tests space, matter, radiation, vacuum, expansion, fractions, equality, acceleration, orientation, scale, target and extension. Its ordered coordinate product has $2^{12} = 4096$ forms. The three volume factors, extra radiation recurrence and invariant vacuum give the admitted transport powers; the terminal budget determines the expansion, equality and acceleration coordinates. The coordinate-membership and acceptance argument of Section 317 applies to this product, with the following native completeness and uniqueness proofs. candidatesComplete, CosmicComponentTransportGrammarBase.lean:L163; uniqueSurvivor, CosmicComponentTransportGrammar.lean:L398.

Grammar source: V3 SPEC, cosmic_component_transport_terminal_law_v1.py:L95.

> Source, proof and provenance: V3 matter_transport, cosmic_component_transport_terminal_law_v1.py:L25; V3 radiation_transport, cosmic_component_transport_terminal_law_v1.py:L30; V3 vacuum_transport, cosmic_component_transport_terminal_law_v1.py:L35; V3 late_squared_expansion, cosmic_component_transport_terminal_law_v1.py:L40; V3 matter_fraction, cosmic_component_transport_terminal_law_v1.py:L47; V3 vacuum_fraction, cosmic_component_transport_terminal_law_v1.py:L53; V3 matter_vacuum_equality_cube, cosmic_component_transport_terminal_law_v1.py:L58; V3 acceleration_onset_cube, cosmic_component_transport_terminal_law_v1.py:L62; V3 _interval_overlap, cosmic_component_transport_terminal_validation_v1.py:L43; V3 _positive_or_empty_difference, cosmic_component_transport_terminal_validation_v1.py:L47; V3 cosmic_transport_measurement_record, cosmic_component_transport_terminal_validation_v1.py:L53; Lean generatedSpatialCount, CosmicComponentTransport.lean:L66; Lean presentClosure, CosmicComponentTransport.lean:L74; Lean everySquaredExpansion, CosmicComponentTransportProofs.lean:L61; Lean everyMatterFraction, CosmicComponentTransportProofs.lean:L72; Lean emptyDifferenceRetained, CosmicComponentTransportMeasurement.lean:L118; Lean positiveDifferenceRetained, CosmicComponentTransportMeasurement.lean:L124; Lean completeRowsRetained, CosmicComponentTransportComparison.lean:L29; Lean missingRowRejects, CosmicComponentTransportComparison.lean:L66; Timeline: Turn 515, line 5091.

473. Unified Constants: the foundation-order object

The Unified Constants Object assembles the admitted electromagnetic, lepton, quark, dark/baryon, Hubble, Planck-hierarchy and vacuum constructions through one rooted incidence graph. It is the distinct v3 Unified Constants entry; it does not rename or replace the Physics Grand Lock. Its physical construction uses the Fold-derived binary count b=2 and generator c=3, spatial rank s=3 and boundary rank t=2. No measured constant selects these quantities.

For the source's dependency probe, c is a whole count at least three. Define the positive least covers

$$d(c)=\min\{k\ge 1 : b^k \ge c^s\}, \qquad u(c)=\min\{k\ge 1 : b^k \ge c^{s+1}\}.$$

The implementation advances support by multiplication with its supplied binary argument until it first covers the carrier. Its constructive run proof establishes both coverage and strict failure at every earlier positive depth. Positive power is repeated multiplication, P(a,1)=a and P(a,n+1)=P(a,n)*a; the positive predecessor of n>1 is the unique p>=1 with p+1=n. These are operations on generated counts, not selectable scientific coefficients.

Write B_d=b^d, B_u=b^u and C=b*d^c. The full sixteen-field foundation vector is

$$F(c)=(b,c,s,t,d,u,A,L,Q_d,Q_u,R,f_D,H,P,V,1/b),$$

where every component follows from the same generated counts:

$$\begin{aligned} A&=B_u+c^b\frac{C+1}{C}, & L&=\frac{1}{bc^d-1},\\ Q_d&=\frac{1}{cB_d-1}, & Q_u&=\frac{1}{cB_u-1},\\ R&=\frac{c^s}{d}, & f_D&=\frac{c^s}{B_d},\\ H&=1+\frac{c-1}{cb^s}, & P&=\frac{B_u-1}{b},\\ V&=\frac{1}{b^{4d}}. \end{aligned}$$

At the admitted generator, 2^4<3^3<=2^5 and 2^6<3^4<=2^7 force d=5 and u=7. Consequently C=2*5^3=250 and

$$
\begin{aligned}
A &= 128 + 9\frac{251}{250} = \frac{34259}{250},\\
L &= \frac{1}{2\cdot 3^5 - 1} = \frac{1}{485},\\
Q_d &= \frac{1}{3\cdot 32 - 1} = \frac{1}{95}, \qquad Q_u = \frac{1}{3\cdot 128 - 1} = \frac{1}{383},\\
R &= \frac{27}{5}, \quad f_D = \frac{27}{32}, \quad H = 1 + \frac{2/3}{8} = \frac{13}{12},\\
P &= \frac{127}{2}, \quad V = \frac{1}{2^{20}}, \quad \frac{1}{b} = \frac{1}{2}.
\end{aligned}
$$

This is a complete ordered vector; no sector is omitted to select a favourable headline.

The source's successor probe replaces c=3 by c=4 while holding b,s,t fixed. It is a controlled dependency test, not an adjustable parameter in the physical model. The cover calculations become 2^5<4^3<=2^6 and 2^7<4^4<=2^8, hence d=6 and u=8. Recomputing every component gives the following complete comparison:

Field	F(3)	F(4)
binary_fibre_count	2	2
generator_count	3	4
spatial_rank	3	3
boundary_rank	2	2
down_cover_depth	5	6
up_cover_depth	7	8
inverse_fine_structure_leading	$\frac{34259}{250}$	$\frac{44065}{162}$
charged_lepton_leading_product	$\frac{1}{485}$	$\frac{1}{8191}$
down_cover_conjugate	$\frac{1}{95}$	$\frac{1}{255}$
up_cover_product	$\frac{1}{383}$	$\frac{1}{1023}$
dark_to_baryon_leading	$\frac{27}{5}$	$\frac{32}{3}$
dark_share_leading	$\frac{27}{32}$	1
hubble_leading	$\frac{13}{12}$	$\frac{35}{32}$
planck_hierarchy_exponent	$\frac{127}{2}$	$\frac{255}{2}$
local_vacuum_energy_floor	$\frac{1}{1048576}$	$\frac{1}{16777216}$
half_One	$\frac{1}{2}$	$\frac{1}{2}$

Exactly twelve fields change: the generator, both covers, inverse fine structure, the lepton product, both quark cover products, dark/baryon ratio, dark share, Hubble ratio, Planck exponent and local vacuum energy floor. Exactly four held fields remain equal: binary count, spatial rank, boundary rank and half-One.

The native proof establishes both complete conjunctions, retaining the source's accepted generator domain rather than restricting the function to the two displayed probes.

Additional proof bindings for the preceding conclusions: Lean completeFoundationFields, UnifiedConstantsFoundation.lean:L98; Lean allTwelveDependents, UnifiedConstantsFoundation.lean:L125.

> Source, proof and provenance: V3 positive_predecessor, unified_constants_object_law_v1.py:L41; V3 positive_power, unified_constants_object_law_v1.py:L50; V3 minimal_binary_cover, unified_constants_object_law_v1.py:L59; V3 foundation_order_vector, unified_constants_object_law_v1.py:L70; Lean runLeast, UnifiedConstantsHelpers.lean:L65; Lean arbitraryBinaryCover, UnifiedConstantsHelpers.lean:L79; Lean originalGenerator, UnifiedConstantsFoundation.lean:L127; Lean originalSuccessor, UnifiedConstantsFoundation.lean:L129; Timeline: Turn 515, line 5099.

474. Unified Constants: terminal continuations and cross-sector equality

The terminal vector reads the existing sector derivations through their original functions. Let a denote the terminal fine-structure constant. Its continued-fraction assembly is

$$D = 250 + \cfrac{1}{175 + \cfrac{1}{245 + \cfrac{1}{343}}}, \qquad a^{-1} = 128 + 9\frac{D+1}{D} = \frac{503846395469}{3676744786}.$$

The full rung ladder is (125,175,245,343): the first rung generates the separate leading-order cover 2*125=250, while the terminal assembly continues that cover with the remaining three rungs. Thus the leading and terminal readings retain their original distinct orders within the same construction.

For the charged-lepton sector, the generated leading denominator is 485. Its sharpened product is

$$L_{\text{sharp}} = \frac{1}{485 - 1/3} = \frac{3}{1454}.$$

The completed three-channel self-coupling contributes the exact positive amount

$$\delta_L = \frac{a^3}{27}\left(5 + \frac{7}{3}a\right),$$

and the terminal product is the source's positive take

$$L_{\text{terminal}} = \frac{3}{1454} - \delta_L.$$

The subtraction is represented by a retained positive remainder, preserving the source's exact domain and positivity check. The original terminal derivation and empirical comparison accompany this reading in the preceding lepton sections.

The terminal quark products use their existing channel reaches: down reach u=7 and up reach u+c=10. Therefore

$$Q_{\text{down,terminal}} = \frac{1}{3 \cdot 2^7 - 1} = \frac{1}{383}, \qquad Q_{\text{up,terminal}} = \frac{1}{3 \cdot 2^{10} - 1} = \frac{1}{3071}.$$

They are terminal continuations of their sector constructions; the source does not assert that these two readings equal the foundation vector's two cover-product fields.

The dark/baryon continuation retains its finite orbit floor 2^5-1=31:

$$R_{\text{leading}} = \frac{27}{5}, \quad R_{\text{terminal}} = \frac{27}{5 + 1/31} = \frac{279}{52}, \quad f_D = \frac{27}{32}, \quad f_B = \frac{5}{32}.$$

The Hubble continuation retains its finite orbit floor 2^7-1=127:

$$H_{\text{leading}} = 1 + \frac{2/3}{8} = \frac{13}{12}, \qquad H_{\text{terminal}} = 1 + \frac{2/3 + 1/127}{8} = \frac{3305}{3048}.$$

These are the precise source-order readings used by this object. The separately typed complete cosmic budget and component transport retain their own original definitions and comparison boundaries.

The Planck sector uses the generated massive predecessor 2^7-1=127. Its exponent and squared hierarchy are

$$P = \frac{127}{2}, \qquad P_{\text{squared}} = 2^{127}.$$

The already derived charged-colour transport preserves the positive factor 1-(2/3)*a, yielding

$$P_{\text{squared,terminal}} = 2^{127}\left(1 - \frac{2}{3}a\right).$$

The vacuum sector supplies its separately typed quantities

$$A_v = \frac{1}{2^{10}}, \qquad E_v = A_v^2 = \frac{1}{2^{20}}, \qquad \Lambda\left(\frac{c_{\text{limit}}}{H_{\text{rate}}}\right)^2 = 3\frac{11}{16} = \frac{33}{16}.$$

A dimensional Lambda still requires the explicit measured rate and limiting-speed references; the Unified Constants assembly introduces neither as a fitted input. Half-One remains 1/2.

The complete terminal vector contains six count/rank/cover fields, two fine-structure readings, three lepton products, two quark products, four dark/baryon readings, two Hubble readings, three Planck readings, three vacuum readings and half-One. Its exact values are:

Terminal field	Exact value
binary_fibre_count	2
generator_count	3
spatial_rank	3
boundary_rank	2
down_cover_depth	5
up_cover_depth	7
inverse_fine_structure_leading	$\frac{34259}{250}$
inverse_fine_structure_terminal	$\frac{503846395469}{3676744786}$
charged_lepton_leading_product	$\frac{1}{485}$

Terminal field	Exact value
charged_lepton_sharpened_product	$\frac{3}{1454}$
charged_lepton_terminal_product	$\frac{1565970939787116880144381546645451095683700058411 5}{7590004776733382935404977537704409308185713931574254}$
down_quark_terminal_product	$\frac{1}{383}$
up_quark_terminal_product	$\frac{1}{3071}$
dark_to_baryon_leading	$\frac{27}{5}$
dark_to_baryon_terminal	$\frac{279}{52}$
dark_share_leading	$\frac{27}{32}$
baryon_share_leading	$\frac{5}{32}$
hubble_leading	$\frac{13}{12}$
hubble_terminal	$\frac{3305}{3048}$
planck_hierarchy_exponent	$\frac{127}{2}$
planck_hierarchy_squared	170141183460469231731687303715884105728
planck_hierarchy_terminal_squared	$\frac{25592393460381748800840516069019941843257249497088 0}{1511539186407}$
local_vacuum_amplitude_floor	$\frac{1}{1024}$
local_vacuum_energy_floor	$\frac{1}{1048576}$
normalized_cosmological_magnitude	$\frac{33}{16}$
half_One	$\frac{1}{2}$

The cross-lock compares the exact fourteen source-selected common-order fields: binary count, generator, spatial rank, boundary rank, both cover depths, leading inverse fine structure, leading lepton product, leading dark/baryon ratio, leading dark share, leading Hubble ratio, Planck exponent, local vacuum energy floor and half-One. The native equality check proves all fourteen simultaneously. It does not collapse a terminal continuation into a different approximation order.

> Source, proof and provenance: V3 admitted_terminal_vector, unified_constants_object_law_v1.py:L118; Lean originalLeadingTerminalEquality, UnifiedConstantsTerminal.lean:L69; Lean originalTwentySixFields, UnifiedConstantsTerminal.lean:L67; Timeline: Turn 515, line 5099.

475. Unified Constants: exact rooted incidence and complete certificate

The source object records the following parent-to-child incidence, with every original edge retained:

One -> Fold
Fold -> binary_fibre_count, generator_count
binary_fibre_count -> down_cover_depth, up_cover_depth, half_One
generator_count -> spatial_rank, lepton_sector, quark_sector, cosmic_partition

spatial_rank -> boundary_rank, down_cover_depth, up_cover_depth
boundary_rank -> fine_structure_sector, vacuum_sector
down_cover_depth -> fine_structure_sector, lepton_sector, dark_baryon_sector, vacuum_sector
up_cover_depth -> fine_structure_sector, quark_sector, hubble_sector, planck_sector
fine_structure_sector -> lepton_sector, quark_sector, planck_sector
cosmic_partition -> dark_baryon_sector, hubble_sector, vacuum_sector

This graph has seventeen nodes and ten nonempty parent rows. Each of the seven sector nodes is reachable from One. The reachability proof instantiates the existing graph theorem equating an admitted graph search with a valid path witness. Explicit paths through the original incidence establish the seven positive results; no new root or bypass edge is inserted. The incoming counts are three each for fine structure, leptons, quarks and vacuum, and two each for dark/baryon, Hubble and Planck. Thus every sector satisfies the original incoming-incidence test.

The seven explicit path witnesses and incoming counts are:

Sector	Path from One	Incoming parents
dark_baryon_sector	One -> Fold -> binary_fibre_count -> down_cover_depth -> dark_baryon_sector	2
fine_structure_sector	One -> Fold -> binary_fibre_count -> down_cover_depth -> fine_structure_sector	3
hubble_sector	One -> Fold -> binary_fibre_count -> up_cover_depth -> hubble_sector	2
lepton_sector	One -> Fold -> generator_count -> lepton_sector	3
planck_sector	One -> Fold -> binary_fibre_count -> up_cover_depth -> planck_sector	2
quark_sector	One -> Fold -> generator_count -> quark_sector	3
vacuum_sector	One -> Fold -> binary_fibre_count -> down_cover_depth -> vacuum_sector	3

The complete eight-field certificate is
(F(3), T, F(4), rooted, shared_incidence,
all_twelve_dependents_move, four_held_controls, fourteen_field_cross_lock).
Its last five components are all true, proved from the definitions above. They are the five original operational witnesses: rooted object, shared incidence, leading/terminal cross-lock, dependency probe and independence control.

The native entry preserves all twelve binary grammar axes: root, Fold, covers, typing, leading, terminal, graph, mutation, control, measurement, extension and rule. Their complete product contains 2^12=4096 alternatives. Every alternative has a kernel-checked source decision and uniqueness proof; the original elimination reasons, source identities, closure and controls remain attached to the construction. All scientific values in the object are read from the admitted derivations or recomputed by the source's explicit dependency probe. The object is formally sealed before comparison with external measurements.

The source grammar tests root, fold, covers, typing, leading, terminal, graph, mutation, control, measurement, extension and rule. Its ordered coordinate product has $2^{12} = 4096$ forms. The rooted graph, complete vectors, generator-successor probe and held controls above provide the root, incidence, mutation and control coordinates. Extension appends a traced node; it cannot rewrite the existing object. The coordinate-membership and acceptance argument of Section 317 applies to this product, with the following native completeness and uniqueness proofs. candidatesComplete, UnifiedConstantsGrammarBase.lean:L163; uniqueSurvivor, UnifiedConstantsGrammar.lean:L398.

Additional proof bindings for the preceding conclusions: Lean allSevenReachable, UnifiedConstantsGraph.lean:L57; Lean everySectorHasIncoming, UnifiedConstantsGraph.lean:L64.

Grammar source: V3 SPEC, unified_constants_object_law_v1.py:L254.

> Source, proof and provenance: V3 dependency_graph, unified_constants_object_law_v1.py:L156; V3 reachable_from_one, unified_constants_object_law_v1.py:L173; V3 theorem_certificate, unified_constants_object_law_v1.py:L196; Lean sectorReachable0, UnifiedConstantsGraph.lean:L36; Lean sectorReachable1, UnifiedConstantsGraph.lean:L39; Lean allOriginalChecks, UnifiedConstantsCertificate.lean:L44; Timeline: Turn 515, line 5099.